

Basic Topics:

1. File Management

- **Commands:** ls, cp, mv, rm, touch
- **Scenario:** List all files in a directory, copy a file, move files between directories, and delete files.

2. Directory Management

- **Commands:** mkdir, rmdir, cd, pwd
- **Scenario:** Create and navigate through directories, and remove empty directories.

3. File Permissions

- **Commands:** chmod, chown, chgrp
- **Scenario:** Change file permissions to control user access.

4. Viewing and Editing Files

- **Commands:** cat, less, more, vi, nano
- **Scenario:** View the contents of a file and edit it using text editors.

5. Managing Links

- **Commands:** ln, ln -s
- **Scenario:** Create hard and soft links to files.

6. Process Management

- **Commands:** ps, top, kill, htop, bg, fg, nice
- **Scenario:** Monitor running processes and terminate or change their priority.

7. Managing Users and Groups

- **Commands:** useradd, userdel, usermod, groupadd
- **Scenario:** Create, modify, and delete users and groups.

8. Package Management (Debian-based)

- **Commands:** apt-get, dpkg
- **Scenario:** Install, remove, and update packages on Debian-based systems.

9. Package Management (RHEL-based)

- **Commands:** yum, dnf, rpm
- **Scenario:** Install, update, and remove packages on Red Hat-based systems.

10. Disk Management

- **Commands:** df, du, fdisk, mount, umount
- **Scenario:** Check disk usage, partition a disk, and mount/unmount file systems.

11. Service Management

- **Commands:** systemctl, service, chkconfig
- **Scenario:** Start, stop, enable, and disable services on a system.

12. Networking Basics

- **Commands:** ip, ifconfig, ping, netstat, traceroute
- **Scenario:** Check network configuration, test connectivity, and display active connections.

13. SSH and Remote Access

- **Commands:** ssh, scp, rsync
- **Scenario:** Securely connect to remote systems and transfer files between them.

14. Cron Jobs

- **Commands:** crontab
- **Scenario:** Schedule regular tasks to automate scripts or system commands.

15. System Logs

- **Commands:** journalctl, dmesg, tail, /var/log
- **Scenario:** View system logs for troubleshooting and auditing purposes.

16. Text Processing Tools

- **Commands:** grep, awk, sed, cut, sort
- **Scenario:** Search, filter, and modify text data from files or command output.

17. Archiving and Compression

- **Commands:** tar, gzip, gunzip, zip, unzip
- **Scenario:** Compress files into archives and extract them.

18. **File Search**

- **Commands:** find, locate
- **Scenario:** Locate files based on name, type, or modification date.

19. **Basic Shell Scripting**

- **Commands:** bash, sh
- **Scenario:** Automate tasks using simple shell scripts.

20. **Environment Variables**

- **Commands:** export, env, set
- **Scenario:** Set and manage environment variables.

Intermediate Topics:

21. **Advanced File Permissions: ACL**

- **Commands:** getfacl, setfacl
- **Scenario:** Assign more granular file permissions using Access Control Lists.

22. **LVM (Logical Volume Manager)**

- **Commands:** pvcreate, vgcreate, lvcreate, lvextend
- **Scenario:** Create and manage logical volumes for flexible disk space allocation.

23. **Kernel Management**

- **Commands:** uname, dmesg, modprobe, lsmod
- **Scenario:** View kernel information and manage kernel modules.

24. **Networking Configuration**

- **Commands:** ip, nmcli, /etc/network/interfaces
- **Scenario:** Set static IP addresses and configure network interfaces.

25. **Firewall Management**

- **Commands:** firewalld, iptables
- **Scenario:** Set up rules to control inbound and outbound traffic on the system.

26. **SELinux**

- **Commands:** getenforce, setenforce, semanage
- **Scenario:** Enable, disable, and troubleshoot SELinux for security policies.

27. **Managing RAID**

- **Commands:** mdadm

- **Scenario:** Create and manage software RAID for redundancy and performance.

28. **Backup and Restore**

- **Commands:** rsync, tar, dd
- **Scenario:** Perform system backups and restore from a backup image.

29. **Swap Management**

- **Commands:** swapon, swapoff, mkswap
- **Scenario:** Configure swap space for system memory management.

30. **NFS (Network File System)**

- **Commands:** mount, nfsstat, /etc/exports
- **Scenario:** Share directories over the network using NFS.

31. **Samba (SMB)**

- **Commands:** smbclient, smb.conf
- **Scenario:** Configure Samba for file sharing between Linux and Windows systems.

32. **Advanced Shell Scripting**

- **Commands:** bash, zsh, functions, loops
- **Scenario:** Create complex automation scripts with loops and conditional statements.

33. **Scheduling with at**

- **Commands:** at, atq, atrm
- **Scenario:** Schedule one-time tasks to run at specific times.

34. **Basic Performance Monitoring**

- **Commands:** top, iotop, vmstat, sar
- **Scenario:** Monitor CPU, memory, and disk I/O performance on the system.

35. **System Boot Process**

- **Commands:** systemd-analyze, grub2, /boot/
- **Scenario:** Analyze boot performance and troubleshoot boot issues.

36. **Creating and Managing Partitions**

- **Commands:** fdisk, parted, mkfs
- **Scenario:** Create, delete, and format disk partitions.

37. **User Authentication with PAM**

- **Commands:** pam-auth-update, /etc/pam.d/
- **Scenario:** Configure Pluggable Authentication Modules for security.

38. **Troubleshooting with strace and lsof**

- **Commands:** strace, lsof
- **Scenario:** Debug program execution and identify open file descriptors.

39. **Setting Up a Web Server (Apache/Nginx)**

- **Commands:** apachectl, nginx, httpd
- **Scenario:** Set up and configure a web server for hosting websites.

40. **Database Management (MySQL/PostgreSQL)**

- **Commands:** mysql, psql
- **Scenario:** Install, configure, and manage a MySQL or PostgreSQL database server.

Advanced Topics:

41. **Kernel Tuning with sysctl**

- **Commands:** sysctl
- **Scenario:** Modify kernel parameters at runtime for performance tuning.

42. **Virtualization with KVM**

- **Commands:** virsh, virt-install
- **Scenario:** Create and manage virtual machines on Linux with KVM.

43. **Docker and Containerization**

- **Commands:** docker, docker-compose
- **Scenario:** Set up, run, and manage containers on Linux.

44. **Automation with Ansible**

- **Commands:** ansible-playbook
- **Scenario:** Automate infrastructure provisioning using Ansible playbooks.

45. **High Availability with Pacemaker and Corosync**

- **Commands:** pcs, crm
- **Scenario:** Set up a highly available cluster for critical services.

46. **Advanced Networking with iptables/nftables**

- **Commands:** iptables, nft
- **Scenario:** Configure complex firewall rules for secure network management.

47. **Load Balancing with HAProxy**

- **Commands:** haproxy, systemctl

- **Scenario:** Set up load balancing for web applications.

48. **Email Server Setup (Postfix)**

- **Commands:** postfix, mail
- **Scenario:** Configure a Postfix mail server for sending and receiving emails.

49. **Advanced Monitoring with Nagios/Zabbix**

- **Commands:** nagios, zabbix-server
- **Scenario:** Set up and configure monitoring tools to track system and service health.

50. **Cloud Integration (AWS/GCP)**

- **Commands:** aws-cli, `gcloud

Basic Topics with Examples:

1. File Management

- Commands: ls, cp, mv, rm, touch
- Example:

ls -l # List all files with details

cp file1.txt /tmp # Copy file1.txt to /tmp

mv file1.txt /tmp # Move file1.txt to /tmp

rm file1.txt # Delete file1.txt

touch file1.txt # Create an empty file named file1.txt

2. Directory Management

- Commands: mkdir, rmdir, cd, pwd
- Example:

mkdir /tmp/new_dir # Create a new directory in /tmp

`cd /tmp/new_dir` # Change directory to /tmp/new_dir

`pwd` # Print working directory

`rmdir /tmp/new_dir` # Remove an empty directory

3. File Permissions

- Commands: `chmod`, `chown`, `chgrp`
- Example:

`chmod 755 file1.txt` # Set file1.txt permissions to `rw-r-xr-x`

`chown user1 file1.txt` # Change ownership of file1.txt to user1

`chgrp group1 file1.txt` # Change the group of file1.txt to group1

4. Viewing and Editing Files

- Commands: `cat`, `less`, `more`, `vi`, `nano`
- Example:

`cat file1.txt` # Display the contents of file1.txt

`less file1.txt` # View large file with less command

`vi file1.txt` # Edit file1.txt with vi editor

`nano file1.txt` # Edit file1.txt with nano editor

5. Managing Links

- Commands: `ln`, `ln -s`
- Example:

`ln file1.txt file1_link # Create a hard link to file1.txt`

`ln -s file1.txt file1_slink # Create a symbolic link to file1.txt`

6. Process Management

- Commands: `ps`, `top`, `kill`, `htop`, `bg`, `fg`, `nice`
- Example:

`ps aux # Show all running processes`

`top # Display real-time process info`

`kill -9 1234 # Forcefully terminate process with PID 1234`

`nice -n 10 myscript # Run myscript with lower priority`

7. Managing Users and Groups

- Commands: `useradd`, `userdel`, `usermod`, `groupadd`
- Example:

`useradd user1 # Add a new user`

`userdel user1 # Delete a user`

`usermod -aG sudo user1 # Add user1 to the sudo group`

`groupadd group1 # Create a new group`

8. Package Management (Debian-based)

- Commands: `apt-get`, `dpkg`
- Example:

apt-get update # Update package repository
apt-get install apache2 # Install Apache web server
dpkg -i package.deb # Install package using dpkg

9. Package Management (RHEL-based)

- Commands: yum, dnf, rpm
- Example:

yum install httpd # Install Apache on RHEL-based systems
dnf update # Update packages on RHEL-based systems
rpm -ivh package.rpm # Install package using rpm

10. Disk Management

- Commands: df, du, fdisk, mount, umount
- Example:

df -h # Show disk usage in human-readable format
du -sh /home/user1 # Show disk usage for a specific directory
fdisk /dev/sdb # Manage partitions on a disk
mount /dev/sdb1 /mnt # Mount the disk /dev/sdb1 to /mnt
umount /mnt # Unmount the disk from /mnt

11. Service Management

- Commands: systemctl, service, chkconfig
- Example:

```
systemctl start httpd    # Start Apache service
systemctl enable httpd   # Enable Apache to start on boot
service apache2 restart  # Restart Apache on Debian-based systems
chkconfig httpd on       # Enable Apache on RHEL-based systems
```

12. Networking Basics

- Commands: ip, ifconfig, ping, netstat, traceroute
- Example:

```
ip addr show    # Show IP address information
ifconfig eth0   # Show details of network interface eth0
ping google.com # Ping a remote host
netstat -tuln   # Show open ports
traceroute google.com # Trace the network route to a remote host
```

13. SSH and Remote Access

- Commands: ssh, scp, rsync
- Example:

```
ssh user@remote_host    # SSH into a remote system
scp file1.txt user@remote:/tmp # Copy file to a remote system
rsync -av /src/ /dest/   # Sync files between local and remote systems
```

14. Cron Jobs

- Commands: crontab
- Example:

```
crontab -e      # Edit cron jobs

# Example cron job: Run script every day at 5 AM
0 5 * * * /home/user/myscript.sh
```

15. System Logs

- Commands: journalctl, dmesg, tail, /var/log
- Example:

```
journalctl -xe  # View detailed system logs

dmesg           # View kernel log messages

tail -f /var/log/syslog # Follow system log in real-time
```

16. Text Processing Tools

- Commands: grep, awk, sed, cut, sort
- Example:

```
grep 'error' /var/log/syslog # Search for 'error' in syslog

awk '{print $1, $2}' file1.txt # Extract and print the first two columns

sed 's/foo/bar/g' file1.txt   # Replace 'foo' with 'bar' in file1.txt

cut -d: -f1 /etc/passwd      # Extract the first field from /etc/passwd

sort file1.txt                # Sort the contents of a file
```

17. Archiving and Compression

- Commands: tar, gzip, gunzip, zip, unzip
- Example:

```
tar -czvf archive.tar.gz /dir    # Create a compressed archive of /dir
```

```
gzip file1.txt                  # Compress file1.txt
```

```
gunzip file1.txt.gz             # Decompress file1.txt.gz
```

```
zip archive.zip file1.txt       # Create a ZIP archive
```

```
unzip archive.zip               # Unzip the archive
```

18. File Search

- Commands: find, locate
- Example:

```
find / -name file1.txt          # Search for file1.txt
```

```
locate file1.txt                 # Locate the file using the updatedb cache
```

19. Basic Shell Scripting

- Commands: bash, sh
- Example:

```
#!/bin/bash
```

```
echo "Hello, World!"            # A basic shell script that prints a message
```

20. Environment Variables

- Commands: export, env, set
- Example:

```
export VAR_NAME="value"      # Set an environment variable
env                          # Display all environment variables
```

Intermediate Topics with Examples:

21. Advanced File Permissions: ACL

- Commands: getfacl, setfacl
- Example:

```
setfacl -m u:user1:rw file1.txt # Add ACL for user1 to file1.txt
getfacl file1.txt               # Display ACL of file1.txt
```

22. LVM (Logical Volume Manager)

- Commands: pvcreate, vgcreate, lvcreate, lvextend
- Example:

```
pvcreate /dev/sdb              # Create a physical volume on /dev/sdb
vgcreate vg_data /dev/sdb      # Create a volume group
lvcreate -L 10G -n lv_data vg_data # Create Lvm
```

Intermediate Topics with Examples (continued):

23. Kernel Management

- Commands: `uname`, `dmesg`, `modprobe`, `lsmod`
- Example:

```
uname -r          # Display the running kernel version
dmesg | grep -i error  # View kernel log messages for errors
modprobe -r module_name  # Remove a kernel module
lsmod             # List all loaded kernel modules
```

24. Networking Configuration

- Commands: `ip`, `nmcli`, `/etc/network/interfaces`
- Example:

```
ip addr add 192.168.1.100/24 dev eth0 # Assign IP address to eth0
nmcli device show                     # Show network device details
nmcli con up "Wired connection 1"    # Bring up a specific network connection
```

25. Firewall Management

- Commands: `firewalld`, `iptables`
- Example:

firewall-cmd --zone=public --add-port=80/tcp --permanent # Open HTTP port 80

firewall-cmd --reload # Reload firewalld rules

iptables -L # List all iptables rules

iptables -A INPUT -p tcp --dport 22 -j ACCEPT # Allow SSH access

26. SELinux

- Commands: getenforce, setenforce, semanage
- Example:

getenforce # Check if SELinux is enforcing or permissive

setenforce 0 # Temporarily set SELinux to permissive mode

semanage fcontext -a -t httpd_sys_content_t "/web(/.*)" # Allow HTTP access to custom web directory

27. Managing RAID

- Commands: mdadm
- Example:

mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sda /dev/sdb # Create RAID1 with 2 devices

mdadm --detail /dev/md0 # Show RAID array details

28. Backup and Restore

- Commands: rsync, tar, dd
- Example:

`rsync -av /source/ /destination/` # Sync files between directories

`tar -czf backup.tar.gz /mydata` # Create a compressed backup

`dd if=/dev/sda of=/backup.img bs=4M` # Create a raw disk backup

29. Swap Management

- Commands: swapon, swapoff, mkswap
- Example:

`mkswap /dev/sdb1` # Create a swap space on /dev/sdb1

`swapon /dev/sdb1` # Enable swap space

`swapoff /dev/sdb1` # Disable swap space

30. NFS (Network File System)

- Commands: mount, nfsstat, /etc/exports
- Example:

```
echo "/shared_dir 192.168.1.0/24(rw, sync, no_root_squash)" >> /etc/exports # Export a directory over NFS
```

```
exportfs -r # Apply NFS export changes
```

```
mount -t nfs 192.168.1.100:/shared_dir /mnt/nfs_mount # Mount the NFS share
```

```
nfsstat # Show NFS statistics
```

31. Samba (SMB)

- Commands: smbclient, smb.conf
- Example:

```
smbclient -L //localhost # List shared directories on a local Samba server
```

```
smb.conf: [shared] # Example Samba configuration
```

```
path = /srv/samba/shared
```

```
read only = no
```

```
browsable = yes
```

```
systemctl restart smb # Restart the Samba service to apply changes
```

32. Advanced Shell Scripting

- Commands: bash, zsh, functions, loops
- Example:

```
#!/bin/bash
```

```
function greet {  
    echo "Hello, $1"  
}  
for user in Alice Bob; do  
    greet $user  
done
```

33. Scheduling with at

- Commands: at, atq, atrm
- Example:

```
at 02:00          # Schedule a task at 2 AM  
at> echo "Backup completed" >> /var/log/backup.log  
atq              # View the queue of scheduled jobs  
atrm 1           # Remove job number 1 from the queue
```

34. Basic Performance Monitoring

- Commands: top, iotop, vmstat, sar
- Example:

```
top             # Monitor system processes in real-time  
iotop           # Monitor disk I/O usage
```

vmstat 5 # Display system memory and CPU usage every 5 seconds
sar -u 1 3 # Show CPU usage every 1 second for 3 intervals

35. System Boot Process

- Commands: systemd-analyze, grub2, /boot/
- Example:

systemd-analyze blame # Show time taken for each service during boot
grub2-mkconfig -o /boot/grub2/grub.cfg # Regenerate GRUB configuration
ls /boot # View contents of the /boot directory

36. Creating and Managing Partitions

- Commands: fdisk, parted, mkfs
- Example:

fdisk /dev/sdb # Create partitions on /dev/sdb
parted /dev/sdb mklabel gpt # Create a GPT partition table
mkfs.ext4 /dev/sdb1 # Format the partition with ext4 filesystem

37. User Authentication with PAM

- Commands: pam-auth-update, /etc/pam.d/
- Example:

```
pam-auth-update          # Update PAM authentication modules
vi /etc/pam.d/sshd       # Edit SSH PAM configuration to restrict root access
```

38. Troubleshooting with strace and lsof

- Commands: strace, lsof
- Example:

```
strace -p 1234           # Trace system calls made by process 1234
lsof -i :80              # List all processes using port 80
```

39. Setting Up a Web Server (Apache/Nginx)

- Commands: apachectl, nginx, httpd
- Example:

```
apachectl start          # Start Apache web server
vi /etc/httpd/conf/httpd.conf # Edit Apache configuration file
systemctl restart nginx  # Restart Nginx service after making changes
```


40. Database Management (MySQL/PostgreSQL)

- Commands: mysql, psql
- Example:

```
mysql -u root -p          # Log in to MySQL
```

```
CREATE DATABASE mydb;      # Create a new MySQL database
```

```
psql -U postgres          # Log in to PostgreSQL
```

```
CREATE DATABASE mydb;      # Create a new PostgreSQL database
```

Advanced Topics with Examples:

41. Kernel Tuning with sysctl

- Commands: sysctl
- Example:

```
sysctl -a                 # List all kernel parameters
```

```
sysctl net.ipv4.ip_forward=1 # Enable IP forwarding
```

```
sysctl -p                 # Apply changes from sysctl.conf
```

42. Virtualization with KVM

- Commands: virsh, virt-install
- Example:

```
virt-install --name myvm --ram 2048 --disk
path=/var/lib/libvirt/images/myvm.img,size=10 --vcpus 2 --os-type linux --os-variant
ubuntu20.04 --network bridge=virbr0 --graphics none --console pty,target_type=serial --
cdrom /var/lib/libvirt/boot/ubuntu.iso
```

```
virsh start myvm          # Start the virtual machine
```

```
virsh list                # List running virtual machines
```

43. Docker and Containerization

- Commands: docker, docker-compose
- Example:

```
docker run -d -p 80:80 nginx          # Run an Nginx container in detached mode
```

```
docker ps                        # List running containers
```

```
docker-compose up -d            # Start containers defined in a docker-compose.yml
```

44. Automation with Ansible

- Commands: ansible, ansible-playbook, ansible-inventory
- Example:

```
# Check the connectivity of the hosts using Ansible ping module
```

```
ansible all -m ping -i inventory
```

Example of running a playbook

ansible-playbook -i inventory deploy_web.yml

deploy_web.yml (sample playbook)

- hosts: webservers

become: yes

tasks:

- name: Install Nginx

yum:

name: nginx

state: present

- name: Start Nginx service

service:

name: nginx

state: started

Scenario: Automate the deployment of Nginx on multiple servers using an Ansible playbook, ensuring installation and service startup.

45. Container Orchestration with Kubernetes

- Commands: kubectl, minikube, kubeadm
- Example:

kubectl get pods

List all running pods

kubectkl apply -f deployment.yml # Apply a Kubernetes configuration (Deployment)

minikube start # Start a local Kubernetes cluster

Example of a basic deployment configuration (deployment.yml)

apiVersion: apps/v1

kind: Deployment

metadata:

 name: nginx-deployment

spec:

 replicas: 3

 selector:

 matchLabels:

 app: nginx

 template:

 metadata:

 labels:

 app: nginx

 spec:

 containers:

 - name: nginx

 image: nginx:1.14.2

 ports:

 - containerPort: 80

Scenario: Deploy a containerized Nginx application with 3 replicas using Kubernetes.

- Commands: haproxy, nginx, keepalived
- Example:

Nginx as a load balancer example

vi /etc/nginx/nginx.conf

Add the following in the config file

```
http {
```

```
    upstream backend {
```

```
        server backend1.example.com;
```

```
        server backend2.example.com;
```

```
    }
```

```
server {
```

```
    listen 80;
```

```
    location / {
```

```
        proxy_pass http://backend;
```

```
    }
```

```
}
```

```
}
```

systemctl restart nginx # Restart Nginx to apply changes

Scenario: Set up load balancing with Nginx to distribute traffic across multiple backend servers.

47. CI/CD Pipelines (Jenkins/GitLab CI)

- Commands: jenkins-cli, gitlab-runner
- Example:

Running Jenkins jobs from the command line

```
java -jar jenkins-cli.jar -s http://localhost:8080/ build my_job
```

GitLab CI pipeline example (.gitlab-ci.yml)

stages:

- build
- deploy

build_job:

stage: build

script:

- echo "Building the project"
- make build

deploy_job:

stage: deploy

script:

- echo "Deploying the project"
- scp build artifact user@server:/path/to/deploy

Scenario: Automate project build and deployment using Jenkins or GitLab CI pipelines.

48. Log Management and Centralization (ELK Stack)

- Commands: logstash, elasticsearch, kibana, filebeat

- Example:

```
# Filebeat configuration example (/etc/filebeat/filebeat.yml)
```

```
filebeat.inputs:
```

```
- type: log
```

```
paths:
```

```
- /var/log/syslog
```

```
output.logstash:
```

```
hosts: ["localhost:5044"]
```

```
# Starting Filebeat to ship logs
```

```
systemctl start filebeat
```

Scenario: Centralize and analyze logs using the ELK stack (Elasticsearch, Logstash, and Kibana) with Filebeat for log collection.

49. High Availability with Pacemaker/Corosync

- Commands: pcs, crm, corosync
- Example:

```
# Authenticate cluster nodes
```

```
pcs cluster auth node1.example.com node2.example.com
```

```
# Create and start the cluster
```

```
pcs cluster setup --name mycluster node1.example.com node2.example.com
```

```
pcs cluster start --all
```

```
# View cluster status
```

```
pcs status
```

```
# Add a resource to the cluster (e.g., an IP address resource)
```

```
pcs resource create VirtualIP ocf:heartbeat:IPaddr2 ip=192.168.1.100 cidr_netmask=24  
op monitor interval=30s
```

Scenario: Set up a highly available cluster with Pacemaker and Corosync for failover of critical resources like IP addresses or services.

50. Linux Security Auditing (Lynis/AIDE)

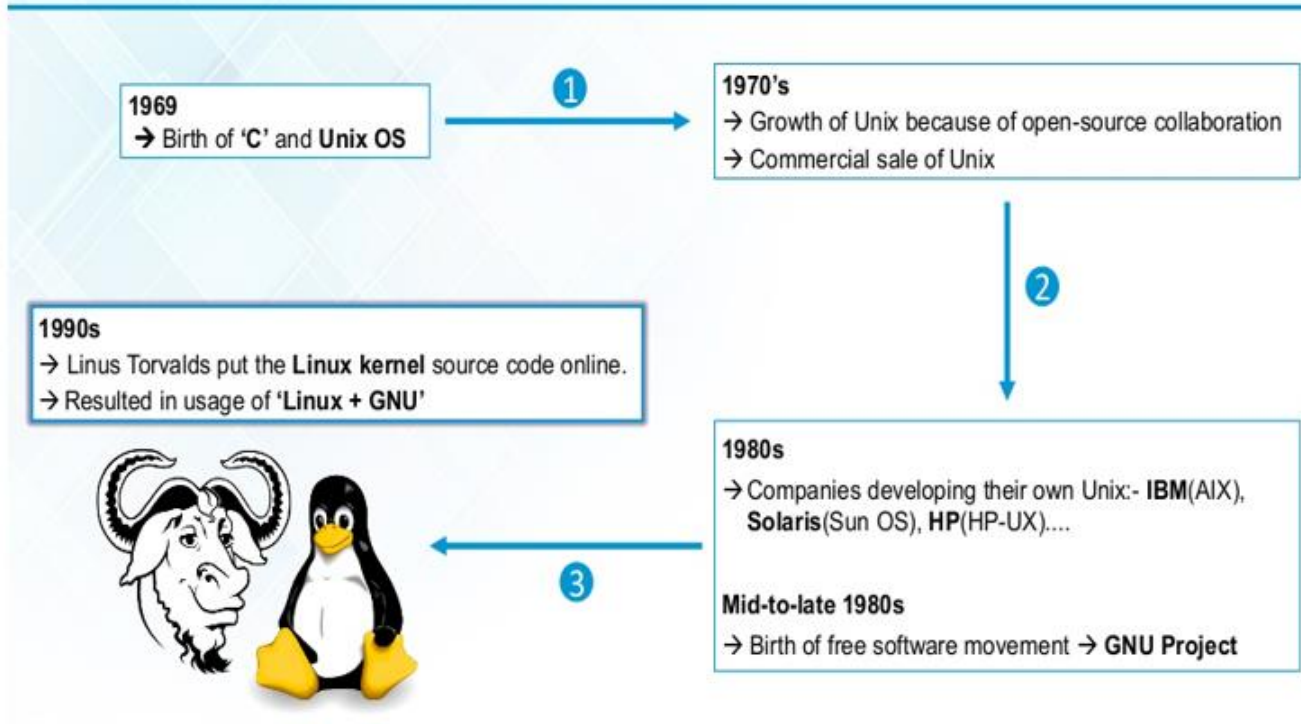
- Commands: lynis, aide, /etc/aide.conf
- Example:

```
# Run a Lynis security audit
```

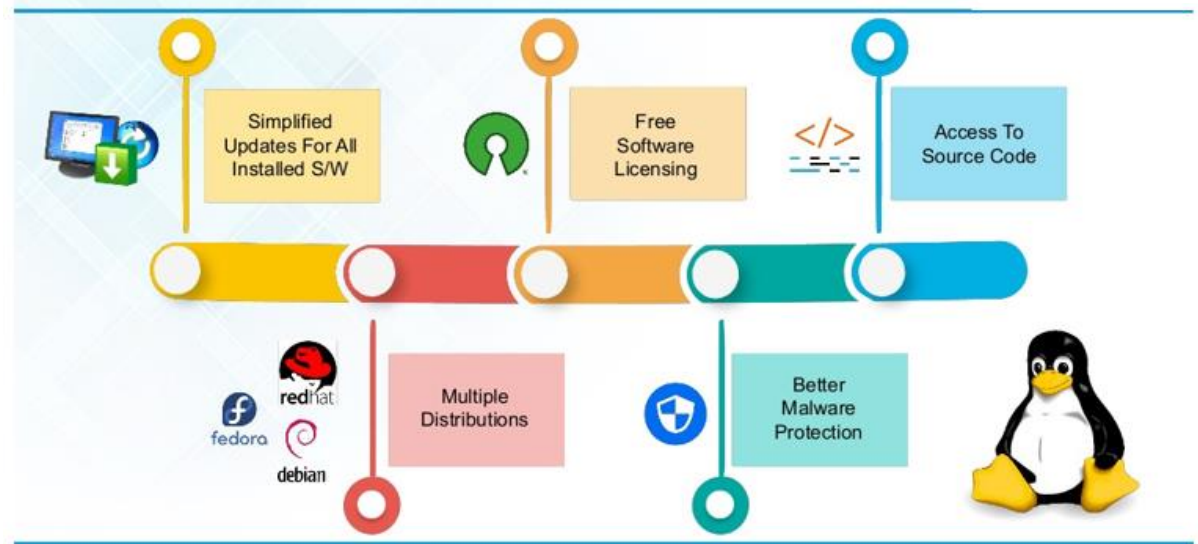
```
lynis audit system
```


Linux Fundamentals for beginners

Birth Of Linux



Linux's Features



Why Use Linux

Why use Linux?

Some of the reasons to use Linux are:

- Low cost and very stable (some Linux servers are not rebooted for over a year, try that with Windows server!)
- Best computing power and inbuilt network support.
- Fastest developing OS, with the most number of developers.
- Most secure OS.
- Configurability
- Convenience
- freedom

Unix Vs Linux

Today Linux is in great demand. You can see the use of Linux everywhere. It's dominating on our servers, desktop, smartphones and even used in some electrical devices like refrigerators.

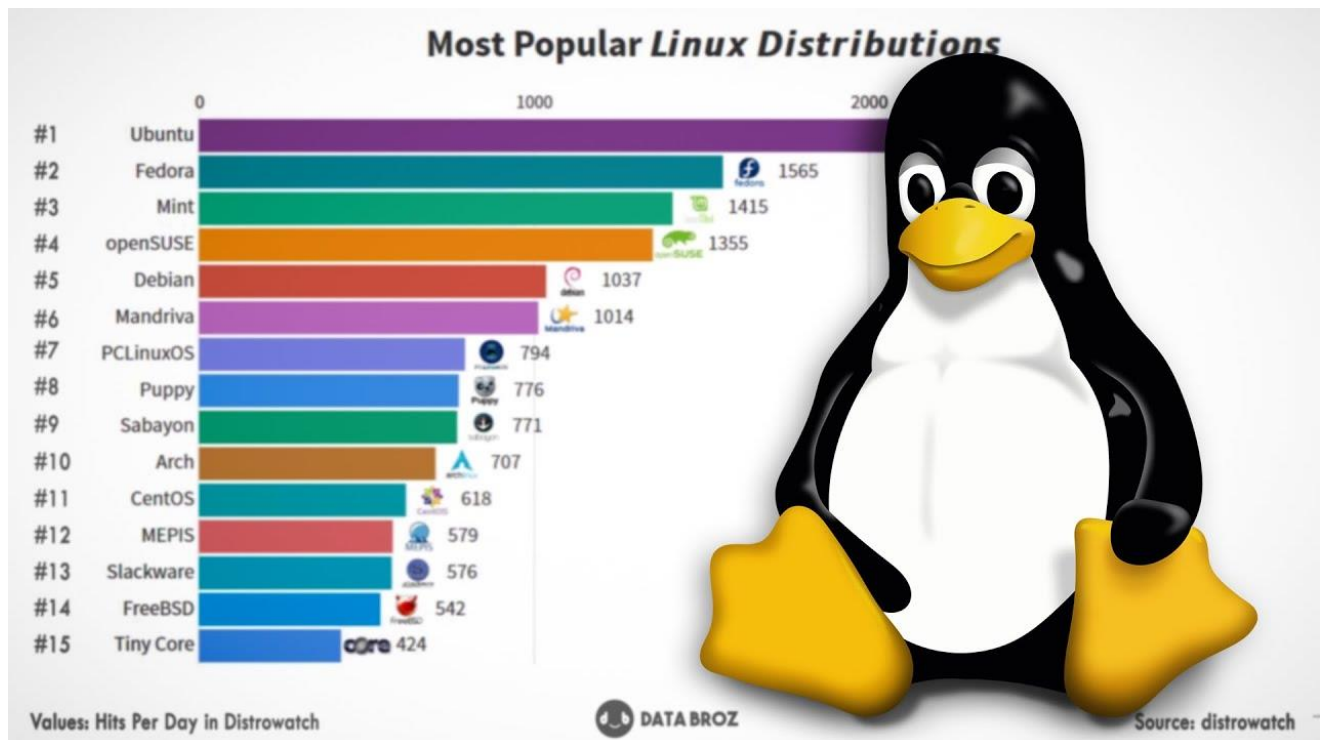
Some people think Unix and Linux as synonyms, but that's not true. Many operating systems were developed to be like Unix but none of them got the popularity as Linux. Linux is the clone of Unix. It has several features similar to Unix, still have some key differences. Before Linux and Windows, computer world was dominated by Unix. Unix is a copyrighted name and IBM AIX, HP-UX and Sun Solaris are only Unix operating system remained till date.

Difference between Linux and Unix

Comparison	Linux	Unix
Definition	It is an open-source operating system which is <i>freely available to everyone</i> .	It is an operating system which <i>can be only used by its copyrighters</i> .
Examples	It has different distros like Ubuntu, Redhat, Fedora, etc	IBM AIX, HP-UX and Sun Solaris.
Users	Nowadays, Linux is in great demand. Anyone can use Linux whether a home user, developer or a student.	It was developed mainly for servers, workstations and mainframes.

Usage	Linux is used everywhere from servers, PC, smartphones, tablets to mainframes and supercomputers.	It is used in servers, workstations and PCs.
Cost	Linux is freely distributed, downloaded, and distributed through magazines also. And priced distros of Linux are also cheaper than Windows.	Unix copyright vendors decide different costs for their respective Unix Operating systems.
Development	As it is open source, it is developed by sharing and collaboration of codes by world-wide developers.	Unix was developed by AT&T Labs, various commercial vendors and non-profit organizations.
Manufacturer	Linux kernel is developed by the community of developers from different parts of the world. Although the father of Linux, Linus Torvalds oversees things.	Unix has three distributions IBM AIX, HP-UX and Sun Solaris. Apple also uses Unix to make OSX operating system.
GUI	Linux is command based but some distros provide GUI based Linux. Gnome and KDE are mostly used GUI.	Initially it was command based OS, but later Common Desktop Environment was created. Most Unix distributions use Gnome.
Interface	The default interface is BASH (Bourne Again SHell). But some distros have developed their own interfaces.	It originally used Bourne shell. But is also compatible with other GUIs.
File system support	Linux supports more file system than Unix.	It also supports file system but lesser than Linux.
Coding	Linux is a Unix clone, behaves like Unix but doesn't contain its code.	Unix contain a completely different coding developed by AT&T Labs.
Operating system	Linux is just the kernel.	Unix is a complete package of Operating system.
Security	It provides higher security. Linux has about 60-100 viruses listed till date.	Unix is also highly secured. It has about 85-120 viruses listed till date
Error detection and solution	As Linux is open-source, whenever a user post any kind of threat, developers from all over the world start working on it. And hence, it provides faster solution.	In Unix, users have to wait for some time for the problem to be resolved.

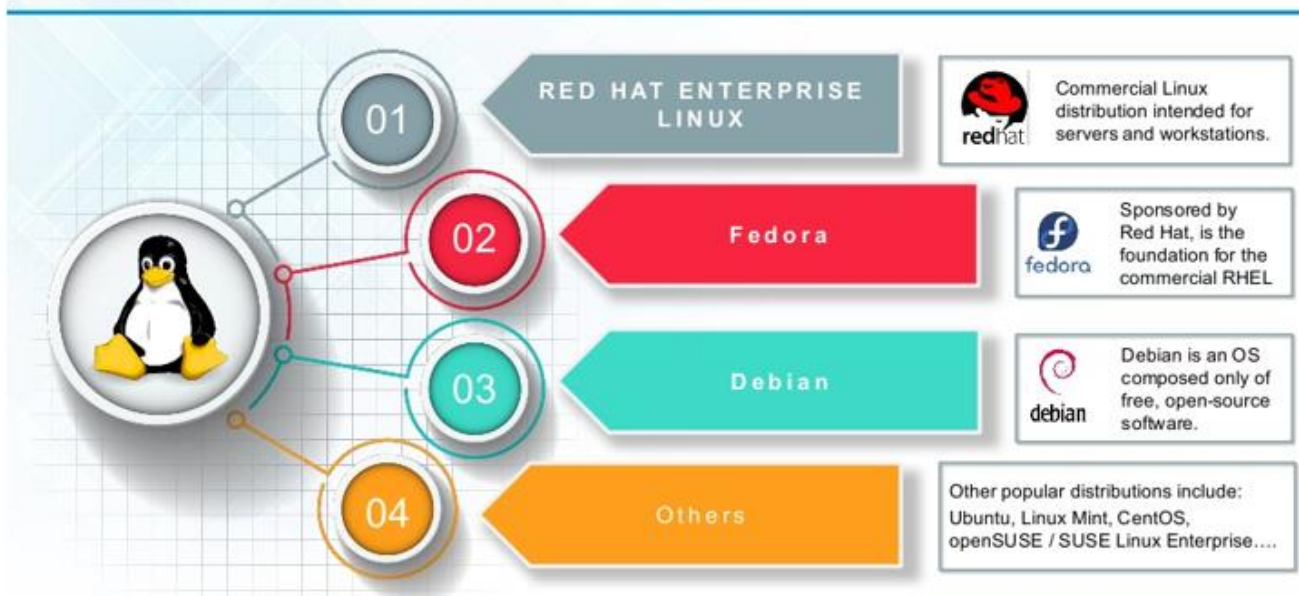
Linux Distributions (Distros)



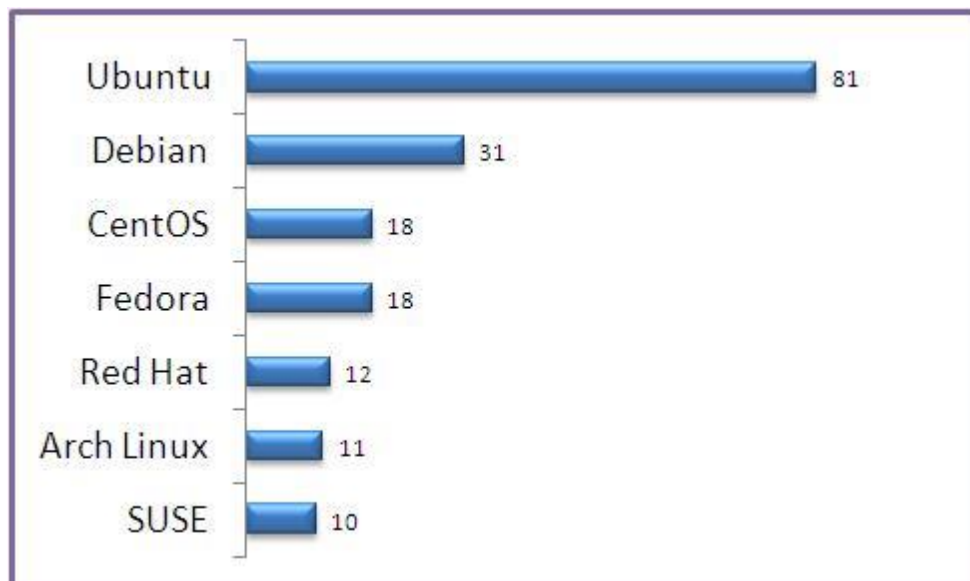
Choosing a Linux Distro

Distribution	Why To Use
UBuntu	It works like Mac OS and easy to use.
Linux mint	It works like windows and should be use by new comers.
Debian	It provides stability but not recommended to a new user.
Fedora	If you want to use red hat and latest software.
Red hat enterprise	To be used commercially.
CentOS	If you want to use red hat but without its trademark.
OpenSUSE	It works same as Fedora but slightly older and more stable.
Arch Linux	It is not for the beginners because every package has to be installed by yourself.

Various Linux Distributions



Top Linux Distribution



Chapter 1. man pages

This chapter will explain the use of **man** pages (also called **manual pages**) on your Unix or Linux computer.

You will learn the **man** command together with related commands like **whereis**, **whatis** and **mandb**.

Most Unix files and commands have pretty good man pages to explain their use. Man pages also come in handy when you are using multiple flavours of Unix or several Linux distributions since options and parameters sometimes vary.

1.1. man \$command

Type **man** followed by a command (for which you want help) and start reading. Press **q** to quit the manpage. Some man pages contain examples (near the end).

```
paul@laika:~$ man whois
Reformatting whois(1), please wait...
```

1.2. man \$configfile

Most **configuration files** have their own manual.

```
paul@laika:~$ man syslog.conf
Reformatting syslog.conf(5), please wait...
```

1.3. man \$daemon

This is also true for most **daemons** (background programs) on your system..

```
paul@laika:~$ man syslogd
Reformatting syslogd(8), please wait...
```

1.4. man -k (apropos)

man -k (or **apropos**) shows a list of man pages containing a string.

```
paul@laika:~$ man -k syslog
lm-syslog-setup (8) - configure laptop mode to switch syslog.conf ...
logger (1) - a shell command interface to the syslog(3) ...
syslog-facility (8) - Setup and remove LOCALx facility for sysklogd
syslog.conf (5) - syslogd(8) configuration file
syslogd (8) - Linux system logging utilities.
syslogd-listfiles (8) - list system logfiles
```

1.5. whatis

To see just the description of a manual page, use **whatis** followed by a string.

```
paul@u810:~$ whatis route
route (8) - show / manipulate the IP routing table
```

1.6. whereis

The location of a manpage can be revealed with **whereis**.

```
paul@laika:~$ whereis -m whois
whois: /usr/share/man/man1/whois.1.gz
```

This file is directly readable by **man**.

```
paul@laika:~$ man /usr/share/man/man1/whois.1.gz
```

1.7. man sections

By now you will have noticed the numbers between the round brackets. **man man** will explain to you that these are section numbers. Executable programs and shell commands reside in section one.

```
1 Executable programs or shell commands
2 System calls (functions provided by the kernel)
3 Library calls (functions within program libraries)
4 Special files (usually found in /dev)
5 File formats and conventions eg /etc/passwd
6 Games
7 Miscellaneous (including macro packages and conventions), e.g. man(7)
8 System administration commands (usually only for root)
9 Kernel routines [Non standard]
```

1.8. man \$section \$file

Therefor, when referring to the man page of the passwd command, you will see it written as **passwd(1)**; when referring to the **passwd file**, you will see it written as **passwd(5)**. The screenshot explains how to open the man page in the correct section.

```
[paul@RHEL52 ~]$ man passwd      # opens the first manual found
[paul@RHEL52 ~]$ man 5 passwd    # opens a page from section 5
```

1.9. man man

If you want to know more about **man**, then Read The Fantastic Manual (RTFM).

Unfortunately, manual pages do not have the answer to everything...

```
paul@laika:~$ man woman
No manual entry for woman
```

1.10. mandb

Should you be convinced that a man page exists, but you can't access it, then try running **mandb** on Debian/Mint.

```
root@laika:~# mandb
0 man subdirectories contained newer manual pages.
0 manual pages were added.
0 stray cats were added.
0 old database entries were purged.
```

Or run **makewhatis** on CentOS/Redhat.

```
[root@centos65 ~]# apropos scsi
scsi: nothing appropriate
[root@centos65 ~]# makewhatis
[root@centos65 ~]# apropos scsi
hpsa                (4)  - HP Smart Array SCSI driver
lsscsi              (8)  - list SCSI devices (or hosts) and their attributes
sd                  (4)  - Driver for SCSI Disk Drives
st                  (4)  - SCSI tape device
```

Chapter 2. working with directories

This module is a brief overview of the most common commands to work with directories: **pwd**, **cd**, **ls**, **mkdir** and **rmdir**. These commands are available on any Linux (or Unix) system.

This module also discusses **absolute** and **relative paths** and **path completion** in the **bash** shell.

2.1. pwd

The **you are here** sign can be displayed with the **pwd** command (Print Working Directory). Go ahead, try it: Open a command line interface (also called a terminal, console or xterm) and type **pwd**. The tool displays your **current directory**.

```
paul@debian8:~$ pwd
/home/paul
```

2.2. cd

You can change your current directory with the **cd** command (Change Directory).

```
paul@debian8$ cd /etc
paul@debian8$ pwd
/etc
paul@debian8$ cd /bin
paul@debian8$ pwd
/bin
paul@debian8$ cd /home/paul/
paul@debian8$ pwd
/home/paul
```

2.2.1. cd ~

The **cd** is also a shortcut to get back into your home directory. Just typing **cd** without a target directory, will put you in your home directory. Typing **cd ~** has the same effect.

```
paul@debian8$ cd /etc
paul@debian8$ pwd
/etc
paul@debian8$ cd
paul@debian8$ pwd
/home/paul
paul@debian8$ cd ~
paul@debian8$ pwd
/home/paul
```

2.2.2. cd ..

To go to the **parent directory** (the one just above your current directory in the directory tree), type **cd ..**.

```
paul@debian8$ pwd
/usr/share/games
paul@debian8$ cd ..
paul@debian8$ pwd
/usr/share
```

*To stay in the current directory, type **cd .** ;-)* We will see useful use of the **.** character representing the current directory later.

2.2.3. cd -

Another useful shortcut with **cd** is to just type **cd -** to go to the previous directory.

```
paul@debian8$ pwd
/home/paul
paul@debian8$ cd /etc
paul@debian8$ pwd
/etc
paul@debian8$ cd -
/home/paul
paul@debian8$ cd -
/etc
```

2.3. absolute and relative paths

You should be aware of **absolute** and **relative paths** in the file tree. When you type a path starting with a **slash (/)**, then the **root** of the file tree is assumed. If you don't start your path with a slash, then the current directory is the assumed starting point.

The screenshot below first shows the current directory **/home/paul**. From within this directory, you have to type **cd /home** instead of **cd home** to go to the **/home** directory.

```
paul@debian8$ pwd
/home/paul
paul@debian8$ cd home
bash: cd: home: No such file or directory
paul@debian8$ cd /home
paul@debian8$ pwd
/home
```

When inside **/home**, you have to type **cd paul** instead of **cd /paul** to enter the subdirectory **paul** of the current directory **/home**.

```
paul@debian8$ pwd
/home
paul@debian8$ cd /paul
bash: cd: /paul: No such file or directory
paul@debian8$ cd paul
paul@debian8$ pwd
/home/paul
```

In case your current directory is the **root directory /**, then both **cd /home** and **cd home** will get you in the **/home** directory.

```
paul@debian8$ pwd
/
paul@debian8$ cd home
paul@debian8$ pwd
/home
paul@debian8$ cd /
paul@debian8$ cd /home
paul@debian8$ pwd
/home
```

This was the last screenshot with **pwd** statements. From now on, the current directory will often be displayed in the prompt. Later in this book we will explain how the shell variable **\$PS1** can be configured to show this.

2.4. path completion

The **tab** key can help you in typing a path without errors. Typing **cd /et** followed by the **tab** key will expand the command line to **cd /etc/**. When typing **cd /Et** followed by the **tab** key, nothing will happen because you typed the wrong **path** (upper case E).

You will need fewer key strokes when using the **tab** key, and you will be sure your typed **path** is correct!

2.5. ls

You can list the contents of a directory with **ls**.

```
paul@debian8:~$ ls
allfiles.txt  dmesg.txt  services  stuff  summer.txt
paul@debian8:~$
```

2.5.1. ls -a

A frequently used option with **ls** is **-a** to show all files. Showing all files means including the **hidden files**. When a file name on a Linux file system starts with a dot, it is considered a **hidden file** and it doesn't show up in regular file listings.

```
paul@debian8:~$ ls
allfiles.txt  dmesg.txt  services  stuff  summer.txt
paul@debian8:~$ ls -a
.  allfiles.txt  .bash_profile  dmesg.txt  .lessht  stuff
.. .bash_history .bashrc        services   .ssh      summer.txt
paul@debian8:~$
```

2.5.2. ls -l

Many times you will be using options with **ls** to display the contents of the directory in different formats or to display different parts of the directory. Typing just **ls** gives you a list of files in the directory. Typing **ls -l** (that is a letter L, not the number 1) gives you a long listing.

```
paul@debian8:~$ ls -l
total 17296
-rw-r--r-- 1 paul paul 17584442 Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul   96650 Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul   19558 Sep 17 00:04 services
drwxr-xr-x 2 paul paul   4096 Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul     0 Sep 17 00:04 summer.txt
```

2.5.3. ls -lh

Another frequently used ls option is **-h**. It shows the numbers (file sizes) in a more human readable format. Also shown below is some variation in the way you can give the options to **ls**. We will explain the details of the output later in this book.

Note that we use the letter L as an option in this screenshot, not the number 1.

```
paul@debian8:~$ ls -l -h
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$ ls -lh
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$ ls -hl
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$ ls -h -l
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$
```

2.6. mkdir

Walking around the Unix file tree is fun, but it is even more fun to create your own directories with **mkdir**. You have to give at least one parameter to **mkdir**, the name of the new directory to be created. Think before you type a leading `/`.

```
paul@debian8:~$ mkdir mydir
paul@debian8:~$ cd mydir
paul@debian8:~/mydir$ ls -al
total 8
drwxr-xr-x  2 paul paul 4096 Sep 17 00:07 .
drwxr-xr-x 48 paul paul 4096 Sep 17 00:07 ..
paul@debian8:~/mydir$ mkdir stuff
paul@debian8:~/mydir$ mkdir otherstuff
paul@debian8:~/mydir$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 otherstuff
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 stuff
paul@debian8:~/mydir$
```

2.6.1. mkdir -p

The following command will fail, because the **parent directory** of **threedirsdeep** does not exist.

```
paul@debian8:~$ mkdir mydir2/mysubdir2/threedirsdeep
mkdir: cannot create directory 'mydir2/mysubdir2/threedirsdeep': No such fi\
le or directory
```

When given the option **-p**, then **mkdir** will create **parent directories** as needed.

```
paul@debian8:~$ mkdir -p mydir2/mysubdir2/threedirsdeep
paul@debian8:~$ cd mydir2
paul@debian8:~/mydir2$ ls -l
total 4
drwxr-xr-x 3 paul paul 4096 Sep 17 00:11 mysubdir2
paul@debian8:~/mydir2$ cd mysubdir2
paul@debian8:~/mydir2/mysubdir2$ ls -l
total 4
drwxr-xr-x 2 paul paul 4096 Sep 17 00:11 threedirsdeep
paul@debian8:~/mydir2/mysubdir2$ cd threedirsdeep/
paul@debian8:~/mydir2/mysubdir2/threedirsdeep$ pwd
/home/paul/mydir2/mysubdir2/threedirsdeep
```

2.7. rmdir

When a directory is empty, you can use **rmdir** to remove the directory.

```
paul@debian8:~/mydir$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 otherstuff
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 stuff
paul@debian8:~/mydir$ rmdir otherstuff
paul@debian8:~/mydir$ cd ..
paul@debian8:~$ rmdir mydir
rmdir: failed to remove 'mydir': Directory not empty
paul@debian8:~$ rmdir mydir/stuff
paul@debian8:~$ rmdir mydir
paul@debian8:~$
```

2.7.1. rmdir -p

And similar to the **mkdir -p** option, you can also use **rmdir** to recursively remove directories.

```
paul@debian8:~$ mkdir -p test42/subdir
paul@debian8:~$ rmdir -p test42/subdir
paul@debian8:~$
```

2.8. solution: working with directories

1. Display your current directory.

```
pwd
```

2. Change to the /etc directory.

```
cd /etc
```

3. Now change to your home directory using only three key presses.

```
cd (and the enter key)
```

4. Change to the /boot/grub directory using only eleven key presses.

```
cd /boot/grub (use the tab key)
```

5. Go to the parent directory of the current directory.

```
cd .. (with space between cd and ..)
```

6. Go to the root directory.

```
cd /
```

7. List the contents of the root directory.

```
ls
```

8. List a long listing of the root directory.

```
ls -l
```

9. Stay where you are, and list the contents of /etc.

```
ls /etc
```

10. Stay where you are, and list the contents of /bin and /sbin.

```
ls /bin /sbin
```

11. Stay where you are, and list the contents of ~.

```
ls ~
```

12. List all the files (including hidden files) in your home directory.

```
ls -al ~
```

13. List the files in /boot in a human readable format.

```
ls -lh /boot
```

14. Create a directory testdir in your home directory.

```
mkdir ~/testdir
```

15. Change to the /etc directory, stay here and create a directory newdir in your home directory.


```
cd /etc ; mkdir ~/newdir
```

16. Create in one command the directories ~/dir1/dir2/dir3 (dir3 is a subdirectory from dir2, and dir2 is a subdirectory from dir1).

```
mkdir -p ~/dir1/dir2/dir3
```

17. Remove the directory testdir.

```
rmdir testdir
```

18. If time permits (or if you are waiting for other students to finish this practice), use and understand **pushd** and **popd**. Use the man page of **bash** to find information about these commands.

```
man bash          # opens the manual
/pushd            # searches for pushd
n                 # next (do this two/three times)
```

The Bash shell has two built-in commands called **pushd** and **popd**. Both commands work with a common stack of previous directories. Pushd adds a directory to the stack and changes to a new current directory, popd removes a directory from the stack and sets the current directory.

```
paul@debian7:/etc$ cd /bin
paul@debian7:/bin$ pushd /lib
/lib /bin
paul@debian7:/lib$ pushd /proc
/proc /lib /bin
paul@debian7:/proc$ popd
/lib /bin
paul@debian7:/lib$ popd
/bin
```

Chapter 3. working with files

In this chapter we learn how to recognise, create, remove, copy and move files using commands like **file**, **touch**, **rm**, **cp**, **mv** and **rename**.

3.1. all files are case sensitive

Files on Linux (or any Unix) are **case sensitive**. This means that **FILE1** is different from **file1**, and **/etc/hosts** is different from **/etc/Hosts** (the latter one does not exist on a typical Linux computer).

This screenshot shows the difference between two files, one with upper case **W**, the other with lower case **w**.

```
paul@laika:~/Linux$ ls
winter.txt  Winter.txt
paul@laika:~/Linux$ cat winter.txt
It is cold.
paul@laika:~/Linux$ cat Winter.txt
It is very cold!
```

3.2. everything is a file

A **directory** is a special kind of **file**, but it is still a (case sensitive!) **file**. Each terminal window (for example **/dev/pts/4**), any hard disk or partition (for example **/dev/sdb1**) and any process are all represented somewhere in the **file system** as a **file**. It will become clear throughout this course that everything on Linux is a **file**.

3.3. file

The **file** utility determines the file type. Linux does not use extensions to determine the file type. The command line does not care whether a file ends in **.txt** or **.pdf**. As a system administrator, you should use the **file** command to determine the file type. Here are some examples on a typical Linux system.

```
paul@laika:~$ file pic33.png
pic33.png: PNG image data, 3840 x 1200, 8-bit/color RGBA, non-interlaced
paul@laika:~$ file /etc/passwd
/etc/passwd: ASCII text
paul@laika:~$ file HelloWorld.c
HelloWorld.c: ASCII C program text
```

The **file** command uses a magic file that contains patterns to recognise file types. The magic file is located in **/usr/share/file/magic**. Type **man 5 magic** for more information.

It is interesting to point out **file -s** for special files like those in **/dev** and **/proc**.

```
root@debian6~# file /dev/sda
/dev/sda: block special
root@debian6~# file -s /dev/sda
/dev/sda: x86 boot sector; partition 1: ID=0x83, active, starthead...
root@debian6~# file /proc/cpuinfo
/proc/cpuinfo: empty
root@debian6~# file -s /proc/cpuinfo
/proc/cpuinfo: ASCII C++ program text
```

3.4. touch

3.4.1. create an empty file

One easy way to create an empty file is with **touch**. (We will see many other ways for creating files later in this book.)

This screenshot starts with an empty directory, creates two files with **touch** and the lists those files.

```
paul@debian7:~$ ls -l
total 0
paul@debian7:~$ touch file42
paul@debian7:~$ touch file33
paul@debian7:~$ ls -l
total 0
-rw-r--r-- 1 paul paul 0 Oct 15 08:57 file33
-rw-r--r-- 1 paul paul 0 Oct 15 08:56 file42
paul@debian7:~$
```

3.4.2. touch -t

The **touch** command can set some properties while creating empty files. Can you determine what is set by looking at the next screenshot? If not, check the manual for **touch**.

```
paul@debian7:~$ touch -t 200505050000 SinkoDeMayo
paul@debian7:~$ touch -t 130207111630 BigBattle.txt
paul@debian7:~$ ls -l
total 0
-rw-r--r-- 1 paul paul 0 Jul 11 1302 BigBattle.txt
-rw-r--r-- 1 paul paul 0 Oct 15 08:57 file33
-rw-r--r-- 1 paul paul 0 Oct 15 08:56 file42
-rw-r--r-- 1 paul paul 0 May 5 2005 SinkoDeMayo
paul@debian7:~$
```

3.5. rm

3.5.1. remove forever

When you no longer need a file, use **rm** to remove it. Unlike some graphical user interfaces, the command line in general does not have a **waste bin** or **trash can** to recover files. When you use **rm** to remove a file, the file is gone. Therefore, be careful when removing files!

```
paul@debian7:~$ ls
BigBattle.txt  file33  file42  SinkoDeMayo
paul@debian7:~$ rm BigBattle.txt
paul@debian7:~$ ls
file33  file42  SinkoDeMayo
paul@debian7:~$
```

3.5.2. rm -i

To prevent yourself from accidentally removing a file, you can type **rm -i**.

```
paul@debian7:~$ ls
file33  file42  SinkoDeMayo
paul@debian7:~$ rm -i file33
rm: remove regular empty file `file33'? yes
paul@debian7:~$ rm -i SinkoDeMayo
rm: remove regular empty file `SinkoDeMayo'? n
paul@debian7:~$ ls
file42  SinkoDeMayo
paul@debian7:~$
```

3.5.3. rm -rf

By default, **rm -r** will not remove non-empty directories. However **rm** accepts several options that will allow you to remove any directory. The **rm -rf** statement is famous because it will erase anything (providing that you have the permissions to do so). When you are logged on as root, be very careful with **rm -rf** (the **f** means **force** and the **r** means **recursive**) since being root implies that permissions don't apply to you. You can literally erase your entire file system by accident.

```
paul@debian7:~$ mkdir test
paul@debian7:~$ rm test
rm: cannot remove `test': Is a directory
paul@debian7:~$ rm -rf test
paul@debian7:~$ ls test
ls: cannot access test: No such file or directory
paul@debian7:~$
```

3.6. cp

3.6.1. copy one file

To copy a file, use **cp** with a source and a target argument.

```
paul@debian7:~$ ls
file42 SinkoDeMayo
paul@debian7:~$ cp file42 file42.copy
paul@debian7:~$ ls
file42 file42.copy SinkoDeMayo
```

3.6.2. copy to another directory

If the target is a directory, then the source files are copied to that target directory.

```
paul@debian7:~$ mkdir dir42
paul@debian7:~$ cp SinkoDeMayo dir42
paul@debian7:~$ ls dir42/
SinkoDeMayo
```

3.6.3. cp -r

To copy complete directories, use **cp -r** (the **-r** option forces **recursive** copying of all files in all subdirectories).

```
paul@debian7:~$ ls
dir42 file42 file42.copy SinkoDeMayo
paul@debian7:~$ cp -r dir42/ dir33
paul@debian7:~$ ls
dir33 dir42 file42 file42.copy SinkoDeMayo
paul@debian7:~$ ls dir33/
SinkoDeMayo
```

3.6.4. copy multiple files to directory

You can also use **cp** to copy multiple files into a directory. In this case, the last argument (a.k.a. the target) must be a directory.

```
paul@debian7:~$ cp file42 file42.copy SinkoDeMayo dir42/
paul@debian7:~$ ls dir42/
file42 file42.copy SinkoDeMayo
```

3.6.5. cp -i

To prevent **cp** from overwriting existing files, use the **-i** (for interactive) option.

```
paul@debian7:~$ cp SinkoDeMayo file42
paul@debian7:~$ cp SinkoDeMayo file42
paul@debian7:~$ cp -i SinkoDeMayo file42
cp: overwrite `file42'? n
paul@debian7:~$
```

3.7. mv

3.7.1. rename files with mv

Use **mv** to rename a file or to move the file to another directory.

```
paul@debian7:~$ ls
dir33 dir42 file42 file42.copy SinkoDeMayo
paul@debian7:~$ mv file42 file33
paul@debian7:~$ ls
dir33 dir42 file33 file42.copy SinkoDeMayo
paul@debian7:~$
```

When you need to rename only one file then **mv** is the preferred command to use.

3.7.2. rename directories with mv

The same **mv** command can be used to rename directories.

```
paul@debian7:~$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 dir33
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 dir42
-rw-r--r-- 1 paul paul  0 Oct 15 09:38 file33
-rw-r--r-- 1 paul paul  0 Oct 15 09:16 file42.copy
-rw-r--r-- 1 paul paul  0 May  5 2005 SinkoDeMayo
paul@debian7:~$ mv dir33 backup
paul@debian7:~$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 backup
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 dir42
-rw-r--r-- 1 paul paul  0 Oct 15 09:38 file33
-rw-r--r-- 1 paul paul  0 Oct 15 09:16 file42.copy
-rw-r--r-- 1 paul paul  0 May  5 2005 SinkoDeMayo
paul@debian7:~$
```

3.7.3. mv -i

The **mv** also has a **-i** switch similar to **cp** and **rm**.

this screenshot shows that **mv -i** will ask permission to overwrite an existing file.

```
paul@debian7:~$ mv -i file33 SinkoDeMayo
mv: overwrite `SinkoDeMayo'? no
paul@debian7:~$
```


3.8. rename

3.8.1. about rename

The **rename** command is one of the rare occasions where the Linux Fundamentals book has to make a distinction between Linux distributions. Almost every command in the **Fundamentals** part of this book works on almost every Linux computer. But **rename** is different.

Try to use **mv** whenever you need to rename only a couple of files.

3.8.2. rename on Debian/Ubuntu

The **rename** command on Debian uses regular expressions (regular expression or short regex are explained in a later chapter) to rename many files at once.

Below a **rename** example that switches all occurrences of txt to png for all file names ending in .txt.

```
paul@debian7:~/test42$ ls
abc.txt file33.txt file42.txt
paul@debian7:~/test42$ rename 's/\.txt/\.png/' *.txt
paul@debian7:~/test42$ ls
abc.png file33.png file42.png
```

This second example switches all (first) occurrences of **file** into **document** for all file names ending in .png.

```
paul@debian7:~/test42$ ls
abc.png file33.png file42.png
paul@debian7:~/test42$ rename 's/file/document/' *.png
paul@debian7:~/test42$ ls
abc.png document33.png document42.png
paul@debian7:~/test42$
```

3.8.3. rename on CentOS/RHEL/Fedora

On Red Hat Enterprise Linux, the syntax of **rename** is a bit different. The first example below renames all *.conf files replacing any occurrence of .conf with .backup.

```
[paul@centos7 ~]$ touch one.conf two.conf three.conf
[paul@centos7 ~]$ rename .conf .backup *.conf
[paul@centos7 ~]$ ls
one.backup three.backup two.backup
[paul@centos7 ~]$
```

The second example renames all (*) files replacing one with ONE.

```
[paul@centos7 ~]$ ls
one.backup three.backup two.backup
[paul@centos7 ~]$ rename one ONE *
[paul@centos7 ~]$ ls
ONE.backup three.backup two.backup
[paul@centos7 ~]$
```

3.9. practice: working with files

1. List the files in the /bin directory
2. Display the type of file of /bin/cat, /etc/passwd and /usr/bin/passwd.
- 3a. Download wolf.jpg and LinuxFun.pdf from <http://linux-training.be> (wget http:// linux-training.be/files/studentfiles/wolf.jpg and wget http://linux-training.be/files/books/LinuxFun.pdf)

```
wget http://linux-training.be/files/studentfiles/wolf.jpg  
wget http://linux-training.be/files/studentfiles/wolf.png  
wget http://linux-training.be/files/books/LinuxFun.pdf
```

- 3b. Display the type of file of wolf.jpg and LinuxFun.pdf
- 3c. Rename wolf.jpg to wolf.pdf (use mv).
- 3d. Display the type of file of wolf.pdf and LinuxFun.pdf.
4. Create a directory ~/touched and enter it.
5. Create the files today.txt and yesterday.txt in touched.
6. Change the date on yesterday.txt to match yesterday's date.
7. Copy yesterday.txt to copy.yesterday.txt
8. Rename copy.yesterday.txt to kim
9. Create a directory called ~/testbackup and copy all files from ~/touched into it.
10. Use one command to remove the directory ~/testbackup and all files into it.
11. Create a directory ~/etcbackup and copy all *.conf files from /etc into it. Did you include all subdirectories of /etc ?
12. Use rename to rename all *.conf files to *.backup . (if you have more than one distro available, try it on all!)

3.10. solution: working with files

1. List the files in the /bin directory

```
ls /bin
```

2. Display the type of file of /bin/cat, /etc/passwd and /usr/bin/passwd.

```
file /bin/cat /etc/passwd /usr/bin/passwd
```

3a. Download wolf.jpg and LinuxFun.pdf from <http://linux-training.be> (wget [http:// linux-training.be/files/studentfiles/wolf.jpg](http://linux-training.be/files/studentfiles/wolf.jpg) and wget <http://linux-training.be/files/books/LinuxFun.pdf>)

```
wget http://linux-training.be/files/studentfiles/wolf.jpg
wget http://linux-training.be/files/studentfiles/wolf.png
wget http://linux-training.be/files/books/LinuxFun.pdf
```

3b. Display the type of file of wolf.jpg and LinuxFun.pdf

```
file wolf.jpg LinuxFun.pdf
```

3c. Rename wolf.jpg to wolf.pdf (use mv).

```
mv wolf.jpg wolf.pdf
```

3d. Display the type of file of wolf.pdf and LinuxFun.pdf.

```
file wolf.pdf LinuxFun.pdf
```

4. Create a directory ~/touched and enter it.

```
mkdir ~/touched ; cd ~/touched
```

5. Create the files today.txt and yesterday.txt in touched.

```
touch today.txt yesterday.txt
```

6. Change the date on yesterday.txt to match yesterday's date.

```
touch -t 200810251405 yesterday.txt (substitute 20081025 with yesterday)
```

7. Copy yesterday.txt to copy.yesterday.txt

```
cp yesterday.txt copy.yesterday.txt
```

8. Rename copy.yesterday.txt to kim

```
mv copy.yesterday.txt kim
```

9. Create a directory called ~/testbackup and copy all files from ~/touched into it.

```
mkdir ~/testbackup ; cp -r ~/touched ~/testbackup/
```

10. Use one command to remove the directory ~/testbackup and all files into it.

```
rm -rf ~/testbackup
```

11. Create a directory ~/etcbackup and copy all *.conf files from /etc into it. Did you include all subdirectories of /etc ?

```
cp -r /etc/*.conf ~/etcbackup
```

Only *.conf files that are directly in /etc/ are copied.

12. Use rename to rename all *.conf files to *.backup . (if you have more than one distro available, try it on all!)

```
On RHEL: touch 1.conf 2.conf ; rename conf backup *.conf
```

```
On Debian: touch 1.conf 2.conf ; rename 's/conf/backup/' *.conf
```

Chapter 4. working with file contents

In this chapter we will look at the contents of **text files** with **head**, **tail**, **cat**, **tac**, **more**, **less** and **strings**.

We will also get a glimpse of the possibilities of tools like **cat** on the command line.

4.1. head

You can use **head** to display the first ten lines of a file.

```
paul@debian7~$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
root@debian7~#
```

The **head** command can also display the first **n** lines of a file.

```
paul@debian7~$ head -4 /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
paul@debian7~$
```

And **head** can also display the first **n** bytes.

```
paul@debian7~$ head -c14 /etc/passwd
root:x:0:0:roopaul@debian7~$
```

4.2. tail

Similar to **head**, the **tail** command will display the last ten lines of a file.

```
paul@debian7~$ tail /etc/services
vboxd          20012/udp
binkp          24554/tcp      # binkp fidonet protocol
asp            27374/tcp      # Address Search Protocol
asp            27374/udp
csync2         30865/tcp      # cluster synchronization tool
dircproxy      57000/tcp      # Detachable IRC Proxy
tfido          60177/tcp      # fidonet EMSI over telnet
fido           60179/tcp      # fidonet EMSI over TCP

# Local services
paul@debian7~$
```

You can give **tail** the number of lines you want to see.

```
paul@debian7~$ tail -3 /etc/services
fido           60179/tcp      # fidonet EMSI over TCP

# Local services
paul@debian7~$
```

The **tail** command has other useful options, some of which we will use during this course.

4.3. cat

The **cat** command is one of the most universal tools, yet all it does is copy **standard input** to **standard output**. In combination with the shell this can be very powerful and diverse. Some examples will give a glimpse into the possibilities. The first example is simple, you can use **cat** to display a file on the screen. If the file is longer than the screen, it will scroll to the end.

```
paul@debian8:~$ cat /etc/resolv.conf
domain linux-training.be
search linux-training.be
nameserver 192.168.1.42
```

4.3.1. concatenate

cat is short for **concatenate**. One of the basic uses of **cat** is to concatenate files into a bigger (or complete) file.

```
paul@debian8:~$ echo one >part1
paul@debian8:~$ echo two >part2
paul@debian8:~$ echo three >part3
paul@debian8:~$ cat part1
one
paul@debian8:~$ cat part2
two
paul@debian8:~$ cat part3
three
paul@debian8:~$ cat part1 part2 part3
one
two
three
paul@debian8:~$ cat part1 part2 part3 >all
paul@debian8:~$ cat all
one
two
three
paul@debian8:~$
```

4.3.2. create files

You can use **cat** to create flat text files. Type the **cat > winter.txt** command as shown in the screenshot below. Then type one or more lines, finishing each line with the enter key. After the last line, type and hold the Control (Ctrl) key and press d.

```
paul@debian8:~$ cat > winter.txt
It is very cold today!
paul@debian8:~$ cat winter.txt
It is very cold today!
paul@debian8:~$
```

The **Ctrl d** key combination will send an **EOF** (End of File) to the running process ending the **cat** command.

4.3.3. custom end marker

You can choose an end marker for **cat** with **<<** as is shown in this screenshot. This construction is called a **here directive** and will end the **cat** command.

```
paul@debian8:~$ cat > hot.txt <<stop
> It is hot today!
> Yes it is summer.
> stop
paul@debian8:~$ cat hot.txt
It is hot today!
Yes it is summer.
paul@debian8:~$
```

4.3.4. copy files

In the third example you will see that **cat** can be used to copy files. We will explain in detail what happens here in the bash shell chapter.

```
paul@debian8:~$ cat winter.txt
It is very cold today!
paul@debian8:~$ cat winter.txt > cold.txt
paul@debian8:~$ cat cold.txt
It is very cold today!
paul@debian8:~$
```

4.4. tac

Just one example will show you the purpose of **tac** (cat backwards).

```
paul@debian8:~$ cat count
one
two
three
four
paul@debian8:~$ tac count
four
three
two
one
```


4.5. more and less

The **more** command is useful for displaying files that take up more than one screen. More will allow you to see the contents of the file page by page. Use the space bar to see the next page, or **q** to quit. Some people prefer the **less** command to **more**.

4.6. strings

With the **strings** command you can display readable ascii strings found in (binary) files. This example locates the **ls** binary then displays readable strings in the binary file (output is truncated).

```
paul@laika:~$ which ls
/bin/ls
paul@laika:~$ strings /bin/ls
/lib/ld-linux.so.2
librt.so.1
__gmon_start__
__Jv_RegisterClasses
clock_gettime
libacl.so.1
...
```

4.7. solution: file contents

1. Display the first 12 lines of **/etc/services**.

```
head -12 /etc/services
```

2. Display the last line of **/etc/passwd**.

```
tail -1 /etc/passwd
```

3. Use **cat** to create a file named **count.txt** that looks like this:

```
cat > count.txt
One
Two
Three
Four
Five (followed by Ctrl-d)
```

4. Use **cp** to make a backup of this file to **cnt.txt**.

```
cp count.txt cnt.txt
```

5. Use **cat** to make a backup of this file to **catcnt.txt**.

```
cat count.txt > catcnt.txt
```

6. Display **catcnt.txt**, but with all lines in reverse order (the last line first).

```
tac catcnt.txt
```

7. Use **more** to display **/etc/services**.

```
more /etc/services
```

8. Display the readable character strings from the **/usr/bin/passwd** command.

```
strings /usr/bin/passwd
```

9. Use **ls** to find the biggest file in **/etc**.

```
ls -lrS /etc
```

10. Open two terminal windows (or tabs) and make sure you are in the same directory in both. Type **echo this is the first line > tailing.txt** in the first terminal, then issue **tail -f tailing.txt** in the second terminal. Now go back to the first terminal and type **echo This is another line >> tailing.txt** (note the double >>), verify that the **tail -f** in the second terminal shows both lines. Stop the **tail -f** with **Ctrl-C**.

11. Use **cat** to create a file named **tailing.txt** that contains the contents of **tailing.txt** followed by the contents of **/etc/passwd**.

```
cat /etc/passwd >> tailing.txt
```

12. Use **cat** to create a file named **tailing.txt** that contains the contents of **tailing.txt** preceded by the contents of **/etc/passwd**.

```
mv tailing.txt tmp.txt ; cat /etc/passwd tmp.txt > tailing.txt
```

Chapter 5. the Linux file tree

This chapter takes a look at the most common directories in the **Linux file tree**. It also shows that on Unix everything is a file.

5.1. filesystem hierarchy standard

Many Linux distributions partially follow the **Filesystem Hierarchy Standard**. The **FHS** may help make more Unix/Linux file system trees conform better in the future. The **FHS** is available online at <http://www.pathname.com/fhs/> where we read: "The filesystem hierarchy standard has been designed to be used by Unix distribution developers, package developers, and system implementers. However, it is primarily intended to be a reference and is not a tutorial on how to manage a Unix filesystem or directory hierarchy."

5.2. man hier

There are some differences in the filesystems between **Linux distributions**. For help about your machine, enter **man hier** to find information about the file system hierarchy. This manual will explain the directory structure on your computer.

5.3. the root directory /

All Linux systems have a directory structure that starts at the **root directory**. The root directory is represented by a **forward slash**, like this: /. Everything that exists on your Linux system can be found below this root directory. Let's take a brief look at the contents of the root directory.

```
[paul@RHELv4u3 ~]$ ls /  
bin  dev  home  media  mnt  proc  sbin  srv  tftpboot  usr  
boot  etc  lib  misc  opt  root  selinux  sys  tmp  var
```

5.4. binary directories

Binaries are files that contain compiled source code (or machine code). Binaries can be **executed** on the computer. Sometimes binaries are called **executables**.

5.4.1. /bin

The **/bin** directory contains **binaries** for use by all users. According to the FHS the **/bin** directory should contain **/bin/cat** and **/bin/date** (among others).

In the screenshot below you see common Unix/Linux commands like cat, cp, cpio, date, dd, echo, grep, and so on. Many of these will be covered in this book.

```
paul@laika:~$ ls /bin
archdetect      egrep           mt              setupcon
autopartition   false          mt-gnu         sh
bash            fgconsole      mv             sh.distrib
bunzip2         fgrep          nano           sleep
bzipcat         fuser          nc             stralign
bzcmp           fusermount     nc.traditional stty
bzdiff          get_mouptions  netcat         su
bzegrep         grep           netstat        sync
bzexe           gunzip         ntfs-3g        sysfs
bzfgrep         gzexe         ntfs-3g.probe  tailf
bzgrep          gzip           parted_devices tar
bzip2           hostname       parted_server  tempfile
bzip2recover    hw-detect      partman        touch
bzless          ip             partman-commit true
bzmore          kbd_mode       perform_recipe ulockmgr
cat             kill           pidof          umount
...
```

5.4.2. other /bin directories

You can find a **/bin subdirectory** in many other directories. A user named **serena** could put her own programs in **/home/serena/bin**.

Some applications, often when installed directly from source will put themselves in **/opt**. A **samba server** installation can use **/opt/samba/bin** to store its binaries.

5.4.3. /sbin

/sbin contains binaries to configure the operating system. Many of the **system binaries** require **root** privilege to perform certain tasks.

Below a screenshot containing **system binaries** to change the ip address, partition a disk and create an ext4 file system.

```
paul@ubu1010:~$ ls -l /sbin/ifconfig /sbin/fdisk /sbin/mkfs.ext4
-rwxr-xr-x 1 root root 97172 2011-02-02 09:56 /sbin/fdisk
-rwxr-xr-x 1 root root 65708 2010-07-02 09:27 /sbin/ifconfig
-rwxr-xr-x 5 root root 55140 2010-08-18 18:01 /sbin/mkfs.ext4
```

5.4.4. /lib

Binaries found in **/bin** and **/sbin** often use **shared libraries** located in **/lib**. Below is a screenshot of the partial contents of **/lib**.

```
paul@laika:~$ ls /lib/libc*
/lib/libc-2.5.so      /lib/libcfont.so.0.0.0  /lib/libcom_err.so.2.1
/lib/libcap.so.1      /lib/libcidn-2.5.so     /lib/libconsole.so.0
/lib/libcap.so.1.10   /lib/libcidn.so.1       /lib/libconsole.so.0.0.0
/lib/libcfont.so.0    /lib/libcom_err.so.2    /lib/libcrypt-2.5.so
```

/lib/modules

Typically, the **Linux kernel** loads kernel modules from **/lib/modules/\$kernel-version/**. This directory is discussed in detail in the Linux kernel chapter.

/lib32 and /lib64

We currently are in a transition between **32-bit** and **64-bit** systems. Therefore, you may encounter directories named **/lib32** and **/lib64** which clarify the register size used during compilation time of the libraries. A 64-bit computer may have some 32-bit binaries and libraries for compatibility with legacy applications. This screenshot uses the **file** utility to demonstrate the difference.

```
paul@laika:~$ file /lib32/libc-2.5.so
/lib32/libc-2.5.so: ELF 32-bit LSB shared object, Intel 80386, \
version 1 (SYSV), for GNU/Linux 2.6.0, stripped
paul@laika:~$ file /lib64/libcap.so.1.10
/lib64/libcap.so.1.10: ELF 64-bit LSB shared object, AMD x86-64, \
version 1 (SYSV), stripped
```

The ELF (**Executable and Linkable Format**) is used in almost every Unix-like operating system since **System V**.

5.4.5. /opt

The purpose of **/opt** is to store **optional** software. In many cases this is software from outside the distribution repository. You may find an empty **/opt** directory on many systems.

A large package can install all its files in **/bin**, **/lib**, **/etc** subdirectories within **/opt/\$packagename/**. If for example the package is called **wp**, then it installs in **/opt/wp**, putting binaries in **/opt/wp/bin** and manpages in **/opt/wp/man**.

5.5. configuration directories

5.5.1. /boot

The **/boot** directory contains all files needed to boot the computer. These files don't change very often. On Linux systems you typically find the **/boot/grub** directory here. **/boot/grub** contains **/boot/grub/grub.cfg** (older systems may still have **/boot/grub/grub.conf**) which defines the boot menu that is displayed before the kernel starts.

5.5.2. /etc

All of the machine-specific **configuration files** should be located in **/etc**. Historically **/etc** stood for **etcetera**, today people often use the **Editable Text Configuration** backronym.

Many times the name of a configuration files is the same as the application, daemon, or protocol with **.conf** added as the extension.

```
paul@laika:~$ ls /etc/*.conf
/etc/adduser.conf          /etc/ld.so.conf          /etc/scrollkeeper.conf
/etc/brltty.conf           /etc/lftp.conf           /etc/sysctl.conf
/etc/ccertificates.conf    /etc/libao.conf          /etc/syslog.conf
/etc/cvs-cron.conf         /etc/logrotate.conf      /etc/ucf.conf
/etc/ddclient.conf         /etc/ltrace.conf         /etc/uniconf.conf
/etc/debconf.conf          /etc/mke2fs.conf         /etc/updatedb.conf
/etc/deluser.conf          /etc/netscsid.conf       /etc/usplash.conf
/etc/fdmount.conf          /etc/nsswitch.conf       /etc/uswsusp.conf
/etc/hdparm.conf           /etc/pam.conf            /etc/vnc.conf
/etc/host.conf             /etc/pnm2ppa.conf        /etc/wodim.conf
/etc/inetd.conf            /etc/povray.conf         /etc/wvdial.conf
/etc/kernel-img.conf       /etc/resolv.conf
paul@laika:~$
```

There is much more to be found in **/etc**.

/etc/init.d/

A lot of Unix/Linux distributions have an **/etc/init.d** directory that contains scripts to start and stop **daemons**. This directory could disappear as Linux migrates to systems that replace the old **init** way of starting all **daemons**.

/etc/X11/

The graphical display (aka **X Window System** or just **X**) is driven by software from the X.org foundation. The configuration file for your graphical display is **/etc/X11/xorg.conf**.

/etc/skel/

The **skeleton** directory **/etc/skel** is copied to the home directory of a newly created user. It usually contains hidden files like a **.bashrc** script.

/etc/sysconfig/

This directory, which is not mentioned in the FHS, contains a lot of **Red Hat Enterprise Linux** configuration files. We will discuss some of them in greater detail. The screenshot below is the **/etc/sysconfig** directory from RHELv4u4 with everything installed.


```

paul@RHELv4u4:~$ ls /etc/sysconfig/
apmd          firstboot    irda         network      saslauthd
apm-scripts   grub         irqbalance   networking   selinux
authconfig    hidd        keyboard     ntpd         spamassassin
autofs        httpd       kudzu        openib.conf  squid
bluetooth     hwconf      lm_sensors   pand         syslog
clock         il8n        mouse        pcmcia       sys-config-sec
console       init        mouse.B      pgsql        sys-config-users
crond         installinfo named         prelink      sys-logviewer
desktop       ipmi        netdump      rawdevices   tux
diskdump      iptables    netdump_id_dsa rhn          vncservers
dund          iptables-cfg netdump_id_dsa.p samba        xinetd
paul@RHELv4u4:~$

```

The file **/etc/sysconfig/firstboot** tells the Red Hat Setup Agent not to run at boot time. If you want to run the Red Hat Setup Agent at the next reboot, then simply remove this file, and run **chkconfig --level 5 firstboot on**. The Red Hat Setup Agent allows you to install the latest updates, create a user account, join the Red Hat Network and more. It will then create the **/etc/sysconfig/firstboot** file again.

```

paul@RHELv4u4:~$ cat /etc/sysconfig/firstboot
RUN_FIRSTBOOT=NO

```

The **/etc/sysconfig/harddisks** file contains some parameters to tune the hard disks. The file explains itself.

You can see hardware detected by **kudzu** in **/etc/sysconfig/hwconf**. Kudzu is software from Red Hat for automatic discovery and configuration of hardware.

The keyboard type and keymap table are set in the **/etc/sysconfig/keyboard** file. For more console keyboard information, check the manual pages of **keymaps(5)**, **dumpkeys(1)**, **loadkeys(1)** and the directory **/lib/kbd/keymaps/**.

```

root@RHELv4u4:/etc/sysconfig# cat keyboard
KEYBOARDTYPE="pc"
KEYTABLE="us"

```

We will discuss networking files in this directory in the networking chapter.

5.6. data directories

5.6.1. /home

Users can store personal or project data under **/home**. It is common (but not mandatory by the fhs) practice to name the users home directory after the user name in the format **/home/\$USERNAME**. For example:

```
paul@ubu606:~$ ls /home
geert  annik  sandra  paul  tom
```

Besides giving every user (or every project or group) a location to store personal files, the home directory of a user also serves as a location to store the user profile. A typical Unix user profile contains many hidden files (files whose file name starts with a dot). The hidden files of the Unix user profiles contain settings specific for that user.

```
paul@ubu606:~$ ls -d /home/paul/.*
/home/paul/.          /home/paul/.bash profile  /home/paul/.ssh
/home/paul/..         /home/paul/.bashrc       /home/paul/.viminfo
/home/paul/.bash history /home/paul/.lessht
```

5.6.2. /root

On many systems **/root** is the default location for personal data and profile of the **root user**. If it does not exist by default, then some administrators create it.

5.6.3. /srv

You may use **/srv** for data that is **served by your system**. The FHS allows locating cvs, rsync, ftp and www data in this location. The FHS also approves administrative naming in /srv, like /srv/project55/ftp and /srv/sales/www.

On Sun Solaris (or Oracle Solaris) **/export** is used for this purpose.

5.6.4. /media

The **/media** directory serves as a mount point for **removable media devices** such as CD-ROM's, digital cameras, and various usb-attached devices. Since **/media** is rather new in the Unix world, you could very well encounter systems running without this directory. Solaris 9 does not have it, Solaris 10 does. Most Linux distributions today mount all removable media in **/media**.

```
paul@debian5:~$ ls /media/
cdrom  cdrom0  usbdisk
```

5.6.5. /mnt

The **/mnt** directory should be empty and should only be used for temporary mount points (according to the FHS).

Unix and Linux administrators used to create many directories here, like /mnt/something/. You likely will encounter many systems with more than one directory created and/or mounted inside **/mnt** to be used for various local and remote filesystems.

5.6.6. /tmp

Applications and users should use **/tmp** to store temporary data when needed. Data stored in **/tmp** may use either disk space or RAM. Both of which are managed by the operating system. Never use **/tmp** to store data that is important or which you wish to archive.

5.7. in memory directories

5.7.1. /dev

Device files in **/dev** appear to be ordinary files, but are not actually located on the hard disk. The **/dev** directory is populated with files as the kernel is recognising hardware.

common physical devices

Common hardware such as hard disk devices are represented by device files in **/dev**. Below a screenshot of SATA device files on a laptop and then IDE attached drives on a desktop. (The detailed meaning of these devices will be discussed later.)

```
#
# SATA or SCSI or USB
#
paul@laika:~$ ls /dev/sd*
/dev/sda /dev/sda1 /dev/sda2 /dev/sda3 /dev/sdb /dev/sdb1 /dev/sdb2

#
# IDE or ATAPI
#
paul@barry:~$ ls /dev/hd*
/dev/hda /dev/hda1 /dev/hda2 /dev/hdb /dev/hdb1 /dev/hdb2 /dev/hdc
```

Besides representing physical hardware, some device files are special. These special devices can be very useful.

/dev/tty and /dev/pts

For example, **/dev/tty1** represents a terminal or console attached to the system. (Don't break your head on the exact terminology of 'terminal' or 'console', what we mean here is a command line interface.) When typing commands in a terminal that is part of a graphical interface like Gnome or KDE, then your terminal will be represented as **/dev/pts/1** (1 can be another number).

/dev/null

On Linux you will find other special devices such as **/dev/null** which can be considered a black hole; it has unlimited storage, but nothing can be retrieved from it. Technically speaking, anything written to **/dev/null** will be discarded. **/dev/null** can be useful to discard unwanted output from commands. */dev/null is not a good location to store your backups ;-).*

5.7.2. /proc conversation with the kernel

/proc is another special directory, appearing to be ordinary files, but not taking up disk space. It is actually a view of the kernel, or better, what the kernel manages, and is a means to interact with it directly. **/proc** is a proc filesystem.

```
paul@RHELv4u4:~$ mount -t proc
```

```
none on /proc type proc (rw)
```

When listing the /proc directory you will see many numbers (on any Unix) and some interesting files (on Linux)

```
mul@laika:~$ ls /proc
1          2339    4724    5418    6587    7201      cmdline    mounts
10175     2523    4729    5421    6596    7204      cpuinfo     mtrr
10211     2783    4741    5658    6599    7206      crypto      net
10239     2975    4873    5661    6638    7214      devices     pagetypeinfo
141       29775   4874    5665    6652    7216      diskstats   partitions
15045     29792   4878    5927    6719    7218      dma         sched_debug
1519      2997    4879    6       6736    7223      driver      scsi
1548      3       4881    6032    6737    7224      execdomains self
1551      30228   4882    6033    6755    7227      fb          slabinfo
1554      3069    5       6145    6762    7260      filesystems stat
1557      31422   5073    6298    6774    7267      fs          swaps
1606      3149    5147    6414    6816    7275      ide         sys
180       31507   5203    6418    6991    7282      interrupts  sysrq-trigger
181       3189    5206    6419    6993    7298      iomem       sysvipc
182       3193    5228    6420    6996    7319      ioports     timer_list
18898     3246    5272    6421    7157    7330      irq         timer_stats
19799     3248    5291    6422    7163    7345      kallsyms    tty
19803     3253    5294    6423    7164    7513      kcore       uptime
19804     3372    5356    6424    7171    7525      key-users   version
1987      4       5370    6425    7175    7529      kmsg        version_signature
1989      42     5379    6426    7188    9964      loadavg     vmcore
2         45     5380    6430    7189      locks       vmnet
20845     4542    5412    6450    7191    asound      meminfo     vmstat
221      46     5414    6551    7192    buddyinfo   misc        zoneinfo
2338     4704    5416    6568    7199    bus         modules
```

Let's investigate the file properties inside **/proc**. Looking at the date and time will display the current date and time showing the files are constantly updated (a view on the kernel).

```
paul@RHELv4u4:~$ date
Mon Jan 29 18:06:32 EST 2007
paul@RHELv4u4:~$ ls -al /proc/cpuinfo
-r--r--r-- 1 root root 0 Jan 29 18:06 /proc/cpuinfo
paul@RHELv4u4:~$
paul@RHELv4u4:~$ ...time passes...
paul@RHELv4u4:~$
paul@RHELv4u4:~$ date
Mon Jan 29 18:10:00 EST 2007
paul@RHELv4u4:~$ ls -al /proc/cpuinfo
-r--r--r-- 1 root root 0 Jan 29 18:10 /proc/cpuinfo
```

Most files in /proc are 0 bytes, yet they contain data--sometimes a lot of data. You can see this by executing cat on files like **/proc/cpuinfo**, which contains information about the CPU.

```
paul@RHELv4u4:~$ file /proc/cpuinfo
/proc/cpuinfo: empty
paul@RHELv4u4:~$ cat /proc/cpuinfo
processor       : 0
vendor_id      : AuthenticAMD
cpu family     : 15
model          : 43
```

```
model name      : AMD Athlon(tm) 64 X2 Dual Core Processor 4600+
stepping       : 1
cpu MHz        : 2398.628
cache size     : 512 KB
fdiv_bug      : no
hlt_bug       : no
f00f_bug      : no
coma_bug      : no
fpu           : yes
fpu_exception  : yes
cpuid level    : 1
wp            : yes
flags         : fpu vme de pse tsc msr pae mce cx8 apic mtrr pge...
bogomips      : 4803.54
```

Just for fun, here is /proc/cpuinfo on a Sun Sunblade 1000...

```
paul@pasha:~$ cat /proc/cpuinfo
cpu : TI UltraSparc III (Cheetah)
fpu : UltraSparc III integrated FPU
promlib : Version 3 Revision 2
prom : 4.2.2
type : sun4u
ncpus probed : 2
ncpus active : 2
Cpu0Bogo : 498.68
Cpu0ClkTck : 000000002cb41780
Cpu1Bogo : 498.68
Cpu1ClkTck : 000000002cb41780
MMU Type : Cheetah
State:
CPU0: online
CPU1: online
```

Most of the files in /proc are read only, some require root privileges, some files are writable, and many files in **/proc/sys** are writable. Let's discuss some of the files in /proc.

/proc/interrupts

On the x86 architecture, **/proc/interrupts** displays the interrupts.

```
paul@RHELv4u4:~$ cat /proc/interrupts
CPU0
 0:   13876877    IO-APIC-edge  timer
 1:         15    IO-APIC-edge  i8042
 8:          1    IO-APIC-edge  rtc
 9:          0    IO-APIC-level  acpi
12:          67    IO-APIC-edge  i8042
14:         128    IO-APIC-edge  ide0
15:       124320    IO-APIC-edge  ide1
169:       111993    IO-APIC-level  ioc0
177:        2428    IO-APIC-level  eth0
NMI:          0
LOC:       13878037
ERR:          0
MIS:          0
```

On a machine with two CPU's, the file looks like this.

```
paul@laika:~$ cat /proc/interrupts
CPU0      CPU1
 0:   860013      0    IO-APIC-edge  timer
 1:    4533      0    IO-APIC-edge  i8042
 7:         0      0    IO-APIC-edge  parport0
 8:   6588227     0    IO-APIC-edge  rtc
10:    2314      0    IO-APIC-fasteoi  acpi
12:    133       0    IO-APIC-edge  i8042
14:         0      0    IO-APIC-edge  libata
15:    72269     0    IO-APIC-edge  libata
18:         1      0    IO-APIC-fasteoi  yenta
19:   115036     0    IO-APIC-fasteoi  eth0
20:   126871     0    IO-APIC-fasteoi  libata, ohci1394
21:    30204     0    IO-APIC-fasteoi  ehci_hcd:usb1, uhci_hcd:usb2
22:    1334      0    IO-APIC-fasteoi  saa7133[0], saa7133[0]
24:   234739     0    IO-APIC-fasteoi  nvidia
NMI:         72      42
LOC:   860000    859994
ERR:         0
```

/proc/kcore

The physical memory is represented in **/proc/kcore**. Do not try to cat this file, instead use a debugger. The size of **/proc/kcore** is the same as your physical memory, plus four bytes.

```
paul@laika:~$ ls -lh /proc/kcore
-r----- 1 root root 2.0G 2007-01-30 08:57 /proc/kcore
paul@laika:~$
```

5.7.3. /sys Linux 2.6 hot plugging

The **/sys** directory was created for the Linux 2.6 kernel. Since 2.6, Linux uses **sysfs** to support **usb** and **IEEE 1394 (FireWire)** hot plug devices. See the manual pages of **udev(8)** (the successor of **devfs**) and **hotplug(8)** for more info (or visit <http://linux-hotplug.sourceforge.net/>).

Basically the **/sys** directory contains kernel information about hardware.

5.8. /usr Unix System Resources

Although **/usr** is pronounced like user, remember that it stands for **Unix System Resources**. The **/usr** hierarchy should contain **shareable, read only** data. Some people choose to mount **/usr** as read only. This can be done from its own partition or from a read only NFS share (NFS is discussed later).

5.8.1. /usr/bin

The **/usr/bin** directory contains a lot of commands.

```
paul@deb508:~$ ls /usr/bin | wc -l
1395
```

(On Solaris the **/bin** directory is a symbolic link to **/usr/bin**.)

5.8.2. /usr/include

The **/usr/include** directory contains general use include files for C.

```
paul@ubu1010:~$ ls /usr/include/
aolib.h      expat_config.h  math.h       search.h
af_vfs.h     expat_external.h mcheck.h     semaphore.h
aio.h        expat.h         memory.h     setjmp.h
AL           fcntl.h        menu.h       sgTTY.h
aliases.h    features.h      mntent.h     shadow.h
...
```

5.8.3. /usr/lib

The **/usr/lib** directory contains libraries that are not directly executed by users or scripts.

```
paul@deb508:~$ ls /usr/lib | head -7
4Suite
ao
apt
arj
aspell
avahi
bonobo
```

5.8.4. /usr/local

The **/usr/local** directory can be used by an administrator to install software locally.

```
paul@deb508:~$ ls /usr/local/
bin  etc  games  include  lib  man  sbin  share  src
paul@deb508:~$ du -sh /usr/local/
128K /usr/local/
```

5.8.5. /usr/share

The **/usr/share** directory contains architecture independent data. As you can see, this is a fairly large directory.

```
paul@deb508:~$ ls /usr/share/ | wc -l
```

```
263
paul@deb508:~$ du -sh /usr/share/
1.3G /usr/share/
```

This directory typically contains **/usr/share/man** for manual pages.

```
paul@deb508:~$ ls /usr/share/man
cs  fr    hu  it.UTF-8  man2 man6  pl.ISO8859-2  sv
de  fr.ISO8859-1  id  ja    man3 man7  pl.UTF-8      tr
es  fr.UTF-8     it  ko    man4 man8  pt_BR        zh_CN
fi  gl    it.ISO8859-1  man1  man5 pl     ru     zh  TW
```

And it contains **/usr/share/games** for all static game data (so no high-scores or play logs).

```
paul@ubu1010:~$ ls /usr/share/games/
openttd  wesnoth
```

5.8.6. /usr/src

The **/usr/src** directory is the recommended location for kernel source files.

```
paul@deb508:~$ ls -l /usr/src/
total 12
drwxr-xr-x  4 root root 4096 2011-02-01 14:43 linux-headers-2.6.26-2-686
drwxr-xr-x 18 root root 4096 2011-02-01 14:43 linux-headers-2.6.26-2-common
drwxr-xr-x  3 root root 4096 2009-10-28 16:01 linux-kbuild-2.6.26
```

5.9. /var variable data

Files that are unpredictable in size, such as log, cache and spool files, should be located in **/var**.

5.9.1. /var/log

The **/var/log** directory serves as a central point to contain all log files.

```
[paul@RHEL4b ~]$ ls /var/log
acpid          cron.2      maillog.2   quagga       secure.4
amanda         cron.3      maillog.3   radius        spooler
anaconda.log   cron.4      maillog.4   rpmpkgs       spooler.1
anaconda.syslog cups        mailman     rpmpkgs.1     spooler.2
anaconda.xlog  dmesg       messages    rpmpkgs.2     spooler.3
audit          exim        messages.1  rpmpkgs.3     spooler.4
boot.log       gdm         messages.2  rpmpkgs.4     squid
boot.log.1     httpd       messages.3  sa            uucp
boot.log.2     iiim        messages.4  samba         vbox
boot.log.3     iptraf      mysqld.log  scrollkeeper.log vmware-tools-guestd
boot.log.4     lastlog     news        secure        wtmp
canna          mail        pgsql       secure.1       wtmp.1
cron           maillog     ppp         secure.2       Xorg.0.log
cron.1         maillog.1  prelink.log secure.3       Xorg.0.log.old
```

5.9.2. /var/log/messages

A typical first file to check when troubleshooting on Red Hat (and derivatives) is the **/var/log/messages** file. By default this file will contain information on what just happened to the system. The file is called **/var/log/syslog** on Debian and Ubuntu.

```
[root@RHEL4b ~]# tail /var/log/messages
Jul 30 05:13:56 anacron: anacron startup succeeded
Jul 30 05:13:56 atd: atd startup succeeded
Jul 30 05:13:57 messagebus: messagebus startup succeeded
Jul 30 05:13:57 cups-config-daemon: cups-config-daemon startup succeeded
Jul 30 05:13:58 haldaemon: haldaemon startup succeeded
Jul 30 05:14:00 fstab-sync[3560]: removed all generated mount points
Jul 30 05:14:01 fstab-sync[3628]: added mount point /media/cdrom for...
Jul 30 05:14:01 fstab-sync[3646]: added mount point /media/floppy for...
Jul 30 05:16:46 sshd(pam_unix)[3662]: session opened for user paul by...
Jul 30 06:06:37 su(pam_unix)[3904]: session opened for user root by paul
```

5.9.3. /var/cache

The **/var/cache** directory can contain **cache data** for several applications.

```
paul@ubul010:~$ ls /var/cache/
apt          dictionaries-common  gdm      man          software-center
binfmts      flashplugin-installer hald     pm-utils
cups         fontconfig           jockey   pppconfig
debconf      fonts                ldconfig samba
```

5.9.4. /var/spool

The **/var/spool** directory typically contains spool directories for **mail** and **cron**, but also serves as a parent directory for other spool files (for example print spool files).

5.9.5. **/var/lib**

The **/var/lib** directory contains application state information.

Red Hat Enterprise Linux for example keeps files pertaining to **rpm** in **/var/lib/rpm/**.

11.9.6. **/var/...**

/var also contains Process ID files in **/var/run** (soon to be replaced with **/run**) and temporary files that survive a reboot in **/var/tmp** and information about file locks in **/var/lock**. There

5.10. solution: file system tree

1. Does the file **/bin/cat** exist ? What about **/bin/dd** and **/bin/echo**. What is the type of these files ?

```
ls /bin/cat ; file /bin/cat
```

```
ls /bin/dd ; file /bin/dd
```

```
ls /bin/echo ; file /bin/echo
```

2. What is the size of the Linux kernel file(s) (vmlinu*) in **/boot** ?

```
ls -lh /boot/vm*
```

3. Create a directory **~/test**. Then issue the following commands:

```
cd ~/test
```

```
dd if=/dev/zero of=zeros.txt count=1 bs=100
```

```
od zeroes.txt
```

dd will copy one times (count=1) a block of size 100 bytes (bs=100) from the file **/dev/zero** to **~/test/zeros.txt**. Can you describe the functionality of **/dev/zero** ?

/dev/zero is a Linux special device. It can be considered a source of zeroes. You cannot send something to **/dev/zero**, but you can read zeroes from it.

4. Now issue the following command:

```
dd if=/dev/random of=random.txt count=1 bs=100 ; od random.txt
```

dd will copy one times (count=1) a block of size 100 bytes (bs=100) from the file **/dev/random** to **~/test/random.txt**. Can you describe the functionality of **/dev/random** ?

/dev/random acts as a **random number generator** on your Linux machine.

5. Issue the following two commands, and look at the first character of each output line.

```
ls -l /dev/sd* /dev/hd*
```

```
ls -l /dev/tty* /dev/input/mou*
```

The first **ls** will show block(b) devices, the second **ls** shows character(c) devices. Can you tell the difference between block and character devices ?

Block devices are always written to (or read from) in blocks. For hard disks, blocks of 512 bytes are common. Character devices act as a stream of characters (or bytes). Mouse and keyboard are typical character devices.

6. Use **cat** to display **/etc/hosts** and **/etc/resolv.conf**. What is your idea about the purpose of these files ?

```
/etc/hosts contains hostnames with their ip address
```

```
/etc/resolv.conf should contain the ip address of a DNS name server.
```


7. Are there any files in **/etc/skel/** ? Check also for hidden files.

```
Issue "ls -al /etc/skel/". Yes, there should be hidden files there.
```

8. Display **/proc/cpuinfo**. On what architecture is your Linux running ?

```
The file should contain at least one line with Intel or other cpu.
```

9. Display **/proc/interrupts**. What is the size of this file ? Where is this file stored ?

The size is zero, yet the file contains data. It is not stored anywhere because **/proc** is a virtual file system that allows you to talk with the kernel. (If you answered "stored in RAM-memory, that is also correct...).

10. Can you enter the **/root** directory ? Are there (hidden) files ?

```
Try "cd /root". The /root directory is not accessible for normal users on most modern Linux sy
```

11. Are **ifconfig**, **fdisk**, **parted**, **shutdown** and **grub-install** present in **/sbin** ? Why are these binaries in **/sbin** and not in **/bin** ?

```
Because those files are only meant for system administrators.
```

12. Is **/var/log** a file or a directory ? What about **/var/spool** ?

```
Both are directories.
```

13. Open two command prompts (Ctrl-Shift-T in gnome-terminal) or terminals (Ctrl-Alt-F1, Ctrl-Alt-F2, ...) and issue the **who am i** in both. Then try to echo a word from one terminal to the other.

```
tty-terminal: echo Hello > /dev/tty1
```

```
pts-terminal: echo Hello > /dev/pts/1
```

14. Read the man page of **random** and explain the difference between **/dev/random** and **/dev/urandom**.

```
man 4 random
```

Chapter 6. commands and arguments

This chapter introduces you to **shell expansion** by taking a close look at **commands** and **arguments**. Knowing **shell expansion** is important because many **commands** on your Linux system are processed and most likely changed by the **shell** before they are executed.

The command line interface or **shell** used on most Linux systems is called **bash**, which stands for **Bourne again shell**. The **bash** shell incorporates features from **sh** (the original Bourne shell), **cs**h (the C shell), and **ksh** (the Korn shell).

This chapter frequently uses the **echo** command to demonstrate shell features. The **echo** command is very simple: it echoes the input that it receives.

```
paul@laika:~$ echo Burtonville
Burtonville
paul@laika:~$ echo Smurfs are blue
Smurfs are blue
```


6.1. arguments

One of the primary features of a shell is to perform a **command line scan**. When you enter a command at the shell's command prompt and press the enter key, then the shell will start scanning that line, cutting it up in **arguments**. While scanning the line, the shell may make many changes to the **arguments** you typed.

This process is called **shell expansion**. When the shell has finished scanning and modifying that line, then it will be executed.

6.2. white space removal

Parts that are separated by one or more consecutive **white spaces** (or tabs) are considered separate **arguments**, any white space is removed. The first **argument** is the command to be executed, the other **arguments** are given to the command. The shell effectively cuts your command into one or more arguments.

This explains why the following four different command lines are the same after **shell expansion**.

```
[paul@RHELv4u3 ~]$ echo Hello World
Hello World
[paul@RHELv4u3 ~]$ echo Hello  World
Hello World
[paul@RHELv4u3 ~]$ echo  Hello  World
Hello World
[paul@RHELv4u3 ~]$ echo      Hello      World
Hello World
```

The **echo** command will display each argument it receives from the shell. The **echo** command will also add a new white space between the arguments it received.

6.3. single quotes

You can prevent the removal of white spaces by quoting the spaces. The contents of the quoted string are considered as one argument. In the screenshot below the **echo** receives only one **argument**.

```
[paul@RHEL4b ~]$ echo 'A line with      single      quotes'
A line with      single      quotes
[paul@RHEL4b ~]$
```

6.4. double quotes

You can also prevent the removal of white spaces by double quoting the spaces. Same as above, **echo** only receives one **argument**.

```
[paul@RHEL4b ~]$ echo "A line with      double      quotes"
A line with      double      quotes
[paul@RHEL4b ~]$
```

Later in this book, when discussing **variables** we will see important differences between single and double quotes.

6.5. echo and quotes

Quoted lines can include special escaped characters recognised by the **echo** command (when using **echo -e**). The screenshot below shows how to use **\n** for a newline and **\t** for a tab (usually eight white spaces).

```
[paul@RHEL4b ~]$ echo -e "A line with \na newline"
A line with
a newline
[paul@RHEL4b ~]$ echo -e 'A line with \na newline'
A line with
a newline
[paul@RHEL4b ~]$ echo -e "A line with \ta tab"
A line with      a tab
[paul@RHEL4b ~]$ echo -e 'A line with \ta tab'
A line with      a tab
[paul@RHEL4b ~]$
```

The **echo** command can generate more than white spaces, tabs and newlines. Look in the man page for a list of options.

6.6. commands

6.6.1. external or built-in commands?

Not all commands are external to the shell, some are **builtin**. **External commands** are programs that have their own binary and reside somewhere in the file system. Many external commands are located in **/bin** or **/sbin**. **Builtin commands** are an integral part of the shell program itself.

6.6.2. type

To find out whether a command given to the shell will be executed as an **external command** or as a **builtin command**, use the **type** command.

```
paul@laika:~$ type cd
cd is a shell builtin
paul@laika:~$ type cat
cat is /bin/cat
```

As you can see, the **cd** command is **builtin** and the **cat** command is **external**.

You can also use this command to show you whether the command is **aliased** or not.

```
paul@laika:~$ type ls
ls is aliased to `ls --color=auto'
```

6.6.3. running external commands

Some commands have both builtin and external versions. When one of these commands is executed, the builtin version takes priority. To run the external version, you must enter the full path to the command.

```
paul@laika:~$ type -a echo
echo is a shell builtin
echo is /bin/echo
paul@laika:~$ /bin/echo Running the external echo command...
Running the external echo command...
```

6.6.4. which

The **which** command will search for binaries in the **\$PATH** environment variable (variables will be explained later). In the screenshot below, it is determined that **cd** is **builtin**, and **ls**, **cp**, **rm**, **mv**, **mkdir**, **pwd**, and **which** are external commands.

```
[root@RHEL4b ~]# which cp ls cd mkdir pwd
/bin/cp
/bin/ls
/usr/bin/which: no cd in (/usr/kerberos/sbin:/usr/kerberos/bin:...
/bin/mkdir
/bin/pwd
```

6.7. aliases

6.7.1. create an alias

The shell allows you to create **aliases**. Aliases are often used to create an easier to remember name for an existing command or to easily supply parameters.

```
[paul@RHELv4u3 ~]$ cat count.txt
one
two
three
[paul@RHELv4u3 ~]$ alias dog=tac
[paul@RHELv4u3 ~]$ dog count.txt
three
two
one
```

6.7.2. abbreviate commands

An **alias** can also be useful to abbreviate an existing command.

```
paul@laika:~$ alias ll='ls -lh --color=auto'
paul@laika:~$ alias c='clear'
paul@laika:~$
```

6.7.3. default options

Aliases can be used to supply commands with default options. The example below shows how to set the **-i** option default when typing **rm**.

```
[paul@RHELv4u3 ~]$ rm -i winter.txt
rm: remove regular file `winter.txt'? no
[paul@RHELv4u3 ~]$ rm winter.txt
[paul@RHELv4u3 ~]$ ls winter.txt
ls: winter.txt: No such file or directory
[paul@RHELv4u3 ~]$ touch winter.txt
[paul@RHELv4u3 ~]$ alias rm='rm -i'
[paul@RHELv4u3 ~]$ rm winter.txt
rm: remove regular empty file `winter.txt'? no
[paul@RHELv4u3 ~]$
```

Some distributions enable default aliases to protect users from accidentally erasing files ('rm -i', 'mv -i', 'cp -i')

6.7.4. viewing aliases

You can provide one or more aliases as arguments to the **alias** command to get their definitions. Providing no arguments gives a complete list of current aliases.

```
paul@laika:~$ alias c ll
alias c='clear'
alias ll='ls -lh --color=auto'
```

6.7.5. unalias

You can undo an alias with the **unalias** command.

```
[paul@RHEL4b ~]$ which rm
/bin/rm
[paul@RHEL4b ~]$ alias rm='rm -i'
[paul@RHEL4b ~]$ which rm
alias rm='rm -i'
        /bin/rm
[paul@RHEL4b ~]$ unalias rm
[paul@RHEL4b ~]$ which rm
/bin/rm
[paul@RHEL4b ~]$
```

6.8. displaying shell expansion

You can display shell expansion with **set -x**, and stop displaying it with **set +x**. You might want to use this further on in this course, or when in doubt about exactly what the shell is doing with your command.

```
[paul@RHELv4u3 ~]$ set -x
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ echo $USER
+ echo paul
paul
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ echo \ $USER
+ echo '$USER'
$USER
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ set +x
+ set +x
[paul@RHELv4u3 ~]$ echo $USER
paul
```

6.9. solution: commands and arguments

1. How many **arguments** are in this line (not counting the command itself).

```
touch '/etc/cron/cron.allow' 'file 42.txt' "file 33.txt"
```

```
answer: three
```

2. Is **tac** a shell builtin command ?

```
type tac
```

3. Is there an existing alias for **rm** ?

```
alias rm
```

4. Read the man page of **rm**, make sure you understand the **-i** option of **rm**. Create and remove a file to test the **-i** option.

```
man rm
```

```
touch testfile
```

```
rm -i testfile
```

5. Execute: **alias rm='rm -i'** . Test your alias with a test file. Does this work as expected ?

```
touch testfile
```

```
rm testfile (should ask for confirmation)
```

6. List all current aliases.

```
alias
```

7a. Create an alias called 'city' that echoes your hometown.

```
alias city='echo Antwerp'
```

7b. Use your alias to test that it works.

```
city (it should display Antwerp)
```

8. Execute **set -x** to display shell expansion for every command.

```
set -x
```

9. Test the functionality of **set -x** by executing your **city** and **rm** aliases.

```
shell should display the resolved aliases and then execute the command:
paul@deb503:~$ set -x
paul@deb503:~$ city
+ echo antwerp
antwerp
```

10 Execute **set +x** to stop displaying shell expansion.

```
set +x
```

11. Remove your city alias.

```
unalias city
```

12. What is the location of the **cat** and the **passwd** commands ?

```
which cat (probably /bin/cat)
```

```
which passwd (probably /usr/bin/passwd)
```

13. Explain the difference between the following commands:

```
echo
```

```
/bin/echo
```

The **echo** command will be interpreted by the shell as the **built-in echo** command. The **/bin/echo** command will make the shell execute the **echo binary** located in the **/bin** directory.

14. Explain the difference between the following commands:

```
echo Hello
```

```
echo -n Hello
```

The **-n** option of the **echo** command will prevent echo from echoing a trailing newline. **echo Hello** will echo six characters in total, **echo -n hello** only echoes five characters.

(The **-n** option might not work in the Korn shell.)

15. Display **A B C** with two spaces between B and C.

```
echo "A B C"
```

16. Complete the following command (do not use spaces) to display exactly the following output:

```
4+4      =8
10+14    =24
```

The solution is to use tabs with **\t**.

```
echo -e "4+4\t=8" ; echo -e "10+14\t=24"
```

17. Use **echo** to display the following exactly:

```
??\
echo '??\
echo -e '??\
echo "??\
echo -e "??\
echo ??\
```

Find two solutions with single quotes, two with double quotes and one without quotes (and say thank you to René and Darioush from Google for this extra).

18. Use one **echo** command to display three words on three lines.

```
echo -e "one \ntwo \nthree"
```

Chapter 7. control operators

In this chapter we put more than one command on the command line using **control operators**. We also briefly discuss related parameters (\$?) and similar special characters(&).

7.1. ; semicolon

You can put two or more commands on the same line separated by a semicolon `;`. The shell will scan the line until it reaches the semicolon. All the arguments before this semicolon will be considered a separate command from all the arguments after the semicolon. Both series will be executed sequentially with the shell waiting for each command to finish before starting the next one.

```
[paul@RHELv4u3 ~]$ echo Hello
Hello
[paul@RHELv4u3 ~]$ echo World
World
[paul@RHELv4u3 ~]$ echo Hello ; echo World
Hello
World
[paul@RHELv4u3 ~]$
```

7.2. & ampersand

When a line ends with an ampersand `&`, the shell will not wait for the command to finish. You will get your shell prompt back, and the command is executed in background. You will get a message when this command has finished executing in background.

```
[paul@RHELv4u3 ~]$ sleep 20 &
[1] 7925
[paul@RHELv4u3 ~]$
...wait 20 seconds...
[paul@RHELv4u3 ~]$
[1]+  Done                  sleep 20
```

The technical explanation of what happens in this case is explained in the chapter about **processes**.

7.3. \$? dollar question mark

The exit code of the previous command is stored in the shell variable `$?`. Actually `$?` is a shell parameter and not a variable, since you cannot assign a value to `$?`.

```
paul@debian5:~/test$ touch file1
paul@debian5:~/test$ echo $?
0
paul@debian5:~/test$ rm file1
paul@debian5:~/test$ echo $?
0
paul@debian5:~/test$ rm file1
rm: cannot remove `file1': No such file or directory
paul@debian5:~/test$ echo $?
1
paul@debian5:~/test$
```

7.4. && double ampersand

The shell will interpret **&&** as a **logical AND**. When using **&&** the second command is executed only if the first one succeeds (returns a zero exit status).

```
paul@barry:~$ echo first && echo second
first
second
paul@barry:~$ zecho first && echo second
-bash: zecho: command not found
```

Another example of the same **logical AND** principle. This example starts with a working **cd** followed by **ls**, then a non-working **cd** which is **not** followed by **ls**.

```
[paul@RHELv4u3 ~]$ cd gen && ls
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
[paul@RHELv4u3 gen]$ cd gen && ls
-bash: cd: gen: No such file or directory
```

7.5. || double vertical bar

The **||** represents a **logical OR**. The second command is executed only when the first command fails (returns a non-zero exit status).

```
paul@barry:~$ echo first || echo second ; echo third
first
third
paul@barry:~$ zecho first || echo second ; echo third
-bash: zecho: command not found
second
third
paul@barry:~$
```

Another example of the same **logical OR** principle.

```
[paul@RHELv4u3 ~]$ cd gen || ls
[paul@RHELv4u3 gen]$ cd gen || ls
-bash: cd: gen: No such file or directory
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
```

7.6. combining && and ||

You can use this logical AND and logical OR to write an **if-then-else** structure on the command line. This example uses **echo** to display whether the **rm** command was successful.

```
paul@laika:~/test$ rm file1 && echo It worked! || echo It failed!
It worked!
paul@laika:~/test$ rm file1 && echo It worked! || echo It failed!
rm: cannot remove `file1': No such file or directory
It failed!
paul@laika:~/test$
```

7.7. # pound sign

Everything written after a **pound sign** (#) is ignored by the shell. This is useful to write a **shell comment**, but has no influence on the command execution or shell expansion.

```
paul@debian4:~$ mkdir test      # we create a directory
paul@debian4:~$ cd test        ##### we enter the directory
paul@debian4:~/test$ ls        # is it empty ?
paul@debian4:~/test$
```

7.8. \ escaping special characters

The backslash \ character enables the use of control characters, but without the shell interpreting it, this is called **escaping** characters.

```
[paul@RHELv4u3 ~]$ echo hello \; world
hello ; world
[paul@RHELv4u3 ~]$ echo hello\ \ \ world
hello  world
[paul@RHELv4u3 ~]$ echo escaping \\ \# \& \\" \'
escaping \ # & " '
[paul@RHELv4u3 ~]$ echo escaping \\? \* \\" \'
escaping \?*\"'
```

7.8.1. end of line backslash

Lines ending in a backslash are continued on the next line. The shell does not interpret the newline character and will wait on shell expansion and execution of the command line until a newline without backslash is encountered.

```
[paul@RHEL4b ~]$ echo This command line \
> is split in three \
> parts
This command line is split in three parts
[paul@RHEL4b ~]$
```

7.9. practice: control operators

0. Each question can be answered by one command line!
1. When you type **passwd**, which file is executed ?
2. What kind of file is that ?
3. Execute the **pwd** command twice. (remember 0.)
4. Execute **ls** after **cd /etc**, but only if **cd /etc** did not error.
5. Execute **cd /etc** after **cd etc**, but only if **cd etc** fails.
6. Echo **it worked** when **touch test42** works, and echo **it failed** when the **touch** failed. All on one command line as a normal user (not root). Test this line in your home directory and in **/bin/**.
7. Execute **sleep 6**, what is this command doing ?
8. Execute **sleep 200** in background (do not wait for it to finish).
9. Write a command line that executes **rm file55**. Your command line should print 'success' if file55 is removed, and print 'failed' if there was a problem.
- (optional)10. Use echo to display "Hello World with strange' characters \ * [} ~ \ \ ." (including all quotes)

7.10. solution: control operators

0. Each question can be answered by one command line!

1. When you type **passwd**, which file is executed ?

```
which passwd
```

2. What kind of file is that ?

```
file /usr/bin/passwd
```

3. Execute the **pwd** command twice. (remember 0.)

```
pwd ; pwd
```

4. Execute **ls** after **cd /etc**, but only if **cd /etc** did not error.

```
cd /etc && ls
```

5. Execute **cd /etc** after **cd etc**, but only if **cd etc** fails.

```
cd etc || cd /etc
```

6. Echo **it worked** when **touch test42** works, and echo **it failed** when the **touch** failed. All on one command line as a normal user (not root). Test this line in your home directory and in **/bin/**.

```
paul@deb503:~$ cd ; touch test42 && echo it worked || echo it failed
it worked
paul@deb503:~$ cd /bin; touch test42 && echo it worked || echo it failed
touch: cannot touch `test42': Permission denied
it failed
```

7. Execute **sleep 6**, what is this command doing ?

```
pausing for six seconds
```

8. Execute **sleep 200** in background (do not wait for it to finish).

```
sleep 200 &
```

9. Write a command line that executes **rm file55**. Your command line should print 'success' if file55 is removed, and print 'failed' if there was a problem.

```
rm file55 && echo success || echo failed
```

(optional)10. Use echo to display "Hello World with strange' characters \ * [] ~ \ \ ." (including all quotes)

```
echo \"Hello World with strange\" characters \\ \\* \\[ \\] \\~ \\|\\| \\|\\. \\|\"
```

```
or
```

```
echo \"\"\"Hello World with strange' characters \\ * [ ] ~ \\|\\. \"\"\"
```

Chapter 8. shell variables

In this chapter we learn to manage environment **variables** in the shell. These **variables** are often needed by applications.

8.1. \$ dollar sign

Another important character interpreted by the shell is the dollar sign **\$**. The shell will look for an **environment variable** named like the string following the **dollar sign** and replace it with the value of the variable (or with nothing if the variable does not exist).

These are some examples using \$HOSTNAME, \$USER, \$UID, \$SHELL, and \$HOME.

```
[paul@RHELv4u3 ~]$ echo This is the $SHELL shell
This is the /bin/bash shell
[paul@RHELv4u3 ~]$ echo This is $SHELL on computer $HOSTNAME
This is /bin/bash on computer RHELv4u3.localdomain
[paul@RHELv4u3 ~]$ echo The userid of $USER is $UID
The userid of paul is 500
[paul@RHELv4u3 ~]$ echo My homedir is $HOME
My homedir is /home/paul
```

8.2. case sensitive

This example shows that shell variables are case sensitive!

```
[paul@RHELv4u3 ~]$ echo Hello $USER
Hello paul
[paul@RHELv4u3 ~]$ echo Hello $user
Hello
```

8.3. creating variables

This example creates the variable **\$MyVar** and sets its value. It then uses **echo** to verify the value.

```
[paul@RHELv4u3 gen]$ MyVar=555
[paul@RHELv4u3 gen]$ echo $MyVar
555
[paul@RHELv4u3 gen]$
```

8.4. quotes

Notice that double quotes still allow the parsing of variables, whereas single quotes prevent this.

```
[paul@RHELv4u3 ~]$ MyVar=555
[paul@RHELv4u3 ~]$ echo $MyVar
555
[paul@RHELv4u3 ~]$ echo "$MyVar"
555
[paul@RHELv4u3 ~]$ echo '$MyVar'
$MyVar
```

The bash shell will replace variables with their value in double quoted lines, but not in single quoted lines.

```
paul@laika:~$ city=Burtonville
paul@laika:~$ echo "We are in $city today."
We are in Burtonville today.
paul@laika:~$ echo 'We are in $city today.'
We are in $city today.
```

8.5. set

You can use the **set** command to display a list of environment variables. On Ubuntu and Debian systems, the **set** command will also list shell functions after the shell variables. Use **set | more** to see the variables then.

8.6. unset

Use the **unset** command to remove a variable from your shell environment.

```
[paul@RHEL4b ~]$ MyVar=8472
[paul@RHEL4b ~]$ echo $MyVar
8472
[paul@RHEL4b ~]$ unset MyVar
[paul@RHEL4b ~]$ echo $MyVar

[paul@RHEL4b ~]$
```


8.7. \$PS1

The **\$PS1** variable determines your shell prompt. You can use backslash escaped special characters like **\u** for the username or **\w** for the working directory. The **bash** manual has a complete reference.

In this example we change the value of **\$PS1** a couple of times.

```
paul@deb503:~$ PS1=prompt
prompt
promptPS1='prompt '
prompt
prompt PS1='> '
>
> PS1='\u@\h$ '
paul@deb503$
paul@deb503$ PS1='\u@\h:\w$ '
paul@deb503:~$
```

To avoid unrecoverable mistakes, you can set normal user prompts to green and the root prompt to red. Add the following to your **.bashrc** for a green user prompt:

```
# color prompt by paul
RED='\[\033[01;31m\]'
WHITE='\[\033[01;00m\]'
GREEN='\[\033[01;32m\]'
BLUE='\[\033[01;34m\]'
export PS1="${debian_chroot:+($debian_chroot)}$GREEN\u$WHITE@$BLUE\h$WHITE\w\$ "
```

8.8. \$PATH

The **\$PATH** variable determines where the shell is looking for commands to execute (unless the command is builtin or aliased). This variable contains a list of directories, separated by colons.

```
[[paul@RHEL4b ~]$ echo $PATH
/usr/kerberos/bin:/usr/local/bin:/bin:/usr/bin:
```

The shell will not look in the current directory for commands to execute! (Looking for executables in the current directory provided an easy way to hack PC-DOS computers). If you want the shell to look in the current directory, then add a **.** at the end of your **\$PATH**.

```
[paul@RHEL4b ~]$ PATH=$PATH:.
[paul@RHEL4b ~]$ echo $PATH
/usr/kerberos/bin:/usr/local/bin:/bin:/usr/bin:.
[paul@RHEL4b ~]$
```

Your path might be different when using **su** instead of **su -** because the latter will take on the environment of the target user. The root user typically has **/sbin** directories added to the **\$PATH** variable.

```
[paul@RHEL3 ~]$ su
Password:
[root@RHEL3 paul]# echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin
[root@RHEL3 paul]# exit
[paul@RHEL3 ~]$ su -
Password:
[root@RHEL3 ~]# echo $PATH
/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin:
[root@RHEL3 ~]#
```

8.9. env

The **env** command without options will display a list of **exported variables**. The difference with **set** with options is that **set** lists all variables, including those not exported to child shells.

But **env** can also be used to start a clean shell (a shell without any inherited environment). The **env -i** command clears the environment for the subshell.

Notice in this screenshot that **bash** will set the **\$SHELL** variable on startup.

```
[paul@RHEL4b ~]$ bash -c 'echo $SHELL $HOME $USER'
/bin/bash /home/paul paul
[paul@RHEL4b ~]$ env -i bash -c 'echo $SHELL $HOME $USER'
/bin/bash
[paul@RHEL4b ~]$
```

You can use the **env** command to set the **\$LANG**, or any other, variable for just one instance of **bash** with one command. The example below uses this to show the influence of the **\$LANG** variable on file globbing (see the chapter on file globbing).

```
[paul@RHEL4b test]$ env LANG=C bash -c 'ls File[a-z]'
Filea  Fileb
[paul@RHEL4b test]$ env LANG=en_US.UTF-8 bash -c 'ls File[a-z]'
Filea  FileA  Fileb  FileB
[paul@RHEL4b test]$
```

8.10. export

You can export shell variables to other shells with the **export** command. This will export the variable to child shells.

```
[paul@RHEL4b ~]$ var3=three
[paul@RHEL4b ~]$ var4=four
[paul@RHEL4b ~]$ export var4
[paul@RHEL4b ~]$ echo $var3 $var4
three four
[paul@RHEL4b ~]$ bash
[paul@RHEL4b ~]$ echo $var3 $var4
four
```

But it will not export to the parent shell (previous screenshot continued).

```
[paul@RHEL4b ~]$ export var5=five
[paul@RHEL4b ~]$ echo $var3 $var4 $var5
four five
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ echo $var3 $var4 $var5
three four
[paul@RHEL4b ~]$
```

8.11. delineate variables

Until now, we have seen that bash interprets a variable starting from a dollar sign, continuing until the first occurrence of a non-alphanumeric character that is not an underscore. In some situations, this can be a problem. This issue can be resolved with curly braces like in this example.

```
[paul@RHEL4b ~]$ prefix=Super
[paul@RHEL4b ~]$ echo Hello $prefixman and $prefixgirl
Hello  and
[paul@RHEL4b ~]$ echo Hello ${prefix}man and ${prefix}girl
Hello Superman and Supergirl
[paul@RHEL4b ~]$
```

8.12. unbound variables

The example below tries to display the value of the **\$MyVar** variable, but it fails because the variable does not exist. By default the shell will display nothing when a variable is unbound (does not exist).

```
[paul@RHELv4u3 gen]$ echo $MyVar

[paul@RHELv4u3 gen]$
```

There is, however, the **nounset** shell option that you can use to generate an error when a variable does not exist.

```
paul@laika:~$ set -u
paul@laika:~$ echo $Myvar
bash: Myvar: unbound variable
paul@laika:~$ set +u
paul@laika:~$ echo $Myvar

paul@laika:~$
```

In the bash shell **set -u** is identical to **set -o nounset** and likewise **set +u** is identical to **set +o nounset**.

8.13. practice: shell variables

1. Use echo to display Hello followed by your username. (use a bash variable!)
2. Create a variable **answer** with a value of **42**.
3. Copy the value of \$LANG to \$MyLANG.
4. List all current shell variables.
5. List all exported shell variables.
6. Do the **env** and **set** commands display your variable ?
6. Destroy your **answer** variable.
7. Create two variables, and **export** one of them.
8. Display the exported variable in an interactive child shell.
9. Create a variable, give it the value 'Dumb', create another variable with value 'do'. Use **echo** and the two variables to echo Dumbledore.
10. Find the list of backslash escaped characters in the manual of bash. Add the time to your **PS1** prompt.

8.14. solution: shell variables

1. Use echo to display Hello followed by your username. (use a bash variable!)

```
echo Hello $USER
```

2. Create a variable **answer** with a value of **42**.

```
answer=42
```

3. Copy the value of \$LANG to \$MyLANG.

```
MyLANG=$LANG
```

4. List all current shell variables.

```
set
```

```
set | more on Ubuntu/Debian
```

5. List all exported shell variables.

```
env  
export  
declare -x
```

6. Do the **env** and **set** commands display your variable ?

```
env | more  
set | more
```

6. Destroy your **answer** variable.

```
unset answer
```

7. Create two variables, and **export** one of them.

```
var1=1; export var2=2
```

8. Display the exported variable in an interactive child shell.

```
bash  
echo $var2
```

9. Create a variable, give it the value 'Dumb', create another variable with value 'do'. Use **echo** and the two variables to echo Dumbledore.

```
varx=Dumb; vary=do  
  
echo ${varx}le${vary}re  
solution by Yves from Dexia : echo $varx'le'$vary're'  
solution by Erwin from Telenet : echo "$varx"le"$vary"re
```

10. Find the list of backslash escaped characters in the manual of bash. Add the time to your **PS1** prompt.

```
PS1='\t \u@\h \W$ '
```

Chapter 9. shell embedding and options

This chapter takes a brief look at **child shells**, **embedded shells** and **shell options**.

9.2. shell options

Both **set** and **unset** are builtin shell commands. They can be used to set options of the bash shell itself. The next example will clarify this. By default, the shell will treat unset variables as a variable having no value. By setting the **-u** option, the shell will treat any reference to unset variables as an error. See the man page of bash for more information.

```
[paul@RHEL4b ~]$ echo $var123

[paul@RHEL4b ~]$ set -u
[paul@RHEL4b ~]$ echo $var123
-bash: var123: unbound variable
[paul@RHEL4b ~]$ set +u
[paul@RHEL4b ~]$ echo $var123

[paul@RHEL4b ~]$
```

To list all the set options for your shell, use **echo \$-**. The **noclobber** (or **-C**) option will be explained later in this book (in the I/O redirection chapter).

```
[paul@RHEL4b ~]$ echo $-
himBH
[paul@RHEL4b ~]$ set -C ; set -u
[paul@RHEL4b ~]$ echo $-
himuBCH
[paul@RHEL4b ~]$ set +C ; set +u
[paul@RHEL4b ~]$ echo $-
himBH
[paul@RHEL4b ~]$
```

When typing **set** without options, you get a list of all variables without function when the shell is on **posix** mode. You can set bash in posix mode typing **set -o posix**.

9.3. solution: shell embedding

1. Find the list of shell options in the man page of **bash**. What is the difference between **set -u** and **set -o nounset**?

read the manual of bash (man bash), search for nounset -- both mean the same thing.

2. Activate **nounset** in your shell. Test that it shows an error message when using non-existing variables.

```
set -u
OR
set -o nounset
```

Both these lines have the same effect.

3. Deactivate nounset.

```
set +u
OR
set +o nounset
```

4. Execute **cd /var** and **ls** in an embedded shell.

```
echo $(cd /var ; ls)
```

The **echo** command is only needed to show the result of the **ls** command. Omitting will result in the shell trying to execute the first file as a command.

5. Create the variable **embvar** in an embedded shell and echo it. Does the variable exist in your current shell now ?

```
echo $(embvar=emb;echo $embvar) ; echo $embvar #the last echo fails
```

```
$embvar does not exist in your current shell
```

6. Explain what "set -x" does. Can this be useful ?

```
It displays shell expansion for troubleshooting your command.
```

(optional)7. Given the following screenshot, add exactly four characters to that command line so that the total output is FirstMiddleLast.

```
[paul@RHEL4b ~]$ echo First; echo Middle; echo Last
```

```
echo -n First; echo -n Middle; echo Last
```

8. Display a **long listing** (**ls -l**) of the **passwd** command using the **which** command inside an embedded shell.

```
ls -l $(which passwd)
```

Chapter 10. shell history

The shell makes it easy for us to repeat commands, this chapter explains how.

10.1. repeating the last command

To repeat the last command in bash, type **!!**. This is pronounced as **bang bang**.

```
paul@debian5:~/test42$ echo this will be repeated > file42.txt
paul@debian5:~/test42$ !!
echo this will be repeated > file42.txt
paul@debian5:~/test42$
```

10.2. repeating other commands

You can repeat other commands using one **bang** followed by one or more characters. The shell will repeat the last command that started with those characters.

```
paul@debian5:~/test42$ touch file42
paul@debian5:~/test42$ cat file42
paul@debian5:~/test42$ !to
touch file42
paul@debian5:~/test42$
```

10.3. history

To see older commands, use **history** to display the shell command history (or use **history n** to see the last n commands).

```
paul@debian5:~/test$ history 10
38  mkdir test
39  cd test
40  touch file1
41  echo hello > file2
42  echo It is very cold today > winter.txt
43  ls
44  ls -l
45  cp winter.txt summer.txt
46  ls -l
47  history 10
```

10.4. !n

When typing **!** followed by the number preceding the command you want repeated, then the shell will echo the command and execute it.

```
paul@debian5:~/test$ !43
ls
file1  file2  summer.txt  winter.txt
```

10.5. Ctrl-r

Another option is to use **ctrl-r** to search in the history. In the screenshot below i only typed **ctrl-r** followed by four characters **apti** and it finds the last command containing these four consecutive characters.

```
paul@debian5:~$  
(reverse-i-search)`apti': sudo aptitude install screen
```

10.6. \$HISTSIZE

The `$HISTSIZE` variable determines the number of commands that will be remembered in your current environment. Most distributions default this variable to 500 or 1000.

```
paul@debian5:~$ echo $HISTSIZE  
500
```

You can change it to any value you like.

```
paul@debian5:~$ HISTSIZE=15000  
paul@debian5:~$ echo $HISTSIZE  
15000
```

10.7. \$HISTFILE

The `$HISTFILE` variable points to the file that contains your history. The **bash** shell defaults this value to `~/.bash_history`.

```
paul@debian5:~$ echo $HISTFILE  
/home/paul/.bash_history
```

A session history is saved to this file when you **exit** the session!

*Closing a gnome-terminal with the mouse, or typing **reboot** as root will NOT save your terminal's history.*

10.8. \$HISTFILESIZE

The number of commands kept in your history file can be set using `$HISTFILESIZE`.

```
paul@debian5:~$ echo $HISTFILESIZE  
15000
```

10.9. prevent recording a command

You can prevent a command from being recorded in **history** using a space prefix.

```
paul@debian8:~/github$ echo abc
abc
paul@debian8:~/github$ echo def
def
paul@debian8:~/github$ echo ghi
ghi
paul@debian8:~/github$ history 3
9501  echo abc
9502  echo ghi
9503  history 3
```

10.10. (optional)regular expressions

It is possible to use **regular expressions** when using the **bang** to repeat commands. The screenshot below switches 1 into 2.

```
paul@debian5:~/test$ cat file1
paul@debian5:~/test$ !c:s/1/2
cat file2
hello
paul@debian5:~/test$
```

10.11. (optional) Korn shell history

Repeating a command in the **Korn shell** is very similar. The Korn shell also has the **history** command, but uses the letter **r** to recall lines from history.

This screenshot shows the history command. Note the different meaning of the parameter.

```
$ history 17
17  clear
18  echo hoi
19  history 12
20  echo world
21  history 17
```

Repeating with **r** can be combined with the line numbers given by the history command, or with the first few letters of the command.

```
$ r e
echo world
world
$ cd /etc
$ r
cd /etc
$
```

10.12. practice: shell history

1. Issue the command **echo The answer to the meaning of life, the universe and everything is 42.**
2. Repeat the previous command using only two characters (there are two solutions!)
3. Display the last 5 commands you typed.
4. Issue the long **echo** from question 1 again, using the line numbers you received from the command in question 3.
5. How many commands can be kept in memory for your current shell session ?
6. Where are these commands stored when exiting the shell ?
7. How many commands can be written to the **history file** when exiting your current shell session ?
8. Make sure your current bash shell remembers the next 5000 commands you type.
9. Open more than one console (by press Ctrl-shift-t in gnome-terminal, or by opening an extra putty.exe in MS Windows) with the same user account. When is command history written to the history file ?

10.13. solution: shell history

1. Issue the command **echo The answer to the meaning of life, the universe and everything is 42.**

```
echo The answer to the meaning of life, the universe and everything is 42
```

2. Repeat the previous command using only two characters (there are two solutions!)

```
!!  
OR  
!e
```

3. Display the last 5 commands you typed.

```
paul@ubu1010:~$ history 5  
52  ls -l  
53  ls  
54  df -h | grep sda  
55  echo The answer to the meaning of life, the universe and everything is 42  
56  history 5
```

You will receive different line numbers.

4. Issue the long **echo** from question 1 again, using the line numbers you received from the command in question 3.

```
paul@ubu1010:~$ !55  
echo The answer to the meaning of life, the universe and everything is 42  
The answer to the meaning of life, the universe and everything is 42
```

5. How many commands can be kept in memory for your current shell session ?

```
echo $HISTSIZE
```

6. Where are these commands stored when exiting the shell ?

```
echo $HISTFILE
```

7. How many commands can be written to the **history file** when exiting your current shell session ?

```
echo $HISTFILESIZE
```

8. Make sure your current bash shell remembers the next 5000 commands you type.

```
HISTSIZE=5000
```

9. Open more than one console (by press Ctrl-shift-t in gnome-terminal, or by opening an extra putty.exe in MS Windows) with the same user account. When is command history written to the history file ?

```
when you type exit
```

Chapter 11. file globbing

The shell is also responsible for **file globbing** (or dynamic filename generation). This chapter will explain **file globbing**.

11.1. * asterisk

The asterisk ***** is interpreted by the shell as a sign to generate filenames, matching the asterisk to any combination of characters (even none). When no path is given, the shell will use filenames in the current directory. See the man page of **glob(7)** for more information. (This is part of LPI topic 1.103.3.)

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File*
File4 File55 FileA Fileab FileAB
[paul@RHELv4u3 gen]$ ls file*
file1 file2 file3 fileab fileabc
[paul@RHELv4u3 gen]$ ls *ile55
File55
[paul@RHELv4u3 gen]$ ls F*ile55
File55
[paul@RHELv4u3 gen]$ ls F*55
File55
[paul@RHELv4u3 gen]$
```

11.2. ? question mark

Similar to the asterisk, the question mark **?** is interpreted by the shell as a sign to generate filenames, matching the question mark with exactly one character.

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File?
File4 FileA
[paul@RHELv4u3 gen]$ ls Fil?4
File4
[paul@RHELv4u3 gen]$ ls Fil??
File4 FileA
[paul@RHELv4u3 gen]$ ls File??
File55 Fileab FileAB
[paul@RHELv4u3 gen]$
```

11.3. [] square brackets

The square bracket [is interpreted by the shell as a sign to generate filenames, matching any of the characters between [and the first subsequent]. The order in this list between the brackets is not important. Each pair of brackets is replaced by exactly one character.

```
[paul@RHELv4u3 gen]$ ls
file1  file2  file3  File4  File55  FileA  fileab  Fileab  FileAB  fileabc
[paul@RHELv4u3 gen]$ ls File[5A]
FileA
[paul@RHELv4u3 gen]$ ls File[A5]
FileA
[paul@RHELv4u3 gen]$ ls File[A5][5b]
File55
[paul@RHELv4u3 gen]$ ls File[a5][5b]
File55  Fileab
[paul@RHELv4u3 gen]$ ls File[a5][5b][abcdefghijklm]
ls: File[a5][5b][abcdefghijklm]: No such file or directory
[paul@RHELv4u3 gen]$ ls file[a5][5b][abcdefghijklm]
fileabc
[paul@RHELv4u3 gen]$
```

You can also exclude characters from a list between square brackets with the exclamation mark !. And you are allowed to make combinations of these **wild cards**.

```
[paul@RHELv4u3 gen]$ ls
file1  file2  file3  File4  File55  FileA  fileab  Fileab  FileAB  fileabc
[paul@RHELv4u3 gen]$ ls file[a5][!Z]
fileab
[paul@RHELv4u3 gen]$ ls file[!5]*
file1  file2  file3  fileab  fileabc
[paul@RHELv4u3 gen]$ ls file[!5]?
fileab
[paul@RHELv4u3 gen]$
```

11.4. a-z and 0-9 ranges

The bash shell will also understand ranges of characters between brackets.

```
[paul@RHELv4u3 gen]$ ls
file1  file3  File55  fileab  FileAB  fileabc
file2  File4  FileA   Fileab  fileab2
[paul@RHELv4u3 gen]$ ls file[a-z]*
fileab  fileab2  fileabc
[paul@RHELv4u3 gen]$ ls file[0-9]
file1  file2  file3
[paul@RHELv4u3 gen]$ ls file[a-z][a-z][0-9]*
fileab2
[paul@RHELv4u3 gen]$
```

11.5. \$LANG and square brackets

But, don't forget the influence of the **LANG** variable. Some languages include lower case letters in an upper case range (and vice versa).

```
paul@RHELv4u4:~/test$ ls [A-Z]ile?
file1  file2  file3  File4
paul@RHELv4u4:~/test$ ls [a-z]ile?
file1  file2  file3  File4
paul@RHELv4u4:~/test$ echo $LANG
en_US.UTF-8
paul@RHELv4u4:~/test$ LANG=C
paul@RHELv4u4:~/test$ echo $LANG
C
paul@RHELv4u4:~/test$ ls [a-z]ile?
file1  file2  file3
paul@RHELv4u4:~/test$ ls [A-Z]ile?
File4
paul@RHELv4u4:~/test$
```

If **\$LC_ALL** is set, then this will also need to be reset to prevent file globbing.

11.6. preventing file globbing

The screenshot below should be no surprise. The **echo *** will echo a ***** when in an empty directory. And it will echo the names of all files when the directory is not empty.

```
paul@ubu1010:~$ mkdir test42
paul@ubu1010:~$ cd test42
paul@ubu1010:~/test42$ echo *
*
paul@ubu1010:~/test42$ touch file42 file33
paul@ubu1010:~/test42$ echo *
file33 file42
```

Globbering can be prevented using quotes or by escaping the special characters, as shown in this screenshot.

```
paul@ubu1010:~/test42$ echo *
file33 file42
paul@ubu1010:~/test42$ echo \*
*
paul@ubu1010:~/test42$ echo '*'
*
paul@ubu1010:~/test42$ echo "*"
*
```

11.7. solution: shell globbing

1. Create a test directory and enter it.

```
mkdir testdir; cd testdir
```

2. Create the following files :

```
file1  
file10  
file11  
file2  
File2  
File3  
file33  
fileAB  
filea  
fileA  
fileAAA  
file(  
file 2
```

(the last one has 6 characters including a space)

```
touch file1 file10 file11 file2 File2 File3  
touch file33 fileAB filea fileA fileAAA  
touch "file("  
touch "file 2"
```

3. List (with ls) all files starting with file

```
ls file*
```

4. List (with ls) all files starting with File

```
ls File*
```

5. List (with ls) all files starting with file and ending in a number.

```
ls file*[0-9]
```

6. List (with ls) all files starting with file and ending with a letter

```
ls file*[a-z]
```

7. List (with ls) all files starting with File and having a digit as fifth character.

```
ls File[0-9]*
```

8. List (with ls) all files starting with File and having a digit as fifth character and nothing else.

```
ls File[0-9]
```

9. List (with ls) all files starting with a letter and ending in a number.

```
ls [a-z]*[0-9]
```

10. List (with ls) all files that have exactly five characters.


```
ls ?????
```

11. List (with ls) all files that start with f or F and end with 3 or A.

```
ls [fF]*[3A]
```

12. List (with ls) all files that start with f have i or R as second character and end in a number.

```
ls f[iR]*[0-9]
```

13. List all files that do not start with the letter F.

```
ls [!F]*
```

14. Copy the value of \$LANG to \$MyLANG.

```
MyLANG=$LANG
```

15. Show the influence of \$LANG in listing A-Z or a-z ranges.

```
see example in book
```

16. You receive information that one of your servers was cracked, the cracker probably replaced the **ls** command. You know that the **echo** command is safe to use. Can **echo** replace **ls** ? How can you list the files in the current directory with **echo** ?

```
echo *
```

17. Is there another command besides cd to change directories ?

```
pushd popd
```

Chapter 12. I/O redirection

One of the powers of the Unix command line is the use of **input/output redirection** and **pipes**.

This chapter explains **redirection** of input, output and error streams.

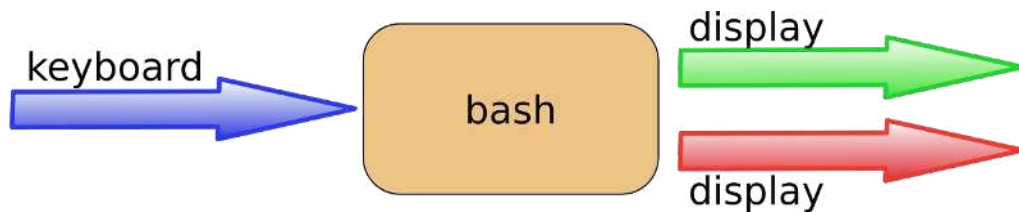
12.1. stdin, stdout, and stderr

The bash shell has three basic streams; it takes input from **stdin** (stream **0**), it sends output to **stdout** (stream **1**) and it sends error messages to **stderr** (stream **2**) .

The drawing below has a graphical interpretation of these three streams.



The keyboard often serves as **stdin**, whereas **stdout** and **stderr** both go to the display. This can be confusing to new Linux users because there is no obvious way to recognize **stdout** from **stderr**. Experienced users know that separating output from errors can be very useful.

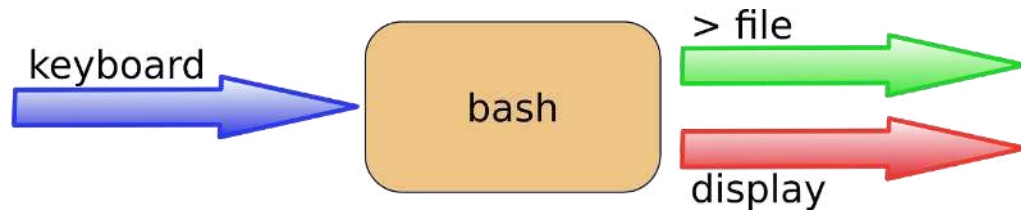


The next sections will explain how to redirect these streams.

12.2. output redirection

12.2.1. > stdout

stdout can be redirected with a **greater than** sign. While scanning the line, the shell will see the > sign and will clear the file.



The > notation is in fact the abbreviation of **1>** (**stdout** being referred to as stream **1**).

```
[paul@RHELv4u3 ~]$ echo It is cold today!  
It is cold today!  
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt  
[paul@RHELv4u3 ~]$ cat winter.txt  
It is cold today!  
[paul@RHELv4u3 ~]$
```

Note that the bash shell effectively **removes** the redirection from the command line before argument 0 is executed. This means that in the case of this command:

```
echo hello > greetings.txt
```

the shell only counts two arguments (echo = argument 0, hello = argument 1). The redirection is removed before the argument counting takes place.

12.2.2. output file is erased

While scanning the line, the shell will see the > sign and **will clear the file!** Since this happens before resolving **argument 0**, this means that even when the command fails, the file will have been cleared!

```
[paul@RHELv4u3 ~]$ cat winter.txt  
It is cold today!  
[paul@RHELv4u3 ~]$ zcho It is cold today! > winter.txt  
-bash: zcho: command not found  
[paul@RHELv4u3 ~]$ cat winter.txt  
[paul@RHELv4u3 ~]$
```

12.2.3. noclobber

Erasing a file while using > can be prevented by setting the **noclobber** option.

```
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ set -o noclobber
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
-bash: winter.txt: cannot overwrite existing file
[paul@RHELv4u3 ~]$ set +o noclobber
[paul@RHELv4u3 ~]$
```

12.2.4. overruling noclobber

The **noclobber** can be overruled with >|.

```
[paul@RHELv4u3 ~]$ set -o noclobber
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
-bash: winter.txt: cannot overwrite existing file
[paul@RHELv4u3 ~]$ echo It is very cold today! >| winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is very cold today!
[paul@RHELv4u3 ~]$
```

12.2.5. >> append

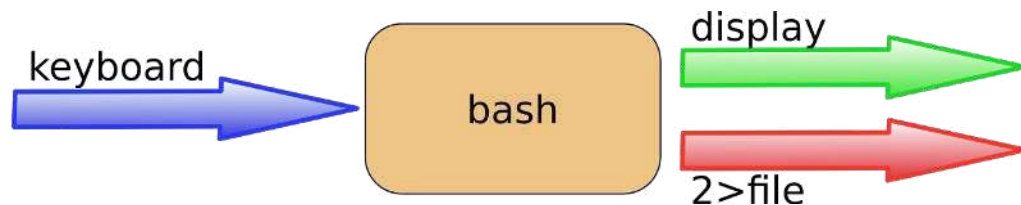
Use >> to **append** output to a file.

```
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ echo Where is the summer ? >> winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
Where is the summer ?
[paul@RHELv4u3 ~]$
```

12.3. error redirection

12.3.1. 2> stderr

Redirecting **stderr** is done with **2>**. This can be very useful to prevent error messages from cluttering your screen.



The screenshot below shows redirection of **stdout** to a file, and **stderr** to **/dev/null**. Writing **1>** is the same as **>**.

```
[paul@RHELv4u3 ~]$ find / > allfiles.txt 2> /dev/null
[paul@RHELv4u3 ~]$
```

18.3.2. 2>&1

To redirect both **stdout** and **stderr** to the same file, use **2>&1**.

```
[paul@RHELv4u3 ~]$ find / > allfiles_and_errors.txt 2>&1
[paul@RHELv4u3 ~]$
```

Note that the order of redirections is significant. For example, the command

```
ls > dirlist 2>&1
```

directs both standard output (file descriptor 1) and standard error (file descriptor 2) to the file `dirlist`, while the command

```
ls 2>&1 > dirlist
```

directs only the standard output to file `dirlist`, because the standard error made a copy of the standard output before the standard output was redirected to `dirlist`.

12.4. output redirection and pipes

By default you cannot `grep` inside **stderr** when using pipes on the command line, because only **stdout** is passed.

```
paul@debian7:~$ rm file42 file33 file1201 | grep file42
rm: cannot remove 'file42': No such file or directory
rm: cannot remove 'file33': No such file or directory
rm: cannot remove 'file1201': No such file or directory
```

With **2>&1** you can force **stderr** to go to **stdout**. This enables the next command in the pipe to act on both streams.

```
paul@debian7:~$ rm file42 file33 file1201 2>&1 | grep file42
rm: cannot remove 'file42': No such file or directory
```

You cannot use both **1>&2** and **2>&1** to switch **stdout** and **stderr**.

```
paul@debian7:~$ rm file42 file33 file1201 2>&1 1>&2 | grep file42
rm: cannot remove 'file42': No such file or directory
paul@debian7:~$ echo file42 2>&1 1>&2 | sed 's/file42/FILE42/'
FILE42
```

You need a third stream to switch **stdout** and **stderr** after a pipe symbol.

```
paul@debian7:~$ echo file42 3>&1 1>&2 2>&3 | sed 's/file42/FILE42/'
file42
paul@debian7:~$ rm file42 3>&1 1>&2 2>&3 | sed 's/file42/FILE42/'
rm: cannot remove 'FILE42': No such file or directory
```

12.5. joining stdout and stderr

The **&>** construction will put both **stdout** and **stderr** in one stream (to a file).

```
paul@debian7:~$ rm file42 &> out_and_err
paul@debian7:~$ cat out_and_err
rm: cannot remove 'file42': No such file or directory
paul@debian7:~$ echo file42 &> out_and_err
paul@debian7:~$ cat out_and_err
file42
paul@debian7:~$
```

12.6. input redirection

12.6.1. < stdin

Redirecting **stdin** is done with < (short for 0<).

```
[paul@RHEL4b ~]$ cat < text.txt
one
two
[paul@RHEL4b ~]$ tr 'onetw' 'ONEZZ' < text.txt
ONE
ZZO
[paul@RHEL4b ~]$
```

12.6.2. << here document

The **here document** (sometimes called here-is-document) is a way to append input until a certain sequence (usually EOF) is encountered. The **EOF** marker can be typed literally or can be called with Ctrl-D.

```
[paul@RHEL4b ~]$ cat <<EOF > text.txt
> one
> two
> EOF
[paul@RHEL4b ~]$ cat text.txt
one
two
[paul@RHEL4b ~]$ cat <<bro1 > text.txt
> brel
> bro1
[paul@RHEL4b ~]$ cat text.txt
brel
[paul@RHEL4b ~]$
```

12.6.3. <<< here string

The **here string** can be used to directly pass strings to a command. The result is the same as using **echo string | command** (but you have one less process running).

```
paul@ubu1110~$ base64 <<< linux-training.be
bGludXgt dHJhaW5pbmcuYmUK
paul@ubu1110~$ base64 -d <<< bGludXgt dHJhaW5pbmcuYmUK
linux-training.be
```

See rfc 3548 for more information about **base64**.

12.7. confusing redirection

The shell will scan the whole line before applying redirection. The following command line is very readable and is correct.

```
cat winter.txt > snow.txt 2> errors.txt
```

But this one is also correct, but less readable.

```
2> errors.txt cat winter.txt > snow.txt
```

Even this will be understood perfectly by the shell.

```
< winter.txt > snow.txt 2> errors.txt cat
```

12.8. quick file clear

So what is the quickest way to clear a file ?

```
>foo
```

And what is the quickest way to clear a file when the **noclobber** option is set ?

12.9. solution: input/output redirection

1. Activate the **noclobber** shell option.

```
set -o noclobber
set -C
```

2. Verify that **noclobber** is active by repeating an **ls** on **/etc/** with redirected output to a file.

```
ls /etc > etc.txt
ls /etc > etc.txt (should not work)
```

4. When listing all shell options, which character represents the **noclobber** option ?

```
echo $- (noclobber is visible as C)
```

5. Deactivate the **noclobber** option.

```
set +o noclobber
```

6. Make sure you have two shells open on the same computer. Create an empty **tailing.txt** file. Then type **tail -f tailing.txt**. Use the second shell to **append** a line of text to that file. Verify that the first shell displays this line.

```
paul@deb503:~$ > tailing.txt
paul@deb503:~$ tail -f tailing.txt
hello
world

in the other shell:
paul@deb503:~$ echo hello >> tailing.txt
paul@deb503:~$ echo world >> tailing.txt
```

7. Create a file that contains the names of five people. Use **cat** and output redirection to create the file and use a **here document** to end the input.

```
paul@deb503:~$ cat > tennis.txt << ace
> Justine Henin
> Venus Williams
> Serena Williams
> Martina Hingis
> Kim Clijsters
> ace
paul@deb503:~$ cat tennis.txt
Justine Henin
Venus Williams
Serena Williams
Martina Hingis
Kim Clijsters
paul@deb503:~$
```

Chapter 13. filters

Commands that are created to be used with a **pipe** are often called **filters**. These **filters** are very small programs that do one specific thing very efficiently. They can be used as **building blocks**.

This chapter will introduce you to the most common **filters**. The combination of simple commands and filters in a long **pipe** allows you to design elegant solutions.

13.1. cat

When between two **pipes**, the **cat** command does nothing (except putting **stdin** on **stdout**).

```
[paul@RHEL4b pipes]$ tac count.txt | cat | cat | cat | cat | cat
five
four
three
two
one
[paul@RHEL4b pipes]$
```

13.2. tee

Writing long **pipes** in Unix is fun, but sometimes you may want intermediate results. This is where **tee** comes in handy. The **tee** filter puts **stdin** on **stdout** and also into a file. So **tee** is almost the same as **cat**, except that it has two identical outputs.

```
[paul@RHEL4b pipes]$ tac count.txt | tee temp.txt | tac
one
two
three
four
five
[paul@RHEL4b pipes]$ cat temp.txt
five
four
three
two
one
[paul@RHEL4b pipes]$
```

13.3. grep

The **grep** filter is famous among Unix users. The most common use of **grep** is to filter lines of text containing (or not containing) a certain string.

```
[paul@RHEL4b pipes]$ cat tennis.txt
Amelie Mauresmo, Fra
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
Venus Williams, USA
[paul@RHEL4b pipes]$ cat tennis.txt | grep Williams
Serena Williams, usa
Venus Williams, USA
```

You can write this without the **cat**.

```
[paul@RHEL4b pipes]$ grep Williams tennis.txt
Serena Williams, usa
Venus Williams, USA
```

One of the most useful options of **grep** is **grep -i** which filters in a case insensitive way.

```
[paul@RHEL4b pipes]$ grep Bel tennis.txt
Justine Henin, Bel
[paul@RHEL4b pipes]$ grep -i Bel tennis.txt
```

```
Kim Clijsters, BEL
Justine Henin, Bel
[paul@RHEL4b pipes]$
```

Another very useful option is **grep -v** which outputs lines not matching the string.

```
[paul@RHEL4b pipes]$ grep -v Fra tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
Venus Williams, USA
[paul@RHEL4b pipes]$
```

And of course, both options can be combined to filter all lines not containing a case insensitive string.

```
[paul@RHEL4b pipes]$ grep -vi usa tennis.txt
Amelie Mauresmo, Fra
Kim Clijsters, BEL
Justine Henin, Bel
[paul@RHEL4b pipes]$
```

With **grep -A1** one line **after** the result is also displayed.

```
paul@debian5:~/pipes$ grep -A1 Henin tennis.txt
Justine Henin, Bel
Serena Williams, usa
```

With **grep -B1** one line **before** the result is also displayed.

```
paul@debian5:~/pipes$ grep -B1 Henin tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
```

With **grep -C1** (context) one line **before** and one **after** are also displayed. All three options (A,B, and C) can display any number of lines (using e.g. A2, B4 or C20).

```
paul@debian5:~/pipes$ grep -C1 Henin tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
```

13.4. cut

The **cut** filter can select columns from files, depending on a delimiter or a count of bytes. The screenshot below uses **cut** to filter for the username and userid in the **/etc/passwd** file. It uses the colon as a delimiter, and selects fields 1 and 3.

```
[paul@RHEL4b pipes]$ cut -d: -f1,3 /etc/passwd | tail -4
Figo:510
Pfaff:511
Harry:516
Hermione:517
[paul@RHEL4b pipes]$
```

When using a space as the delimiter for **cut**, you have to quote the space.

```
[paul@RHEL4b pipes]$ cut -d" " -f1 tennis.txt
Amelie
Kim
Justine
Serena
Venus
[paul@RHEL4b pipes]$
```

This example uses **cut** to display the second to the seventh character of **/etc/passwd**.

```
[paul@RHEL4b pipes]$ cut -c2-7 /etc/passwd | tail -4
igo:x:
faff:x
arry:x
ermion
[paul@RHEL4b pipes]$
```

13.5. tr

You can translate characters with **tr**. The screenshot shows the translation of all occurrences of **e** to **E**.

```
[paul@RHEL4b pipes]$ cat tennis.txt | tr 'e' 'E'
AmElie MaurEsmo, Fra
Kim CliJstErs, BEL
JustinE HEnin, BEl
SErEna Williams, usa
VEnus Williams, USA
```

Here we set all letters to uppercase by defining two ranges.

```
[paul@RHEL4b pipes]$ cat tennis.txt | tr 'a-z' 'A-Z'
AMELIE MAURES MO, FRA
KIM CLIJSTERS, BEL
JUSTINE HENIN, BEL
SERENA WILLIAMS, USA
VENUS WILLIAMS, USA
[paul@RHEL4b pipes]$
```

Here we translate all newlines to spaces.

```
[paul@RHEL4b pipes]$ cat count.txt
one
two
```

```
three
four
five
[paul@RHEL4b pipes]$ cat count.txt | tr '\n' ' '
one two three four five [paul@RHEL4b pipes]$
```

The **tr -s** filter can also be used to squeeze multiple occurrences of a character to one.

```
[paul@RHEL4b pipes]$ cat spaces.txt
one    two        three
      four   five six
[paul@RHEL4b pipes]$ cat spaces.txt | tr -s ' '
one two three
  four five six
[paul@RHEL4b pipes]$
```

You can also use **tr** to 'encrypt' texts with **rot13**.

```
[paul@RHEL4b pipes]$ cat count.txt | tr 'a-z' 'nopqrstuvwxyzabcdefghijklm'
bar
gjb
guerr
sbhe
svir
[paul@RHEL4b pipes]$ cat count.txt | tr 'a-z' 'n-za-m'
bar
gjb
guerr
sbhe
svir
[paul@RHEL4b pipes]$
```

This last example uses **tr -d** to delete characters.

```
paul@debian5:~/pipes$ cat tennis.txt | tr -d e
Amlı Maursmo, Fra
Kim Clijstrs, BEL
Justin Hnin, Bl
Srna Williams, usa
Vnus Williams, USA
```

13.6. wc

Counting words, lines and characters is easy with **wc**.

```
[paul@RHEL4b pipes]$ wc tennis.txt
 5  15 100 tennis.txt
[paul@RHEL4b pipes]$ wc -l tennis.txt
5 tennis.txt
[paul@RHEL4b pipes]$ wc -w tennis.txt
15 tennis.txt
[paul@RHEL4b pipes]$ wc -c tennis.txt
100 tennis.txt
[paul@RHEL4b pipes]$
```


13.7. sort

The **sort** filter will default to an alphabetical sort.

```
paul@debian5:~/pipes$ cat music.txt
Queen
Brel
Led Zeppelin
Abba
paul@debian5:~/pipes$ sort music.txt
Abba
Brel
Led Zeppelin
Queen
```

But the **sort** filter has many options to tweak its usage. This example shows sorting different columns (column 1 or column 2).

```
[paul@RHEL4b pipes]$ sort -k1 country.txt
Belgium, Brussels, 10
France, Paris, 60
Germany, Berlin, 100
Iran, Teheran, 70
Italy, Rome, 50
[paul@RHEL4b pipes]$ sort -k2 country.txt
Germany, Berlin, 100
Belgium, Brussels, 10
France, Paris, 60
Italy, Rome, 50
Iran, Teheran, 70
```

The screenshot below shows the difference between an alphabetical sort and a numerical sort (both on the third column).

```
[paul@RHEL4b pipes]$ sort -k3 country.txt
Belgium, Brussels, 10
Germany, Berlin, 100
Italy, Rome, 50
France, Paris, 60
Iran, Teheran, 70
[paul@RHEL4b pipes]$ sort -n -k3 country.txt
Belgium, Brussels, 10
Italy, Rome, 50
France, Paris, 60
Iran, Teheran, 70
Germany, Berlin, 100
```

13.8. uniq

With **uniq** you can remove duplicates from a **sorted list**.

```
paul@debian5:~/pipes$ cat music.txt
Queen
Brel
Queen
Abba
paul@debian5:~/pipes$ sort music.txt
Abba
Brel
Queen
Queen
paul@debian5:~/pipes$ sort music.txt |uniq
Abba
Brel
Queen
```

uniq can also count occurrences with the **-c** option.

```
paul@debian5:~/pipes$ sort music.txt |uniq -c
  1 Abba
  1 Brel
  2 Queen
```

13.9. comm

Comparing streams (or files) can be done with the **comm**. By default **comm** will output three columns. In this example, Abba, Cure and Queen are in both lists, Bowie and Sweet are only in the first file, Turner is only in the second.

```
paul@debian5:~/pipes$ cat > list1.txt
Abba
Bowie
Cure
Queen
Sweet
paul@debian5:~/pipes$ cat > list2.txt
Abba
Cure
Queen
Turner
paul@debian5:~/pipes$ comm list1.txt list2.txt
      Abba
Bowie
      Cure
      Queen
Sweet
      Turner
```

The output of **comm** can be easier to read when outputting only a single column. The digits point out which output columns should not be displayed.

```
paul@debian5:~/pipes$ comm -12 list1.txt list2.txt
Abba
Cure
Queen
paul@debian5:~/pipes$ comm -13 list1.txt list2.txt
Turner
paul@debian5:~/pipes$ comm -23 list1.txt list2.txt
Bowie
Sweet
```

13.10. od

European humans like to work with ascii characters, but computers store files in bytes. The example below creates a simple file, and then uses **od** to show the contents of the file in hexadecimal bytes

```
paul@laika:~/test$ cat > text.txt
abcdefg
1234567
paul@laika:~/test$ od -t x1 text.txt
0000000 61 62 63 64 65 66 67 0a 31 32 33 34 35 36 37 0a
0000020
```

The same file can also be displayed in octal bytes.

```
paul@laika:~/test$ od -b text.txt
0000000 141 142 143 144 145 146 147 012 061 062 063 064 065 066 067 012
0000020
```

And here is the file in ascii (or backslashed) characters.

```
paul@laika:~/test$ od -c text.txt
0000000  a  b  c  d  e  f  g  \n  1  2  3  4  5  6  7  \n
0000020
```

13.11. sed

The stream **editor sed** can perform editing functions in the stream, using **regular expressions**.

```
paul@debian5:~/pipes$ echo level5 | sed 's/5/42/'
level42
paul@debian5:~/pipes$ echo level5 | sed 's/level/jump/'
jump5
```

Add **g** for global replacements (all occurrences of the string per line).

```
paul@debian5:~/pipes$ echo level5 level7 | sed 's/level/jump/'
jump5 level7
paul@debian5:~/pipes$ echo level5 level7 | sed 's/level/jump/g'
jump5 jump7
```

With **d** you can remove lines from a stream containing a character.

```
paul@debian5:~/test42$ cat tennis.txt
Venus Williams, USA
Martina Hingis, SUI
Justine Henin, BE
Serena williams, USA
Kim Clijsters, BE
Yanina Wickmayer, BE
paul@debian5:~/test42$ cat tennis.txt | sed '/BE/d'
Venus Williams, USA
Martina Hingis, SUI
Serena williams, USA
```

13.12. pipe examples

13.12.1. who | wc

How many users are logged on to this system ?

```
[paul@RHEL4b pipes]$ who
root      tty1          Jul 25 10:50
paul      pts/0           Jul 25 09:29 (laika)
Harry    pts/1           Jul 25 12:26 (barry)
paul      pts/2           Jul 25 12:26 (pasha)
[paul@RHEL4b pipes]$ who | wc -l
4
```

13.12.2. who | cut | sort

Display a sorted list of logged on users.

```
[paul@RHEL4b pipes]$ who | cut -d' ' -f1 | sort
Harry
paul
paul
root
```

Display a sorted list of logged on users, but every user only once .

```
[paul@RHEL4b pipes]$ who | cut -d' ' -f1 | sort | uniq
Harry
paul
root
```

13.12.3. grep | cut

Display a list of all bash **user accounts** on this computer. Users accounts are explained in detail later.

```
paul@debian5:~$ grep bash /etc/passwd
root:x:0:0:root:/root:/bin/bash
paul:x:1000:1000:paul,,,:/home/paul:/bin/bash
serena:x:1001:1001:./home/serena:/bin/bash
paul@debian5:~$ grep bash /etc/passwd | cut -d: -f1
root
paul
serena
```

13.13. solution: filters

1. Put a sorted list of all bash users in bashusers.txt.

```
grep bash /etc/passwd | cut -d: -f1 | sort > bashusers.txt
```

2. Put a sorted list of all logged on users in onlineusers.txt.

```
who | cut -d' ' -f1 | sort > onlineusers.txt
```

3. Make a list of all filenames in **/etc** that contain the string **conf** in their filename.

```
ls /etc | grep conf
```

4. Make a sorted list of all files in **/etc** that contain the case insensitive string **conf** in their filename.

```
ls /etc | grep -i conf | sort
```

5. Look at the output of **/sbin/ifconfig**. Write a line that displays only ip address and the subnet mask.

```
/sbin/ifconfig | head -2 | grep 'inet ' | tr -s ' ' | cut -d' ' -f3,5
```

6. Write a line that removes all non-letters from a stream.

```
paul@deb503:~$ cat text
This is, yes really! , a text with ?&* too many str$ange# characters ;-)
paul@deb503:~$ cat text | tr -d ',!$?.*&^%#@;()-'
This is yes really a text with too many strange characters
```

7. Write a line that receives a text file, and outputs all words on a separate line.

```
paul@deb503:~$ cat text2
it is very cold today without the sun

paul@deb503:~$ cat text2 | tr ' ' '\n'
it
is
very
cold
today
without
the
sun
```

8. Write a spell checker on the command line. (There may be a dictionary in **/usr/share/dict/**.)

```
paul@rhel ~$ echo "The zun is shining today" > text

paul@rhel ~$ cat > DICT
is
shining
sun
the
```

today

```
paul@rhel ~$ cat text | tr 'A-Z ' 'a-z\n' | sort | uniq | comm -23 - DICT  
zun
```

You could also add the solution from question number 6 to remove non-letters, and **tr -s ' '** to remove redundant spaces.

Chapter 14. basic Unix tools

This chapter introduces commands to **find** or **locate** files and to **compress** files, together with other common tools that were not discussed before. While the tools discussed here are technically not considered **filters**, they can be used in **pipes**.

14.1. find

The **find** command can be very useful at the start of a pipe to search for files. Here are some examples. You might want to add **2>/dev/null** to the command lines to avoid cluttering your screen with error messages.

Find all files in **/etc** and put the list in etcfiles.txt

```
find /etc > etcfiles.txt
```

Find all files of the entire system and put the list in allfiles.txt

```
find / > allfiles.txt
```

Find files that end in .conf in the current directory (and all subdirs).

```
find . -name "*.conf"
```

Find files of type file (not directory, pipe or etc.) that end in .conf.

```
find . -type f -name "*.conf"
```

Find files of type directory that end in .bak .

```
find /data -type d -name "*.bak"
```

Find files that are newer than file42.txt

```
find . -newer file42.txt
```

Find can also execute another command on every file found. This example will look for *.odf files and copy them to /backup/.

```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

Find can also execute, after your confirmation, another command on every file found. This example will remove *.odf files if you approve of it for every file found.

```
find /data -name "*.odf" -ok rm {} \;
```

14.2. locate

The **locate** tool is very different from **find** in that it uses an index to locate files. This is a lot faster than traversing all the directories, but it also means that it is always outdated. If the index does not exist yet, then you have to create it (as root on Red Hat Enterprise Linux) with the **updatedb** command.

```
[paul@RHEL4b ~]$ locate Samba
warning: locate: could not open database: /var/lib/slocate/slocate.db:...
warning: You need to run the 'updatedb' command (as root) to create th...
Please have a look at /etc/updatedb.conf to enable the daily cron job.
[paul@RHEL4b ~]$ updatedb
fatal error: updatedb: You are not authorized to create a default sloc...
[paul@RHEL4b ~]$ su -
Password:
[root@RHEL4b ~]# updatedb
[root@RHEL4b ~]#
```

Most Linux distributions will schedule the **updatedb** to run once every day.

14.3. date

The **date** command can display the date, time, time zone and more.

```
paul@rhel55 ~$ date
Sat Apr 17 12:44:30 CEST 2010
```

A date string can be customised to display the format of your choice. Check the man page for more options.

```
paul@rhel55 ~$ date +%A %d-%m-%Y
Saturday 17-04-2010
```

Time on any Unix is calculated in number of seconds since 1969 (the first second being the first second of the first of January 1970). Use **date +%s** to display Unix time in seconds.

```
paul@rhel55 ~$ date +%s
1271501080
```

When will this seconds counter reach two thousand million ?

```
paul@rhel55 ~$ date -d '1970-01-01 + 2000000000 seconds'
Wed May 18 04:33:20 CEST 2033
```

14.4. cal

The **cal** command displays the current month, with the current day highlighted.

```
paul@rhel55 ~$ cal
      April 2010
Su Mo Tu We Th Fr Sa
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30
```

You can select any month in the past or the future.

```
paul@rhel55 ~$ cal 2 1970
      February 1970
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
```

14.5. sleep

The **sleep** command is sometimes used in scripts to wait a number of seconds. This example shows a five second **sleep**.

```
paul@rhel55 ~$ sleep 5
paul@rhel55 ~$
```

14.6. time

The **time** command can display how long it takes to execute a command. The **date** command takes only a little time.

```
paul@rhel55 ~$ time date
Sat Apr 17 13:08:27 CEST 2010

real    0m0.014s
user    0m0.008s
sys     0m0.006s
```

The **sleep 5** command takes five **real** seconds to execute, but consumes little **cpu time**.

```
paul@rhel55 ~$ time sleep 5

real    0m5.018s
user    0m0.005s
sys     0m0.011s
```

This **bzip2** command compresses a file and uses a lot of **cpu time**.

```
paul@rhel55 ~$ time bzip2 text.txt

real    0m2.368s
user    0m0.847s
sys     0m0.539s
```

14.7. gzip - gunzip

Users never have enough disk space, so compression comes in handy. The **gzip** command can make files take up less space.

```
paul@rhel155 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
paul@rhel155 ~$ gzip text.txt
paul@rhel155 ~$ ls -lh text.txt.gz
-rw-rw-r-- 1 paul paul 760K Apr 17 13:11 text.txt.gz
```

You can get the original back with **gunzip**.

```
paul@rhel155 ~$ gunzip text.txt.gz
paul@rhel155 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
```

14.8. zcat - zmore

Text files that are compressed with **gzip** can be viewed with **zcat** and **zmore**.

```
paul@rhel155 ~$ head -4 text.txt
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
paul@rhel155 ~$ gzip text.txt
paul@rhel155 ~$ zcat text.txt.gz | head -4
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
```

14.9. bzip2 - bunzip2

Files can also be compressed with **bzip2** which takes a little more time than **gzip**, but compresses better.

```
paul@rhel155 ~$ bzip2 text.txt
paul@rhel155 ~$ ls -lh text.txt.bz2
-rw-rw-r-- 1 paul paul 569K Apr 17 13:11 text.txt.bz2
```

Files can be uncompressed again with **bunzip2**.

```
paul@rhel155 ~$ bunzip2 text.txt.bz2
paul@rhel155 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
```

14.10. bzip2 - bzip2

And in the same way **bzcat** and **bzmore** can display files compressed with **bzip2**.

```
paul@rhel155 ~$ bzip2 text.txt
paul@rhel155 ~$ bzcat text.txt.bz2 | head -4
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
```


14.11. solution: basic Unix tools

1. Explain the difference between these two commands. This question is very important. If you don't know the answer, then look back at the **shell** chapter.

```
find /data -name "*.txt"
```

```
find /data -name *.txt
```

When ***.txt** is quoted then the shell will not touch it. The **find** tool will look in the **/data** for all files ending in **.txt**.

When ***.txt** is not quoted then the shell might expand this (when one or more files that ends in **.txt** exist in the current directory). The **find** might show a different result, or can result in a syntax error.

2. Explain the difference between these two statements. Will they both work when there are 200 **.odf** files in **/data** ? How about when there are 2 million **.odf** files ?

```
find /data -name "*.odf" > data_odf.txt
```

```
find /data/*.odf > data_odf.txt
```

The first **find** will output all **.odf** filenames in **/data** and all subdirectories. The shell will redirect this to a file.

The second find will output all files named **.odf** in **/data** and will also output all files that exist in directories named ***.odf** (in **/data**).

With two million files the command line would be expanded beyond the maximum that the shell can accept. The last part of the command line would be lost.

3. Write a find command that finds all files created after January 30th 2010.

```
touch -t 201001302359 marker_date  
find . -type f -newer marker_date
```

There is another solution :

```
find . -type f -newerat "20100130 23:59:59"
```

4. Write a find command that finds all ***.odf** files created in September 2009.

```
touch -t 200908312359 marker_start  
touch -t 200910010000 marker_end  
find . -type f -name "*.odf" -newer marker_start ! -newer marker_end
```

The exclamation mark **! -newer** can be read as **not newer**.

5. Count the number of ***.conf** files in **/etc** and all its subdirs.

```
find /etc -type f -name '*.conf' | wc -l
```

6. Here are two commands that do the same thing: copy ***.odf** files to **/backup/** . What would be a reason to replace the first command with the second ? Again, this is an important question.

```
cp -r /data/*.odf /backup/
```

```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

The first might fail when there are too many files to fit on one command line.

7. Create a file called **loctest.txt**. Can you find this file with **locate** ? Why not ? How do you make locate find this file ?

You cannot locate this with **locate** because it is not yet in the index.

```
updatedb
```

8. Use find and -exec to rename all .htm files to .html.

```
paul@rhel55 ~$ find . -name '*.htm'
./one.htm
./two.htm
paul@rhel55 ~$ find . -name '*.htm' -exec mv {} {}1 \;
paul@rhel55 ~$ find . -name '*.htm*'
./one.html
./two.html
```

9. Issue the **date** command. Now display the date in YYYY/MM/DD format.

```
date +%Y/%m/%d
```

10. Issue the **cal** command. Display a calendar of 1582 and 1752. Notice anything special ?

```
cal 1582
```

The calendars are different depending on the country. Check <http://linux-training.be/files/studentfiles/dates.txt>

Chapter 15. regular expressions

Regular expressions are a very powerful tool in Linux. They can be used with a variety of programs like bash, vi, rename, grep, sed, and more.

This chapter introduces you to the basics of **regular expressions**.

15.1. regex versions

There are three different versions of regular expression syntax:

```
BRE: Basic Regular Expressions  
ERE: Extended Regular Expressions  
PRCE: Perl Regular Expressions
```

Depending on the tool being used, one or more of these syntaxes can be used.

For example the **grep** tool has the **-E** option to force a string to be read as ERE while **-G** forces BRE and **-P** forces PRCE.

Note that **grep** also has **-F** to force the string to be read literally.

The **sed** tool also has options to choose a regex syntax.

Read the manual of the tools you use!

15.2. grep

15.2.1. print lines matching a pattern

grep is a popular Linux tool to search for lines that match a certain pattern. Below are some examples of the simplest **regular expressions**.

This is the contents of the test file. This file contains three lines (or three **newline** characters).

```
paul@rhel65:~$ cat names
Tania
Laura
Valentina
```

When **grepping** for a single character, only the lines containing that character are returned.

```
paul@rhel65:~$ grep u names
Laura
paul@rhel65:~$ grep e names
Valentina
paul@rhel65:~$ grep i names
Tania
Valentina
```

The pattern matching in this example should be very straightforward; if the given character occurs on a line, then **grep** will return that line.

15.2.2. concatenating characters

Two concatenated characters will have to be concatenated in the same way to have a match.

This example demonstrates that **ia** will match **Tania** but not **Valentina** and **in** will match **Valentina** but not **Tania**.

```
paul@rhel65:~$ grep a names
Tania
Laura
Valentina
paul@rhel65:~$ grep ia names
Tania
paul@rhel65:~$ grep in names
Valentina
paul@rhel65:~$
```

15.2.3. one or the other

PRCE and ERE both use the pipe symbol to signify OR. In this example we **grep** for lines containing the letter i or the letter a.

```
paul@debian7:~$ cat list
Tania
Laura
paul@debian7:~$ grep -E 'i|a' list
Tania
Laura
```

Note that we use the **-E** switch of grep to force interpretation of our string as an ERE.

We need to **escape** the pipe symbol in a BRE to get the same logical OR.

```
paul@debian7:~$ grep -G 'i|a' list
paul@debian7:~$ grep -G 'i\\|a' list
Tania
Laura
```

15.2.4. one or more

The ***** signifies zero, one or more occurrences of the previous and the **+** signifies one or more of the previous.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o*' list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o+' list2
lol
lool
loool
paul@debian7:~$
```

15.2.5. match the end of a string

For the following examples, we will use this file.

```
paul@debian7:~$ cat names
Tania
Laura
Valentina
Fleur
Floor
```

The two examples below show how to use the **dollar character** to match the end of a string.

```
paul@debian7:~$ grep a$ names
Tania
Laura
Valentina
paul@debian7:~$ grep r$ names
Fleur
Floor
```

15.2.6. match the start of a string

The **caret character** (^) will match a string at the start (or the beginning) of a line.

Given the same file as above, here are two examples.

```
paul@debian7:~$ grep ^Val names
Valentina
paul@debian7:~$ grep ^F names
Fleur
Floor
```

Both the dollar sign and the little hat are called **anchors** in a regex.

15.2.7. separating words

Regular expressions use a **\b** sequence to reference a word separator. Take for example this file:

```
paul@debian7:~$ cat text
The governor is governing.
The winter is over.
Can you get over there?
```

Simply grepping for **over** will give too many results.

```
paul@debian7:~$ grep over text
The governor is governing.
The winter is over.
Can you get over there?
```

Surrounding the searched word with spaces is not a good solution (because other characters can be word separators). This screenshot below show how to use **\b** to find only the searched word:

```
paul@debian7:~$ grep '\bover\b' text
The winter is over.
Can you get over there?
paul@debian7:~$
```

Note that **grep** also has a **-w** option to grep for words.

```
paul@debian7:~$ cat text
The governor is governing.
The winter is over.
Can you get over there?
paul@debian7:~$ grep -w over text
The winter is over.
Can you get over there?
paul@debian7:~$
```

15.2.8. grep features

Sometimes it is easier to combine a simple regex with **grep** options, than it is to write a more complex regex. These options were discussed before:

```
grep -i
grep -v
grep -w
grep -A5
grep -B5
grep -C5
```

15.2.9. preventing shell expansion of a regex

The dollar sign is a special character, both for the regex and also for the shell (remember variables and embedded shells). Therefore it is advised to always quote the regex, this prevents shell expansion.

```
paul@debian7:~$ grep 'r$' names
Fleur
Floor
```

15.3. rename

15.3.1. the rename command

On Debian Linux the **/usr/bin/rename** command is a link to **/usr/bin/prename** installed by the **perl** package.

```
paul@pi ~ $ dpkg -S $(readlink -f $(which rename))
perl: /usr/bin/prename
```

Red Hat derived systems do not install the same **rename** command, so this section does not describe **rename** on Red Hat (unless you copy the perl script manually).

There is often confusion on the internet about the rename command because solutions that work fine in Debian (and Ubuntu, xubuntu, Mint, ...) cannot be used in Red Hat (and CentOS, Fedora, ...).

15.3.2. perl

The **rename** command is actually a perl script that uses **perl regular expressions**. The complete manual for these can be found by typing **perldoc perlrequick** (after installing **perldoc**).

```
root@pi:~# aptitude install perl-doc
The following NEW packages will be installed:
  perl-doc
0 packages upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 8,170 kB of archives. After unpacking 13.2 MB will be used.
Get: 1 http://mirrordirector.raspbian.org/raspbian/ wheezy/main perl-do...
Fetched 8,170 kB in 19s (412 kB/s)
Selecting previously unselected package perl-doc.
(Reading database ... 67121 files and directories currently installed.)
Unpacking perl-doc (from .../perl-doc_5.14.2-21+rpi2_all.deb) ...
Adding 'diversion of /usr/bin/perldoc to /usr/bin/perldoc.stub by perl-doc'
Processing triggers for man-db ...
Setting up perl-doc (5.14.2-21+rpi2) ...

root@pi:~# perldoc perlrequick
```

15.3.3. well known syntax

The most common use of the **rename** is to search for filenames matching a certain **string** and replacing this string with an **other string**.

This is often presented as **s/string/other string/** as seen in this example:

```
paul@pi ~ $ ls
abc      allfiles.TXT  bllfiles.TXT  Scratch      tennis2.TXT
abc.conf backup        cllfiles.TXT  temp.TXT     tennis.TXT
paul@pi ~ $ rename 's/TXT/text/' *
paul@pi ~ $ ls
abc      allfiles.text  bllfiles.text  Scratch      tennis2.text
abc.conf backup        cllfiles.text  temp.text    tennis.text
```

And here is another example that uses **rename** with the well know syntax to change the extensions of the same files once more:

```
paul@pi ~ $ ls
abc      allfiles.text  bllfiles.text  Scratch      tennis2.text
abc.conf backup        cllfiles.text  temp.text    tennis.text
paul@pi ~ $ rename 's/text/txt/' *.text
paul@pi ~ $ ls
abc      allfiles.txt  bllfiles.txt  Scratch      tennis2.txt
abc.conf backup        cllfiles.txt  temp.txt     tennis.txt
paul@pi ~ $
```

These two examples appear to work because the strings we used only exist at the end of the filename. Remember that file extensions have no meaning in the bash shell.

The next example shows what can go wrong with this syntax.

```
paul@pi ~ $ touch atxt.txt
paul@pi ~ $ rename 's/txt/problem/' atxt.txt
paul@pi ~ $ ls
abc      allfiles.txt  backup        cllfiles.txt  temp.txt     tennis.txt
abc.conf aproblem.txt  bllfiles.txt  Scratch      tennis2.txt
paul@pi ~ $
```

Only the first occurrence of the searched string is replaced.

15.3.4. a global replace

The syntax used in the previous example can be described as **s/regex/replacement/**. This is simple and straightforward, you enter a **regex** between the first two slashes and a **replacement string** between the last two.

This example expands this syntax only a little, by adding a **modifier**.

```
paul@pi ~ $ rename -n 's/TXT/txt/g' aTXT.TXT
aTXT.TXT renamed as atxt.txt
paul@pi ~ $
```

The syntax we use now can be described as **s/regex/replacement/g** where **s** signifies **switch** and **g** stands for **global**.

Note that this example used the **-n** switch to show what is being done (instead of actually renaming the file).

15.3.5. case insensitive replace

Another **modifier** that can be useful is **i**. this example shows how to replace a case insensitive string with another string.

```
paul@debian7:~/files$ ls
file1.text  file2.TEXT  file3.txt
paul@debian7:~/files$ rename 's/.text/.txt/i' *
paul@debian7:~/files$ ls
file1.txt  file2.txt  file3.txt
paul@debian7:~/files$
```

15.3.6. renaming extensions

Command line Linux has no knowledge of MS-DOS like extensions, but many end users and graphical application do use them.

Here is an example on how to use **rename** to only rename the file extension. It uses the dollar sign to mark the ending of the filename.

```
paul@pi ~ $ ls *.txt
allfiles.txt  bllfiles.txt  cllfiles.txt  really.txt.txt  temp.txt  tennis.txt
paul@pi ~ $ rename 's/.txt$/.TXT/' *.txt
paul@pi ~ $ ls *.TXT
allfiles.TXT  bllfiles.TXT  cllfiles.TXT  really.txt.TXT
temp.TXT      tennis.TXT
paul@pi ~ $
```

Note that the **dollar sign** in the regex means **at the end**. Without the dollar sign this command would fail on the really.txt.txt file.

15.4. sed

15.4.1. stream editor

The **stream editor** or short **sed** uses **regex** for stream editing.

In this example **sed** is used to replace a string.

```
echo Sunday | sed 's/Sun/Mon/'  
Monday
```

The slashes can be replaced by a couple of other characters, which can be handy in some cases to improve readability.

```
echo Sunday | sed 's:Sun:Mon:'  
Monday  
echo Sunday | sed 's_Sun_Mon_'  
Monday  
echo Sunday | sed 's|Sun|Mon|'  
Monday
```

15.4.2. interactive editor

While **sed** is meant to be used in a stream, it can also be used interactively on a file.

```
paul@debian7:~/files$ echo Sunday > today  
paul@debian7:~/files$ cat today  
Sunday  
paul@debian7:~/files$ sed -i 's/Sun/Mon/' today  
paul@debian7:~/files$ cat today  
Monday
```

15.4.3. simple back referencing

The **ampersand** character can be used to reference the searched (and found) string.

In this example the **ampersand** is used to double the occurrence of the found string.

```
echo Sunday | sed 's/Sun/&&/'  
SunSunday  
echo Sunday | sed 's/day/&&/'  
Sundayday
```

15.4.4. back referencing

Parentheses (often called round brackets) are used to group sections of the regex so they can later be referenced.

Consider this simple example:

```
paul@debian7:~$ echo Sunday | sed 's_\(Sun\)_\1ny_'  
Sunnyday  
paul@debian7:~$ echo Sunday | sed 's_\(Sun\)_\1ny \1_'  
Sunny Sunday
```

15.4.5. a dot for any character

In a **regex** a simple dot can signify any character.

```
paul@debian7:~$ echo 2014-04-01 | sed 's/....-.... /YYYY-MM-DD/'  
YYYY-MM-DD  
paul@debian7:~$ echo abcd-ef-gh | sed 's/....-.... /YYYY-MM-DD/'  
YYYY-MM-DD
```

15.4.6. multiple back referencing

When more than one pair of **parentheses** is used, each of them can be referenced separately by consecutive numbers.

```
paul@debian7:~$ echo 2014-04-01 | sed 's/\(...\) - \(...\) - \(...\) /\1+\2+\3/'  
2014+04+01  
paul@debian7:~$ echo 2014-04-01 | sed 's/\(....\) - \(...\) - \(...\) /\3:\2:\1/'  
01:04:2014
```

This feature is called **grouping**.

15.4.7. white space

The `\s` can refer to white space such as a space or a tab.

This example looks for white spaces (`\s`) globally and replaces them with 1 space.

```
paul@debian7:~$ echo -e 'today\tis\twarm'
today    is        warm
paul@debian7:~$ echo -e 'today\tis\twarm' | sed 's_\s_ _g'
today is warm
```

15.4.8. optional occurrence

A question mark signifies that the previous is **optional**.

The example below searches for three consecutive letter o, but the third o is optional.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'ooo?' list2
lool
loool
paul@debian7:~$ cat list2 | sed 's/ooo\?/A/'
ll
lol
lAl
lAl
```


15.4.9. exactly n times

You can demand an exact number of times the oprevious has to occur.

This example wants exactly three o's.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o{3}' list2
loool
paul@debian7:~$ cat list2 | sed 's/o\{3\}/A/'
ll
lol
lool
lAl
paul@debian7:~$
```

15.4.10. between n and m times

And here we demand exactly from minimum 2 to maximum 3 times.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o{2,3}' list2
lool
loool
paul@debian7:~$ grep 'o\{2,3\}' list2
lool
loool
paul@debian7:~$ cat list2 | sed 's/o\{2,3\}/A/'
ll
lol
lAl
lAl
paul@debian7:~$
```

15.5. bash history

The **bash shell** can also interpret some regular expressions.

This example shows how to manipulate the exclamation mark history feature of the bash shell.

```
paul@debian7:~$ mkdir hist
paul@debian7:~$ cd hist/
paul@debian7:~/hist$ touch file1 file2 file3
paul@debian7:~/hist$ ls -l file1
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file1
paul@debian7:~/hist$ !l
ls -l file1
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file1
paul@debian7:~/hist$ !l:s/1/3
ls -l file3
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file3
paul@debian7:~/hist$
```

This also works with the history numbers in bash.

```
paul@debian7:~/hist$ history 6
2089  mkdir hist
2090  cd hist/
2091  touch file1 file2 file3
2092  ls -l file1
2093  ls -l file3
2094  history 6
paul@debian7:~/hist$ !2092
ls -l file1
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file1
paul@debian7:~/hist$ !2092:s/1/2
ls -l file2
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file2
paul@debian7:~/hist$
```

Chapter 16. scripting introduction

Shells like **bash** and **Korn** have support for programming constructs that can be saved as **scripts**. These **scripts** in turn then become more **shell** commands. Many Linux commands are **scripts**. **User profile scripts** are run when a user logs on and **init scripts** are run when a **daemon** is stopped or started.

This means that system administrators also need basic knowledge of **scripting** to understand how their servers and their applications are started, updated, upgraded, patched, maintained, configured and removed, and also to understand how a user environment is built.

The goal of this chapter is to give you enough information to be able to read and understand scripts. Not to become a writer of complex scripts.

16.1. prerequisites

You should have read and understood **part III shell expansion** and **part IV pipes and commands** before starting this chapter.

16.2. hello world

Just like in every programming course, we start with a simple **hello_world** script. The following script will output **Hello World**.

```
echo Hello World
```

After creating this simple script in **vi** or with **echo**, you'll have to **chmod +x hello_world** to make it executable. And unless you add the scripts directory to your path, you'll have to type the path to the script for the shell to be able to find it.

```
[paul@RHEL4a ~]$ echo echo Hello World > hello_world
[paul@RHEL4a ~]$ chmod +x hello_world
[paul@RHEL4a ~]$ ./hello_world
Hello World
[paul@RHEL4a ~]$
```

16.3. she-bang

Let's expand our example a little further by putting **#!/bin/bash** on the first line of the script. The **#!** is called a **she-bang** (sometimes called **sha-bang**), where the **she-bang** is the first two characters of the script.

```
#!/bin/bash
echo Hello World
```

You can never be sure which shell a user is running. A script that works flawlessly in **bash** might not work in **ksh**, **cs****h**, or **dash**. To instruct a shell to run your script in a certain shell, you can start your script with a **she-bang** followed by the shell it is supposed to run in. This script will run in a bash shell.

```
#!/bin/bash
echo -n hello
echo A bash subshell `echo -n hello`
```

This script will run in a Korn shell (unless **/bin/ksh** is a hard link to **/bin/bash**). The **/etc/shells** file contains a list of shells on your system.

```
#!/bin/ksh
echo -n hello
echo a Korn subshell `echo -n hello`
```

16.4. comment

Let's expand our example a little further by adding comment lines.

```
#!/bin/bash
#
# Hello World Script
#
echo Hello World
```

16.5. variables

Here is a simple example of a variable inside a script.

```
#!/bin/bash
#
# simple variable in script
#
var1=4
echo var1 = $var1
```

Scripts can contain variables, but since scripts are run in their own shell, the variables do not survive the end of the script.

```
[paul@RHEL4a ~]$ echo $var1

[paul@RHEL4a ~]$ ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1

[paul@RHEL4a ~]$
```

16.6. sourcing a script

Luckily, you can force a script to run in the same shell; this is called **sourcing** a script.

```
[paul@RHEL4a ~]$ source ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1
4
[paul@RHEL4a ~]$
```

The above is identical to the below.

```
[paul@RHEL4a ~]$ . ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1
4
[paul@RHEL4a ~]$
```

16.7. troubleshooting a script

Another way to run a script in a separate shell is by typing **bash** with the name of the script as a parameter.

```
paul@debian6~/test$ bash runme
42
```

Expanding this to **bash -x** allows you to see the commands that the shell is executing (after shell expansion).

```
paul@debian6~/test$ bash -x runme
+ var4=42
+ echo 42
42
paul@debian6~/test$ cat runme
# the runme script
var4=42
echo $var4
paul@debian6~/test$
```

Notice the absence of the commented (#) line, and the replacement of the variable before execution of **echo**.

16.8. prevent setuid root spoofing

Some user may try to perform **setuid** based script **root spoofing**. This is a rare but possible attack. To improve script security and to avoid interpreter spoofing, you need to add **--** after the **#!/bin/bash**, which disables further option processing so the shell will not accept any options.

```
#!/bin/bash -
or
#!/bin/bash --
```

Any arguments after the **--** are treated as filenames and arguments. An argument of **-** is equivalent to **--**.

16.9. solution: introduction to scripting

0. Give each script a different name, keep them for later!

1. Write a script that outputs the name of a city.

```
$ echo 'echo Antwerp' > first.bash
$ chmod +x first.bash
$ ./first.bash
Antwerp
```

2. Make sure the script runs in the bash shell.

```
$ cat first.bash
#!/bin/bash
echo Antwerp
```

3. Make sure the script runs in the Korn shell.

```
$ cat first.bash
#!/bin/ksh
echo Antwerp
```

Note that while first.bash will technically work as a Korn shell script, the name ending in .bash is confusing.

4. Create a script that defines two variables, and outputs their value.

```
$ cat second.bash
#!/bin/bash

var33=300
var42=400

echo $var33 $var42
```

5. The previous script does not influence your current shell (the variables do not exist outside of the script). Now run the script so that it influences your current shell.

```
source second.bash
```

6. Is there a shorter way to **source** the script ?

```
. ./second.bash
```

7. Comment your scripts so that you know what they are doing.

```
$ cat second.bash
#!/bin/bash
# script to test variables and sourcing

# define two variables
var33=300
var42=400

# output the value of these variables
echo $var33 $var42
```

Chapter 17. scripting loops

17.1. test []

The **test** command can test whether something is true or false. Let's start by testing whether 10 is greater than 55.

```
[paul@RHEL4b ~]$ test 10 -gt 55 ; echo $?  
1  
[paul@RHEL4b ~]$
```

The test command returns 1 if the test fails. And as you see in the next screenshot, test returns 0 when a test succeeds.

```
[paul@RHEL4b ~]$ test 56 -gt 55 ; echo $?  
0  
[paul@RHEL4b ~]$
```

If you prefer true and false, then write the test like this.

```
[paul@RHEL4b ~]$ test 56 -gt 55 && echo true || echo false  
true  
[paul@RHEL4b ~]$ test 6 -gt 55 && echo true || echo false  
false
```

The test command can also be written as square brackets, the screenshot below is identical to the one above.

```
[paul@RHEL4b ~]$ [ 56 -gt 55 ] && echo true || echo false  
true  
[paul@RHEL4b ~]$ [ 6 -gt 55 ] && echo true || echo false  
false
```

Below are some example tests. Take a look at **man test** to see more options for tests.

[-d foo]	Does the directory foo exist ?
[-e bar]	Does the file bar exist ?
['/etc' = \$PWD]	Is the string /etc equal to the variable \$PWD ?
[\$1 != 'secret']	Is the first parameter different from secret ?
[55 -lt \$bar]	Is 55 less than the value of \$bar ?
[\$foo -ge 1000]	Is the value of \$foo greater or equal to 1000 ?
["abc" < \$bar]	Does abc sort before the value of \$bar ?
[-f foo]	Is foo a regular file ?
[-r bar]	Is bar a readable file ?
[foo -nt bar]	Is file foo newer than file bar ?
[-o nounset]	Is the shell option nounset set ?

Tests can be combined with logical AND and OR.

```
paul@RHEL4b:~$ [ 66 -gt 55 -a 66 -lt 500 ] && echo true || echo false  
true  
paul@RHEL4b:~$ [ 66 -gt 55 -a 660 -lt 500 ] && echo true || echo false  
false  
paul@RHEL4b:~$ [ 66 -gt 55 -o 660 -lt 500 ] && echo true || echo false  
true
```

17.2. if then else

The **if then else** construction is about choice. If a certain condition is met, then execute something, else execute something else. The example below tests whether a file exists, and if the file exists then a proper message is echoed.

```
#!/bin/bash

if [ -f isit.txt ]
then echo isit.txt exists!
else echo isit.txt not found!
fi
```

If we name the above script 'choice', then it executes like this.

```
[paul@RHEL4a scripts]$ ./choice
isit.txt not found!
[paul@RHEL4a scripts]$ touch isit.txt
[paul@RHEL4a scripts]$ ./choice
isit.txt exists!
[paul@RHEL4a scripts]$
```

17.3. if then elif

You can nest a new **if** inside an **else** with **elif**. This is a simple example.

```
#!/bin/bash
count=42
if [ $count -eq 42 ]
then
    echo "42 is correct."
elif [ $count -gt 42 ]
then
    echo "Too much."
else
    echo "Not enough."
fi
```

17.4. for loop

The example below shows the syntax of a classical **for loop** in bash.

```
for i in 1 2 4
do
    echo $i
done
```

An example of a **for loop** combined with an embedded shell.

```
#!/bin/ksh
for counter in `seq 1 20`
do
    echo counting from 1 to 20, now at $counter
    sleep 1
done
```

The same example as above can be written without the embedded shell using the bash **{from..to}** shorthand.

```
#!/bin/bash
for counter in {1..20}
do
    echo counting from 1 to 20, now at $counter
    sleep 1
done
```

This **for loop** uses file globbing (from the shell expansion). Putting the instruction on the command line has identical functionality.

```
kahlan@solexp11$ ls
count.ksh  go.ksh
kahlan@solexp11$ for file in *.ksh ; do cp $file $file.backup ; done
kahlan@solexp11$ ls
count.ksh  count.ksh.backup  go.ksh  go.ksh.backup
```

17.5. while loop

Below a simple example of a **while loop**.

```
i=100;
while [ $i -ge 0 ] ;
do
    echo Counting down, from 100 to 0, now at $i;
    let i--;
done
```

Endless loops can be made with **while true** or **while :**, where the **colon** is the equivalent of **no operation** in the **Korn** and **bash** shells.

```
#!/bin/ksh
# endless loop
while :
do
    echo hello
    sleep 1
done
```

17.6. until loop

Below a simple example of an **until loop**.

```
let i=100;
until [ $i -le 0 ] ;
do
    echo Counting down, from 100 to 1, now at $i;
    let i--;
done
```

17.7. solution: scripting tests and loops

1. Write a script that uses a **for** loop to count from 3 to 7.

```
#!/bin/bash

for i in 3 4 5 6 7
do
    echo Counting from 3 to 7, now at $i
done
```

2. Write a script that uses a **for** loop to count from 1 to 17000.

```
#!/bin/bash

for i in `seq 1 17000`
do
    echo Counting from 1 to 17000, now at $i
done
```

3. Write a script that uses a **while** loop to count from 3 to 7.

```
#!/bin/bash

i=3
while [ $i -le 7 ]
do
    echo Counting from 3 to 7, now at $i
    let i=i+1
done
```

4. Write a script that uses an **until** loop to count down from 8 to 4.

```
#!/bin/bash

i=8
until [ $i -lt 4 ]
do
    echo Counting down from 8 to 4, now at $i
    let i=i-1
done
```

5. Write a script that counts the number of files ending in **.txt** in the current directory.

```
#!/bin/bash

let i=0
for file in *.txt
do
    let i++
done
echo "There are $i files ending in .txt"
```

6. Wrap an **if** statement around the script so it is also correct when there are zero files ending in **.txt**.

```
#!/bin/bash

ls *.txt > /dev/null 2>&1
if [ $? -ne 0 ]
```

```
then echo "There are 0 files ending in .txt"
else
  let i=0
  for file in *.txt
  do
    let i++
  done
  echo "There are $i files ending in .txt"
fi
```

Chapter 18. scripting parameters

18.1. script parameters

A **bash** shell script can have parameters. The numbering you see in the script below continues if you have more parameters. You also have special parameters containing the number of parameters, a string of all of them, and also the process id, and the last return code. The man page of **bash** has a full list.

```
#!/bin/bash
echo The first argument is $1
echo The second argument is $2
echo The third argument is $3

echo \$ $$  PID of the script
echo \# $#  count arguments
echo \? $?  last return code
echo \* $*  all the arguments
```

Below is the output of the script above in action.

```
[paul@RHEL4a scripts]$ ./pars one two three
The first argument is one
The second argument is two
The third argument is three
$ 5610 PID of the script
# 3 count arguments
? 0 last return code
* one two three all the arguments
```

Once more the same script, but with only two parameters.

```
[paul@RHEL4a scripts]$ ./pars 1 2
The first argument is 1
The second argument is 2
The third argument is
$ 5612 PID of the script
# 2 count arguments
? 0 last return code
* 1 2 all the arguments
[paul@RHEL4a scripts]$
```

Here is another example, where we use **\$0**. The **\$0** parameter contains the name of the script.

```
paul@debian6~$ cat myname
echo this script is called $0
paul@debian6~$ ./myname
this script is called ./myname
paul@debian6~$ mv myname test42
paul@debian6~$ ./test42
this script is called ./test42
```

18.2. shift through parameters

The **shift** statement can parse all **parameters** one by one. This is a sample script.

```
kahlan@solexp11$ cat shift.ksh
#!/bin/ksh

if [ "$#" == "0" ]
then
    echo You have to give at least one parameter.
    exit 1
fi

while (( $# ))
do
    echo You gave me $1
    shift
done
```

Below is some sample output of the script above.

```
kahlan@solexp11$ ./shift.ksh one
You gave me one
kahlan@solexp11$ ./shift.ksh one two three 1201 "33 42"
You gave me one
You gave me two
You gave me three
You gave me 1201
You gave me 33 42
kahlan@solexp11$ ./shift.ksh
You have to give at least one parameter.
```

18.3. runtime input

You can ask the user for input with the **read** command in a script.

```
#!/bin/bash
echo -n Enter a number:
read number
```

18.4. sourcing a config file

The **source** (as seen in the shell chapters) can be used to source a configuration file.

Below a sample configuration file for an application.

```
[paul@RHEL4a scripts]$ cat myApp.conf
# The config file of myApp

# Enter the path here
myAppPath=/var/myApp

# Enter the number of quines here
quines=5
```

And here an application that uses this file.

```
[paul@RHEL4a scripts]$ cat myApp.bash
#!/bin/bash
#
# Welcome to the myApp application
#

. ./myApp.conf

echo There are $quines quines
```

The running application can use the values inside the sourced configuration file.

```
[paul@RHEL4a scripts]$ ./myApp.bash
There are 5 quines
[paul@RHEL4a scripts]$
```

18.5. get script options with getopt

The **getopts** function allows you to parse options given to a command. The following script allows for any combination of the options a, f and z.

```
kahlan@solexp11$ cat options.ksh
#!/bin/ksh

while getopts ":afz" option;
do
  case $option in
    a)
      echo received -a
      ;;
    f)
      echo received -f
      ;;
    z)
      echo received -z
      ;;
    *)
      echo "invalid option -$OPTARG"
      ;;
  esac
done
```

This is sample output from the script above. First we use correct options, then we enter twice an invalid option.

```
kahlan@solexp11$ ./options.ksh
kahlan@solexp11$ ./options.ksh -af
received -a
received -f
kahlan@solexp11$ ./options.ksh -zfg
received -z
received -f
invalid option -g
kahlan@solexp11$ ./options.ksh -a -b -z
received -a
invalid option -b
received -z
```

You can also check for options that need an argument, as this example shows.

```
kahlan@solexp11$ cat argoptions.ksh
#!/bin/ksh

while getopts ":af:z" option;
do
    case $option in
        a)
            echo received -a
            ;;
        f)
            echo received -f with $OPTARG
            ;;
        z)
            echo received -z
            ;;
        :)
            echo "option -$OPTARG needs an argument"
            ;;
        *)
            echo "invalid option -$OPTARG"
            ;;
    esac
done
```

This is sample output from the script above.

```
kahlan@solexp11$ ./argoptions.ksh -a -f hello -z
received -a
received -f with hello
received -z
kahlan@solexp11$ ./argoptions.ksh -zaf 42
received -z
received -a
received -f with 42
kahlan@solexp11$ ./argoptions.ksh -zf
received -z
option -f needs an argument
```

18.6. get shell options with shopt

You can toggle the values of variables controlling optional shell behaviour with the **shopt** built-in shell command. The example below first verifies whether the `cdspell` option is set; it is not. The next `shopt` command sets the value, and the third `shopt` command verifies that the option really is set. You can now use minor spelling mistakes in the `cd` command. The man page of `bash` has a complete list of options.

```
paul@laika:~$ shopt -q cdspell ; echo $?
1
paul@laika:~$ shopt -s cdspell
paul@laika:~$ shopt -q cdspell ; echo $?
0
paul@laika:~$ cd /Etc
/etc
```

18.7. solution: parameters and options

1. Write a script that receives four parameters, and outputs them in reverse order.

```
echo $4 $3 $2 $1
```

2. Write a script that receives two parameters (two filenames) and outputs whether those files exist.

```
#!/bin/bash

if [ -f $1 ]
then echo $1 exists!
else echo $1 not found!
fi

if [ -f $2 ]
then echo $2 exists!
else echo $2 not found!
fi
```

3. Write a script that asks for a filename. Verify existence of the file, then verify that you own the file, and whether it is writable. If not, then make it writable.
4. Make a configuration file for the previous script. Put a logging switch in the config file, logging means writing detailed output of everything the script does to a log file in /tmp.

Chapter 19. more scripting

19.1. eval

eval reads arguments as input to the shell (the resulting commands are executed). This allows using the value of a variable as a variable.

```
paul@deb503:~/test42$ answer=42
paul@deb503:~/test42$ word=answer
paul@deb503:~/test42$ eval x=\$$word ; echo $x
42
```

Both in **bash** and **Korn** the arguments can be quoted.

```
kahlan@solexp11$ answer=42
kahlan@solexp11$ word=answer
kahlan@solexp11$ eval "y=\$$word" ; echo $y
42
```

Sometimes the **eval** is needed to have correct parsing of arguments. Consider this example where the **date** command receives one parameter **1 week ago**.

```
paul@debian6~$ date --date="1 week ago"
Thu Mar  8 21:36:25 CET 2012
```

When we set this command in a variable, then executing that variable fails unless we use **eval**.

```
paul@debian6~$ lastweek='date --date="1 week ago"'
paul@debian6~$ $lastweek
date: extra operand `ago'
Try `date --help' for more information.
paul@debian6~$ eval $lastweek
Thu Mar  8 21:36:39 CET 2012
```

19.2. (())

The **(())** allows for evaluation of numerical expressions.

```
paul@deb503:~/test42$ (( 42 > 33 )) && echo true || echo false
true
paul@deb503:~/test42$ (( 42 > 1201 )) && echo true || echo false
false
paul@deb503:~/test42$ var42=42
paul@deb503:~/test42$ (( 42 == var42 )) && echo true || echo false
true
paul@deb503:~/test42$ (( 42 == $var42 )) && echo true || echo false
true
paul@deb503:~/test42$ var42=33
paul@deb503:~/test42$ (( 42 == var42 )) && echo true || echo false
false
```

19.3. let

The **let** built-in shell function instructs the shell to perform an evaluation of arithmetic expressions. It will return 0 unless the last arithmetic expression evaluates to 0.

```
[paul@RHEL4b ~]$ let x="3 + 4" ; echo $x
7
[paul@RHEL4b ~]$ let x="10 + 100/10" ; echo $x
20
[paul@RHEL4b ~]$ let x="10-2+100/10" ; echo $x
18
[paul@RHEL4b ~]$ let x="10*2+100/10" ; echo $x
30
```

The **shell** can also convert between different bases.

```
[paul@RHEL4b ~]$ let x="0xFF" ; echo $x
255
[paul@RHEL4b ~]$ let x="0xC0" ; echo $x
192
[paul@RHEL4b ~]$ let x="0xA8" ; echo $x
168
[paul@RHEL4b ~]$ let x="8#70" ; echo $x
56
[paul@RHEL4b ~]$ let x="8#77" ; echo $x
63
[paul@RHEL4b ~]$ let x="16#c0" ; echo $x
192
```

There is a difference between assigning a variable directly, or using **let** to evaluate the arithmetic expressions (even if it is just assigning a value).

```
kahlan@solexp11$ dec=15 ; oct=017 ; hex=0x0f
kahlan@solexp11$ echo $dec $oct $hex
15 017 0x0f
kahlan@solexp11$ let dec=15 ; let oct=017 ; let hex=0x0f
kahlan@solexp11$ echo $dec $oct $hex
15 15 15
```

19.4. case

You can sometimes simplify nested if statements with a **case** construct.

```
[paul@RHEL4b ~]$ ./help
What animal did you see ? lion
You better start running fast!
[paul@RHEL4b ~]$ ./help
What animal did you see ? dog
Don't worry, give it a cookie.
[paul@RHEL4b ~]$ cat help
#!/bin/bash
#
# Wild Animals Helpdesk Advice
#
echo -n "What animal did you see ? "
read animal
case $animal in
    "lion" | "tiger")
        echo "You better start running fast!"
        ;;
    "cat")
        echo "Let that mouse go..."
        ;;
    "dog")
        echo "Don't worry, give it a cookie."
        ;;
    "chicken" | "goose" | "duck" )
        echo "Eggs for breakfast!"
        ;;
    "liger")
        echo "Approach and say 'Ah you big fluffy kitty...'"
        ;;
    "babelfish")
        echo "Did it fall out your ear ?"
        ;;
    *)
        echo "You discovered an unknown animal, name it!"
        ;;
esac
[paul@RHEL4b ~]$
```

19.5. shell functions

Shell **functions** can be used to group commands in a logical way.

```
kahlan@solexp11$ cat funcs.ksh
#!/bin/ksh

function greetings {
echo Hello World!
echo and hello to $USER to!
}

echo We will now call a function
greetings
echo The end
```

This is sample output from this script with a **function**.

```
kahlan@solexp11$ ./funcs.ksh
We will now call a function
Hello World!
and hello to kahlan to!
The end
```

A shell function can also receive parameters.

```
kahlan@solexp11$ cat addfunc.ksh
#!/bin/ksh

function plus {
let result="$1 + $2"
echo $1 + $2 = $result
}

plus 3 10
plus 20 13
plus 20 22
```

This script produces the following output.

```
kahlan@solexp11$ ./addfunc.ksh
3 + 10 = 13
20 + 13 = 33
20 + 22 = 42
```

19.6. solution : more scripting

1. Write a script that asks for two numbers, and outputs the sum and product (as shown here).

```
Enter a number: 5
Enter another number: 2

Sum:      5 + 2 = 7
Product:   5 x 2 = 10
```

```
#!/bin/bash

echo -n "Enter a number : "
read n1

echo -n "Enter another number : "
read n2

let sum="$n1+$n2"
let pro="$n1*$n2"

echo -e "Sum\t: $n1 + $n2 = $sum"
echo -e "Product\t: $n1 * $n2 = $pro"
```

2. Improve the previous script to test that the numbers are between 1 and 100, exit with an error if necessary.

```
echo -n "Enter a number between 1 and 100 : "
read n1

if [ $n1 -lt 1 -o $n1 -gt 100 ]
then
    echo Wrong number...
    exit 1
fi
```

3. Improve the previous script to congratulate the user if the sum equals the product.

```
if [ $sum -eq $pro ]
then echo Congratulations $sum == $pro
fi
```

4. Write a script with a case insensitive case statement, using the shopt nocasematch option. The nocasematch option is reset to the value it had before the scripts started.

```
#!/bin/bash
#
# Wild Animals Case Insensitive Helpdesk Advice
#

if shopt -q nocasematch; then
    nocase=yes;
else
    nocase=no;
    shopt -s nocasematch;
fi

echo -n "What animal did you see ? "
read animal

case $animal in
```

```

"lion" | "tiger")
    echo "You better start running fast!"
;;
"cat")
    echo "Let that mouse go..."
;;
"dog")
    echo "Don't worry, give it a cookie."
;;
"chicken" | "goose" | "duck" )
    echo "Eggs for breakfast!"
;;
"liger")
    echo "Approach and say 'Ah you big fluffy kitty.'"
;;
"babelefish")
    echo "Did it fall out your ear ?"
;;
*)
    echo "You discovered an unknown animal, name it!"
;;
esac

if [ nocase = yes ] ; then
    shopt -s nocasematch;
else
    shopt -u nocasematch;
fi

```

5. If time permits (or if you are waiting for other students to finish this practice), take a look at Linux system scripts in /etc/init.d and /etc/rc.d and try to understand them. Where does execution of a script start in /etc/init.d/samba ? There are also some hidden scripts in ~, we

Chapter 20. introduction to users

This little chapter will teach you how to identify your user account on a Unix computer using commands like **who am i**, **id**, and more.

In a second part you will learn how to become another user with the **su** command.

And you will learn how to run a program as another user with **sudo**.

20.1. whoami

The **whoami** command tells you your username.

```
[paul@centos7 ~]$ whoami
paul
[paul@centos7 ~]$
```

20.2. who

The **who** command will give you information about who is logged on the system.

```
[paul@centos7 ~]$ who
root      pts/0      2014-10-10 23:07 (10.104.33.101)
paul      pts/1      2014-10-10 23:30 (10.104.33.101)
laura     pts/2      2014-10-10 23:34 (10.104.33.96)
tania     pts/3      2014-10-10 23:39 (10.104.33.91)
[paul@centos7 ~]$
```

20.3. who am i

With **who am i** the **who** command will display only the line pointing to your current session.

```
[paul@centos7 ~]$ who am i
paul      pts/1      2014-10-10 23:30 (10.104.33.101)
[paul@centos7 ~]$
```

20.4. w

The **w** command shows you who is logged on and what they are doing.

```
[paul@centos7 ~]$ w
 23:34:07 up 31 min,  2 users,  load average: 0.00, 0.01, 0.02
USER      TTY      LOGIN@  IDLE   JCPU   PCPU WHAT
root      pts/0    23:07   15.00s  0.01s  0.01s top
paul      pts/1    23:30    7.00s  0.00s  0.00s w
[paul@centos7 ~]$
```

20.5. id

The **id** command will give you your user id, primary group id, and a list of the groups that you belong to.

```
paul@debian7:~$ id
uid=1000(paul) gid=1000(paul) groups=1000(paul)
```

On RHEL/CentOS you will also get **SELinux** context information with this command.

```
[root@centos7 ~]# id
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r\
:unconfined_t:s0-s0:c0.c1023
```


20.6. su to another user

The **su** command allows a user to run a shell as another user.

```
laura@debian7:~$ su tania
Password:
tania@debian7:/home/laura$
```

20.7. su to root

Yes you can also **su** to become **root**, when you know the **root** password.

```
laura@debian7:~$ su root
Password:
root@debian7:/home/laura#
```

20.8. su as root

You need to know the password of the user you want to substitute to, unless your are logged in as **root**. The **root** user can become any existing user without knowing that user's password.

```
root@debian7:~# id
uid=0(root) gid=0(root) groups=0(root)
root@debian7:~# su - valentina
valentina@debian7:~$
```

20.9. su - \$username

By default, the **su** command maintains the same shell environment. To become another user and also get the target user's environment, issue the **su -** command followed by the target username.

```
root@debian7:~# su laura
laura@debian7:/root$ exit
exit
root@debian7:~# su - laura
laura@debian7:~$ pwd
/home/laura
```

20.10. su -

When no username is provided to **su** or **su -**, the command will assume **root** is the target.

```
tania@debian7:~$ su -
Password:
root@debian7:~#
```

20.11. run a program as another user

The `sudo` program allows a user to start a program with the credentials of another user. Before this works, the system administrator has to set up the `/etc/sudoers` file. This can be useful to delegate administrative tasks to another user (without giving the root password).

The screenshot below shows the usage of **sudo**. User **paul** received the right to run **useradd** with the credentials of **root**. This allows **paul** to create new users on the system without becoming **root** and without knowing the **root password**.

First the command fails for **paul**.

```
paul@debian7:~$ /usr/sbin/useradd -m valentina
useradd: Permission denied.
useradd: cannot lock /etc/passwd; try again later.
```

But with **sudo** it works.

```
paul@debian7:~$ sudo /usr/sbin/useradd -m valentina
[sudo] password for paul:
paul@debian7:~$
```

20.12. visudo

Check the man page of **visudo** before playing with the `/etc/sudoers` file. Editing the **sudoers** is out of scope for this fundamentals book.

```
paul@rhel65:~$ apropos visudo
visudo                (8) - edit the sudoers file
paul@rhel65:~$
```

20.13. sudo su -

On some Linux systems like Ubuntu and Xubuntu, the **root** user does not have a password set. This means that it is not possible to login as **root** (extra security). To perform tasks as **root**, the first user is given all **sudo rights** via the **/etc/sudoers**. In fact all users that are members of the admin group can use sudo to run all commands as root.

```
root@laika:~# grep admin /etc/sudoers
# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL
```

The end result of this is that the user can type **sudo su -** and become root without having to enter the root password. The sudo command does require you to enter your own password. Thus the password prompt in the screenshot below is for sudo, not for su.

```
paul@laika:~$ sudo su -
Password:
root@laika:~#
```

20.14. sudo logging

Using **sudo** without authorization will result in a severe warning:

```
paul@rhel65:~$ sudo su -

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for paul:
paul is not in the sudoers file. This incident will be reported.
paul@rhel65:~$
```

The root user can see this in the **/var/log/secure** on Red Hat and in **/var/log/auth.log** on Debian).

```
root@rhel65:~# tail /var/log/secure | grep sudo | tr -s ' '
Apr 13 16:03:42 rhel65 sudo: paul : user NOT in sudoers ; TTY=pts/0 ; PWD=\
/home/paul ; USER=root ; COMMAND=/bin/su -
root@rhel65:~#
```

20.15. solution: introduction to users

1. Run a command that displays only your currently logged on user name.

```
laura@debian7:~$ whoami
laura
laura@debian7:~$ echo $USER
laura
```

2. Display a list of all logged on users.

```
laura@debian7:~$ who
laura      pts/0      2014-10-13 07:22 (10.104.33.101)
laura@debian7:~$
```

3. Display a list of all logged on users including the command they are running at this very moment.

```
laura@debian7:~$ w
07:47:02 up 16 min,  2 users,  load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@      IDLE   JCPU   PCPU WHAT
root      pts/0    10.104.33.101  07:30      6.00s   0.04s   0.00s w
root      pts/1    10.104.33.101  07:46      6.00s   0.01s   0.00s sleep 42
laura@debian7:~$
```

4. Display your user name and your unique user identification (userid).

```
laura@debian7:~$ id
uid=1005(laura) gid=1007(laura) groups=1007(laura)
laura@debian7:~$
```

5. Use **su** to switch to another user account (unless you are root, you will need the password of the other account). And get back to the previous account.

```
laura@debian7:~$ su tania
Password:
tania@debian7:/home/laura$ id
uid=1006(tania) gid=1008(tania) groups=1008(tania)
tania@debian7:/home/laura$ exit
laura@debian7:~$
```

6. Now use **su -** to switch to another user and notice the difference.

```
laura@debian7:~$ su - tania
Password:
tania@debian7:~$ pwd
/home/tania
tania@debian7:~$ logout
laura@debian7:~$
```

Note that **su -** gets you into the home directory of **Tania**.

7. Try to create a new user account (when using your normal user account). this should fail. (Details on adding user accounts are explained in the next chapter.)

```
laura@debian7:~$ useradd valentina
-su: useradd: command not found
laura@debian7:~$ /usr/sbin/useradd valentina
useradd: Permission denied.
useradd: cannot lock /etc/passwd; try again later.
```

It is possible that **useradd** is located in **/sbin/useradd** on your computer.

8. Now try the same, but with **sudo** before your command.

```
laura@debian7:~$ sudo /usr/sbin/useradd valentina
[sudo] password for laura:
laura is not in the sudoers file.  This incident will be reported.
laura@debian7:~$
```

Notice that **laura** has no permission to use the **sudo** on this system.

Chapter 21. user management

This chapter will teach you how to use **useradd**, **usermod** and **userdel** to create, modify and remove user accounts.

You will need **root** access on a Linux computer to complete this chapter.

21.1. user management

User management on Linux can be done in three complementary ways. You can use the **graphical** tools provided by your distribution. These tools have a look and feel that depends on the distribution. If you are a novice Linux user on your home system, then use the graphical tool that is provided by your distribution. This will make sure that you do not run into problems.

Another option is to use **command line tools** like `useradd`, `usermod`, `gpasswd`, `passwd` and others. Server administrators are likely to use these tools, since they are familiar and very similar across many different distributions. This chapter will focus on these command line tools.

A third and rather extremist way is to **edit the local configuration files** directly using `vi` (or `vim`). Do not attempt this as a novice on production systems!

21.2. /etc/passwd

The local user database on Linux (and on most Unixes) is `/etc/passwd`.

```
[root@RHEL5 ~]# tail /etc/passwd
inge:x:518:524:art dealer:/home/inge:/bin/ksh
ann:x:519:525:flute player:/home/ann:/bin/bash
frederik:x:520:526:rubius poet:/home/frederik:/bin/bash
steven:x:521:527:roman emperor:/home/steven:/bin/bash
pascale:x:522:528:artist:/home/pascale:/bin/ksh
geert:x:524:530:kernel developer:/home/geert:/bin/bash
wim:x:525:531:master damuti:/home/wim:/bin/bash
sandra:x:526:532:radish stresser:/home/sandra:/bin/bash
annelies:x:527:533:sword fighter:/home/annelies:/bin/bash
laura:x:528:534:art dealer:/home/laura:/bin/ksh
```

As you can see, this file contains seven columns separated by a colon. The columns contain the username, an x, the user id, the primary group id, a description, the name of the home directory, and the login shell.

More information can be found by typing `man 5 passwd`.

```
[root@RHEL5 ~]# man 5 passwd
```

21.3. root

The **root** user also called the **superuser** is the most powerful account on your Linux system. This user can do almost anything, including the creation of other users. The root user always has `userid 0` (regardless of the name of the account).

```
[root@RHEL5 ~]# head -1 /etc/passwd
root:x:0:0:root:/root:/bin/bash
```


21.4. useradd

You can add users with the **useradd** command. The example below shows how to add a user named yanina (last parameter) and at the same time forcing the creation of the home directory (-m), setting the name of the home directory (-d), and setting a description (-c).

```
[root@RHEL5 ~]# useradd -m -d /home/yanina -c "yanina wickmayer" yanina
[root@RHEL5 ~]# tail -1 /etc/passwd
yanina:x:529:529:yanina wickmayer:/home/yanina:/bin/bash
```

The user named yanina received userid 529 and **primary group** id 529.

21.5. /etc/default/useradd

Both Red Hat Enterprise Linux and Debian/Ubuntu have a file called **/etc/default/useradd** that contains some default user options. Besides using cat to display this file, you can also use **useradd -D**.

```
[root@RHEL4 ~]# useradd -D
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
```

21.6. userdel

You can delete the user yanina with **userdel**. The -r option of userdel will also remove the home directory.

```
[root@RHEL5 ~]# userdel -r yanina
```

21.7. usermod

You can modify the properties of a user with the **usermod** command. This example uses **usermod** to change the description of the user harry.

```
[root@RHEL4 ~]# tail -1 /etc/passwd
harry:x:516:520:harry potter:/home/harry:/bin/bash
[root@RHEL4 ~]# usermod -c 'wizard' harry
[root@RHEL4 ~]# tail -1 /etc/passwd
harry:x:516:520:wizard:/home/harry:/bin/bash
```

21.8. creating home directories

The easiest way to create a home directory is to supply the **-m** option with **useradd** (it is likely set as a default option on Linux).

A less easy way is to create a home directory manually with **mkdir** which also requires setting the owner and the permissions on the directory with **chmod** and **chown** (both commands are discussed in detail in another chapter).

```
[root@RHEL5 ~]# mkdir /home/laura
[root@RHEL5 ~]# chown laura:laura /home/laura
[root@RHEL5 ~]# chmod 700 /home/laura
[root@RHEL5 ~]# ls -ld /home/laura/
drwx----- 2 laura laura 4096 Jun 24 15:17 /home/laura/
```

21.9. /etc/skel/

When using **useradd** the **-m** option, the **/etc/skel/** directory is copied to the newly created home directory. The **/etc/skel/** directory contains some (usually hidden) files that contain profile settings and default values for applications. In this way **/etc/skel/** serves as a default home directory and as a default user profile.

```
[root@RHEL5 ~]# ls -la /etc/skel/
total 48
drwxr-xr-x  2 root root  4096 Apr  1 00:11 .
drwxr-xr-x 97 root root 12288 Jun 24 15:36 ..
-rw-r--r--  1 root root    24 Jul 12  2006 .bash_logout
-rw-r--r--  1 root root   176 Jul 12  2006 .bash_profile
-rw-r--r--  1 root root   124 Jul 12  2006 .bashrc
```

21.10. deleting home directories

The **-r** option of **userdel** will make sure that the home directory is deleted together with the user account.

```
[root@RHEL5 ~]# ls -ld /home/wim/
drwx----- 2 wim wim 4096 Jun 24 15:19 /home/wim/
[root@RHEL5 ~]# userdel -r wim
[root@RHEL5 ~]# ls -ld /home/wim/
ls: /home/wim/: No such file or directory
```

21.11. login shell

The `/etc/passwd` file specifies the **login shell** for the user. In the screenshot below you can see that user **annelies** will log in with the `/bin/bash` shell, and user **laura** with the `/bin/ksh` shell.

```
[root@RHEL5 ~]# tail -2 /etc/passwd
annelies:x:527:533:sword fighter:/home/annelies:/bin/bash
laura:x:528:534:art dealer:/home/laura:/bin/ksh
```

You can use the `usermod` command to change the shell for a user.

```
[root@RHEL5 ~]# usermod -s /bin/bash laura
[root@RHEL5 ~]# tail -1 /etc/passwd
laura:x:528:534:art dealer:/home/laura:/bin/bash
```

21.12. chsh

Users can change their login shell with the **chsh** command. First, user **harry** obtains a list of available shells (he could also have done a `cat /etc/shells`) and then changes his login shell to the **Korn shell** (`/bin/ksh`). At the next login, harry will default into ksh instead of bash.

```
[laura@centos7 ~]$ chsh -l
/bin/sh
/bin/bash
/sbin/nologin
/usr/bin/sh
/usr/bin/bash
/usr/sbin/nologin
/bin/ksh
/bin/tcsh
/bin/csh
[laura@centos7 ~]$
```

Note that the `-l` option does not exist on Debian and that the above screenshot assumes that **ksh** and **csh** shells are installed.

The screenshot below shows how **laura** can change her default shell (active on next login).

```
[laura@centos7 ~]$ chsh -s /bin/ksh
Changing shell for laura.
Password:
Shell changed.
```

21.13. solution: user management

1. Create a user account named **serena**, including a home directory and a description (or comment) that reads **Serena Williams**. Do all this in one single command.

```
root@debian7:~# useradd -m -c 'Serena Williams' serena
```

2. Create a user named **venus**, including home directory, bash shell, a description that reads **Venus Williams** all in one single command.

```
root@debian7:~# useradd -m -c "Venus Williams" -s /bin/bash venus
```

3. Verify that both users have correct entries in `/etc/passwd`, `/etc/shadow` and `/etc/group`.

```
root@debian7:~# tail -2 /etc/passwd
serena:x:1008:1010:Serena Williams:/home/serena:/bin/sh
venus:x:1009:1011:Venus Williams:/home/venus:/bin/bash
root@debian7:~# tail -2 /etc/shadow
serena:!:16358:0:99999:7:::
venus:!:16358:0:99999:7:::
root@debian7:~# tail -2 /etc/group
serena:x:1010:
venus:x:1011:
```

4. Verify that their home directory was created.

```
root@debian7:~# ls -lrt /home | tail -2
drwxr-xr-x 2 serena  serena  4096 Oct 15 10:50 serena
drwxr-xr-x 2 venus   venus   4096 Oct 15 10:59 venus
root@debian7:~#
```

5. Create a user named **einstime** with **/bin/date** as his default logon shell.

```
root@debian7:~# useradd -s /bin/date einstime
```

Or even better:

```
root@debian7:~# useradd -s $(which date) einstime
```

7. What happens when you log on with the **einstime** user ? Can you think of a useful real world example for changing a user's login shell to an application ?

```
root@debian7:~# su - einstime
Wed Oct 15 11:05:56 UTC 2014 # You get the output of the date command
root@debian7:~#
```

It can be useful when users need to access only one application on the server. Just logging in opens the application for them, and closing the application automatically logs them out.

8. Create a file named **welcome.txt** and make sure every new user will see this file in their home directory.

```
root@debian7:~# echo Hello > /etc/skel/welcome.txt
```

9. Verify this setup by creating (and deleting) a test user account.

```
root@debian7:~# useradd -m test
root@debian7:~# ls -l /home/test
total 4
-rw-r--r-- 1 test test 6 Oct 15 11:16 welcome.txt
root@debian7:~# userdel -r test
root@debian7:~#
```

10. Change the default login shell for the **serena** user to **/bin/bash**. Verify before and after you make this change.

```
root@debian7:~# grep serena /etc/passwd
serena:x:1008:1010:Serena Williams:/home/serena:/bin/sh
root@debian7:~# usermod -s /bin/bash serena
root@debian7:~# grep serena /etc/passwd
serena:x:1008:1010:Serena Williams:/home/serena:/bin/bash
root@debian7:~#
```

Chapter 22. user passwords

This chapter will tell you more about passwords for local users.

Three methods for setting passwords are explained; using the **passwd** command, using **openssl passwd**, and using the **crypt** function in a C program.

The chapter will also discuss password settings and disabling, suspending or locking accounts.

22.1. passwd

Passwords of users can be set with the **passwd** command. Users will have to provide their old password before twice entering the new one.

```
[tania@centos7 ~]$ passwd
Changing password for user tania.
Changing password for tania.
(current) UNIX password:
New password:
BAD PASSWORD: The password is shorter than 8 characters
New password:
BAD PASSWORD: The password is a palindrome
New password:
BAD PASSWORD: The password is too similar to the old one
passwd: Have exhausted maximum number of retries for service
```

As you can see, the **passwd** tool will do some basic verification to prevent users from using too simple passwords. The **root** user does not have to follow these rules (there will be a warning though). The **root** user also does not have to provide the old password before entering the new password twice.

```
root@debian7:~# passwd tania
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

22.2. shadow file

User passwords are encrypted and kept in **/etc/shadow**. The **/etc/shadow** file is read only and can only be read by root. We will see in the file permissions section how it is possible for users to change their password. For now, you will have to know that users can change their password with the **/usr/bin/passwd** command.

```
[root@centos7 ~]# tail -4 /etc/shadow
paul:$6$ikp2Xta5BT.Tml.p$2TZjNnOYNNQKpwLJqoGJbVsZG5/Fti8ovBRd.VzRbiDSl7TEq\
IaSMH.TeBKnTS/SjlMruW8qffc0JNORW.BTW1:16338:0:99999:7:::
tania:$6$8Z/zovxj$9qvoqT8i9KIrmN.k4EQwAF5ryz5yzNwEvYjAa9L5XVXQu.z4DlvpMREH\
eQpQzvRnqFdKkVj17H5ST.c79HDZw0:16356:0:99999:7:::
laura:$6$glDuTY5e$/NYYWLxfHgZFWeoujaXSMcR.Mz.lGOxtcxFocFVJNb98nbTPhWFXfKWG\
SyYh1WCv6763Wq54.w24Yr3uAZBOM/:16356:0:99999:7:::
valentina:$6$jZa6PVI$luQgqR6En9mZB6mKJ3LXRB4CnFko6LRhbb.v4iqUk9MVreui1lv7\
GxHOUDSKA0N55ZRNhGHa6T2ouFnVno/0o1:16356:0:99999:7:::
[root@centos7 ~]#
```

The **/etc/shadow** file contains nine colon separated columns. The nine fields contain (from left to right) the user name, the encrypted password (note that only inge and laura have an encrypted password), the day the password was last changed (day 1 is January 1, 1970), number of days the password must be left unchanged, password expiry day, warning number of days before password expiry, number of days after expiry before disabling the account, and the day the account was disabled (again, since 1970). The last field has no meaning yet.

All the passwords in the screenshot above are hashes of **hunter2**.

22.3. encryption with passwd

Passwords are stored in an encrypted format. This encryption is done by the **crypt** function. The easiest (and recommended) way to add a user with a password to the system is to add the user with the **useradd -m user** command, and then set the user's password with **passwd**.

```
[root@RHEL4 ~]# useradd -m xavier
[root@RHEL4 ~]# passwd xavier
Changing password for user xavier.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
[root@RHEL4 ~]#
```

22.4. encryption with openssl

Another way to create users with a password is to use the **-p** option of **useradd**, but that option requires an encrypted password. You can generate this encrypted password with the **openssl passwd** command.

The **openssl passwd** command will generate several distinct hashes for the same password, for this it uses a **salt**.

```
paul@rhel65:~$ openssl passwd hunter2
86jcUNlnGDFpY
paul@rhel65:~$ openssl passwd hunter2
Yj7mDO9OAnvq6
paul@rhel65:~$ openssl passwd hunter2
YqDcJeGoDbzKA
paul@rhel65:~$
```

This **salt** can be chosen and is visible as the first two characters of the hash.

```
paul@rhel65:~$ openssl passwd -salt 42 hunter2
42ZrbtP1Ze8G.
paul@rhel65:~$ openssl passwd -salt 42 hunter2
42ZrbtP1Ze8G.
paul@rhel65:~$ openssl passwd -salt 42 hunter2
42ZrbtP1Ze8G.
paul@rhel65:~$
```

This example shows how to create a user with password.

```
root@rhel65:~# useradd -m -p $(openssl passwd hunter2) mohamed
```

Note that this command puts the password in your command history!

22.5. encryption with crypt

A third option is to create your own C program using the crypt function, and compile this into a command.

```
paul@rhel65:~$ cat MyCrypt.c
#include <stdio.h>
#define_USE_XOPEN
#include <unistd.h>

int main(int argc, char** argv)
{
    if(argc==3)
    {
        printf("%s\n", crypt(argv[1],argv[2]));
    }
    else
    {
        printf("Usage: MyCrypt $password $salt\n" );
    }
    return 0;
}
```

This little program can be compiled with **gcc** like this.

```
paul@rhel65:~$ gcc MyCrypt.c -o MyCrypt -lcrypt
```

To use it, we need to give two parameters to MyCrypt. The first is the unencrypted password, the second is the salt. The salt is used to perturb the encryption algorithm in one of 4096 different ways. This variation prevents two users with the same password from having the same entry in **/etc/shadow**.

```
paul@rhel65:~$ ./MyCrypt hunter2 42
42ZrbtP1Ze8G.
paul@rhel65:~$ ./MyCrypt hunter2 33
33d6taYSiEUXI
```

Did you notice that the first two characters of the password are the **salt**?

The standard output of the crypt function is using the DES algorithm which is old and can be cracked in minutes. A better method is to use **md5** passwords which can be recognized by a salt starting with \$1\$.

```
paul@rhel65:~$ ./MyCrypt hunter2 '$1$42'
$1$42$716Y3xT5282XmZrtDOF9f0
paul@rhel65:~$ ./MyCrypt hunter2 '$6$42'
$6$42$OqFFAVnI3gTSYG0yI9TZWX9cpyQzwIop7HwpG1LLEsNBiMr4w6OvLX1KDa./UpwXfrFkli...
```

The **md5** salt can be up to eight characters long. The salt is displayed in **/etc/shadow** between the second and third \$, so never use the password as the salt!

```
paul@rhel65:~$ ./MyCrypt hunter2 '$1$hunter2'
$1$hunter2$YVxrxDmidq7Xf8Gdt6qM2.
```

22.6. /etc/login.defs

The **/etc/login.defs** file contains some default settings for user passwords like password aging and length settings. (You will also find the numerical limits of user ids and group ids and whether or not a home directory should be created by default).

```
root@rhel65:~# grep ^PASS /etc/login.defs
PASS_MAX_DAYS    99999
PASS_MIN_DAYS    0
PASS_MIN_LEN     5
PASS_WARN_AGE    7
```

Debian also has this file.

```
root@debian7:~# grep PASS /etc/login.defs
# PASS_MAX_DAYS    Maximum number of days a password may be used.
# PASS_MIN_DAYS    Minimum number of days allowed between password changes.
# PASS_WARN_AGE    Number of days warning given before a password expires.
PASS_MAX_DAYS    99999
PASS_MIN_DAYS    0
PASS_WARN_AGE    7
#PASS_CHANGE_TRIES
#PASS_ALWAYS_WARN
#PASS_MIN_LEN
#PASS_MAX_LEN
# NO_PASSWORD_CONSOLE
root@debian7:~#
```

22.7. chage

The **chage** command can be used to set an expiration date for a user account (-E), set a minimum (-m) and maximum (-M) password age, a password expiration date, and set the number of warning days before the password expiration date. Much of this functionality is also available from the **passwd** command. The **-l** option of **chage** will list these settings for a user.

```
root@rhel65:~# chage -l paul
Last password change                : Mar 27, 2014
Password expires                    : never
Password inactive                   : never
Account expires                     : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
root@rhel65:~#
```

22.8. disabling a password

Passwords in **/etc/shadow** cannot begin with an exclamation mark. When the second field in **/etc/passwd** starts with an exclamation mark, then the password can not be used.

Using this feature is often called **locking**, **disabling**, or **suspending** a user account. Besides **vi** (or **vim**) you can also accomplish this with **usermod**.

The first command in the next screenshot will show the hashed password of **laura** in **/etc/shadow**. The next command disables the password of **laura**, making it impossible for Laura to authenticate using this password.

```
root@debian7:~# grep laura /etc/shadow | cut -c1-70
laura:$6$JYj4JZqp$stwwWACp3OtE1R2aZuE87j.nbW.puDkNUYV7mCHfCVMa3CoDUJV
root@debian7:~# usermod -L laura
```

As you can see below, the password hash is simply preceded with an exclamation mark.

```
root@debian7:~# grep laura /etc/shadow | cut -c1-70
laura:!!$6$JYj4JZqp$stwwWACp3OtE1R2aZuE87j.nbW.puDkNUYV7mCHfCVMa3CoDUJV
root@debian7:~#
```

The root user (and users with **sudo** rights on **su**) still will be able to **su** into the **laura** account (because the password is not needed here). Also note that **laura** will still be able to login if she has set up passwordless ssh!

```
root@debian7:~# su - laura
laura@debian7:~$
```

You can unlock the account again with **usermod -U**.

```
root@debian7:~# usermod -U laura
root@debian7:~# grep laura /etc/shadow | cut -c1-70
laura:$6$JYj4JZqp$stwwWACp3OtE1R2aZuE87j.nbW.puDkNUYV7mCHfCVMa3CoDUJV
```

Watch out for tiny differences in the command line options of **passwd**, **usermod**, and **useradd** on different Linux distributions. Verify the local files when using features like "**disabling**, **suspending**, or **locking**" on user accounts and their passwords.

22.9. editing local files

If you still want to manually edit the **/etc/passwd** or **/etc/shadow**, after knowing these commands for password management, then use **vim** instead of **vi(m)** directly. The **vim** tool will do proper locking of the file.

```
[root@RHEL5 ~]# vim /etc/passwd
vim: the password file is busy (/etc/ptmp present)
```

22.10. solution: user passwords

1. Set the password for **serena** to **hunter2**.

```
root@debian7:~# passwd serena
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

2. Also set a password for **venus** and then lock the **venus** user account with **usermod**. Verify the locking in **/etc/shadow** before and after you lock it.

```
root@debian7:~# passwd venus
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@debian7:~# grep venus /etc/shadow | cut -c1-70
venus:$6$gswzXICW$uSnKFV1kFKZmTPaMVS4AvNA/KO27OxN0v5LHdV9ed0gTyXrjUeM/
root@debian7:~# usermod -L venus
root@debian7:~# grep venus /etc/shadow | cut -c1-70
venus:!!$6$gswzXICW$uSnKFV1kFKZmTPaMVS4AvNA/KO27OxN0v5LHdV9ed0gTyXrjUeM
```

Note that **usermod -L** precedes the password hash with an exclamation mark (!).

3. Use **passwd -d** to disable the **serena** password. Verify the **serena** line in **/etc/shadow** before and after disabling.

```
root@debian7:~# grep serena /etc/shadow | cut -c1-70
serena:$6$Es$omrPE$F2Ypu8kpLrfKdW0v/UIwA5jrYyBD2nwZ/dt.i/IypRgiPZSdB/B
root@debian7:~# passwd -d serena
passwd: password expiry information changed.
root@debian7:~# grep serena /etc/shadow
serena::16358:0:99999:7:::
root@debian7:~#
```

4. What is the difference between locking a user account and disabling a user account's password like we just did with **usermod -L** and **passwd -d**?

Locking will prevent the user from logging on to the system with his password by putting a **!** in front of the password in **/etc/shadow**.

Disabling with **passwd** will erase the password from **/etc/shadow**.

5. Try changing the password of **serena** to **serena** as **serena**.

```
log on as serena, then execute: passwd serena... it should fail!
```

6. Make sure **serena** has to change her password in 10 days.

```
chage -M 10 serena
```

7. Make sure every new user needs to change their password every 10 days.

```
vi /etc/login.defs (and change PASS_MAX_DAYS to 10)
```

8. Take a backup as root of **/etc/shadow**. Use **vi** to copy an encrypted **hunter2** hash from **venus** to **serena**. Can **serena** now log on with **hunter2** as a password ?

Yes.

9. Why use **vipw** instead of **vi** ? What could be the problem when using **vi** or **vim** ?

vipw will give a warning when someone else is already using that file (with **vipw**).

10. Use **chsh** to list all shells (only works on RHEL/CentOS/Fedora), and compare to **cat /etc/shells**.

```
chsh -l  
cat /etc/shells
```

11. Which **useradd** option allows you to name a home directory ?

-d

12. How can you see whether the password of user **serena** is locked or unlocked ? Give a solution with **grep** and a solution with **passwd**.

```
grep serena /etc/shadow
```

```
passwd -S serena
```

Chapter 23. user profiles

Logged on users have a number of preset (and customized) aliases, variables, and functions, but where do they come from ? The **shell** uses a number of startup files that are executed (or rather **sourced**) whenever the shell is invoked. What follows is an overview of startup scripts.

23.1. system profile

Both the **bash** and the **ksh** shell will verify the existence of **/etc/profile** and **source** it if it exists.

When reading this script, you will notice (both on Debian and on Red Hat Enterprise Linux) that it builds the **PATH** environment variable (among others). The script might also change the **PS1** variable, set the **HOSTNAME** and execute even more scripts like **/etc/inputrc**

This screenshot uses **grep** to show **PATH** manipulation in **/etc/profile** on Debian.

```
root@debian7:~# grep PATH /etc/profile
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
PATH="/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games"
export PATH
root@debian7:~#
```

This screenshot uses **grep** to show **PATH** manipulation in **/etc/profile** on RHEL7/CentOS7.

```
[root@centos7 ~]# grep PATH /etc/profile
case ":{PATH}:" in
    PATH=$PATH:$1
    PATH=$1:$PATH
export PATH USER LOGNAME MAIL HOSTNAME HISTSIZE HISTCONTROL
[root@centos7 ~]#
```

The **root user** can use this script to set aliases, functions, and variables for every user on the system.

23.2. ~/.bash_profile

When this file exists in the home directory, then **bash** will source it. On Debian Linux 5/6/7 this file does not exist by default.

RHEL7/CentOS7 uses a small **~/.bash_profile** where it checks for the existence of **~/.bashrc** and then sources it. It also adds **\$HOME/bin** to the **\$PATH** variable.

```
[root@rhel7 ~]# cat /home/paul/.bash_profile
# .bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/.local/bin:$HOME/bin

export PATH
[root@rhel7 ~]#
```

23.3. ~/.bash_login

When **.bash_profile** does not exist, then **bash** will check for **~/.bash_login** and source it.

Neither Debian nor Red Hat have this file by default.

23.4. ~/.profile

When neither **~/.bash_profile** and **~/.bash_login** exist, then **bash** will verify the existence of **~/.profile** and execute it. This file does not exist by default on Red Hat.

On Debian this script can execute **~/.bashrc** and will add **\$HOME/bin** to the **\$PATH** variable.

```
root@debian7:~# tail -11 /home/paul/.profile
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
        . "$HOME/.bashrc"
    fi
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi
```

RHEL/CentOS does not have this file by default.

23.5. ~/.bashrc

The **~/.bashrc** script is often sourced by other scripts. Let us take a look at what it does by default.

Red Hat uses a very simple **~/.bashrc**, checking for **/etc/bashrc** and sourcing it. It also leaves room for custom aliases and functions.

```
[root@rhel7 ~]# cat /home/paul/.bashrc
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi

# Uncomment the following line if you don't like systemctl's auto-paging feature:
# export SYSTEMD_PAGER=

# User specific aliases and functions
```

On Debian this script is quite a bit longer and configures **\$PS1**, some history variables and a number of active and inactive aliases.

```
root@debian7:~# wc -l /home/paul/.bashrc
110 /home/paul/.bashrc
```

23.6. ~/.bash_logout

When exiting **bash**, it can execute **~/.bash_logout**.

Debian use this opportunity to clear the console screen.

```
serena@deb503:~$ cat ~/.bash_logout
# ~/.bash_logout: executed by bash(1) when login shell exits.

# when leaving the console clear the screen to increase privacy

if [ "$SHLVL" = 1 ]; then
    [ -x /usr/bin/clear_console ] && /usr/bin/clear_console -q
fi
```

Red Hat Enterprise Linux 5 will simple call the **/usr/bin/clear** command in this script.

```
[serena@rhel53 ~]$ cat ~/.bash_logout
# ~/.bash_logout

/usr/bin/clear
```

Red Hat Enterprise Linux 6 and 7 create this file, but leave it empty (except for a comment).

```
paul@rhel65:~$ cat ~/.bash_logout
# ~/.bash_logout
```

23.7. Debian overview

Below is a table overview of when Debian is running any of these bash startup scripts.

Table 30.1. Debian User Environment

script	su	su -	ssh	gdm
~/bashrc	no	yes	yes	yes
~/profile	no	yes	yes	yes
/etc/profile	no	yes	yes	yes
/etc/bash.bashrc	yes	no	no	yes

23.8. RHEL5 overview

Below is a table overview of when Red Hat Enterprise Linux 5 is running any of these bash startup scripts.

Table 30.2. Red Hat User Environment

script	su	su -	ssh	gdm
~/bashrc	yes	yes	yes	yes
~/bash_profile	no	yes	yes	yes
/etc/profile	no	yes	yes	yes
/etc/bashrc	yes	yes	yes	yes

23.9. solution: user profiles

1. Make a list of all the profile files on your system.

```
ls -a ~ ; ls -l /etc/pro* /etc/bash*
```

2. Read the contents of each of these, often they **source** extra scripts.
3. Put a unique variable, alias and function in each of those files.
4. Try several different ways to obtain a shell (su, su -, ssh, tmux, gnome-terminal, Ctrl- alt-F1, ...) and verify which of your custom variables, aliases and function are present in your environment.
5. Do you also know the order in which they are executed?

```
same name aliases, functions and variables will overwrite each other
```

6. When an application depends on a setting in \$HOME/.profile, does it matter whether \$HOME/.bash_profile exists or not ?

```
Yes it does matter. (man bash /INVOCATION)
```

Chapter 24. groups

Users can be listed in **groups**. Groups allow you to set permissions on the group level instead of having to set permissions for every individual user.

Every Unix or Linux distribution will have a graphical tool to manage groups. Novice users are advised to use this graphical tool. More experienced users can use command line tools to manage users, but be careful: Some distributions do not allow the mixed use of GUI and CLI tools to manage groups (YaST in Novell Suse). Senior administrators can edit the relevant files directly with **vi** or **vigr**.

24.1. groupadd

Groups can be created with the **groupadd** command. The example below shows the creation of five (empty) groups.

```
root@laika:~# groupadd tennis
root@laika:~# groupadd football
root@laika:~# groupadd snooker
root@laika:~# groupadd formula1
root@laika:~# groupadd salsa
```

24.2. group file

Users can be a member of several groups. Group membership is defined by the **/etc/group** file.

```
root@laika:~# tail -5 /etc/group
tennis:x:1006:
football:x:1007:
snooker:x:1008:
formula1:x:1009:
salsa:x:1010:
root@laika:~#
```

The first field is the group's name. The second field is the group's (encrypted) password (can be empty). The third field is the group identification or **GID**. The fourth field is the list of members, these groups have no members.

24.3. groups

A user can type the **groups** command to see a list of groups where the user belongs to.

```
[harry@RHEL4b ~]$ groups
harry sports
[harry@RHEL4b ~]$
```


24.4. usermod

Group membership can be modified with the `useradd` or **usermod** command.

```
root@laika:~# usermod -a -G tennis inge
root@laika:~# usermod -a -G tennis katrien
root@laika:~# usermod -a -G salsa katrien
root@laika:~# usermod -a -G snooker sandra
root@laika:~# usermod -a -G formula1 annelies
root@laika:~# tail -5 /etc/group
tennis:x:1006:inge,katrien
football:x:1007:
snooker:x:1008:sandra
formula1:x:1009:annelies
salsa:x:1010:katrien
root@laika:~#
```

Be careful when using **usermod** to add users to groups. By default, the **usermod** command will **remove** the user from every group of which he is a member if the group is not listed in the command! Using the **-a** (append) switch prevents this behaviour.

24.5. groupmod

You can change the group name with the **groupmod** command.

```
root@laika:~# groupmod -n darts snooker
root@laika:~# tail -5 /etc/group
tennis:x:1006:inge,katrien
football:x:1007:
formula1:x:1009:annelies
salsa:x:1010:katrien
darts:x:1008:sandra
```

24.6. groupdel

You can permanently remove a group with the **groupdel** command.

```
root@laika:~# groupdel tennis
root@laika:~#
```

24.7. gpasswd

You can delegate control of group membership to another user with the **gpasswd** command. In the example below we delegate permissions to add and remove group members to serena for the sports group. Then we **su** to serena and add harry to the sports group.

```
[root@RHEL4b ~]# gpasswd -A serena sports
[root@RHEL4b ~]# su - serena
[serena@RHEL4b ~]$ id harry
uid=516(harry) gid=520(harry) groups=520(harry)
[serena@RHEL4b ~]$ gpasswd -a harry sports
Adding user harry to group sports
[serena@RHEL4b ~]$ id harry
uid=516(harry) gid=520(harry) groups=520(harry), 522(sports)
[serena@RHEL4b ~]$ tail -1 /etc/group
sports:x:522:serena,venus,harry
[serena@RHEL4b ~]$
```

Group administrators do not have to be a member of the group. They can remove themselves from a group, but this does not influence their ability to add or remove members.

```
[serena@RHEL4b ~]$ gpasswd -d serena sports
Removing user serena from group sports
[serena@RHEL4b ~]$ exit
```

Information about group administrators is kept in the **/etc/gshadow** file.

```
[root@RHEL4b ~]# tail -1 /etc/gshadow
sports:!:serena:venus,harry
[root@RHEL4b ~]#
```

To remove all group administrators from a group, use the **gpasswd** command to set an empty administrators list.

```
[root@RHEL4b ~]# gpasswd -A "" sports
```

24.8. newgrp

You can start a **child shell** with a new temporary **primary group** using the **newgrp** command.

```
root@rhel65:~# mkdir prigroup
root@rhel65:~# cd prigroup/
root@rhel65:~/prigroup# touch standard.txt
root@rhel65:~/prigroup# ls -l
total 0
-rw-r--r--. 1 root root 0 Apr 13 17:49 standard.txt
root@rhel65:~/prigroup# echo $SHLVL
1
root@rhel65:~/prigroup# newgrp tennis
root@rhel65:~/prigroup# echo $SHLVL
2
root@rhel65:~/prigroup# touch newgrp.txt
root@rhel65:~/prigroup# ls -l
total 0
-rw-r--r--. 1 root tennis 0 Apr 13 17:49 newgrp.txt
-rw-r--r--. 1 root root 0 Apr 13 17:49 standard.txt
root@rhel65:~/prigroup# exit
exit
root@rhel65:~/prigroup#
```

24.9. vigr

Similar to vipw, the **vigr** command can be used to manually edit the **/etc/group** file, since it will do proper locking of the file. Only experienced senior administrators should use **vi** or

24.10. solution: groups

1. Create the groups tennis, football and sports.

```
groupadd tennis ; groupadd football ; groupadd sports
```

2. In one command, make venus a member of tennis and sports.

```
usermod -a -G tennis,sports venus
```

3. Rename the football group to foot.

```
groupmod -n foot football
```

4. Use vi to add serena to the tennis group.

```
vi /etc/group
```

5. Use the id command to verify that serena is a member of tennis.

```
id (and after logoff logon serena should be member)
```

6. Make someone responsible for managing group membership of foot and sports. Test that it works.

```
gpasswd -A (to make manager)
```

```
gpasswd -a (to add member)
```

Chapter 25. standard file permissions

This chapter contains details about basic file security through **file ownership** and **file permissions**.

25.1. file ownership

25.1.1. user owner and group owner

The **users** and **groups** of a system can be locally managed in **/etc/passwd** and **/etc/group**, or they can be in a NIS, LDAP, or Samba domain. These users and groups can **own** files. Actually, every file has a **user owner** and a **group owner**, as can be seen in the following screenshot.

```
paul@rhel65:~/owners$ ls -lh
total 636K
-rw-r--r--. 1 paul snooker 1.1K Apr  8 18:47 data.odt
-rw-r--r--. 1 paul paul    626K Apr  8 18:46 file1
-rw-r--r--. 1 root tennis  185 Apr  8 18:46 file2
-rw-rw-r--. 1 root root    0 Apr  8 18:47 stuff.txt
paul@rhel65:~/owners$
```

User paul owns three files; file1 has paul as **user owner** and has the group paul as **group owner**, data.odt is **group owned** by the group snooker, file2 by the group tennis.

The last file is called stuff.txt and is owned by the root user and the root group.

25.1.2. listing user accounts

You can use the following command to list all local user accounts.

```
paul@debian7~$ cut -d: -f1 /etc/passwd | column
root          ntp           sam           bert          naomi
daemon        mysql         tom           rino          matthias2
bin           paul          wouter       antonio       bram
sys           maarten      robrecht     simon         fabrice
sync          kevin        bilal        sven          chimene
games         yuri         dimitri      wouter2       messagebus
man           william      ahmed        tarik         roger
lp            yves         dylan        jan           frank
mail          kris         robin        ian           toon
news          hamid        matthias     ivan          rinus
uucp          vladimir    ben          azeddine      eddy
proxy         abiy         mike         eric          bram2
www-data      david        kevin2       kamel         keith
backup        chahid       kenzo        ischa         jesse
list          stef         aaron        bart          frederick
irc           joeri        lorenzo      omer          hans
gnats         glenn        jens         kurt          dries
nobody        yannick      ruben        steve         steve2
libuuid       christof     jelle        constantin    tomas
Debian-exim   george       stefaan      sam2          johan
statd         joost        marc         bjorn         tom2
sshd          arno         thomas       ronald
```

25.1.3. chgrp

You can change the group owner of a file using the **chgrp** command.

```
root@rhel65:/home/paul/owners# ls -l file2
-rw-r--r--. 1 root tennis 185 Apr  8 18:46 file2
root@rhel65:/home/paul/owners# chgrp snooker file2
root@rhel65:/home/paul/owners# ls -l file2
-rw-r--r--. 1 root snooker 185 Apr  8 18:46 file2
root@rhel65:/home/paul/owners#
```

25.1.4. chown

The user owner of a file can be changed with **chown** command.

```
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root paul 0 2008-08-06 14:11 FileForPaul
root@laika:/home/paul# chown paul FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 paul paul 0 2008-08-06 14:11 FileForPaul
```

You can also use **chown** to change both the user owner and the group owner.

```
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 paul paul 0 2008-08-06 14:11 FileForPaul
root@laika:/home/paul# chown root:project42 FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root project42 0 2008-08-06 14:11 FileForPaul
```


25.2. list of special files

When you use **ls -l**, for each file you can see ten characters before the user and group owner. The first character tells us the type of file. Regular files get a **-**, directories get a **d**, symbolic links are shown with an **l**, pipes get a **p**, character devices a **c**, block devices a **b**, and sockets an **s**.

Table 32.1. Unix special files

first character	file type
-	normal file
d	directory
l	symbolic link
p	named pipe
b	block device
c	character device
s	socket

Below a screenshot of a character device (the console) and a block device (the hard disk).

```
paul@debian6lt~$ ls -ld /dev/console /dev/sda
crw----- 1 root root  5, 1 Mar 15 12:45 /dev/console
brw-rw---- 1 root disk  8, 0 Mar 15 12:45 /dev/sda
```

And here you can see a directory, a regular file and a symbolic link.

```
paul@debian6lt~$ ls -ld /etc /etc/hosts /etc/motd
drwxr-xr-x 128 root root 12288 Mar 15 18:34 /etc
-rw-r--r--  1 root root   372 Dec 10 17:36 /etc/hosts
lrwxrwxrwx  1 root root    13 Dec  5 10:36 /etc/motd -> /var/run/motd
```

25.3. permissions

25.3.1. rwx

The nine characters following the file type denote the permissions in three triplets. A permission can be **r** for read access, **w** for write access, and **x** for execute. You need the **r** permission to list (**ls**) the contents of a directory. You need the **x** permission to enter (**cd**) a directory. You need the **w** permission to create files in or remove files from a directory.

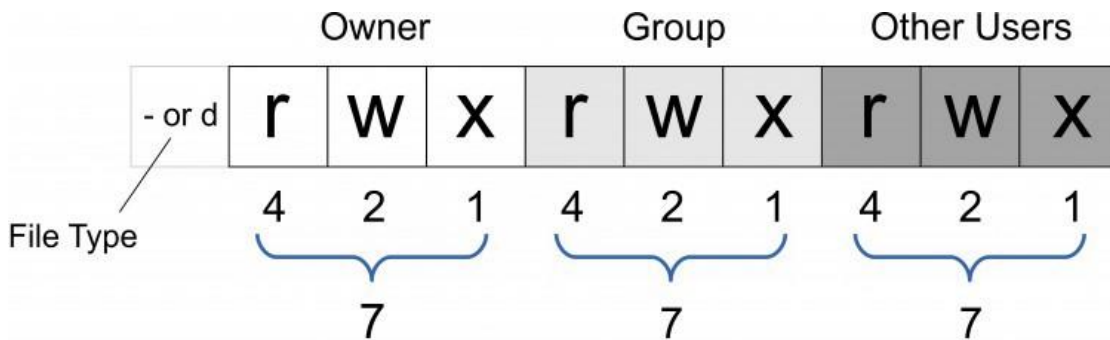


Table 32.2. standard Unix file permissions

permission	on a file	on a directory
r (read)	read file contents (cat)	read directory contents (ls)
w (write)	change file contents (vi)	create files in (touch)
x (execute)	execute the file	enter the directory (cd)

25.3.2. three sets of rwx

We already know that the output of **ls -l** starts with ten characters for each file. This screenshot shows a regular file (because the first character is a -).

```
paul@RHELv4u4:~/test$ ls -l proc42.bash
-rwxr-xr-- 1 paul proj 984 Feb  6 12:01 proc42.bash
```

Below is a table describing the function of all ten characters.

Table 32.3. Unix file permissions position

position	characters	function
1	-	this is a regular file
2-4	rwx	permissions for the user owner
5-7	r-x	permissions for the group owner
8-10	r--	permissions for others

When you are the **user owner** of a file, then the **user owner permissions** apply to you. The rest of the permissions have no influence on your access to the file.

When you belong to the **group** that is the **group owner** of a file, then the **group owner permissions** apply to you. The rest of the permissions have no influence on your access to

the file.

When you are not the **user owner** of a file and you do not belong to the **group owner**, then the **others permissions** apply to you. The rest of the permissions have no influence on your access to the file.

25.3.3. permission examples

Some example combinations on files and directories are seen in this screenshot. The name of the file explains the permissions.

```
paul@laika:~/perms$ ls -lh
total 12K
drwxr-xr-x 2 paul paul 4.0K 2007-02-07 22:26 AllEnter_UserCreateDelete
-rwxrwxrwx 1 paul paul  0 2007-02-07 22:21 EveryoneFullControl.txt
-r--r----- 1 paul paul  0 2007-02-07 22:21 OnlyOwnersRead.txt
-rwxrwx--- 1 paul paul  0 2007-02-07 22:21 OwnersAll_RestNothing.txt
dr-xr-x--- 2 paul paul 4.0K 2007-02-07 22:25 UserAndGroupEnter
dr-x----- 2 paul paul 4.0K 2007-02-07 22:25 OnlyUserEnter
paul@laika:~/perms$
```

To summarise, the first **rwX** triplet represents the permissions for the **user owner**. The second triplet corresponds to the **group owner**; it specifies permissions for all members of that group. The third triplet defines permissions for all **other** users that are not the user owner and are not a member of the group owner.

25.3.4. setting permissions (chmod)

Permissions can be changed with **chmod**. The first example gives the user owner execute permissions.

```
paul@laika:~/perms$ ls -l permissions.txt
-rw-r--r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod u+x permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxr--r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

This example removes the group owners read permission.

```
paul@laika:~/perms$ chmod g-r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx---r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

This example removes the others read permission.

```
paul@laika:~/perms$ chmod o-r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx----- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

This example gives all of them the write permission.

```
paul@laika:~/perms$ chmod a+w permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx-w--w- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

You don't even have to type the a.

```
paul@laika:~/perms$ chmod +x permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx-wx-wx 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

You can also set explicit permissions.

```
paul@laika:~/perms$ chmod u=rw permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw--wx-wx 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Feel free to make any kind of combination.

```
paul@laika:~/perms$ chmod u=rw,g=rw,o=r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw-rw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Even fishy combinations are accepted by chmod.

```
paul@laika:~/perms$ chmod u=rwx,ug+rw,o=r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxrw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

25.3.5. setting octal permissions

Most Unix administrators will use the **old school** octal system to talk about and set permissions. Look at the triplet bitwise, equating r to 4, w to 2, and x to 1.

Table 32.4. Octal permissions

binary	octal	permission
000	0	---
001	1	--x
010	2	-w-
011	3	-wx
100	4	r--
101	5	r-x
110	6	rw-
111	7	rwX

This makes **777** equal to **rw-rw-rwX** and by the same logic, **654** mean **rw-r-xr--** . The **chmod** command will accept these numbers.

```
paul@laika:~/perms$ chmod 777 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxrwxrwx 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod 664 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw-rw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod 750 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxr-x--- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

25.3.6. umask

When creating a file or directory, a set of default permissions are applied. These default permissions are determined by the **umask**. The **umask** specifies permissions that you do not want set on by default. You can display the **umask** with the **umask** command.

```
[Harry@RHEL4b ~]$ umask
0002
[Harry@RHEL4b ~]$ touch test
[Harry@RHEL4b ~]$ ls -l test
-rw-rw-r-- 1 Harry Harry 0 Jul 24 06:03 test
[Harry@RHEL4b ~]$
```

As you can also see, the file is also not executable by default. This is a general security feature among Unixes; newly created files are never executable by default. You have to explicitly do a **chmod +x** to make a file executable. This also means that the 1 bit in the **umask** has no meaning--a **umask** of 0022 is the same as 0033.

25.3.7. mkdir -m

When creating directories with **mkdir** you can use the **-m** option to set the **mode**. This screenshot explains.

```
paul@debian5~$ mkdir -m 700 MyDir
paul@debian5~$ mkdir -m 777 Public
paul@debian5~$ ls -dl MyDir/ Public/
drwx----- 2 paul paul 4096 2011-10-16 19:16 MyDir/
drwxrwxrwx 2 paul paul 4096 2011-10-16 19:16 Public/
```

25.3.8. cp -p

To preserve permissions and time stamps from source files, use **cp -p**.

```
paul@laika:~/perms$ cp file* cp
paul@laika:~/perms$ cp -p file* cpp
paul@laika:~/perms$ ll *
-rwx----- 1 paul paul    0 2008-08-25 13:26 file33
-rwxr-x--- 1 paul paul    0 2008-08-25 13:26 file42

cp:
total 0
-rwx----- 1 paul paul 0 2008-08-25 13:34 file33
-rwxr-x--- 1 paul paul 0 2008-08-25 13:34 file42

cpp:
total 0
-rwx----- 1 paul paul 0 2008-08-25 13:26 file33
-rwxr-x--- 1 paul paul 0 2008-08-25 13:26 file42
```

25.4. solution: standard file permissions

1. As normal user, create a directory ~/permissions. Create a file owned by yourself in there.

```
mkdir ~/permissions ; touch ~/permissions/myfile.txt
```

2. Copy a file owned by root from /etc/ to your permissions dir, who owns this file now ?

```
cp /etc/hosts ~/permissions/
```

The copy is owned by you.

3. As root, create a file in the users ~/permissions directory.

```
(become root)# touch /home/username/permissions/rootfile
```

4. As normal user, look at who owns this file created by root.

```
ls -l ~/permissions
```

The file created by root is owned by root.

5. Change the ownership of all files in ~/permissions to yourself.

```
chown user ~/permissions/*
```

You cannot become owner of the file that belongs to root.

6. Make sure you have all rights to these files, and others can only read.

```
chmod 644 (on files)
```

```
chmod 755 (on directories)
```

7. With chmod, is 770 the same as rwxrwx--- ?

yes

8. With chmod, is 664 the same as r-xr-xr-- ?

No

9. With chmod, is 400 the same as r----- ?

yes

10. With chmod, is 734 the same as rwxr-xr-- ?

no

11a. Display the umask in octal and in symbolic form.

```
umask ; umask -S
```

11b. Set the umask to 077, but use the symbolic format to set it. Verify that this works.

```
umask -S u=rwx,go=
```


12. Create a file as root, give only read to others. Can a normal user read this file ? Test writing to this file with vi.

```
(become root)
# echo hello > /home/username/root.txt
# chmod 744 /home/username/root.txt
(become user)
vi ~/root.txt
```

13a. Create a file as normal user, give only read to others. Can another normal user read this file ? Test writing to this file with vi.

```
echo hello > file ; chmod 744 file
```

Yes, others can read this file

13b. Can root read this file ? Can root write to this file with vi ?

Yes, root can read and write to this file. Permissions do not apply to root.

14. Create a directory that belongs to a group, where every member of that group can read and write to files, and create files. Make sure that people can only delete their own files.

```
mkdir /home/project42 ; groupadd project42
chgrp project42 /home/project42 ; chmod 775 /home/project42
```

You can not yet do the last part of this exercise...

Chapter 26. advanced file permissions

26.1. sticky bit on directory

You can set the **sticky bit** on a directory to prevent users from removing files that they do not own as a user owner. The sticky bit is displayed at the same location as the x permission for others. The sticky bit is represented by a **t** (meaning x is also there) or a **T** (when there is no x for others).

```
root@RHELv4u4:~# mkdir /project55
root@RHELv4u4:~# ls -ld /project55
drwxr-xr-x  2 root root 4096 Feb  7 17:38 /project55
root@RHELv4u4:~# chmod +t /project55/
root@RHELv4u4:~# ls -ld /project55
drwxr-xr-t  2 root root 4096 Feb  7 17:38 /project55
root@RHELv4u4:~#
```

The **sticky bit** can also be set with octal permissions, it is binary 1 in the first of four triplets.

```
root@RHELv4u4:~# chmod 1775 /project55/
root@RHELv4u4:~# ls -ld /project55
drwxrwxr-t  2 root root 4096 Feb  7 17:38 /project55
root@RHELv4u4:~#
```

You will typically find the **sticky bit** on the **/tmp** directory.

```
root@barry:~# ls -ld /tmp
drwxrwxrwt 6 root root 4096 2009-06-04 19:02 /tmp
```

26.2. setgid bit on directory

setgid can be used on directories to make sure that all files inside the directory are owned by the group owner of the directory. The **setgid** bit is displayed at the same location as the x permission for group owner. The **setgid** bit is represented by an **s** (meaning x is also there) or a **S** (when there is no x for the group owner). As this example shows, even though **root** does not belong to the group **proj55**, the files created by root in **/project55** will belong to **proj55** since the **setgid** is set.

```
root@RHELv4u4:~# groupadd proj55
root@RHELv4u4:~# chown root:proj55 /project55/
root@RHELv4u4:~# chmod 2775 /project55/
root@RHELv4u4:~# touch /project55/fromroot.txt
root@RHELv4u4:~# ls -ld /project55/
drwxrwsr-x  2 root proj55 4096 Feb  7 17:45 /project55/
root@RHELv4u4:~# ls -l /project55/
total 4
-rw-r--r--  1 root proj55 0 Feb  7 17:45 fromroot.txt
root@RHELv4u4:~#
```

You can use the **find** command to find all **setgid** directories.

```
paul@laika:~$ find / -type d -perm -2000 2> /dev/null
/var/log/mysql
/var/log/news
/var/local
...
```

26.3. setgid and setuid on regular files

These two permissions cause an executable file to be executed with the permissions of the **file owner** instead of the **executing owner**. This means that if any user executes a program that belongs to the **root user**, and the **setuid** bit is set on that program, then the program runs as **root**. This can be dangerous, but sometimes this is good for security.

Take the example of passwords; they are stored in **/etc/shadow** which is only readable by **root**. (The **root** user never needs permissions anyway.)

```
root@RHELv4u4:~# ls -l /etc/shadow
-r----- 1 root root 1260 Jan 21 07:49 /etc/shadow
```

Changing your password requires an update of this file, so how can normal non-root users do this? Let's take a look at the permissions on the **/usr/bin/passwd**.

```
root@RHELv4u4:~# ls -l /usr/bin/passwd
-r-s--x--x 1 root root 21200 Jun 17 2005 /usr/bin/passwd
```

When running the **passwd** program, you are executing it with **root** credentials.

You can use the **find** command to find all **setuid** programs.

```
paul@laika:~$ find /usr/bin -type f -perm -04000
/usr/bin/arping
/usr/bin/kgrantpty
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/fping6
/usr/bin/passwd
/usr/bin/gpasswd
...
```

In most cases, setting the **setuid** bit on executables is sufficient. Setting the **setgid** bit will result in these programs to run with the credentials of their group owner.

26.4. setuid on sudo

The **sudo** binary has the **setuid** bit set, so any user can run it with the effective userid of root.

```
paul@rhel65:~$ ls -l $(which sudo)
---s--x--x. 1 root root 123832 Oct 7 2013 /usr/bin/sudo
paul@rhel65:~$
```

26.5. solution: sticky, setuid and setgid bits

1a. Set up a directory, owned by the group sports.

```
groupadd sports
```

```
mkdir /home/sports
```

```
chown root:sports /home/sports
```

1b. Members of the sports group should be able to create files in this directory.

```
chmod 770 /home/sports
```

1c. All files created in this directory should be group-owned by the sports group.

```
chmod 2770 /home/sports
```

1d. Users should be able to delete only their own user-owned files.

```
chmod +t /home/sports
```

1e. Test that this works!

Log in with different users (group members and others and root), create files and watch the permissions. Try changing and deleting files...

2. Verify the permissions on **/usr/bin/passwd**. Remove the **setuid**, then try changing your password as a normal user. Reset the permissions back and try again.

```
root@deb503:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
root@deb503:~# chmod 755 /usr/bin/passwd
root@deb503:~# ls -l /usr/bin/passwd
-rwxr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
```

A normal user cannot change password now.

```
root@deb503:~# chmod 4755 /usr/bin/passwd
root@deb503:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
```

3. If time permits (or if you are waiting for other students to finish this practice), read about file attributes in the man page of `chattr` and `lsattr`. Try setting the `i` attribute on a file and test that it works.

```
paul@laika:~$ sudo su -
[sudo] password for paul:
root@laika:~# mkdir attr
root@laika:~# cd attr/
root@laika:~/attr# touch file42
root@laika:~/attr# lsattr
-----
./file42
root@laika:~/attr# chattr +i file42
```

```
root@laika:~/attr# lsattr
----i----- ./file42
root@laika:~/attr# rm -rf file42
rm: cannot remove `file42': Operation not permitted
root@laika:~/attr# chattr -i file42
root@laika:~/attr# rm -rf file42
root@laika:~/attr#
```

Chapter 27. access control lists

Standard Unix permissions might not be enough for some organisations. This chapter introduces **access control lists** or **acl's** to further protect files and directories.

27.1. acl in /etc/fstab

File systems that support **access control lists**, or **acls**, have to be mounted with the **acl** option listed in **/etc/fstab**. In the example below, you can see that the root file system has **acl** support, whereas **/home/data** does not.

```
root@laika:~# tail -4 /etc/fstab
/dev/sda1      /              ext3      acl,relatime 0 1
/dev/sdb2      /home/data     auto      noacl,defaults 0 0
pasha:/home/r  /home/pasha   nfs       defaults     0 0
wolf:/srv/data /home/wolf     nfs       defaults     0 0
```

27.2. getfacl

Reading **acls** can be done with **/usr/bin/getfacl**. This screenshot shows how to read the **acl** of **file33** with **getfacl**.

```
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
group::r--
mask::rwx
other::r--
```

27.3. setfacl

Writing or changing **acls** can be done with **/usr/bin/setfacl**. These screenshots show how to change the **acl** of **file33** with **setfacl**.

First we add **user sandra** with octal permission **7** to the **acl**.

```
paul@laika:~/test$ setfacl -m u:sandra:7 file33
```

Then we add the **group tennis** with octal permission **6** to the **acl** of the same file.

```
paul@laika:~/test$ setfacl -m g:tennis:6 file33
```

The result is visible with **getfacl**.

```
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
user:sandra:rwx
group::r--
group:tennis:rw-
mask::rwx
other::r--
```

27.4. remove an acl entry

The **-x** option of the **setfacl** command will remove an **acl** entry from the targeted file.

```
paul@laika:~/test$ setfacl -m u:sandra:7 file33
paul@laika:~/test$ getfacl file33 | grep sandra
user:sandra:rw-
paul@laika:~/test$ setfacl -x sandra file33
paul@laika:~/test$ getfacl file33 | grep sandra
```

Note that omitting the **u** or **g** when defining the **acl** for an account will default it to a user account.

27.5. remove the complete acl

The **-b** option of the **setfacl** command will remove the **acl** from the targeted file.

```
paul@laika:~/test$ setfacl -b file33
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
group::r--
other::r--
```

27.6. the acl mask

The **acl mask** defines the maximum effective permissions for any entry in the **acl**. This **mask** is calculated every time you execute the **setfacl** or **chmod** commands.

You can prevent the calculation by using the **--no-mask** switch.

```
paul@laika:~/test$ setfacl --no-mask -m u:sandra:7 file33
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
user:sandra:rw-   #effective:rw-
group::r--
mask::rw-
other::r--
```

27.7. eiciel

Desktop users might want to use **eiciel** to manage **acls** with a graphical tool.



You will need to install **eiciel** and **nautilus-actions** to have an extra tab in **nautilus** to manage **acls**.

```
paul@laika:~$ sudo aptitude install eiciel nautilus-actions
```

Chapter 28. file links

An average computer using Linux has a file system with many **hard links** and **symbolic links**.

To understand links in a file system, you first have to understand what an **inode** is.

28.1. inodes

28.1.1. inode contents

An **inode** is a data structure that contains metadata about a file. When the file system stores a new file on the hard disk, it stores not only the contents (data) of the file, but also extra properties like the name of the file, the creation date, its permissions, the owner of the file, and more. All this information (except the name of the file and the contents of the file) is stored in the **inode** of the file.

The **ls -l** command will display some of the inode contents, as seen in this screenshot.

```
root@rhel53 ~# ls -ld /home/project42/
drwxr-xr-x 4 root pro42 4.0K Mar 27 14:29 /home/project42/
```

28.1.2. inode table

The **inode table** contains all of the **inodes** and is created when you create the file system (with **mkfs**). You can use the **df -i** command to see how many **inodes** are used and free on mounted file systems.

```
root@rhel53 ~# df -i
Filesystem              Inodes    IUsed    IFree  IUse% Mounted on
/dev/mapper/VolGroup00-LogVol100
                        4947968  115326  4832642    3% /
/dev/hda1                26104      45    26059    1% /boot
tmpfs                   64417        1    64416    1% /dev/shm
/dev/sda1                262144    2207   259937    1% /home/project42
/dev/sdb1                 74400    5519    68881    8% /home/project33
/dev/sdb5                  0          0         0    - /home/sales
/dev/sdb6               100744     11   100733    1% /home/research
```

In the **df -i** screenshot above you can see the **inode** usage for several mounted **file systems**. You don't see numbers for **/dev/sdb5** because it is a **fat** file system.

28.1.3. inode number

Each **inode** has a unique number (the inode number). You can see the **inode** numbers with the **ls -li** command.

```
paul@RHELv4u4:~/test$ touch file1
paul@RHELv4u4:~/test$ touch file2
paul@RHELv4u4:~/test$ touch file3
paul@RHELv4u4:~/test$ ls -li
total 12
817266 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file1
817267 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file3
paul@RHELv4u4:~/test$
```

These three files were created one after the other and got three different **inodes** (the first column). All the information you see with this **ls** command resides in the **inode**, except for the filename (which is contained in the directory).

28.1.4. inode and file contents

Let's put some data in one of the files.

```
paul@RHELv4u4:~/test$ ls -li
total 16
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 1 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
paul@RHELv4u4:~/test$ cat file2
It is winter now and it is very cold.
We do not like the cold, we prefer hot summer nights.
paul@RHELv4u4:~/test$
```

The data that is displayed by the **cat** command is not in the **inode**, but somewhere else on the disk. The **inode** contains a pointer to that data.

28.2. about directories

28.2.1. a directory is a table

A **directory** is a special kind of file that contains a table which maps filenames to inodes. Listing our current directory with **ls -ali** will display the contents of the directory file.

```
paul@RHELv4u4:~/test$ ls -ali
total 32
817262 drwxrwxr-x 2 paul paul 4096 Feb 5 15:42 .
800768 drwx----- 16 paul paul 4096 Feb 5 15:42 ..
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 1 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
paul@RHELv4u4:~/test$
```

28.2.2. . and ..

You can see five names, and the mapping to their five inodes. The dot **.** is a mapping to itself, and the dotdot **..** is a mapping to the parent directory. The three other names are mappings to different inodes.

28.3. hard links

28.3.1. creating hard links

When we create a **hard link** to a file with **ln**, an extra entry is added in the directory. A new file name is mapped to an existing inode.

```
paul@RHELv4u4:~/test$ ln file2 hardlink_to_file2
paul@RHELv4u4:~/test$ ls -li
total 24
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 2 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
817270 -rw-rw-r-- 2 paul paul 92 Feb 5 15:42 hardlink_to_file2
paul@RHELv4u4:~/test$
```

Both files have the same inode, so they will always have the same permissions and the same owner. Both files will have the same content. Actually, both files are equal now, meaning you can safely remove the original file, the hardlinked file will remain. The inode contains a counter, counting the number of hard links to itself. When the counter drops to zero, then the inode is emptied.

28.3.2. finding hard links

You can use the **find** command to look for files with a certain inode. The screenshot below shows how to search for all filenames that point to **inode** 817270. Remember that an **inode** number is unique to its partition.

```
paul@RHELv4u4:~/test$ find / -inum 817270 2> /dev/null
/home/paul/test/file2
/home/paul/test/hardlink_to_file2
```

28.4. symbolic links

Symbolic links (sometimes called **soft links**) do not link to inodes, but create a name to name mapping. Symbolic links are created with **ln -s**. As you can see below, the **symbolic link** gets an inode of its own.

```
paul@RHELv4u4:~/test$ ln -s file2 symlink_to_file2
paul@RHELv4u4:~/test$ ls -li
total 32
817273 -rw-rw-r-- 1 paul paul 13 Feb 5 17:06 file1
817270 -rw-rw-r-- 2 paul paul 106 Feb 5 17:04 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
817270 -rw-rw-r-- 2 paul paul 106 Feb 5 17:04 hardlink_to_file2
817267 lrwxrwxrwx 1 paul paul 5 Feb 5 16:55 symlink_to_file2 -> file2
paul@RHELv4u4:~/test$
```

Permissions on a symbolic link have no meaning, since the permissions of the target apply. Hard links are limited to their own partition (because they point to an inode), symbolic links can link anywhere (other file systems, even networked).

28.5. removing links

Links can be removed with **rm**.

```
paul@laika:~$ touch data.txt
paul@laika:~$ ln -s data.txt sl_data.txt
paul@laika:~$ ln data.txt hl_data.txt
paul@laika:~$ rm sl_data.txt
paul@laika:~$ rm hl_data.txt
```

28.6. solution : links

1. Create two files named winter.txt and summer.txt, put some text in them.

```
echo cold > winter.txt ; echo hot > summer.txt
```

2. Create a hard link to winter.txt named hlwinter.txt.

```
ln winter.txt hlwinter.txt
```

3. Display the inode numbers of these three files, the hard links should have the same inode.

```
ls -li winter.txt summer.txt hlwinter.txt
```

4. Use the find command to list the two hardlinked files

```
find . -inum xyz #replace xyz with the inode number
```

5. Everything about a file is in the inode, except two things : name them!

The name of the file is in a directory, and the contents is somewhere on the disk.

6. Create a symbolic link to summer.txt called slsummer.txt.

```
ln -s summer.txt slsummer.txt
```


7. Find all files with inode number 2. What does this information tell you ?

It tells you there is more than one inode table (one for every formatted partition + virtual file systems)

8. Look at the directories /etc/init.d/ /etc/rc.d/ /etc/rc3.d/ ... do you see the links ?

```
ls -l /etc/init.d
```

```
ls -l /etc/rc2.d
```

```
ls -l /etc/rc3.d
```

9. Look in /lib with ls -l...

```
ls -l /lib
```

10. Use **find** to look in your home directory for regular files that do not(!) have one hard link.

```
find ~ ! -links 1 -type f
```

Appendix A. keyboard settings

A.1. about keyboard layout

Many people (like US-Americans) prefer the default US-qwerty keyboard layout. So when you are not from the USA and want a local keyboard layout on your system, then the best practice is to select this keyboard at installation time. Then the keyboard layout will always be correct. Also, whenever you use ssh to remotely manage a Linux system, your local keyboard layout will be used, independent of the server keyboard configuration. So you will not find much information on changing keyboard layout on the fly on linux, because not many people need it. Below are some tips to help you.

A.2. X Keyboard Layout

This is the relevant portion in `/etc/X11/xorg.conf`, first for Belgian azerty, then for US-qwerty.

```
[paul@RHEL5 ~]$ grep -i xkb /etc/X11/xorg.conf
Option      "XkbModel"  "pc105"
Option      "XkbLayout" "be"
```

```
[paul@RHEL5 ~]$ grep -i xkb /etc/X11/xorg.conf
Option      "XkbModel"  "pc105"
Option      "XkbLayout" "us"
```

When in Gnome or KDE or any other graphical environment, look in the graphical menu in preferences, there will be a keyboard section to choose your layout. Use the graphical menu instead of editing `xorg.conf`.

A.3. shell keyboard layout

When in bash, take a look in the `/etc/sysconfig/keyboard` file. Below a sample US-qwerty configuration, followed by a Belgian azerty configuration.

```
[paul@RHEL5 ~]$ cat /etc/sysconfig/keyboard
KEYBOARDTYPE="pc"
KEYTABLE="us"
```

```
[paul@RHEL5 ~]$ cat /etc/sysconfig/keyboard
KEYBOARDTYPE="pc"
KEYTABLE="be-latin1"
```

The keymaps themselves can be found in `/usr/share/keymaps` or `/lib/kbd/keymaps`.

```
[paul@RHEL5 ~]$ ls -l /lib/kbd/keymaps/
total 52
drwxr-xr-x 2 root root 4096 Apr  1 00:14 amiga
```



```
drwxr-xr-x 2 root root 4096 Apr 1 00:14 atari
drwxr-xr-x 8 root root 4096 Apr 1 00:14 i386
drwxr-xr-x 2 root root 4096 Apr 1 00:14 include
drwxr-xr-x 4 root root 4096 Apr 1 00:14 mac
lrwxrwxrwx 1 root root    3 Apr 1 00:14 ppc -> mac
drwxr-xr-x 2 root root 4096 Apr 1 00:14 sun
```

Appendix B. hardware

B.1. buses

B.1.1. about buses

Hardware components communicate with the **Central Processing Unit** or **cpu** over a **bus**. The most common buses today are **usb**, **pci**, **agp**, **pci-express** and **pcmcia** aka **pc-card**. These are all **Plug and Play** buses.

Older **x86** computers often had **isa** buses, which can be configured using **jumper**s or **dip switches**.

B.1.2. /proc/bus

To list the buses recognised by the Linux kernel on your computer, look at the contents of the **/proc/bus/** directory (screenshot from Ubuntu 7.04 and RHEL4u4 below).

```
root@laika:~# ls /proc/bus/
input  pccard  pci  usb
```

```
[root@RHEL4b ~]# ls /proc/bus/
input  pci  usb
```

Can you guess which of these two screenshots was taken on a laptop ?

B.1.3. /usr/sbin/lssusb

To list all the usb devices connected to your system, you could read the contents of **/proc/bus/usb/devices** (if it exists) or you could use the more readable output of **lsusb**, which is executed here on a SPARC system with Ubuntu.

```
root@shaka:~# lsusb
Bus 001 Device 002: ID 0430:0100 Sun Microsystems, Inc. 3-button Mouse
Bus 001 Device 003: ID 0430:0005 Sun Microsystems, Inc. Type 6 Keyboard
Bus 001 Device 001: ID 04b0:0136 Nikon Corp. Coolpix 7900 (storage)
root@shaka:~#
```

B.1.4. /var/lib/usbutils/usb.ids

The **/var/lib/usbutils/usb.ids** file contains a gzipped list of all known usb devices.

```
paul@barry:~$ zmore /var/lib/usbutils/usb.ids | head
-----> /var/lib/usbutils/usb.ids <-----
#
# List of USB ID's
#
# Maintained by Vojtech Pavlik <vojtech@suse.cz>
```

```
# If you have any new entries, send them to the maintainer.
# The latest version can be obtained from
# http://www.linux-usb.org/usb.ids
#
# $Id: usb.ids,v 1.225 2006/07/13 04:18:02 dbrownell Exp $
```

B.1.5. /usr/sbin/lspci

To get a list of all pci devices connected, you could take a look at `/proc/bus/pci` or run `lspci` (partial output below).

```
paul@laika:~$ lspci
...
00:06.0 FireWire (IEEE 1394): Texas Instruments TSB43AB22/A IEEE-139...
00:08.0 Ethernet controller: Realtek Semiconductor Co., Ltd. RTL-816...
00:09.0 Multimedia controller: Philips Semiconductors SAA7133/SAA713...
00:0a.0 Network controller: RaLink RT2500 802.11g Cardbus/mini-PCI
00:0f.0 RAID bus controller: VIA Technologies, Inc. VIA VT6420 SATA ...
00:0f.1 IDE interface: VIA Technologies, Inc. VT82C586A/B/VT82C686/A...
00:10.0 USB Controller: VIA Technologies, Inc. VT82xxxxx UHCI USB 1....
00:10.1 USB Controller: VIA Technologies, Inc. VT82xxxxx UHCI USB 1....
...
```

B.2. interrupts

B.2.1. about interrupts

An **interrupt request** or **IRQ** is a request from a device to the CPU. A device raises an interrupt when it requires the attention of the CPU (could be because the device has data ready to be read by the CPU).

Since the introduction of pci, irq's can be shared among devices.

Interrupt 0 is always reserved for the timer, interrupt 1 for the keyboard. IRQ 2 is used as a channel for IRQ's 8 to 15, and thus is the same as IRQ 9.

B.2.2. /proc/interrupts

You can see a listing of interrupts on your system in `/proc/interrupts`.

```
paul@laika:~$ cat /proc/interrupts
      CPU0      CPU1
0:   1320048      555  IO-APIC-edge    timer
1:     10224        7  IO-APIC-edge    i8042
7:         0         0  IO-APIC-edge    parport0
8:         2         1  IO-APIC-edge    rtc
10:       3062      21  IO-APIC-fasteoi  acpi
12:        131        2  IO-APIC-edge    i8042
15:      47073        0  IO-APIC-edge    idel
18:         0         1  IO-APIC-fasteoi  yenta
19:      31056        1  IO-APIC-fasteoi  libata, ohci1394
20:      19042        1  IO-APIC-fasteoi  eth0
21:      44052        1  IO-APIC-fasteoi  uhci_hcd:usb1, uhci_hcd:usb2,...
22:     188352        1  IO-APIC-fasteoi  ra0
```

```
23:    632444    1 IO-APIC-fastecoi  nvidia
24:    1585    1 IO-APIC-fastecoi  VIA82XX-MODEM, VIA8237
```

B.2.3. dmesg

You can also use **dmesg** to find irq's allocated at boot time.

```
paul@laika:~$ dmesg | grep "irq 1[45]"
[ 28.930069] ata3: PATA max UDMA/133 cmd 0x1f0 ctl 0x3f6 bmdma 0x2090 irq 14
[ 28.930071] ata4: PATA max UDMA/133 cmd 0x170 ctl 0x376 bmdma 0x2098 irq 15
```

B.3. io ports

B.3.1. about io ports

Communication in the other direction, from CPU to device, happens through **IO ports**. The CPU writes data or control codes to the IO port of the device. But this is not only a one way communication, the CPU can also use a device's IO port to read status information about the device. Unlike interrupts, ports cannot be shared!

B.3.2. /proc/ioports

You can see a listing of your system's IO ports via **/proc/ioports**.

```
[root@RHEL4b ~]# cat /proc/ioports
0000-001f : dma1
0020-0021 : pic1
0040-0043 : timer0
0050-0053 : timer1
0060-006f : keyboard
0070-0077 : rtc
0080-008f : dma page reg
00a0-00a1 : pic2
00c0-00df : dma2
00f0-00ff : fpu
0170-0177 : ide1
02f8-02ff : serial
...
```

B.4. dma

B.4.1. about dma

A device that needs a lot of data, interrupts and ports can pose a heavy load on the cpu. With **dma** or **Direct Memory Access** a device can gain (temporary) access to a specific range of the **ram** memory.

B.4.2. /proc/dma

Looking at **/proc/dma** might not give you the information that you want, since it only contains currently assigned **dma** channels for **isa** devices.

```
root@laika:~# cat /proc/dma
1: parport0
4: cascade
```

pci devices that are using dma are not listed in **/proc/dma**, in this case **dmesg** can be useful. The screenshot below shows that during boot the parallel port received dma channel 1, and the Infrared port received dma channel 3.

```
root@laika:~# dmesg | egrep -C 1 'dma 1|dma 3'
[ 20.576000] parport: PnPBIOS parport detected.
[ 20.580000] parport0: PC-style at 0x378 (0x778), irq 7, dma 1...
[ 20.764000] irda_init()
--
[ 21.204000] pnp: Device 00:0b activated.
[ 21.204000] nsc_ircc_pnp_probe() : From PnP, found firbase 0x2F8...
[ 21.204000] nsc-ircc, chip->init
```

Appendix C. License

GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document "free" in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of "copyleft", which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The "Document", below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as "you". You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A "Modified Version" of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A "Secondary Section" is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document's overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The "Invariant Sections" are certain Secondary Sections whose titles

are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The "Cover Texts" are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A "Transparent" copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not "Transparent" is called "Opaque".

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The "Title Page" means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, "Title Page" means the text near the most prominent appearance of the work's title, preceding the beginning of the body of the text.

The "publisher" means any person or entity that distributes copies of the Document to the public.

A section "Entitled XYZ" means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as "Acknowledgements", "Dedications", "Endorsements", or "History".) To "Preserve the Title" of such a section when you modify the Document means that it remains a section "Entitled XYZ" according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either

commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- * A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.

* B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.

* C. State on the Title page the name of the publisher of the Modified Version, as the publisher.

* D. Preserve all the copyright notices of the Document.

* E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.

* F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.

* G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.

* H. Include an unaltered copy of this License.

* I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.

* J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.

* K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.

* L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.

* M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.

* N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.

* O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of,

you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements".

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an "aggregate" if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled "Acknowledgements", "Dedications", or "History", the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License "or any later version" applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies

that a proxy can decide which future versions of this License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

"Massive Multiauthor Collaboration Site" (or "MMC Site") means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A "Massive Multiauthor Collaboration" (or "MMC") contained in the site means any set of copyrightable works thus published on the MMC site.

"CC-BY-SA" means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

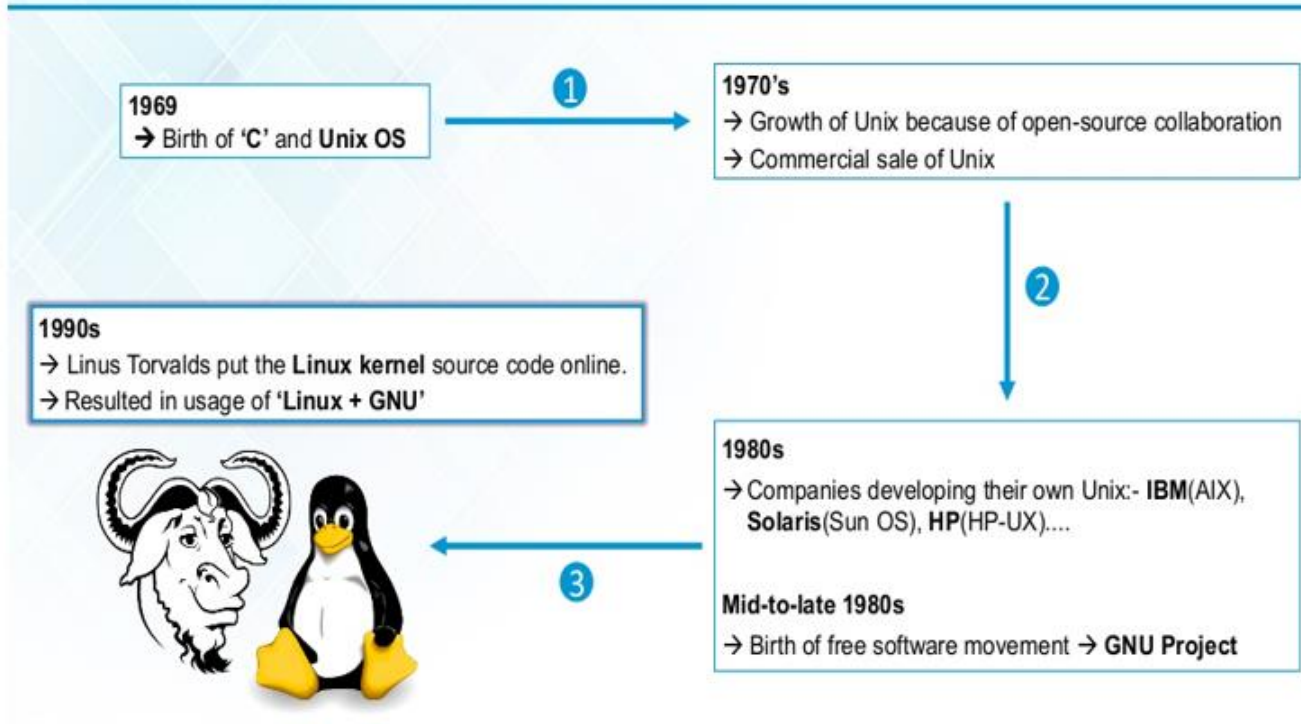
"Incorporate" means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is "eligible for relicensing" if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

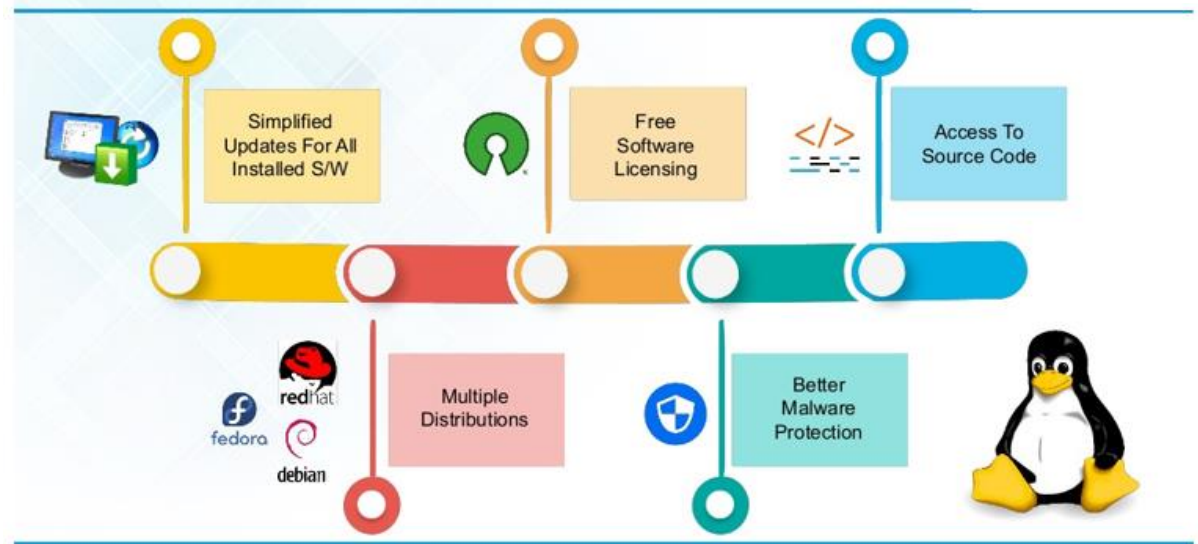
The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.

Linux Fundamentals for beginners

Birth Of Linux



Linux's Features



Why Use Linux

Why use Linux?

Some of the reasons to use Linux are:

- Low cost and very stable (some Linux servers are not rebooted for over a year, try that with Windows server!)
- Best computing power and inbuilt network support.
- Fastest developing OS, with the most number of developers.
- Most secure OS.
- Configurability
- Convenience
- freedom

Unix Vs Linux

Today Linux is in great demand. You can see the use of Linux everywhere. It's dominating on our servers, desktop, smartphones and even used in some electrical devices like refrigerators.

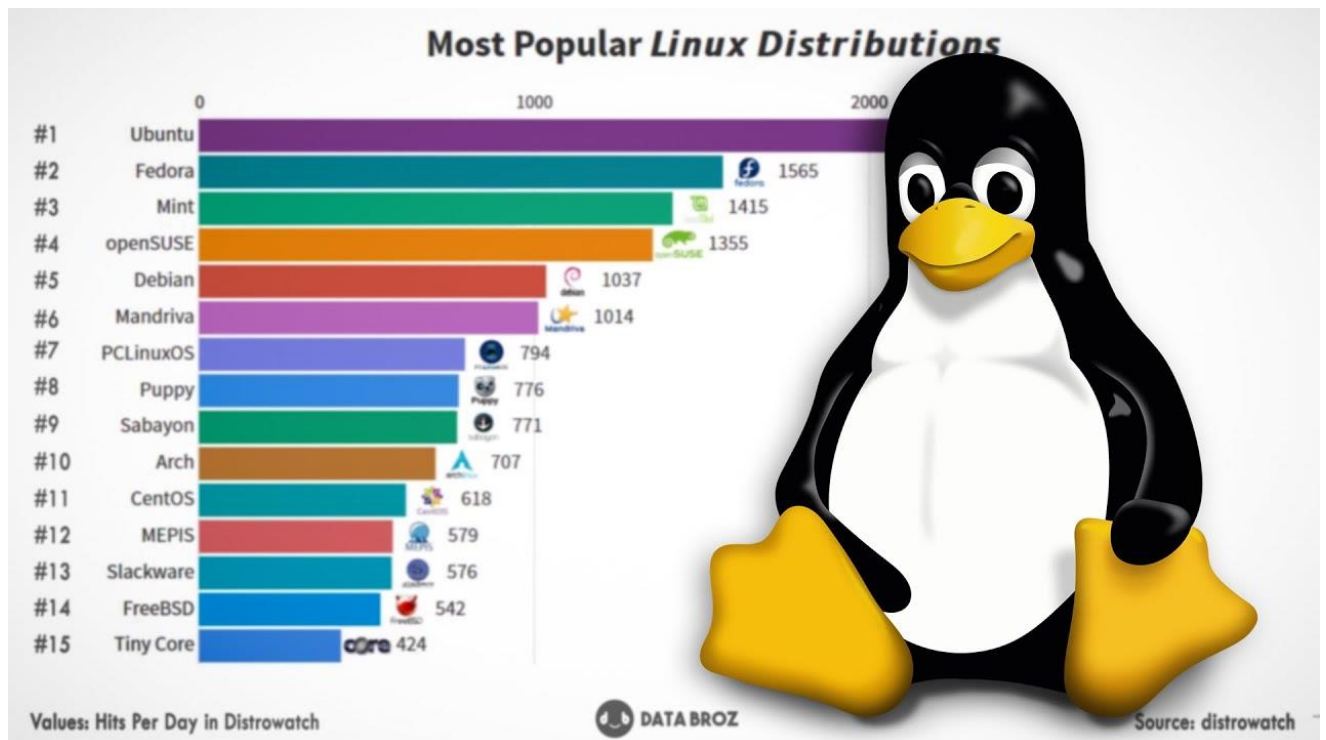
Some people think Unix and Linux as synonyms, but that's not true. Many operating systems were developed to be like Unix but none of them got the popularity as Linux. Linux is the clone of Unix. It has several features similar to Unix, still have some key differences. Before Linux and Windows, computer world was dominated by Unix. Unix is a copyrighted name and IBM AIX, HP-UX and Sun Solaris are only Unix operating system remained till date.

Difference between Linux and Unix

Comparison	Linux	Unix
Definition	It is an open-source operating system which is <i>freely available to everyone</i> .	It is an operating system which <i>can be only used by its copyrighters</i> .
Examples	It has different distros like Ubuntu, Redhat, Fedora, etc	IBM AIX, HP-UX and Sun Solaris.
Users	Nowadays, Linux is in great demand. Anyone can use Linux whether a home user, developer or a student.	It was developed mainly for servers, workstations and mainframes.

Usage	Linux is used everywhere from servers, PC, smartphones, tablets to mainframes and supercomputers.	It is used in servers, workstations and PCs.
Cost	Linux is freely distributed, downloaded, and distributed through magazines also. And priced distros of Linux are also cheaper than Windows.	Unix copyright vendors decide different costs for their respective Unix Operating systems.
Development	As it is open source, it is developed by sharing and collaboration of codes by world-wide developers.	Unix was developed by AT&T Labs, various commercial vendors and non-profit organizations.
Manufacturer	Linux kernel is developed by the community of developers from different parts of the world. Although the father of Linux, Linus Torvalds oversees things.	Unix has three distributions IBM AIX, HP-UX and Sun Solaris. Apple also uses Unix to make OSX operating system.
GUI	Linux is command based but some distros provide GUI based Linux. Gnome and KDE are mostly used GUI.	Initially it was command based OS, but later Common Desktop Environment was created. Most Unix distributions use Gnome.
Interface	The default interface is BASH (Bourne Again SHell). But some distros have developed their own interfaces.	It originally used Bourne shell. But is also compatible with other GUIs.
File system support	Linux supports more file system than Unix.	It also supports file system but lesser than Linux.
Coding	Linux is a Unix clone, behaves like Unix but doesn't contain its code.	Unix contain a completely different coding developed by AT&T Labs.
Operating system	Linux is just the kernel.	Unix is a complete package of Operating system.
Security	It provides higher security. Linux has about 60-100 viruses listed till date.	Unix is also highly secured. It has about 85-120 viruses listed till date
Error detection and solution	As Linux is open-source, whenever a user post any kind of threat, developers from all over the world start working on it. And hence, it provides faster solution.	In Unix, users have to wait for some time for the problem to be resolved.

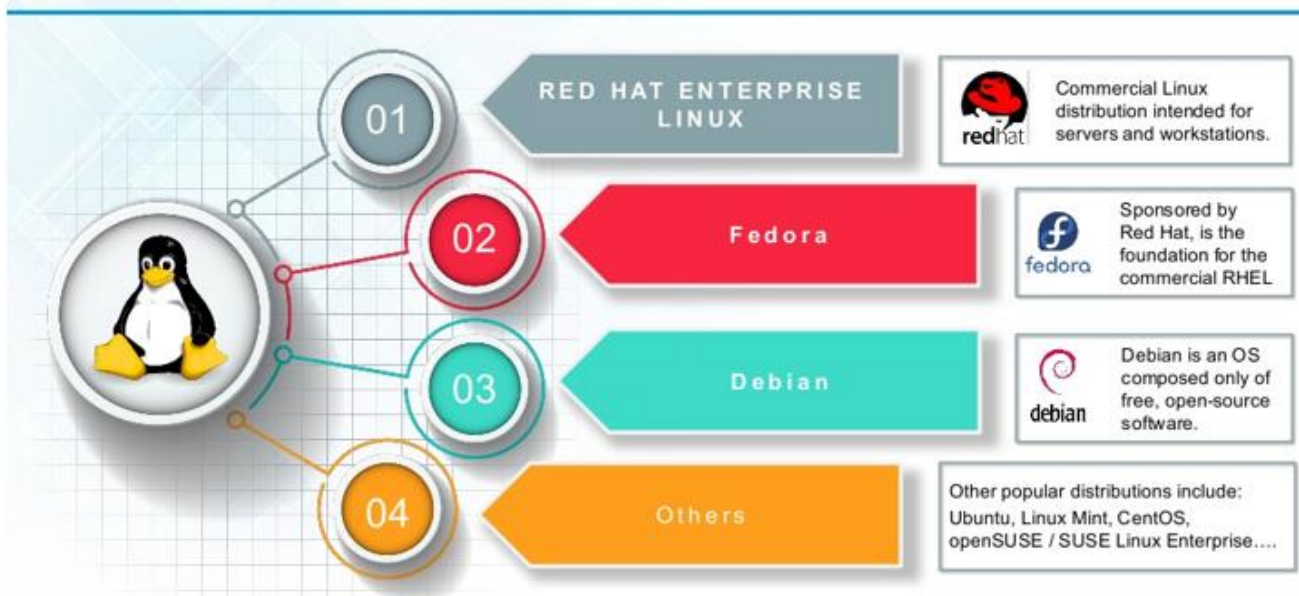
Linux Distributions (Distros)



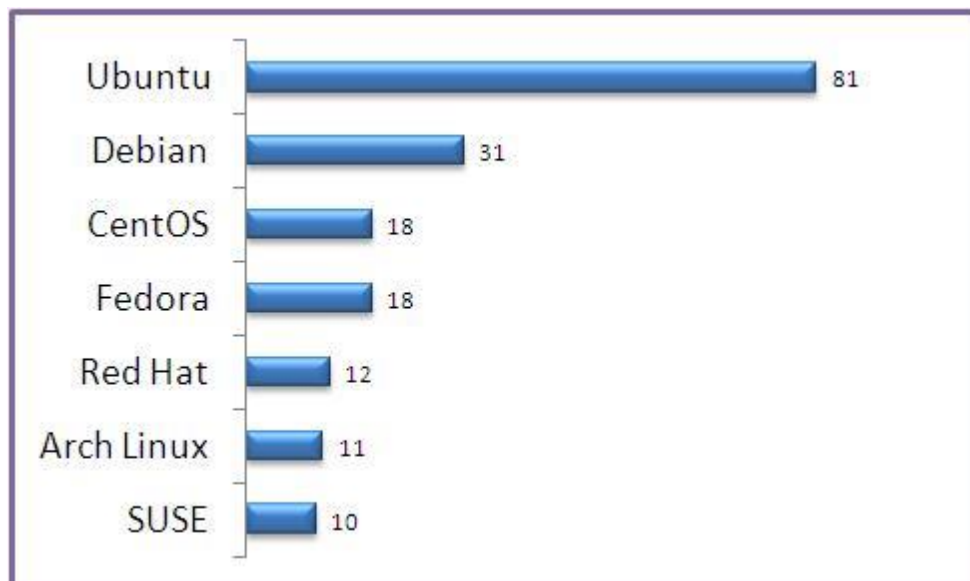
Choosing a Linux Distro

Distribution	Why To Use
UBuntu	It works like Mac OS and easy to use.
Linux mint	It works like windows and should be use by new comers.
Debian	It provides stability but not recommended to a new user.
Fedora	If you want to use red hat and latest software.
Red hat enterprise	To be used commercially.
CentOS	If you want to use red hat but without its trademark.
OpenSUSE	It works same as Fedora but slightly older and more stable.
Arch Linux	It is not for the beginners because every package has to be installed by yourself.

Various Linux Distributions



Top Linux Distribution



Chapter 1. man pages

This chapter will explain the use of **man** pages (also called **manual pages**) on your Unix or Linux computer.

You will learn the **man** command together with related commands like **whereis**, **whatis** and **mandb**.

Most Unix files and commands have pretty good man pages to explain their use. Man pages also come in handy when you are using multiple flavours of Unix or several Linux distributions since options and parameters sometimes vary.

1.1. man \$command

Type **man** followed by a command (for which you want help) and start reading. Press **q** to quit the manpage. Some man pages contain examples (near the end).

```
paul@laika:~$ man whois
Reformatting whois(1), please wait...
```

1.2. man \$configfile

Most **configuration files** have their own manual.

```
paul@laika:~$ man syslog.conf
Reformatting syslog.conf(5), please wait...
```

1.3. man \$daemon

This is also true for most **daemons** (background programs) on your system..

```
paul@laika:~$ man syslogd
Reformatting syslogd(8), please wait...
```

1.4. man -k (apropos)

man -k (or **apropos**) shows a list of man pages containing a string.

```
paul@laika:~$ man -k syslog
lm-syslog-setup (8) - configure laptop mode to switch syslog.conf ...
logger (1) - a shell command interface to the syslog(3) ...
syslog-facility (8) - Setup and remove LOCALx facility for sysklogd
syslog.conf (5) - syslogd(8) configuration file
syslogd (8) - Linux system logging utilities.
syslogd-listfiles (8) - list system logfiles
```

1.5. whatis

To see just the description of a manual page, use **whatis** followed by a string.

```
paul@u810:~$ whatis route
route (8) - show / manipulate the IP routing table
```

1.6. whereis

The location of a manpage can be revealed with **whereis**.

```
paul@laika:~$ whereis -m whois
whois: /usr/share/man/man1/whois.1.gz
```

This file is directly readable by **man**.

```
paul@laika:~$ man /usr/share/man/man1/whois.1.gz
```

1.7. man sections

By now you will have noticed the numbers between the round brackets. **man man** will explain to you that these are section numbers. Executable programs and shell commands reside in section one.

```
1 Executable programs or shell commands
2 System calls (functions provided by the kernel)
3 Library calls (functions within program libraries)
4 Special files (usually found in /dev)
5 File formats and conventions eg /etc/passwd
6 Games
7 Miscellaneous (including macro packages and conventions), e.g. man(7)
8 System administration commands (usually only for root)
9 Kernel routines [Non standard]
```

1.8. man \$section \$file

Therefor, when referring to the man page of the passwd command, you will see it written as **passwd(1)**; when referring to the **passwd file**, you will see it written as **passwd(5)**. The screenshot explains how to open the man page in the correct section.

```
[paul@RHEL52 ~]$ man passwd      # opens the first manual found
[paul@RHEL52 ~]$ man 5 passwd    # opens a page from section 5
```

1.9. man man

If you want to know more about **man**, then Read The Fantastic Manual (RTFM).

Unfortunately, manual pages do not have the answer to everything...

```
paul@laika:~$ man woman
No manual entry for woman
```

1.10. mandb

Should you be convinced that a man page exists, but you can't access it, then try running **mandb** on Debian/Mint.

```
root@laika:~# mandb
0 man subdirectories contained newer manual pages.
0 manual pages were added.
0 stray cats were added.
0 old database entries were purged.
```

Or run **makewhatis** on CentOS/Redhat.

```
[root@centos65 ~]# apropos scsi
scsi: nothing appropriate
[root@centos65 ~]# makewhatis
[root@centos65 ~]# apropos scsi
hpsa                (4)  - HP Smart Array SCSI driver
lsscsi              (8)  - list SCSI devices (or hosts) and their attributes
sd                  (4)  - Driver for SCSI Disk Drives
st                  (4)  - SCSI tape device
```

Chapter 2. working with directories

This module is a brief overview of the most common commands to work with directories: **pwd**, **cd**, **ls**, **mkdir** and **rmdir**. These commands are available on any Linux (or Unix) system.

This module also discusses **absolute** and **relative paths** and **path completion** in the **bash** shell.

2.1. pwd

The **you are here** sign can be displayed with the **pwd** command (Print Working Directory). Go ahead, try it: Open a command line interface (also called a terminal, console or xterm) and type **pwd**. The tool displays your **current directory**.

```
paul@debian8:~$ pwd
/home/paul
```

2.2. cd

You can change your current directory with the **cd** command (Change Directory).

```
paul@debian8$ cd /etc
paul@debian8$ pwd
/etc
paul@debian8$ cd /bin
paul@debian8$ pwd
/bin
paul@debian8$ cd /home/paul/
paul@debian8$ pwd
/home/paul
```

2.2.1. cd ~

The **cd** is also a shortcut to get back into your home directory. Just typing **cd** without a target directory, will put you in your home directory. Typing **cd ~** has the same effect.

```
paul@debian8$ cd /etc
paul@debian8$ pwd
/etc
paul@debian8$ cd
paul@debian8$ pwd
/home/paul
paul@debian8$ cd ~
paul@debian8$ pwd
/home/paul
```

2.2.2. cd ..

To go to the **parent directory** (the one just above your current directory in the directory tree), type **cd ..**.

```
paul@debian8$ pwd
/usr/share/games
paul@debian8$ cd ..
paul@debian8$ pwd
/usr/share
```

*To stay in the current directory, type **cd .** ;-)* We will see useful use of the **.** character representing the current directory later.

2.2.3. cd -

Another useful shortcut with **cd** is to just type **cd -** to go to the previous directory.

```
paul@debian8$ pwd
/home/paul
paul@debian8$ cd /etc
paul@debian8$ pwd
/etc
paul@debian8$ cd -
/home/paul
paul@debian8$ cd -
/etc
```

2.3. absolute and relative paths

You should be aware of **absolute** and **relative paths** in the file tree. When you type a path starting with a **slash (/)**, then the **root** of the file tree is assumed. If you don't start your path with a slash, then the current directory is the assumed starting point.

The screenshot below first shows the current directory **/home/paul**. From within this directory, you have to type **cd /home** instead of **cd home** to go to the **/home** directory.

```
paul@debian8$ pwd
/home/paul
paul@debian8$ cd home
bash: cd: home: No such file or directory
paul@debian8$ cd /home
paul@debian8$ pwd
/home
```

When inside **/home**, you have to type **cd paul** instead of **cd /paul** to enter the subdirectory **paul** of the current directory **/home**.

```
paul@debian8$ pwd
/home
paul@debian8$ cd /paul
bash: cd: /paul: No such file or directory
paul@debian8$ cd paul
paul@debian8$ pwd
/home/paul
```

In case your current directory is the **root directory /**, then both **cd /home** and **cd home** will get you in the **/home** directory.

```
paul@debian8$ pwd
/
paul@debian8$ cd home
paul@debian8$ pwd
/home
paul@debian8$ cd /
paul@debian8$ cd /home
paul@debian8$ pwd
/home
```

This was the last screenshot with **pwd** statements. From now on, the current directory will often be displayed in the prompt. Later in this book we will explain how the shell variable **\$PS1** can be configured to show this.

2.4. path completion

The **tab** key can help you in typing a path without errors. Typing **cd /et** followed by the **tab** key will expand the command line to **cd /etc/**. When typing **cd /Et** followed by the **tab** key, nothing will happen because you typed the wrong **path** (upper case E).

You will need fewer key strokes when using the **tab** key, and you will be sure your typed **path** is correct!

2.5. ls

You can list the contents of a directory with **ls**.

```
paul@debian8:~$ ls
allfiles.txt  dmesg.txt  services  stuff  summer.txt
paul@debian8:~$
```

2.5.1. ls -a

A frequently used option with **ls** is **-a** to show all files. Showing all files means including the **hidden files**. When a file name on a Linux file system starts with a dot, it is considered a **hidden file** and it doesn't show up in regular file listings.

```
paul@debian8:~$ ls
allfiles.txt  dmesg.txt  services  stuff  summer.txt
paul@debian8:~$ ls -a
.  allfiles.txt  .bash_profile  dmesg.txt  .lessht  stuff
.. .bash_history .bashrc        services   .ssh      summer.txt
paul@debian8:~$
```

2.5.2. ls -l

Many times you will be using options with **ls** to display the contents of the directory in different formats or to display different parts of the directory. Typing just **ls** gives you a list of files in the directory. Typing **ls -l** (that is a letter L, not the number 1) gives you a long listing.

```
paul@debian8:~$ ls -l
total 17296
-rw-r--r-- 1 paul paul 17584442 Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul   96650 Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul   19558 Sep 17 00:04 services
drwxr-xr-x 2 paul paul    4096 Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul      0 Sep 17 00:04 summer.txt
```

2.5.3. ls -lh

Another frequently used ls option is **-h**. It shows the numbers (file sizes) in a more human readable format. Also shown below is some variation in the way you can give the options to **ls**. We will explain the details of the output later in this book.

Note that we use the letter L as an option in this screenshot, not the number 1.

```
paul@debian8:~$ ls -l -h
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$ ls -lh
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$ ls -hl
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$ ls -h -l
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$
```

2.6. mkdir

Walking around the Unix file tree is fun, but it is even more fun to create your own directories with **mkdir**. You have to give at least one parameter to **mkdir**, the name of the new directory to be created. Think before you type a leading `/`.

```
paul@debian8:~$ mkdir mydir
paul@debian8:~$ cd mydir
paul@debian8:~/mydir$ ls -al
total 8
drwxr-xr-x  2 paul paul 4096 Sep 17 00:07 .
drwxr-xr-x 48 paul paul 4096 Sep 17 00:07 ..
paul@debian8:~/mydir$ mkdir stuff
paul@debian8:~/mydir$ mkdir otherstuff
paul@debian8:~/mydir$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 otherstuff
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 stuff
paul@debian8:~/mydir$
```

2.6.1. mkdir -p

The following command will fail, because the **parent directory** of **threedirsdeep** does not exist.

```
paul@debian8:~$ mkdir mydir2/mysubdir2/threedirsdeep
mkdir: cannot create directory 'mydir2/mysubdir2/threedirsdeep': No such fi\
le or directory
```

When given the option **-p**, then **mkdir** will create **parent directories** as needed.

```
paul@debian8:~$ mkdir -p mydir2/mysubdir2/threedirsdeep
paul@debian8:~$ cd mydir2
paul@debian8:~/mydir2$ ls -l
total 4
drwxr-xr-x 3 paul paul 4096 Sep 17 00:11 mysubdir2
paul@debian8:~/mydir2$ cd mysubdir2
paul@debian8:~/mydir2/mysubdir2$ ls -l
total 4
drwxr-xr-x 2 paul paul 4096 Sep 17 00:11 threedirsdeep
paul@debian8:~/mydir2/mysubdir2$ cd threedirsdeep/
paul@debian8:~/mydir2/mysubdir2/threedirsdeep$ pwd
/home/paul/mydir2/mysubdir2/threedirsdeep
```

2.7. rmdir

When a directory is empty, you can use **rmdir** to remove the directory.

```
paul@debian8:~/mydir$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 otherstuff
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 stuff
paul@debian8:~/mydir$ rmdir otherstuff
paul@debian8:~/mydir$ cd ..
paul@debian8:~$ rmdir mydir
rmdir: failed to remove 'mydir': Directory not empty
paul@debian8:~$ rmdir mydir/stuff
paul@debian8:~$ rmdir mydir
paul@debian8:~$
```

2.7.1. rmdir -p

And similar to the **mkdir -p** option, you can also use **rmdir** to recursively remove directories.

```
paul@debian8:~$ mkdir -p test42/subdir
paul@debian8:~$ rmdir -p test42/subdir
paul@debian8:~$
```

2.8. solution: working with directories

1. Display your current directory.

```
pwd
```

2. Change to the /etc directory.

```
cd /etc
```

3. Now change to your home directory using only three key presses.

```
cd (and the enter key)
```

4. Change to the /boot/grub directory using only eleven key presses.

```
cd /boot/grub (use the tab key)
```

5. Go to the parent directory of the current directory.

```
cd .. (with space between cd and ..)
```

6. Go to the root directory.

```
cd /
```

7. List the contents of the root directory.

```
ls
```

8. List a long listing of the root directory.

```
ls -l
```

9. Stay where you are, and list the contents of /etc.

```
ls /etc
```

10. Stay where you are, and list the contents of /bin and /sbin.

```
ls /bin /sbin
```

11. Stay where you are, and list the contents of ~.

```
ls ~
```

12. List all the files (including hidden files) in your home directory.

```
ls -al ~
```

13. List the files in /boot in a human readable format.

```
ls -lh /boot
```

14. Create a directory testdir in your home directory.

```
mkdir ~/testdir
```

15. Change to the /etc directory, stay here and create a directory newdir in your home directory.

```
cd /etc ; mkdir ~/newdir
```

16. Create in one command the directories ~/dir1/dir2/dir3 (dir3 is a subdirectory from dir2, and dir2 is a subdirectory from dir1).

```
mkdir -p ~/dir1/dir2/dir3
```

17. Remove the directory testdir.

```
rmdir testdir
```

18. If time permits (or if you are waiting for other students to finish this practice), use and understand **pushd** and **popd**. Use the man page of **bash** to find information about these commands.

```
man bash          # opens the manual
/pushd            # searches for pushd
n                 # next (do this two/three times)
```

The Bash shell has two built-in commands called **pushd** and **popd**. Both commands work with a common stack of previous directories. Pushd adds a directory to the stack and changes to a new current directory, popd removes a directory from the stack and sets the current directory.

```
paul@debian7:/etc$ cd /bin
paul@debian7:/bin$ pushd /lib
/lib /bin
paul@debian7:/lib$ pushd /proc
/proc /lib /bin
paul@debian7:/proc$ popd
/lib /bin
paul@debian7:/lib$ popd
/bin
```

Chapter 3. working with files

In this chapter we learn how to recognise, create, remove, copy and move files using commands like **file**, **touch**, **rm**, **cp**, **mv** and **rename**.

3.1. all files are case sensitive

Files on Linux (or any Unix) are **case sensitive**. This means that **FILE1** is different from **file1**, and **/etc/hosts** is different from **/etc/Hosts** (the latter one does not exist on a typical Linux computer).

This screenshot shows the difference between two files, one with upper case **W**, the other with lower case **w**.

```
paul@laika:~/Linux$ ls
winter.txt  Winter.txt
paul@laika:~/Linux$ cat winter.txt
It is cold.
paul@laika:~/Linux$ cat Winter.txt
It is very cold!
```

3.2. everything is a file

A **directory** is a special kind of **file**, but it is still a (case sensitive!) **file**. Each terminal window (for example **/dev/pts/4**), any hard disk or partition (for example **/dev/sdb1**) and any process are all represented somewhere in the **file system** as a **file**. It will become clear throughout this course that everything on Linux is a **file**.

3.3. file

The **file** utility determines the file type. Linux does not use extensions to determine the file type. The command line does not care whether a file ends in **.txt** or **.pdf**. As a system administrator, you should use the **file** command to determine the file type. Here are some examples on a typical Linux system.

```
paul@laika:~$ file pic33.png
pic33.png: PNG image data, 3840 x 1200, 8-bit/color RGBA, non-interlaced
paul@laika:~$ file /etc/passwd
/etc/passwd: ASCII text
paul@laika:~$ file HelloWorld.c
HelloWorld.c: ASCII C program text
```

The **file** command uses a magic file that contains patterns to recognise file types. The magic file is located in **/usr/share/file/magic**. Type **man 5 magic** for more information.

It is interesting to point out **file -s** for special files like those in **/dev** and **/proc**.

```
root@debian6~# file /dev/sda
/dev/sda: block special
root@debian6~# file -s /dev/sda
/dev/sda: x86 boot sector; partition 1: ID=0x83, active, starthead...
root@debian6~# file /proc/cpuinfo
/proc/cpuinfo: empty
root@debian6~# file -s /proc/cpuinfo
/proc/cpuinfo: ASCII C++ program text
```

3.4. touch

3.4.1. create an empty file

One easy way to create an empty file is with **touch**. (We will see many other ways for creating files later in this book.)

This screenshot starts with an empty directory, creates two files with **touch** and the lists those files.

```
paul@debian7:~$ ls -l
total 0
paul@debian7:~$ touch file42
paul@debian7:~$ touch file33
paul@debian7:~$ ls -l
total 0
-rw-r--r-- 1 paul paul 0 Oct 15 08:57 file33
-rw-r--r-- 1 paul paul 0 Oct 15 08:56 file42
paul@debian7:~$
```

3.4.2. touch -t

The **touch** command can set some properties while creating empty files. Can you determine what is set by looking at the next screenshot? If not, check the manual for **touch**.

```
paul@debian7:~$ touch -t 200505050000 SinkoDeMayo
paul@debian7:~$ touch -t 130207111630 BigBattle.txt
paul@debian7:~$ ls -l
total 0
-rw-r--r-- 1 paul paul 0 Jul 11 1302 BigBattle.txt
-rw-r--r-- 1 paul paul 0 Oct 15 08:57 file33
-rw-r--r-- 1 paul paul 0 Oct 15 08:56 file42
-rw-r--r-- 1 paul paul 0 May 5 2005 SinkoDeMayo
paul@debian7:~$
```

3.5. rm

3.5.1. remove forever

When you no longer need a file, use **rm** to remove it. Unlike some graphical user interfaces, the command line in general does not have a **waste bin** or **trash can** to recover files. When you use **rm** to remove a file, the file is gone. Therefore, be careful when removing files!

```
paul@debian7:~$ ls
BigBattle.txt  file33  file42  SinkoDeMayo
paul@debian7:~$ rm BigBattle.txt
paul@debian7:~$ ls
file33  file42  SinkoDeMayo
paul@debian7:~$
```

3.5.2. rm -i

To prevent yourself from accidentally removing a file, you can type **rm -i**.

```
paul@debian7:~$ ls
file33  file42  SinkoDeMayo
paul@debian7:~$ rm -i file33
rm: remove regular empty file `file33'? yes
paul@debian7:~$ rm -i SinkoDeMayo
rm: remove regular empty file `SinkoDeMayo'? n
paul@debian7:~$ ls
file42  SinkoDeMayo
paul@debian7:~$
```

3.5.3. rm -rf

By default, **rm -r** will not remove non-empty directories. However **rm** accepts several options that will allow you to remove any directory. The **rm -rf** statement is famous because it will erase anything (providing that you have the permissions to do so). When you are logged on as root, be very careful with **rm -rf** (the **f** means **force** and the **r** means **recursive**) since being root implies that permissions don't apply to you. You can literally erase your entire file system by accident.

```
paul@debian7:~$ mkdir test
paul@debian7:~$ rm test
rm: cannot remove `test': Is a directory
paul@debian7:~$ rm -rf test
paul@debian7:~$ ls test
ls: cannot access test: No such file or directory
paul@debian7:~$
```

3.6. cp

3.6.1. copy one file

To copy a file, use **cp** with a source and a target argument.

```
paul@debian7:~$ ls
file42 SinkoDeMayo
paul@debian7:~$ cp file42 file42.copy
paul@debian7:~$ ls
file42 file42.copy SinkoDeMayo
```

3.6.2. copy to another directory

If the target is a directory, then the source files are copied to that target directory.

```
paul@debian7:~$ mkdir dir42
paul@debian7:~$ cp SinkoDeMayo dir42
paul@debian7:~$ ls dir42/
SinkoDeMayo
```

3.6.3. cp -r

To copy complete directories, use **cp -r** (the **-r** option forces **recursive** copying of all files in all subdirectories).

```
paul@debian7:~$ ls
dir42 file42 file42.copy SinkoDeMayo
paul@debian7:~$ cp -r dir42/ dir33
paul@debian7:~$ ls
dir33 dir42 file42 file42.copy SinkoDeMayo
paul@debian7:~$ ls dir33/
SinkoDeMayo
```

3.6.4. copy multiple files to directory

You can also use **cp** to copy multiple files into a directory. In this case, the last argument (a.k.a. the target) must be a directory.

```
paul@debian7:~$ cp file42 file42.copy SinkoDeMayo dir42/
paul@debian7:~$ ls dir42/
file42 file42.copy SinkoDeMayo
```

3.6.5. cp -i

To prevent **cp** from overwriting existing files, use the **-i** (for interactive) option.

```
paul@debian7:~$ cp SinkoDeMayo file42
paul@debian7:~$ cp SinkoDeMayo file42
paul@debian7:~$ cp -i SinkoDeMayo file42
cp: overwrite `file42'? n
paul@debian7:~$
```

3.7. mv

3.7.1. rename files with mv

Use **mv** to rename a file or to move the file to another directory.

```
paul@debian7:~$ ls
dir33 dir42 file42 file42.copy SinkoDeMayo
paul@debian7:~$ mv file42 file33
paul@debian7:~$ ls
dir33 dir42 file33 file42.copy SinkoDeMayo
paul@debian7:~$
```

When you need to rename only one file then **mv** is the preferred command to use.

3.7.2. rename directories with mv

The same **mv** command can be used to rename directories.

```
paul@debian7:~$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 dir33
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 dir42
-rw-r--r-- 1 paul paul  0 Oct 15 09:38 file33
-rw-r--r-- 1 paul paul  0 Oct 15 09:16 file42.copy
-rw-r--r-- 1 paul paul  0 May  5  2005 SinkoDeMayo
paul@debian7:~$ mv dir33 backup
paul@debian7:~$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 backup
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 dir42
-rw-r--r-- 1 paul paul  0 Oct 15 09:38 file33
-rw-r--r-- 1 paul paul  0 Oct 15 09:16 file42.copy
-rw-r--r-- 1 paul paul  0 May  5  2005 SinkoDeMayo
paul@debian7:~$
```

3.7.3. mv -i

The **mv** also has a **-i** switch similar to **cp** and **rm**.

this screenshot shows that **mv -i** will ask permission to overwrite an existing file.

```
paul@debian7:~$ mv -i file33 SinkoDeMayo
mv: overwrite `SinkoDeMayo'? no
paul@debian7:~$
```

3.8. rename

3.8.1. about rename

The **rename** command is one of the rare occasions where the Linux Fundamentals book has to make a distinction between Linux distributions. Almost every command in the **Fundamentals** part of this book works on almost every Linux computer. But **rename** is different.

Try to use **mv** whenever you need to rename only a couple of files.

3.8.2. rename on Debian/Ubuntu

The **rename** command on Debian uses regular expressions (regular expression or short regex are explained in a later chapter) to rename many files at once.

Below a **rename** example that switches all occurrences of txt to png for all file names ending in .txt.

```
paul@debian7:~/test42$ ls
abc.txt file33.txt file42.txt
paul@debian7:~/test42$ rename 's/\.txt/\.png/' *.txt
paul@debian7:~/test42$ ls
abc.png file33.png file42.png
```

This second example switches all (first) occurrences of **file** into **document** for all file names ending in .png.

```
paul@debian7:~/test42$ ls
abc.png file33.png file42.png
paul@debian7:~/test42$ rename 's/file/document/' *.png
paul@debian7:~/test42$ ls
abc.png document33.png document42.png
paul@debian7:~/test42$
```

3.8.3. rename on CentOS/RHEL/Fedora

On Red Hat Enterprise Linux, the syntax of **rename** is a bit different. The first example below renames all *.conf files replacing any occurrence of .conf with .backup.

```
[paul@centos7 ~]$ touch one.conf two.conf three.conf
[paul@centos7 ~]$ rename .conf .backup *.conf
[paul@centos7 ~]$ ls
one.backup three.backup two.backup
[paul@centos7 ~]$
```

The second example renames all (*) files replacing one with ONE.

```
[paul@centos7 ~]$ ls
one.backup three.backup two.backup
[paul@centos7 ~]$ rename one ONE *
[paul@centos7 ~]$ ls
ONE.backup three.backup two.backup
[paul@centos7 ~]$
```


3.9. practice: working with files

1. List the files in the /bin directory
2. Display the type of file of /bin/cat, /etc/passwd and /usr/bin/passwd.
- 3a. Download wolf.jpg and LinuxFun.pdf from <http://linux-training.be> (wget http:// linux-training.be/files/studentfiles/wolf.jpg and wget http://linux-training.be/files/books/LinuxFun.pdf)

```
wget http://linux-training.be/files/studentfiles/wolf.jpg  
wget http://linux-training.be/files/studentfiles/wolf.png  
wget http://linux-training.be/files/books/LinuxFun.pdf
```

- 3b. Display the type of file of wolf.jpg and LinuxFun.pdf
- 3c. Rename wolf.jpg to wolf.pdf (use mv).
- 3d. Display the type of file of wolf.pdf and LinuxFun.pdf.
4. Create a directory ~/touched and enter it.
5. Create the files today.txt and yesterday.txt in touched.
6. Change the date on yesterday.txt to match yesterday's date.
7. Copy yesterday.txt to copy.yesterday.txt
8. Rename copy.yesterday.txt to kim
9. Create a directory called ~/testbackup and copy all files from ~/touched into it.
10. Use one command to remove the directory ~/testbackup and all files into it.
11. Create a directory ~/etcbackup and copy all *.conf files from /etc into it. Did you include all subdirectories of /etc ?
12. Use rename to rename all *.conf files to *.backup . (if you have more than one distro available, try it on all!)

3.10. solution: working with files

1. List the files in the /bin directory

```
ls /bin
```

2. Display the type of file of /bin/cat, /etc/passwd and /usr/bin/passwd.

```
file /bin/cat /etc/passwd /usr/bin/passwd
```

3a. Download wolf.jpg and LinuxFun.pdf from <http://linux-training.be> (wget [http:// linux-training.be/files/studentfiles/wolf.jpg](http://linux-training.be/files/studentfiles/wolf.jpg) and wget <http://linux-training.be/files/books/LinuxFun.pdf>)

```
wget http://linux-training.be/files/studentfiles/wolf.jpg
wget http://linux-training.be/files/studentfiles/wolf.png
wget http://linux-training.be/files/books/LinuxFun.pdf
```

3b. Display the type of file of wolf.jpg and LinuxFun.pdf

```
file wolf.jpg LinuxFun.pdf
```

3c. Rename wolf.jpg to wolf.pdf (use mv).

```
mv wolf.jpg wolf.pdf
```

3d. Display the type of file of wolf.pdf and LinuxFun.pdf.

```
file wolf.pdf LinuxFun.pdf
```

4. Create a directory ~/touched and enter it.

```
mkdir ~/touched ; cd ~/touched
```

5. Create the files today.txt and yesterday.txt in touched.

```
touch today.txt yesterday.txt
```

6. Change the date on yesterday.txt to match yesterday's date.

```
touch -t 200810251405 yesterday.txt (substitute 20081025 with yesterday)
```

7. Copy yesterday.txt to copy.yesterday.txt

```
cp yesterday.txt copy.yesterday.txt
```

8. Rename copy.yesterday.txt to kim

```
mv copy.yesterday.txt kim
```

9. Create a directory called ~/testbackup and copy all files from ~/touched into it.

```
mkdir ~/testbackup ; cp -r ~/touched ~/testbackup/
```

10. Use one command to remove the directory ~/testbackup and all files into it.

```
rm -rf ~/testbackup
```

11. Create a directory ~/etcbackup and copy all *.conf files from /etc into it. Did you include all subdirectories of /etc ?

```
cp -r /etc/*.conf ~/etcbackup
```

Only *.conf files that are directly in /etc/ are copied.

12. Use rename to rename all *.conf files to *.backup . (if you have more than one distro available, try it on all!)

```
On RHEL: touch 1.conf 2.conf ; rename conf backup *.conf
```

```
On Debian: touch 1.conf 2.conf ; rename 's/conf/backup/' *.conf
```

Chapter 4. working with file contents

In this chapter we will look at the contents of **text files** with **head**, **tail**, **cat**, **tac**, **more**, **less** and **strings**.

We will also get a glimpse of the possibilities of tools like **cat** on the command line.

4.1. head

You can use **head** to display the first ten lines of a file.

```
paul@debian7~$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
root@debian7~#
```

The **head** command can also display the first **n** lines of a file.

```
paul@debian7~$ head -4 /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
paul@debian7~$
```

And **head** can also display the first **n** bytes.

```
paul@debian7~$ head -c14 /etc/passwd
root:x:0:0:roopaul@debian7~$
```

4.2. tail

Similar to **head**, the **tail** command will display the last ten lines of a file.

```
paul@debian7~$ tail /etc/services
vboxd          20012/udp
binkp          24554/tcp      # binkp fidonet protocol
asp            27374/tcp      # Address Search Protocol
asp            27374/udp
csync2         30865/tcp      # cluster synchronization tool
dircproxy      57000/tcp      # Detachable IRC Proxy
tfido          60177/tcp      # fidonet EMSI over telnet
fido           60179/tcp      # fidonet EMSI over TCP

# Local services
paul@debian7~$
```

You can give **tail** the number of lines you want to see.

```
paul@debian7~$ tail -3 /etc/services
fido           60179/tcp      # fidonet EMSI over TCP

# Local services
paul@debian7~$
```

The **tail** command has other useful options, some of which we will use during this course.

4.3. cat

The **cat** command is one of the most universal tools, yet all it does is copy **standard input** to **standard output**. In combination with the shell this can be very powerful and diverse. Some examples will give a glimpse into the possibilities. The first example is simple, you can use **cat** to display a file on the screen. If the file is longer than the screen, it will scroll to the end.

```
paul@debian8:~$ cat /etc/resolv.conf
domain linux-training.be
search linux-training.be
nameserver 192.168.1.42
```

4.3.1. concatenate

cat is short for **concatenate**. One of the basic uses of **cat** is to concatenate files into a bigger (or complete) file.

```
paul@debian8:~$ echo one >part1
paul@debian8:~$ echo two >part2
paul@debian8:~$ echo three >part3
paul@debian8:~$ cat part1
one
paul@debian8:~$ cat part2
two
paul@debian8:~$ cat part3
three
paul@debian8:~$ cat part1 part2 part3
one
two
three
paul@debian8:~$ cat part1 part2 part3 >all
paul@debian8:~$ cat all
one
two
three
paul@debian8:~$
```

4.3.2. create files

You can use **cat** to create flat text files. Type the **cat > winter.txt** command as shown in the screenshot below. Then type one or more lines, finishing each line with the enter key. After the last line, type and hold the Control (Ctrl) key and press d.

```
paul@debian8:~$ cat > winter.txt
It is very cold today!
paul@debian8:~$ cat winter.txt
It is very cold today!
paul@debian8:~$
```

The **Ctrl d** key combination will send an **EOF** (End of File) to the running process ending the **cat** command.

4.3.3. custom end marker

You can choose an end marker for **cat** with **<<** as is shown in this screenshot. This construction is called a **here directive** and will end the **cat** command.

```
paul@debian8:~$ cat > hot.txt <<stop
> It is hot today!
> Yes it is summer.
> stop
paul@debian8:~$ cat hot.txt
It is hot today!
Yes it is summer.
paul@debian8:~$
```

4.3.4. copy files

In the third example you will see that **cat** can be used to copy files. We will explain in detail what happens here in the bash shell chapter.

```
paul@debian8:~$ cat winter.txt
It is very cold today!
paul@debian8:~$ cat winter.txt > cold.txt
paul@debian8:~$ cat cold.txt
It is very cold today!
paul@debian8:~$
```

4.4. tac

Just one example will show you the purpose of **tac** (cat backwards).

```
paul@debian8:~$ cat count
one
two
three
four
paul@debian8:~$ tac count
four
three
two
one
```

4.5. more and less

The **more** command is useful for displaying files that take up more than one screen. More will allow you to see the contents of the file page by page. Use the space bar to see the next page, or **q** to quit. Some people prefer the **less** command to **more**.

4.6. strings

With the **strings** command you can display readable ascii strings found in (binary) files. This example locates the **ls** binary then displays readable strings in the binary file (output is truncated).

```
paul@laika:~$ which ls
/bin/ls
paul@laika:~$ strings /bin/ls
/lib/ld-linux.so.2
librt.so.1
__gmon_start__
__Jv_RegisterClasses
clock_gettime
libacl.so.1
...
```

4.7. solution: file contents

1. Display the first 12 lines of **/etc/services**.

```
head -12 /etc/services
```

2. Display the last line of **/etc/passwd**.

```
tail -1 /etc/passwd
```

3. Use **cat** to create a file named **count.txt** that looks like this:

```
cat > count.txt
One
Two
Three
Four
Five (followed by Ctrl-d)
```

4. Use **cp** to make a backup of this file to **cnt.txt**.

```
cp count.txt cnt.txt
```

5. Use **cat** to make a backup of this file to **catcnt.txt**.

```
cat count.txt > catcnt.txt
```

6. Display **catcnt.txt**, but with all lines in reverse order (the last line first).

```
tac catcnt.txt
```

7. Use **more** to display **/etc/services**.


```
more /etc/services
```

8. Display the readable character strings from the **/usr/bin/passwd** command.

```
strings /usr/bin/passwd
```

9. Use **ls** to find the biggest file in **/etc**.

```
ls -lrs /etc
```

10. Open two terminal windows (or tabs) and make sure you are in the same directory in both. Type **echo this is the first line > tailing.txt** in the first terminal, then issue **tail -f tailing.txt** in the second terminal. Now go back to the first terminal and type **echo This is another line >> tailing.txt** (note the double >>), verify that the **tail -f** in the second terminal shows both lines. Stop the **tail -f** with **Ctrl-C**.

11. Use **cat** to create a file named **tailing.txt** that contains the contents of **tailing.txt** followed by the contents of **/etc/passwd**.

```
cat /etc/passwd >> tailing.txt
```

12. Use **cat** to create a file named **tailing.txt** that contains the contents of **tailing.txt** preceded by the contents of **/etc/passwd**.

```
mv tailing.txt tmp.txt ; cat /etc/passwd tmp.txt > tailing.txt
```

Chapter 5. the Linux file tree

This chapter takes a look at the most common directories in the **Linux file tree**. It also shows that on Unix everything is a file.

5.1. filesystem hierarchy standard

Many Linux distributions partially follow the **Filesystem Hierarchy Standard**. The **FHS** may help make more Unix/Linux file system trees conform better in the future. The **FHS** is available online at <http://www.pathname.com/fhs/> where we read: "The filesystem hierarchy standard has been designed to be used by Unix distribution developers, package developers, and system implementers. However, it is primarily intended to be a reference and is not a tutorial on how to manage a Unix filesystem or directory hierarchy."

5.2. man hier

There are some differences in the filesystems between **Linux distributions**. For help about your machine, enter **man hier** to find information about the file system hierarchy. This manual will explain the directory structure on your computer.

5.3. the root directory /

All Linux systems have a directory structure that starts at the **root directory**. The root directory is represented by a **forward slash**, like this: /. Everything that exists on your Linux system can be found below this root directory. Let's take a brief look at the contents of the root directory.

```
[paul@RHELv4u3 ~]$ ls /  
bin  dev  home  media  mnt  proc  sbin  srv  tftpboot  usr  
boot  etc  lib  misc  opt  root  selinux  sys  tmp  var
```

5.4. binary directories

Binaries are files that contain compiled source code (or machine code). Binaries can be **executed** on the computer. Sometimes binaries are called **executables**.

5.4.1. /bin

The **/bin** directory contains **binaries** for use by all users. According to the FHS the **/bin** directory should contain **/bin/cat** and **/bin/date** (among others).

In the screenshot below you see common Unix/Linux commands like cat, cp, cpio, date, dd, echo, grep, and so on. Many of these will be covered in this book.

```
paul@laika:~$ ls /bin
archdetect      egrep            mt               setupcon
autopartition   false            mt-gnu           sh
bash            fgconsole        mv               sh.distrib
bunzip2          fgrep            nano             sleep
bzipcat          fuser            nc               stralign
bzipcmp          fusermount       nc.traditional  stty
bzdiff           get_mouptoptions netcat            su
bzegrep          grep             netstat          sync
bzexe            gunzip           ntfs-3g           sysfs
bzfgrep          gzexe            ntfs-3g.probe    tailf
bzgrep           gzip             parted_devices   tar
bzip2            hostname         parted_server     tempfile
bzip2recover     hw-detect        partman           touch
bzless           ip               partman-commit    true
bzmore           kbd_mode         perform_recipe    ulockmgr
cat              kill             pidof             umount
...
```

5.4.2. other /bin directories

You can find a **/bin subdirectory** in many other directories. A user named **serena** could put her own programs in **/home/serena/bin**.

Some applications, often when installed directly from source will put themselves in **/opt**. A **samba server** installation can use **/opt/samba/bin** to store its binaries.

5.4.3. /sbin

/sbin contains binaries to configure the operating system. Many of the **system binaries** require **root** privilege to perform certain tasks.

Below a screenshot containing **system binaries** to change the ip address, partition a disk and create an ext4 file system.

```
paul@ubu1010:~$ ls -l /sbin/ifconfig /sbin/fdisk /sbin/mkfs.ext4
-rwxr-xr-x 1 root root 97172 2011-02-02 09:56 /sbin/fdisk
-rwxr-xr-x 1 root root 65708 2010-07-02 09:27 /sbin/ifconfig
-rwxr-xr-x 5 root root 55140 2010-08-18 18:01 /sbin/mkfs.ext4
```

5.4.4. /lib

Binaries found in **/bin** and **/sbin** often use **shared libraries** located in **/lib**. Below is a screenshot of the partial contents of **/lib**.

```
paul@laika:~$ ls /lib/libc*
/lib/libc-2.5.so      /lib/libcfont.so.0.0.0  /lib/libcom_err.so.2.1
/lib/libcap.so.1      /lib/libcidn-2.5.so     /lib/libconsole.so.0
/lib/libcap.so.1.10   /lib/libcidn.so.1       /lib/libconsole.so.0.0.0
/lib/libcfont.so.0    /lib/libcom_err.so.2    /lib/libcrypt-2.5.so
```

/lib/modules

Typically, the **Linux kernel** loads kernel modules from **/lib/modules/\$kernel-version/**. This directory is discussed in detail in the Linux kernel chapter.

/lib32 and /lib64

We currently are in a transition between **32-bit** and **64-bit** systems. Therefore, you may encounter directories named **/lib32** and **/lib64** which clarify the register size used during compilation time of the libraries. A 64-bit computer may have some 32-bit binaries and libraries for compatibility with legacy applications. This screenshot uses the **file** utility to demonstrate the difference.

```
paul@laika:~$ file /lib32/libc-2.5.so
/lib32/libc-2.5.so: ELF 32-bit LSB shared object, Intel 80386, \
version 1 (SYSV), for GNU/Linux 2.6.0, stripped
paul@laika:~$ file /lib64/libcap.so.1.10
/lib64/libcap.so.1.10: ELF 64-bit LSB shared object, AMD x86-64, \
version 1 (SYSV), stripped
```

The ELF (**Executable and Linkable Format**) is used in almost every Unix-like operating system since **System V**.

5.4.5. /opt

The purpose of **/opt** is to store **optional** software. In many cases this is software from outside the distribution repository. You may find an empty **/opt** directory on many systems.

A large package can install all its files in **/bin**, **/lib**, **/etc** subdirectories within **/opt/\$packagename/**. If for example the package is called **wp**, then it installs in **/opt/wp**, putting binaries in **/opt/wp/bin** and manpages in **/opt/wp/man**.

5.5. configuration directories

5.5.1. /boot

The **/boot** directory contains all files needed to boot the computer. These files don't change very often. On Linux systems you typically find the **/boot/grub** directory here. **/boot/grub** contains **/boot/grub/grub.cfg** (older systems may still have **/boot/grub/grub.conf**) which defines the boot menu that is displayed before the kernel starts.

5.5.2. /etc

All of the machine-specific **configuration files** should be located in **/etc**. Historically **/etc** stood for **etcetera**, today people often use the **Editable Text Configuration** backronym.

Many times the name of a configuration files is the same as the application, daemon, or protocol with **.conf** added as the extension.

```
paul@laika:~$ ls /etc/*.conf
/etc/adduser.conf      /etc/ld.so.conf      /etc/scrollkeeper.conf
/etc/brltty.conf       /etc/lftp.conf       /etc/sysctl.conf
/etc/ccertificates.conf /etc/libao.conf      /etc/syslog.conf
/etc/cvs-cron.conf     /etc/logrotate.conf  /etc/ucf.conf
/etc/ddclient.conf     /etc/ltrace.conf     /etc/uniconf.conf
/etc/debconf.conf      /etc/mke2fs.conf     /etc/updatedb.conf
/etc/deluser.conf      /etc/netscsid.conf   /etc/usplash.conf
/etc/fdmount.conf      /etc/nsswitch.conf   /etc/uswsusp.conf
/etc/hdparm.conf       /etc/pam.conf        /etc/vnc.conf
/etc/host.conf         /etc/pnm2ppa.conf    /etc/wodim.conf
/etc/inetd.conf        /etc/povray.conf     /etc/wvdial.conf
/etc/kernel-img.conf   /etc/resolv.conf
paul@laika:~$
```

There is much more to be found in **/etc**.

/etc/init.d/

A lot of Unix/Linux distributions have an **/etc/init.d** directory that contains scripts to start and stop **daemons**. This directory could disappear as Linux migrates to systems that replace the old **init** way of starting all **daemons**.

/etc/X11/

The graphical display (aka **X Window System** or just **X**) is driven by software from the X.org foundation. The configuration file for your graphical display is **/etc/X11/xorg.conf**.

/etc/skel/

The **skeleton** directory **/etc/skel** is copied to the home directory of a newly created user. It usually contains hidden files like a **.bashrc** script.

/etc/sysconfig/

This directory, which is not mentioned in the FHS, contains a lot of **Red Hat Enterprise Linux** configuration files. We will discuss some of them in greater detail. The screenshot below is the **/etc/sysconfig** directory from RHELv4u4 with everything installed.

```
paul@RHELv4u4:~$ ls /etc/sysconfig/
apmd          firstboot    irda         network      saslauthd
apm-scripts   grub         irqbalance   networking   selinux
authconfig    hidd         keyboard     ntpd         spamassassin
autofs        httpd        kudzu        openib.conf  squid
bluetooth     hwconf       lm_sensors   pand          syslog
clock         il8n         mouse        pcmcia        sys-config-sec
console       init         mouse.B      pgsql         sys-config-users
crond         installinfo  named        prelink       sys-logviewer
desktop       ipmi         netdump      rawdevices   tux
diskdump      iptables     netdump_id_dsa  rhn          vncservers
dund          iptables-cfg netdump_id_dsa.p samba         xinetd
paul@RHELv4u4:~$
```

The file **/etc/sysconfig/firstboot** tells the Red Hat Setup Agent not to run at boot time. If you want to run the Red Hat Setup Agent at the next reboot, then simply remove this file, and run **chkconfig --level 5 firstboot on**. The Red Hat Setup Agent allows you to install the latest updates, create a user account, join the Red Hat Network and more. It will then create the **/etc/sysconfig/firstboot** file again.

```
paul@RHELv4u4:~$ cat /etc/sysconfig/firstboot
RUN_FIRSTBOOT=NO
```

The **/etc/sysconfig/harddisks** file contains some parameters to tune the hard disks. The file explains itself.

You can see hardware detected by **kudzu** in **/etc/sysconfig/hwconf**. Kudzu is software from Red Hat for automatic discovery and configuration of hardware.

The keyboard type and keymap table are set in the **/etc/sysconfig/keyboard** file. For more console keyboard information, check the manual pages of **keymaps(5)**, **dumpkeys(1)**, **loadkeys(1)** and the directory **/lib/kbd/keymaps/**.

```
root@RHELv4u4:/etc/sysconfig# cat keyboard
KEYBOARDTYPE="pc"
KEYTABLE="us"
```

We will discuss networking files in this directory in the networking chapter.

5.6. data directories

5.6.1. /home

Users can store personal or project data under **/home**. It is common (but not mandatory by the fhs) practice to name the users home directory after the user name in the format **/home/\$USERNAME**. For example:

```
paul@ubu606:~$ ls /home
geert  annik  sandra  paul  tom
```

Besides giving every user (or every project or group) a location to store personal files, the home directory of a user also serves as a location to store the user profile. A typical Unix user profile contains many hidden files (files whose file name starts with a dot). The hidden files of the Unix user profiles contain settings specific for that user.

```
paul@ubu606:~$ ls -d /home/paul/.*
/home/paul/.          /home/paul/.bash profile  /home/paul/.ssh
/home/paul/..         /home/paul/.bashrc       /home/paul/.viminfo
/home/paul/.bash history /home/paul/.lessht
```

5.6.2. /root

On many systems **/root** is the default location for personal data and profile of the **root user**. If it does not exist by default, then some administrators create it.

5.6.3. /srv

You may use **/srv** for data that is **served by your system**. The FHS allows locating cvs, rsync, ftp and www data in this location. The FHS also approves administrative naming in /srv, like /srv/project55/ftp and /srv/sales/www.

On Sun Solaris (or Oracle Solaris) **/export** is used for this purpose.

5.6.4. /media

The **/media** directory serves as a mount point for **removable media devices** such as CD-ROM's, digital cameras, and various usb-attached devices. Since **/media** is rather new in the Unix world, you could very well encounter systems running without this directory. Solaris 9 does not have it, Solaris 10 does. Most Linux distributions today mount all removable media in **/media**.

```
paul@debian5:~$ ls /media/
cdrom  cdrom0  usbdisk
```

5.6.5. /mnt

The **/mnt** directory should be empty and should only be used for temporary mount points (according to the FHS).

Unix and Linux administrators used to create many directories here, like /mnt/something/. You likely will encounter many systems with more than one directory created and/or mounted inside **/mnt** to be used for various local and remote filesystems.

5.6.6. /tmp

Applications and users should use **/tmp** to store temporary data when needed. Data stored in **/tmp** may use either disk space or RAM. Both of which are managed by the operating system. Never use **/tmp** to store data that is important or which you wish to archive.

5.7. in memory directories

5.7.1. /dev

Device files in **/dev** appear to be ordinary files, but are not actually located on the hard disk. The **/dev** directory is populated with files as the kernel is recognising hardware.

common physical devices

Common hardware such as hard disk devices are represented by device files in **/dev**. Below a screenshot of SATA device files on a laptop and then IDE attached drives on a desktop. (The detailed meaning of these devices will be discussed later.)

```
#
# SATA or SCSI or USB
#
paul@laika:~$ ls /dev/sd*
/dev/sda /dev/sda1 /dev/sda2 /dev/sda3 /dev/sdb /dev/sdb1 /dev/sdb2

#
# IDE or ATAPI
#
paul@barry:~$ ls /dev/hd*
/dev/hda /dev/hda1 /dev/hda2 /dev/hdb /dev/hdb1 /dev/hdb2 /dev/hdc
```

Besides representing physical hardware, some device files are special. These special devices can be very useful.

/dev/tty and /dev/pts

For example, **/dev/tty1** represents a terminal or console attached to the system. (Don't break your head on the exact terminology of 'terminal' or 'console', what we mean here is a command line interface.) When typing commands in a terminal that is part of a graphical interface like Gnome or KDE, then your terminal will be represented as **/dev/pts/1** (1 can be another number).

/dev/null

On Linux you will find other special devices such as **/dev/null** which can be considered a black hole; it has unlimited storage, but nothing can be retrieved from it. Technically speaking, anything written to **/dev/null** will be discarded. **/dev/null** can be useful to discard unwanted output from commands. */dev/null is not a good location to store your backups ;-).*

5.7.2. /proc conversation with the kernel

/proc is another special directory, appearing to be ordinary files, but not taking up disk space. It is actually a view of the kernel, or better, what the kernel manages, and is a means to interact with it directly. **/proc** is a proc filesystem.

```
paul@RHELv4u4:~$ mount -t proc
```

```
none on /proc type proc (rw)
```

When listing the /proc directory you will see many numbers (on any Unix) and some interesting files (on Linux)

```
mul@laika:~$ ls /proc
1          2339    4724    5418    6587    7201      cmdline    mounts
10175      2523    4729    5421    6596    7204      cpuinfo     mtrr
10211      2783    4741    5658    6599    7206      crypto      net
10239      2975    4873    5661    6638    7214      devices     pagetypeinfo
141        29775    4874    5665    6652    7216      diskstats   partitions
15045      29792    4878    5927    6719    7218      dma         sched_debug
1519       2997    4879    6      6736    7223      driver      scsi
1548       3      4881    6032    6737    7224      execdomains self
1551       30228  4882    6033    6755    7227      fb          slabinfo
1554       3069    5      6145    6762    7260      filesystems stat
1557       31422  5073    6298    6774    7267      fs          swaps
1606       3149    5147    6414    6816    7275      ide         sys
180        31507    5203    6418    6991    7282      interrupts  sysrq-trigger
181        3189    5206    6419    6993    7298      iomem       sysvipc
182        3193    5228    6420    6996    7319      ioports     timer_list
18898      3246    5272    6421    7157    7330      irq         timer_stats
19799      3248    5291    6422    7163    7345      kallsyms    tty
19803      3253    5294    6423    7164    7513      kcore       uptime
19804      3372    5356    6424    7171    7525      key-users   version
1987       4      5370    6425    7175    7529      kmsg        version_signature
1989       42     5379    6426    7188    9964      loadavg     vmcore
2          45     5380    6430    7189      locks       vmnet
20845      4542    5412    6450    7191    asound      meminfo     vmstat
221        46     5414    6551    7192    buddyinfo   misc        zoneinfo
2338      4704    5416    6568    7199    bus         modules
```

Let's investigate the file properties inside **/proc**. Looking at the date and time will display the current date and time showing the files are constantly updated (a view on the kernel).

```
paul@RHELv4u4:~$ date
Mon Jan 29 18:06:32 EST 2007
paul@RHELv4u4:~$ ls -al /proc/cpuinfo
-r--r--r-- 1 root root 0 Jan 29 18:06 /proc/cpuinfo
paul@RHELv4u4:~$
paul@RHELv4u4:~$ ...time passes...
paul@RHELv4u4:~$
paul@RHELv4u4:~$ date
Mon Jan 29 18:10:00 EST 2007
paul@RHELv4u4:~$ ls -al /proc/cpuinfo
-r--r--r-- 1 root root 0 Jan 29 18:10 /proc/cpuinfo
```

Most files in /proc are 0 bytes, yet they contain data--sometimes a lot of data. You can see this by executing cat on files like **/proc/cpuinfo**, which contains information about the CPU.

```
paul@RHELv4u4:~$ file /proc/cpuinfo
/proc/cpuinfo: empty
paul@RHELv4u4:~$ cat /proc/cpuinfo
processor       : 0
vendor_id      : AuthenticAMD
cpu family     : 15
model          : 43
```

```
model name      : AMD Athlon(tm) 64 X2 Dual Core Processor 4600+
stepping       : 1
cpu MHz        : 2398.628
cache size     : 512 KB
fdiv_bug      : no
hlt_bug       : no
f00f_bug      : no
coma_bug      : no
fpu           : yes
fpu_exception  : yes
cpuid level    : 1
wp            : yes
flags         : fpu vme de pse tsc msr pae mce cx8 apic mtrr pge...
bogomips      : 4803.54
```

Just for fun, here is /proc/cpuinfo on a Sun Sunblade 1000...

```
paul@pasha:~$ cat /proc/cpuinfo
cpu : TI UltraSparc III (Cheetah)
fpu : UltraSparc III integrated FPU
promlib : Version 3 Revision 2
prom : 4.2.2
type : sun4u
ncpus probed : 2
ncpus active : 2
Cpu0Bogo : 498.68
Cpu0ClkTck : 000000002cb41780
Cpu1Bogo : 498.68
Cpu1ClkTck : 000000002cb41780
MMU Type : Cheetah
State:
CPU0: online
CPU1: online
```

Most of the files in /proc are read only, some require root privileges, some files are writable, and many files in **/proc/sys** are writable. Let's discuss some of the files in /proc.

/proc/interrupts

On the x86 architecture, **/proc/interrupts** displays the interrupts.

```
paul@RHELv4u4:~$ cat /proc/interrupts
CPU0
 0:   13876877    IO-APIC-edge  timer
 1:         15    IO-APIC-edge  i8042
 8:          1    IO-APIC-edge  rtc
 9:          0    IO-APIC-level  acpi
12:          67    IO-APIC-edge  i8042
14:         128    IO-APIC-edge  ide0
15:       124320    IO-APIC-edge  ide1
169:       111993    IO-APIC-level  ioc0
177:        2428    IO-APIC-level  eth0
NMI:          0
LOC:       13878037
ERR:          0
MIS:          0
```

On a machine with two CPU's, the file looks like this.

```
paul@laika:~$ cat /proc/interrupts
CPU0      CPU1
 0:   860013      0    IO-APIC-edge  timer
 1:    4533      0    IO-APIC-edge  i8042
 7:         0      0    IO-APIC-edge  parport0
 8:   6588227     0    IO-APIC-edge  rtc
10:    2314      0    IO-APIC-fasteoi  acpi
12:    133       0    IO-APIC-edge  i8042
14:         0      0    IO-APIC-edge  libata
15:    72269     0    IO-APIC-edge  libata
18:         1      0    IO-APIC-fasteoi  yenta
19:   115036     0    IO-APIC-fasteoi  eth0
20:   126871     0    IO-APIC-fasteoi  libata, ohci1394
21:    30204     0    IO-APIC-fasteoi  ehci_hcd:usb1, uhci_hcd:usb2
22:    1334      0    IO-APIC-fasteoi  saa7133[0], saa7133[0]
24:   234739     0    IO-APIC-fasteoi  nvidia
NMI:         72      42
LOC:   860000    859994
ERR:         0
```

/proc/kcore

The physical memory is represented in **/proc/kcore**. Do not try to cat this file, instead use a debugger. The size of **/proc/kcore** is the same as your physical memory, plus four bytes.

```
paul@laika:~$ ls -lh /proc/kcore
-r----- 1 root root 2.0G 2007-01-30 08:57 /proc/kcore
paul@laika:~$
```

5.7.3. /sys Linux 2.6 hot plugging

The **/sys** directory was created for the Linux 2.6 kernel. Since 2.6, Linux uses **sysfs** to support **usb** and **IEEE 1394 (FireWire)** hot plug devices. See the manual pages of **udev(8)** (the successor of **devfs**) and **hotplug(8)** for more info (or visit <http://linux-hotplug.sourceforge.net/>).

Basically the **/sys** directory contains kernel information about hardware.

5.8. /usr Unix System Resources

Although **/usr** is pronounced like user, remember that it stands for **Unix System Resources**. The **/usr** hierarchy should contain **shareable, read only** data. Some people choose to mount **/usr** as read only. This can be done from its own partition or from a read only NFS share (NFS is discussed later).

5.8.1. /usr/bin

The **/usr/bin** directory contains a lot of commands.

```
paul@deb508:~$ ls /usr/bin | wc -l
1395
```

(On Solaris the **/bin** directory is a symbolic link to **/usr/bin**.)

5.8.2. /usr/include

The **/usr/include** directory contains general use include files for C.

```
paul@ubu1010:~$ ls /usr/include/
aolib.h      expat_config.h  math.h       search.h
af_vfs.h     expat_external.h mcheck.h     semaphore.h
aio.h        expat.h         memory.h     setjmp.h
AL           fcntl.h        menu.h       sgTTY.h
aliases.h    features.h      mntent.h     shadow.h
...
```

5.8.3. /usr/lib

The **/usr/lib** directory contains libraries that are not directly executed by users or scripts.

```
paul@deb508:~$ ls /usr/lib | head -7
4Suite
ao
apt
arj
aspell
avahi
bonobo
```

5.8.4. /usr/local

The **/usr/local** directory can be used by an administrator to install software locally.

```
paul@deb508:~$ ls /usr/local/
bin  etc  games  include  lib  man  sbin  share  src
paul@deb508:~$ du -sh /usr/local/
128K /usr/local/
```

5.8.5. /usr/share

The **/usr/share** directory contains architecture independent data. As you can see, this is a fairly large directory.

```
paul@deb508:~$ ls /usr/share/ | wc -l
```



```
263
paul@deb508:~$ du -sh /usr/share/
1.3G /usr/share/
```

This directory typically contains **/usr/share/man** for manual pages.

```
paul@deb508:~$ ls /usr/share/man
cs  fr    hu  it.UTF-8  man2 man6  pl.ISO8859-2  sv
de  fr.ISO8859-1  id  ja    man3 man7  pl.UTF-8      tr
es  fr.UTF-8      it  ko    man4 man8  pt_BR        zh_CN
fi  gl    it.ISO8859-1  man1  man5  pl     ru        zh  TW
```

And it contains **/usr/share/games** for all static game data (so no high-scores or play logs).

```
paul@ubu1010:~$ ls /usr/share/games/
openttd wesnoth
```

5.8.6. /usr/src

The **/usr/src** directory is the recommended location for kernel source files.

```
paul@deb508:~$ ls -l /usr/src/
total 12
drwxr-xr-x  4 root root 4096 2011-02-01 14:43 linux-headers-2.6.26-2-686
drwxr-xr-x 18 root root 4096 2011-02-01 14:43 linux-headers-2.6.26-2-common
drwxr-xr-x  3 root root 4096 2009-10-28 16:01 linux-kbuild-2.6.26
```

5.9. /var variable data

Files that are unpredictable in size, such as log, cache and spool files, should be located in **/var**.

5.9.1. /var/log

The **/var/log** directory serves as a central point to contain all log files.

```
[paul@RHEL4b ~]$ ls /var/log
acpid          cron.2      maillog.2   quagga      secure.4
amanda         cron.3      maillog.3   radius      spooler
anaconda.log   cron.4      maillog.4   rpmpkgs     spooler.1
anaconda.syslog cups        mailman     rpmpkgs.1   spooler.2
anaconda.xlog  dmesg       messages    rpmpkgs.2   spooler.3
audit          exim        messages.1  rpmpkgs.3   spooler.4
boot.log       gdm         messages.2  rpmpkgs.4   squid
boot.log.1     httpd       messages.3  sa          uucp
boot.log.2     iiim        messages.4  samba       vbox
boot.log.3     iptraf      mysqld.log  scrollkeeper.log vmware-tools-guestd
boot.log.4     lastlog     news        secure      wtmp
canna          mail        pgsql       secure.1     wtmp.1
cron           maillog     ppp         secure.2     Xorg.0.log
cron.1         maillog.1   prelink.log secure.3     Xorg.0.log.old
```

5.9.2. /var/log/messages

A typical first file to check when troubleshooting on Red Hat (and derivatives) is the **/var/log/messages** file. By default this file will contain information on what just happened to the system. The file is called **/var/log/syslog** on Debian and Ubuntu.

```
[root@RHEL4b ~]# tail /var/log/messages
Jul 30 05:13:56 anacron: anacron startup succeeded
Jul 30 05:13:56 atd: atd startup succeeded
Jul 30 05:13:57 messagebus: messagebus startup succeeded
Jul 30 05:13:57 cups-config-daemon: cups-config-daemon startup succeeded
Jul 30 05:13:58 haldaemon: haldaemon startup succeeded
Jul 30 05:14:00 fstab-sync[3560]: removed all generated mount points
Jul 30 05:14:01 fstab-sync[3628]: added mount point /media/cdrom for...
Jul 30 05:14:01 fstab-sync[3646]: added mount point /media/floppy for...
Jul 30 05:16:46 sshd(pam_unix)[3662]: session opened for user paul by...
Jul 30 06:06:37 su(pam_unix)[3904]: session opened for user root by paul
```

5.9.3. /var/cache

The **/var/cache** directory can contain **cache data** for several applications.

```
paul@ubul010:~$ ls /var/cache/
apt          dictionaries-common  gdm      man          software-center
binfmts      flashplugin-installer hald      pm-utils
cups         fontconfig           jockey   pppconfig
debconf      fonts                ldconfig samba
```

5.9.4. /var/spool

The **/var/spool** directory typically contains spool directories for **mail** and **cron**, but also serves as a parent directory for other spool files (for example print spool files).

5.9.5. **/var/lib**

The **/var/lib** directory contains application state information.

Red Hat Enterprise Linux for example keeps files pertaining to **rpm** in **/var/lib/rpm/**.

11.9.6. **/var/...**

/var also contains Process ID files in **/var/run** (soon to be replaced with **/run**) and temporary files that survive a reboot in **/var/tmp** and information about file locks in **/var/lock**. There

5.10. solution: file system tree

1. Does the file **/bin/cat** exist ? What about **/bin/dd** and **/bin/echo**. What is the type of these files ?

```
ls /bin/cat ; file /bin/cat
```

```
ls /bin/dd ; file /bin/dd
```

```
ls /bin/echo ; file /bin/echo
```

2. What is the size of the Linux kernel file(s) (vmlinu*) in **/boot** ?

```
ls -lh /boot/vm*
```

3. Create a directory **~/test**. Then issue the following commands:

```
cd ~/test
```

```
dd if=/dev/zero of=zeros.txt count=1 bs=100
```

```
od zeroes.txt
```

dd will copy one times (count=1) a block of size 100 bytes (bs=100) from the file **/dev/zero** to **~/test/zeros.txt**. Can you describe the functionality of **/dev/zero** ?

/dev/zero is a Linux special device. It can be considered a source of zeroes. You cannot send something to **/dev/zero**, but you can read zeroes from it.

4. Now issue the following command:

```
dd if=/dev/random of=random.txt count=1 bs=100 ; od random.txt
```

dd will copy one times (count=1) a block of size 100 bytes (bs=100) from the file **/dev/random** to **~/test/random.txt**. Can you describe the functionality of **/dev/random** ?

/dev/random acts as a **random number generator** on your Linux machine.

5. Issue the following two commands, and look at the first character of each output line.

```
ls -l /dev/sd* /dev/hd*
```

```
ls -l /dev/tty* /dev/input/mou*
```

The first **ls** will show block(b) devices, the second **ls** shows character(c) devices. Can you tell the difference between block and character devices ?

Block devices are always written to (or read from) in blocks. For hard disks, blocks of 512 bytes are common. Character devices act as a stream of characters (or bytes). Mouse and keyboard are typical character devices.

6. Use **cat** to display **/etc/hosts** and **/etc/resolv.conf**. What is your idea about the purpose of these files ?

```
/etc/hosts contains hostnames with their ip address
```

```
/etc/resolv.conf should contain the ip address of a DNS name server.
```


7. Are there any files in **/etc/skel/** ? Check also for hidden files.

```
Issue "ls -al /etc/skel/". Yes, there should be hidden files there.
```

8. Display **/proc/cpuinfo**. On what architecture is your Linux running ?

```
The file should contain at least one line with Intel or other cpu.
```

9. Display **/proc/interrupts**. What is the size of this file ? Where is this file stored ?

The size is zero, yet the file contains data. It is not stored anywhere because **/proc** is a virtual file system that allows you to talk with the kernel. (If you answered "stored in RAM-memory, that is also correct...).

10. Can you enter the **/root** directory ? Are there (hidden) files ?

```
Try "cd /root". The /root directory is not accessible for normal users on most modern Linux sy
```

11. Are **ifconfig**, **fdisk**, **parted**, **shutdown** and **grub-install** present in **/sbin** ? Why are these binaries in **/sbin** and not in **/bin** ?

```
Because those files are only meant for system administrators.
```

12. Is **/var/log** a file or a directory ? What about **/var/spool** ?

```
Both are directories.
```

13. Open two command prompts (Ctrl-Shift-T in gnome-terminal) or terminals (Ctrl-Alt-F1, Ctrl-Alt-F2, ...) and issue the **who am i** in both. Then try to echo a word from one terminal to the other.

```
tty-terminal: echo Hello > /dev/tty1
```

```
pts-terminal: echo Hello > /dev/pts/1
```

14. Read the man page of **random** and explain the difference between **/dev/random** and **/dev/urandom**.

```
man 4 random
```

Chapter 6. commands and arguments

This chapter introduces you to **shell expansion** by taking a close look at **commands** and **arguments**. Knowing **shell expansion** is important because many **commands** on your Linux system are processed and most likely changed by the **shell** before they are executed.

The command line interface or **shell** used on most Linux systems is called **bash**, which stands for **Bourne again shell**. The **bash** shell incorporates features from **sh** (the original Bourne shell), **cs**h (the C shell), and **ksh** (the Korn shell).

This chapter frequently uses the **echo** command to demonstrate shell features. The **echo** command is very simple: it echoes the input that it receives.

```
paul@laika:~$ echo Burtonville
Burtonville
paul@laika:~$ echo Smurfs are blue
Smurfs are blue
```

6.1. arguments

One of the primary features of a shell is to perform a **command line scan**. When you enter a command at the shell's command prompt and press the enter key, then the shell will start scanning that line, cutting it up in **arguments**. While scanning the line, the shell may make many changes to the **arguments** you typed.

This process is called **shell expansion**. When the shell has finished scanning and modifying that line, then it will be executed.

6.2. white space removal

Parts that are separated by one or more consecutive **white spaces** (or tabs) are considered separate **arguments**, any white space is removed. The first **argument** is the command to be executed, the other **arguments** are given to the command. The shell effectively cuts your command into one or more arguments.

This explains why the following four different command lines are the same after **shell expansion**.

```
[paul@RHELv4u3 ~]$ echo Hello World
Hello World
[paul@RHELv4u3 ~]$ echo Hello   World
Hello World
[paul@RHELv4u3 ~]$ echo    Hello    World
Hello World
[paul@RHELv4u3 ~]$      echo      Hello      World
Hello World
```

The **echo** command will display each argument it receives from the shell. The **echo** command will also add a new white space between the arguments it received.

6.3. single quotes

You can prevent the removal of white spaces by quoting the spaces. The contents of the quoted string are considered as one argument. In the screenshot below the **echo** receives only one **argument**.

```
[paul@RHEL4b ~]$ echo 'A line with      single      quotes'
A line with      single      quotes
[paul@RHEL4b ~]$
```

6.4. double quotes

You can also prevent the removal of white spaces by double quoting the spaces. Same as above, **echo** only receives one **argument**.

```
[paul@RHEL4b ~]$ echo "A line with      double      quotes"
A line with      double      quotes
[paul@RHEL4b ~]$
```

Later in this book, when discussing **variables** we will see important differences between single and double quotes.

6.5. echo and quotes

Quoted lines can include special escaped characters recognised by the **echo** command (when using **echo -e**). The screenshot below shows how to use **\n** for a newline and **\t** for a tab (usually eight white spaces).

```
[paul@RHEL4b ~]$ echo -e "A line with \na newline"
A line with
a newline
[paul@RHEL4b ~]$ echo -e 'A line with \na newline'
A line with
a newline
[paul@RHEL4b ~]$ echo -e "A line with \ta tab"
A line with      a tab
[paul@RHEL4b ~]$ echo -e 'A line with \ta tab'
A line with      a tab
[paul@RHEL4b ~]$
```

The **echo** command can generate more than white spaces, tabs and newlines. Look in the man page for a list of options.

6.6. commands

6.6.1. external or built-in commands?

Not all commands are external to the shell, some are **builtin**. **External commands** are programs that have their own binary and reside somewhere in the file system. Many external commands are located in **/bin** or **/sbin**. **Builtin commands** are an integral part of the shell program itself.

6.6.2. type

To find out whether a command given to the shell will be executed as an **external command** or as a **builtin command**, use the **type** command.

```
paul@laika:~$ type cd
cd is a shell builtin
paul@laika:~$ type cat
cat is /bin/cat
```

As you can see, the **cd** command is **builtin** and the **cat** command is **external**.

You can also use this command to show you whether the command is **aliased** or not.

```
paul@laika:~$ type ls
ls is aliased to `ls --color=auto'
```

6.6.3. running external commands

Some commands have both builtin and external versions. When one of these commands is executed, the builtin version takes priority. To run the external version, you must enter the full path to the command.

```
paul@laika:~$ type -a echo
echo is a shell builtin
echo is /bin/echo
paul@laika:~$ /bin/echo Running the external echo command...
Running the external echo command...
```

6.6.4. which

The **which** command will search for binaries in the **\$PATH** environment variable (variables will be explained later). In the screenshot below, it is determined that **cd** is **builtin**, and **ls**, **cp**, **rm**, **mv**, **mkdir**, **pwd**, and **which** are external commands.

```
[root@RHEL4b ~]# which cp ls cd mkdir pwd
/bin/cp
/bin/ls
/usr/bin/which: no cd in (/usr/kerberos/sbin:/usr/kerberos/bin:...
/bin/mkdir
/bin/pwd
```

6.7. aliases

6.7.1. create an alias

The shell allows you to create **aliases**. Aliases are often used to create an easier to remember name for an existing command or to easily supply parameters.

```
[paul@RHELv4u3 ~]$ cat count.txt
one
two
three
[paul@RHELv4u3 ~]$ alias dog=tac
[paul@RHELv4u3 ~]$ dog count.txt
three
two
one
```

6.7.2. abbreviate commands

An **alias** can also be useful to abbreviate an existing command.

```
paul@laika:~$ alias ll='ls -lh --color=auto'
paul@laika:~$ alias c='clear'
paul@laika:~$
```

6.7.3. default options

Aliases can be used to supply commands with default options. The example below shows how to set the **-i** option default when typing **rm**.

```
[paul@RHELv4u3 ~]$ rm -i winter.txt
rm: remove regular file `winter.txt'? no
[paul@RHELv4u3 ~]$ rm winter.txt
[paul@RHELv4u3 ~]$ ls winter.txt
ls: winter.txt: No such file or directory
[paul@RHELv4u3 ~]$ touch winter.txt
[paul@RHELv4u3 ~]$ alias rm='rm -i'
[paul@RHELv4u3 ~]$ rm winter.txt
rm: remove regular empty file `winter.txt'? no
[paul@RHELv4u3 ~]$
```

Some distributions enable default aliases to protect users from accidentally erasing files ('rm -i', 'mv -i', 'cp -i')

6.7.4. viewing aliases

You can provide one or more aliases as arguments to the **alias** command to get their definitions. Providing no arguments gives a complete list of current aliases.

```
paul@laika:~$ alias c ll
alias c='clear'
alias ll='ls -lh --color=auto'
```

6.7.5. unalias

You can undo an alias with the **unalias** command.

```
[paul@RHEL4b ~]$ which rm
/bin/rm
[paul@RHEL4b ~]$ alias rm='rm -i'
[paul@RHEL4b ~]$ which rm
alias rm='rm -i'
        /bin/rm
[paul@RHEL4b ~]$ unalias rm
[paul@RHEL4b ~]$ which rm
/bin/rm
[paul@RHEL4b ~]$
```

6.8. displaying shell expansion

You can display shell expansion with **set -x**, and stop displaying it with **set +x**. You might want to use this further on in this course, or when in doubt about exactly what the shell is doing with your command.

```
[paul@RHELv4u3 ~]$ set -x
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ echo $USER
+ echo paul
paul
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ echo \ $USER
+ echo '$USER'
$USER
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ set +x
+ set +x
[paul@RHELv4u3 ~]$ echo $USER
paul
```

6.9. solution: commands and arguments

1. How many **arguments** are in this line (not counting the command itself).

```
touch '/etc/cron/cron.allow' 'file 42.txt' "file 33.txt"
```

```
answer: three
```

2. Is **tac** a shell builtin command ?

```
type tac
```

3. Is there an existing alias for **rm** ?

```
alias rm
```

4. Read the man page of **rm**, make sure you understand the **-i** option of **rm**. Create and remove a file to test the **-i** option.

```
man rm
```

```
touch testfile
```

```
rm -i testfile
```

5. Execute: **alias rm='rm -i'** . Test your alias with a test file. Does this work as expected ?

```
touch testfile
```

```
rm testfile (should ask for confirmation)
```

6. List all current aliases.

```
alias
```

7a. Create an alias called 'city' that echoes your hometown.

```
alias city='echo Antwerp'
```

7b. Use your alias to test that it works.

```
city (it should display Antwerp)
```

8. Execute **set -x** to display shell expansion for every command.

```
set -x
```

9. Test the functionality of **set -x** by executing your **city** and **rm** aliases.

```
shell should display the resolved aliases and then execute the command:
paul@deb503:~$ set -x
paul@deb503:~$ city
+ echo antwerp
antwerp
```

10 Execute **set +x** to stop displaying shell expansion.

```
set +x
```

11. Remove your city alias.

```
unalias city
```

12. What is the location of the **cat** and the **passwd** commands ?

```
which cat (probably /bin/cat)
```

```
which passwd (probably /usr/bin/passwd)
```

13. Explain the difference between the following commands:

```
echo
```

```
/bin/echo
```

The **echo** command will be interpreted by the shell as the **built-in echo** command. The **/bin/echo** command will make the shell execute the **echo binary** located in the **/bin** directory.

14. Explain the difference between the following commands:

```
echo Hello
```

```
echo -n Hello
```

The **-n** option of the **echo** command will prevent echo from echoing a trailing newline. **echo Hello** will echo six characters in total, **echo -n hello** only echoes five characters.

(The **-n** option might not work in the Korn shell.)

15. Display **A B C** with two spaces between B and C.

```
echo "A B C"
```

16. Complete the following command (do not use spaces) to display exactly the following output:

```
4+4      =8
10+14    =24
```

The solution is to use tabs with **\t**.

```
echo -e "4+4\t=8" ; echo -e "10+14\t=24"
```

17. Use **echo** to display the following exactly:

```
??\
echo '??\
echo -e '??\
echo "??\
echo -e "??\
echo ??\
```

Find two solutions with single quotes, two with double quotes and one without quotes (and say thank you to René and Darioush from Google for this extra).

18. Use one **echo** command to display three words on three lines.

```
echo -e "one \ntwo \nthree"
```

Chapter 7. control operators

In this chapter we put more than one command on the command line using **control operators**. We also briefly discuss related parameters (\$?) and similar special characters(&).

7.1. ; semicolon

You can put two or more commands on the same line separated by a semicolon `;`. The shell will scan the line until it reaches the semicolon. All the arguments before this semicolon will be considered a separate command from all the arguments after the semicolon. Both series will be executed sequentially with the shell waiting for each command to finish before starting the next one.

```
[paul@RHELv4u3 ~]$ echo Hello
Hello
[paul@RHELv4u3 ~]$ echo World
World
[paul@RHELv4u3 ~]$ echo Hello ; echo World
Hello
World
[paul@RHELv4u3 ~]$
```

7.2. & ampersand

When a line ends with an ampersand `&`, the shell will not wait for the command to finish. You will get your shell prompt back, and the command is executed in background. You will get a message when this command has finished executing in background.

```
[paul@RHELv4u3 ~]$ sleep 20 &
[1] 7925
[paul@RHELv4u3 ~]$
...wait 20 seconds...
[paul@RHELv4u3 ~]$
[1]+  Done                  sleep 20
```

The technical explanation of what happens in this case is explained in the chapter about **processes**.

7.3. \$? dollar question mark

The exit code of the previous command is stored in the shell variable `$?`. Actually `$?` is a shell parameter and not a variable, since you cannot assign a value to `$?`.

```
paul@debian5:~/test$ touch file1
paul@debian5:~/test$ echo $?
0
paul@debian5:~/test$ rm file1
paul@debian5:~/test$ echo $?
0
paul@debian5:~/test$ rm file1
rm: cannot remove `file1': No such file or directory
paul@debian5:~/test$ echo $?
1
paul@debian5:~/test$
```


7.4. && double ampersand

The shell will interpret **&&** as a **logical AND**. When using **&&** the second command is executed only if the first one succeeds (returns a zero exit status).

```
paul@barry:~$ echo first && echo second
first
second
paul@barry:~$ zecho first && echo second
-bash: zecho: command not found
```

Another example of the same **logical AND** principle. This example starts with a working **cd** followed by **ls**, then a non-working **cd** which is **not** followed by **ls**.

```
[paul@RHELv4u3 ~]$ cd gen && ls
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
[paul@RHELv4u3 gen]$ cd gen && ls
-bash: cd: gen: No such file or directory
```

7.5. || double vertical bar

The **||** represents a **logical OR**. The second command is executed only when the first command fails (returns a non-zero exit status).

```
paul@barry:~$ echo first || echo second ; echo third
first
third
paul@barry:~$ zecho first || echo second ; echo third
-bash: zecho: command not found
second
third
paul@barry:~$
```

Another example of the same **logical OR** principle.

```
[paul@RHELv4u3 ~]$ cd gen || ls
[paul@RHELv4u3 gen]$ cd gen || ls
-bash: cd: gen: No such file or directory
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
```

7.6. combining && and ||

You can use this logical AND and logical OR to write an **if-then-else** structure on the command line. This example uses **echo** to display whether the **rm** command was successful.

```
paul@laika:~/test$ rm file1 && echo It worked! || echo It failed!
It worked!
paul@laika:~/test$ rm file1 && echo It worked! || echo It failed!
rm: cannot remove `file1': No such file or directory
It failed!
paul@laika:~/test$
```

7.7. # pound sign

Everything written after a **pound sign** (#) is ignored by the shell. This is useful to write a **shell comment**, but has no influence on the command execution or shell expansion.

```
paul@debian4:~$ mkdir test      # we create a directory
paul@debian4:~$ cd test        ##### we enter the directory
paul@debian4:~/test$ ls        # is it empty ?
paul@debian4:~/test$
```

7.8. \ escaping special characters

The backslash \ character enables the use of control characters, but without the shell interpreting it, this is called **escaping** characters.

```
[paul@RHELv4u3 ~]$ echo hello \; world
hello ; world
[paul@RHELv4u3 ~]$ echo hello\ \ \ world
hello  world
[paul@RHELv4u3 ~]$ echo escaping \\ \# \& \\" \'
escaping \ # & " '
[paul@RHELv4u3 ~]$ echo escaping \\? \* \\" \'
escaping \?*\"'
```

7.8.1. end of line backslash

Lines ending in a backslash are continued on the next line. The shell does not interpret the newline character and will wait on shell expansion and execution of the command line until a newline without backslash is encountered.

```
[paul@RHEL4b ~]$ echo This command line \
> is split in three \
> parts
This command line is split in three parts
[paul@RHEL4b ~]$
```

7.9. practice: control operators

0. Each question can be answered by one command line!
1. When you type **passwd**, which file is executed ?
2. What kind of file is that ?
3. Execute the **pwd** command twice. (remember 0.)
4. Execute **ls** after **cd /etc**, but only if **cd /etc** did not error.
5. Execute **cd /etc** after **cd etc**, but only if **cd etc** fails.
6. Echo **it worked** when **touch test42** works, and echo **it failed** when the **touch** failed. All on one command line as a normal user (not root). Test this line in your home directory and in **/bin/**.
7. Execute **sleep 6**, what is this command doing ?
8. Execute **sleep 200** in background (do not wait for it to finish).
9. Write a command line that executes **rm file55**. Your command line should print 'success' if file55 is removed, and print 'failed' if there was a problem.
- (optional)10. Use echo to display "Hello World with strange' characters \ * [} ~ \ \ ." (including all quotes)

7.10. solution: control operators

0. Each question can be answered by one command line!

1. When you type **passwd**, which file is executed ?

```
which passwd
```

2. What kind of file is that ?

```
file /usr/bin/passwd
```

3. Execute the **pwd** command twice. (remember 0.)

```
pwd ; pwd
```

4. Execute **ls** after **cd /etc**, but only if **cd /etc** did not error.

```
cd /etc && ls
```

5. Execute **cd /etc** after **cd etc**, but only if **cd etc** fails.

```
cd etc || cd /etc
```

6. Echo **it worked** when **touch test42** works, and echo **it failed** when the **touch** failed. All on one command line as a normal user (not root). Test this line in your home directory and in **/bin/**.

```
paul@deb503:~$ cd ; touch test42 && echo it worked || echo it failed
it worked
paul@deb503:~$ cd /bin; touch test42 && echo it worked || echo it failed
touch: cannot touch `test42': Permission denied
it failed
```

7. Execute **sleep 6**, what is this command doing ?

```
pausing for six seconds
```

8. Execute **sleep 200** in background (do not wait for it to finish).

```
sleep 200 &
```

9. Write a command line that executes **rm file55**. Your command line should print 'success' if file55 is removed, and print 'failed' if there was a problem.

```
rm file55 && echo success || echo failed
```

(optional)10. Use echo to display "Hello World with strange' characters \ * [] ~ \ \." (including all quotes)

```
echo \"Hello World with strange\" characters \\ \\* \\[ \\] \\~ \\\\ \\\" \\\"
```

```
or
```

```
echo \"\"\"Hello World with strange' characters \\ * [ ] ~ \\\" . \"\"\"
```

Chapter 8. shell variables

In this chapter we learn to manage environment **variables** in the shell. These **variables** are often needed by applications.

8.1. \$ dollar sign

Another important character interpreted by the shell is the dollar sign **\$**. The shell will look for an **environment variable** named like the string following the **dollar sign** and replace it with the value of the variable (or with nothing if the variable does not exist).

These are some examples using \$HOSTNAME, \$USER, \$UID, \$SHELL, and \$HOME.

```
[paul@RHELv4u3 ~]$ echo This is the $SHELL shell
This is the /bin/bash shell
[paul@RHELv4u3 ~]$ echo This is $SHELL on computer $HOSTNAME
This is /bin/bash on computer RHELv4u3.localdomain
[paul@RHELv4u3 ~]$ echo The userid of $USER is $UID
The userid of paul is 500
[paul@RHELv4u3 ~]$ echo My homedir is $HOME
My homedir is /home/paul
```

8.2. case sensitive

This example shows that shell variables are case sensitive!

```
[paul@RHELv4u3 ~]$ echo Hello $USER
Hello paul
[paul@RHELv4u3 ~]$ echo Hello $user
Hello
```

8.3. creating variables

This example creates the variable **\$MyVar** and sets its value. It then uses **echo** to verify the value.

```
[paul@RHELv4u3 gen]$ MyVar=555
[paul@RHELv4u3 gen]$ echo $MyVar
555
[paul@RHELv4u3 gen]$
```

8.4. quotes

Notice that double quotes still allow the parsing of variables, whereas single quotes prevent this.

```
[paul@RHELv4u3 ~]$ MyVar=555
[paul@RHELv4u3 ~]$ echo $MyVar
555
[paul@RHELv4u3 ~]$ echo "$MyVar"
555
[paul@RHELv4u3 ~]$ echo '$MyVar'
$MyVar
```

The bash shell will replace variables with their value in double quoted lines, but not in single quoted lines.

```
paul@laika:~$ city=Burtonville
paul@laika:~$ echo "We are in $city today."
We are in Burtonville today.
paul@laika:~$ echo 'We are in $city today.'
We are in $city today.
```

8.5. set

You can use the **set** command to display a list of environment variables. On Ubuntu and Debian systems, the **set** command will also list shell functions after the shell variables. Use **set | more** to see the variables then.

8.6. unset

Use the **unset** command to remove a variable from your shell environment.

```
[paul@RHEL4b ~]$ MyVar=8472
[paul@RHEL4b ~]$ echo $MyVar
8472
[paul@RHEL4b ~]$ unset MyVar
[paul@RHEL4b ~]$ echo $MyVar

[paul@RHEL4b ~]$
```

8.7. \$PS1

The **\$PS1** variable determines your shell prompt. You can use backslash escaped special characters like **\u** for the username or **\w** for the working directory. The **bash** manual has a complete reference.

In this example we change the value of **\$PS1** a couple of times.

```
paul@deb503:~$ PS1=prompt
prompt
promptPS1='prompt '
prompt
prompt PS1='> '
>
> PS1='\u@\h$ '
paul@deb503$
paul@deb503$ PS1='\u@\h:\w$ '
paul@deb503:~$
```

To avoid unrecoverable mistakes, you can set normal user prompts to green and the root prompt to red. Add the following to your **.bashrc** for a green user prompt:

```
# color prompt by paul
RED='\[\033[01;31m\]'
WHITE='\[\033[01;00m\]'
GREEN='\[\033[01;32m\]'
BLUE='\[\033[01;34m\]'
export PS1="${debian_chroot:+($debian_chroot)}$GREEN\u$WHITE@$BLUE\h$WHITE\w\$ "
```


8.8. \$PATH

The **\$PATH** variable determines where the shell is looking for commands to execute (unless the command is builtin or aliased). This variable contains a list of directories, separated by colons.

```
[[paul@RHEL4b ~]$ echo $PATH
/usr/kerberos/bin:/usr/local/bin:/bin:/usr/bin:
```

The shell will not look in the current directory for commands to execute! (Looking for executables in the current directory provided an easy way to hack PC-DOS computers). If you want the shell to look in the current directory, then add a **.** at the end of your **\$PATH**.

```
[paul@RHEL4b ~]$ PATH=$PATH:.
[paul@RHEL4b ~]$ echo $PATH
/usr/kerberos/bin:/usr/local/bin:/bin:/usr/bin:.
[paul@RHEL4b ~]$
```

Your path might be different when using **su** instead of **su -** because the latter will take on the environment of the target user. The root user typically has **/sbin** directories added to the **\$PATH** variable.

```
[paul@RHEL3 ~]$ su
Password:
[root@RHEL3 paul]# echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin
[root@RHEL3 paul]# exit
[paul@RHEL3 ~]$ su -
Password:
[root@RHEL3 ~]# echo $PATH
/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin:
[root@RHEL3 ~]#
```

8.9. env

The **env** command without options will display a list of **exported variables**. The difference with **set** with options is that **set** lists all variables, including those not exported to child shells.

But **env** can also be used to start a clean shell (a shell without any inherited environment). The **env -i** command clears the environment for the subshell.

Notice in this screenshot that **bash** will set the **\$SHELL** variable on startup.

```
[paul@RHEL4b ~]$ bash -c 'echo $SHELL $HOME $USER'
/bin/bash /home/paul paul
[paul@RHEL4b ~]$ env -i bash -c 'echo $SHELL $HOME $USER'
/bin/bash
[paul@RHEL4b ~]$
```

You can use the **env** command to set the **\$LANG**, or any other, variable for just one instance of **bash** with one command. The example below uses this to show the influence of the **\$LANG** variable on file globbing (see the chapter on file globbing).

```
[paul@RHEL4b test]$ env LANG=C bash -c 'ls File[a-z]'
Filea  Fileb
[paul@RHEL4b test]$ env LANG=en_US.UTF-8 bash -c 'ls File[a-z]'
Filea  FileA  Fileb  FileB
[paul@RHEL4b test]$
```

8.10. export

You can export shell variables to other shells with the **export** command. This will export the variable to child shells.

```
[paul@RHEL4b ~]$ var3=three
[paul@RHEL4b ~]$ var4=four
[paul@RHEL4b ~]$ export var4
[paul@RHEL4b ~]$ echo $var3 $var4
three four
[paul@RHEL4b ~]$ bash
[paul@RHEL4b ~]$ echo $var3 $var4
four
```

But it will not export to the parent shell (previous screenshot continued).

```
[paul@RHEL4b ~]$ export var5=five
[paul@RHEL4b ~]$ echo $var3 $var4 $var5
four five
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ echo $var3 $var4 $var5
three four
[paul@RHEL4b ~]$
```

8.11. delineate variables

Until now, we have seen that bash interprets a variable starting from a dollar sign, continuing until the first occurrence of a non-alphanumeric character that is not an underscore. In some situations, this can be a problem. This issue can be resolved with curly braces like in this example.

```
[paul@RHEL4b ~]$ prefix=Super
[paul@RHEL4b ~]$ echo Hello $prefixman and $prefixgirl
Hello  and
[paul@RHEL4b ~]$ echo Hello ${prefix}man and ${prefix}girl
Hello Superman and Supergirl
[paul@RHEL4b ~]$
```

8.12. unbound variables

The example below tries to display the value of the **\$MyVar** variable, but it fails because the variable does not exist. By default the shell will display nothing when a variable is unbound (does not exist).

```
[paul@RHELv4u3 gen]$ echo $MyVar

[paul@RHELv4u3 gen]$
```

There is, however, the **nounset** shell option that you can use to generate an error when a variable does not exist.

```
paul@laika:~$ set -u
paul@laika:~$ echo $Myvar
bash: Myvar: unbound variable
paul@laika:~$ set +u
paul@laika:~$ echo $Myvar

paul@laika:~$
```

In the bash shell **set -u** is identical to **set -o nounset** and likewise **set +u** is identical to **set +o nounset**.

8.13. practice: shell variables

1. Use echo to display Hello followed by your username. (use a bash variable!)
2. Create a variable **answer** with a value of **42**.
3. Copy the value of \$LANG to \$MyLANG.
4. List all current shell variables.
5. List all exported shell variables.
6. Do the **env** and **set** commands display your variable ?
6. Destroy your **answer** variable.
7. Create two variables, and **export** one of them.
8. Display the exported variable in an interactive child shell.
9. Create a variable, give it the value 'Dumb', create another variable with value 'do'. Use **echo** and the two variables to echo Dumbledore.
10. Find the list of backslash escaped characters in the manual of bash. Add the time to your **PS1** prompt.

8.14. solution: shell variables

1. Use `echo` to display Hello followed by your username. (use a bash variable!)

```
echo Hello $USER
```

2. Create a variable **answer** with a value of **42**.

```
answer=42
```

3. Copy the value of `$LANG` to `$MyLANG`.

```
MyLANG=$LANG
```

4. List all current shell variables.

```
set
```

```
set | more on Ubuntu/Debian
```

5. List all exported shell variables.

```
env  
export  
declare -x
```

6. Do the **env** and **set** commands display your variable ?

```
env | more  
set | more
```

6. Destroy your **answer** variable.

```
unset answer
```

7. Create two variables, and **export** one of them.

```
var1=1; export var2=2
```

8. Display the exported variable in an interactive child shell.

```
bash  
echo $var2
```

9. Create a variable, give it the value 'Dumb', create another variable with value 'do'. Use **echo** and the two variables to echo Dumbledore.

```
varx=Dumb; vary=do  
  
echo ${varx}le${vary}re  
solution by Yves from Dexia : echo $varx'le'$vary're'  
solution by Erwin from Telenet : echo "$varx"le"$vary"re
```

10. Find the list of backslash escaped characters in the manual of `bash`. Add the time to your **PS1** prompt.

```
PS1='\t \u@\h \W$ '
```

Chapter 9. shell embedding and options

This chapter takes a brief look at **child shells**, **embedded shells** and **shell options**.

9.2. shell options

Both **set** and **unset** are builtin shell commands. They can be used to set options of the bash shell itself. The next example will clarify this. By default, the shell will treat unset variables as a variable having no value. By setting the **-u** option, the shell will treat any reference to unset variables as an error. See the man page of bash for more information.

```
[paul@RHEL4b ~]$ echo $var123

[paul@RHEL4b ~]$ set -u
[paul@RHEL4b ~]$ echo $var123
-bash: var123: unbound variable
[paul@RHEL4b ~]$ set +u
[paul@RHEL4b ~]$ echo $var123

[paul@RHEL4b ~]$
```

To list all the set options for your shell, use **echo \$-**. The **noclobber** (or **-C**) option will be explained later in this book (in the I/O redirection chapter).

```
[paul@RHEL4b ~]$ echo $-
himBH
[paul@RHEL4b ~]$ set -C ; set -u
[paul@RHEL4b ~]$ echo $-
himuBCH
[paul@RHEL4b ~]$ set +C ; set +u
[paul@RHEL4b ~]$ echo $-
himBH
[paul@RHEL4b ~]$
```

When typing **set** without options, you get a list of all variables without function when the shell is on **posix** mode. You can set bash in posix mode typing **set -o posix**.

9.3. solution: shell embedding

1. Find the list of shell options in the man page of **bash**. What is the difference between **set -u** and **set -o nounset**?

read the manual of bash (man bash), search for nounset -- both mean the same thing.

2. Activate **nounset** in your shell. Test that it shows an error message when using non-existing variables.

```
set -u
OR
set -o nounset
```

Both these lines have the same effect.

3. Deactivate nounset.

```
set +u
OR
set +o nounset
```

4. Execute **cd /var** and **ls** in an embedded shell.

```
echo $(cd /var ; ls)
```


The **echo** command is only needed to show the result of the **ls** command. Omitting will result in the shell trying to execute the first file as a command.

5. Create the variable **embvar** in an embedded shell and echo it. Does the variable exist in your current shell now ?

```
echo $(embvar=emb;echo $embvar) ; echo $embvar #the last echo fails
```

```
$embvar does not exist in your current shell
```

6. Explain what "set -x" does. Can this be useful ?

```
It displays shell expansion for troubleshooting your command.
```

(optional)7. Given the following screenshot, add exactly four characters to that command line so that the total output is FirstMiddleLast.

```
[paul@RHEL4b ~]$ echo First; echo Middle; echo Last
```

```
echo -n First; echo -n Middle; echo Last
```

8. Display a **long listing** (**ls -l**) of the **passwd** command using the **which** command inside an embedded shell.

```
ls -l $(which passwd)
```

Chapter 10. shell history

The shell makes it easy for us to repeat commands, this chapter explains how.

10.1. repeating the last command

To repeat the last command in bash, type **!!**. This is pronounced as **bang bang**.

```
paul@debian5:~/test42$ echo this will be repeated > file42.txt
paul@debian5:~/test42$ !!
echo this will be repeated > file42.txt
paul@debian5:~/test42$
```

10.2. repeating other commands

You can repeat other commands using one **bang** followed by one or more characters. The shell will repeat the last command that started with those characters.

```
paul@debian5:~/test42$ touch file42
paul@debian5:~/test42$ cat file42
paul@debian5:~/test42$ !to
touch file42
paul@debian5:~/test42$
```

10.3. history

To see older commands, use **history** to display the shell command history (or use **history n** to see the last n commands).

```
paul@debian5:~/test$ history 10
38  mkdir test
39  cd test
40  touch file1
41  echo hello > file2
42  echo It is very cold today > winter.txt
43  ls
44  ls -l
45  cp winter.txt summer.txt
46  ls -l
47  history 10
```

10.4. !n

When typing **!** followed by the number preceding the command you want repeated, then the shell will echo the command and execute it.

```
paul@debian5:~/test$ !43
ls
file1  file2  summer.txt  winter.txt
```

10.5. Ctrl-r

Another option is to use **ctrl-r** to search in the history. In the screenshot below i only typed **ctrl-r** followed by four characters **apti** and it finds the last command containing these four consecutive characters.

```
paul@debian5:~$  
(reverse-i-search)`apti': sudo aptitude install screen
```

10.6. \$HISTSIZE

The `$HISTSIZE` variable determines the number of commands that will be remembered in your current environment. Most distributions default this variable to 500 or 1000.

```
paul@debian5:~$ echo $HISTSIZE  
500
```

You can change it to any value you like.

```
paul@debian5:~$ HISTSIZE=15000  
paul@debian5:~$ echo $HISTSIZE  
15000
```

10.7. \$HISTFILE

The `$HISTFILE` variable points to the file that contains your history. The **bash** shell defaults this value to `~/.bash_history`.

```
paul@debian5:~$ echo $HISTFILE  
/home/paul/.bash_history
```

A session history is saved to this file when you **exit** the session!

*Closing a gnome-terminal with the mouse, or typing **reboot** as root will NOT save your terminal's history.*

10.8. \$HISTFILESIZE

The number of commands kept in your history file can be set using `$HISTFILESIZE`.

```
paul@debian5:~$ echo $HISTFILESIZE  
15000
```

10.9. prevent recording a command

You can prevent a command from being recorded in **history** using a space prefix.

```
paul@debian8:~/github$ echo abc
abc
paul@debian8:~/github$ echo def
def
paul@debian8:~/github$ echo ghi
ghi
paul@debian8:~/github$ history 3
9501  echo abc
9502  echo ghi
9503  history 3
```

10.10. (optional)regular expressions

It is possible to use **regular expressions** when using the **bang** to repeat commands. The screenshot below switches 1 into 2.

```
paul@debian5:~/test$ cat file1
paul@debian5:~/test$ !c:s/1/2
cat file2
hello
paul@debian5:~/test$
```

10.11. (optional) Korn shell history

Repeating a command in the **Korn shell** is very similar. The Korn shell also has the **history** command, but uses the letter **r** to recall lines from history.

This screenshot shows the history command. Note the different meaning of the parameter.

```
$ history 17
17  clear
18  echo hoi
19  history 12
20  echo world
21  history 17
```

Repeating with **r** can be combined with the line numbers given by the history command, or with the first few letters of the command.

```
$ r e
echo world
world
$ cd /etc
$ r
cd /etc
$
```

10.12. practice: shell history

1. Issue the command **echo The answer to the meaning of life, the universe and everything is 42.**
2. Repeat the previous command using only two characters (there are two solutions!)
3. Display the last 5 commands you typed.
4. Issue the long **echo** from question 1 again, using the line numbers you received from the command in question 3.
5. How many commands can be kept in memory for your current shell session ?
6. Where are these commands stored when exiting the shell ?
7. How many commands can be written to the **history file** when exiting your current shell session ?
8. Make sure your current bash shell remembers the next 5000 commands you type.
9. Open more than one console (by press Ctrl-shift-t in gnome-terminal, or by opening an extra putty.exe in MS Windows) with the same user account. When is command history written to the history file ?

10.13. solution: shell history

1. Issue the command **echo The answer to the meaning of life, the universe and everything is 42.**

```
echo The answer to the meaning of life, the universe and everything is 42
```

2. Repeat the previous command using only two characters (there are two solutions!)

```
!!  
OR  
!e
```

3. Display the last 5 commands you typed.

```
paul@ubu1010:~$ history 5  
52  ls -l  
53  ls  
54  df -h | grep sda  
55  echo The answer to the meaning of life, the universe and everything is 42  
56  history 5
```

You will receive different line numbers.

4. Issue the long **echo** from question 1 again, using the line numbers you received from the command in question 3.

```
paul@ubu1010:~$ !55  
echo The answer to the meaning of life, the universe and everything is 42  
The answer to the meaning of life, the universe and everything is 42
```

5. How many commands can be kept in memory for your current shell session ?

```
echo $HISTSIZE
```

6. Where are these commands stored when exiting the shell ?

```
echo $HISTFILE
```

7. How many commands can be written to the **history file** when exiting your current shell session ?

```
echo $HISTFILESIZE
```

8. Make sure your current bash shell remembers the next 5000 commands you type.

```
HISTSIZE=5000
```

9. Open more than one console (by press Ctrl-shift-t in gnome-terminal, or by opening an extra putty.exe in MS Windows) with the same user account. When is command history written to the history file ?

```
when you type exit
```

Chapter 11. file globbing

The shell is also responsible for **file globbing** (or dynamic filename generation). This chapter will explain **file globbing**.

11.1. * asterisk

The asterisk ***** is interpreted by the shell as a sign to generate filenames, matching the asterisk to any combination of characters (even none). When no path is given, the shell will use filenames in the current directory. See the man page of **glob(7)** for more information. (This is part of LPI topic 1.103.3.)

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File*
File4 File55 FileA Fileab FileAB
[paul@RHELv4u3 gen]$ ls file*
file1 file2 file3 fileab fileabc
[paul@RHELv4u3 gen]$ ls *ile55
File55
[paul@RHELv4u3 gen]$ ls F*ile55
File55
[paul@RHELv4u3 gen]$ ls F*55
File55
[paul@RHELv4u3 gen]$
```

11.2. ? question mark

Similar to the asterisk, the question mark **?** is interpreted by the shell as a sign to generate filenames, matching the question mark with exactly one character.

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File?
File4 FileA
[paul@RHELv4u3 gen]$ ls Fil?4
File4
[paul@RHELv4u3 gen]$ ls Fil??
File4 FileA
[paul@RHELv4u3 gen]$ ls File??
File55 Fileab FileAB
[paul@RHELv4u3 gen]$
```

11.3. [] square brackets

The square bracket `[]` is interpreted by the shell as a sign to generate filenames, matching any of the characters between `[` and the first subsequent `]`. The order in this list between the brackets is not important. Each pair of brackets is replaced by exactly one character.

```
[paul@RHELv4u3 gen]$ ls
file1  file2  file3  File4  File55  FileA  fileab  Fileab  FileAB  fileabc
[paul@RHELv4u3 gen]$ ls File[5A]
FileA
[paul@RHELv4u3 gen]$ ls File[A5]
FileA
[paul@RHELv4u3 gen]$ ls File[A5][5b]
File55
[paul@RHELv4u3 gen]$ ls File[a5][5b]
File55  Fileab
[paul@RHELv4u3 gen]$ ls File[a5][5b][abcdefghijklm]
ls: File[a5][5b][abcdefghijklm]: No such file or directory
[paul@RHELv4u3 gen]$ ls file[a5][5b][abcdefghijklm]
fileabc
[paul@RHELv4u3 gen]$
```

You can also exclude characters from a list between square brackets with the exclamation mark `!`. And you are allowed to make combinations of these **wild cards**.

```
[paul@RHELv4u3 gen]$ ls
file1  file2  file3  File4  File55  FileA  fileab  Fileab  FileAB  fileabc
[paul@RHELv4u3 gen]$ ls file[a5][!Z]
fileab
[paul@RHELv4u3 gen]$ ls file[!5]*
file1  file2  file3  fileab  fileabc
[paul@RHELv4u3 gen]$ ls file[!5]?
fileab
[paul@RHELv4u3 gen]$
```

11.4. a-z and 0-9 ranges

The bash shell will also understand ranges of characters between brackets.

```
[paul@RHELv4u3 gen]$ ls
file1  file3  File55  fileab  FileAB  fileabc
file2  File4  FileA   Fileab  fileab2
[paul@RHELv4u3 gen]$ ls file[a-z]*
fileab  fileab2  fileabc
[paul@RHELv4u3 gen]$ ls file[0-9]
file1  file2  file3
[paul@RHELv4u3 gen]$ ls file[a-z][a-z][0-9]*
fileab2
[paul@RHELv4u3 gen]$
```

11.5. \$LANG and square brackets

But, don't forget the influence of the **LANG** variable. Some languages include lower case letters in an upper case range (and vice versa).

```
paul@RHELv4u4:~/test$ ls [A-Z]ile?
file1  file2  file3  File4
paul@RHELv4u4:~/test$ ls [a-z]ile?
file1  file2  file3  File4
paul@RHELv4u4:~/test$ echo $LANG
en_US.UTF-8
paul@RHELv4u4:~/test$ LANG=C
paul@RHELv4u4:~/test$ echo $LANG
C
paul@RHELv4u4:~/test$ ls [a-z]ile?
file1  file2  file3
paul@RHELv4u4:~/test$ ls [A-Z]ile?
File4
paul@RHELv4u4:~/test$
```

If **\$LC_ALL** is set, then this will also need to be reset to prevent file globbing.

11.6. preventing file globbing

The screenshot below should be no surprise. The **echo *** will echo a ***** when in an empty directory. And it will echo the names of all files when the directory is not empty.

```
paul@ubu1010:~$ mkdir test42
paul@ubu1010:~$ cd test42
paul@ubu1010:~/test42$ echo *
*
paul@ubu1010:~/test42$ touch file42 file33
paul@ubu1010:~/test42$ echo *
file33 file42
```

Globbering can be prevented using quotes or by escaping the special characters, as shown in this screenshot.

```
paul@ubu1010:~/test42$ echo *
file33 file42
paul@ubu1010:~/test42$ echo \*
*
paul@ubu1010:~/test42$ echo '*'
*
paul@ubu1010:~/test42$ echo "*"
*
```

11.7. solution: shell globbing

1. Create a test directory and enter it.

```
mkdir testdir; cd testdir
```

2. Create the following files :

```
file1  
file10  
file11  
file2  
File2  
File3  
file33  
fileAB  
filea  
fileA  
fileAAA  
file(  
file 2
```

(the last one has 6 characters including a space)

```
touch file1 file10 file11 file2 File2 File3  
touch file33 fileAB filea fileA fileAAA  
touch "file("  
touch "file 2"
```

3. List (with ls) all files starting with file

```
ls file*
```

4. List (with ls) all files starting with File

```
ls File*
```

5. List (with ls) all files starting with file and ending in a number.

```
ls file*[0-9]
```

6. List (with ls) all files starting with file and ending with a letter

```
ls file*[a-z]
```

7. List (with ls) all files starting with File and having a digit as fifth character.

```
ls File[0-9]*
```

8. List (with ls) all files starting with File and having a digit as fifth character and nothing else.

```
ls File[0-9]
```

9. List (with ls) all files starting with a letter and ending in a number.

```
ls [a-z]*[0-9]
```

10. List (with ls) all files that have exactly five characters.


```
ls ?????
```

11. List (with ls) all files that start with f or F and end with 3 or A.

```
ls [fF]*[3A]
```

12. List (with ls) all files that start with f have i or R as second character and end in a number.

```
ls f[iR]*[0-9]
```

13. List all files that do not start with the letter F.

```
ls [!F]*
```

14. Copy the value of \$LANG to \$MyLANG.

```
MyLANG=$LANG
```

15. Show the influence of \$LANG in listing A-Z or a-z ranges.

```
see example in book
```

16. You receive information that one of your servers was cracked, the cracker probably replaced the **ls** command. You know that the **echo** command is safe to use. Can **echo** replace **ls** ? How can you list the files in the current directory with **echo** ?

```
echo *
```

17. Is there another command besides cd to change directories ?

```
pushd popd
```

Chapter 12. I/O redirection

One of the powers of the Unix command line is the use of **input/output redirection** and **pipes**.

This chapter explains **redirection** of input, output and error streams.

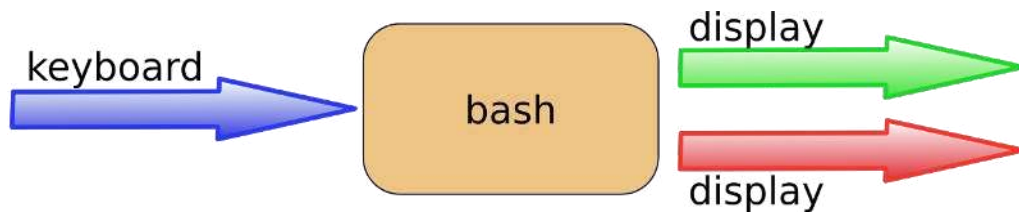
12.1. stdin, stdout, and stderr

The bash shell has three basic streams; it takes input from **stdin** (stream **0**), it sends output to **stdout** (stream **1**) and it sends error messages to **stderr** (stream **2**) .

The drawing below has a graphical interpretation of these three streams.



The keyboard often serves as **stdin**, whereas **stdout** and **stderr** both go to the display. This can be confusing to new Linux users because there is no obvious way to recognize **stdout** from **stderr**. Experienced users know that separating output from errors can be very useful.

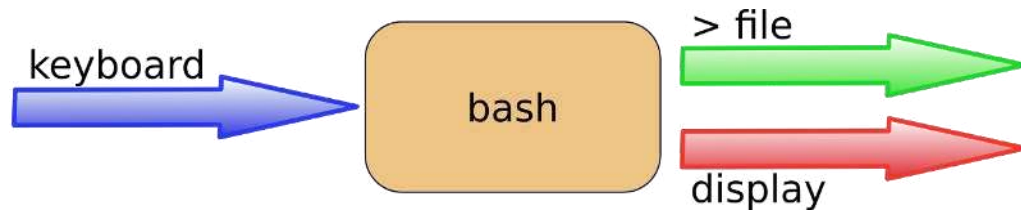


The next sections will explain how to redirect these streams.

12.2. output redirection

12.2.1. > stdout

stdout can be redirected with a **greater than** sign. While scanning the line, the shell will see the > sign and will clear the file.



The > notation is in fact the abbreviation of **1>** (**stdout** being referred to as stream **1**).

```
[paul@RHELv4u3 ~]$ echo It is cold today!
It is cold today!
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$
```

Note that the bash shell effectively **removes** the redirection from the command line before argument 0 is executed. This means that in the case of this command:

```
echo hello > greetings.txt
```

the shell only counts two arguments (echo = argument 0, hello = argument 1). The redirection is removed before the argument counting takes place.

12.2.2. output file is erased

While scanning the line, the shell will see the > sign and **will clear the file!** Since this happens before resolving **argument 0**, this means that even when the command fails, the file will have been cleared!

```
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ zcho It is cold today! > winter.txt
-bash: zcho: command not found
[paul@RHELv4u3 ~]$ cat winter.txt
[paul@RHELv4u3 ~]$
```

12.2.3. noclobber

Erasing a file while using > can be prevented by setting the **noclobber** option.

```
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ set -o noclobber
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
-bash: winter.txt: cannot overwrite existing file
[paul@RHELv4u3 ~]$ set +o noclobber
[paul@RHELv4u3 ~]$
```

12.2.4. overruling noclobber

The **noclobber** can be overruled with >|.

```
[paul@RHELv4u3 ~]$ set -o noclobber
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
-bash: winter.txt: cannot overwrite existing file
[paul@RHELv4u3 ~]$ echo It is very cold today! >| winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is very cold today!
[paul@RHELv4u3 ~]$
```

12.2.5. >> append

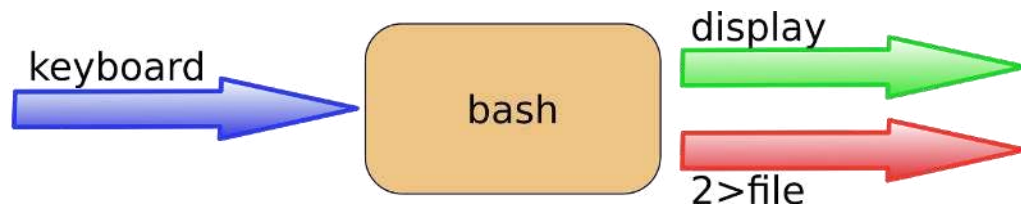
Use >> to **append** output to a file.

```
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ echo Where is the summer ? >> winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
Where is the summer ?
[paul@RHELv4u3 ~]$
```

12.3. error redirection

12.3.1. 2> stderr

Redirecting **stderr** is done with **2>**. This can be very useful to prevent error messages from cluttering your screen.



The screenshot below shows redirection of **stdout** to a file, and **stderr** to **/dev/null**. Writing **1>** is the same as **>**.

```
[paul@RHELv4u3 ~]$ find / > allfiles.txt 2> /dev/null
[paul@RHELv4u3 ~]$
```

18.3.2. 2>&1

To redirect both **stdout** and **stderr** to the same file, use **2>&1**.

```
[paul@RHELv4u3 ~]$ find / > allfiles_and_errors.txt 2>&1
[paul@RHELv4u3 ~]$
```

Note that the order of redirections is significant. For example, the command

```
ls > dirlist 2>&1
```

directs both standard output (file descriptor 1) and standard error (file descriptor 2) to the file `dirlist`, while the command

```
ls 2>&1 > dirlist
```

directs only the standard output to file `dirlist`, because the standard error made a copy of the standard output before the standard output was redirected to `dirlist`.

12.4. output redirection and pipes

By default you cannot `grep` inside **stderr** when using pipes on the command line, because only **stdout** is passed.

```
paul@debian7:~$ rm file42 file33 file1201 | grep file42
rm: cannot remove 'file42': No such file or directory
rm: cannot remove 'file33': No such file or directory
rm: cannot remove 'file1201': No such file or directory
```

With **2>&1** you can force **stderr** to go to **stdout**. This enables the next command in the pipe to act on both streams.

```
paul@debian7:~$ rm file42 file33 file1201 2>&1 | grep file42
rm: cannot remove 'file42': No such file or directory
```

You cannot use both **1>&2** and **2>&1** to switch **stdout** and **stderr**.

```
paul@debian7:~$ rm file42 file33 file1201 2>&1 1>&2 | grep file42
rm: cannot remove 'file42': No such file or directory
paul@debian7:~$ echo file42 2>&1 1>&2 | sed 's/file42/FILE42/'
FILE42
```

You need a third stream to switch **stdout** and **stderr** after a pipe symbol.

```
paul@debian7:~$ echo file42 3>&1 1>&2 2>&3 | sed 's/file42/FILE42/'
file42
paul@debian7:~$ rm file42 3>&1 1>&2 2>&3 | sed 's/file42/FILE42/'
rm: cannot remove 'FILE42': No such file or directory
```

12.5. joining stdout and stderr

The **&>** construction will put both **stdout** and **stderr** in one stream (to a file).

```
paul@debian7:~$ rm file42 &> out_and_err
paul@debian7:~$ cat out_and_err
rm: cannot remove 'file42': No such file or directory
paul@debian7:~$ echo file42 &> out_and_err
paul@debian7:~$ cat out_and_err
file42
paul@debian7:~$
```

12.6. input redirection

12.6.1. < stdin

Redirecting **stdin** is done with < (short for 0<).

```
[paul@RHEL4b ~]$ cat < text.txt
one
two
[paul@RHEL4b ~]$ tr 'onetw' 'ONEZZ' < text.txt
ONE
ZZO
[paul@RHEL4b ~]$
```

12.6.2. << here document

The **here document** (sometimes called here-is-document) is a way to append input until a certain sequence (usually EOF) is encountered. The **EOF** marker can be typed literally or can be called with Ctrl-D.

```
[paul@RHEL4b ~]$ cat <<EOF > text.txt
> one
> two
> EOF
[paul@RHEL4b ~]$ cat text.txt
one
two
[paul@RHEL4b ~]$ cat <<bro1 > text.txt
> brel
> bro1
[paul@RHEL4b ~]$ cat text.txt
brel
[paul@RHEL4b ~]$
```

12.6.3. <<< here string

The **here string** can be used to directly pass strings to a command. The result is the same as using **echo string | command** (but you have one less process running).

```
paul@ubu1110~$ base64 <<< linux-training.be
bGludXgt dHJhaW5pbmcuYmUK
paul@ubu1110~$ base64 -d <<< bGludXgt dHJhaW5pbmcuYmUK
linux-training.be
```

See rfc 3548 for more information about **base64**.

12.7. confusing redirection

The shell will scan the whole line before applying redirection. The following command line is very readable and is correct.

```
cat winter.txt > snow.txt 2> errors.txt
```

But this one is also correct, but less readable.

```
2> errors.txt cat winter.txt > snow.txt
```

Even this will be understood perfectly by the shell.

```
< winter.txt > snow.txt 2> errors.txt cat
```

12.8. quick file clear

So what is the quickest way to clear a file ?

```
>foo
```

And what is the quickest way to clear a file when the **noclobber** option is set ?

12.9. solution: input/output redirection

1. Activate the **noclobber** shell option.

```
set -o noclobber
set -C
```

2. Verify that **noclobber** is active by repeating an **ls** on **/etc/** with redirected output to a file.

```
ls /etc > etc.txt
ls /etc > etc.txt (should not work)
```

4. When listing all shell options, which character represents the **noclobber** option ?

```
echo $- (noclobber is visible as C)
```

5. Deactivate the **noclobber** option.

```
set +o noclobber
```

6. Make sure you have two shells open on the same computer. Create an empty **tailing.txt** file. Then type **tail -f tailing.txt**. Use the second shell to **append** a line of text to that file. Verify that the first shell displays this line.

```
paul@deb503:~$ > tailing.txt
paul@deb503:~$ tail -f tailing.txt
hello
world

in the other shell:
paul@deb503:~$ echo hello >> tailing.txt
paul@deb503:~$ echo world >> tailing.txt
```

7. Create a file that contains the names of five people. Use **cat** and output redirection to create the file and use a **here document** to end the input.

```
paul@deb503:~$ cat > tennis.txt << ace
> Justine Henin
> Venus Williams
> Serena Williams
> Martina Hingis
> Kim Clijsters
> ace
paul@deb503:~$ cat tennis.txt
Justine Henin
Venus Williams
Serena Williams
Martina Hingis
Kim Clijsters
paul@deb503:~$
```

Chapter 13. filters

Commands that are created to be used with a **pipe** are often called **filters**. These **filters** are very small programs that do one specific thing very efficiently. They can be used as **building blocks**.

This chapter will introduce you to the most common **filters**. The combination of simple commands and filters in a long **pipe** allows you to design elegant solutions.

13.1. cat

When between two **pipes**, the **cat** command does nothing (except putting **stdin** on **stdout**).

```
[paul@RHEL4b pipes]$ tac count.txt | cat | cat | cat | cat | cat
five
four
three
two
one
[paul@RHEL4b pipes]$
```

13.2. tee

Writing long **pipes** in Unix is fun, but sometimes you may want intermediate results. This is where **tee** comes in handy. The **tee** filter puts **stdin** on **stdout** and also into a file. So **tee** is almost the same as **cat**, except that it has two identical outputs.

```
[paul@RHEL4b pipes]$ tac count.txt | tee temp.txt | tac
one
two
three
four
five
[paul@RHEL4b pipes]$ cat temp.txt
five
four
three
two
one
[paul@RHEL4b pipes]$
```

13.3. grep

The **grep** filter is famous among Unix users. The most common use of **grep** is to filter lines of text containing (or not containing) a certain string.

```
[paul@RHEL4b pipes]$ cat tennis.txt
Amelie Mauresmo, Fra
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
Venus Williams, USA
[paul@RHEL4b pipes]$ cat tennis.txt | grep Williams
Serena Williams, usa
Venus Williams, USA
```

You can write this without the **cat**.

```
[paul@RHEL4b pipes]$ grep Williams tennis.txt
Serena Williams, usa
Venus Williams, USA
```

One of the most useful options of **grep** is **grep -i** which filters in a case insensitive way.

```
[paul@RHEL4b pipes]$ grep Bel tennis.txt
Justine Henin, Bel
[paul@RHEL4b pipes]$ grep -i Bel tennis.txt
```

```
Kim Clijsters, BEL
Justine Henin, Bel
[paul@RHEL4b pipes]$
```

Another very useful option is **grep -v** which outputs lines not matching the string.

```
[paul@RHEL4b pipes]$ grep -v Fra tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
Venus Williams, USA
[paul@RHEL4b pipes]$
```

And of course, both options can be combined to filter all lines not containing a case insensitive string.

```
[paul@RHEL4b pipes]$ grep -vi usa tennis.txt
Amelie Mauresmo, Fra
Kim Clijsters, BEL
Justine Henin, Bel
[paul@RHEL4b pipes]$
```

With **grep -A1** one line **after** the result is also displayed.

```
paul@debian5:~/pipes$ grep -A1 Henin tennis.txt
Justine Henin, Bel
Serena Williams, usa
```

With **grep -B1** one line **before** the result is also displayed.

```
paul@debian5:~/pipes$ grep -B1 Henin tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
```

With **grep -C1** (context) one line **before** and one **after** are also displayed. All three options (A,B, and C) can display any number of lines (using e.g. A2, B4 or C20).

```
paul@debian5:~/pipes$ grep -C1 Henin tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
```

13.4. cut

The **cut** filter can select columns from files, depending on a delimiter or a count of bytes. The screenshot below uses **cut** to filter for the username and userid in the **/etc/passwd** file. It uses the colon as a delimiter, and selects fields 1 and 3.

```
[paul@RHEL4b pipes]$ cut -d: -f1,3 /etc/passwd | tail -4
Figo:510
Pfaff:511
Harry:516
Hermione:517
[paul@RHEL4b pipes]$
```

When using a space as the delimiter for **cut**, you have to quote the space.

```
[paul@RHEL4b pipes]$ cut -d" " -f1 tennis.txt
Amelie
Kim
Justine
Serena
Venus
[paul@RHEL4b pipes]$
```

This example uses **cut** to display the second to the seventh character of **/etc/passwd**.

```
[paul@RHEL4b pipes]$ cut -c2-7 /etc/passwd | tail -4
igo:x:
faff:x
arry:x
ermion
[paul@RHEL4b pipes]$
```

13.5. tr

You can translate characters with **tr**. The screenshot shows the translation of all occurrences of **e** to **E**.

```
[paul@RHEL4b pipes]$ cat tennis.txt | tr 'e' 'E'
AmElie MaurEsmo, Fra
Kim CliJstErs, BEL
JustinE HEnin, BEl
SErEna Williams, usa
VEnus Williams, USA
```

Here we set all letters to uppercase by defining two ranges.

```
[paul@RHEL4b pipes]$ cat tennis.txt | tr 'a-z' 'A-Z'
AMELIE MAURES MO, FRA
KIM CLIJSTERS, BEL
JUSTINE HENIN, BEL
SERENA WILLIAMS, USA
VENUS WILLIAMS, USA
[paul@RHEL4b pipes]$
```

Here we translate all newlines to spaces.

```
[paul@RHEL4b pipes]$ cat count.txt
one
two
```

```
three
four
five
[paul@RHEL4b pipes]$ cat count.txt | tr '\n' ' '
one two three four five [paul@RHEL4b pipes]$
```

The **tr -s** filter can also be used to squeeze multiple occurrences of a character to one.

```
[paul@RHEL4b pipes]$ cat spaces.txt
one    two        three
      four   five six
[paul@RHEL4b pipes]$ cat spaces.txt | tr -s ' '
one two three
  four five six
[paul@RHEL4b pipes]$
```

You can also use **tr** to 'encrypt' texts with **rot13**.

```
[paul@RHEL4b pipes]$ cat count.txt | tr 'a-z' 'nopqrstuvwxyzabcdefghijklm'
bar
gjb
guerr
sbhe
svir
[paul@RHEL4b pipes]$ cat count.txt | tr 'a-z' 'n-za-m'
bar
gjb
guerr
sbhe
svir
[paul@RHEL4b pipes]$
```

This last example uses **tr -d** to delete characters.

```
paul@debian5:~/pipes$ cat tennis.txt | tr -d e
Amlı Maursmo, Fra
Kim Clijstrs, BEL
Justin Hnin, Bl
Srna Williams, usa
Vnus Williams, USA
```

13.6. wc

Counting words, lines and characters is easy with **wc**.

```
[paul@RHEL4b pipes]$ wc tennis.txt
 5  15 100 tennis.txt
[paul@RHEL4b pipes]$ wc -l tennis.txt
5 tennis.txt
[paul@RHEL4b pipes]$ wc -w tennis.txt
15 tennis.txt
[paul@RHEL4b pipes]$ wc -c tennis.txt
100 tennis.txt
[paul@RHEL4b pipes]$
```

13.7. sort

The **sort** filter will default to an alphabetical sort.

```
paul@debian5:~/pipes$ cat music.txt
Queen
Brel
Led Zeppelin
Abba
paul@debian5:~/pipes$ sort music.txt
Abba
Brel
Led Zeppelin
Queen
```

But the **sort** filter has many options to tweak its usage. This example shows sorting different columns (column 1 or column 2).

```
[paul@RHEL4b pipes]$ sort -k1 country.txt
Belgium, Brussels, 10
France, Paris, 60
Germany, Berlin, 100
Iran, Teheran, 70
Italy, Rome, 50
[paul@RHEL4b pipes]$ sort -k2 country.txt
Germany, Berlin, 100
Belgium, Brussels, 10
France, Paris, 60
Italy, Rome, 50
Iran, Teheran, 70
```

The screenshot below shows the difference between an alphabetical sort and a numerical sort (both on the third column).

```
[paul@RHEL4b pipes]$ sort -k3 country.txt
Belgium, Brussels, 10
Germany, Berlin, 100
Italy, Rome, 50
France, Paris, 60
Iran, Teheran, 70
[paul@RHEL4b pipes]$ sort -n -k3 country.txt
Belgium, Brussels, 10
Italy, Rome, 50
France, Paris, 60
Iran, Teheran, 70
Germany, Berlin, 100
```


13.8. uniq

With **uniq** you can remove duplicates from a **sorted list**.

```
paul@debian5:~/pipes$ cat music.txt
Queen
Brel
Queen
Abba
paul@debian5:~/pipes$ sort music.txt
Abba
Brel
Queen
Queen
paul@debian5:~/pipes$ sort music.txt |uniq
Abba
Brel
Queen
```

uniq can also count occurrences with the **-c** option.

```
paul@debian5:~/pipes$ sort music.txt |uniq -c
 1 Abba
 1 Brel
 2 Queen
```

13.9. comm

Comparing streams (or files) can be done with the **comm**. By default **comm** will output three columns. In this example, Abba, Cure and Queen are in both lists, Bowie and Sweet are only in the first file, Turner is only in the second.

```
paul@debian5:~/pipes$ cat > list1.txt
Abba
Bowie
Cure
Queen
Sweet
paul@debian5:~/pipes$ cat > list2.txt
Abba
Cure
Queen
Turner
paul@debian5:~/pipes$ comm list1.txt list2.txt
      Abba
Bowie
      Cure
      Queen
Sweet
      Turner
```

The output of **comm** can be easier to read when outputting only a single column. The digits point out which output columns should not be displayed.

```
paul@debian5:~/pipes$ comm -12 list1.txt list2.txt
Abba
Cure
Queen
paul@debian5:~/pipes$ comm -13 list1.txt list2.txt
Turner
paul@debian5:~/pipes$ comm -23 list1.txt list2.txt
Bowie
Sweet
```

13.10. od

European humans like to work with ascii characters, but computers store files in bytes. The example below creates a simple file, and then uses **od** to show the contents of the file in hexadecimal bytes

```
paul@laika:~/test$ cat > text.txt
abcdefg
1234567
paul@laika:~/test$ od -t x1 text.txt
0000000 61 62 63 64 65 66 67 0a 31 32 33 34 35 36 37 0a
0000020
```

The same file can also be displayed in octal bytes.

```
paul@laika:~/test$ od -b text.txt
0000000 141 142 143 144 145 146 147 012 061 062 063 064 065 066 067 012
0000020
```

And here is the file in ascii (or backslashed) characters.

```
paul@laika:~/test$ od -c text.txt
0000000  a  b  c  d  e  f  g  \n  1  2  3  4  5  6  7  \n
0000020
```

13.11. sed

The stream **editor** **sed** can perform editing functions in the stream, using **regular expressions**.

```
paul@debian5:~/pipes$ echo level5 | sed 's/5/42/'
level42
paul@debian5:~/pipes$ echo level5 | sed 's/level/jump/'
jump5
```

Add **g** for global replacements (all occurrences of the string per line).

```
paul@debian5:~/pipes$ echo level5 level7 | sed 's/level/jump/'
jump5 level7
paul@debian5:~/pipes$ echo level5 level7 | sed 's/level/jump/g'
jump5 jump7
```

With **d** you can remove lines from a stream containing a character.

```
paul@debian5:~/test42$ cat tennis.txt
Venus Williams, USA
Martina Hingis, SUI
Justine Henin, BE
Serena williams, USA
Kim Clijsters, BE
Yanina Wickmayer, BE
paul@debian5:~/test42$ cat tennis.txt | sed '/BE/d'
Venus Williams, USA
Martina Hingis, SUI
Serena williams, USA
```

13.12. pipe examples

13.12.1. who | wc

How many users are logged on to this system ?

```
[paul@RHEL4b pipes]$ who
root      tty1          Jul 25 10:50
paul      pts/0           Jul 25 09:29 (laika)
Harry    pts/1           Jul 25 12:26 (barry)
paul      pts/2           Jul 25 12:26 (pasha)
[paul@RHEL4b pipes]$ who | wc -l
4
```

13.12.2. who | cut | sort

Display a sorted list of logged on users.

```
[paul@RHEL4b pipes]$ who | cut -d' ' -f1 | sort
Harry
paul
paul
root
```

Display a sorted list of logged on users, but every user only once .

```
[paul@RHEL4b pipes]$ who | cut -d' ' -f1 | sort | uniq
Harry
paul
root
```

13.12.3. grep | cut

Display a list of all bash **user accounts** on this computer. Users accounts are explained in detail later.

```
paul@debian5:~$ grep bash /etc/passwd
root:x:0:0:root:/root:/bin/bash
paul:x:1000:1000:paul,,,:/home/paul:/bin/bash
serena:x:1001:1001:./home/serena:/bin/bash
paul@debian5:~$ grep bash /etc/passwd | cut -d: -f1
root
paul
serena
```

13.13. solution: filters

1. Put a sorted list of all bash users in bashusers.txt.

```
grep bash /etc/passwd | cut -d: -f1 | sort > bashusers.txt
```

2. Put a sorted list of all logged on users in onlineusers.txt.

```
who | cut -d' ' -f1 | sort > onlineusers.txt
```

3. Make a list of all filenames in **/etc** that contain the string **conf** in their filename.

```
ls /etc | grep conf
```

4. Make a sorted list of all files in **/etc** that contain the case insensitive string **conf** in their filename.

```
ls /etc | grep -i conf | sort
```

5. Look at the output of **/sbin/ifconfig**. Write a line that displays only ip address and the subnet mask.

```
/sbin/ifconfig | head -2 | grep 'inet ' | tr -s ' ' | cut -d' ' -f3,5
```

6. Write a line that removes all non-letters from a stream.

```
paul@deb503:~$ cat text
This is, yes really! , a text with ?&* too many str$ange# characters ;-)
paul@deb503:~$ cat text | tr -d ',!$?.*&^%#@;()-'
This is yes really a text with too many strange characters
```

7. Write a line that receives a text file, and outputs all words on a separate line.

```
paul@deb503:~$ cat text2
it is very cold today without the sun

paul@deb503:~$ cat text2 | tr ' ' '\n'
it
is
very
cold
today
without
the
sun
```

8. Write a spell checker on the command line. (There may be a dictionary in **/usr/share/dict/**.)

```
paul@rhel ~$ echo "The zun is shining today" > text

paul@rhel ~$ cat > DICT
is
shining
sun
the
```

today

```
paul@rhel ~$ cat text | tr 'A-Z ' 'a-z\n' | sort | uniq | comm -23 - DICT  
zun
```

You could also add the solution from question number 6 to remove non-letters, and **`tr -s ' '`** to remove redundant spaces.

Chapter 14. basic Unix tools

This chapter introduces commands to **find** or **locate** files and to **compress** files, together with other common tools that were not discussed before. While the tools discussed here are technically not considered **filters**, they can be used in **pipes**.

14.1. find

The **find** command can be very useful at the start of a pipe to search for files. Here are some examples. You might want to add **2>/dev/null** to the command lines to avoid cluttering your screen with error messages.

Find all files in **/etc** and put the list in **etcfiles.txt**

```
find /etc > etcfiles.txt
```

Find all files of the entire system and put the list in **allfiles.txt**

```
find / > allfiles.txt
```

Find files that end in **.conf** in the current directory (and all subdirs).

```
find . -name "*.conf"
```

Find files of type file (not directory, pipe or etc.) that end in **.conf**.

```
find . -type f -name "*.conf"
```

Find files of type directory that end in **.bak**.

```
find /data -type d -name "*.bak"
```

Find files that are newer than **file42.txt**

```
find . -newer file42.txt
```

Find can also execute another command on every file found. This example will look for ***.odf** files and copy them to **/backup/**.

```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

Find can also execute, after your confirmation, another command on every file found. This example will remove ***.odf** files if you approve of it for every file found.

```
find /data -name "*.odf" -ok rm {} \;
```

14.2. locate

The **locate** tool is very different from **find** in that it uses an index to locate files. This is a lot faster than traversing all the directories, but it also means that it is always outdated. If the index does not exist yet, then you have to create it (as root on Red Hat Enterprise Linux) with the **updatedb** command.

```
[paul@RHEL4b ~]$ locate Samba
warning: locate: could not open database: /var/lib/slocate/slocate.db:...
warning: You need to run the 'updatedb' command (as root) to create th...
Please have a look at /etc/updatedb.conf to enable the daily cron job.
[paul@RHEL4b ~]$ updatedb
fatal error: updatedb: You are not authorized to create a default sloc...
[paul@RHEL4b ~]$ su -
Password:
[root@RHEL4b ~]# updatedb
[root@RHEL4b ~]#
```

Most Linux distributions will schedule the **updatedb** to run once every day.

14.3. date

The **date** command can display the date, time, time zone and more.

```
paul@rhel55 ~$ date
Sat Apr 17 12:44:30 CEST 2010
```

A date string can be customised to display the format of your choice. Check the man page for more options.

```
paul@rhel55 ~$ date +%A %d-%m-%Y
Saturday 17-04-2010
```

Time on any Unix is calculated in number of seconds since 1969 (the first second being the first second of the first of January 1970). Use **date +%s** to display Unix time in seconds.

```
paul@rhel55 ~$ date +%s
1271501080
```

When will this seconds counter reach two thousand million ?

```
paul@rhel55 ~$ date -d '1970-01-01 + 2000000000 seconds'
Wed May 18 04:33:20 CEST 2033
```

14.4. cal

The **cal** command displays the current month, with the current day highlighted.

```
paul@rhel55 ~$ cal
      April 2010
Su Mo Tu We Th Fr Sa
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30
```

You can select any month in the past or the future.

```
paul@rhel55 ~$ cal 2 1970
      February 1970
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
```

14.5. sleep

The **sleep** command is sometimes used in scripts to wait a number of seconds. This example shows a five second **sleep**.

```
paul@rhel55 ~$ sleep 5
paul@rhel55 ~$
```

14.6. time

The **time** command can display how long it takes to execute a command. The **date** command takes only a little time.

```
paul@rhel55 ~$ time date
Sat Apr 17 13:08:27 CEST 2010

real    0m0.014s
user    0m0.008s
sys     0m0.006s
```

The **sleep 5** command takes five **real** seconds to execute, but consumes little **cpu time**.

```
paul@rhel55 ~$ time sleep 5

real    0m5.018s
user    0m0.005s
sys     0m0.011s
```

This **bzip2** command compresses a file and uses a lot of **cpu time**.

```
paul@rhel55 ~$ time bzip2 text.txt

real    0m2.368s
user    0m0.847s
sys     0m0.539s
```

14.7. gzip - gunzip

Users never have enough disk space, so compression comes in handy. The **gzip** command can make files take up less space.

```
paul@rhel155 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
paul@rhel155 ~$ gzip text.txt
paul@rhel155 ~$ ls -lh text.txt.gz
-rw-rw-r-- 1 paul paul 760K Apr 17 13:11 text.txt.gz
```

You can get the original back with **gunzip**.

```
paul@rhel155 ~$ gunzip text.txt.gz
paul@rhel155 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
```

14.8. zcat - zmore

Text files that are compressed with **gzip** can be viewed with **zcat** and **zmore**.

```
paul@rhel155 ~$ head -4 text.txt
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
paul@rhel155 ~$ gzip text.txt
paul@rhel155 ~$ zcat text.txt.gz | head -4
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
```

14.9. bzip2 - bunzip2

Files can also be compressed with **bzip2** which takes a little more time than **gzip**, but compresses better.

```
paul@rhel155 ~$ bzip2 text.txt
paul@rhel155 ~$ ls -lh text.txt.bz2
-rw-rw-r-- 1 paul paul 569K Apr 17 13:11 text.txt.bz2
```

Files can be uncompressed again with **bunzip2**.

```
paul@rhel155 ~$ bunzip2 text.txt.bz2
paul@rhel155 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
```

14.10. bzip2 - bzip2

And in the same way **bzcat** and **bzmore** can display files compressed with **bzip2**.

```
paul@rhel155 ~$ bzip2 text.txt
paul@rhel155 ~$ bzcat text.txt.bz2 | head -4
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
```

14.11. solution: basic Unix tools

1. Explain the difference between these two commands. This question is very important. If you don't know the answer, then look back at the **shell** chapter.

```
find /data -name "*.txt"
```

```
find /data -name *.txt
```

When ***.txt** is quoted then the shell will not touch it. The **find** tool will look in the **/data** for all files ending in **.txt**.

When ***.txt** is not quoted then the shell might expand this (when one or more files that ends in **.txt** exist in the current directory). The **find** might show a different result, or can result in a syntax error.

2. Explain the difference between these two statements. Will they both work when there are 200 **.odf** files in **/data** ? How about when there are 2 million **.odf** files ?

```
find /data -name "*.odf" > data_odf.txt
```

```
find /data/*.odf > data_odf.txt
```

The first **find** will output all **.odf** filenames in **/data** and all subdirectories. The shell will redirect this to a file.

The second find will output all files named **.odf** in **/data** and will also output all files that exist in directories named ***.odf** (in **/data**).

With two million files the command line would be expanded beyond the maximum that the shell can accept. The last part of the command line would be lost.

3. Write a find command that finds all files created after January 30th 2010.

```
touch -t 201001302359 marker_date  
find . -type f -newer marker_date
```

There is another solution :

```
find . -type f -newerat "20100130 23:59:59"
```

4. Write a find command that finds all ***.odf** files created in September 2009.

```
touch -t 200908312359 marker_start  
touch -t 200910010000 marker_end  
find . -type f -name "*.odf" -newer marker_start ! -newer marker_end
```

The exclamation mark **! -newer** can be read as **not newer**.

5. Count the number of ***.conf** files in **/etc** and all its subdirs.

```
find /etc -type f -name '*.conf' | wc -l
```

6. Here are two commands that do the same thing: copy ***.odf** files to **/backup/** . What would be a reason to replace the first command with the second ? Again, this is an important question.

```
cp -r /data/*.odf /backup/
```



```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

The first might fail when there are too many files to fit on one command line.

7. Create a file called **loctest.txt**. Can you find this file with **locate** ? Why not ? How do you make locate find this file ?

You cannot locate this with **locate** because it is not yet in the index.

```
updatedb
```

8. Use find and -exec to rename all .htm files to .html.

```
paul@rhel55 ~$ find . -name '*.htm'
./one.htm
./two.htm
paul@rhel55 ~$ find . -name '*.htm' -exec mv {} {}1 \;
paul@rhel55 ~$ find . -name '*.htm*'
./one.html
./two.html
```

9. Issue the **date** command. Now display the date in YYYY/MM/DD format.

```
date +%Y/%m/%d
```

10. Issue the **cal** command. Display a calendar of 1582 and 1752. Notice anything special ?

```
cal 1582
```

The calendars are different depending on the country. Check <http://linux-training.be/files/studentfiles/dates.txt>

Chapter 15. regular expressions

Regular expressions are a very powerful tool in Linux. They can be used with a variety of programs like bash, vi, rename, grep, sed, and more.

This chapter introduces you to the basics of **regular expressions**.

15.1. regex versions

There are three different versions of regular expression syntax:

```
BRE: Basic Regular Expressions  
ERE: Extended Regular Expressions  
PRCE: Perl Regular Expressions
```

Depending on the tool being used, one or more of these syntaxes can be used.

For example the **grep** tool has the **-E** option to force a string to be read as ERE while **-G** forces BRE and **-P** forces PRCE.

Note that **grep** also has **-F** to force the string to be read literally.

The **sed** tool also has options to choose a regex syntax.

Read the manual of the tools you use!

15.2. grep

15.2.1. print lines matching a pattern

grep is a popular Linux tool to search for lines that match a certain pattern. Below are some examples of the simplest **regular expressions**.

This is the contents of the test file. This file contains three lines (or three **newline** characters).

```
paul@rhel65:~$ cat names
Tania
Laura
Valentina
```

When **grepping** for a single character, only the lines containing that character are returned.

```
paul@rhel65:~$ grep u names
Laura
paul@rhel65:~$ grep e names
Valentina
paul@rhel65:~$ grep i names
Tania
Valentina
```

The pattern matching in this example should be very straightforward; if the given character occurs on a line, then **grep** will return that line.

15.2.2. concatenating characters

Two concatenated characters will have to be concatenated in the same way to have a match.

This example demonstrates that **ia** will match **Tania** but not **Valentina** and **in** will match **Valentina** but not **Tania**.

```
paul@rhel65:~$ grep a names
Tania
Laura
Valentina
paul@rhel65:~$ grep ia names
Tania
paul@rhel65:~$ grep in names
Valentina
paul@rhel65:~$
```

15.2.3. one or the other

PRCE and ERE both use the pipe symbol to signify OR. In this example we **grep** for lines containing the letter i or the letter a.

```
paul@debian7:~$ cat list
Tania
Laura
paul@debian7:~$ grep -E 'i|a' list
Tania
Laura
```

Note that we use the **-E** switch of grep to force interpretation of our string as an ERE.

We need to **escape** the pipe symbol in a BRE to get the same logical OR.

```
paul@debian7:~$ grep -G 'i|a' list
paul@debian7:~$ grep -G 'i\\|a' list
Tania
Laura
```

15.2.4. one or more

The ***** signifies zero, one or more occurrences of the previous and the **+** signifies one or more of the previous.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o*' list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o+' list2
lol
lool
loool
paul@debian7:~$
```

15.2.5. match the end of a string

For the following examples, we will use this file.

```
paul@debian7:~$ cat names
Tania
Laura
Valentina
Fleur
Floor
```

The two examples below show how to use the **dollar character** to match the end of a string.

```
paul@debian7:~$ grep a$ names
Tania
Laura
Valentina
paul@debian7:~$ grep r$ names
Fleur
Floor
```

15.2.6. match the start of a string

The **caret character** (^) will match a string at the start (or the beginning) of a line.

Given the same file as above, here are two examples.

```
paul@debian7:~$ grep ^Val names
Valentina
paul@debian7:~$ grep ^F names
Fleur
Floor
```

Both the dollar sign and the little hat are called **anchors** in a regex.

15.2.7. separating words

Regular expressions use a **\b** sequence to reference a word separator. Take for example this file:

```
paul@debian7:~$ cat text
The governor is governing.
The winter is over.
Can you get over there?
```

Simply grepping for **over** will give too many results.

```
paul@debian7:~$ grep over text
The governor is governing.
The winter is over.
Can you get over there?
```

Surrounding the searched word with spaces is not a good solution (because other characters can be word separators). This screenshot below show how to use **\b** to find only the searched word:

```
paul@debian7:~$ grep '\bover\b' text
The winter is over.
Can you get over there?
paul@debian7:~$
```

Note that **grep** also has a **-w** option to grep for words.

```
paul@debian7:~$ cat text
The governor is governing.
The winter is over.
Can you get over there?
paul@debian7:~$ grep -w over text
The winter is over.
Can you get over there?
paul@debian7:~$
```


15.2.8. grep features

Sometimes it is easier to combine a simple regex with **grep** options, than it is to write a more complex regex. These options were discussed before:

```
grep -i
grep -v
grep -w
grep -A5
grep -B5
grep -C5
```

15.2.9. preventing shell expansion of a regex

The dollar sign is a special character, both for the regex and also for the shell (remember variables and embedded shells). Therefore it is advised to always quote the regex, this prevents shell expansion.

```
paul@debian7:~$ grep 'r$' names
Fleur
Floor
```

15.3. rename

15.3.1. the rename command

On Debian Linux the **/usr/bin/rename** command is a link to **/usr/bin/prename** installed by the **perl** package.

```
paul@pi ~ $ dpkg -S $(readlink -f $(which rename))
perl: /usr/bin/prename
```

Red Hat derived systems do not install the same **rename** command, so this section does not describe **rename** on Red Hat (unless you copy the perl script manually).

There is often confusion on the internet about the rename command because solutions that work fine in Debian (and Ubuntu, xubuntu, Mint, ...) cannot be used in Red Hat (and CentOS, Fedora, ...).

15.3.2. perl

The **rename** command is actually a perl script that uses **perl regular expressions**. The complete manual for these can be found by typing **perldoc perlrequick** (after installing **perldoc**).

```
root@pi:~# aptitude install perl-doc
The following NEW packages will be installed:
  perl-doc
0 packages upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 8,170 kB of archives. After unpacking 13.2 MB will be used.
Get: 1 http://mirrordirector.raspbian.org/raspbian/ wheezy/main perl-do...
Fetched 8,170 kB in 19s (412 kB/s)
Selecting previously unselected package perl-doc.
(Reading database ... 67121 files and directories currently installed.)
Unpacking perl-doc (from .../perl-doc_5.14.2-21+rpi2_all.deb) ...
Adding 'diversion of /usr/bin/perldoc to /usr/bin/perldoc.stub by perl-doc'
Processing triggers for man-db ...
Setting up perl-doc (5.14.2-21+rpi2) ...

root@pi:~# perldoc perlrequick
```

15.3.3. well known syntax

The most common use of the **rename** is to search for filenames matching a certain **string** and replacing this string with an **other string**.

This is often presented as **s/string/other string/** as seen in this example:

```
paul@pi ~ $ ls
abc      allfiles.TXT  bllfiles.TXT  Scratch      tennis2.TXT
abc.conf backup        cllfiles.TXT  temp.TXT     tennis.TXT
paul@pi ~ $ rename 's/TXT/text/' *
paul@pi ~ $ ls
abc      allfiles.text  bllfiles.text  Scratch      tennis2.text
abc.conf backup        cllfiles.text  temp.text    tennis.text
```

And here is another example that uses **rename** with the well know syntax to change the extensions of the same files once more:

```
paul@pi ~ $ ls
abc      allfiles.text  bllfiles.text  Scratch      tennis2.text
abc.conf backup        cllfiles.text  temp.text    tennis.text
paul@pi ~ $ rename 's/text/txt/' *.text
paul@pi ~ $ ls
abc      allfiles.txt  bllfiles.txt  Scratch      tennis2.txt
abc.conf backup        cllfiles.txt  temp.txt     tennis.txt
paul@pi ~ $
```

These two examples appear to work because the strings we used only exist at the end of the filename. Remember that file extensions have no meaning in the bash shell.

The next example shows what can go wrong with this syntax.

```
paul@pi ~ $ touch atxt.txt
paul@pi ~ $ rename 's/txt/problem/' atxt.txt
paul@pi ~ $ ls
abc      allfiles.txt  backup        cllfiles.txt  temp.txt     tennis.txt
abc.conf aproblem.txt  bllfiles.txt  Scratch      tennis2.txt
paul@pi ~ $
```

Only the first occurrence of the searched string is replaced.

15.3.4. a global replace

The syntax used in the previous example can be described as **s/regex/replacement/**. This is simple and straightforward, you enter a **regex** between the first two slashes and a **replacement string** between the last two.

This example expands this syntax only a little, by adding a **modifier**.

```
paul@pi ~ $ rename -n 's/TXT/txt/g' aTXT.TXT
aTXT.TXT renamed as atxt.txt
paul@pi ~ $
```

The syntax we use now can be described as **s/regex/replacement/g** where **s** signifies **switch** and **g** stands for **global**.

Note that this example used the **-n** switch to show what is being done (instead of actually renaming the file).

15.3.5. case insensitive replace

Another **modifier** that can be useful is **i**. this example shows how to replace a case insensitive string with another string.

```
paul@debian7:~/files$ ls
file1.text  file2.TEXT  file3.txt
paul@debian7:~/files$ rename 's/.text/.txt/i' *
paul@debian7:~/files$ ls
file1.txt  file2.txt  file3.txt
paul@debian7:~/files$
```

15.3.6. renaming extensions

Command line Linux has no knowledge of MS-DOS like extensions, but many end users and graphical application do use them.

Here is an example on how to use **rename** to only rename the file extension. It uses the dollar sign to mark the ending of the filename.

```
paul@pi ~ $ ls *.txt
allfiles.txt  bllfiles.txt  cllfiles.txt  really.txt.txt  temp.txt  tennis.txt
paul@pi ~ $ rename 's/.txt$/.TXT/' *.txt
paul@pi ~ $ ls *.TXT
allfiles.TXT  bllfiles.TXT  cllfiles.TXT  really.txt.TXT
temp.TXT      tennis.TXT
paul@pi ~ $
```

Note that the **dollar sign** in the regex means **at the end**. Without the dollar sign this command would fail on the really.txt.txt file.

15.4. sed

15.4.1. stream editor

The **stream editor** or short **sed** uses **regex** for stream editing.

In this example **sed** is used to replace a string.

```
echo Sunday | sed 's/Sun/Mon/'  
Monday
```

The slashes can be replaced by a couple of other characters, which can be handy in some cases to improve readability.

```
echo Sunday | sed 's:Sun:Mon:'  
Monday  
echo Sunday | sed 's_Sun_Mon_'  
Monday  
echo Sunday | sed 's|Sun|Mon|'  
Monday
```

15.4.2. interactive editor

While **sed** is meant to be used in a stream, it can also be used interactively on a file.

```
paul@debian7:~/files$ echo Sunday > today  
paul@debian7:~/files$ cat today  
Sunday  
paul@debian7:~/files$ sed -i 's/Sun/Mon/' today  
paul@debian7:~/files$ cat today  
Monday
```

15.4.3. simple back referencing

The **ampersand** character can be used to reference the searched (and found) string.

In this example the **ampersand** is used to double the occurrence of the found string.

```
echo Sunday | sed 's/Sun/&&/'  
SunSunday  
echo Sunday | sed 's/day/&&/'  
Sundayday
```

15.4.4. back referencing

Parentheses (often called round brackets) are used to group sections of the regex so they can later be referenced.

Consider this simple example:

```
paul@debian7:~$ echo Sunday | sed 's_\(Sun\)_\1ny_'  
Sunnyday  
paul@debian7:~$ echo Sunday | sed 's_\(Sun\)_\1ny \1_'  
Sunny Sunday
```

15.4.5. a dot for any character

In a **regex** a simple dot can signify any character.

```
paul@debian7:~$ echo 2014-04-01 | sed 's/....-.... /YYYY-MM-DD/'  
YYYY-MM-DD  
paul@debian7:~$ echo abcd-ef-gh | sed 's/....-.... /YYYY-MM-DD/'  
YYYY-MM-DD
```

15.4.6. multiple back referencing

When more than one pair of **parentheses** is used, each of them can be referenced separately by consecutive numbers.

```
paul@debian7:~$ echo 2014-04-01 | sed 's/\(...\) - \(...\) - \(...\) /\1+\2+\3/'  
2014+04+01  
paul@debian7:~$ echo 2014-04-01 | sed 's/\(....\) - \(...\) - \(...\) /\3:\2:\1/'  
01:04:2014
```

This feature is called **grouping**.

15.4.7. white space

The `\s` can refer to white space such as a space or a tab.

This example looks for white spaces (`\s`) globally and replaces them with 1 space.

```
paul@debian7:~$ echo -e 'today\tis\twarm'
today    is        warm
paul@debian7:~$ echo -e 'today\tis\twarm' | sed 's_\s_ _g'
today is warm
```

15.4.8. optional occurrence

A question mark signifies that the previous is **optional**.

The example below searches for three consecutive letter o, but the third o is optional.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'ooo?' list2
lool
loool
paul@debian7:~$ cat list2 | sed 's/ooo\?/A/'
ll
lol
lAl
lAl
```

15.4.9. exactly n times

You can demand an exact number of times the oprevious has to occur.

This example wants exactly three o's.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o{3}' list2
loool
paul@debian7:~$ cat list2 | sed 's/o\{3\}/A/'
ll
lol
lool
lAl
paul@debian7:~$
```

15.4.10. between n and m times

And here we demand exactly from minimum 2 to maximum 3 times.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o{2,3}' list2
lool
loool
paul@debian7:~$ grep 'o\{2,3\}' list2
lool
loool
paul@debian7:~$ cat list2 | sed 's/o\{2,3\}/A/'
ll
lol
lAl
lAl
paul@debian7:~$
```

15.5. bash history

The **bash shell** can also interpret some regular expressions.

This example shows how to manipulate the exclamation mark history feature of the bash shell.

```
paul@debian7:~$ mkdir hist
paul@debian7:~$ cd hist/
paul@debian7:~/hist$ touch file1 file2 file3
paul@debian7:~/hist$ ls -l file1
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file1
paul@debian7:~/hist$ !l
ls -l file1
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file1
paul@debian7:~/hist$ !l:s/1/3
ls -l file3
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file3
paul@debian7:~/hist$
```

This also works with the history numbers in bash.

```
paul@debian7:~/hist$ history 6
2089  mkdir hist
2090  cd hist/
2091  touch file1 file2 file3
2092  ls -l file1
2093  ls -l file3
2094  history 6
paul@debian7:~/hist$ !2092
ls -l file1
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file1
paul@debian7:~/hist$ !2092:s/1/2
ls -l file2
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file2
paul@debian7:~/hist$
```

Chapter 16. scripting introduction

Shells like **bash** and **Korn** have support for programming constructs that can be saved as **scripts**. These **scripts** in turn then become more **shell** commands. Many Linux commands are **scripts**. **User profile scripts** are run when a user logs on and **init scripts** are run when a **daemon** is stopped or started.

This means that system administrators also need basic knowledge of **scripting** to understand how their servers and their applications are started, updated, upgraded, patched, maintained, configured and removed, and also to understand how a user environment is built.

The goal of this chapter is to give you enough information to be able to read and understand scripts. Not to become a writer of complex scripts.

16.1. prerequisites

You should have read and understood **part III shell expansion** and **part IV pipes and commands** before starting this chapter.

16.2. hello world

Just like in every programming course, we start with a simple **hello_world** script. The following script will output **Hello World**.

```
echo Hello World
```

After creating this simple script in **vi** or with **echo**, you'll have to **chmod +x hello_world** to make it executable. And unless you add the scripts directory to your path, you'll have to type the path to the script for the shell to be able to find it.

```
[paul@RHEL4a ~]$ echo echo Hello World > hello_world
[paul@RHEL4a ~]$ chmod +x hello_world
[paul@RHEL4a ~]$ ./hello_world
Hello World
[paul@RHEL4a ~]$
```

16.3. she-bang

Let's expand our example a little further by putting **#!/bin/bash** on the first line of the script. The **#!** is called a **she-bang** (sometimes called **sha-bang**), where the **she-bang** is the first two characters of the script.

```
#!/bin/bash
echo Hello World
```

You can never be sure which shell a user is running. A script that works flawlessly in **bash** might not work in **ksh**, **cs****h**, or **dash**. To instruct a shell to run your script in a certain shell, you can start your script with a **she-bang** followed by the shell it is supposed to run in. This script will run in a bash shell.

```
#!/bin/bash
echo -n hello
echo A bash subshell `echo -n hello`
```

This script will run in a Korn shell (unless **/bin/ksh** is a hard link to **/bin/bash**). The **/etc/shells** file contains a list of shells on your system.

```
#!/bin/ksh
echo -n hello
echo a Korn subshell `echo -n hello`
```

16.4. comment

Let's expand our example a little further by adding comment lines.

```
#!/bin/bash
#
# Hello World Script
#
echo Hello World
```

16.5. variables

Here is a simple example of a variable inside a script.

```
#!/bin/bash
#
# simple variable in script
#
var1=4
echo var1 = $var1
```

Scripts can contain variables, but since scripts are run in their own shell, the variables do not survive the end of the script.

```
[paul@RHEL4a ~]$ echo $var1

[paul@RHEL4a ~]$ ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1

[paul@RHEL4a ~]$
```

16.6. sourcing a script

Luckily, you can force a script to run in the same shell; this is called **sourcing** a script.

```
[paul@RHEL4a ~]$ source ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1
4
[paul@RHEL4a ~]$
```

The above is identical to the below.

```
[paul@RHEL4a ~]$ . ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1
4
[paul@RHEL4a ~]$
```

16.7. troubleshooting a script

Another way to run a script in a separate shell is by typing **bash** with the name of the script as a parameter.

```
paul@debian6~/test$ bash runme
42
```

Expanding this to **bash -x** allows you to see the commands that the shell is executing (after shell expansion).

```
paul@debian6~/test$ bash -x runme
+ var4=42
+ echo 42
42
paul@debian6~/test$ cat runme
# the runme script
var4=42
echo $var4
paul@debian6~/test$
```

Notice the absence of the commented (#) line, and the replacement of the variable before execution of **echo**.

16.8. prevent setuid root spoofing

Some user may try to perform **setuid** based script **root spoofing**. This is a rare but possible attack. To improve script security and to avoid interpreter spoofing, you need to add **--** after the **#!/bin/bash**, which disables further option processing so the shell will not accept any options.

```
#!/bin/bash -
or
#!/bin/bash --
```

Any arguments after the **--** are treated as filenames and arguments. An argument of **-** is equivalent to **--**.

16.9. solution: introduction to scripting

0. Give each script a different name, keep them for later!

1. Write a script that outputs the name of a city.

```
$ echo 'echo Antwerp' > first.bash
$ chmod +x first.bash
$ ./first.bash
Antwerp
```

2. Make sure the script runs in the bash shell.

```
$ cat first.bash
#!/bin/bash
echo Antwerp
```

3. Make sure the script runs in the Korn shell.

```
$ cat first.bash
#!/bin/ksh
echo Antwerp
```

Note that while first.bash will technically work as a Korn shell script, the name ending in .bash is confusing.

4. Create a script that defines two variables, and outputs their value.

```
$ cat second.bash
#!/bin/bash

var33=300
var42=400

echo $var33 $var42
```

5. The previous script does not influence your current shell (the variables do not exist outside of the script). Now run the script so that it influences your current shell.

```
source second.bash
```

6. Is there a shorter way to **source** the script ?

```
. ./second.bash
```

7. Comment your scripts so that you know what they are doing.

```
$ cat second.bash
#!/bin/bash
# script to test variables and sourcing

# define two variables
var33=300
var42=400

# output the value of these variables
echo $var33 $var42
```

Chapter 17. scripting loops

17.1. test []

The **test** command can test whether something is true or false. Let's start by testing whether 10 is greater than 55.

```
[paul@RHEL4b ~]$ test 10 -gt 55 ; echo $?
1
[paul@RHEL4b ~]$
```

The test command returns 1 if the test fails. And as you see in the next screenshot, test returns 0 when a test succeeds.

```
[paul@RHEL4b ~]$ test 56 -gt 55 ; echo $?
0
[paul@RHEL4b ~]$
```

If you prefer true and false, then write the test like this.

```
[paul@RHEL4b ~]$ test 56 -gt 55 && echo true || echo false
true
[paul@RHEL4b ~]$ test 6 -gt 55 && echo true || echo false
false
```

The test command can also be written as square brackets, the screenshot below is identical to the one above.

```
[paul@RHEL4b ~]$ [ 56 -gt 55 ] && echo true || echo false
true
[paul@RHEL4b ~]$ [ 6 -gt 55 ] && echo true || echo false
false
```

Below are some example tests. Take a look at **man test** to see more options for tests.

[-d foo]	Does the directory foo exist ?
[-e bar]	Does the file bar exist ?
['/etc' = \$PWD]	Is the string /etc equal to the variable \$PWD ?
[\$1 != 'secret']	Is the first parameter different from secret ?
[55 -lt \$bar]	Is 55 less than the value of \$bar ?
[\$foo -ge 1000]	Is the value of \$foo greater or equal to 1000 ?
["abc" < \$bar]	Does abc sort before the value of \$bar ?
[-f foo]	Is foo a regular file ?
[-r bar]	Is bar a readable file ?
[foo -nt bar]	Is file foo newer than file bar ?
[-o nounset]	Is the shell option nounset set ?

Tests can be combined with logical AND and OR.

```
paul@RHEL4b:~$ [ 66 -gt 55 -a 66 -lt 500 ] && echo true || echo false
true
paul@RHEL4b:~$ [ 66 -gt 55 -a 660 -lt 500 ] && echo true || echo false
false
paul@RHEL4b:~$ [ 66 -gt 55 -o 660 -lt 500 ] && echo true || echo false
true
```

17.2. if then else

The **if then else** construction is about choice. If a certain condition is met, then execute something, else execute something else. The example below tests whether a file exists, and if the file exists then a proper message is echoed.

```
#!/bin/bash

if [ -f isit.txt ]
then echo isit.txt exists!
else echo isit.txt not found!
fi
```

If we name the above script 'choice', then it executes like this.

```
[paul@RHEL4a scripts]$ ./choice
isit.txt not found!
[paul@RHEL4a scripts]$ touch isit.txt
[paul@RHEL4a scripts]$ ./choice
isit.txt exists!
[paul@RHEL4a scripts]$
```

17.3. if then elif

You can nest a new **if** inside an **else** with **elif**. This is a simple example.

```
#!/bin/bash
count=42
if [ $count -eq 42 ]
then
    echo "42 is correct."
elif [ $count -gt 42 ]
then
    echo "Too much."
else
    echo "Not enough."
fi
```

17.4. for loop

The example below shows the syntax of a classical **for loop** in bash.

```
for i in 1 2 4
do
    echo $i
done
```

An example of a **for loop** combined with an embedded shell.

```
#!/bin/ksh
for counter in `seq 1 20`
do
    echo counting from 1 to 20, now at $counter
    sleep 1
done
```

The same example as above can be written without the embedded shell using the bash **{from..to}** shorthand.

```
#!/bin/bash
for counter in {1..20}
do
    echo counting from 1 to 20, now at $counter
    sleep 1
done
```

This **for loop** uses file globbing (from the shell expansion). Putting the instruction on the command line has identical functionality.

```
kahlan@solexp11$ ls
count.ksh  go.ksh
kahlan@solexp11$ for file in *.ksh ; do cp $file $file.backup ; done
kahlan@solexp11$ ls
count.ksh  count.ksh.backup  go.ksh  go.ksh.backup
```

17.5. while loop

Below a simple example of a **while loop**.

```
i=100;
while [ $i -ge 0 ] ;
do
    echo Counting down, from 100 to 0, now at $i;
    let i--;
done
```

Endless loops can be made with **while true** or **while :**, where the **colon** is the equivalent of **no operation** in the **Korn** and **bash** shells.

```
#!/bin/ksh
# endless loop
while :
do
    echo hello
    sleep 1
done
```

17.6. until loop

Below a simple example of an **until loop**.

```
let i=100;
until [ $i -le 0 ] ;
do
    echo Counting down, from 100 to 1, now at $i;
    let i--;
done
```

17.7. solution: scripting tests and loops

1. Write a script that uses a **for** loop to count from 3 to 7.

```
#!/bin/bash

for i in 3 4 5 6 7
do
    echo Counting from 3 to 7, now at $i
done
```

2. Write a script that uses a **for** loop to count from 1 to 17000.

```
#!/bin/bash

for i in `seq 1 17000`
do
    echo Counting from 1 to 17000, now at $i
done
```

3. Write a script that uses a **while** loop to count from 3 to 7.

```
#!/bin/bash

i=3
while [ $i -le 7 ]
do
    echo Counting from 3 to 7, now at $i
    let i=i+1
done
```

4. Write a script that uses an **until** loop to count down from 8 to 4.

```
#!/bin/bash

i=8
until [ $i -lt 4 ]
do
    echo Counting down from 8 to 4, now at $i
    let i=i-1
done
```

5. Write a script that counts the number of files ending in **.txt** in the current directory.

```
#!/bin/bash

let i=0
for file in *.txt
do
    let i++
done
echo "There are $i files ending in .txt"
```

6. Wrap an **if** statement around the script so it is also correct when there are zero files ending in **.txt**.

```
#!/bin/bash

ls *.txt > /dev/null 2>&1
if [ $? -ne 0 ]
```

```
then echo "There are 0 files ending in .txt"
else
  let i=0
  for file in *.txt
  do
    let i++
  done
  echo "There are $i files ending in .txt"
fi
```

Chapter 18. scripting parameters

18.1. script parameters

A **bash** shell script can have parameters. The numbering you see in the script below continues if you have more parameters. You also have special parameters containing the number of parameters, a string of all of them, and also the process id, and the last return code. The man page of **bash** has a full list.

```
#!/bin/bash
echo The first argument is $1
echo The second argument is $2
echo The third argument is $3

echo \$ $$  PID of the script
echo \# $#  count arguments
echo \? $?  last return code
echo \* $*  all the arguments
```

Below is the output of the script above in action.

```
[paul@RHEL4a scripts]$ ./pars one two three
The first argument is one
The second argument is two
The third argument is three
$ 5610 PID of the script
# 3 count arguments
? 0 last return code
* one two three all the arguments
```

Once more the same script, but with only two parameters.

```
[paul@RHEL4a scripts]$ ./pars 1 2
The first argument is 1
The second argument is 2
The third argument is
$ 5612 PID of the script
# 2 count arguments
? 0 last return code
* 1 2 all the arguments
[paul@RHEL4a scripts]$
```

Here is another example, where we use **\$0**. The **\$0** parameter contains the name of the script.

```
paul@debian6~$ cat myname
echo this script is called $0
paul@debian6~$ ./myname
this script is called ./myname
paul@debian6~$ mv myname test42
paul@debian6~$ ./test42
this script is called ./test42
```


18.2. shift through parameters

The **shift** statement can parse all **parameters** one by one. This is a sample script.

```
kahlan@solexp11$ cat shift.ksh
#!/bin/ksh

if [ "$#" == "0" ]
then
    echo You have to give at least one parameter.
    exit 1
fi

while (( $# ))
do
    echo You gave me $1
    shift
done
```

Below is some sample output of the script above.

```
kahlan@solexp11$ ./shift.ksh one
You gave me one
kahlan@solexp11$ ./shift.ksh one two three 1201 "33 42"
You gave me one
You gave me two
You gave me three
You gave me 1201
You gave me 33 42
kahlan@solexp11$ ./shift.ksh
You have to give at least one parameter.
```

18.3. runtime input

You can ask the user for input with the **read** command in a script.

```
#!/bin/bash
echo -n Enter a number:
read number
```

18.4. sourcing a config file

The **source** (as seen in the shell chapters) can be used to source a configuration file.

Below a sample configuration file for an application.

```
[paul@RHEL4a scripts]$ cat myApp.conf
# The config file of myApp

# Enter the path here
myAppPath=/var/myApp

# Enter the number of quines here
quines=5
```

And here an application that uses this file.

```
[paul@RHEL4a scripts]$ cat myApp.bash
#!/bin/bash
#
# Welcome to the myApp application
#

. ./myApp.conf

echo There are $quines quines
```

The running application can use the values inside the sourced configuration file.

```
[paul@RHEL4a scripts]$ ./myApp.bash
There are 5 quines
[paul@RHEL4a scripts]$
```

18.5. get script options with getopt

The **getopts** function allows you to parse options given to a command. The following script allows for any combination of the options a, f and z.

```
kahlan@solexp11$ cat options.ksh
#!/bin/ksh

while getopts ":afz" option;
do
  case $option in
    a)
      echo received -a
      ;;
    f)
      echo received -f
      ;;
    z)
      echo received -z
      ;;
    *)
      echo "invalid option -$OPTARG"
      ;;
  esac
done
```

This is sample output from the script above. First we use correct options, then we enter twice an invalid option.

```
kahlan@solexp11$ ./options.ksh
kahlan@solexp11$ ./options.ksh -af
received -a
received -f
kahlan@solexp11$ ./options.ksh -zfg
received -z
received -f
invalid option -g
kahlan@solexp11$ ./options.ksh -a -b -z
received -a
invalid option -b
received -z
```

You can also check for options that need an argument, as this example shows.

```
kahlan@solexp11$ cat argoptions.ksh
#!/bin/ksh

while getopts ":af:z" option;
do
    case $option in
        a)
            echo received -a
            ;;
        f)
            echo received -f with $OPTARG
            ;;
        z)
            echo received -z
            ;;
        :)
            echo "option -$OPTARG needs an argument"
            ;;
        *)
            echo "invalid option -$OPTARG"
            ;;
    esac
done
```

This is sample output from the script above.

```
kahlan@solexp11$ ./argoptions.ksh -a -f hello -z
received -a
received -f with hello
received -z
kahlan@solexp11$ ./argoptions.ksh -zaf 42
received -z
received -a
received -f with 42
kahlan@solexp11$ ./argoptions.ksh -zf
received -z
option -f needs an argument
```

18.6. get shell options with shopt

You can toggle the values of variables controlling optional shell behaviour with the **shopt** built-in shell command. The example below first verifies whether the `cdspell` option is set; it is not. The next `shopt` command sets the value, and the third `shopt` command verifies that the option really is set. You can now use minor spelling mistakes in the `cd` command. The man page of `bash` has a complete list of options.

```
paul@laika:~$ shopt -q cdspell ; echo $?
1
paul@laika:~$ shopt -s cdspell
paul@laika:~$ shopt -q cdspell ; echo $?
0
paul@laika:~$ cd /Etc
/etc
```

18.7. solution: parameters and options

1. Write a script that receives four parameters, and outputs them in reverse order.

```
echo $4 $3 $2 $1
```

2. Write a script that receives two parameters (two filenames) and outputs whether those files exist.

```
#!/bin/bash

if [ -f $1 ]
then echo $1 exists!
else echo $1 not found!
fi

if [ -f $2 ]
then echo $2 exists!
else echo $2 not found!
fi
```

3. Write a script that asks for a filename. Verify existence of the file, then verify that you own the file, and whether it is writable. If not, then make it writable.
4. Make a configuration file for the previous script. Put a logging switch in the config file, logging means writing detailed output of everything the script does to a log file in /tmp.

Chapter 19. more scripting

19.1. eval

eval reads arguments as input to the shell (the resulting commands are executed). This allows using the value of a variable as a variable.

```
paul@deb503:~/test42$ answer=42
paul@deb503:~/test42$ word=answer
paul@deb503:~/test42$ eval x=\$$word ; echo $x
42
```

Both in **bash** and **Korn** the arguments can be quoted.

```
kahlan@solexp11$ answer=42
kahlan@solexp11$ word=answer
kahlan@solexp11$ eval "y=\$$word" ; echo $y
42
```

Sometimes the **eval** is needed to have correct parsing of arguments. Consider this example where the **date** command receives one parameter **1 week ago**.

```
paul@debian6~$ date --date="1 week ago"
Thu Mar  8 21:36:25 CET 2012
```

When we set this command in a variable, then executing that variable fails unless we use **eval**.

```
paul@debian6~$ lastweek='date --date="1 week ago"'
paul@debian6~$ $lastweek
date: extra operand `ago'
Try `date --help' for more information.
paul@debian6~$ eval $lastweek
Thu Mar  8 21:36:39 CET 2012
```

19.2. (())

The **(())** allows for evaluation of numerical expressions.

```
paul@deb503:~/test42$ (( 42 > 33 )) && echo true || echo false
true
paul@deb503:~/test42$ (( 42 > 1201 )) && echo true || echo false
false
paul@deb503:~/test42$ var42=42
paul@deb503:~/test42$ (( 42 == var42 )) && echo true || echo false
true
paul@deb503:~/test42$ (( 42 == $var42 )) && echo true || echo false
true
paul@deb503:~/test42$ var42=33
paul@deb503:~/test42$ (( 42 == var42 )) && echo true || echo false
false
```


19.3. let

The **let** built-in shell function instructs the shell to perform an evaluation of arithmetic expressions. It will return 0 unless the last arithmetic expression evaluates to 0.

```
[paul@RHEL4b ~]$ let x="3 + 4" ; echo $x
7
[paul@RHEL4b ~]$ let x="10 + 100/10" ; echo $x
20
[paul@RHEL4b ~]$ let x="10-2+100/10" ; echo $x
18
[paul@RHEL4b ~]$ let x="10*2+100/10" ; echo $x
30
```

The **shell** can also convert between different bases.

```
[paul@RHEL4b ~]$ let x="0xFF" ; echo $x
255
[paul@RHEL4b ~]$ let x="0xC0" ; echo $x
192
[paul@RHEL4b ~]$ let x="0xA8" ; echo $x
168
[paul@RHEL4b ~]$ let x="8#70" ; echo $x
56
[paul@RHEL4b ~]$ let x="8#77" ; echo $x
63
[paul@RHEL4b ~]$ let x="16#c0" ; echo $x
192
```

There is a difference between assigning a variable directly, or using **let** to evaluate the arithmetic expressions (even if it is just assigning a value).

```
kahlan@solexp11$ dec=15 ; oct=017 ; hex=0x0f
kahlan@solexp11$ echo $dec $oct $hex
15 017 0x0f
kahlan@solexp11$ let dec=15 ; let oct=017 ; let hex=0x0f
kahlan@solexp11$ echo $dec $oct $hex
15 15 15
```

19.4. case

You can sometimes simplify nested if statements with a **case** construct.

```
[paul@RHEL4b ~]$ ./help
What animal did you see ? lion
You better start running fast!
[paul@RHEL4b ~]$ ./help
What animal did you see ? dog
Don't worry, give it a cookie.
[paul@RHEL4b ~]$ cat help
#!/bin/bash
#
# Wild Animals Helpdesk Advice
#
echo -n "What animal did you see ? "
read animal
case $animal in
    "lion" | "tiger")
        echo "You better start running fast!"
        ;;
    "cat")
        echo "Let that mouse go..."
        ;;
    "dog")
        echo "Don't worry, give it a cookie."
        ;;
    "chicken" | "goose" | "duck" )
        echo "Eggs for breakfast!"
        ;;
    "liger")
        echo "Approach and say 'Ah you big fluffy kitty...'"
        ;;
    "babelfish")
        echo "Did it fall out your ear ?"
        ;;
    *)
        echo "You discovered an unknown animal, name it!"
        ;;
esac
[paul@RHEL4b ~]$
```

19.5. shell functions

Shell **functions** can be used to group commands in a logical way.

```
kahlan@solexp11$ cat funcs.ksh
#!/bin/ksh

function greetings {
echo Hello World!
echo and hello to $USER to!
}

echo We will now call a function
greetings
echo The end
```

This is sample output from this script with a **function**.

```
kahlan@solexp11$ ./funcs.ksh
We will now call a function
Hello World!
and hello to kahlan to!
The end
```

A shell function can also receive parameters.

```
kahlan@solexp11$ cat addfunc.ksh
#!/bin/ksh

function plus {
let result="$1 + $2"
echo $1 + $2 = $result
}

plus 3 10
plus 20 13
plus 20 22
```

This script produces the following output.

```
kahlan@solexp11$ ./addfunc.ksh
3 + 10 = 13
20 + 13 = 33
20 + 22 = 42
```

19.6. solution : more scripting

1. Write a script that asks for two numbers, and outputs the sum and product (as shown here).

```
Enter a number: 5
Enter another number: 2

Sum:      5 + 2 = 7
Product:   5 x 2 = 10
```

```
#!/bin/bash

echo -n "Enter a number : "
read n1

echo -n "Enter another number : "
read n2

let sum="$n1+$n2"
let pro="$n1*$n2"

echo -e "Sum\t: $n1 + $n2 = $sum"
echo -e "Product\t: $n1 * $n2 = $pro"
```

2. Improve the previous script to test that the numbers are between 1 and 100, exit with an error if necessary.

```
echo -n "Enter a number between 1 and 100 : "
read n1

if [ $n1 -lt 1 -o $n1 -gt 100 ]
then
    echo Wrong number...
    exit 1
fi
```

3. Improve the previous script to congratulate the user if the sum equals the product.

```
if [ $sum -eq $pro ]
then echo Congratulations $sum == $pro
fi
```

4. Write a script with a case insensitive case statement, using the shopt nocasematch option. The nocasematch option is reset to the value it had before the scripts started.

```
#!/bin/bash
#
# Wild Animals Case Insensitive Helpdesk Advice
#

if shopt -q nocasematch; then
    nocase=yes;
else
    nocase=no;
    shopt -s nocasematch;
fi

echo -n "What animal did you see ? "
read animal

case $animal in
```

```

"lion" | "tiger")
    echo "You better start running fast!"
;;
"cat")
    echo "Let that mouse go..."
;;
"dog")
    echo "Don't worry, give it a cookie."
;;
"chicken" | "goose" | "duck" )
    echo "Eggs for breakfast!"
;;
"liger")
    echo "Approach and say 'Ah you big fluffy kitty.'"
;;
"babelfish")
    echo "Did it fall out your ear ?"
;;
*)
    echo "You discovered an unknown animal, name it!"
;;
esac

if [ nocase = yes ] ; then
    shopt -s nocasematch;
else
    shopt -u nocasematch;
fi

```

5. If time permits (or if you are waiting for other students to finish this practice), take a look at Linux system scripts in /etc/init.d and /etc/rc.d and try to understand them. Where does execution of a script start in /etc/init.d/samba ? There are also some hidden scripts in ~, we

Chapter 20. introduction to users

This little chapter will teach you how to identify your user account on a Unix computer using commands like **who am i**, **id**, and more.

In a second part you will learn how to become another user with the **su** command.

And you will learn how to run a program as another user with **sudo**.

20.1. whoami

The **whoami** command tells you your username.

```
[paul@centos7 ~]$ whoami
paul
[paul@centos7 ~]$
```

20.2. who

The **who** command will give you information about who is logged on the system.

```
[paul@centos7 ~]$ who
root      pts/0      2014-10-10 23:07 (10.104.33.101)
paul      pts/1      2014-10-10 23:30 (10.104.33.101)
laura     pts/2      2014-10-10 23:34 (10.104.33.96)
tania     pts/3      2014-10-10 23:39 (10.104.33.91)
[paul@centos7 ~]$
```

20.3. who am i

With **who am i** the **who** command will display only the line pointing to your current session.

```
[paul@centos7 ~]$ who am i
paul      pts/1      2014-10-10 23:30 (10.104.33.101)
[paul@centos7 ~]$
```

20.4. w

The **w** command shows you who is logged on and what they are doing.

```
[paul@centos7 ~]$ w
 23:34:07 up 31 min,  2 users,  load average: 0.00, 0.01, 0.02
USER      TTY      LOGIN@  IDLE   JCPU   PCPU WHAT
root      pts/0    23:07   15.00s  0.01s  0.01s top
paul      pts/1    23:30    7.00s  0.00s  0.00s w
[paul@centos7 ~]$
```

20.5. id

The **id** command will give you your user id, primary group id, and a list of the groups that you belong to.

```
paul@debian7:~$ id
uid=1000(paul) gid=1000(paul) groups=1000(paul)
```

On RHEL/CentOS you will also get **SELinux** context information with this command.

```
[root@centos7 ~]# id
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r\
:unconfined_t:s0-s0:c0.c1023
```

20.6. su to another user

The **su** command allows a user to run a shell as another user.

```
laura@debian7:~$ su tania
Password:
tania@debian7:/home/laura$
```

20.7. su to root

Yes you can also **su** to become **root**, when you know the **root** password.

```
laura@debian7:~$ su root
Password:
root@debian7:/home/laura#
```

20.8. su as root

You need to know the password of the user you want to substitute to, unless your are logged in as **root**. The **root** user can become any existing user without knowing that user's password.

```
root@debian7:~# id
uid=0(root) gid=0(root) groups=0(root)
root@debian7:~# su - valentina
valentina@debian7:~$
```

20.9. su - \$username

By default, the **su** command maintains the same shell environment. To become another user and also get the target user's environment, issue the **su -** command followed by the target username.

```
root@debian7:~# su laura
laura@debian7:/root$ exit
exit
root@debian7:~# su - laura
laura@debian7:~$ pwd
/home/laura
```

20.10. su -

When no username is provided to **su** or **su -**, the command will assume **root** is the target.

```
tania@debian7:~$ su -
Password:
root@debian7:~#
```


20.11. run a program as another user

The `sudo` program allows a user to start a program with the credentials of another user. Before this works, the system administrator has to set up the `/etc/sudoers` file. This can be useful to delegate administrative tasks to another user (without giving the root password).

The screenshot below shows the usage of **sudo**. User **paul** received the right to run **useradd** with the credentials of **root**. This allows **paul** to create new users on the system without becoming **root** and without knowing the **root password**.

First the command fails for **paul**.

```
paul@debian7:~$ /usr/sbin/useradd -m valentina
useradd: Permission denied.
useradd: cannot lock /etc/passwd; try again later.
```

But with **sudo** it works.

```
paul@debian7:~$ sudo /usr/sbin/useradd -m valentina
[sudo] password for paul:
paul@debian7:~$
```

20.12. visudo

Check the man page of **visudo** before playing with the `/etc/sudoers` file. Editing the **sudoers** is out of scope for this fundamentals book.

```
paul@rhel65:~$ apropos visudo
visudo                (8) - edit the sudoers file
paul@rhel65:~$
```

20.13. sudo su -

On some Linux systems like Ubuntu and Xubuntu, the **root** user does not have a password set. This means that it is not possible to login as **root** (extra security). To perform tasks as **root**, the first user is given all **sudo rights** via the **/etc/sudoers**. In fact all users that are members of the admin group can use sudo to run all commands as root.

```
root@laika:~# grep admin /etc/sudoers
# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL
```

The end result of this is that the user can type **sudo su -** and become root without having to enter the root password. The sudo command does require you to enter your own password. Thus the password prompt in the screenshot below is for sudo, not for su.

```
paul@laika:~$ sudo su -
Password:
root@laika:~#
```

20.14. sudo logging

Using **sudo** without authorization will result in a severe warning:

```
paul@rhel65:~$ sudo su -

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for paul:
paul is not in the sudoers file. This incident will be reported.
paul@rhel65:~$
```

The root user can see this in the **/var/log/secure** on Red Hat and in **/var/log/auth.log** on Debian).

```
root@rhel65:~# tail /var/log/secure | grep sudo | tr -s ' '
Apr 13 16:03:42 rhel65 sudo: paul : user NOT in sudoers ; TTY=pts/0 ; PWD=\
/home/paul ; USER=root ; COMMAND=/bin/su -
root@rhel65:~#
```

20.15. solution: introduction to users

1. Run a command that displays only your currently logged on user name.

```
laura@debian7:~$ whoami
laura
laura@debian7:~$ echo $USER
laura
```

2. Display a list of all logged on users.

```
laura@debian7:~$ who
laura pts/0 2014-10-13 07:22 (10.104.33.101)
laura@debian7:~$
```

3. Display a list of all logged on users including the command they are running at this very moment.

```
laura@debian7:~$ w
07:47:02 up 16 min, 2 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@      IDLE   JCPU   PCPU WHAT
root      pts/0    10.104.33.101  07:30      6.00s  0.04s  0.00s w
root      pts/1    10.104.33.101  07:46      6.00s  0.01s  0.00s sleep 42
laura@debian7:~$
```

4. Display your user name and your unique user identification (userid).

```
laura@debian7:~$ id
uid=1005(laura) gid=1007(laura) groups=1007(laura)
laura@debian7:~$
```

5. Use **su** to switch to another user account (unless you are root, you will need the password of the other account). And get back to the previous account.

```
laura@debian7:~$ su tania
Password:
tania@debian7:/home/laura$ id
uid=1006(tania) gid=1008(tania) groups=1008(tania)
tania@debian7:/home/laura$ exit
laura@debian7:~$
```

6. Now use **su -** to switch to another user and notice the difference.

```
laura@debian7:~$ su - tania
Password:
tania@debian7:~$ pwd
/home/tania
tania@debian7:~$ logout
laura@debian7:~$
```

Note that **su -** gets you into the home directory of **Tania**.

7. Try to create a new user account (when using your normal user account). this should fail. (Details on adding user accounts are explained in the next chapter.)

```
laura@debian7:~$ useradd valentina
-su: useradd: command not found
laura@debian7:~$ /usr/sbin/useradd valentina
useradd: Permission denied.
useradd: cannot lock /etc/passwd; try again later.
```

It is possible that **useradd** is located in **/sbin/useradd** on your computer.

8. Now try the same, but with **sudo** before your command.

```
laura@debian7:~$ sudo /usr/sbin/useradd valentina
[sudo] password for laura:
laura is not in the sudoers file.  This incident will be reported.
laura@debian7:~$
```

Notice that **laura** has no permission to use the **sudo** on this system.

Chapter 21. user management

This chapter will teach you how to use **useradd**, **usermod** and **userdel** to create, modify and remove user accounts.

You will need **root** access on a Linux computer to complete this chapter.

21.1. user management

User management on Linux can be done in three complementary ways. You can use the **graphical** tools provided by your distribution. These tools have a look and feel that depends on the distribution. If you are a novice Linux user on your home system, then use the graphical tool that is provided by your distribution. This will make sure that you do not run into problems.

Another option is to use **command line tools** like `useradd`, `usermod`, `gpasswd`, `passwd` and others. Server administrators are likely to use these tools, since they are familiar and very similar across many different distributions. This chapter will focus on these command line tools.

A third and rather extremist way is to **edit the local configuration files** directly using `vi` (or `vim`). Do not attempt this as a novice on production systems!

21.2. /etc/passwd

The local user database on Linux (and on most Unixes) is `/etc/passwd`.

```
[root@RHEL5 ~]# tail /etc/passwd
inge:x:518:524:art dealer:/home/inge:/bin/ksh
ann:x:519:525:flute player:/home/ann:/bin/bash
frederik:x:520:526:rubius poet:/home/frederik:/bin/bash
steven:x:521:527:roman emperor:/home/steven:/bin/bash
pascal:x:522:528:artist:/home/pascal:/bin/ksh
geert:x:524:530:kernel developer:/home/geert:/bin/bash
wim:x:525:531:master damuti:/home/wim:/bin/bash
sandra:x:526:532:radish stresser:/home/sandra:/bin/bash
annelies:x:527:533:sword fighter:/home/annelies:/bin/bash
laura:x:528:534:art dealer:/home/laura:/bin/ksh
```

As you can see, this file contains seven columns separated by a colon. The columns contain the username, an x, the user id, the primary group id, a description, the name of the home directory, and the login shell.

More information can be found by typing `man 5 passwd`.

```
[root@RHEL5 ~]# man 5 passwd
```

21.3. root

The **root** user also called the **superuser** is the most powerful account on your Linux system. This user can do almost anything, including the creation of other users. The root user always has `userid 0` (regardless of the name of the account).

```
[root@RHEL5 ~]# head -1 /etc/passwd
root:x:0:0:root:/root:/bin/bash
```

21.4. useradd

You can add users with the **useradd** command. The example below shows how to add a user named yanina (last parameter) and at the same time forcing the creation of the home directory (-m), setting the name of the home directory (-d), and setting a description (-c).

```
[root@RHEL5 ~]# useradd -m -d /home/yanina -c "yanina wickmayer" yanina
[root@RHEL5 ~]# tail -1 /etc/passwd
yanina:x:529:529:yanina wickmayer:/home/yanina:/bin/bash
```

The user named yanina received userid 529 and **primary group** id 529.

21.5. /etc/default/useradd

Both Red Hat Enterprise Linux and Debian/Ubuntu have a file called **/etc/default/useradd** that contains some default user options. Besides using cat to display this file, you can also use **useradd -D**.

```
[root@RHEL4 ~]# useradd -D
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
```

21.6. userdel

You can delete the user yanina with **userdel**. The -r option of userdel will also remove the home directory.

```
[root@RHEL5 ~]# userdel -r yanina
```

21.7. usermod

You can modify the properties of a user with the **usermod** command. This example uses **usermod** to change the description of the user harry.

```
[root@RHEL4 ~]# tail -1 /etc/passwd
harry:x:516:520:harry potter:/home/harry:/bin/bash
[root@RHEL4 ~]# usermod -c 'wizard' harry
[root@RHEL4 ~]# tail -1 /etc/passwd
harry:x:516:520:wizard:/home/harry:/bin/bash
```


21.8. creating home directories

The easiest way to create a home directory is to supply the **-m** option with **useradd** (it is likely set as a default option on Linux).

A less easy way is to create a home directory manually with **mkdir** which also requires setting the owner and the permissions on the directory with **chmod** and **chown** (both commands are discussed in detail in another chapter).

```
[root@RHEL5 ~]# mkdir /home/laura
[root@RHEL5 ~]# chown laura:laura /home/laura
[root@RHEL5 ~]# chmod 700 /home/laura
[root@RHEL5 ~]# ls -ld /home/laura/
drwx----- 2 laura laura 4096 Jun 24 15:17 /home/laura/
```

21.9. /etc/skel/

When using **useradd** the **-m** option, the **/etc/skel/** directory is copied to the newly created home directory. The **/etc/skel/** directory contains some (usually hidden) files that contain profile settings and default values for applications. In this way **/etc/skel/** serves as a default home directory and as a default user profile.

```
[root@RHEL5 ~]# ls -la /etc/skel/
total 48
drwxr-xr-x  2 root root  4096 Apr  1 00:11 .
drwxr-xr-x 97 root root 12288 Jun 24 15:36 ..
-rw-r--r--  1 root root    24 Jul 12  2006 .bash_logout
-rw-r--r--  1 root root   176 Jul 12  2006 .bash_profile
-rw-r--r--  1 root root   124 Jul 12  2006 .bashrc
```

21.10. deleting home directories

The **-r** option of **userdel** will make sure that the home directory is deleted together with the user account.

```
[root@RHEL5 ~]# ls -ld /home/wim/
drwx----- 2 wim wim 4096 Jun 24 15:19 /home/wim/
[root@RHEL5 ~]# userdel -r wim
[root@RHEL5 ~]# ls -ld /home/wim/
ls: /home/wim/: No such file or directory
```

21.11. login shell

The `/etc/passwd` file specifies the **login shell** for the user. In the screenshot below you can see that user **annelies** will log in with the `/bin/bash` shell, and user **laura** with the `/bin/ksh` shell.

```
[root@RHEL5 ~]# tail -2 /etc/passwd
annelies:x:527:533:sword fighter:/home/annelies:/bin/bash
laura:x:528:534:art dealer:/home/laura:/bin/ksh
```

You can use the `usermod` command to change the shell for a user.

```
[root@RHEL5 ~]# usermod -s /bin/bash laura
[root@RHEL5 ~]# tail -1 /etc/passwd
laura:x:528:534:art dealer:/home/laura:/bin/bash
```

21.12. chsh

Users can change their login shell with the **chsh** command. First, user **harry** obtains a list of available shells (he could also have done a `cat /etc/shells`) and then changes his login shell to the **Korn shell** (`/bin/ksh`). At the next login, harry will default into ksh instead of bash.

```
[laura@centos7 ~]$ chsh -l
/bin/sh
/bin/bash
/sbin/nologin
/usr/bin/sh
/usr/bin/bash
/usr/sbin/nologin
/bin/ksh
/bin/tcsh
/bin/csh
[laura@centos7 ~]$
```

Note that the `-l` option does not exist on Debian and that the above screenshot assumes that **ksh** and **csh** shells are installed.

The screenshot below shows how **laura** can change her default shell (active on next login).

```
[laura@centos7 ~]$ chsh -s /bin/ksh
Changing shell for laura.
Password:
Shell changed.
```

21.13. solution: user management

1. Create a user account named **serena**, including a home directory and a description (or comment) that reads **Serena Williams**. Do all this in one single command.

```
root@debian7:~# useradd -m -c 'Serena Williams' serena
```

2. Create a user named **venus**, including home directory, bash shell, a description that reads **Venus Williams** all in one single command.

```
root@debian7:~# useradd -m -c "Venus Williams" -s /bin/bash venus
```

3. Verify that both users have correct entries in `/etc/passwd`, `/etc/shadow` and `/etc/group`.

```
root@debian7:~# tail -2 /etc/passwd
serena:x:1008:1010:Serena Williams:/home/serena:/bin/sh
venus:x:1009:1011:Venus Williams:/home/venus:/bin/bash
root@debian7:~# tail -2 /etc/shadow
serena:!:16358:0:99999:7:::
venus:!:16358:0:99999:7:::
root@debian7:~# tail -2 /etc/group
serena:x:1010:
venus:x:1011:
```

4. Verify that their home directory was created.

```
root@debian7:~# ls -lrt /home | tail -2
drwxr-xr-x 2 serena  serena  4096 Oct 15 10:50 serena
drwxr-xr-x 2 venus   venus   4096 Oct 15 10:59 venus
root@debian7:~#
```

5. Create a user named **einstime** with **/bin/date** as his default logon shell.

```
root@debian7:~# useradd -s /bin/date einstime
```

Or even better:

```
root@debian7:~# useradd -s $(which date) einstime
```

7. What happens when you log on with the **einstime** user ? Can you think of a useful real world example for changing a user's login shell to an application ?

```
root@debian7:~# su - einstime
Wed Oct 15 11:05:56 UTC 2014 # You get the output of the date command
root@debian7:~#
```

It can be useful when users need to access only one application on the server. Just logging in opens the application for them, and closing the application automatically logs them out.

8. Create a file named **welcome.txt** and make sure every new user will see this file in their home directory.

```
root@debian7:~# echo Hello > /etc/skel/welcome.txt
```

9. Verify this setup by creating (and deleting) a test user account.

```
root@debian7:~# useradd -m test
root@debian7:~# ls -l /home/test
total 4
-rw-r--r-- 1 test test 6 Oct 15 11:16 welcome.txt
root@debian7:~# userdel -r test
root@debian7:~#
```

10. Change the default login shell for the **serena** user to **/bin/bash**. Verify before and after you make this change.

```
root@debian7:~# grep serena /etc/passwd
serena:x:1008:1010:Serena Williams:/home/serena:/bin/sh
root@debian7:~# usermod -s /bin/bash serena
root@debian7:~# grep serena /etc/passwd
serena:x:1008:1010:Serena Williams:/home/serena:/bin/bash
root@debian7:~#
```

Chapter 22. user passwords

This chapter will tell you more about passwords for local users.

Three methods for setting passwords are explained; using the **passwd** command, using **openssl passwd**, and using the **crypt** function in a C program.

The chapter will also discuss password settings and disabling, suspending or locking accounts.

22.1. passwd

Passwords of users can be set with the **passwd** command. Users will have to provide their old password before twice entering the new one.

```
[tania@centos7 ~]$ passwd
Changing password for user tania.
Changing password for tania.
(current) UNIX password:
New password:
BAD PASSWORD: The password is shorter than 8 characters
New password:
BAD PASSWORD: The password is a palindrome
New password:
BAD PASSWORD: The password is too similar to the old one
passwd: Have exhausted maximum number of retries for service
```

As you can see, the **passwd** tool will do some basic verification to prevent users from using too simple passwords. The **root** user does not have to follow these rules (there will be a warning though). The **root** user also does not have to provide the old password before entering the new password twice.

```
root@debian7:~# passwd tania
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

22.2. shadow file

User passwords are encrypted and kept in **/etc/shadow**. The **/etc/shadow** file is read only and can only be read by root. We will see in the file permissions section how it is possible for users to change their password. For now, you will have to know that users can change their password with the **/usr/bin/passwd** command.

```
[root@centos7 ~]# tail -4 /etc/shadow
paul:$6$ikp2Xta5BT.Tml.p$2TZjNnOYNNQKpwLJqoGJbVsZG5/Fti8ovBRd.VzRbiDSl7TEq\
IaSMH.TeBKnTS/SjlMruW8qffc0JNORW.BTW1:16338:0:99999:7:::
tania:$6$8Z/zovxj$9qvoqT8i9KIrmN.k4EQwAF5ryz5yzNwEvYjAa9L5XVXQu.z4DlvpMREH\
eQpQzvRnqFdKkVj17H5ST.c79HDZw0:16356:0:99999:7:::
laura:$6$glDuTY5e$/NYYWLxfHgZFWeoujaXSMcR.Mz.lGOxtcxFocFVJNb98nbTPhWFXfKWG\
SyYh1WCv6763Wq54.w24Yr3uAZBOM/:16356:0:99999:7:::
valentina:$6$jZa6PVI$luQgqR6En9mZB6mKJ3LXRB4CnFko6LRhbh.v4iqUk9MVreui1lv7\
GxHOUDSKA0N55ZRNhGHa6T2ouFnVno/0o1:16356:0:99999:7:::
[root@centos7 ~]#
```

The **/etc/shadow** file contains nine colon separated columns. The nine fields contain (from left to right) the user name, the encrypted password (note that only inge and laura have an encrypted password), the day the password was last changed (day 1 is January 1, 1970), number of days the password must be left unchanged, password expiry day, warning number of days before password expiry, number of days after expiry before disabling the account, and the day the account was disabled (again, since 1970). The last field has no meaning yet.

All the passwords in the screenshot above are hashes of **hunter2**.

22.3. encryption with passwd

Passwords are stored in an encrypted format. This encryption is done by the **crypt** function. The easiest (and recommended) way to add a user with a password to the system is to add the user with the **useradd -m user** command, and then set the user's password with **passwd**.

```
[root@RHEL4 ~]# useradd -m xavier
[root@RHEL4 ~]# passwd xavier
Changing password for user xavier.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
[root@RHEL4 ~]#
```

22.4. encryption with openssl

Another way to create users with a password is to use the **-p** option of **useradd**, but that option requires an encrypted password. You can generate this encrypted password with the **openssl passwd** command.

The **openssl passwd** command will generate several distinct hashes for the same password, for this it uses a **salt**.

```
paul@rhel65:~$ openssl passwd hunter2
86jcUNlnGDFpY
paul@rhel65:~$ openssl passwd hunter2
Yj7mDO9OAnvq6
paul@rhel65:~$ openssl passwd hunter2
YqDcJeGoDbzKA
paul@rhel65:~$
```

This **salt** can be chosen and is visible as the first two characters of the hash.

```
paul@rhel65:~$ openssl passwd -salt 42 hunter2
42ZrbtP1Ze8G.
paul@rhel65:~$ openssl passwd -salt 42 hunter2
42ZrbtP1Ze8G.
paul@rhel65:~$ openssl passwd -salt 42 hunter2
42ZrbtP1Ze8G.
paul@rhel65:~$
```

This example shows how to create a user with password.

```
root@rhel65:~# useradd -m -p $(openssl passwd hunter2) mohamed
```

Note that this command puts the password in your command history!

22.5. encryption with crypt

A third option is to create your own C program using the crypt function, and compile this into a command.

```
paul@rhel65:~$ cat MyCrypt.c
#include <stdio.h>
#define_USE_XOPEN
#include <unistd.h>

int main(int argc, char** argv)
{
    if(argc==3)
    {
        printf("%s\n", crypt(argv[1],argv[2]));
    }
    else
    {
        printf("Usage: MyCrypt $password $salt\n" );
    }
    return 0;
}
```

This little program can be compiled with **gcc** like this.

```
paul@rhel65:~$ gcc MyCrypt.c -o MyCrypt -lcrypt
```

To use it, we need to give two parameters to MyCrypt. The first is the unencrypted password, the second is the salt. The salt is used to perturb the encryption algorithm in one of 4096 different ways. This variation prevents two users with the same password from having the same entry in **/etc/shadow**.

```
paul@rhel65:~$ ./MyCrypt hunter2 42
42ZrbtP1Ze8G.
paul@rhel65:~$ ./MyCrypt hunter2 33
33d6taYSiEUXI
```

Did you notice that the first two characters of the password are the **salt**?

The standard output of the crypt function is using the DES algorithm which is old and can be cracked in minutes. A better method is to use **md5** passwords which can be recognized by a salt starting with \$1\$.

```
paul@rhel65:~$ ./MyCrypt hunter2 '$1$42'
$1$42$716Y3xT5282XmZrtDOF9f0
paul@rhel65:~$ ./MyCrypt hunter2 '$6$42'
$6$42$OqFFAVnI3gTSYG0yI9TZWX9cpyQzwIop7HwpG1LLEsNBiMr4w6OvLX1KDa./UpwXfrFkli...
```

The **md5** salt can be up to eight characters long. The salt is displayed in **/etc/shadow** between the second and third \$, so never use the password as the salt!

```
paul@rhel65:~$ ./MyCrypt hunter2 '$1$hunter2'
$1$hunter2$YVxrxDmidq7Xf8Gdt6qM2.
```

22.6. /etc/login.defs

The **/etc/login.defs** file contains some default settings for user passwords like password aging and length settings. (You will also find the numerical limits of user ids and group ids and whether or not a home directory should be created by default).

```
root@rhel65:~# grep ^PASS /etc/login.defs
PASS_MAX_DAYS      99999
PASS_MIN_DAYS      0
PASS_MIN_LEN       5
PASS_WARN_AGE      7
```

Debian also has this file.

```
root@debian7:~# grep PASS /etc/login.defs
# PASS_MAX_DAYS      Maximum number of days a password may be used.
# PASS_MIN_DAYS      Minimum number of days allowed between password changes.
# PASS_WARN_AGE      Number of days warning given before a password expires.
PASS_MAX_DAYS      99999
PASS_MIN_DAYS      0
PASS_WARN_AGE      7
#PASS_CHANGE_TRIES
#PASS_ALWAYS_WARN
#PASS_MIN_LEN
#PASS_MAX_LEN
# NO_PASSWORD_CONSOLE
root@debian7:~#
```

22.7. chage

The **chage** command can be used to set an expiration date for a user account (-E), set a minimum (-m) and maximum (-M) password age, a password expiration date, and set the number of warning days before the password expiration date. Much of this functionality is also available from the **passwd** command. The **-l** option of **chage** will list these settings for a user.

```
root@rhel65:~# chage -l paul
Last password change                : Mar 27, 2014
Password expires                    : never
Password inactive                   : never
Account expires                     : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
root@rhel65:~#
```

22.8. disabling a password

Passwords in **/etc/shadow** cannot begin with an exclamation mark. When the second field in **/etc/passwd** starts with an exclamation mark, then the password can not be used.

Using this feature is often called **locking**, **disabling**, or **suspending** a user account. Besides **vi** (or **vim**) you can also accomplish this with **usermod**.

The first command in the next screenshot will show the hashed password of **laura** in **/etc/shadow**. The next command disables the password of **laura**, making it impossible for Laura to authenticate using this password.

```
root@debian7:~# grep laura /etc/shadow | cut -c1-70
laura:$6$JYj4JZqp$stwwWACp3OtE1R2aZuE87j.nbW.puDkNUYV7mCHfCVMa3CoDUJV
root@debian7:~# usermod -L laura
```

As you can see below, the password hash is simply preceded with an exclamation mark.

```
root@debian7:~# grep laura /etc/shadow | cut -c1-70
laura:!!$6$JYj4JZqp$stwwWACp3OtE1R2aZuE87j.nbW.puDkNUYV7mCHfCVMa3CoDUJV
root@debian7:~#
```

The root user (and users with **sudo** rights on **su**) still will be able to **su** into the **laura** account (because the password is not needed here). Also note that **laura** will still be able to login if she has set up passwordless ssh!

```
root@debian7:~# su - laura
laura@debian7:~$
```

You can unlock the account again with **usermod -U**.

```
root@debian7:~# usermod -U laura
root@debian7:~# grep laura /etc/shadow | cut -c1-70
laura:$6$JYj4JZqp$stwwWACp3OtE1R2aZuE87j.nbW.puDkNUYV7mCHfCVMa3CoDUJV
```

Watch out for tiny differences in the command line options of **passwd**, **usermod**, and **useradd** on different Linux distributions. Verify the local files when using features like "**disabling**, **suspending**, or **locking**" on user accounts and their passwords.

22.9. editing local files

If you still want to manually edit the **/etc/passwd** or **/etc/shadow**, after knowing these commands for password management, then use **vim** instead of **vi(m)** directly. The **vim** tool will do proper locking of the file.

```
[root@RHEL5 ~]# vim /etc/passwd
vim: the password file is busy (/etc/ptmp present)
```

22.10. solution: user passwords

1. Set the password for **serena** to **hunter2**.

```
root@debian7:~# passwd serena
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

2. Also set a password for **venus** and then lock the **venus** user account with **usermod**. Verify the locking in **/etc/shadow** before and after you lock it.

```
root@debian7:~# passwd venus
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@debian7:~# grep venus /etc/shadow | cut -c1-70
venus:$6$gswzXICW$uSnKFV1kFKZmTPaMVS4AvNA/KO27OxN0v5LHdV9ed0gTyXrjUeM/
root@debian7:~# usermod -L venus
root@debian7:~# grep venus /etc/shadow | cut -c1-70
venus:!!$6$gswzXICW$uSnKFV1kFKZmTPaMVS4AvNA/KO27OxN0v5LHdV9ed0gTyXrjUeM
```

Note that **usermod -L** precedes the password hash with an exclamation mark (!).

3. Use **passwd -d** to disable the **serena** password. Verify the **serena** line in **/etc/shadow** before and after disabling.

```
root@debian7:~# grep serena /etc/shadow | cut -c1-70
serena:$6$Es$omrPE$F2Ypu8kpLrfKdW0v/UIwA5jrYyBD2nwZ/dt.i/IypRgiPZSdB/B
root@debian7:~# passwd -d serena
passwd: password expiry information changed.
root@debian7:~# grep serena /etc/shadow
serena::16358:0:99999:7:::
root@debian7:~#
```

4. What is the difference between locking a user account and disabling a user account's password like we just did with **usermod -L** and **passwd -d**?

Locking will prevent the user from logging on to the system with his password by putting a **!** in front of the password in **/etc/shadow**.

Disabling with **passwd** will erase the password from **/etc/shadow**.

5. Try changing the password of **serena** to **serena** as **serena**.

```
log on as serena, then execute: passwd serena... it should fail!
```

6. Make sure **serena** has to change her password in 10 days.

```
chage -M 10 serena
```

7. Make sure every new user needs to change their password every 10 days.

```
vi /etc/login.defs (and change PASS_MAX_DAYS to 10)
```

8. Take a backup as root of **/etc/shadow**. Use **vi** to copy an encrypted **hunter2** hash from **venus** to **serena**. Can **serena** now log on with **hunter2** as a password ?

Yes.

9. Why use **vipw** instead of **vi** ? What could be the problem when using **vi** or **vim** ?

vipw will give a warning when someone else is already using that file (with **vipw**).

10. Use **chsh** to list all shells (only works on RHEL/CentOS/Fedora), and compare to **cat /etc/shells**.

```
chsh -l  
cat /etc/shells
```

11. Which **useradd** option allows you to name a home directory ?

-d

12. How can you see whether the password of user **serena** is locked or unlocked ? Give a solution with **grep** and a solution with **passwd**.

```
grep serena /etc/shadow
```

```
passwd -S serena
```

Chapter 23. user profiles

Logged on users have a number of preset (and customized) aliases, variables, and functions, but where do they come from ? The **shell** uses a number of startup files that are executed (or rather **sourced**) whenever the shell is invoked. What follows is an overview of startup scripts.

23.1. system profile

Both the **bash** and the **ksh** shell will verify the existence of **/etc/profile** and **source** it if it exists.

When reading this script, you will notice (both on Debian and on Red Hat Enterprise Linux) that it builds the **PATH** environment variable (among others). The script might also change the **PS1** variable, set the **HOSTNAME** and execute even more scripts like **/etc/inputrc**

This screenshot uses **grep** to show **PATH** manipulation in **/etc/profile** on Debian.

```
root@debian7:~# grep PATH /etc/profile
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
PATH="/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games"
export PATH
root@debian7:~#
```

This screenshot uses **grep** to show **PATH** manipulation in **/etc/profile** on RHEL7/CentOS7.

```
[root@centos7 ~]# grep PATH /etc/profile
case ":{PATH}:" in
    PATH=$PATH:$1
    PATH=$1:$PATH
export PATH USER LOGNAME MAIL HOSTNAME HISTSIZE HISTCONTROL
[root@centos7 ~]#
```

The **root user** can use this script to set aliases, functions, and variables for every user on the system.

23.2. ~/.bash_profile

When this file exists in the home directory, then **bash** will source it. On Debian Linux 5/6/7 this file does not exist by default.

RHEL7/CentOS7 uses a small **~/.bash_profile** where it checks for the existence of **~/.bashrc** and then sources it. It also adds **\$HOME/bin** to the **\$PATH** variable.

```
[root@rhel7 ~]# cat /home/paul/.bash_profile
# ~/.bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/.local/bin:$HOME/bin

export PATH
[root@rhel7 ~]#
```


23.3. ~/.bash_login

When **.bash_profile** does not exist, then **bash** will check for **~/.bash_login** and source it.

Neither Debian nor Red Hat have this file by default.

23.4. ~/.profile

When neither **~/.bash_profile** and **~/.bash_login** exist, then **bash** will verify the existence of **~/.profile** and execute it. This file does not exist by default on Red Hat.

On Debian this script can execute **~/.bashrc** and will add **\$HOME/bin** to the **\$PATH** variable.

```
root@debian7:~# tail -11 /home/paul/.profile
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
        . "$HOME/.bashrc"
    fi
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi
```

RHEL/CentOS does not have this file by default.

23.5. ~/.bashrc

The **~/.bashrc** script is often sourced by other scripts. Let us take a look at what it does by default.

Red Hat uses a very simple **~/.bashrc**, checking for **/etc/bashrc** and sourcing it. It also leaves room for custom aliases and functions.

```
[root@rhel7 ~]# cat /home/paul/.bashrc
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi

# Uncomment the following line if you don't like systemctl's auto-paging feature:
# export SYSTEMD_PAGER=

# User specific aliases and functions
```

On Debian this script is quite a bit longer and configures **\$PS1**, some history variables and a number of active and inactive aliases.

```
root@debian7:~# wc -l /home/paul/.bashrc
110 /home/paul/.bashrc
```

23.6. ~/.bash_logout

When exiting **bash**, it can execute **~/.bash_logout**.

Debian use this opportunity to clear the console screen.

```
serena@deb503:~$ cat ~/.bash_logout
# ~/.bash_logout: executed by bash(1) when login shell exits.

# when leaving the console clear the screen to increase privacy

if [ "$SHLVL" = 1 ]; then
    [ -x /usr/bin/clear_console ] && /usr/bin/clear_console -q
fi
```

Red Hat Enterprise Linux 5 will simple call the **/usr/bin/clear** command in this script.

```
[serena@rhel53 ~]$ cat ~/.bash_logout
# ~/.bash_logout

/usr/bin/clear
```

Red Hat Enterprise Linux 6 and 7 create this file, but leave it empty (except for a comment).

```
paul@rhel65:~$ cat ~/.bash_logout
# ~/.bash_logout
```

23.7. Debian overview

Below is a table overview of when Debian is running any of these bash startup scripts.

Table 30.1. Debian User Environment

script	su	su -	ssh	gdm
~/bashrc	no	yes	yes	yes
~/profile	no	yes	yes	yes
/etc/profile	no	yes	yes	yes
/etc/bash.bashrc	yes	no	no	yes

23.8. RHEL5 overview

Below is a table overview of when Red Hat Enterprise Linux 5 is running any of these bash startup scripts.

Table 30.2. Red Hat User Environment

script	su	su -	ssh	gdm
~/bashrc	yes	yes	yes	yes
~/bash_profile	no	yes	yes	yes
/etc/profile	no	yes	yes	yes
/etc/bashrc	yes	yes	yes	yes

23.9. solution: user profiles

1. Make a list of all the profile files on your system.

```
ls -a ~ ; ls -l /etc/pro* /etc/bash*
```

2. Read the contents of each of these, often they **source** extra scripts.
3. Put a unique variable, alias and function in each of those files.
4. Try several different ways to obtain a shell (su, su -, ssh, tmux, gnome-terminal, Ctrl- alt-F1, ...) and verify which of your custom variables, aliases and function are present in your environment.
5. Do you also know the order in which they are executed?

```
same name aliases, functions and variables will overwrite each other
```

6. When an application depends on a setting in \$HOME/.profile, does it matter whether \$HOME/.bash_profile exists or not ?

```
Yes it does matter. (man bash /INVOCATION)
```

Chapter 24. groups

Users can be listed in **groups**. Groups allow you to set permissions on the group level instead of having to set permissions for every individual user.

Every Unix or Linux distribution will have a graphical tool to manage groups. Novice users are advised to use this graphical tool. More experienced users can use command line tools to manage users, but be careful: Some distributions do not allow the mixed use of GUI and CLI tools to manage groups (YaST in Novell Suse). Senior administrators can edit the relevant files directly with **vi** or **vigr**.

24.1. groupadd

Groups can be created with the **groupadd** command. The example below shows the creation of five (empty) groups.

```
root@laika:~# groupadd tennis
root@laika:~# groupadd football
root@laika:~# groupadd snooker
root@laika:~# groupadd formula1
root@laika:~# groupadd salsa
```

24.2. group file

Users can be a member of several groups. Group membership is defined by the **/etc/group** file.

```
root@laika:~# tail -5 /etc/group
tennis:x:1006:
football:x:1007:
snooker:x:1008:
formula1:x:1009:
salsa:x:1010:
root@laika:~#
```

The first field is the group's name. The second field is the group's (encrypted) password (can be empty). The third field is the group identification or **GID**. The fourth field is the list of members, these groups have no members.

24.3. groups

A user can type the **groups** command to see a list of groups where the user belongs to.

```
[harry@RHEL4b ~]$ groups
harry sports
[harry@RHEL4b ~]$
```

24.4. usermod

Group membership can be modified with the `useradd` or **usermod** command.

```
root@laika:~# usermod -a -G tennis inge
root@laika:~# usermod -a -G tennis katrien
root@laika:~# usermod -a -G salsa katrien
root@laika:~# usermod -a -G snooker sandra
root@laika:~# usermod -a -G formula1 annelies
root@laika:~# tail -5 /etc/group
tennis:x:1006:inge,katrien
football:x:1007:
snooker:x:1008:sandra
formula1:x:1009:annelies
salsa:x:1010:katrien
root@laika:~#
```

Be careful when using **usermod** to add users to groups. By default, the **usermod** command will **remove** the user from every group of which he is a member if the group is not listed in the command! Using the **-a** (append) switch prevents this behaviour.

24.5. groupmod

You can change the group name with the **groupmod** command.

```
root@laika:~# groupmod -n darts snooker
root@laika:~# tail -5 /etc/group
tennis:x:1006:inge,katrien
football:x:1007:
formula1:x:1009:annelies
salsa:x:1010:katrien
darts:x:1008:sandra
```

24.6. groupdel

You can permanently remove a group with the **groupdel** command.

```
root@laika:~# groupdel tennis
root@laika:~#
```


24.7. gpasswd

You can delegate control of group membership to another user with the **gpasswd** command. In the example below we delegate permissions to add and remove group members to serena for the sports group. Then we **su** to serena and add harry to the sports group.

```
[root@RHEL4b ~]# gpasswd -A serena sports
[root@RHEL4b ~]# su - serena
[serena@RHEL4b ~]$ id harry
uid=516(harry) gid=520(harry) groups=520(harry)
[serena@RHEL4b ~]$ gpasswd -a harry sports
Adding user harry to group sports
[serena@RHEL4b ~]$ id harry
uid=516(harry) gid=520(harry) groups=520(harry), 522(sports)
[serena@RHEL4b ~]$ tail -1 /etc/group
sports:x:522:serena,venus,harry
[serena@RHEL4b ~]$
```

Group administrators do not have to be a member of the group. They can remove themselves from a group, but this does not influence their ability to add or remove members.

```
[serena@RHEL4b ~]$ gpasswd -d serena sports
Removing user serena from group sports
[serena@RHEL4b ~]$ exit
```

Information about group administrators is kept in the **/etc/gshadow** file.

```
[root@RHEL4b ~]# tail -1 /etc/gshadow
sports:!:serena:venus,harry
[root@RHEL4b ~]#
```

To remove all group administrators from a group, use the **gpasswd** command to set an empty administrators list.

```
[root@RHEL4b ~]# gpasswd -A "" sports
```

24.8. newgrp

You can start a **child shell** with a new temporary **primary group** using the **newgrp** command.

```
root@rhel65:~# mkdir prigroup
root@rhel65:~# cd prigroup/
root@rhel65:~/prigroup# touch standard.txt
root@rhel65:~/prigroup# ls -l
total 0
-rw-r--r--. 1 root root 0 Apr 13 17:49 standard.txt
root@rhel65:~/prigroup# echo $SHLVL
1
root@rhel65:~/prigroup# newgrp tennis
root@rhel65:~/prigroup# echo $SHLVL
2
root@rhel65:~/prigroup# touch newgrp.txt
root@rhel65:~/prigroup# ls -l
total 0
-rw-r--r--. 1 root tennis 0 Apr 13 17:49 newgrp.txt
-rw-r--r--. 1 root root 0 Apr 13 17:49 standard.txt
root@rhel65:~/prigroup# exit
exit
root@rhel65:~/prigroup#
```

24.9. vigr

Similar to vipw, the **vigr** command can be used to manually edit the **/etc/group** file, since it will do proper locking of the file. Only experienced senior administrators should use **vi** or

24.10. solution: groups

1. Create the groups tennis, football and sports.

```
groupadd tennis ; groupadd football ; groupadd sports
```

2. In one command, make venus a member of tennis and sports.

```
usermod -a -G tennis,sports venus
```

3. Rename the football group to foot.

```
groupmod -n foot football
```

4. Use vi to add serena to the tennis group.

```
vi /etc/group
```

5. Use the id command to verify that serena is a member of tennis.

```
id (and after logoff logon serena should be member)
```

6. Make someone responsible for managing group membership of foot and sports. Test that it works.

```
gpasswd -A (to make manager)
```

```
gpasswd -a (to add member)
```

Chapter 25. standard file permissions

This chapter contains details about basic file security through **file ownership** and **file permissions**.

25.1. file ownership

25.1.1. user owner and group owner

The **users** and **groups** of a system can be locally managed in `/etc/passwd` and `/etc/group`, or they can be in a NIS, LDAP, or Samba domain. These users and groups can **own** files. Actually, every file has a **user owner** and a **group owner**, as can be seen in the following screenshot.

```
paul@rhel65:~/owners$ ls -lh
total 636K
-rw-r--r--. 1 paul snooker 1.1K Apr  8 18:47 data.odt
-rw-r--r--. 1 paul paul    626K Apr  8 18:46 file1
-rw-r--r--. 1 root tennis  185 Apr  8 18:46 file2
-rw-rw-r--. 1 root root    0 Apr  8 18:47 stuff.txt
paul@rhel65:~/owners$
```

User paul owns three files; file1 has paul as **user owner** and has the group paul as **group owner**, data.odt is **group owned** by the group snooker, file2 by the group tennis.

The last file is called stuff.txt and is owned by the root user and the root group.

25.1.2. listing user accounts

You can use the following command to list all local user accounts.

```
paul@debian7~$ cut -d: -f1 /etc/passwd | column
root          ntp           sam           bert          naomi
daemon        mysql         tom           rino          matthias2
bin           paul          wouter       antonio       bram
sys           maarten      robrecht     simon         fabrice
sync          kevin        bilal        sven          chimene
games         yuri         dimitri      wouter2       messagebus
man           william      ahmed        tarik         roger
lp            yves         dylan        jan           frank
mail          kris         robin        ian           toon
news          hamid        matthias     ivan          rinus
uucp          vladimir    ben          azeddine      eddy
proxy         abiy         mike         eric          bram2
www-data      david        kevin2       kamel         keith
backup        chahid       kenzo        ischa         jesse
list          stef         aaron        bart          frederick
irc           joeri        lorenzo      omer          hans
gnats         glenn        jens         kurt          dries
nobody        yannick      ruben        steve         steve2
libuuid       christof     jelle        constantin    tomas
Debian-exim   george       stefaan      sam2          johan
statd         joost        marc         bjorn         tom2
sshd          arno         thomas       ronald
```

25.1.3. chgrp

You can change the group owner of a file using the **chgrp** command.

```
root@rhel65:/home/paul/owners# ls -l file2
-rw-r--r--. 1 root tennis 185 Apr  8 18:46 file2
root@rhel65:/home/paul/owners# chgrp snooker file2
root@rhel65:/home/paul/owners# ls -l file2
-rw-r--r--. 1 root snooker 185 Apr  8 18:46 file2
root@rhel65:/home/paul/owners#
```

25.1.4. chown

The user owner of a file can be changed with **chown** command.

```
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root paul 0 2008-08-06 14:11 FileForPaul
root@laika:/home/paul# chown paul FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 paul paul 0 2008-08-06 14:11 FileForPaul
```

You can also use **chown** to change both the user owner and the group owner.

```
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 paul paul 0 2008-08-06 14:11 FileForPaul
root@laika:/home/paul# chown root:project42 FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root project42 0 2008-08-06 14:11 FileForPaul
```

25.2. list of special files

When you use **ls -l**, for each file you can see ten characters before the user and group owner. The first character tells us the type of file. Regular files get a **-**, directories get a **d**, symbolic links are shown with an **l**, pipes get a **p**, character devices a **c**, block devices a **b**, and sockets an **s**.

Table 32.1. Unix special files

first character	file type
-	normal file
d	directory
l	symbolic link
p	named pipe
b	block device
c	character device
s	socket

Below a screenshot of a character device (the console) and a block device (the hard disk).

```
paul@debian6lt~$ ls -ld /dev/console /dev/sda
crw----- 1 root root  5, 1 Mar 15 12:45 /dev/console
brw-rw---- 1 root disk  8, 0 Mar 15 12:45 /dev/sda
```

And here you can see a directory, a regular file and a symbolic link.

```
paul@debian6lt~$ ls -ld /etc /etc/hosts /etc/motd
drwxr-xr-x 128 root root 12288 Mar 15 18:34 /etc
-rw-r--r--  1 root root   372 Dec 10 17:36 /etc/hosts
lrwxrwxrwx  1 root root    13 Dec  5 10:36 /etc/motd -> /var/run/motd
```


25.3. permissions

25.3.1. rwx

The nine characters following the file type denote the permissions in three triplets. A permission can be **r** for read access, **w** for write access, and **x** for execute. You need the **r** permission to list (**ls**) the contents of a directory. You need the **x** permission to enter (**cd**) a directory. You need the **w** permission to create files in or remove files from a directory.

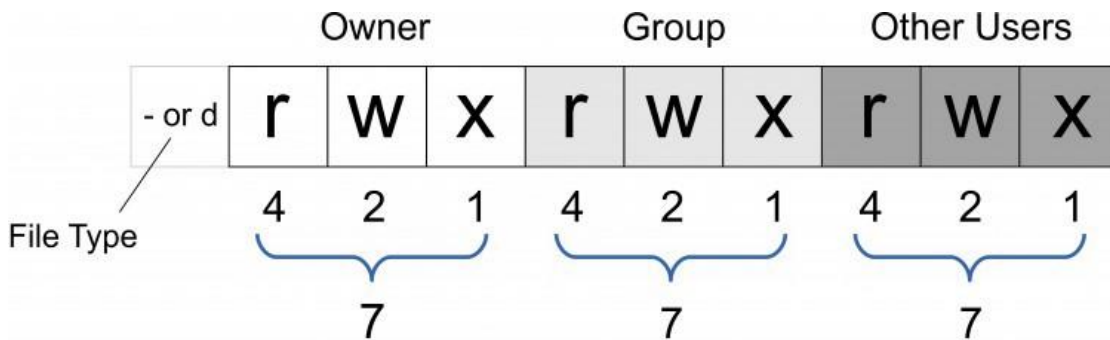


Table 32.2. standard Unix file permissions

permission	on a file	on a directory
r (read)	read file contents (cat)	read directory contents (ls)
w (write)	change file contents (vi)	create files in (touch)
x (execute)	execute the file	enter the directory (cd)

25.3.2. three sets of rwx

We already know that the output of **ls -l** starts with ten characters for each file. This screenshot shows a regular file (because the first character is a -).

```
paul@RHELv4u4:~/test$ ls -l proc42.bash
-rwxr-xr-- 1 paul proj 984 Feb  6 12:01 proc42.bash
```

Below is a table describing the function of all ten characters.

Table 32.3. Unix file permissions position

position	characters	function
1	-	this is a regular file
2-4	rwx	permissions for the user owner
5-7	r-x	permissions for the group owner
8-10	r--	permissions for others

When you are the **user owner** of a file, then the **user owner permissions** apply to you. The rest of the permissions have no influence on your access to the file.

When you belong to the **group** that is the **group owner** of a file, then the **group owner permissions** apply to you. The rest of the permissions have no influence on your access to

the file.

When you are not the **user owner** of a file and you do not belong to the **group owner**, then the **others permissions** apply to you. The rest of the permissions have no influence on your access to the file.

25.3.3. permission examples

Some example combinations on files and directories are seen in this screenshot. The name of the file explains the permissions.

```
paul@laika:~/perms$ ls -lh
total 12K
drwxr-xr-x 2 paul paul 4.0K 2007-02-07 22:26 AllEnter_UserCreateDelete
-rwxrwxrwx 1 paul paul  0 2007-02-07 22:21 EveryoneFullControl.txt
-r--r----- 1 paul paul  0 2007-02-07 22:21 OnlyOwnersRead.txt
-rwxrwx--- 1 paul paul  0 2007-02-07 22:21 OwnersAll_RestNothing.txt
dr-xr-x--- 2 paul paul 4.0K 2007-02-07 22:25 UserAndGroupEnter
dr-x----- 2 paul paul 4.0K 2007-02-07 22:25 OnlyUserEnter
paul@laika:~/perms$
```

To summarise, the first **rwX** triplet represents the permissions for the **user owner**. The second triplet corresponds to the **group owner**; it specifies permissions for all members of that group. The third triplet defines permissions for all **other** users that are not the user owner and are not a member of the group owner.

25.3.4. setting permissions (chmod)

Permissions can be changed with **chmod**. The first example gives the user owner execute permissions.

```
paul@laika:~/perms$ ls -l permissions.txt
-rw-r--r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod u+x permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxr--r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

This example removes the group owners read permission.

```
paul@laika:~/perms$ chmod g-r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx---r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

This example removes the others read permission.

```
paul@laika:~/perms$ chmod o-r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx----- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

This example gives all of them the write permission.

```
paul@laika:~/perms$ chmod a+w permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx-w--w- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

You don't even have to type the a.

```
paul@laika:~/perms$ chmod +x permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx-wx-wx 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

You can also set explicit permissions.

```
paul@laika:~/perms$ chmod u=rw permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw--wx-wx 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Feel free to make any kind of combination.

```
paul@laika:~/perms$ chmod u=rw,g=rw,o=r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw-rw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Even fishy combinations are accepted by chmod.

```
paul@laika:~/perms$ chmod u=rwx,ug+rw,o=r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxrw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

25.3.5. setting octal permissions

Most Unix administrators will use the **old school** octal system to talk about and set permissions. Look at the triplet bitwise, equating r to 4, w to 2, and x to 1.

Table 32.4. Octal permissions

binary	octal	permission
000	0	---
001	1	--x
010	2	-w-
011	3	-wx
100	4	r--
101	5	r-x
110	6	rw-
111	7	rwX

This makes **777** equal to **rw-rw-rw-** and by the same logic, **654** mean **rw-r-xr--** . The **chmod** command will accept these numbers.

```
paul@laika:~/perms$ chmod 777 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxrwxrwx 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod 664 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw-rw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod 750 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxr-x--- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

25.3.6. umask

When creating a file or directory, a set of default permissions are applied. These default permissions are determined by the **umask**. The **umask** specifies permissions that you do not want set on by default. You can display the **umask** with the **umask** command.

```
[Harry@RHEL4b ~]$ umask
0002
[Harry@RHEL4b ~]$ touch test
[Harry@RHEL4b ~]$ ls -l test
-rw-rw-r-- 1 Harry Harry 0 Jul 24 06:03 test
[Harry@RHEL4b ~]$
```

As you can also see, the file is also not executable by default. This is a general security feature among Unixes; newly created files are never executable by default. You have to explicitly do a **chmod +x** to make a file executable. This also means that the 1 bit in the **umask** has no meaning--a **umask** of 0022 is the same as 0033.

25.3.7. mkdir -m

When creating directories with **mkdir** you can use the **-m** option to set the **mode**. This screenshot explains.

```
paul@debian5~$ mkdir -m 700 MyDir
paul@debian5~$ mkdir -m 777 Public
paul@debian5~$ ls -dl MyDir/ Public/
drwx----- 2 paul paul 4096 2011-10-16 19:16 MyDir/
drwxrwxrwx 2 paul paul 4096 2011-10-16 19:16 Public/
```

25.3.8. cp -p

To preserve permissions and time stamps from source files, use **cp -p**.

```
paul@laika:~/perms$ cp file* cp
paul@laika:~/perms$ cp -p file* cpp
paul@laika:~/perms$ ll *
-rwx----- 1 paul paul    0 2008-08-25 13:26 file33
-rwxr-x--- 1 paul paul    0 2008-08-25 13:26 file42

cp:
total 0
-rwx----- 1 paul paul 0 2008-08-25 13:34 file33
-rwxr-x--- 1 paul paul 0 2008-08-25 13:34 file42

cpp:
total 0
-rwx----- 1 paul paul 0 2008-08-25 13:26 file33
-rwxr-x--- 1 paul paul 0 2008-08-25 13:26 file42
```

25.4. solution: standard file permissions

1. As normal user, create a directory ~/permissions. Create a file owned by yourself in there.

```
mkdir ~/permissions ; touch ~/permissions/myfile.txt
```

2. Copy a file owned by root from /etc/ to your permissions dir, who owns this file now ?

```
cp /etc/hosts ~/permissions/
```

The copy is owned by you.

3. As root, create a file in the users ~/permissions directory.

```
(become root)# touch /home/username/permissions/rootfile
```

4. As normal user, look at who owns this file created by root.

```
ls -l ~/permissions
```

The file created by root is owned by root.

5. Change the ownership of all files in ~/permissions to yourself.

```
chown user ~/permissions/*
```

You cannot become owner of the file that belongs to root.

6. Make sure you have all rights to these files, and others can only read.

```
chmod 644 (on files)
```

```
chmod 755 (on directories)
```

7. With chmod, is 770 the same as rwxrwx--- ?

yes

8. With chmod, is 664 the same as r-xr-xr-- ?

No

9. With chmod, is 400 the same as r----- ?

yes

10. With chmod, is 734 the same as rwxr-xr-- ?

no

11a. Display the umask in octal and in symbolic form.

```
umask ; umask -S
```

11b. Set the umask to 077, but use the symbolic format to set it. Verify that this works.

```
umask -S u=rwx,go=
```

12. Create a file as root, give only read to others. Can a normal user read this file ? Test writing to this file with vi.

```
(become root)
# echo hello > /home/username/root.txt
# chmod 744 /home/username/root.txt
(become user)
vi ~/root.txt
```

13a. Create a file as normal user, give only read to others. Can another normal user read this file ? Test writing to this file with vi.

```
echo hello > file ; chmod 744 file
```

Yes, others can read this file

13b. Can root read this file ? Can root write to this file with vi ?

Yes, root can read and write to this file. Permissions do not apply to root.

14. Create a directory that belongs to a group, where every member of that group can read and write to files, and create files. Make sure that people can only delete their own files.

```
mkdir /home/project42 ; groupadd project42
chgrp project42 /home/project42 ; chmod 775 /home/project42
```

You can not yet do the last part of this exercise...

Chapter 26. advanced file permissions

26.1. sticky bit on directory

You can set the **sticky bit** on a directory to prevent users from removing files that they do not own as a user owner. The sticky bit is displayed at the same location as the x permission for others. The sticky bit is represented by a **t** (meaning x is also there) or a **T** (when there is no x for others).

```
root@RHELv4u4:~# mkdir /project55
root@RHELv4u4:~# ls -ld /project55
drwxr-xr-x 2 root root 4096 Feb  7 17:38 /project55
root@RHELv4u4:~# chmod +t /project55/
root@RHELv4u4:~# ls -ld /project55
drwxr-xr-t 2 root root 4096 Feb  7 17:38 /project55
root@RHELv4u4:~#
```

The **sticky bit** can also be set with octal permissions, it is binary 1 in the first of four triplets.

```
root@RHELv4u4:~# chmod 1775 /project55/
root@RHELv4u4:~# ls -ld /project55
drwxrwxr-t 2 root root 4096 Feb  7 17:38 /project55
root@RHELv4u4:~#
```

You will typically find the **sticky bit** on the **/tmp** directory.

```
root@barry:~# ls -ld /tmp
drwxrwxrwt 6 root root 4096 2009-06-04 19:02 /tmp
```

26.2. setgid bit on directory

setgid can be used on directories to make sure that all files inside the directory are owned by the group owner of the directory. The **setgid** bit is displayed at the same location as the x permission for group owner. The **setgid** bit is represented by an **s** (meaning x is also there) or a **S** (when there is no x for the group owner). As this example shows, even though **root** does not belong to the group **proj55**, the files created by root in **/project55** will belong to **proj55** since the **setgid** is set.

```
root@RHELv4u4:~# groupadd proj55
root@RHELv4u4:~# chown root:proj55 /project55/
root@RHELv4u4:~# chmod 2775 /project55/
root@RHELv4u4:~# touch /project55/fromroot.txt
root@RHELv4u4:~# ls -ld /project55/
drwxrwsr-x 2 root proj55 4096 Feb  7 17:45 /project55/
root@RHELv4u4:~# ls -l /project55/
total 4
-rw-r--r-- 1 root proj55 0 Feb  7 17:45 fromroot.txt
root@RHELv4u4:~#
```

You can use the **find** command to find all **setgid** directories.

```
paul@laika:~$ find / -type d -perm -2000 2> /dev/null
/var/log/mysql
/var/log/news
/var/local
...
```

26.3. setgid and setuid on regular files

These two permissions cause an executable file to be executed with the permissions of the **file owner** instead of the **executing owner**. This means that if any user executes a program that belongs to the **root user**, and the **setuid** bit is set on that program, then the program runs as **root**. This can be dangerous, but sometimes this is good for security.

Take the example of passwords; they are stored in **/etc/shadow** which is only readable by **root**. (The **root** user never needs permissions anyway.)

```
root@RHELv4u4:~# ls -l /etc/shadow
-r----- 1 root root 1260 Jan 21 07:49 /etc/shadow
```

Changing your password requires an update of this file, so how can normal non-root users do this? Let's take a look at the permissions on the **/usr/bin/passwd**.

```
root@RHELv4u4:~# ls -l /usr/bin/passwd
-r-s--x--x 1 root root 21200 Jun 17 2005 /usr/bin/passwd
```

When running the **passwd** program, you are executing it with **root** credentials.

You can use the **find** command to find all **setuid** programs.

```
paul@laika:~$ find /usr/bin -type f -perm -04000
/usr/bin/arping
/usr/bin/kgrantpty
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/fping6
/usr/bin/passwd
/usr/bin/gpasswd
...
```

In most cases, setting the **setuid** bit on executables is sufficient. Setting the **setgid** bit will result in these programs to run with the credentials of their group owner.

26.4. setuid on sudo

The **sudo** binary has the **setuid** bit set, so any user can run it with the effective userid of root.

```
paul@rhel65:~$ ls -l $(which sudo)
---s--x--x. 1 root root 123832 Oct 7 2013 /usr/bin/sudo
paul@rhel65:~$
```

26.5. solution: sticky, setuid and setgid bits

1a. Set up a directory, owned by the group sports.

```
groupadd sports
```

```
mkdir /home/sports
```

```
chown root:sports /home/sports
```

1b. Members of the sports group should be able to create files in this directory.

```
chmod 770 /home/sports
```

1c. All files created in this directory should be group-owned by the sports group.

```
chmod 2770 /home/sports
```

1d. Users should be able to delete only their own user-owned files.

```
chmod +t /home/sports
```

1e. Test that this works!

Log in with different users (group members and others and root), create files and watch the permissions. Try changing and deleting files...

2. Verify the permissions on **/usr/bin/passwd**. Remove the **setuid**, then try changing your password as a normal user. Reset the permissions back and try again.

```
root@deb503:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
root@deb503:~# chmod 755 /usr/bin/passwd
root@deb503:~# ls -l /usr/bin/passwd
-rwxr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
```

A normal user cannot change password now.

```
root@deb503:~# chmod 4755 /usr/bin/passwd
root@deb503:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
```

3. If time permits (or if you are waiting for other students to finish this practice), read about file attributes in the man page of `chattr` and `lsattr`. Try setting the `i` attribute on a file and test that it works.

```
paul@laika:~$ sudo su -
[sudo] password for paul:
root@laika:~# mkdir attr
root@laika:~# cd attr/
root@laika:~/attr# touch file42
root@laika:~/attr# lsattr
----- ./file42
root@laika:~/attr# chattr +i file42
```

```
root@laika:~/attr# lsattr
----i----- ./file42
root@laika:~/attr# rm -rf file42
rm: cannot remove `file42': Operation not permitted
root@laika:~/attr# chattr -i file42
root@laika:~/attr# rm -rf file42
root@laika:~/attr#
```

Chapter 27. access control lists

Standard Unix permissions might not be enough for some organisations. This chapter introduces **access control lists** or **acl's** to further protect files and directories.

27.1. acl in /etc/fstab

File systems that support **access control lists**, or **acls**, have to be mounted with the **acl** option listed in **/etc/fstab**. In the example below, you can see that the root file system has **acl** support, whereas **/home/data** does not.

```
root@laika:~# tail -4 /etc/fstab
/dev/sda1      /              ext3          acl,relatime  0 1
/dev/sdb2      /home/data     auto          noacl,defaults 0 0
pasha:/home/r  /home/pasha   nfs          defaults      0 0
wolf:/srv/data /home/wolf     nfs          defaults      0 0
```

27.2. getfacl

Reading **acls** can be done with **/usr/bin/getfacl**. This screenshot shows how to read the **acl** of **file33** with **getfacl**.

```
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
group::r--
mask::rwx
other::r--
```

27.3. setfacl

Writing or changing **acls** can be done with **/usr/bin/setfacl**. These screenshots show how to change the **acl** of **file33** with **setfacl**.

First we add **user sandra** with octal permission **7** to the **acl**.

```
paul@laika:~/test$ setfacl -m u:sandra:7 file33
```

Then we add the **group tennis** with octal permission **6** to the **acl** of the same file.

```
paul@laika:~/test$ setfacl -m g:tennis:6 file33
```

The result is visible with **getfacl**.

```
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
user:sandra:rwx
group::r--
group:tennis:rw-
mask::rwx
other::r--
```


27.4. remove an acl entry

The **-x** option of the **setfacl** command will remove an **acl** entry from the targeted file.

```
paul@laika:~/test$ setfacl -m u:sandra:7 file33
paul@laika:~/test$ getfacl file33 | grep sandra
user:sandra:rw-
paul@laika:~/test$ setfacl -x sandra file33
paul@laika:~/test$ getfacl file33 | grep sandra
```

Note that omitting the **u** or **g** when defining the **acl** for an account will default it to a user account.

27.5. remove the complete acl

The **-b** option of the **setfacl** command will remove the **acl** from the targeted file.

```
paul@laika:~/test$ setfacl -b file33
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
group::r--
other::r--
```

27.6. the acl mask

The **acl mask** defines the maximum effective permissions for any entry in the **acl**. This **mask** is calculated every time you execute the **setfacl** or **chmod** commands.

You can prevent the calculation by using the **--no-mask** switch.

```
paul@laika:~/test$ setfacl --no-mask -m u:sandra:7 file33
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
user:sandra:rw-   #effective:rw-
group::r--
mask::rw-
other::r--
```

27.7. eiciel

Desktop users might want to use **eiciel** to manage **acls** with a graphical tool.



You will need to install **eiciel** and **nautilus-actions** to have an extra tab in **nautilus** to manage **acls**.

```
paul@laika:~$ sudo aptitude install eiciel nautilus-actions
```

Chapter 28. file links

An average computer using Linux has a file system with many **hard links** and **symbolic links**.

To understand links in a file system, you first have to understand what an **inode** is.

28.1. inodes

28.1.1. inode contents

An **inode** is a data structure that contains metadata about a file. When the file system stores a new file on the hard disk, it stores not only the contents (data) of the file, but also extra properties like the name of the file, the creation date, its permissions, the owner of the file, and more. All this information (except the name of the file and the contents of the file) is stored in the **inode** of the file.

The **ls -l** command will display some of the inode contents, as seen in this screenshot.

```
root@rhel53 ~# ls -ld /home/project42/
drwxr-xr-x 4 root pro42 4.0K Mar 27 14:29 /home/project42/
```

28.1.2. inode table

The **inode table** contains all of the **inodes** and is created when you create the file system (with **mkfs**). You can use the **df -i** command to see how many **inodes** are used and free on mounted file systems.

```
root@rhel53 ~# df -i
Filesystem              Inodes    IUsed    IFree  IUse% Mounted on
/dev/mapper/VolGroup00-LogVol100
                        4947968   115326  4832642    3% /
/dev/hda1                26104      45    26059    1% /boot
tmpfs                   64417        1    64416    1% /dev/shm
/dev/sda1               262144    2207   259937    1% /home/project42
/dev/sdb1               74400    5519    68881    8% /home/project33
/dev/sdb5                 0          0         0    - /home/sales
/dev/sdb6              100744     11   100733    1% /home/research
```

In the **df -i** screenshot above you can see the **inode** usage for several mounted **file systems**. You don't see numbers for **/dev/sdb5** because it is a **fat** file system.

28.1.3. inode number

Each **inode** has a unique number (the inode number). You can see the **inode** numbers with the **ls -li** command.

```
paul@RHELv4u4:~/test$ touch file1
paul@RHELv4u4:~/test$ touch file2
paul@RHELv4u4:~/test$ touch file3
paul@RHELv4u4:~/test$ ls -li
total 12
817266 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file1
817267 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file3
paul@RHELv4u4:~/test$
```

These three files were created one after the other and got three different **inodes** (the first column). All the information you see with this **ls** command resides in the **inode**, except for the filename (which is contained in the directory).

28.1.4. inode and file contents

Let's put some data in one of the files.

```
paul@RHELv4u4:~/test$ ls -li
total 16
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 1 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
paul@RHELv4u4:~/test$ cat file2
It is winter now and it is very cold.
We do not like the cold, we prefer hot summer nights.
paul@RHELv4u4:~/test$
```

The data that is displayed by the **cat** command is not in the **inode**, but somewhere else on the disk. The **inode** contains a pointer to that data.

28.2. about directories

28.2.1. a directory is a table

A **directory** is a special kind of file that contains a table which maps filenames to inodes. Listing our current directory with **ls -ali** will display the contents of the directory file.

```
paul@RHELv4u4:~/test$ ls -ali
total 32
817262 drwxrwxr-x 2 paul paul 4096 Feb 5 15:42 .
800768 drwx----- 16 paul paul 4096 Feb 5 15:42 ..
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 1 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
paul@RHELv4u4:~/test$
```

28.2.2. . and ..

You can see five names, and the mapping to their five inodes. The dot **.** is a mapping to itself, and the dotdot **..** is a mapping to the parent directory. The three other names are mappings to different inodes.

28.3. hard links

28.3.1. creating hard links

When we create a **hard link** to a file with **ln**, an extra entry is added in the directory. A new file name is mapped to an existing inode.

```
paul@RHELv4u4:~/test$ ln file2 hardlink_to_file2
paul@RHELv4u4:~/test$ ls -li
total 24
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 2 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
817270 -rw-rw-r-- 2 paul paul 92 Feb 5 15:42 hardlink_to_file2
paul@RHELv4u4:~/test$
```

Both files have the same inode, so they will always have the same permissions and the same owner. Both files will have the same content. Actually, both files are equal now, meaning you can safely remove the original file, the hardlinked file will remain. The inode contains a counter, counting the number of hard links to itself. When the counter drops to zero, then the inode is emptied.

28.3.2. finding hard links

You can use the **find** command to look for files with a certain inode. The screenshot below shows how to search for all filenames that point to **inode** 817270. Remember that an **inode** number is unique to its partition.

```
paul@RHELv4u4:~/test$ find / -inum 817270 2> /dev/null
/home/paul/test/file2
/home/paul/test/hardlink_to_file2
```

28.4. symbolic links

Symbolic links (sometimes called **soft links**) do not link to inodes, but create a name to name mapping. Symbolic links are created with **ln -s**. As you can see below, the **symbolic link** gets an inode of its own.

```
paul@RHELv4u4:~/test$ ln -s file2 symlink_to_file2
paul@RHELv4u4:~/test$ ls -li
total 32
817273 -rw-rw-r-- 1 paul paul 13 Feb 5 17:06 file1
817270 -rw-rw-r-- 2 paul paul 106 Feb 5 17:04 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
817270 -rw-rw-r-- 2 paul paul 106 Feb 5 17:04 hardlink_to_file2
817267 lrwxrwxrwx 1 paul paul 5 Feb 5 16:55 symlink_to_file2 -> file2
paul@RHELv4u4:~/test$
```

Permissions on a symbolic link have no meaning, since the permissions of the target apply. Hard links are limited to their own partition (because they point to an inode), symbolic links can link anywhere (other file systems, even networked).

28.5. removing links

Links can be removed with **rm**.

```
paul@laika:~$ touch data.txt
paul@laika:~$ ln -s data.txt sl_data.txt
paul@laika:~$ ln data.txt hl_data.txt
paul@laika:~$ rm sl_data.txt
paul@laika:~$ rm hl_data.txt
```

28.6. solution : links

1. Create two files named winter.txt and summer.txt, put some text in them.

```
echo cold > winter.txt ; echo hot > summer.txt
```

2. Create a hard link to winter.txt named hlwinter.txt.

```
ln winter.txt hlwinter.txt
```

3. Display the inode numbers of these three files, the hard links should have the same inode.

```
ls -li winter.txt summer.txt hlwinter.txt
```

4. Use the find command to list the two hardlinked files

```
find . -inum xyz #replace xyz with the inode number
```

5. Everything about a file is in the inode, except two things : name them!

The name of the file is in a directory, and the contents is somewhere on the disk.

6. Create a symbolic link to summer.txt called slsummer.txt.

```
ln -s summer.txt slsummer.txt
```

7. Find all files with inode number 2. What does this information tell you ?

It tells you there is more than one inode table (one for every formatted partition + virtual file systems)

8. Look at the directories /etc/init.d/ /etc/rc.d/ /etc/rc3.d/ ... do you see the links ?

```
ls -l /etc/init.d
```

```
ls -l /etc/rc2.d
```

```
ls -l /etc/rc3.d
```

9. Look in /lib with ls -l...

```
ls -l /lib
```

10. Use **find** to look in your home directory for regular files that do not(!) have one hard link.

```
find ~ ! -links 1 -type f
```


Appendix A. keyboard settings

A.1. about keyboard layout

Many people (like US-Americans) prefer the default US-qwerty keyboard layout. So when you are not from the USA and want a local keyboard layout on your system, then the best practice is to select this keyboard at installation time. Then the keyboard layout will always be correct. Also, whenever you use ssh to remotely manage a Linux system, your local keyboard layout will be used, independent of the server keyboard configuration. So you will not find much information on changing keyboard layout on the fly on linux, because not many people need it. Below are some tips to help you.

A.2. X Keyboard Layout

This is the relevant portion in `/etc/X11/xorg.conf`, first for Belgian azerty, then for US-qwerty.

```
[paul@RHEL5 ~]$ grep -i xkb /etc/X11/xorg.conf
Option      "XkbModel"  "pc105"
Option      "XkbLayout" "be"
```

```
[paul@RHEL5 ~]$ grep -i xkb /etc/X11/xorg.conf
Option      "XkbModel"  "pc105"
Option      "XkbLayout" "us"
```

When in Gnome or KDE or any other graphical environment, look in the graphical menu in preferences, there will be a keyboard section to choose your layout. Use the graphical menu instead of editing `xorg.conf`.

A.3. shell keyboard layout

When in bash, take a look in the `/etc/sysconfig/keyboard` file. Below a sample US-qwerty configuration, followed by a Belgian azerty configuration.

```
[paul@RHEL5 ~]$ cat /etc/sysconfig/keyboard
KEYBOARDTYPE="pc"
KEYTABLE="us"
```

```
[paul@RHEL5 ~]$ cat /etc/sysconfig/keyboard
KEYBOARDTYPE="pc"
KEYTABLE="be-latin1"
```

The keymaps themselves can be found in `/usr/share/keymaps` or `/lib/kbd/keymaps`.

```
[paul@RHEL5 ~]$ ls -l /lib/kbd/keymaps/
total 52
drwxr-xr-x 2 root root 4096 Apr  1 00:14 amiga
```



```
drwxr-xr-x 2 root root 4096 Apr 1 00:14 atari
drwxr-xr-x 8 root root 4096 Apr 1 00:14 i386
drwxr-xr-x 2 root root 4096 Apr 1 00:14 include
drwxr-xr-x 4 root root 4096 Apr 1 00:14 mac
lrwxrwxrwx 1 root root    3 Apr 1 00:14 ppc -> mac
drwxr-xr-x 2 root root 4096 Apr 1 00:14 sun
```

Appendix B. hardware

B.1. buses

B.1.1. about buses

Hardware components communicate with the **Central Processing Unit** or **cpu** over a **bus**. The most common buses today are **usb**, **pci**, **agp**, **pci-express** and **pcmcia** aka **pc-card**. These are all **Plug and Play** buses.

Older **x86** computers often had **isa** buses, which can be configured using **jumper**s or **dip switches**.

B.1.2. /proc/bus

To list the buses recognised by the Linux kernel on your computer, look at the contents of the **/proc/bus/** directory (screenshot from Ubuntu 7.04 and RHEL4u4 below).

```
root@laika:~# ls /proc/bus/
input  pccard  pci  usb
```

```
[root@RHEL4b ~]# ls /proc/bus/
input  pci  usb
```

Can you guess which of these two screenshots was taken on a laptop ?

B.1.3. /usr/sbin/lusb

To list all the usb devices connected to your system, you could read the contents of **/proc/bus/usb/devices** (if it exists) or you could use the more readable output of **lsusb**, which is executed here on a SPARC system with Ubuntu.

```
root@shaka:~# lsusb
Bus 001 Device 002: ID 0430:0100 Sun Microsystems, Inc. 3-button Mouse
Bus 001 Device 003: ID 0430:0005 Sun Microsystems, Inc. Type 6 Keyboard
Bus 001 Device 001: ID 04b0:0136 Nikon Corp. Coolpix 7900 (storage)
root@shaka:~#
```

B.1.4. /var/lib/usbutils/usb.ids

The **/var/lib/usbutils/usb.ids** file contains a gzipped list of all known usb devices.

```
paul@barry:~$ zmore /var/lib/usbutils/usb.ids | head
-----> /var/lib/usbutils/usb.ids <-----
#
# List of USB ID's
#
# Maintained by Vojtech Pavlik <vojtech@suse.cz>
```

```
# If you have any new entries, send them to the maintainer.
# The latest version can be obtained from
# http://www.linux-usb.org/usb.ids
#
# $Id: usb.ids,v 1.225 2006/07/13 04:18:02 dbrownell Exp $
```

B.1.5. /usr/sbin/lspci

To get a list of all pci devices connected, you could take a look at **/proc/bus/pci** or run **lspci** (partial output below).

```
paul@laika:~$ lspci
...
00:06.0 FireWire (IEEE 1394): Texas Instruments TSB43AB22/A IEEE-139...
00:08.0 Ethernet controller: Realtek Semiconductor Co., Ltd. RTL-816...
00:09.0 Multimedia controller: Philips Semiconductors SAA7133/SAA713...
00:0a.0 Network controller: RaLink RT2500 802.11g Cardbus/mini-PCI
00:0f.0 RAID bus controller: VIA Technologies, Inc. VIA VT6420 SATA ...
00:0f.1 IDE interface: VIA Technologies, Inc. VT82C586A/B/VT82C686/A...
00:10.0 USB Controller: VIA Technologies, Inc. VT82xxxxx UHCI USB 1....
00:10.1 USB Controller: VIA Technologies, Inc. VT82xxxxx UHCI USB 1....
...
```

B.2. interrupts

B.2.1. about interrupts

An **interrupt request** or **IRQ** is a request from a device to the CPU. A device raises an interrupt when it requires the attention of the CPU (could be because the device has data ready to be read by the CPU).

Since the introduction of pci, irq's can be shared among devices.

Interrupt 0 is always reserved for the timer, interrupt 1 for the keyboard. IRQ 2 is used as a channel for IRQ's 8 to 15, and thus is the same as IRQ 9.

B.2.2. /proc/interrupts

You can see a listing of interrupts on your system in **/proc/interrupts**.

```
paul@laika:~$ cat /proc/interrupts
      CPU0      CPU1
0:  1320048      555  IO-APIC-edge    timer
1:   10224        7  IO-APIC-edge    i8042
7:         0        0  IO-APIC-edge    parport0
8:         2        1  IO-APIC-edge    rtc
10:       3062      21  IO-APIC-fasteoi  acpi
12:        131        2  IO-APIC-edge    i8042
15:      47073        0  IO-APIC-edge    idel
18:         0        1  IO-APIC-fasteoi  yenta
19:      31056        1  IO-APIC-fasteoi  libata, ohci1394
20:      19042        1  IO-APIC-fasteoi  eth0
21:      44052        1  IO-APIC-fasteoi  uhci_hcd:usb1, uhci_hcd:usb2,...
22:     188352        1  IO-APIC-fasteoi  ra0
```

```
23:    632444    1 IO-APIC-fastecoi  nvidia
24:    1585    1 IO-APIC-fastecoi  VIA82XX-MODEM, VIA8237
```

B.2.3. dmesg

You can also use **dmesg** to find irq's allocated at boot time.

```
paul@laika:~$ dmesg | grep "irq 1[45]"
[ 28.930069] ata3: PATA max UDMA/133 cmd 0x1f0 ctl 0x3f6 bmdma 0x2090 irq 14
[ 28.930071] ata4: PATA max UDMA/133 cmd 0x170 ctl 0x376 bmdma 0x2098 irq 15
```

B.3. io ports

B.3.1. about io ports

Communication in the other direction, from CPU to device, happens through **IO ports**. The CPU writes data or control codes to the IO port of the device. But this is not only a one way communication, the CPU can also use a device's IO port to read status information about the device. Unlike interrupts, ports cannot be shared!

B.3.2. /proc/ioports

You can see a listing of your system's IO ports via **/proc/ioports**.

```
[root@RHEL4b ~]# cat /proc/ioports
0000-001f : dma1
0020-0021 : pic1
0040-0043 : timer0
0050-0053 : timer1
0060-006f : keyboard
0070-0077 : rtc
0080-008f : dma page reg
00a0-00a1 : pic2
00c0-00df : dma2
00f0-00ff : fpu
0170-0177 : ide1
02f8-02ff : serial
...
```

B.4. dma

B.4.1. about dma

A device that needs a lot of data, interrupts and ports can pose a heavy load on the cpu. With **dma** or **Direct Memory Access** a device can gain (temporary) access to a specific range of the **ram** memory.

B.4.2. /proc/dma

Looking at **/proc/dma** might not give you the information that you want, since it only contains currently assigned **dma** channels for **isa** devices.

```
root@laika:~# cat /proc/dma
1: parport0
4: cascade
```

pci devices that are using dma are not listed in **/proc/dma**, in this case **dmesg** can be useful. The screenshot below shows that during boot the parallel port received dma channel 1, and the Infrared port received dma channel 3.

```
root@laika:~# dmesg | egrep -C 1 'dma 1|dma 3'
[    20.576000] parport: PnPBIOS parport detected.
[    20.580000] parport0: PC-style at 0x378 (0x778), irq 7, dma 1...
[    20.764000] irda_init()
--
[    21.204000] pnp: Device 00:0b activated.
[    21.204000] nsc_ircc_pnp_probe() : From PnP, found firbase 0x2F8...
[    21.204000] nsc-ircc, chip->init
```

Appendix C. License

GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document "free" in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of "copyleft", which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The "Document", below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as "you". You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A "Modified Version" of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A "Secondary Section" is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document's overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The "Invariant Sections" are certain Secondary Sections whose titles

are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The "Cover Texts" are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A "Transparent" copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not "Transparent" is called "Opaque".

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The "Title Page" means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, "Title Page" means the text near the most prominent appearance of the work's title, preceding the beginning of the body of the text.

The "publisher" means any person or entity that distributes copies of the Document to the public.

A section "Entitled XYZ" means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as "Acknowledgements", "Dedications", "Endorsements", or "History".) To "Preserve the Title" of such a section when you modify the Document means that it remains a section "Entitled XYZ" according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either

commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- * A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.

* B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.

* C. State on the Title page the name of the publisher of the Modified Version, as the publisher.

* D. Preserve all the copyright notices of the Document.

* E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.

* F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.

* G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.

* H. Include an unaltered copy of this License.

* I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.

* J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.

* K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.

* L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.

* M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.

* N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.

* O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of,

you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements".

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an "aggregate" if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled "Acknowledgements", "Dedications", or "History", the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License "or any later version" applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies

that a proxy can decide which future versions of this License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

"Massive Multiauthor Collaboration Site" (or "MMC Site") means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A "Massive Multiauthor Collaboration" (or "MMC") contained in the site means any set of copyrightable works thus published on the MMC site.

"CC-BY-SA" means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

"Incorporate" means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is "eligible for relicensing" if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.

Linux

Contents:

1. Basic Commands
2. Help Commands
3. File System Hierarchy
4. Editors in Linux
5. Permissions and ownership
6. Processes and Signal Commands
7. Package Management Commands
8. Networking Commands
9. Disk Management Commands
10. User and Group Commands

Commands

1. PWD - Present Working Directory
 - ⇒ It Displays present working directory
 - Syntax: pwd**
2. Ls – List of files
 - ⇒ It displays list of files or directories of present working directory
 - Syntax: ls**
 - ⇒ ls / -It Displays list of files or directories in root directory.
 - ⇒ ls -l : To view the long list of file and directory.
 - ⇒ ls -a: To view the hidden files.
 - ⇒ ls -al: To view the hidden files in long format.
 - ⇒ ls D*: It display the file or directory, starting character with D.
 - ⇒ ls -d ????: It displays the file or directory, whose length is of 4 characters.
 - ⇒ ls dir_name: Display the list of files or directories in the specific directory.
3. Cd - Change directory
 - ⇒ It is used to switch from one directory to another directory.
 - Syntax: cd**
 - ⇒ **cd dir_name**: It change to the specified directory.
 - ⇒ **cd ..** : It goes to one step back means previous working directory.
 - ⇒ **cd ../..** : It goes to two step back.
4. Cat : Concatenate and combining files.
 - ⇒ It is used create a file.
 - ⇒ It is used to display the content of the file.
 - ⇒ It is used to combining of files and copying the content to the another file.
 - ⇒ It is used to append the content to the file.

- Syntax: cat > file_name** – It creates a file and then prompt us to write content. Use ctrl+d to save the content.
- ⇒ **cat file_name** : It displays the content of file.
 - ⇒ **cat file_name1 file_name2 > file_name3** – Combining of two files and copying the content to the another new file.
 - ⇒ **cat >> file_name1**: Appending the content to the existing file.
5. Touch : Creates an empty file
- ⇒ It is used to create empty file and also creates multiple empty files at a time.
- Syntax: touch file_name** – Creates an empty file.
- ⇒ **Touch file_name1 file_name2** – Creates multiple empty files simultaneously.
6. MKDIR – Making Directory
- ⇒ It creates directory and also create directory structure.
- Syntax: mkdir dir_name** – Creates a directory.
- ⇒ **mkdir -p dir_structure** – Creates a nested directory structure.
 - ⇒ **Note** : To view the nested directory structure, we follow below syntax
 - ⇒ **ls -R**
7. RMDIR – Remove Directory.
- ⇒ It removes the directory and also removes the directory structure.
- Syntax: rmdir dir_name** – Removes a directory.
- ⇒ **rmdir dir_structure** – Removes a top level sub-directory in the directory in the directory structure.
 - ⇒ **rmdir -p dir_structure** – Removes a directory structure with sub-directories.
8. RM – Remove files and directories.
- ⇒ It removes files and directories in the home directory.
- Syntax: rm file_name** – Removes file.
- ⇒ **rm -rf dir_structure** – Removes the whole directory structure.
9. Mv – Rename and move.
- ⇒ It is used to move the file from one file to another.
- Syntax: mv file_name1 file_name2** – It moves the content from file_name1 to the file_name2. There is no content in the file_name1
10. Cp – Copying the file.
- ⇒ It is used to copying the file.
- Syntax: cp file_name1 file_name2** – It copying the content from file_name1 to the file_name2. Both the files having the same content.
11. Echo – It is used to displaying lines of text or string and use directions to store the content in a new file.
- Syntax: echo “Write content here” > file_name**

SOME HELP COMMANDS

1. **whatis <command_name>**: The command gives a one-line description of a command.

2. **Man <command_name>:** The man command provides detailed documentation about commands.
3. **Info <command_name>:** The info command shows a more detailed and structured form of documentation, often used for GNU utilities.
4. **<command_name> --help:** Many commands have a --help flag that displays a brief summary of how to use the command and its options.
5. **Which <command_name>:** The which command shows the location of an executable file for a command.

FILE SYSTEM HIERARCHY

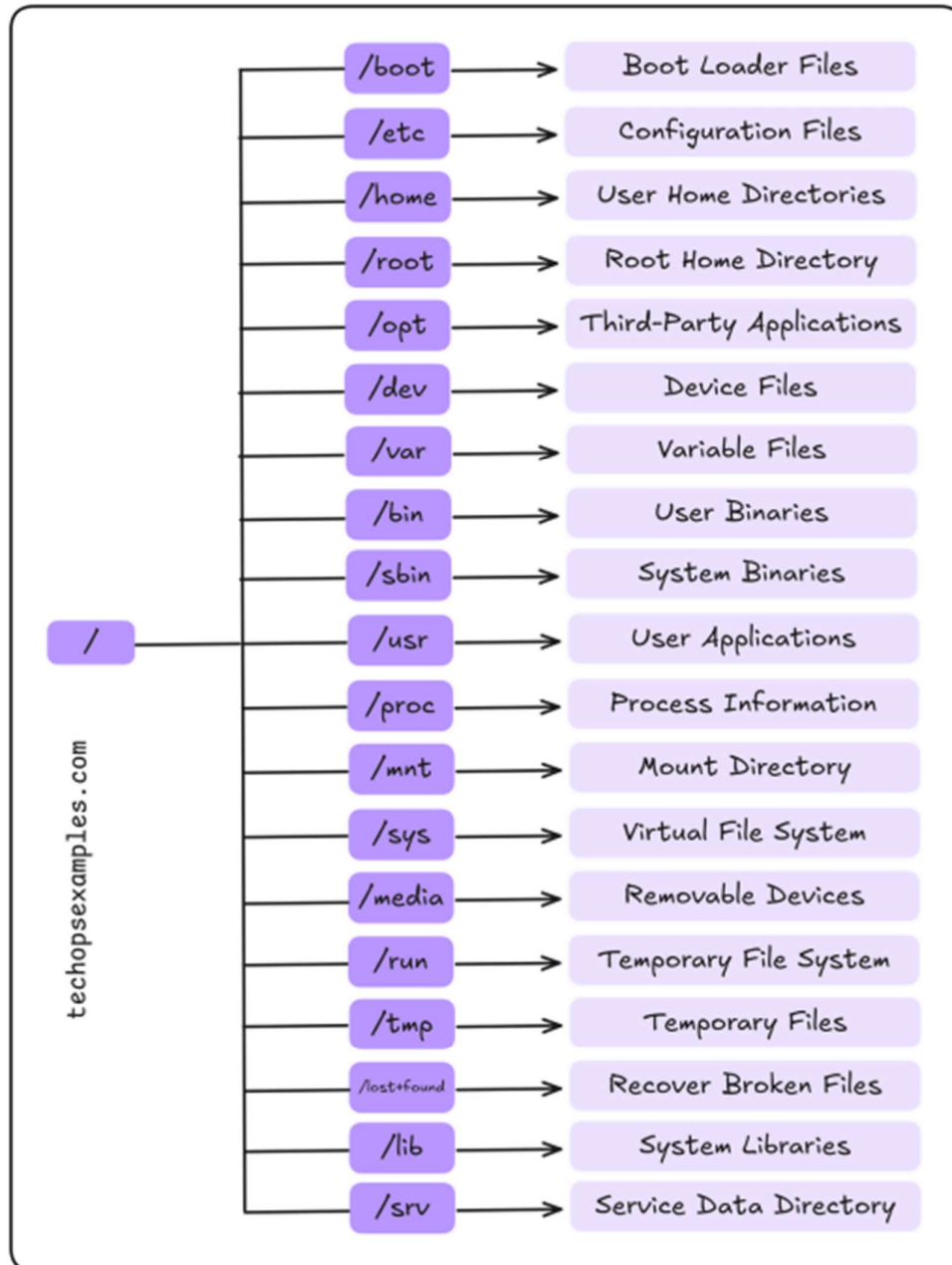
The Linux File System is also known as the **Filesystem Hierarchy Standard(FHS)**, provides a standard directory structure for Unix-Like Operating Systems.

Directories and their Purpose:

1. **Root Directory(/):** It is base of the file system hierarchy, where all other directories and files are nested. It's the starting point of the directory structure.
2. **Boot Loader Files(/boot):** It stores boot loader files, including the Linux kernel and initial RAM disk.
3. **Configuration Files (/etc):** It contains system-wide configuration files. It is important to note that /etc should hold static configuration files and not binaries.
4. **User Home Directories (/home):** The directory for user home directories, where personal files and user-specific settings are stored.
5. **Root Home Directory (/root):** The home directory of the root (superuser), containing configuration and environment files specific to root. Only the root user has access to this directory.

6. **Process Information (/proc):** A virtual file system that provides process and kernel information as files, dynamically generated by the sys

Linux File system hierarchy



7. **Mount Directory (/mnt) :** It is used for temporarily mounting file systems.
8. **Virtual File System (/sys):** It provides information about devices, drivers, and some kernel features.
9. **Removable Devices (/media):** It serves as a mount point for removable media like CD-ROMs.

10. **Temporary File System (/run):** It contains run-time variable data, such as information about the system since the last boot.
11. **Temporary Files (/tmp):** A directory for temporary files, which may not be preserved between system reboots.
12. **Recover Broken Files (/lost+found):** Created during the recovery process after a system crash or improper shutdown. It contains recovered files that were part of the file system but became corrupted or lost.
13. **System Libraries (/lib):** It includes essential libraries needed for the binaries in /bin and /sbin.
14. **Service Data Directory (/srv):** Contains data for services offered by the system, such as web server data for Apache or FTP data for FTP servers. This directory holds files related to server operations.
15. **Third-Party Applications (/opt):** Contains add-on software packages and third-party applications that are not part of the default system. For example, large software installations like Java or other proprietary programs can be placed here.
16. **Device Files (/dev):** It contains device files that represent hardware components and other system devices.
17. **Variable Files (/var):** It contains variable files, such as logs and spool files, whose content is expected to continually change during normal operation.
18. **User Binaries (/bin):** It contains essential user command binaries that are required for basic system operation and are available to all users, such as `ls`, `cp`, `mv`, and `rm`. These are crucial commands for interacting with the system.
19. **System Binaries (/sbin):** It stores essential system binaries, like fsck, init, and route.
20. **User Applications (/usr):** A second hierarchy for read-only user data, including the majority of user utilities and applications.
21. **Process Information (/proc):** A virtual file system that provides process and kernel information as files, dynamically generated by the sys.

EDITORS IN LINUX

The most commonly used text editors in Linux can vary depending on the user's expertise, environment (command-line or graphical), and purpose.

1. Vim/Vi (Command-Line)

Popularity: Widely used by system administrators, developers, and advanced users.

Reason: Pre-installed on almost every Linux distribution, powerful for fast text manipulation, and highly customizable.

Strengths:

- a. Extremely efficient for large-scale editing tasks.
- b. Once familiar, it can dramatically speed up text manipulation

Syntax: `vi file_name`

2. Nano (Command-Line)

Popularity: Popular among beginners and for quick file edits.

Reason: Simple to use, user-friendly, and often set as the default text editor on many distributions.

Strengths:

- a. Ideal for quick, straightforward edits in the terminal.
- b. No steep learning curve compared to Vim or Emacs

Syntax: `nano file_name`

3. Emacs (Command-Line and Graphical)

Popularity: Preferred by developers, power users, and enthusiasts who value extensibility.

Reason: Highly customizable with support for extensive plugins and can serve as more than just a text editor (e.g., for project management or email).

Strengths:

- a. Highly extensible with Lisp-based scripting.
- b. Versatile (can be used for coding, note-taking, task management, etc.).

Syntax: `emacs <filename>`

4. gedit (Graphical, GNOME Default Editor)

Popularity: Commonly used in GNOME desktop environments for casual or simple text editing.

Reason: Lightweight, user-friendly, and comes pre-installed with GNOME desktop environments.

Strengths:

- a. Clean graphical interface.
- b. Good for basic text editing, configuration files, and script writing.

Syntax: `gedit <filename>`

5. Visual Studio Code (VS Code) (Graphical)

Popularity: Extremely popular among developers, especially for coding projects.

Reason: Extensive plugin marketplace, cross-platform, and support for many programming languages.

Strengths:

- a. Rich features for coding, such as IntelliSense, debugging, and Git integration.
- b. Lightweight and extendable with numerous plugins.

Syntax: `code <filename>`

PERMISSIONS AND OWNERSHIP IN LINUX

In Linux, permissions and ownership are fundamental concepts for file management and security. They control who can read, write, or execute files and directories.

1. Ownership

Every file and directory in Linux has an owner and a group associated with it. There are three types of ownership:

- **User (Owner):** The user who owns the file. This is usually the person who created the file.
- **Group:** A set of users who share access to the file. Any user in the group can be given specific permissions to the file.
- **Others:** All other users who are not the owner or part of the group

2. Permissions

Linux permissions define what actions the owner, group, and others can perform on a file or directory. There are three types of permissions:

- **Read (r):** Allows a user to read the contents of the file or list the contents of a directory.
- **Write (w):** Allows a user to modify the contents of a file or create and delete files within a directory.
- **Execute (x):** Allows a user to execute a file (if it is a script).

These permissions can be represented numerically:

* r = 4

* w = 2

* x = 1

For each user category (Owner, Group, Others), permissions are represented as a three-digit number:

* 777: Full permissions (rwx) for Owner, Group, and Others.

* 755: Full permissions (rwx) for the Owner, and read/execute (r-x) for Group and Others. * *

* 644: Read/write (rw-) for the Owner, and read-only (r--) for Group and Others.

Example Breakdown of 777

* Owner (7): Read (4), Write (2), Execute (1) = 7 (rwx)

* Group (7): Read (4), Write (2), Execute (1) = 7 (rwx)

* Others (7): Read (4), Write (2), Execute (1) = 7 (rwx)

COMMANDS:

1. **ls -l**: Displays a detailed list of files and directories, including their permissions and ownership
2. **Chown**: To change the ownership of a file or directory
Syntax: chown user_name file_name

To view the file, use the following command.

ls -l

3. **Chgrp**: Changes group ownership of a file or directory.
Syntax: chgrp g file_name
4. **Chmod**: Changes the permissions of a file or directory.
Syntax: chmod [permissions] file_name

Methods for changing permissions:

- a. **Symbolic Method**: Symbols are used with +, -, and = to add, remove, or set permissions.
 - ⇒ **chmod u+x file.txt**: Add execute permission for the owner (user)
 - ⇒ **chmod g-w file.txt**: Remove write permission for the group
 - ⇒ **chmod a=r file.txt**: Set read-only permissions for everyone
- b. **Numeric (Octal) Method**:
 - ⇒ **chmod 755 file.txt**: Add all permissions to the user, add read and execute permissions to the group, add read and execute permissions to the others.

PROCESSES AND SIGNAL COMMANDS IN LINUX

In a Linux system, processes represent running programs or tasks. Every process is assigned a unique Process ID (PID). Managing these processes is essential for system performance, debugging, and ensuring stability.

Signals allow you to communicate with processes, for instance, to terminate or modify their behaviour, making it easier to automate tasks, manage workloads, and maintain system health

COMMANDS:

1. **ps**: Displays information about running processes.
Syntax: ps
 - ⇒ **ps aux**: Displays all running processes in detail for all users.
 - ⇒ **ps -ef**: Another format to list all processes in full detail.
2. **Top**: Displays a real-time view of active processes.
Syntax: top -This command shows processes along with CPU and memory usage, and updates dynamically.
3. **Kill**: Sends a signal (usually to terminate) to a process.

Syntax: kill signal PID -This sends the signal to forcibly terminate process with PID.

4. **Killall:** Kills all processes by name, rather than by PID.

Syntax: killall

⇒ **killall process_name** -This kills all instances of the firefox process.

5. **Htop:** A more user-friendly version of **top** that provides better visuals and controls.

Syntax: htop - Allows you to easily kill processes, view tree structures

6. **Nice:** Adjusts the priority (niceness) of a process.

Syntax: nice -n priority command

7. **Renice:** To change the priority of an already running process

Syntax: renice [priority] PID

PACKAGE MANAGEMENT IN LINUX

Package Management in Linux refers to the process of installing, updating, and removing software packages on a Linux system. Different Linux distributions use different package managers, but the core concepts remain similar.

Debian-based systems (e.g., Ubuntu, Debian):

1. **APT** (Advanced Package Tool)
2. Package files: **.deb**

Red Hat-based systems (e.g., CentOS, Fedora, RHEL):

3. **YUM** (Yellowdog Updater Modified) / **DNF** (Dandified YUM)
4. Package files: **.rpm**

COMMANDS:

1. **sudo apt update:** Update package lists
 - ⇒ **sudo apt upgrade:** Upgrade installed packages
 - ⇒ **sudo apt install package_name:** Install a package
 - ⇒ **sudo apt remove package_name:** Remove a package
 - ⇒ **sudo apt purge package_name:** Purge a package (remove along with configuration files)
 - ⇒ **apt search package_name:** Search for a package
 - ⇒ **apt show package_name:** show package details.
2. **sudo dnf check-update:** Update package lists
 - ⇒ **sudo dnf upgrade:** Upgrade installed packages
 - ⇒ **sudo dnf install package_name:** Install a package
 - ⇒ **sudo dnf remove package_name:** Remove a package
 - ⇒ **dnf search package_name:** Search for a package
 - ⇒ **dnf info package_name:** show package details.

NETWORKING COMMANDS

1. **Ifconfig:** It Displays or configures network interface, such as IP addresses.
Syntax: `ifconfig eth0` – It displays interface information.
 - ⇒ `ifconfig enpos3 192.189.2.6 netmask 255.255.255.8` – It changes the IP address and also netmask in the interface.
 - ⇒ `sudo ifconfig down`: It is used to disable or shut down the network interface.
 - ⇒ `sudo ifconfig up`: It is used to enable or bring up a network interface.
2. **Ping:** It checks the connectivity between your system and a remote host.
Syntax: `ping google.com` - Displays network connectivity between your machine and Google's servers.
3. **Netstat:** Displays network connections, routing tables, interface statistics.
Syntax: `netstat`- It displays a list of active connections for TCP, UDP, and other protocols.
 - ⇒ `netstat -l`: It displays only the ports that are in a **listening** state.
 - ⇒ `netstat -lt`: It displays all the TCP ports that are currently in the listening state.
 - ⇒ `netstat -lu`: It displays all the UDP ports that are currently in the listening state.
 - ⇒ `netstat -n`: It displays active connections using numeric IP addresses.
 - ⇒ `netstat -t`: It displays lists all active TCP connections.
 - ⇒ `netstat -u`: It displays lists all active UDP connections.
4. **Curl:** It transfers data from or to a server using various protocols (HTTP, FTP, etc.).
Syntax: `curl http://example.com`
5. **Wget:** It is used for downloading files from the web.
Syntax: `wget http://example.com/file.zip`
6. **Nslookup:** The nslookup command in Linux is used to query Domain Name System (DNS) servers and obtain information about domain names or IP addresses. It is primarily used for troubleshooting DNS-related issues and performing DNS lookups.
Syntax: `nslookup google.com`- This command queries the default DNS server and returns the IP addresses associated with google.com.
 - ⇒ `nslookup 8.8.8.8` - This performs a reverse DNS lookup, translating an IP address back to its domain name.
 - ⇒ `nslookup google.com 8.8.8.8`: This command performs a DNS query for google.com using Google's public DNS server (8.8.8.8) instead of the system's default DNS server.
7. **Traceroute:** The traceroute command in Linux is used to trace the path that packets take to reach a destination, showing each hop along the route from your machine to the target.
Syntax: `traceroute google.com`- This command will show the route that packets take to reach Google's servers, including all intermediate routers (hops) along the way.
 - ⇒ `traceroute -m 10 google.com`: Limits the traceroute to a maximum of 10 hops.

DISK MANAGEMENT COMMANDS

It is used to managing storage devices, partitions, and file systems. Here are some commonly used commands for managing disks in Linux.

1. **Df (Disk Usage):** It shows the amount of disk space used and available on file systems.
Syntax: `df -h` (h – human readable format)
2. **Du (Directory Disk Usage):** It summarizes disk usage for directories and files.
Syntax: `du -sh /home/user/` (s – shows size)
3. **Fdisk (Disk partitioning):** A command-line utility to create, modify, or delete partitions on a disk.
Syntax: `sudo fdisk/dev/sda`
4. **Mount (Mount File Systems):** It is used to mounts a file system to a specified directory.
Syntax: `sudo mount /dev/sda1 /mnt`
Note: To view the mount file systems use the following command.
`mount | grep "^/dev"`
5. **Umount (Unmount File Systems):** Unmounts a file system
Syntax: `sudo umount /mnt`
6. **Lsblk (List Block Devices):** Displays information about all available block devices (disks and partitions).
Syntax: `lsblk`

USER AND GROUP MANAGEMENT

In Linux, managing users and groups is essential for system administration, access control, and security.

1. **Useradd:** Adds a new user to the system.
Syntax: `useradd user_name`
⇒ `useradd -m -d /home/newuser user_name` - To specify a home directory with useradd.

To check the groups, use the following command
`tail /etc/passwd`
⇒ `useradd -u 2024 -c "Hi" -d /home/newuser -s /bin/csh user_name` – Displays new user with all the properties.
2. **Passwd (Set or Change Password):** Sets or changes the password for a user.
Syntax: `sudo passwd username`

To view the passwd properties use the following command
`grep user_name /etc/shadow`

3. **Userdel (Delete a User):** Deletes a user from the system.
Syntax: `userdel user_name`
⇒ `userdel -r username:` remove the user's home directory.

4. **Usermod (Modify User Account):** Modifies a user's settings such as username, home directory, group, etc
- ⇒ **sudo usermod -l newname oldname:** This command changes the login name of a user from oldname to newname.
 - ⇒ **sudo usermod -d /new/home/directory -m username:** Use this to change the location of a user's home directory. The -m option moves the content of the current home directory to the new directory.
 - ⇒ **sudo usermod -L username:** Locks a user's account, preventing them from logging in.
 - ⇒ **sudo usermod -U username:** Unlocks a previously locked account.
 - ⇒ **sudo usermod -u 1001 username:** The -u option changes the user's numerical user ID.
 - ⇒ **sudo usermod -r username:** The -r option modifies the user to a system user account (typically used for system services).
 - ⇒ **sudo usermod -s /bin/bash username:** The -s option changes the default shell for the user (e.g., changing to /bin/bash).

5. **Groupadd (Create a new Group): Adds a new group to the system.**

Syntax: sudo groupadd groupname

6. **gpasswd (Administer / Assign Group Passwords):** Assigns a password to a group

Syntax: sudo gpasswd groupname

7. **groups (Show Groups for a User):** Displays the groups a user is a member of.

Syntax: groups username

8. **groupdel (Delete the group): Removes a group from the system.**

Syntax: sudo groupdel group_name

9. **groupmod:** The groupmod command in Linux is used to modify an existing group's attributes, such as changing the group name or the Group ID (GID).

Syntax: sudo groupmod [options] groupname

- ⇒ **sudo groupmod -n new_name old_name:** You can use groupmod to rename an existing group.
- ⇒ **sudo groupmod -g 1001 developers:** You can change the numeric Group ID (GID) of an existing group.
- ⇒ **sudo groupmod -n <name> -g <id> staff:** You can change both the name and GID of a group in a single command.



Linux & Git cheat sheet

File Operations:

- **ls**: Lists all files and directories in the present working directory
- **ls -R**: Lists files in sub-directories as well
- **ls -a**: Shows hidden files
- **ls -al**: Lists files and directories with detailed information 📄📁
- **cd directoryname**: Changes the directory 📁
- **cd ..**: Moves one level up ↗
- **pwd**: Displays the present working directory 📄
- **cat > filename**: Creates a new file 📄✍
- **cat filename**: Displays the file content 📄👁
- **touch filename**: Creates or modifies a file 🛠📄
- **rm filename**: Deletes a file ✖📄
- **cp source destination**: Copies files from source to destination ➡📄
- **mv source destination**: Moves files from source to destination 🔄📄
- **find / -name filename**: Finds a file or a directory by its name 🔍📄📁
- **file filename**: Determines the file type 📄?
- **less filename**: Views the file content page by page 📄📄
- **head filename**: Views the first ten lines of a file 📄⬆TOP
- **tail filename**: Views the last ten lines of a file 📄⬆END
- **du -h --max-depth=1**: Shows the size of each directory 📁📏

Directory Operations:

- **mkdir directoryname**: Creates a new directory 📁NEW
- **rmdir directoryname**: Deletes a directory ✖📁
- **cp -r source destination**: Copies directories recursively ➡📁
- **mv olddir newdir**: Renames directories 🔄📁
- **find / -type d -name directoryname**: Finds a directory starting from root 🔍📁

Process Operations:

- **ps**: Displays your currently active processes 🔄
- **top**: Displays all running processes 🔄
- **kill pid**: Kills the process with given pid ✖🔄

- **pkill name:** Kills the process with the given name ❌🔄
- **bg:** Resumes suspended jobs without bringing them to foreground ▶🔄
- **fg:** Brings the most recent job to foreground ▶
- **renice +n [pid]:** Change the priority of a running process 🔄⚙️
- **&>filename:** Redirects both the stdout and the stderr to the file 🔄📄
- **1>filename:** Redirect the stdout to file 🔄📄
- **2>filename:** Redirect stderr to file 🔄📄

🔒 File Permissions:

- **chmod octal filename:** Change the permissions of file to octal 🗑️📄
- **chown ownername filename:** Change file owner 👤📄
- **chgrp groupname filename:** Change group owner 👤📄

🔗 Networking:

- **ping host:** Ping a host and outputs results 🌐
- **netstat -pnltu:** Display various network related information 🔍🌐
- **ssh user@host:** Remote login into the host as user 🌐
- **wget url:** Download files from the web 🌐↓
- **curl url:** Sends a request to a URL and returns the response 🌐🔄

📁 Archives and Compression:

- **tar cf file.tar files:** Create a tar named file.tar containing files 🗑️📄
- **tar xf file.tar:** Extract the files from file.tar 🗑️📁
- **gzip file:** Compresses file and renames it to file.gz 📄➡️📄.gz
- **gzip -d file.gz:** Decompresses file.gz back to file 📄.gz➡️📄
- **zip -r file.zip files:** Create a zip archive named file.zip 🗑️📄
- **unzip file.zip:** Extract the contents of a zip file 🗑️📁

🔍 Text Processing:

- **grep pattern files:** Search for pattern in files 🔍📄
- **grep -r pattern dir:** Search recursively for pattern in dir 🔍📁
- **echo 'text':** Prints text 🗣️
- **sed 's/string1/string2/g' filename:** Replaces string1 with string2 in filename 🔄📄
- **diff file1 file2:** Compares two files and shows the differences ↔️📄
- **wc filename:** Count lines, words, and characters in a file 📄📊
- **awk:** A versatile programming language for working on files 📄🐱
- **sed -i 's/string1/string2/g' filename:** Replace string1 with string2 in filename 🔄📄 (In-place edit)
- **cut -d':' -f1 /etc/passwd:** Cut out the first field of each line in /etc/passwd, using colon as a field delimiter 🔪📄

💾 Disk Usage:

- **df:** Shows disk usage 🗑️📊
- **du:** Shows directory space usage 📁📊
- **free:** Show memory and swap usage 📈🔄

- **whereis app:** Show possible locations of app 🔍📁

💻 System Info:

- **date:** Show the current date and time 📅🕒
- **cal:** Show this month's calendar 📅
- **uptime:** Show current uptime 🕒
- **w:** Display who is online 👤
- **uname -a:** Show kernel information 🖥️🔍

📦 Package Installations:

- **sudo apt-get update:** Updates package lists for upgrades 🔄📦
- **sudo apt-get upgrade:** Upgrades all upgradable packages 🔄📦
- **sudo apt-get install pkgname:** Install pkgname 📦
- **sudo apt-get remove pkgname:** Removes pkgname 📦❌

🔧 Others (mostly used in scripts):

- **command | grep pattern:** Pipe the output of command to grep for searching ▶️🔍

🔍 Search and Find:

- **locate filename:** Find a file by its name 🔍📄
- **whereis programname:** Locate the binary, source, and manual page files for a command 🔍📁
- **which commandname:** Shows the full path of (shell) commands 🔍🚀

📦 Compression / Archives:

- **tar -cvf archive.tar dirname/:** Create a tar archive 📦📁
- **tar -xvf archive.tar:** Extract a tar archive 📦📄
- **tar -jcvf archive.tar.bz2 dirname/:** Create a compressed bz2 archive 📦📁
- **tar -jxvf archive.tar.bz2:** Extract a bz2 archive 📦📄

💻 Shell Scripting:

- **#!/bin/bash:** Shebang line to specify the script interpreter 🔄▼
- **\$0, \$1, ..., \$9, \${10}, \${11}:** Script arguments 🔄₁₂₃₄
- **if [condition]; then ... fi:** if statement in bash scripts 🔄🔒
- **for i in {1..10}; do ... done:** for loop in bash scripts 🔄🔄
- **while [condition]; do ... done:** while loop in bash scripts 🔄🔄
- **function name() {...}:** Define a function 🔄🎯

🔧 Environment Variables:

- **env:** Display all environment variables 🌐🔧
- **echo \$VARIABLE:** Display the value of an environment variable 👤🔧
- **export VARIABLE=value:** Set the value of an environment variable 🔄🔧
- **alias new_command='old_command options':** Create a new command that executes the old command with the specified options 📄NEW🔧

- **echo \$PATH:** Print the PATH environment variable 🛠️🔑
- **export PATH=\$PATH:/new/path:** Add /new/path to the PATH 🛠️🔑

🔄 Others (mostly used in scripts):

- **command1 ; command2:** Run command1 and then command2 🔄🔄
- **command1 && command2:** Run command2 if command1 is successful 🔄✅
- **command1 || command2:** Run command2 if command1 is not successful 🔄❌
- **command &:** Run command in background 🔄🕒

Remember, you can always use the **man command** (e.g., **man ls**) to get more information about each command. Happy coding! 🚀

💻 System Monitoring and Performance:

- **iostat:** Reports Central Processing Unit (CPU) statistics and input/output statistics for devices, partitions, and network filesystems 🔄📈
- **vmstat:** Reports information about processes, memory, paging, block IO, traps, disks, and CPU activity 🔄📈
- **htop:** An interactive process viewer for Unix systems 🔄💻

💿 Disk Usage:

- **dd if=/dev/zero of=/tmp/output.img bs=8k count=256k:** Create a file of a certain size for testing disk speed 🗄️📏
 - This command will create a file named output.img in the /tmp directory with a size of approximately 2 GB (8 KB * 256 KB). It will be filled with zeros.

example:

```
$ dd if=/dev/zero of=/tmp/output.img bs=8k count=256k
262144+0 records in
262144+0 records out
```

2147483648 bytes (2.1 GB, 2.0 GiB) copied, 2.18953 s, 981 MB/s

Note: The output of the dd command may vary based on your system's performance.

- **hdparm -Tt /dev/sda:** Measure the read speed of your hard drive 🗄️📶
 - This command will measure the read speed of your hard drive (/dev/sda) and display the result in MB/sec.

example:

```
$ sudo hdparm -Tt /dev/sda
```

/dev/sda:

Timing cached reads: 20124 MB in 2.00 seconds = 10076.39 MB/sec

Timing buffered disk reads: 588 MB in 3.01 seconds = 195.47 MB/sec

💡 Others:

- **yes > /dev/null &**: Use this command to push a system to its limit 💡🚩
 - Running this command will continuously print the letter "y" and redirect the output to /dev/null, which discards it. The process will run in the background.
- **example:**

```
$ yes > /dev/null &
```

```
[1] 12345
```

The number in brackets [1] and the PID 12345 represent the background job's ID and process ID, respectively.

- **:(){ :|:& };:** A fork bomb – handle with care. Do not run this command on a production system 💣🚫
 - **:(){ :|:& };:**
Warning: Do not run this command on a production system or any system you care about. It is a fork bomb and can quickly consume system resources, causing the system to become unresponsive or crash.

A fork bomb is a self-replicating program that creates a large number of child processes, overwhelming the system. It can be executed as follows:

```
$ :(){ :|:& };:
```

If you accidentally run this command, you may need to restart your system to recover from its effects.

📅 Cron Jobs:

- **crontab -l**: List all your cron jobs 📄👤
- **crontab -e**: Edit your cron jobs 📄✎
- **crontab -r**: Remove all your cron jobs ✖📄
- **crontab -v**: Display the last time you edited your cron jobs 🕒📄
- **crontab file**: Install a cron job from a file 📄🔄
- **@reboot command**: Schedule a job to run at startup 📄🕒🔄

Git Cheat sheet

🔄 Git Commands:

- **git init**: Initialize a local git repository 🔄📁
- **git clone url**: Create a local copy of a remote repository 🔄🔗📁

git add filename: Add a file to the staging area +

git commit -m "Commit message": Commit changes with a message 

git status: Check the status of the working directory 

git pull: Pull latest changes from the remote repository 

git push: Push changes to the remote repository 

git branch: List all local branches 

git branch branchname: Create a new branch +

git checkout branchname: Switch to a branch ↔

git merge branchname: Merge a branch into the active branch 

git stash: Stash changes in a dirty working directory 

git stash apply: Apply changes from a stash ➡

git log: View commit history 

git reset: Reset your HEAD pointer to a previous commit ←
BACK

git rm filename: Remove a file from version control 

git rebase: Reapply commits on top of another base tip 

Version Control (Git commands):

git revert: Create a new commit that undoes all of the changes made in a particular commit, then apply it to the current branch ←
BACK

git cherry-pick commitID: Apply the changes introduced by some existing commits 

Linux Cheat Sheet

File and Directory Management

1. **ls** - Lists directory contents.

- ls - Basic list.
- ls -l - List with details.
- Example: ls -la Directory

2. **cd** - Change the current directory.

- cd Directory - Changes to "Directory".
- Example: cd /home

3. **pwd** - Print the current directory path.

- pwd - Outputs the current directory.
- Example: pwd

4. **mkdir** - Create a new directory.

- mkdir Directory - Creates "Directory".
- Example: mkdir /home/Directory

5. **rmdir** - Remove an empty directory.

- rmdir Directory - Deletes "Directory".
- Example: rmdir /home/Directory

6. **rm** - Delete files and directories.

- `rm file` - Deletes "file".
- `rm -r Directory` - Deletes "Directory" recursively.
- Example: `rm -rf /home/Directory`

7. **cp** - Copy files or directories.

- `cp file new_file` - Copies "file" to "new_file".
- `cp -r Directory new_directory` - Copies "Directory" recursively.
- Example: `cp -r Directory /backup/`

8. **mv** - Move or rename files and directories.

- `mv file new_file` - Renames "file" to "new_file".
- Example: `mv file /home/Directory/`

9. **touch** - Create an empty file.

- `touch file` - Creates "file".
- Example: `touch /home/Directory/newfile`

10. **cat** - Concatenate and display file contents.

- `cat file` - Shows contents of "file".
- Example: `cat file`

11. **less** - View file content interactively.
 - `less file` - Opens "file" in viewer.
 - Example: `less file`
12. **more** - View file content page-by-page.
 - `more file` - Views file with paging.
 - Example: `more file`
13. **head** - Show the first few lines of a file.
 - `head -n 5 file` - Shows first 5 lines.
 - Example: `head file`
14. **tail** - Show the last few lines of a file.
 - `tail -n 5 file` - Shows last 5 lines.
 - `tail -f file` - Shows last lines with live updates.
 - Example: `tail -f /var/log/syslog`
15. **find** - Search for files and directories.
 - `find Directory -name "*.txt"` - Searches for .txt files.
 - Example: `find /home -type f -name "*.sh"`
16. **locate** - Find files by name (needs mlocate package).
 - `locate file` - Searches for "file".
 - Example: `locate file`

17. **du** - Disk usage of files and directories.
 - du -sh Directory - Shows size of "Directory".
 - Example: du -h Directory
18. **df** - Display disk space usage.
 - df -h - Disk usage in human-readable format.
 - Example: df -h
19. **stat** - Display detailed file or directory status.
 - stat file - Shows stats of "file".
 - Example: stat file
20. **tree** - Show directory structure in tree format.
 - tree Directory - Displays "Directory" structure.
 - Example: tree /home

File Permissions and Ownership

21. **chmod** - Change file permissions.
 - chmod 755 file - Sets permissions for "file".
 - Example: chmod +x script.sh
22. **chown** - Change file ownership.
 - chown user file - Changes owner to "user".
 - Example: chown root file
23. **chgrp** - Change group ownership.
 - chgrp group file - Changes group to "group".
 - Example: chgrp admin file
24. **umask** - Set default permissions for new files.
 - umask 022 - Default file permissions.
 - Example: umask

Process Management

25. **ps** - Show active processes.
 - ps aux - Shows all processes.
 - Example: ps aux | grep nginx
26. **top** - Monitor system processes in real time.
 - top - Live view of processes.
 - Example: top
27. **htop** - Enhanced process viewer (requires htop).
 - htop - Interactive process viewer.
 - Example: htop
28. **kill** - Terminate a process by ID.
 - kill 1234 - Ends process with PID 1234.
 - Example: kill 1234
29. **killall** - Kill processes by name.
 - killall firefox - Kills all "firefox" processes.
 - Example: killall nginx
30. **pkill** - Kill processes by pattern.
 - pkill -u user - Kills all processes for "user".
 - Example: pkill -u john

Networking

31. **ping** - Check network connectivity.
 - ping google.com - Sends ping requests.
 - Example: ping -c 4 google.com
32. **ifconfig** - Display network configuration (deprecated).
 - ifconfig - Shows network interfaces.
 - Example: ifconfig
33. **traceroute** - Track packet route to destination.
 - traceroute google.com - Shows route to "google.com".
 - Example: traceroute google.com
34. **nslookup** - Query DNS records.
 - nslookup google.com - DNS lookup for "google.com".
 - Example: nslookup example.com
35. **netstat** - Network connections, routing tables.
 - netstat -tuln - Lists open ports.
 - Example: netstat -an

Compression and Archiving

- 37. **tar** - Archive multiple files.
 - tar -cvf archive.tar Directory - Creates a tar archive.
 - Example: tar -xvf archive.tar
- 38. **zip** - Compress files into a zip archive.
 - zip archive.zip file1 file2 - Creates "archive.zip".
 - Example: zip -r archive.zip Directory
- 39. **unzip** - Extract files from a zip archive.
 - unzip archive.zip - Extracts "archive.zip".
 - Example: unzip archive.zip
- 40. **gzip** - Compress a file.
 - gzip file - Compresses "file" as "file.gz".
 - Example: gzip -d file.gz
- 41. **bzip2** - Compress with bzip2.
 - bzip2 file - Compresses "file".
 - Example: bzip2 -d file.bz2

System Information

- 42. **uname** - Display system information.
 - `uname -a` - Full system details.
 - Example: `uname -r`
- 43. **uptime** - Show system uptime.
 - `uptime` - Outputs how long system has been up.
 - Example: `uptime`
- 44. **free** - Memory usage.
 - `free -h` - Shows memory in human-readable form.
 - Example: `free`
- 45. **dmesg** - Print or control kernel messages.
 - `dmesg | less` - Shows boot logs.
 - Example: `dmesg | grep error`
- 46. **df** - Disk space usage of filesystems.
 - `df -h` - Human-readable format.
 - Example: `df /home`

User Management

- 47. **useradd** - Create a new user.
 - useradd username - Adds "username".
 - Example: useradd admin
- 48. **usermod** - Modify user accounts.
 - usermod -aG sudo user - Adds "user" to "sudo" group.
 - Example: usermod -aG docker user
- 49. **passwd** - Change user password.
 - passwd user - Changes password for "user".
 - Example: passwd root
- 50. **whoami** - Display the current logged-in user.
 - whoami - Shows username.
 - Example: whoami

Data Manipulation

51. **awk** - Pattern scanning and processing.
 - awk '{print \$1}' file - Prints first column.
 - Example: awk '{print \$2}' file
52. **sed** - Stream editor for modifying files.
 - sed 's/old/new/' file - Replace text.
 - Example: sed -i 's/foo/bar/g' file
53. **sort** - Sort file content.
 - sort file - Sorts file lines.
 - Example: sort file
54. **uniq** - Report or omit repeated lines.
 - uniq file - Removes duplicates.
 - Example: uniq -c file
55. **grep** - Search within files.
 - grep "pattern" file - Searches for "pattern".
 - Example: grep -i error file

File and Directory Management (Continued)

56. **wc** - Count lines, words, and characters in a file.
- **Usage:** wc file - Displays line, word, and character counts for "file".
 - **Example:** wc -l file - Counts lines in "file".
57. **ln** - Create links between files.
- **Usage:** ln file link_name - Creates a hard link.
 - ln -s file link_name - Creates a symbolic (soft) link.
 - **Example:** ln -s /path/to/file symlink_name
58. **alias** - Create command shortcuts.
- **Usage:** alias ll='ls -la' - Creates an alias for ls -la.
 - **Example:** alias dir='ls -d */'
59. **unalias** - Remove an alias.
- **Usage:** unalias ll - Removes the alias "ll".
 - **Example:** unalias dir
60. **rename** - Rename multiple files.
- **Usage:** rename 's/old/new/' *.txt - Renames files by replacing "old" with "new" in .txt files.
 - **Example:** rename 's/2023/2024/' *.log

Process Management (Continued)

61. **bg** - Run a job in the background.
- **Usage:** bg - Resumes a paused job in the background.
 - **Example:** bg %1 - Backgrounds job number 1.
62. **fg** - Bring a job to the foreground.
- **Usage:** fg %1 - Brings job number 1 to the foreground.
 - **Example:** fg %2
63. **jobs** - List all active jobs.
- **Usage:** jobs - Displays active jobs with their statuses.
 - **Example:** jobs -l
64. **nice** - Run a command with adjusted priority.
- **Usage:** nice -n 10 command - Runs a command with priority 10.
 - **Example:** nice -n 15 ./script.sh
65. **renice** - Change the priority of a running process.
- **Usage:** renice -n 5 -p PID - Changes the priority of process "PID".
 - **Example:** renice -n 10 -p 1234

66. **nohup** - Run a command immune to hangups.

- **Usage:** nohup command & - Executes command, ignoring hangups.
- **Example:** nohup ./script.sh &

67. **pgrep** - Find processes by name.

- **Usage:** pgrep process_name - Finds process IDs with the specified name.
- **Example:** pgrep nginx

68. **pmap** - Display memory map of a process.

- **Usage:** pmap PID - Shows memory details of process "PID".
- **Example:** pmap 1234

69. **uptime** - Display system uptime and load.

- **Usage:** uptime - Shows system uptime and average load.
- **Example:** uptime

Networking (Continued)

70. **curl** - Transfer data to/from a server.

- **Usage:** curl URL - Fetches data from URL.
- **Example:** curl -O https://example.com/file.txt

71. **wget** - Download files from the internet.

- **Usage:** wget URL - Downloads file from the specified URL.
- **Example:** wget https://example.com/file.zip

72. **scp** - Securely copy files between hosts.

- **Usage:** scp source_file user@remote:/path - Copies file to a remote server.
- **Example:** scp file.txt user@remote-server:/home/user

73. **rsync** - Synchronize files and directories.

- **Usage:** rsync -av source/ destination/ - Syncs source to destination.
- **Example:** rsync -avz Directory user@remote:/backup/

74. **telnet** - Connect to remote servers (insecure).

- **Usage:** telnet host port - Connects to a specified host and port.
- **Example:** telnet localhost 80

75. **netcat (nc)** - Network utility for debugging.

- **Usage:** nc -zv host port - Checks if port is open on the host.
- **Example:** nc -zv google.com 443

76. **ip route** - Display or manipulate IP routing table.

- **Usage:** ip route show - Shows the routing table.
- **Example:** ip route add 192.168.1.0/24 dev eth0

77. **dig** - DNS lookup tool.

- **Usage:** dig domain.com - Fetches DNS records for a domain.
- **Example:** dig +short google.com

78. **arp** - Display or manipulate ARP cache.

- **Usage:** arp -a - Lists the ARP table.
- **Example:** arp -d IP_ADDRESS

Text Processing

79. **echo** - Display a line of text.

- **Usage:** echo "Hello World" - Outputs text to the screen.
- **Example:** echo \$PATH

80. **tr** - Translate characters.

- **Usage:** echo "text" | tr 'a-z' 'A-Z' - Converts lowercase to uppercase.
- **Example:** echo "hello" | tr 'a-z' 'A-Z'

81. **cut** - Remove sections from lines.

- **Usage:** cut -d ' ' -f 1 file - Displays the first field from each line.
- **Example:** cut -d: -f1 /etc/passwd

82. **paste** - Merge lines of files.

- **Usage:** paste file1 file2 - Joins lines from two files.
- **Example:** paste file1 file2

83. **xargs** - Execute commands from standard input.

- **Usage:** cat file | xargs echo - Passes arguments to echo.
- **Example:** find . -name "*.log" | xargs rm

84. **nl** - Number lines in a file.

- **Usage:** nl file - Numbers each line.
- **Example:** nl file

85. **sort** - Sort lines in a file.

- **Usage:** sort file - Sorts lines alphabetically.
- **Example:** sort -n file (numerical sort)

86. **uniq** - Filter out repeated lines.

- **Usage:** uniq file - Removes duplicate lines.
- **Example:** sort file | uniq

87. **tee** - Read from input and write to files.

- **Usage:** command | tee file - Writes output to file and displays it.
- **Example:** ls | tee file

Scheduling and Automation

88. **cron** - Schedule regular tasks.

- **Usage:** `crontab -e` - Opens the cron editor.
- **Example:** `0 5 * * * /path/to/script.sh` (Runs script daily at 5 AM)

89. **at** - Schedule one-time tasks.

- **Usage:** `echo "command" | at time` - Executes command at specified time.
- **Example:** `echo "backup.sh" | at 2:00 AM`

System Monitoring and Information

90. **vmstat** - Reports virtual memory statistics.

- **Usage:** vmstat - Shows memory usage.
- **Example:** vmstat 5 (updates every 5 seconds)

91. **iostat** - Reports CPU and I/O statistics.

- **Usage:** iostat - Shows CPU and disk I/O stats.
- **Example:** iostat -d 2

92. **lsof** - List open files.

- **Usage:** lsof - Shows open files and processes.
- **Example:** lsof | grep file

93. **ss** - Display socket statistics.

- **Usage:** ss -tuln - Shows listening ports and connections.
- **Example:** ss -s

94. **free** - Show memory usage.

- **Usage:** free -h - Memory stats in human-readable format.
- **Example:** free

95. **mpstat** - Display CPU usage.

- **Usage:** mpstat - Shows per-processor CPU usage.
- **Example:** mpstat -P ALL 1

96. **sar** - Collect and report system activity.

- **Usage:** `sar -u 1 5` - CPU usage every second for 5 times.
- **Example:** `sar -r` (memory stats)

Security and Access Control

97. **iptables** - Set up firewall rules.

- **Usage:** iptables -L - Lists rules.
- **Example:** iptables -A INPUT -p tcp --dport 22 -j ACCEPT

98. **chmod** - Change file permissions.

- **Usage:** chmod 755 file - Sets permissions.
- **Example:** chmod +x script.sh

99. **chown** - Change file ownership.

- **Usage:** chown user:group file - Changes ownership.
- **Example:** chown user file

100. **passwd** - Change user password. - **Usage:**

passwd user - Sets a password for "user". -

Example: passwd (for current user)

Happy Learning





Networking on linux

1. Getting Info About the Network Interfaces (`ifconfig` , `ip` , `route`)

`ifconfig` , `ip` , and `route` are essential commands for managing network interfaces in Linux. `ifconfig` displays network interface details, while `ip` is a more modern tool for configuring IP addresses and routes. The `route` command shows and modifies the IP routing table, helping manage network traffic paths.

a. Displaying Interface Information

- Using `ifconfig`
 - `ifconfig` : Display information about enabled interfaces.
 - `ifconfig -a` , `ip address show` : Display information about all interfaces (enabled and disabled).
 - `ifconfig enp0s3` , `ip addr show dev enp0s3` : Display info about a specific interface.
- Using `ip`
 - `ip -4 address` : Show only IPv4 info.
 - `ip -6 address` : Show only IPv6 info.
 - `ip link show` : Display L2 info including MAC address.
 - `ip link show dev enp0s3` : Display L2 info for a specific interface.
- Using `route`
 - `route` : Display the default gateway.
 - `route -n` : Display numerical addresses for the default gateway.
 - `ip route show` : Display the routing table.
 - `systemd-resolve --status` : Display the DNS servers.

2. Setting the Network Interfaces

a. Disabling an interface

- `ifconfig enp0s3 down`
- `ip link set enp0s3 down`

b. Activating an interface.

- `ifconfig enp0s3 up`
- `ip link set enp0s3 up`

c. Checking Interface Status

- `ifconfig -a`
- `ip link show dev enp0s3`

d. Setting an IP Address on an Interface

- `ifconfig enp0s3 192.168.0.222/24 up`
- `ip address del 192.168.0.111/24 dev enp0s3`
- `ip address add 192.168.0.112/24 dev enp0s3`

e. Setting a Secondary IP Address on a Sub-Interface

- `ifconfig enp0s3:1 10.0.0.1/24`

f. Deleting and Setting a New Default Gateway using `route`

- Using `route`
 - `route del default gw 192.168.0.1`
 - `route add default gw 192.168.0.2`

- Using `ip`
 - `ip route del default`
 - `ip route add default via 192.168.0.1`

g. Changing the MAC Address

- Using `ifconfig`
 - `ifconfig enp0s3 down`
 - `ifconfig enp0s3 hw ether 08:00:27:51:05:a1`

- `ifconfig enp0s3 up`
- Using `ip`
 - `ip link set dev enp0s3 address 08:00:27:51:05:a3`

3. Network Static Configuration Using netplan (Ubuntu)

a. Stopping and Disabling the Network Manager

- `sudo systemctl stop NetworkManager` : Stop the Network Manager.
- `sudo systemctl disable NetworkManager` : Disable the Network Manager.
- `sudo systemctl status NetworkManager` : Check the status of the Network Manager.
- `sudo systemctl is-enabled NetworkManager` : Verify if the Network Manager is disabled.

b. Creating a YAML Configuration File in `/etc/netplan`

YAML Configuration Example:

```
yamlCopy code
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.0.20/24
      gateway4: "192.168.0.1"
      nameservers:
        addresses:
          - "8.8.8.8"
          - "8.8.4.4"
```

c. Applying and Checking the New Configuration

- `sudo netplan apply` : Apply the new Netplan configuration.
- `ifconfig` : Display network interface configuration.
- `route -a` : Display the routing table.

4. OpenSSH (Secure Shell)

a. Usage

- Secure Remote Management of Servers, Routers, other Networking Devices
- Network File Copy: rsync, scp, sftp, winscp
- Tunneling, SSH Port Forwarding
- **sshd** is the **SSH server (daemon)** and **ssh** or **putty** is the client.

b. Installation of OpenSSH (client and server)

- **Ubuntu:** `sudo apt update && sudo apt install openssh-server openssh-client`
- **CentOS:** `sudo dnf install openssh-server openssh-clients`
- `sudo systemctl status ssh` : Checking the status of the SSH service.
- `sudo systemctl [start|restart|stop] ssh` : Start, restart, or stop the SSH service.
- `sudo systemctl [enable|disable] ssh` : Enable or disable SSH auto-start on boot.
- `/etc/ssh/sshd_config` : Server configuration file.
- `/etc/ssh/ssh_config` : Client configuration file.

c. Controlling the SSHd Daemon

```
ssh -p 22 username@server_ip      # Connect using default S
ssh -p 22 -l username server_ip   # Connect with a specific
ssh -v -p 22 username@server_ip   # Connect in verbose mode

# Ubuntu
sudo systemctl status ssh         # Check SSH status
```

```

sudo systemctl stop ssh      # Stop SSH service
sudo systemctl restart ssh   # Restart SSH service
sudo systemctl enable ssh    # Enable SSH to start on boot
sudo systemctl is-enabled ssh # Check if SSH is enabled on boot

# CentOS
sudo systemctl status sshd    # Check SSH status
sudo systemctl stop sshd     # Stop SSH service
sudo systemctl restart sshd   # Restart SSH service
sudo systemctl enable sshd    # Enable SSH to start on boot
sudo systemctl is-enabled sshd # Check if SSH is enabled on boot

```

d. Securing and Hardening the SSHd daemon

- `man sshd_config` : Manpage of sshd_config for detailed configuration options.
- Change the configuration file (**/etc/ssh/sshd_config**) like the following and then restart the server
 1. **Change the Port:** `Port 2278`
 2. **Disable Direct Root Login:** `PermitRootLogin no`
 3. **Limit Users' SSH Access:** `AllowUsers stud u1 u2 john`
 4. **Activate Public Key Authentication:** `PubkeyAuthentication yes`
 5. **Disable Password Authentication:** `PasswordAuthentication no`
 6. **Use Only SSH Protocol Version 2:** `Protocol 2`
 7. **Other configurations:**
 - a. `ClientAliveInterval 300`
 - b. `ClientAliveCountMax 0`
 - c. `MaxAuthTries 2`
 - d. `MaxStartups 3`
 - e. `LoginGraceTime 20`

5. Copying Files using SCP and RSYNC

`scp` (Secure Copy) and `rsync` are powerful command-line tools used for transferring files between local and remote systems securely. `scp` uses SSH to copy files with encryption, ensuring data security during transfer. It's simple but doesn't handle partial transfers efficiently. `rsync`, on the other hand, is more versatile, offering features like incremental file transfer, bandwidth control, and file synchronization. It only transfers the changed parts of files, making it faster and more efficient for syncing large directories or performing regular backups.

SCP

- `scp a.txt john@80.0.0.1:~` : Copy a local file to a remote destination.
- `scp -P 2288 a.txt john@80.0.0.1:~` : Copy a local file to a remote destination using a custom port.
- `scp -P 2290 john@80.0.0.1:~/a.txt .` : Copy a file from a remote destination to the current directory.
- `scp -P 2290 -r projects/ john@80.0.0.1:~` : Copy a local directory to a remote destination using the `r` option for recursion.

RSYNC

- `sudo rsync -av /etc/ ~/etc-backup/` : Synchronize a directory.
- `sudo rsync -av --delete /etc/ ~/etc-backup/` : Mirror a directory, deleting files from the destination that no longer exist in the source.
- `rsync -av --exclude-from='~/exclude.txt' source_directory/ destination_directory/` : Exclude files listed in the `exclude.txt` file during synchronization.
 - **exclude.txt** example:
 - `.avi`
 - `music/`
 - `abc.mkv`
- `rsync -av --exclude='*.mkv' --exclude='movie1.avi' source_directory/ destination_directory/` : Exclude specific files from synchronization.
- `sudo rsync -av -e ssh /etc/ student@192.168.0.108:~/etc-backup/` : Synchronize a directory over the network using SSH.
- `sudo rsync -av -e 'ssh -p 2267' /etc/ student@192.168.0.108:~/etc-backup/` : Synchronize a directory over the network using SSH with a custom port.

6. WGET - File Downloading Tool

`wget` is a command-line utility used for downloading files from the web. It supports HTTP, HTTPS, and FTP protocols, making it a versatile tool for retrieving content from remote servers. One of its key features is the ability to resume interrupted downloads, ensuring reliable file transfers. Additionally, `wget` can download entire websites for offline viewing, making it a powerful tool for managing downloads and automating data retrieval in scripts.

a. Installing Wget

- `apt install wget` : Install wget on Ubuntu.
- `dnf install wget` : Install wget on CentOS.

b. Downloading Files

- `wget https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Download a file to the current directory.
- `wget -c https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Resume a previously stopped download.
- `mkdir kali` : Create a directory named **kali**.
- `wget -P kali/ https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Save the file into the **kali** directory.

c. Advanced Download Options

- `wget --limit-rate=100k -P kali/ https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Limit download speed to 100kB per second while saving to the **kali** directory.
- `wget -i urls.txt` : Download multiple files from URLs listed in **urls.txt**.

d. Background Downloads

- `wget -b -P kali/ https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Start the download in the background.
- `tail -f wget-log` : Monitor the download log to check its status.

e. Downloading Websites

- `wget --mirror --convert-links --adjust-extension --page-requisites --no-parent http://example.org` : Get a full offline copy of a website.
 - `wget -mkEpng http://example.org` : An alternative command to mirror a website for offline viewing.
-

7. NETSTAT and SS

`netstat` is a command-line tool used for displaying network connections, routing tables, interface statistics, and more. It helps in monitoring network activity, troubleshooting issues, and seeing which ports are open or being used by different processes. However, it is gradually being replaced by `ss` due to its faster performance and more detailed output.

`ss` (Socket Statistics) is a more modern and efficient utility than `netstat` for displaying socket-related information. It provides detailed statistics on active connections, showing TCP, UDP, and Unix sockets. It is faster, consumes fewer resources, and offers advanced filtering options to narrow down the network data you need to analyze.

a. Displaying Open Ports and Connections

- `sudo netstat -tupan` : Display all open ports and connections.
- `sudo ss -tupan` : Display open ports and connections using `ss`.
- `netstat -tupan | grep :80` : Check if port 80 is open.

8. LSOF (List Open Files)

a. Listing Open Files

- `lsof` : List all open files on the system.
 - `lsof -u username` : List all files opened by processes of a specific user.
 - `lsof -c sshd` : List all files opened by a specific process (e.g., `sshd`).
 - `lsof -iTCP -sTCP:LISTEN` : List all open TCP ports.
 - `lsof -iTCP -sTCP:LISTEN -nP` : List open TCP ports, showing numeric addresses without resolving hostnames.
-

9. Scanning Hosts and Networks using Nmap

`nmap` (Network Mapper) is a powerful open-source tool used for network discovery and security auditing. It is widely used by network administrators and penetration testers to identify hosts and services on a network, detect open ports, and determine potential vulnerabilities. With various scanning techniques, `nmap` can detect operating systems, identify running services, and create a comprehensive network map, making it an essential tool for network security and troubleshooting. Only scan your own hosts and servers. Scanning networks is your responsibility.

a. Basic Scans

- `nmap -sS 192.168.0.1` : Perform a SYN scan (half-open scan, requires root).
- `nmap -sT 192.168.0.1` : Perform a TCP connect scan.

b. Port Scans

- `nmap -p- 192.168.0.1` : Scan all ports (0-65535).
- `nmap -p 20,22-100,443,1000-2000 192.168.0.1` : Scan specific ports (20, 22-100, 443, 1000-2000).

c. Version and OS Detection

- `nmap -p 22,80 -sV 192.168.0.1` : Detect services and versions.
- `nmap -O 192.168.0.1` : Perform OS detection.
- `nmap -A 192.168.0.1` : Enable OS detection, version detection, script scanning, and traceroute.

d. Network and Host Scans

- `nmap -sP 192.168.0.0/24` : Perform a ping scan on an entire network.
- `nmap -Pn 192.168.0.0/24` : Treat all hosts as online and skip host discovery.
- `nmap -sS 192.168.0.0/24 --exclude 192.168.0.10` : Exclude a specific IP from the scan.

e. Saving Output and Target Lists

- `nmap -oN output.txt 192.168.0.1` : Save scan results to a file.
- `nmap -p 80 -iL hosts.txt` : Read target IPs from a file and scan port 80.
- `nmap -n -iL hosts.txt -p 80 -oN output.txt` : Scan from a list of hosts, disable reverse DNS, and save output to a file.

100 Most Important Linux Commands For DevOps Engineers

File System Management:

1. **ls**: List directory contents.

```
ls
```

Example Output:

```
file1.txt  file2.txt  directory1  directory2
```

2. **cd**: Change directory.

```
cd directory_name
```

Example Output: (No Output, just changes directory)

3. **pwd**: Print working directory.

```
pwd
```

Example Output:

```
/home/user/directory_name
```

4. **mkdir**: Make directory.

```
mkdir directory_name
```

Example Output: (No Output, creates directory)

5. **rm**: Remove files or directories.

```
rm file_name
```

Example Output: (No Output, removes file)

6. **cp**: Copy files or directories.

```
cp source_file destination_file
```

Example Output: (No Output, copies file)

7. **mv**: Move or rename files or directories.

```
mv old_file_name new_file_name
```

Example Output: (No Output, renames file)

8. **touch**: Create an empty file.

```
touch file_name
```

Example Output: (No Output, creates empty file)

9. **cat**: Concatenate and display files.

```
cat file_name
```

Example Output:

```
Contents of file
```

10. **less/more**: View file contents page by page.

```
less file_name
```

Example Output:

```
Contents of file
```

File Manipulation:

11. **grep**: Search for a pattern in files.

```
grep pattern file_name
```

Example Output:

```
Matching lines containing pattern
```

12. **find**: Search for files in a directory hierarchy.

```
find directory_name -name file_name
```

Example Output:

```
/directory_name/file_name
```

13. **chmod**: Change file permissions.

```
chmod permissions file_name
```

Example Output: (No Output, changes file permissions)

14. **chown**: Change file owner and group.

```
chown user:group file_name
```

Example Output: (No Output, changes file owner and group)

Archive Management:

15. **tar**: Manipulate archive files.

```
tar -czvf archive_name.tar.gz directory_to_compress
```

Example Output: (No Output, creates tar.gz archive)

16. **gzip/gunzip**: Compress or decompress files.

```
gzip file_name
```

Example Output: (No Output, compresses file)

17. **zip/unzip**: Compress or decompress zip files.

```
zip -r archive_name.zip directory_to_compress
```

Example Output: (No Output, creates zip archive)

Process Management:

18. **ps**: Display information about running processes.

```
ps
```

Example Output:

PID	TTY	TIME	CMD
123	pts/1	00:00:00	bash

19. **kill**: Terminate processes.

```
kill process_id
```

Example Output: (No Output, terminates process)

20. **top**: Display and update sorted information about processes.

```
top
```

Example Output:

```
Tasks: 100 total, 1 running, 99 sleeping
```

Network Management:

21. **ifconfig/ip**: Display or configure network interface parameters.

```
ifconfig
```

Example Output:

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
```

22. **ping**: Test a network connection.

```
ping host_name
```

Example Output:

```
PING host_name (192.168.1.1) 56(84) bytes of data.
```

23. **ssh**: Securely connect to a remote server.

```
ssh user@host
```

Example Output: (No Output, connects to remote server)

24. **scp**: Securely copy files between hosts.

```
scp file user@host:/path
```

Example Output: (No Output, copies file to remote host)

System Information:

25. **df**: Display disk space usage.

```
df
```

Example Output:

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
/dev/sda1	12345678	98765	11223344	10%	/

26. **du**: Estimate file space usage.

```
du
```

Example Output:

```
12345678 .
```

27. **uname**: Print system information.

```
uname -a
```

Example Output:

```
Linux hostname 4.4.0-18362-Microsoft #1-Microsoft
```

28. **uptime**: Show how long the system has been running.

```
uptime
```

Example Output:

```
12:34:56 up 10 days, 2:30, 1 user, load average: 0.00, 0.01, 0.05
```

Users and Permissions:

29. **passwd**: Change user password.

```
passwd
```

Example Output: (No Output, changes user password)

30. **useradd**: Create a new user.

```
useradd username
```

Example Output: (No Output, creates new user)

31. **userdel**: Delete a user account.

```
userdel username
```

Example Output: (No Output, deletes user account)

32. **usermod**: Modify a user account.

```
usermod -aG group_name username
```

Example Output: (No Output, modifies user account)

33. **groupadd**: Create a new group.

```
groupadd group_name
```

Example Output: (No Output, creates new group)

34. **groupdel**: Delete a group.

```
groupdel group_name
```

Example Output: (No Output, deletes group)

Package Management:

35. **yum/dnf**: Package manager for RPM-based Linux distributions.

```
yum install package_name
```

Example Output: (No Output, installs package)

36. **apt/apt-get**: Package manager for Debian-based Linux distributions.

```
apt-get install package_name
```

Example Output: (No Output, installs package)

37. **rpm**: Package manager for RPM-based

Linux distributions. `bash rpm -i package.rpm`

Example Output: (No Output, installs package)

38. **dpkg**: Package manager for Debian-based Linux distributions.

```
dpkg -i package.deb
```

Example Output: (No Output, installs package)

System Maintenance:

39. **cron**: Schedule tasks to run periodically.

```
crontab -e
```

Example Output: (No Output, opens cron file for editing)

40. **at**: Schedule a one-time task.

```
at now + 1 hour
```

Example Output: (No Output, schedules task)

System Control:

41. **systemctl**: Control the systemd system and service manager.

```
systemctl start|stop|restart service_name
```

Example Output: (No Output, starts/stops/restarts service)

42. **journalctl**: Query and display messages from the journal.

```
journalctl
```

Example Output: (No Output, displays system journal)

Miscellaneous:

43. **hostname**: Display or set the system's hostname.

```
hostname
```

Example Output:

```
Hostname
```

44. **whoami**: Print the current user.

```
whoami
```

Example Output:

```
Username
```

Monitoring and Performance:

45. **top**: Display and update sorted information about processes.

```
top
```

Example Output:

```
top - 12:34:56 up 10 days, 2:30, 1 user, load average: 0.00, 0.01, 0.05
Tasks: 100 total, 1 running, 99 sleeping
```

46. **htop**: Interactive process viewer.

```
htop
```

Example Output:

```
CPU%  MEM%
10.2   5.0
```

47. **vmstat**: Report virtual memory statistics.

```
vmstat
```

Example Output:

```
procs -----memory----- ---swap-- -----io----- -system-- ---
---cpu-----
 r  b   swpd   free   buff   cache   si   so   bi   bo   in   cs us
sy id wa st
 1  0       0 123456  78901 2345678    0    0    0    0    0    0  0
0  0  0  0
```

48. **sar**: Collect, report, or save system activity information.

```
sar
```

Example Output:

```
12:00:00 PM   CPU   %user   %nice   %system   %iowait   %steal   %idle
```

49. **iostat**: Report CPU and I/O statistics.

```
iostat
```

Example Output:

```
avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.12    0.04    0.21    0.10    0.00   99.53
```

Text Processing:

50. **sed**: Stream editor for filtering and transforming text.

```
sed 's/old_string/new_string/g' file_name
```

Example Output:

```
Modified contents of file
```

51. **awk**: A versatile programming language for working on files.

```
awk '{print $1}' file_name
```

Example Output:

```
Column_1_Data
```

52. **grep**: Search for a pattern in files.

```
grep pattern file_name
```

Example Output:

```
Matching lines containing pattern
```

53. **sort**: Sort lines of text files.

```
sort file_name
```

Example Output:

```
Sorted lines
```

Input/Output:

54. **tee**: Redirect output to multiple files or displays.

```
command | tee file1 file2
```

Example Output: (No Output, redirects output to files)

55. **tail**: Output the last part of files.

```
tail file_name
```

Example Output:

```
Last 10 lines of file
```

56. **head**: Output the first part of files.

```
head file_name
```

Example Output:

```
First 10 lines of file
```

57. **less/more**: View file contents page by page.

```
less file_name
```

Example Output:

```
Contents of file
```

Compression and Archiving:

58. **gzip/gunzip**: Compress or decompress files.

```
gzip file_name
```

Example Output: (No Output, compresses file)

59. **zip/unzip**: Compress or decompress zip files.

```
zip -r archive_name.zip directory_to_compress
```

Example Output: (No Output, creates zip archive)

60. **tar**: Manipulate archive files.

```
tar -czvf archive_name.tar.gz directory_to_compress
```

Example Output: (No Output, creates tar.gz archive)

System Information:

61. **df**: Display disk space usage.

```
df
```

Example Output:

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
/dev/sda1	12345678	98765	11223344	10%	/

62. **free**: Display amount of free and used memory in the system.

```
free -m
```

Example Output:

total	used	free	shared	buffers	cached
Mem:	1234	567	890	123	456
789					

63. **ps**: Display information about running processes.

```
ps
```

Example Output:

PID	TTY	TIME	CMD
123	pts/1	00:00:00	bash

Networking:

64. **netstat**: Display network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.

```
netstat
```

Example Output:

```
Active Internet connections
```

65. **lsof**: List open files.

```
lsof
```

Example Output:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
---------	-----	------	----	------	--------	----------	------	------

66. **ping**: Test a network connection.

```
ping host_name
```

Example Output:

```
PING host_name (192.168.1.1) 56(84) bytes of data.
```

67. **ssh**: Securely connect to a remote server.

```
ssh user@host
```

Example Output: (No Output, connects to remote server)

68. **scp**: Securely copy files between hosts.

```
scp file user@host:/path
```

Example Output: (No Output, copies file to remote host)

69. **wget**: Retrieve files from the internet via HTTP, HTTPS, or FTP.

```
wget URL
```

Example Output: (No Output, retrieves file)

70. **curl**: Transfer data from or to a server.

```
curl URL
```

Example Output: (No Output, transfers data)

Process Management:

71. **kill**: Terminate processes.

```
kill process_id
```

Example Output: (No Output, terminates process)

User and Group Management:

72. **groups**: Print group memberships for a user.

```
groups username
```

Example Output:

```
username : group1 group2
```

System Information:

73. **hostname**: Display or set the system's hostname.

```
hostname
```

Example Output:

```
Hostname
```

74. **whoami**: Print the current user.

```
whoami
```

Example Output:

```
Username
```

75. **uptime**: Show how long the system has been running.

```
uptime
```

Example Output:

```
12:34:56 up 10 days, 2:30, 1 user, load average: 0.00, 0.01, 0.05
```

76. **date**: Display or set the system date and time.

```
date
```

Example Output:

```
Mon Apr  4 12:34:56 UTC 2024
```

77. **uname**: Print system information.

```
uname -a
```

Example Output:

```
Linux hostname 4.4.0-18362-Microsoft #1-Microsoft
```

78. **lsblk**: List block devices.

```
lsblk
```

Example Output:

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	20G	0	disk	
└─sda1	8:1	0	20G	0	part	/

79. **df**: Display disk space usage.

```
df -h
```

Example Output:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	20G	9.8G	9.2G	52%	/

80. **free**: Display amount of free and used memory in the system.

```
free -m
```

Example Output:

	total	used	free	shared	buff/cache
available					
Mem:	2002	1054	346	199	602
648					

81. **ps**: Display information about running processes.

```
ps
```

Example Output:

PID	TTY	TIME	CMD
123	pts/1	00:00:00	bash

82. **lsof**: List open files.

```
lsof
```

Example Output:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
---------	-----	------	----	------	--------	----------	------	------

83. **netstat**: Display network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.

```
netstat
```

Example Output:

```
Active Internet connections
```

System Control:

84. **systemctl**: Control the systemd system and service manager.

```
systemctl start|stop|restart service_name
```

Example Output: (No Output, starts/stops/restarts service)

85. **journalctl**: Query and display messages from the journal.

```
journalctl
```

Example Output: (No Output, displays system journal)

Package Management:

86. **yum/dnf**: Package manager for RPM-based Linux distributions.

```
yum install package_name
```

Example Output: (No Output, installs package)

87. **apt/apt-get**: Package manager for Debian-based Linux distributions.

```
apt-get install package_name
```

Example Output: (No Output, installs package)

88. **rpm**: Package manager for RPM-based Linux distributions.

```
rpm -i package.rpm
```

Example Output: (No Output, installs package)

89. **dpkg**: Package manager for Debian-based Linux distributions.

```
dpkg -i package.deb
```

Example Output: (No Output, installs package)

System Maintenance:

90. **cron:** Schedule tasks to run periodically.

```
crontab -e
```

Example Output: (No Output, opens cron file for editing)

91. **at:** Schedule a one-time task.

```
at now + 1 hour
```

Example Output: (No Output, schedules task)

Miscellaneous:

92. **su:** Run a command with substitute user and group ID.

```
su - username
```

Example Output: (No Output, switches user)

93. **sudo:** Execute a command as another user.

```
sudo command
```

Example Output: (No Output, executes command as another user)

94. **chmod:** Change file mode bits.

```
chmod 755 file_name
```

Example Output: (No Output, changes file permissions)

95. **chown:** Change file owner and group.

```
chown user:group file_name
```

Example Output: (No Output, changes file owner and group)

96. **passwd**: Change user password.

```
passwd
```

Example Output: (No Output, changes user password)

97. **useradd**: Create a new user or update default new user information.

```
useradd username
```

Example Output: (No Output, creates new user)

98. **userdel**: Delete a user account and related files.

```
userdel username
```

Example Output: (No Output, deletes user account)

99. **usermod**: Modify a user account.

```
usermod -aG group_name username
```

Example Output: (No Output, modifies user account)

100. **groupadd**: Create a new group. `bash groupadd group_name` Example Output: (No Output, creates new group)

1. User and Group Management

- **useradd** - Add a new user
- **usermod** - Modify user account
- **userdel** - Delete a user
- **groupadd** - Add a new group
- **groupdel** - Delete a group
- **chgrp** - Change group ownership of a file
- **passwd** - Change user password
- **Who** - Who is logged in

2. File and Directory Management

- **ls** - List directory contents
- **cd** - Change directory
- **pwd** - Print working directory
- **mkdir** - Make a new directory
- **rmdir** - Remove a directory
- **rm** - Remove files or directories
- **cp** - Copy files or directories
- **mv** - Move or rename files or directories

- **touch** - Create empty files
- **find** - Search for file in a directory hierarchy
- **locate** - Find files by name

3. File Management

- **cat** - Concatenate and display file contents
- **more** - View file content page by page
- **less** - View file content with navigation
- **head** - View the first part of a file
- **tail** - View the last part of a file
- **nano** - Text editor to edit files
- **vi** - Text editor to edit files
- **touch** - Create an empty file or update file timestamp
- **ln** - Create hard or symbolic links
- **file** - Determine file type
- **stat** - Display file or file system status
- **find** - Search for files in a directory hierarchy
- **locate** - Find files by name (using a database)

- **updatedb** - Update the database used by **locate**
- **xargs** - Build and execute command lines from standard input

4. File Permissions and Ownership

- **chmod** - Change file permissions
- **chown** - Change file owner and group
- **chgrp** - Change group ownership
- **umask** - Set file creation mask
- **getfacl** - Get file access control lists (ACL)
- **setfacl** - Set file access control lists (ACL)
- **ls -l** - List files with detailed permissions, owner, and group
- **stat** - Display file or file system status
- **acl** - View or modify Access Control Lists (ACL) on files

5. Disk Management

- **df** - Display free disk space
- **du** - Estimate file space usage
- **fdisk** - Partition table manipulator
- **parted** - A tool for creating and managing disk partitions

- **lsblk** - List information about block devices
- **blkid** - Find or display block device attributes
- **mount** - Mount a file system
- **umount** - Unmount a file system
- **mkfs** - Create a file system
- **fsck** - Check and repair file systems
- **tune2fs** - Adjust tunable file system parameters
- **resize2fs** - Resize an ext2, ext3, or ext4 file system
- **lvcreate** - Create a logical volume
- **vgcreate** - Create a volume group
- **pvccreate** - Prepare a physical volume for LVM
- **lsattr** - List file attributes on a file system
- **chattr** - Change file attributes on a file system

6. Process Management

- **ps** - Display current processes
- **top** - Display dynamic process information
- **htop** - Interactive process viewer

- **kill** - Terminate a process by ID
- **killall** - Kill all processes by name
- **bg** - Resume a suspended process in the background
- **fg** - Bring a process to the foreground
- **nice** - Set process priority
- **renice** - Change the priority of running processes
- **jobs** - List active jobs
- **pgrep** - Search for processes by name or other attributes
- **pkill** - Kill processes by name or other attributes
- **ps tree** - Display processes in a tree format
- **lsof** - List open files and the processes that opened them
- **strace** - Trace system calls and signals
- **timeout** - Run a command with a time limit
- **watch** - Execute a program periodically and show output

7. Network Management

- **ifconfig** - Display and configure network interfaces
- **ip a** - Show all network interfaces and their IPs
- **ip r** - Display the routing table

- `ip link set <interface> up/down` - Enable or disable a network interface
- `ping <host>` - Send ICMP echo request to check connectivity
- `traceroute <host>` - Trace the route packets take to a network host
- `netstat -tulnp` - Show open ports and listening services
- `ss -tulnp` - Display socket statistics and listening services
- `hostname -I` - Show the system's IP address
- `dig <domain>` - Query DNS records
- `nslookup <domain>` - Lookup DNS details
- `whois <domain>` - Get domain registration details
- `curl -I <URL>` - Fetch HTTP headers from a website
- `wget <URL>` - Download files from the web
- `scp <file> user@host:/path` - Securely copy files over SSH
- `rsync -avz <src> <dest>` - Sync files between local and remote locations
- `tcpdump -i eth0` - Capture network packets on an interface
- `nmap <IP>` - Scan a network for open ports

8. System and Monitoring

- `uptime` - Show system uptime and load average

- **dmesg** - Print system boot and hardware logs
- **free -m** - Display memory usage in MB
- **vmstat** - Report system performance statistics
- **iostat** - Display CPU and disk I/O statistics
- **mpstat** - Show CPU usage per core
- **top** - Display dynamic process and system information
- **htop** - Interactive process and resource monitoring
- **sar -u 5 10** - Monitor CPU usage at 5-second intervals
- **iotop** - Display disk read/write usage per process
- **lsof** - List open files by processes
- **watch -n 2 <command>** - Run a command every 2 seconds
- **journalctl -xe** - View system logs with extended details
- **systemctl status <service>** - Check the status of a service
- **service <service> status** - View service status (legacy systems)
- **df -h** - Show disk space usage in a human-readable format
- **du -sh <directory>** - Display the size of a directory
- **ps aux --sort=-%mem** - List processes sorted by memory usage
- **netstat -tulnp** - Show active network connections and listening services

9. Text Processing

- `cat <file>` - Display file contents
- `tac <file>` - Display file contents in reverse order
- `head -n <num> <file>` - Show the first <num> lines of a file
- `tail -n <num> <file>` - Show the last <num> lines of a file
- `tail -f <file>` - Continuously display new lines in a file
- `grep '<pattern>' <file>` - Search for a pattern in a file
- `grep -r '<pattern>' <directory>` - Recursively search for a pattern in a directory
- `awk '{print $1, $2}' <file>` - Process and extract columns from a file
- `sed 's/old/new/g' <file>` - Replace text in a file
- `cut -d':' -f1 <file>` - Extract specific fields from a file
- `sort <file>` - Sort file contents
- `uniq <file>` - Remove duplicate lines from a file
- `wc -l <file>` - Count lines in a file
- `wc -w <file>` - Count words in a file
- `wc -c <file>` - Count bytes in a file
- `tr 'a-z' 'A-Z' <file>` - Convert lowercase to uppercase
- `diff <file1> <file2>` - Compare two files line by line

- `cmp <file1> <file2>` - Compare two files byte by byte
- `tee <file>` - Read from standard input and write to both a file and standard output

10. Package Management

For Debian-based systems (Ubuntu, Debian)

- `apt update` - Update package lists
- `apt upgrade` - Upgrade all installed packages
- `apt install <package>` - Install a package
- `apt remove <package>` - Remove a package
- `apt purge <package>` - Remove a package and its config files
- `dpkg -i <package>.deb` - Install a .deb package
- `dpkg -r <package>` - Remove a package installed via dpkg
- `dpkg -l` - List installed packages
- `apt-cache search <package>` - Search for a package
- `apt autoremove` - Remove unnecessary packages
- `apt clean` - Clear the package cache

For RHEL-based systems (CentOS, Rocky Linux, AlmaLinux, Fedora)

- `yum update` - Update installed packages (for older systems)

- `dnf update` - Update installed packages (for newer systems)
- `yum install <package>` - Install a package
- `dnf install <package>` - Install a package (newer distros)
- `yum remove <package>` - Remove a package
- `dnf remove <package>` - Remove a package (newer distros)
- `rpm -ivh <package>.rpm` - Install an `.rpm` package
- `rpm -e <package>` - Remove an installed `.rpm` package
- `yum list installed` - List installed packages
- `dnf list installed` - List installed packages (newer distros)
- `yum search <package>` - Search for a package
- `dnf search <package>` - Search for a package (newer distros)
- `yum clean all` - Clean cached package data

11. Archive and Compression

Working with tar files

- `tar -cvf archive.tar <files>` - Create a tar archive
- `tar -xvf archive.tar` - Extract a tar archive
- `tar -tvf archive.tar` - List files in a tar archive

- `tar -czvf archive.tar.gz <files>` - Create a compressed tar.gz archive
- `tar -xzvf archive.tar.gz` - Extract a tar.gz archive
- `tar -cJvf archive.tar.xz <files>` - Create a tar.xz archive
- `tar -xJvf archive.tar.xz` - Extract a tar.xz archive

Working with zip files

- `zip archive.zip <files>` - Create a zip archive
- `unzip archive.zip` - Extract a zip archive
- `unzip -l archive.zip` - List contents of a zip file
- `zip -r archive.zip <directory>` - Zip a directory

Working with gzip & bzip2

- `gzip <file>` - Compress a file using gzip
- `gunzip <file>.gz` - Decompress a gzip file
- `bzip2 <file>` - Compress a file using bzip2
- `bunzip2 <file>.bz2` - Decompress a bzip2 file

12. Shutdown and Reboot

- `shutdown -h now` - Shutdown the system immediately
- `shutdown -h +10` - Shutdown the system in 10 minutes

- `shutdown -h 22:00` - Schedule shutdown at 10:00 PM
- `shutdown -c` - Cancel a scheduled shutdown
- `poweroff` - Immediately power off the system
- `halt` - Halt the system
- `reboot` - Reboot the system immediately
- `shutdown -r now` - Reboot the system immediately
- `shutdown -r +5` - Reboot the system in 5 minutes
- `systemctl reboot` - Reboot using systemd

13. Task Scheduling

- `crontab -e` - Edit cron jobs for the current user
- `crontab -l` - List scheduled cron jobs
- `crontab -r` - Remove all cron jobs for the current user
- `crontab -u <user> -l` - List cron jobs for a specific user
- `at <time>` - Schedule a command to run at specified time

14. System Diagnostics and Troubleshooting

- `journalctl -xe` - View detailed system logs
- `journalctl -f` - Follow live logs in real-time

- **journalctl --since "1 hour ago"** - View logs from the last hour
- **dmesg | less** - View system boot logs
- **dmesg | grep error** - Check for errors in system logs
- **cat /var/log/syslog** - View general system logs (Debian-based)
- **cat /var/log/messages** - View general system logs (RHEL-based)
- **uptime** - Show system uptime and load
- **free -m** - Display available and used memory in MB
- **top** - Monitor real-time CPU, memory, and process usage
- **htop** - Interactive system monitoring (if installed)
- **vmstat 1 5** - Display system performance stats
- **iostat** - Show CPU and disk usage statistics

Linux- Common Issue and Solution.

Common issues in Linux and their solutions.

1. **Package Management Issues:**

- **Issue:** Unable to install a package.
- **Solution:** Use the package manager for your distribution (e.g., `apt` for Debian/Ubuntu, `yum` for CentOS) and ensure repositories are properly configured.

2. **Network Configuration:**

- **Issue:** No internet connection.
- **Solution:** Check network cables, run network diagnostic commands (`ifconfig`, `ping`), and verify DNS settings.

3. **Permission Problems:**

- **Issue:** Permission denied while trying to access or modify files.
- **Solution:** Use `chmod` or `chown` to adjust file permissions or ownership. Be cautious not to modify critical system files.

4. **Boot Problems:**

- **Issue:** System not booting properly.
- **Solution:** Use a live CD/USB to access the system, repair the bootloader (e.g., Grub), or fix corrupted system files.

5. **Update Issues:**

- **Issue:** Software updates fail.
- **Solution:** Check repositories, update the package list (`sudo apt update`), and then upgrade packages (`sudo apt upgrade`).

6. **Disk Space Problems:**

- **Issue:** Running out of disk space.
- **Solution:** Identify large files/directories (`du` or `df` commands) and remove unnecessary files. Be careful not to delete critical system files.

7. **Graphical Interface Issues:**

- **Issue:** Desktop environment not loading.
- **Solution:** Check graphics drivers, reinstall desktop environment, or switch to a different one.

8. **Kernel Panic:**

- **Issue:** Kernel panic during boot.
- **Solution:** Boot into a different kernel version, update the kernel, or reinstall the kernel package.

9. ****Authentication Problems:****

- ****Issue:**** Unable to log in.
- ****Solution:**** Verify username and password, check for CAPS LOCK, and reset password if needed.

10. ****Swap Space Issues:****

- ****Issue:**** Insufficient swap space.
- ****Solution:**** Resize or add swap space using ``swapon`` or by modifying the ``/etc/fstab`` file.

11. ****File System Corruption:****

- ****Issue:**** File system errors.
- ****Solution:**** Run file system checks (``fsck``) on unmounted partitions to fix errors.

12. ****Incompatible Libraries:****

- ****Issue:**** Missing or incompatible libraries.
- ****Solution:**** Install required libraries using the package manager.

13. ****Systemd Service Failures:****

- ****Issue:**** Service not starting.
- ****Solution:**** Check service status with ``systemctl status``, review logs, and restart the service.

14. ****Sound Issues:****

- ****Issue:**** No sound or distorted audio.

- **Solution:** Check audio settings, reinstall audio drivers, or update kernel modules.

15. **Bluetooth Connectivity Problems:**

- **Issue:** Bluetooth devices not connecting.
- **Solution:** Restart Bluetooth service, update Bluetooth drivers, and check hardware compatibility.

16. **Firewall Configuration:**

- **Issue:** Unable to access services due to firewall.
- **Solution:** Adjust firewall rules using tools like `ufw` or `firewalld`.

17. **SELinux Issues:**

- **Issue:** SELinux denials.
- **Solution:** Review SELinux logs, adjust SELinux policies, or set SELinux to permissive mode.

18. **Hardware Recognition:**

- **Issue:** Hardware not recognized.
- **Solution:** Update kernel, check hardware compatibility, and install necessary drivers.

19. **X Server Problems:**

- **Issue:** GUI not starting.

- **Solution:** Check X server logs, update graphics drivers, and review display manager settings.

20. **Cron Job Failures:**

- **Issue:** Scheduled tasks not running.
- **Solution:** Review cron job logs, check syntax, and ensure proper permissions.

21. **SSH Connection Issues:**

- **Issue:** Unable to connect via SSH.
- **Solution:** Check SSH configuration, verify firewall rules, and restart SSH service.

22. **Locale Problems:**

- **Issue:** Incorrect date, time, or language settings.
- **Solution:** Set the correct locale using `locale-gen` and update configuration files.

23. **Package Dependency Errors:**

- **Issue:** Unmet dependencies during package installation.
- **Solution:** Use the package manager to resolve dependencies (`sudo apt-f install` for Debian/Ubuntu).

24. ****RAID Configuration Issues:****

- ****Issue:**** RAID array not functioning.
- ****Solution:**** Check RAID configuration, replace faulty drives, and rebuild the array.

25. ****Kernel Module Loading:****

- ****Issue:**** Unable to load kernel modules.
- ****Solution:**** Use ``modprobe`` to manually load modules and update kernel headers if needed.

26. ****System Logs Filling Up:****

- ****Issue:**** Disk space consumed by logs.
- ****Solution:**** Configure log rotation settings and periodically clean up old logs.

27. ****System Freezing:****

- ****Issue:**** System becomes unresponsive.
- ****Solution:**** Check for resource-intensive processes, update kernel, and review hardware health.

28. ****DNS Resolution Problems:****

- ****Issue:**** Unable to resolve domain names.
- ****Solution:**** Check DNS settings in ``/etc/resolv.conf`` and test DNS resolution using tools like ``nslookup``.

29. ****NTP Synchronization Issues:****

- ****Issue:**** Incorrect system time.
- ****Solution:**** Configure NTP to synchronize system time (``sudo timedatectl set-ntp true``).

30. ****System Hanging on Shutdown:****

- ****Issue:**** System doesn't shut down properly.
- ****Solution:**** Investigate systemd issues, update kernel, and check for hardware problems.

31. ****LVM (Logical Volume Management) Issues:****

- ****Issue:**** Problems with logical volumes.
- ****Solution:**** Check volume group and logical volume status, and repair if necessary.

32. ****SMTP Email Sending Issues:****

- ****Issue:**** Unable to send emails.
- ****Solution:**** Review mail server logs, check SMTP configuration, and ensure proper authentication.

33. ****GPU Driver Problems:****

- ****Issue:**** Graphics card driver issues.
- ****Solution:**** Install or update graphics drivers based on the GPU manufacturer.

34. ****Systemd Timer Issues:****

- **Issue:** Scheduled systemd timers not triggering.
- **Solution:** Verify timer unit files, check logs, and restart the timer service.

35. **NFS (Network File System) Mounting Problems:**

- **Issue:** Unable to mount NFS shares.
- **Solution:** Verify NFS server and client configurations, and check network connectivity.

36. **System Resource Exhaustion:**

- **Issue:** High CPU, memory, or disk usage.
- **Solution:** Identify and address resource-intensive processes, optimize system settings, and consider hardware upgrades.

37. **Samba File Sharing Issues:**

- **Issue:** Problems with Samba shares.
- **Solution:** Review Samba configuration, check file permissions, and restart the Samba service.

38. **Slow Boot Times:**

- **Issue:** System takes a long time to boot.
- **Solution:** Analyze boot logs, disable unnecessary services, and update or remove problematic software.

39. ****Incorrect Keyboard Layout:****

- ****Issue:**** Incorrect keyboard mapping.
- ****Solution:**** Set the correct keyboard layout using tools like ``setxkbmap`` or through system settings.

40. ****Stale Lock Files:****

- ****Issue:**** Application won't start due to stale lock files.
- ****Solution:**** Identify and remove lock files, ensuring no active instances of the application are running.

41. ****Incomplete Software Installation:****

- ****Issue:**** Software installation interrupted or incomplete.
- ****Solution:**** Remove incomplete installations, clean up package cache, and reinstall the software.

42. ****Stale Package Dependencies:****

- ****Issue:**** Old or conflicting package dependencies.
- ****Solution:**** Use the package manager to remove obsolete dependencies (``sudo apt autoremove`` for Debian/Ubuntu).

43. ****System Recovery After Power Failure:****

- ****Issue:**** System doesn't recover properly after a power outage.
- ****Solution:**** Configure BIOS settings, use UPS (Uninterruptible Power Supply), and consider graceful shutdown options.

44. ****High System Load:****

- ****Issue:**** System load is consistently high.
- ****Solution:**** Identify resource-hungry processes using tools like `top` or `htop` and address the underlying cause.

45. ****DNS Cache Issues:****

- ****Issue:**** DNS cache causing resolution problems.
- ****Solution:**** Clear DNS cache using appropriate commands (`systemctl restart systemd-resolved` for systemd-based systems).

46. ****Unexpected System Reboots:****

- ****Issue:**** System reboots without warning.
- ****Solution:**** Investigate logs for kernel panics, hardware issues, or software conflicts.

47. System File Corruption:

- Issue: Critical system files corrupted.

- Solution: Restore from backups, reinstall affected packages, or repair the system using a live CD/USB.

48. Issues with External Storage:

- Issue: External drives not recognized.
- Solution: Check cables, use different USB ports, and ensure external drives are properly formatted.

49. Stuck Software Updates:

- Issue: Software updates seem stuck.
- Solution: Verify internet connection, check update logs, and try updating packages individually.

50. Systemd Journal Corruption:

- Issue: Corruption in systemd journal files.
- Solution: Clear or rotate journal files using appropriate commands (`journalctl--rotate`).

51. User Permission Problems:

- Issue: Regular user unable to perform certain tasks.
- Solution: Add the user to appropriate groups, adjust file permissions, or use `sudo` for elevated privileges.

52. Kernel Module Compilation Errors:

- Issue: Compiling kernel modules fails.

- Solution: Install kernel headers, check compiler dependencies, and follow module-specific build instructions.

53. GPU Driver Module Compilation:

- Issue: Compiling GPU driver modules fails.
- Solution: Install necessary development tools and headers, follow GPU driver installation instructions, and ensure compatibility with the kernel.

54. Stuck Print Jobs:

- Issue: Print jobs stuck in the queue.
- Solution: Restart the print service (cups), check for printer connectivity, and clear the print queue.

55. Invalid Shell Configuration:

- Issue: Invalid shell configurations prevent login.
- Solution: Use a rescue mode or live CD to edit shell configuration files (/etc/profile, ~/.bashrc, etc.).

56. PHP Configuration Issues:

- Issue: PHP scripts not executing as expected.
- Solution: Review PHP configuration files, check for syntax errors, and restart the web server.

57. Apache/Nginx Configuration Errors:

- Issue: Web server misconfigurations.

- Solution: Check Apache or Nginx configuration files for errors, and restart the web server.

58. DNS Server Configuration:

- Issue: Incorrect DNS server settings.
- Solution: Verify `/etc/resolv.conf`, check DNS server configurations, and restart the networking service.

59. Systemd Service Masking:

- Issue: Service masked and unable to start.
- Solution: Unmask the service using `systemctl unmask`, then start and enable it.

60. SSH Key Authentication Failures:

- Issue: Unable to authenticate via SSH keys.
- Solution: Check key permissions, ensure correct public key is added to `~/.ssh/authorized_keys`, and review SSH daemon configuration.

61. Slow MySQL/MariaDB Performance:

- Issue: Database server performance issues.
- Solution: Optimize database queries, increase system resources, and fine-tune MySQL/MariaDB configuration.

62. System Font Issues:

- Issue: Font rendering problems.
- Solution: Install necessary fonts, clear font caches, and configure font settings in desktop environments.

63. JVM Heap Issues:

- Issue: Java applications experiencing memory problems.
- Solution: Adjust JVM heap settings, analyze garbage collection logs, and update Java runtime if needed.

64. LDAP Authentication Problems:

- Issue: Issues with LDAP authentication.
- Solution: Verify LDAP server settings, check network connectivity, and review PAM (Pluggable Authentication Modules) configurations.

65. Sudden Network Disconnections:

- Issue: Frequent network disconnections.
- Solution: Check network hardware, update network drivers, and review syslog or journal logs for network-related errors.

66. Network Bonding Issues:

- Issue: Problems with bonded network interfaces.
- Solution: Verify bonding configurations, check network cables, and troubleshoot switch and router settings.

67. Java Application Server Startup Failures:

- Issue: Java application server not starting.
- Solution: Review server logs, check Java version compatibility, and inspect configuration files.

68. System Clock Drift:

- Issue: System clock out of sync.
- Solution: Configure NTP, set the hardware clock (hwclock), and ensure time zone settings are correct.

69. GPG Keyring Problems:

- Issue: GPG keyring errors.
- Solution: Update or import GPG keys, check keyring permissions, and verify keyring integrity.

70. System Sleep/Wake Issues:

- Issue: Problems with sleep or wake functions.
- Solution: Check power management settings, update kernel or graphics drivers, and review ACPI configurations.

71. Wi-Fi Connection Drops:

- Issue: Unstable Wi-Fi connections.
- Solution: Update Wi-Fi drivers, adjust power management settings, and check for interference or signal issues.

72. FTP Server Configuration:

- Issue: FTP server not functioning.
- Solution: Check FTP server logs, review configurations, and ensure necessary ports are open.

73. Invalid Partition Table:

- Issue: Invalid or corrupted partition table.
- Solution: Use tools like gdisk or parted to repair or recreate the partition table.

74. Disk I/O Issues:

- Issue: Slow disk I/O performance.
- Solution: Check disk health with tools like smartctl, optimize file systems, and monitor I/O metrics.

75. Incomplete Software Removal:

- Issue: Residual files after software removal.
- Solution: Manually remove leftover files and configurations after uninstalling software.

76. System Stalls During Heavy Load:

- Issue: System becomes unresponsive under heavy load.
- Solution: Optimize system settings, tune kernel parameters, and consider hardware upgrades.

77. Invalid SSL/TLS Certificates:

- Issue: SSL/TLS certificate errors.
- Solution: Renew or replace expired certificates, ensure correct file permissions, and check web server configurations.

78. XFS or Btrfs File System Issues:

- Issue: Problems with XFS or Btrfs file systems.
- Solution: Check file system consistency (xfs_repair or btrfs scrub), and address any reported errors.

79. Kernel Oops or Panics:

- Issue: Kernel Oops messages or panics.

- Solution: Analyze kernel logs, update kernel, and identify and address faulty hardware or drivers.

80. Systemd Timer Misconfigurations:

- Issue: Incorrect systemd timer configurations.
- Solution: Review timer unit files, verify scheduling settings, and restart the timer service.

81. Unresponsive Web Applications:

- Issue: Web applications not responding.
- Solution: Check application logs, database connections, and review server resource usage.

82. System Clutter:

- Issue: Accumulation of temporary files.
- Solution: Periodically clean up temporary directories (/tmp), use tools like bleachbit, and monitor disk space usage.

83. System Monitoring Setup:

- Issue: Inadequate system monitoring.
- Solution: Set up monitoring tools (e.g., Nagios, Prometheus) to track system metrics and receive alerts.

84. SSL/TLS Vulnerabilities:

- Issue: Vulnerabilities in SSL/TLS protocols.
- Solution: Regularly update OpenSSL or other SSL/TLS libraries, and monitor security advisories.

85. Bash Scripting Errors:

- Issue: Errors in Bash scripts.

- Solution: Review script syntax, check for variable scope issues, and debug using `bash-x`.

86. ****Systemd Service Dependencies:****

- ****Issue:**** Service fails due to missing dependencies.
- ****Solution:**** Adjust dependencies in systemd service files, and ensure required services are started.

87. ****Failed RAID Rebuild:****

- ****Issue:**** RAID array rebuild fails.
- ****Solution:**** Identify and replace faulty drives, and initiate the RAID rebuild manually if needed.

88. ****Invalid SSL/TLS Cipher Suites:****

- ****Issue:**** Weak or deprecated SSL/TLS cipher suites.
- ****Solution:**** Update SSL/TLS configurations, disable weak ciphers, and follow best practices for secure configurations.

89. ****Docker Container Networking Issues:****

- ****Issue:**** Networking problems with Docker containers.

- **Solution:** Check container network configurations, update Docker, and ensure proper firewall rules.

90. **System Shutdown Delays:**

- **Issue:** System takes a long time to shut down.
- **Solution:** Investigate systemd delays, review shutdown logs, and address any issues causing delays.

91. **SELinux AVC Denials:**

- **Issue:** SELinux AVC denials in logs.
- **Solution:** Analyze SELinux logs, create custom SELinux policies if necessary, and update security contexts.

92. **Nginx Reverse Proxy Configuration:**

- **Issue:** Reverse proxy misconfigurations.
- **Solution:** Review Nginx configuration, check proxy pass settings, and ensure backend server connectivity.

93. **Systemd Resolved DNS Caching:**

- **Issue:** Systemd-resolved DNS caching problems.

- **Solution:** Clear DNS cache (``sudo systemd-resolve--flush-caches``), and review resolved service settings.

94. **Systemd Journal Disk Usage:**

- **Issue:** Excessive disk usage by systemd journal.
- **Solution:** Adjust journal rotation settings, and periodically clear old journal entries.

95. **Git Repository Corruption:**

- **Issue:** Corruption in Git repositories.
- **Solution:** Use Git tools (``git fsck``), restore from backups, and check for disk errors.

96. **Virtual Machine Snapshots:**

- **Issue:** Issues with virtual machine snapshots.
- **Solution:** Review virtual machine snapshot configurations, and ensure there is sufficient disk space.

97. **Email Delivery Failures:**

- **Issue:** Email messages not being delivered.
- **Solution:** Check mail server logs, verify DNS configurations, and ensure mail server reputation.

98. **Automount Configuration Problems:**

- **Issue:** Automounted filesystems not mounting.
- **Solution:** Check automount configurations, verify NFS or other network mount connectivity, and review logs.

99. **GPG Key Expiry:**

- **Issue:** GPG keys expiring.
- **Solution:** Renew GPG keys, update key expiration dates, and distribute updated keys.

100. **Chroot Jail Escape:**

- **Issue:** Escaping from a chroot jail.
- **Solution:** Regularly update software within the chroot environment, use hardened configurations, and monitor for security advisories.

Linux- Common Issue and Solution.

Common issues in Linux and their solutions.

1. **Package Management Issues:**

- **Issue:** Unable to install a package.
- **Solution:** Use the package manager for your distribution (e.g., `apt` for Debian/Ubuntu, `yum` for CentOS) and ensure repositories are properly configured.

2. **Network Configuration:**

- **Issue:** No internet connection.
- **Solution:** Check network cables, run network diagnostic commands (`ifconfig`, `ping`), and verify DNS settings.

3. **Permission Problems:**

- **Issue:** Permission denied while trying to access or modify files.
- **Solution:** Use `chmod` or `chown` to adjust file permissions or ownership. Be cautious not to modify critical system files.

4. **Boot Problems:**

- **Issue:** System not booting properly.
- **Solution:** Use a live CD/USB to access the system, repair the bootloader (e.g., Grub), or fix corrupted system files.

5. **Update Issues:**

- **Issue:** Software updates fail.
- **Solution:** Check repositories, update the package list (`sudo apt update`), and then upgrade packages (`sudo apt upgrade`).

6. **Disk Space Problems:**

- **Issue:** Running out of disk space.
- **Solution:** Identify large files/directories (`du` or `df` commands) and remove unnecessary files. Be careful not to delete critical system files.

7. **Graphical Interface Issues:**

- **Issue:** Desktop environment not loading.
- **Solution:** Check graphics drivers, reinstall desktop environment, or switch to a different one.

8. **Kernel Panic:**

- **Issue:** Kernel panic during boot.
- **Solution:** Boot into a different kernel version, update the kernel, or reinstall the kernel package.

9. ****Authentication Problems:****

- ****Issue:**** Unable to log in.
- ****Solution:**** Verify username and password, check for CAPS LOCK, and reset password if needed.

10. ****Swap Space Issues:****

- ****Issue:**** Insufficient swap space.
- ****Solution:**** Resize or add swap space using ``swapon`` or by modifying the ``/etc/fstab`` file.

11. ****File System Corruption:****

- ****Issue:**** File system errors.
- ****Solution:**** Run file system checks (``fsck``) on unmounted partitions to fix errors.

12. ****Incompatible Libraries:****

- ****Issue:**** Missing or incompatible libraries.
- ****Solution:**** Install required libraries using the package manager.

13. ****Systemd Service Failures:****

- ****Issue:**** Service not starting.
- ****Solution:**** Check service status with ``systemctl status``, review logs, and restart the service.

14. ****Sound Issues:****

- ****Issue:**** No sound or distorted audio.

- **Solution:** Check audio settings, reinstall audio drivers, or update kernel modules.

15. **Bluetooth Connectivity Problems:**

- **Issue:** Bluetooth devices not connecting.
- **Solution:** Restart Bluetooth service, update Bluetooth drivers, and check hardware compatibility.

16. **Firewall Configuration:**

- **Issue:** Unable to access services due to firewall.
- **Solution:** Adjust firewall rules using tools like `ufw` or `firewalld`.

17. **SELinux Issues:**

- **Issue:** SELinux denials.
- **Solution:** Review SELinux logs, adjust SELinux policies, or set SELinux to permissive mode.

18. **Hardware Recognition:**

- **Issue:** Hardware not recognized.
- **Solution:** Update kernel, check hardware compatibility, and install necessary drivers.

19. **X Server Problems:**

- **Issue:** GUI not starting.

- **Solution:** Check X server logs, update graphics drivers, and review display manager settings.

20. **Cron Job Failures:**

- **Issue:** Scheduled tasks not running.
- **Solution:** Review cron job logs, check syntax, and ensure proper permissions.

21. **SSH Connection Issues:**

- **Issue:** Unable to connect via SSH.
- **Solution:** Check SSH configuration, verify firewall rules, and restart SSH service.

22. **Locale Problems:**

- **Issue:** Incorrect date, time, or language settings.
- **Solution:** Set the correct locale using `locale-gen` and update configuration files.

23. **Package Dependency Errors:**

- **Issue:** Unmet dependencies during package installation.
- **Solution:** Use the package manager to resolve dependencies (`sudo apt-f install` for Debian/Ubuntu).

24. ****RAID Configuration Issues:****

- ****Issue:**** RAID array not functioning.
- ****Solution:**** Check RAID configuration, replace faulty drives, and rebuild the array.

25. ****Kernel Module Loading:****

- ****Issue:**** Unable to load kernel modules.
- ****Solution:**** Use ``modprobe`` to manually load modules and update kernel headers if needed.

26. ****System Logs Filling Up:****

- ****Issue:**** Disk space consumed by logs.
- ****Solution:**** Configure log rotation settings and periodically clean up old logs.

27. ****System Freezing:****

- ****Issue:**** System becomes unresponsive.
- ****Solution:**** Check for resource-intensive processes, update kernel, and review hardware health.

28. ****DNS Resolution Problems:****

- ****Issue:**** Unable to resolve domain names.
- ****Solution:**** Check DNS settings in ``/etc/resolv.conf`` and test DNS resolution using tools like ``nslookup``.

29. ****NTP Synchronization Issues:****

- ****Issue:**** Incorrect system time.
- ****Solution:**** Configure NTP to synchronize system time (``sudo timedatectl set-ntp true``).

30. ****System Hanging on Shutdown:****

- ****Issue:**** System doesn't shut down properly.
- ****Solution:**** Investigate systemd issues, update kernel, and check for hardware problems.

31. ****LVM (Logical Volume Management) Issues:****

- ****Issue:**** Problems with logical volumes.
- ****Solution:**** Check volume group and logical volume status, and repair if necessary.

32. ****SMTP Email Sending Issues:****

- ****Issue:**** Unable to send emails.
- ****Solution:**** Review mail server logs, check SMTP configuration, and ensure proper authentication.

33. ****GPU Driver Problems:****

- ****Issue:**** Graphics card driver issues.
- ****Solution:**** Install or update graphics drivers based on the GPU manufacturer.

34. ****Systemd Timer Issues:****

- **Issue:** Scheduled systemd timers not triggering.
- **Solution:** Verify timer unit files, check logs, and restart the timer service.

35. **NFS (Network File System) Mounting Problems:**

- **Issue:** Unable to mount NFS shares.
- **Solution:** Verify NFS server and client configurations, and check network connectivity.

36. **System Resource Exhaustion:**

- **Issue:** High CPU, memory, or disk usage.
- **Solution:** Identify and address resource-intensive processes, optimize system settings, and consider hardware upgrades.

37. **Samba File Sharing Issues:**

- **Issue:** Problems with Samba shares.
- **Solution:** Review Samba configuration, check file permissions, and restart the Samba service.

38. **Slow Boot Times:**

- **Issue:** System takes a long time to boot.
- **Solution:** Analyze boot logs, disable unnecessary services, and update or remove problematic software.

39. ****Incorrect Keyboard Layout:****

- ****Issue:**** Incorrect keyboard mapping.
- ****Solution:**** Set the correct keyboard layout using tools like ``setxkbmap`` or through system settings.

40. ****Stale Lock Files:****

- ****Issue:**** Application won't start due to stale lock files.
- ****Solution:**** Identify and remove lock files, ensuring no active instances of the application are running.

41. ****Incomplete Software Installation:****

- ****Issue:**** Software installation interrupted or incomplete.
- ****Solution:**** Remove incomplete installations, clean up package cache, and reinstall the software.

42. ****Stale Package Dependencies:****

- ****Issue:**** Old or conflicting package dependencies.
- ****Solution:**** Use the package manager to remove obsolete dependencies (``sudo apt autoremove`` for Debian/Ubuntu).

43. ****System Recovery After Power Failure:****

- ****Issue:**** System doesn't recover properly after a power outage.
- ****Solution:**** Configure BIOS settings, use UPS (Uninterruptible Power Supply), and consider graceful shutdown options.

44. ****High System Load:****

- ****Issue:**** System load is consistently high.
- ****Solution:**** Identify resource-hungry processes using tools like `top` or `htop` and address the underlying cause.

45. ****DNS Cache Issues:****

- ****Issue:**** DNS cache causing resolution problems.
- ****Solution:**** Clear DNS cache using appropriate commands (`systemctl restart systemd-resolved` for systemd-based systems).

46. ****Unexpected System Reboots:****

- ****Issue:**** System reboots without warning.
- ****Solution:**** Investigate logs for kernel panics, hardware issues, or software conflicts.

47. System File Corruption:

- Issue: Critical system files corrupted.

- Solution: Restore from backups, reinstall affected packages, or repair the system using a live CD/USB.

48. Issues with External Storage:

- Issue: External drives not recognized.
- Solution: Check cables, use different USB ports, and ensure external drives are properly formatted.

49. Stuck Software Updates:

- Issue: Software updates seem stuck.
- Solution: Verify internet connection, check update logs, and try updating packages individually.

50. Systemd Journal Corruption:

- Issue: Corruption in systemd journal files.
- Solution: Clear or rotate journal files using appropriate commands (`journalctl--rotate`).

51. User Permission Problems:

- Issue: Regular user unable to perform certain tasks.
- Solution: Add the user to appropriate groups, adjust file permissions, or use `sudo` for elevated privileges.

52. Kernel Module Compilation Errors:

- Issue: Compiling kernel modules fails.

- Solution: Install kernel headers, check compiler dependencies, and follow module-specific build instructions.

53. GPU Driver Module Compilation:

- Issue: Compiling GPU driver modules fails.
- Solution: Install necessary development tools and headers, follow GPU driver installation instructions, and ensure compatibility with the kernel.

54. Stuck Print Jobs:

- Issue: Print jobs stuck in the queue.
- Solution: Restart the print service (cups), check for printer connectivity, and clear the print queue.

55. Invalid Shell Configuration:

- Issue: Invalid shell configurations prevent login.
- Solution: Use a rescue mode or live CD to edit shell configuration files (/etc/profile, ~/.bashrc, etc.).

56. PHP Configuration Issues:

- Issue: PHP scripts not executing as expected.
- Solution: Review PHP configuration files, check for syntax errors, and restart the web server.

57. Apache/Nginx Configuration Errors:

- Issue: Web server misconfigurations.

- Solution: Check Apache or Nginx configuration files for errors, and restart the web server.

58. DNS Server Configuration:

- Issue: Incorrect DNS server settings.
- Solution: Verify `/etc/resolv.conf`, check DNS server configurations, and restart the networking service.

59. Systemd Service Masking:

- Issue: Service masked and unable to start.
- Solution: Unmask the service using `systemctl unmask`, then start and enable it.

60. SSH Key Authentication Failures:

- Issue: Unable to authenticate via SSH keys.
- Solution: Check key permissions, ensure correct public key is added to `~/.ssh/authorized_keys`, and review SSH daemon configuration.

61. Slow MySQL/MariaDB Performance:

- Issue: Database server performance issues.
- Solution: Optimize database queries, increase system resources, and fine-tune MySQL/MariaDB configuration.

62. System Font Issues:

- Issue: Font rendering problems.
- Solution: Install necessary fonts, clear font caches, and configure font settings in desktop environments.

63. JVM Heap Issues:

- Issue: Java applications experiencing memory problems.
- Solution: Adjust JVM heap settings, analyze garbage collection logs, and update Java runtime if needed.

64. LDAP Authentication Problems:

- Issue: Issues with LDAP authentication.
- Solution: Verify LDAP server settings, check network connectivity, and review PAM (Pluggable Authentication Modules) configurations.

65. Sudden Network Disconnections:

- Issue: Frequent network disconnections.
- Solution: Check network hardware, update network drivers, and review syslog or journal logs for network-related errors.

66. Network Bonding Issues:

- Issue: Problems with bonded network interfaces.
- Solution: Verify bonding configurations, check network cables, and troubleshoot switch and router settings.

67. Java Application Server Startup Failures:

- Issue: Java application server not starting.
- Solution: Review server logs, check Java version compatibility, and inspect configuration files.

68. System Clock Drift:

- Issue: System clock out of sync.
- Solution: Configure NTP, set the hardware clock (hwclock), and ensure time zone settings are correct.

69. GPG Keyring Problems:

- Issue: GPG keyring errors.
- Solution: Update or import GPG keys, check keyring permissions, and verify keyring integrity.

70. System Sleep/Wake Issues:

- Issue: Problems with sleep or wake functions.
- Solution: Check power management settings, update kernel or graphics drivers, and review ACPI configurations.

71. Wi-Fi Connection Drops:

- Issue: Unstable Wi-Fi connections.
- Solution: Update Wi-Fi drivers, adjust power management settings, and check for interference or signal issues.

72. FTP Server Configuration:

- Issue: FTP server not functioning.
- Solution: Check FTP server logs, review configurations, and ensure necessary ports are open.

73. Invalid Partition Table:

- Issue: Invalid or corrupted partition table.
- Solution: Use tools like gdisk or parted to repair or recreate the partition table.

74. Disk I/O Issues:

- Issue: Slow disk I/O performance.
- Solution: Check disk health with tools like smartctl, optimize file systems, and monitor I/O metrics.

75. Incomplete Software Removal:

- Issue: Residual files after software removal.
- Solution: Manually remove leftover files and configurations after uninstalling software.

76. System Stalls During Heavy Load:

- Issue: System becomes unresponsive under heavy load.
- Solution: Optimize system settings, tune kernel parameters, and consider hardware upgrades.

77. Invalid SSL/TLS Certificates:

- Issue: SSL/TLS certificate errors.
- Solution: Renew or replace expired certificates, ensure correct file permissions, and check web server configurations.

78. XFS or Btrfs File System Issues:

- Issue: Problems with XFS or Btrfs file systems.
- Solution: Check file system consistency (xfs_repair or btrfs scrub), and address any reported errors.

79. Kernel Oops or Panics:

- Issue: Kernel Oops messages or panics.

- Solution: Analyze kernel logs, update kernel, and identify and address faulty hardware or drivers.

80. Systemd Timer Misconfigurations:

- Issue: Incorrect systemd timer configurations.
- Solution: Review timer unit files, verify scheduling settings, and restart the timer service.

81. Unresponsive Web Applications:

- Issue: Web applications not responding.
- Solution: Check application logs, database connections, and review server resource usage.

82. System Clutter:

- Issue: Accumulation of temporary files.
- Solution: Periodically clean up temporary directories (/tmp), use tools like bleachbit, and monitor disk space usage.

83. System Monitoring Setup:

- Issue: Inadequate system monitoring.
- Solution: Set up monitoring tools (e.g., Nagios, Prometheus) to track system metrics and receive alerts.

84. SSL/TLS Vulnerabilities:

- Issue: Vulnerabilities in SSL/TLS protocols.
- Solution: Regularly update OpenSSL or other SSL/TLS libraries, and monitor security advisories.

85. Bash Scripting Errors:

- Issue: Errors in Bash scripts.

- Solution: Review script syntax, check for variable scope issues, and debug using `bash-x`.

86. ****Systemd Service Dependencies:****

- ****Issue:**** Service fails due to missing dependencies.
- ****Solution:**** Adjust dependencies in systemd service files, and ensure required services are started.

87. ****Failed RAID Rebuild:****

- ****Issue:**** RAID array rebuild fails.
- ****Solution:**** Identify and replace faulty drives, and initiate the RAID rebuild manually if needed.

88. ****Invalid SSL/TLS Cipher Suites:****

- ****Issue:**** Weak or deprecated SSL/TLS cipher suites.
- ****Solution:**** Update SSL/TLS configurations, disable weak ciphers, and follow best practices for secure configurations.

89. ****Docker Container Networking Issues:****

- ****Issue:**** Networking problems with Docker containers.

- **Solution:** Check container network configurations, update Docker, and ensure proper firewall rules.

90. **System Shutdown Delays:**

- **Issue:** System takes a long time to shut down.
- **Solution:** Investigate systemd delays, review shutdown logs, and address any issues causing delays.

91. **SELinux AVC Denials:**

- **Issue:** SELinux AVC denials in logs.
- **Solution:** Analyze SELinux logs, create custom SELinux policies if necessary, and update security contexts.

92. **Nginx Reverse Proxy Configuration:**

- **Issue:** Reverse proxy misconfigurations.
- **Solution:** Review Nginx configuration, check proxy pass settings, and ensure backend server connectivity.

93. **Systemd Resolved DNS Caching:**

- **Issue:** Systemd-resolved DNS caching problems.

- **Solution:** Clear DNS cache (``sudo systemd-resolve--flush-caches``), and review resolved service settings.

94. **Systemd Journal Disk Usage:**

- **Issue:** Excessive disk usage by systemd journal.
- **Solution:** Adjust journal rotation settings, and periodically clear old journal entries.

95. **Git Repository Corruption:**

- **Issue:** Corruption in Git repositories.
- **Solution:** Use Git tools (``git fsck``), restore from backups, and check for disk errors.

96. **Virtual Machine Snapshots:**

- **Issue:** Issues with virtual machine snapshots.
- **Solution:** Review virtual machine snapshot configurations, and ensure there is sufficient disk space.

97. **Email Delivery Failures:**

- **Issue:** Email messages not being delivered.
- **Solution:** Check mail server logs, verify DNS configurations, and ensure mail server reputation.

98. **Automount Configuration Problems:**

- **Issue:** Automounted filesystems not mounting.
- **Solution:** Check automount configurations, verify NFS or other network mount connectivity, and review logs.

99. **GPG Key Expiry:**

- **Issue:** GPG keys expiring.
- **Solution:** Renew GPG keys, update key expiration dates, and distribute updated keys.

100. **Chroot Jail Escape:**

- **Issue:** Escaping from a chroot jail.
- **Solution:** Regularly update software within the chroot environment, use hardened configurations, and monitor for security advisories.

Command	
ls	Lists a directory's content
pwd	Shows the current working directory's path
cd	Changes the working directory
mkdir	Creates a new directory
rm	Deletes a file
cp	Copies files and directories, including their content
mv	Moves or renames files and directories
touch	Creates a new empty file
file	Checks a file's type
zip and unzip	Creates and extracts a ZIP archive
tar	Archives files without compression in a TAR format
nano, vi, and jed	Edits a file with a text editor
cat	Lists, combines, and writes a file's content as a standard output
grep	Searches a string within a file
sed	Finds, replaces, or deletes patterns in a file
head	Displays a file's first ten lines
tail	Prints a file's last ten lines
awk	Finds and manipulates patterns in a file
sort	Reorders a file's content
cut	Sections and prints lines from a file
diff	Compares two files' content and their differences
tee	Prints command outputs in Terminal and a file

locate	Finds files in a system's database
find	Outputs a file or folder's location
sudo	Runs a command as a superuser
su	Runs programs in the current shell as another user
chmod	Modifies a file's read, write, and execute permissions
chown	Changes a file, directory, or symbolic link's ownership
useradd and userdel	Creates and removes a user account
df	Displays the system's overall disk space usage
du	Checks a file or directory's storage consumption
top	Displays running processes and the system's resource usage
htop	Works like top but with an interactive user interface
ps	Creates a snapshot of all running processes
uname	Prints information about your machine's kernel, name, and hardware
hostname	Shows your system's hostname
time	Calculates commands' execution time
systemctl	Manages system services
watch	Runs another command continuously
jobs	Displays a shell's running processes with their statuses
kill	Terminates a running process
shutdown	Turns off or restarts the system
ping	Checks the system's network connectivity
wget	Downloads files from a URL

curl	Transmits data between servers using URLs
scp	Securely copies files or directories to another system
rsync	Synchronizes content between directories or machines
ifconfig	Displays the system's network interfaces and their configurations
netstat	Shows the system's network information, like routing and sockets
traceroute	Tracks a packet's hops to its destination
nslookup	Queries a domain's IP address and vice versa
dig	Displays DNS information, including record types
history	Lists previously run commands
man	Shows a command's manual
echo	Prints a message as a standard output
ln	Links files or directories
alias and unalias	Sets and removes an alias for a file or command
cal	Displays a calendar in Terminal
apt-get	Manages Debian-based distros package libraries

Linux Commands for File and Directory Management

This section will explore basic Linux commands for file and directory management.

1. ls command

The **ls** command lists files and directories in your system. Here's the syntax:

```
ls [/directory/folder/path]
```

```
root@srv:/# ls /directory/folder/path
file1.txt
```

If you remove the path, the **ls** command will show the current working directory's content. You can modify the command using these options:

- **-R** – lists all the files in the subdirectories.
- **-a** – shows all files, including hidden ones.
- **-lh** – converts sizes to readable formats, such as **MB**, **GB**, and **TB**.

2. pwd command

The **pwd** command prints your current working directory's path, like **/home/directory/path**. Here's the command syntax:

```
pwd [option]
```

It supports two options. The **-L** or **--logical** option prints environment variable content, including symbolic links. Meanwhile, **-P** or **--physical** outputs the current directory's actual path.

```
root@srv:/directory/folder/path# pwd
/directory/folder/path
```

3. cd command

Use the **cd** command to navigate the Linux files and directories. To use it, run this syntax with sudo privileges:

```
cd /directory/folder/path
```

```
root@srv:/# cd /directory/folder/path
root@srv:/directory/folder/path#
```

Depending on your current location, it requires either the full path or the directory name. For example, omit **/username** from **/username/directory/folder** if you are already within it.

Omitting the arguments will take you to the home folder. Here are some navigation shortcuts:

- **cd ~[username]** – goes to another user's home directory.
- **cd ..** – moves one directory up.
- **cd-** – switches to the previous directory.

4. mkdir command

Use the **mkdir** command to create one or multiple directories and set their permissions. Ensure you are authorized to make a new folder in the parent directory. Here's the basic syntax:

```
mkdir [option] [directory_name]
```

To create a folder within a directory, use the path as the command parameter. For example, **mkdir music/songs** will create a **songs** folder inside **music**. Here are several common **mkdir** command options:

- **-p** – creates a directory between two existing folders. For example, **mkdir -p Music/2024/Songs** creates a new **2024** directory.
- **-m** – sets the folder permissions. For instance, enter **mkdir -m777 directory** to create a directory with read, write, and execute permissions for all users.
- **-v** – prints a message for each created directory.

```
root@srv:/# mkdir -v new-folder
mkdir: created directory 'new-folder'
```

5. rmdir command

Use the **rmdir** command to delete an empty directory in Linux. The user must have **sudo** privileges in the parent directory. Here's the syntax:

```
rmdir [option] directory_name
```

If the folder contains a subdirectory, the command will return an error. To force delete a non-empty directory, use the **-p** option.

6. rm command

Use the **rm** command to permanently delete files within a directory. Here's the general syntax:

```
rm [filename1] [filename2] [filename3]
```

Adjust the number of files in the command according to your needs. If you encounter an error, ensure you have the **write** permission in the directory.

To modify the command, add the following options:

- **-i** – prompts a confirmation before deletion.
- **-f** – allows file removal without a confirmation.

- **-r** – deletes files and directories recursively.

Warning! Use the **rm** command with caution since deletion is irreversible. Avoid using the **-r** and **-f** options since they may wipe all your files. Always add the **-i** option to avoid accidental deletion.

7. cp command

Use the **cp** command to copy files or directories, including their content, from your current location to another. It has various use cases, such as:

Copying one file from the current directory to another folder. Specify the file name and target path:

```
cp filename.txt /home/username/Documents
```

Duplicating multiple files to a directory. Enter the file names and the destination path:

```
cp filename1.txt filename2.txt filename3.txt /home/username/Documents
```

Copying a file's content to another within the same directory. Enter the source and the destination file:

```
cp filename1.txt filename2.txt
```

Duplicating an entire directory. Pass the **-R** flag followed by the source and destination directory:

```
cp -R /home/username/Documents /home/username/Documents_backup
```

8. mv command

Use the **mv** command to move or rename files and directories. To move items, enter the file name followed by the destination directory:

```
mv filename.txt /home/username/Documents
```

Meanwhile, use the following syntax to rename a file in Linux with the **mv** command:

```
mv old_filename.txt new_filename.txt
```

9. touch command

The **touch** command lets you create an empty file in a specific directory path. Here's the syntax:

```
touch [option] /home/directory/path/file.txt
```

If you omit the path, the command will create the item in the current folder. You can also use **touch** to generate and modify a timestamp in the Linux command line.

10. file command

The **file** command lets you check a file type – whether it is a text, image, or binary. Here's the syntax:

```
file filename.txt
```

To bulk-check multiple files, list them individually or use their path if they are in the same directory. Add the **-k** option to display more detailed information and **-i** to show the file's MIME type.

11. zip, unzip commands

The **zip** command lets you compress items into a **ZIP** file with the optimal compression ratio. Here's the syntax:

```
zip [options] zipfile file1 file2...
```

For example, this command compresses **note.txt** into **archive.zip** in the current working directory:

```
zip archive.zip note.txt
```

Use the **unzip** command to extract the compressed file. Here's the syntax:

```
unzip [option] file_name.zip
```

12. tar command

The **tar** command archives multiple items into a **TAR** file – a format similar to **ZIP** with optional compression. Here's the syntax:

```
tar [options] [archive_file] [target file or directory]
```

For instance, enter the following to create a new **newarchive.tar** archive in the **/home/user/Documents** directory:

```
tar -cvzf newarchive.tar /home/user/Documents
```

Linux Commands for Text Processing and Searching

The following section explores several Linux commands for processing and searching text.

13. nano, vi, jed commands

Linux lets users edit files using a text editor like **nano**, **vi**, or **jed**. While most distributions include **nano** and **vi**, users must install **jed** manually. All these tools have the same command syntax:

```
nano filename
```

```
vi filename
```

```
jed filename
```

If the target file doesn't exist, these editors will create one. We recommend **nano** if you want to quickly edit text files. Meanwhile, use **vi** or **jed** for scripting and programming.

14. cat command

Concatenate or **cat** is one of the most used Linux commands. It lists, combines, and writes file content to the standard output. Here's the syntax:

```
cat filename.txt
```

There are various ways to use the **cat** command:

- **cat > file.txt** –creates a new file.
- **cat file1.txt file2.txt > file3.txt** –merges **file1.txt** with **file2.txt** and stores the output in **filename3.txt**.
- **tac file.txt** – displays content in reverse order.

15. grep command

The **global regular expression** or **grep** command lets you find a word by searching the content of a file. This Linux command prints all lines containing the matching strings, which is useful for filtering large log files.

For example, to display lines containing **blue** in the **notepad.txt** file, enter:

```
grep blue notepad.txt
```

16. sed command

The **sed** command lets you find, replace, and delete patterns in a file without using a text editor. Here's the general syntax:

```
sed [option] 'script' input_file
```

The script contains the searched regular expression pattern, the replacement string, and subcommands. Use the **s** subcommand to replace matching patterns or **d** to delete them.

At the end, specify the file containing the pattern to modify. Here's an example of a command that replaces **red** in **colors.txt** and **hue.txt** with **blue**:

```
sed 's/red/blue' colors.txt hue.txt
```

17. head command

The **head** command prints the first ten lines of a text file or piped data in your command-line interface. Here's the general syntax:

```
head [option] [file]
```

For instance, to view the first ten lines of **note.txt** in the current directory, enter:

```
head note.txt
```

The **head** command accepts several options, such as:

- **-n** – changes the number of lines printed. For example, **head -n 5** shows the first five lines.
- **-c** – prints the file's first customized number of bytes.
- **-q** – disables headers specifying the file name.

```
root@srv:/directory/folder/path# head -n 5 file1.txt
This is the first line
This is the second line
This is the third line
This is the fourth line
This is the fifth line
```

18. tail command

The **tail** command displays the last ten lines of a file, which is useful for checking new data and errors. Here's the syntax:

```
tail [option] [file]
```

For example, enter the following to show the last ten lines of the **colors.txt** file:

```
tail -n colors.txt
```

19. awk command

The **awk** command scans regular expression patterns in a file to retrieve or manipulate matching data. Here's the basic syntax:

```
awk '/regex pattern/{action}' input_file.txt
```

The action can be mathematical operations, conditional statements like **if**, output expressions such as **print**, and a **delete** command. It also contains the **\$n** notation, which refers to a field in the current line.

To add multiple actions, list them based on the execution order, separated using semicolons. For example, this command contains mathematical, conditional, and output statements:

```
awk -F':' '{ total += $2; students[$1] = $2 } END { average = total /  
length(students); print "Average:", average; print "Above average:"; for (student  
in students) if (students[student] > average) print student }' score.txt
```

```
root@srv: ~:/directory/folder/path# awk -F':' '{ total += $2; students[$1] =  
$2 } END { average = total / length(students); print "Average:", average; print  
"Above average:"; for (student in students) if (students[student] > average) pri  
nt student }' score.txt  
Average: 80.5  
Above average:  
Student B  
Student C
```

20. sort command

The **sort** command rearranges lines in a file in a specific order. It doesn't modify the actual file and only prints the result as Terminal outputs. Here's the syntax:

```
sort [option] [file]
```

By default, this command will sort the lines in alphabetical order, from A to Z. To modify the sorting, use these options:

- **-o** – redirects the command outputs to another file.
- **-r** – reverses the sorting order to descending.
- **-n** – sorts the file numerically.
- **-k** – reorders data in a specific field.

```
root@srv:/directory/folder/path# sort file1.txt
Line A
Line B
Line C
Line D
```

21. cut command

The **cut** command retrieves sections from a file and prints the result as Terminal outputs. Here's the syntax:

```
cut [option] [file]
```

Instead of a file, you can use data from standard input. To determine how the command sections the line, use the following options:

- **-f** – selects a specific field.
- **-b** – cuts the line by a specified byte size.
- **-c** – sections the line using a specified character.
- **-d** – separates lines based on delimiters.

You can combine these options, use a range, and specify multiple values. For example, this command extracts the third to fifth field from a comma-separated list:

```
cut -d',' -f3-5 list.txt
```

```
root@srv:/directory/folder/path# cut -d',' -f3-5 list.txt
item3, item4, item5
item3, item4, item5
item3, item4, item5
item3, item4, item5
```

22. diff command

The **diff** command compares two files' content and outputs the differences. It is used to alter a program without modifying the code. Here's the general format:

```
diff [option] file1 file2
```

Below are some acceptable options:

- **-c** – displays the difference between two files in a context form.
- **-u** – shows the output without redundant information.
- **-i** – makes the **diff** command case insensitive.

```
root@srv:/directory/folder/path# diff list.txt list2.txt
1,4c1,4
< Item1, item2, item3, item4, item5
< Item1, item2, item3, item4, item5
< Item1, item2, item3, item4, item5
< Item1, item2, item3, item4, item5
---
> Item2, item2, item3, item4, item5
> Item1, item2, item4, item4, item5
> Item1, item3, item3, item4, item5
> Item5, item3, item3, item4, item5
```

23. tee command

The **tee** command writes the user's input to Terminal's output and files. Here's the basic syntax:

```
command | tee [option] file1
```

For example, the following pings Google and prints the output in Terminal, **ping_result.txt**, and the **19092024.txt** file:

```
ping google.com | tee ping_result.txt 19092024.txt
```

24. locate command

The **locate** command lets you find a file in the database system. Add the **-i** option to turn off case sensitivity and an asterisk (*) to find content with multiple keywords. For example:

```
locate -i school*note
```

The command searches for files containing **school** and **note**, regardless of their letter case.

25. find command

Use the **find** command to search for files within a specific directory. Here's the syntax:

```
find [option] [path] [expression]
```

For example, to find a file called **file1.txt** within the **directory** folder and its subdirectories, use this command:

```
find /home -name file1.txt
```

```
root@srv:/# find /directory -name file1.txt
/directory/folder/path/file1.txt
```

If you omit the path, the command will search the current working directory. You can also find directories using the following:

```
find ./ -type d -name directoryname
```

Linux Commands for User and Permission Management

Below are several Linux commands for managing the system's users and permissions.

26. sudo command

Superuser do or **sudo** is one of the most basic commands in Linux. It runs your command with administrative or root permissions. Here's the general syntax:

```
sudo (command)
```

When you run a sudo command, Terminal will request the root password. For example, this snippet runs **useradd** with the superuser privilege:

```
sudo useradd username
```

You can also add an option, such as:

- **-k** – invalidates the timestamp file.
- **-g** – executes commands as a specified group name or ID.
- **-h** – runs commands on the host.

Warning! Running a command with sudo privileges can modify all aspects of your system. Since misusing it may break your system, run the command with caution and only if you understand its possible repercussions.

27. su command

The **su** command lets you run a program in the Linux shell as a different user. It is useful to connect via SSH while the root user is disabled. Here's the syntax:

```
su [options] [username [argument]]
```

Without any option or argument, this command runs through **root** and prompts you to use the **sudo** privileges temporarily. Some options are:

- **-p** – keeps the same shell environment, consisting of **HOME**, **SHELL**, **USER**, and **LOGNAME**.

- **-s** – lets you specify another shell environment to run.
- **-l** – runs a login script to switch users. It requires you to enter the user's password.

To check the current shell's user account, run the **whoami** command:

```
root@srv:/# su user1
$ whoami
user1
```

28. chmod command

The **chmod** command modifies directory or file permissions in Linux. Here's the basic syntax:

```
chmod [option] [permission] [file_name]
```

In Linux, each file is associated with three user classes – **owner**, **group member**, and **others**. It also has three permissions – **read**, **write**, and **execute**. If an owner wants to grant all permissions to every user, the command looks like this:

```
chmod -rwxrwxrwx note.txt
```

29. chown command

The **chown** command lets you change a file, directory, or symbolic link's ownership to the specified username. Here's the syntax:

```
chown [option] owner[:group] file(s)
```

For example, to make **linuxuser2** the owner of **filename.txt**, use:

```
chown linuxuser2 filename.txt
```

30. useradd, userdel commands

Use **useradd** to create a new Linux user account and change its password with the **passwd** command. Here are the syntaxes:

```
useradd [option] username
```

```
passwd username
```

Both the **useradd** and **passwd** commands require sudo privileges. To delete a user, use the **userdel** command:

```
userdel username
```

Linux Commands for System Information and Management

This section will list common Linux commands for querying system information and management.

31. df command

Use the **df** command to check a Linux system's disk space usage in percentage and kilobyte (**KB**). Here's the syntax:

```
df [options] [file]
```

If you don't specify the item, this command will display information about every mounted file system. These are some acceptable options:

- **-m** – displays information on the file system usage in **MBs**.
- **-k** – prints file system usage in **KBs**.
- **-T** – shows the file system **type** in a new column.

32. du command

Use **du** to check a file or directory's storage consumption. Remember to specify the directory path when using this command, for example:

```
du /home/user/Documents
```

The **du** command has several options, such as:

- **-s** – shows the specified folder's total size.
- **-m** – provides folder and file information in **MB**.
- **-k** – displays information in **KB**.
- **-h** – informs the displayed folders and files' last modification date.

33. top command

The **top** command displays running processes and the system's real-time condition, including resource utilization. It helps identify resource-intensive processes, enabling you to disable them easily.

To run the command, enter **top** into your command-line interface.

34. htop command

The **htop** command is an interactive program for monitoring system resources and server processes. Unlike **top**, it offers additional features like mouse operation and visual indicators. Here's the command syntax:

```
htop [options]
```

It supports options such as:

- **-d** – shows the delay between updates in tenths of seconds.
- **-C** – enables monochrome mode.
- **-h** – displays the help message and exits.

35. ps command

The **ps** command creates a snapshot of all running processes in your system. Executing it without an option or argument will list the running processes in the shell with the following information:

- Unique process ID (**PID**).
- Type of the terminal (**TTY**).
- Running time (**TIME**).
- Command that launches the process (**CMD**).

The **ps** command accepts several options, including:

- **-T** – displays all processes associated with the current shell session.
- **-u username** – lists processes associated with a specific user.
- **-A** – shows all the running processes.

Suggested Reading

Learn more about using **top**, **htop**, and **ps** commands to check running processes in a Linux system.

36. uname command

The **uname** or **unix name** command prints information about your machine, including its hardware, system name, and Linux kernel. Here's the basic syntax:

```
uname [option]
```

While you can use it without an option, add the following to modify the command:

- **-a** – prints all the system information.

- **-s** – outputs the kernel name.
- **-n** – shows the system's node hostname.

```
root@srv:/# uname -asn
Linux srv      5.15.0-1038-kvm #43-Ubuntu SMP Mon Jul 17 19:45:17 UTC 2023 x86_
64 x86_64 x86_64 GNU/Linux
```

37. hostname command

Run the **hostname** command to display the system's hostname. Here's the syntax:

```
hostname [option]
```

You can run it without an option or use the following:

- **-a** – displays the hostname's alias.
- **-A** – shows the machine's Fully Qualified Domain Name (FQDN).
- **-i** – outputs the machine's IP address.

```
root@srv:/# hostname
srv
```

38. time command

Use **time** to measure commands' execution time. Here's the syntax:

```
time [commandname]
```

To measure a series of commands, separate them using semicolons or double ampersands (**&&**). For example, we will measure **cd**, **touch**, and **chmod** commands' overall execution time:

```
time cd /home/directory/path; touch bashscript.sh; chmod +x bashscript.sh
```

39. systemctl command

The **systemctl** command lets you manage installed services in your Linux system. Here's the basic syntax:

```
systemctl [commandname] [service_name]
```

To use the command, the user must have **root** privilege. It has several use cases, including starting, restarting, and terminating a service. You can also check a service's status and dependencies.

The **systemctl** command is only available in Linux distributions with the **Systemd init system**. Check our article on listing and managing Linux services to learn more about other systems' commands.

40. watch command

The **watch** command lets the user continuously run another utility at a specific interval and print the results as a standard output. Here's the syntax:

```
watch [option] command
```

It is useful for monitoring command output changes. To modify its behavior, use the following options:

- **-d** – displays the differences between command executions.
- **-n** – changes the default two-second interval.
- **-t** – disables the header containing the time interval, command, timestamp, and hostname.

41. jobs command

The **jobs** command displays a shell's running processes with their statuses. It is only available in **csh**, **bash**, **tcsh**, and **ksh** shells. Here's the basic syntax:

```
jobs [options] jobID
```

To check the status of jobs in the current shell, enter **jobs** without any arguments in Terminal. The command will return an empty output if your system doesn't have running jobs. You can also add the following options:

- **-l** – lists process IDs and their information.
- **-n** – shows jobs whose statuses have changed since the last notification.
- **-p** – displays process IDs only.

42. kill command

Use the **kill** command to terminate an unresponsive program using its identification number (PID). To check the PID, run the following command:

```
ps ux
```

To stop the program, enter the syntax below:

```
kill [signal_option] pid
```

There are 64 signals for terminating a program, but **SIGTERM** and **SIGKILL** are the most common. **SIGTERM** is the default signal that lets the program save its progress before stopping. Meanwhile, **SIGKILL** forces programs to stop and discard unsaved progress.

43. shutdown command

The Linux **shutdown** command lets you turn off or restart your system at a specific time. Here's the syntax:

```
shutdown [option] [time] "message"
```

You can use an absolute time in a 24-hour format or a relative one like **+5** to schedule it in five minutes. The **message** is a notification sent to logged-in users about the system shutdown.

Instead of shutting down, restart the system using the **-r** option. To cancel a scheduled reboot, run the command with the **-c** option.

Linux Commands for Network Management and Troubleshooting

Here are commonly used Linux commands for managing and troubleshooting network connections.

44. ping command

The **ping** command is one of the most used commands in Linux. It lets you check whether a network or server is reachable, which is useful for troubleshooting connectivity issues. Here's the syntax:

```
ping [option] [hostname_or_IP_address]
```

For example, run the following to check the connection and response time to **Google**:

```
ping google.com
```

45. wget command

Use the **wget** command to download files from the internet using **HTTP**, **HTTPS**, or **FTP** protocols. Here's the syntax:

```
wget [option] [url]
```

For example, enter the following to download the latest version of WordPress:

```
wget https://wordpress.org/latest.zip
```

46. curl command

The **curl** command transfers data between servers. Its common usage is for retrieving a web page's content to your system using its URL. Here's the syntax:

```
curl [option] URL
```

However, you can add various options to modify the **curl** command behavior for other tasks. Some of the most popular ones include:

- **-o** or **-O** – downloads files from a URL.
- **-X** – changes the default HTTP GET method.
- **-H** – sends a custom header to the URL.
- **-F** – uploads a file to the specified destination.

47. scp command

The **scp** command securely copies files or directories between systems over a network. Here's the syntax:

```
scp [option] [source username@IP]:/[directory and file name] [destination username@IP]:/[destination directory]
```

For a local machine, omit the hostname and IP address. Use the following options to modify the copying behavior:

- **-P** – changes the port for copying. The default is **22**.
- **-l** – limits the **scp** command's bandwidth.
- **-C** – compresses transferred data to make it smaller.

48. rsync command

The **rsync** command lets you sync files or folders between two destinations to ensure they have the same content. Here's the syntax:

```
rsync [options] source destination
```

If your destination or source is a folder, enter the directory path like **/home/directory/path**. To sync a remote server, use its hostname and IP address, like **host@185.185.185.185**.

This command has various options:

- **-a** – enables archive mode to preserve file permissions, dates, and other attributes.
- **-v** – shows visual information about the transferred file.
- **-z** – compresses the transferred file data to reduce their size.

49. ifconfig command

The **ifconfig** command lets you list and configure your system's network interface. In newer Linux distros, it is equivalent to the **ip** command. Here's the basic syntax:

```
ifconfig [interface] [option]
```

Running it without arguments displays information about all network interfaces in your system. To check a specific interface, add its name as an argument without an option. For a more specific task, use the following options:

- **-s** – summarizes the network interfaces and their configuration. This option goes before the interface name.
- **up** and **down** – enables and disables a network interface.
- **inet** and **inet6** – assigns an IPv4 and IPv6 address to a network interface.
- **netmask** – specifies the subnet mask to use with an IPv4 address.

```
root@srv:/# ifconfig -s eth0
Iface      MTU      RX-OK RX-ERR RX-DRP RX-OVR      TX-OK TX-ERR TX-DRP TX-OVR Flg
eth0       1500 15712059      0      0 0      397819      0      0      0 BMRU
```

50. netstat command

The **netstat** command is used to display your system's network information, like sockets and routing. Here's the command syntax:

```
netstat [option]
```

Use various options to modify the displayed information. Some common ones are:

- **-a** – displays listening and closed sockets.
- **-t** – shows TCP connections.
- **-u** – lists UDP connections.
- **-r** – displays routing tables.
- **-i** – shows information about network interfaces.
- **-p** – lists programs' names and process IDs.
- **-c** – continuously outputs network information for real-time monitoring.

```
root@srv:/# netstat -i
Kernel Interface table
Iface      MTU      RX-OK RX-ERR RX-DRP RX-OVR      TX-OK TX-ERR TX-DRP TX-OVR Flg
eth0       1500 15713216      0      0 0      397897      0      0      0 BMRU
lo         65536      5942      0      0 0      5942      0      0      0 LRU
```


51. traceroute command

The **traceroute** command tracks a packet's path when it moves to another host over a network. It gives you information about the involved routers and travel time. Here's the syntax:

```
traceroute [option] destination
```

You can use a domain, hostname, or IP address as the destination. Add the following options for more detailed packet monitoring:

- **-m** – sets each packet's maximum hops.
- **-n** – prevents the command from resolving IP addresses to hostnames for quicker tracing.
- **-I** – changes the default UDP packets to ICMP.
- **-w** – adds a timeout in seconds.

```
root@srv:/# traceroute -m 5 google.com
traceroute to google.com (142.251.42.46), 5 hops max, 60 byte packets
 1 _gateway (169.254.1.1) 0.333 ms 0.298 ms 0.282 ms
 2 * * *
 3 * * *
 4 43.241.1.1 (43.241.1.1) 0.860 ms 0.686 ms 43.241.1.1 (43.241.1.1) 0.961 ms
 5 59.163.29.81.static.vsnl.net.in (59.163.29.81) 1.476 ms 2.545 ms 6.596 ms
```

52. nslookup command

The **nslookup** command queries a DNS server to find out the domain associated with an IP address and vice versa. Here's the syntax:

```
nslookup [options] domain-or-ip [server]
```

If you don't specify the DNS server to use, **nslookup** will use the default resolver from your system or internet service provider. This command supports various options, with some commonly used ones being:

- **-type=** – queries specific information, like the IP address type or MX record.
- **-port=** – sets the DNS server's port number for the query.
- **-retry=** – repeats the query a specific number of times upon failure.
- **-debug** – enables the debug mode to provide more information about the query.

```
root@srv:/# nslookup google.com
Server:          127.0.0.53
Address:         127.0.0.53#53

Non-authoritative answer:
Name:   google.com
Address: 142.251.42.46
Name:   google.com
Address: 2404:6800:4009:830::200e
```

53. dig command

The **dig** or **domain information groper** command gathers DNS data from a domain. Unlike **nslookup**, it is more detailed and versatile. Here's the syntax:

```
dig [option] target [query_type]
```

Replace **target** with a domain name. By default, this command only shows **A** record type. Change **query_type** to check a specific type or use **ANY** to query all of them. To run a reverse DNS lookup, add the **-x** option and use the IP address as the target.

Miscellaneous Linux Commands

In this section, we will list Linux commands with various functions.

54. history command

Enter **history** to list previously executed commands. It lets you reuse the commands without rewriting them. To use it, enter this syntax with sudo privileges:

```
history [option]
```

```
root@srv:/# history
 1  sudo apt-get install node
 2  sudo apt-get install npm
 3  sudo apt-get install nodejs
 4  mkdir nodejsapp
```

To rerun a specific utility, enter an exclamation mark (!) followed by the command's list number. For example, use the following to rerun the **255th** command:

```
!255
```

This command supports many options, such as:

- **-c** – clears the history list.
- **-d offset** – deletes the history entry at the **OFFSET** position.
- **-a** – appends history lines.

55. man command

The **man** command provides a user manual of any Linux Terminal utilities, including their names, descriptions, and options. It consists of nine sections:

- Executable programs or shell commands
- System calls
- Library calls
- Games
- Special files
- File formats and conventions
- System administration commands
- Kernel routines
- Miscellaneous

Here's the command syntax:

```
man [option] [section_number] command_name
```

If you only use the command name as the parameter, Terminal displays the full user manual. Here's an example command to query section **1** of the **ls** command manual:

```
man 1 ls
```

```
root@srv:/# man 1 ls
LS(1)                                User Commands                                LS(1)

NAME
    ls - list directory contents

SYNOPSIS
    ls [OPTION]... [FILE]...

DESCRIPTION
    List information about the FILES (the current directory by default).
    Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-
    fied.

    Mandatory arguments to long options are mandatory for short options
    too.
```

56. echo command

The **echo** command displays a line of text as a standard output. Here's the basic command syntax:

```
echo [option] [string]
```

For example, you can display **Hostinger Tutorials** by entering:

```
echo "Hostinger Tutorials"
```

```
root@srv:/# echo "Hostinger Tutorials"
Hostinger Tutorials
```

This command supports many options, such as:

- **-n** – displays the output without the trailing newline.
- **-e** – enables the interpretation of the following backslash escapes:
- **\b** – removes spaces in between a text.
- **\c** – produces no further output.

57. ln command

The **ln** command lets you create links between files or directories to simplify system management. Here's the syntax:

```
ln [option] [source] [destination]
```

The command will create the target file or directory and link it to the source. By default, it creates a hard link, meaning the new item connects to the same data block as the source.

58. alias, unalias commands

The **alias** command instructs the shell to replace a string with another, allowing you to create a shortcut for a program, file name, or text. Here's the syntax:

```
alias name=string
```

For example, enter the following to make **k** the alias for the **kill** command:

```
alias k='kill'
```

This command doesn't give any output. To check the alias associated with a command, run the following:

```
alias command_name
```

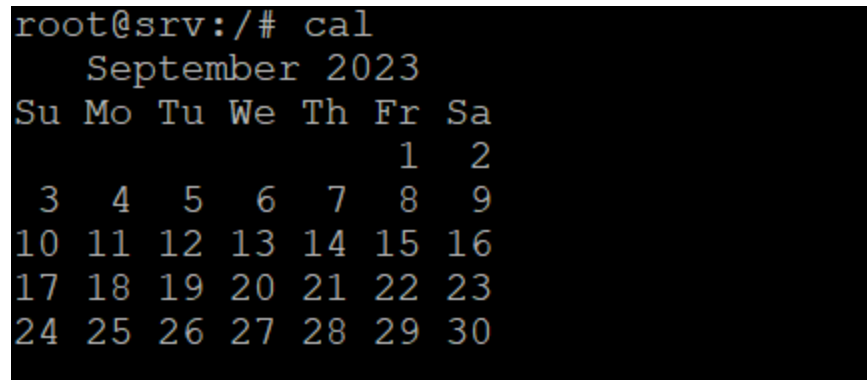
To delete an existing alias, use the **unalias** command with the following syntax:

```
unalias [alias_name]
```

59. cal command

The **cal** command outputs a calendar in the Linux Terminal. It will show the current date if you don't specify the month and year. Here's the syntax:

```
cal [option] [month] [year]
```



```
root@srv:/# cal
    September 2023
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
```

The month is in the numerical representation from **1–12**. To modify the command output, add the following options:

- **-1** – outputs the calendar in a single line.
- **-3** – shows the previous, current, and next month.
- **-A** and **-B** – displays the specified number of months after and before the current one.
- **-m** – starts the calendar with Monday instead of Sunday.

60. apt-get command

apt-get is a command line tool for handling Advanced Package Tool (APT) libraries in Debian-based Linux, like Ubuntu. It requires **sudo** or **root** privileges.

This Linux command lets you manage, update, remove, and install software, including its dependencies. Here's the main syntax:

```
apt-get [options] (command)
```

These are the most common commands to use with **apt-get**:

- **update** –synchronizes the package files from their sources.
- **upgrade** –installs the latest version of all installed packages.
- **check** – updates the package cache and checks broken dependencies.

```
root@srv:/# apt-get update
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Hit:3 https://dl.yarnpkg.com/debian stable InRelease
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Hit:5 https://d17k9fuiwb52nc.cloudfront.net jammy InRelease
Get:6 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [
```

Linux Commands Tips and Tricks

Here are some tips for using Linux commands and Terminal to improve your system management efficiency:

- Add the **-help** option to list the full usage of a command.
- Use the **exit** command to close Terminal.
- Enter the **clear** command to clean the Terminal screen.
- Press the **Tab** button to autofill after entering a command with an argument.
- Use **Ctrl + C** to terminate a running command.
- Press **Ctrl + Z** to pause a working command.
- Use **Ctrl + A** to move to the beginning of the line.
- Press **Ctrl + E** to bring you to the end of the line.
- Separate multiple commands using **semicolons (;)** or **double ampersands (&&)**.

Other Useful Linux Commands to Learn

[How to Configure and Manage Ubuntu Firewall with the UFW Command](#)

[How to Install and Use the Linux Screen Command](#)

[How to List Users in Ubuntu with Linux Commands](#)

[How to Change Password in Linux](#)

Use **compgen -c**, **help**, or **man -k** . to list all available Linux commands in your system. To check a command usage and its options, use **-help**. For example, enter **apt -help** to check the available options for the **apt** command.

How Do I Open Terminal in Linux?

In a GUI-based Linux, Terminal is located in the **Utility** menu or accessible using the **Ctrl + Alt + T** shortcut. For Linux servers, Terminal is opened by default after you log in to the system.

What Does the Syntax Command -option Argument Mean?

It is the typical order of a command-line utility in Unix-like operating systems like Linux. **Command** refers to the program you want to run, and **-option** modifies its behavior. Meanwhile, an argument is the input data the utility is operating on.

What Do [...] and <...> Mean in Command Usage?

In Linux command syntax, both [...] and <...> enclose placeholders. While [...] is used for an optional parameter, <...> is for required components. For example, **options** are enclosed in square brackets, and **input files** are in angular ones.

Why Am I Getting a “Permission Denied” Error?

Permission denied means the command requires **root** or **superuser** privilege to run. To gain the privilege, add **sudo** at the beginning of each command or switch to the superuser using **sudo su**. If the error is related to files or directories, change their permissions using the **chmod** command.

Linux Beginner Level Interview Questions and Answers:

1.What are the Distributions in Linux?

Ans). Linux distributions offer a flexible and open-source alternative to proprietary operating systems. Users can select from a wide variety of distros based on their specific needs, whether they require a stable server environment or a user-friendly desktop experience. The diversity in distributions ensures that there is an option available for virtually every computing requirement

Distribution	Description	Target Audience
Ubuntu	User-friendly, widely used	General users
Fedora	Cutting-edge technology testing	Developers
CentOS	Community-supported RHEL variant	Servers
Debian	Known for stability	General users
Kali Linux	Security-focused	Penetration testers
Arch Linux	Customizable, rolling release	Experienced users

2.What is the Linux Directory Structure?

Ans). **/**: The root directory, the top-level directory for the entire file system.

Key Directories

- **/bin**: Essential command binaries for all users.
- **/sbin**: System administration binaries, typically for the root user.
- **/etc**: Configuration files for system-wide settings.
- **/dev**: Device files representing hardware components.
- **/proc**: Virtual filesystem providing process and kernel information.
- **/var**: Variable data files, such as logs and mail spools.
- **/usr**: User-related programs, libraries, and documentation.
- **/tmp**: Temporary files used by applications.
- **/home**: User home directories for personal files.
- **/opt**: Optional software packages not included in the standard distribution.

3. what Is Inode value?

Ans). An **inode** (short for "index node") is a crucial data structure in Linux and UNIX-based file systems that stores metadata about files and directories.

Each file and directory is assigned a unique inode number, which serves as an identifier within the filesystem.

Key Characteristics of Inodes

- **Metadata Storage:** Inodes contain important information such as:
 - File type (regular file, directory, etc.)
 - File size
 - Ownership (user ID and group ID)
 - Permissions (read, write, execute)
 - Timestamps (last access, last modification, last change)
- **Unique Identification:** Each inode number is unique within a filesystem but can be reused across different filesystems. The combination of the filesystem ID and the inode number creates a globally unique identifier.
- **File Access:** When accessing a file, the system uses the inode number to locate the corresponding metadata and determine where the actual data blocks are stored on disk.

4. explain about commands **awk**, **sort**, **grep**, **umask** and **chmod**?

Ans). **awk:** The **awk** command in Linux is a powerful text processing tool used for pattern scanning and processing. It allows users to manipulate and analyze text files, making it especially useful for tasks involving data extraction and reporting.

Ex: `$awk '/pattern/' filename`

`$ awk '{print $1}' filename`

`$ awk '{sum += $1} END {print sum}' filename`

`$ awk -F: '{print $1}' /etc/passwd`

Sort: The **sort** command in Linux is a powerful utility used for sorting lines of text files or input streams in a specified order. It enhances data organization and readability, making it easier to analyze and filter information.

Common Options

- **-b:** Ignore leading blanks when determining sort keys.
- **-c:** Check if the input is already sorted; output any unsorted lines.
- **-f:** Perform case-insensitive sorting.
- **-n:** Sort numerically, treating the contents as numbers.
- **-r:** Sort in reverse order.
- **-k:** Specify which column to sort by (e.g., `-k2` to sort by the second column).

Ex: `$sort filename.txt`

`$ sort -n numbers.txt`

`$ sort -k2 employee_data.txt (sorts the lines based on the column)`

`$ sort -u filename.txt (this will remove the duplicate lines)`

\$ sort -c filename.txt (check the file is sorted and outputs any unsorted lines)

\$ sort file1.txt file2.txt (concatenates and sorts the contents from both the files)

\$ sort -M months.txt (sorts based on the month names)

grep: The grep command in Linux is a powerful utility used for searching and matching text patterns within files. Its name stands for "global regular expression print," and it is widely used for tasks such as filtering output, searching logs, and extracting specific information from text files.

Common Options

- -i: Ignore case distinctions (case-insensitive search).
- -v: Invert the match; show lines that do not match the pattern.
- -r or -R: Recursively search through directories.
- -n: Show line numbers of matching lines.
- -c: Count the number of matching lines.
- --color: Highlight matching strings in the output.

Ex: \$grep "error" /var/log/syslog

\$ grep -r "config" /etc/

umask: The umask (user file-creation mode mask) is a critical command in Linux that determines the default permissions for newly created files and directories. It specifies which permissions should not be granted by default, effectively masking certain permission bits.

chmod: The **chmod** command in Linux is used to change the permissions of files and directories, allowing users to control who can read, write, or execute a file.

5. what is the difference when changing permissions using symbolic notation and octal(numeric) notation?

Ans)

Feature	Symbolic Notation	Numeric (Octal) Notation
Format	Letters (e.g., u+x)	Octal numbers (e.g., 755)
Flexibility	Allows partial modifications	Sets all specified permissions
Complexity	More descriptive but longer	Shorter but less descriptive
Common Use Cases	Fine-grained control	Quick permission setting

6.What is the difference between **cp** and **mv**?

Ans) cp (Copy): The cp command is used to create a duplicate of a file or directory at a specified location. The original file remains intact after the operation. ideal for creating backups or duplicates of files and directories.

mv (Move): The mv command is used to move a file or directory from one location to another or to rename it. When using mv, the original file is removed from its source location. Useful for organizing files by moving them into different directories or renaming them

7. **chown** and **chgrp**?

Ans) **chown:** The chown command in Linux is used to change the ownership of files and directories.

chgrp: The chgrp command in Linux is used to change the group ownership of files and directories.

8. what are commands **find** and **sed** used for?

Ans) **find:** The find command is used to search for files and directories within a specified directory hierarchy based on various criteria. It can search based on file names, types, sizes, modification times, permissions, and more.

Sed: The sed (stream editor) command is used for parsing and transforming text in a pipeline. It is commonly used for text substitution, deletion, and insertion.

EX: \$sed 's/old_text/new_text/' filename (this replaces the old text with new text in the specified file)

\$ find /path/to/search -type f -name "*.txt" -exec sed -i 's/old_text/new_text/g' {} (This command finds all .txt files in the specified directory and replaces "old_text" with "new_text" in each file.)

9. **wget** & **curl** commands?

Ans) **wget:** Primarily designed for downloading files from the web, including entire websites. It excels in recursive downloads and is user-friendly for basic file retrieval tasks.

curl: A versatile tool for transferring data to and from servers using a wide range of protocols. It is often used for API interactions and supports more complex operations.

Supported Protocols:

- **wget:** Supports HTTP, HTTPS, and FTP protocols.
- **curl:** Supports a broader range of protocols, including HTTP, HTTPS, FTP, FTPS, SCP, SFTP, TFTP, LDAP, and more

10. what is **crontab**?

ans) **Crontab** (short for "cron table") is a command-line utility in Unix-like operating systems, including Linux, that allows users to schedule tasks (known as cron jobs) to be executed automatically at specified times or intervals. It is an essential tool for automating repetitive tasks such as backups, system maintenance, and running scripts.

Syntax: MIN HOUR DOM MON DOW CMD

Ex: 0 0 * * * /path/to/script.sh

11. **tee** command?

Ans) The **tee** command in Linux is a utility that reads from standard input and writes the output to both standard output (the terminal) and one or more specified files simultaneously. This command is particularly useful for capturing the output of commands while still displaying it on the screen

Ex: \$df -h | tee disk_usage.txt (this command shows the disk usages and saves the output in file)

\$echo "Hello, World!" | tee file1.txt file2.txt (This writes "Hello, World!" to both file1.txt and file2.txt.)

12) how to check stats on server?

Ans) **top**: Displays real-time information about system processes, including CPU and memory usage.

Vmstat: Provides information about virtual memory, processes, CPU activity, and I/O statistics.

Ex: \$vmstat 2 10 (updates every 2 seconds for 10 reports)

iostat: Monitors system input/output device loading by observing the time devices are active relative to their average transfer rates.

Ex: iostat [interval] [count]

netstat: Provides network statistics, including active connections and listening ports.

Other commonly used commands are: **du -sh**, **df -h**, **netstat -tuln**, **free -h**.



Linux Networking Commands

1. ifconfig

- Display network interface information.

```
Ifconfig
```

2. ip

- Show/manipulate routing, devices, policy routing, and tunnels.

```
ip address show
```

3. route

- Display or manipulate the IP routing table.

```
route -n
```

4. ping

- Send ICMP ECHO_REQUEST to network hosts.

```
ping google.com
```

5. traceroute

- Print the route packets trace to network host.

```
traceroute google.com
```

6. netstat

- Print network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.

```
netstat -an
```

7. ss

- Display socket statistics.

```
ss -tulpn
```

8. hostname

- Show or set the system's host name.

```
Hostname
```

9. dig

- DNS lookup utility.

```
dig google.com
```

10. nslookup

- Query Internet name servers interactively.

```
nslookup google.com
```

11. route

- Manipulate routing tables.

```
route add default gw 192.168.1.1
```

12. iptables

- Administration tool for IPv4 packet filtering and NAT.

```
iptables -L
```

13. tcpdump

- Dump traffic on a network.

```
tcpdump -i eth0
```

14. sshd

- OpenSSH daemon.

```
service sshd restart
```

15. telnet

- User interface to the TELNET protocol.

```
telnet google.com 80
```

16. scp

- Secure copy (remote file copy program).

```
scp file.txt user@remote:/path/to/destination
```

17. wget

- Non-interactive network downloader.

```
wget http://example.com/file.zip
```

18. curl

- Command line tool for transferring data with URL syntax.

```
curl http://example.com/api
```


19. iptraf

- Interactive color IP LAN monitor.

Iptraf

20. iftop

- Display bandwidth usage on an interface.

Iftop

21. nmap

- Network exploration tool and security scanner.

nmap -sP 192.168.1.0/24

22. lsof

- List open files.

lsof -i :80

23. ethtool

- Display or change ethernet card settings.

ethtool eth0

24. arp

- Display or modify the ARP cache.

```
arp -a
```

25. route

- Display or modify the IP routing table.

```
Route
```

26. ss

- Display socket statistics.

```
ss -s
```

27. hostnamectl

- Control the system hostname and related settings.

```
hostnamectl status
```

28. resolvconf

- Manage DNS information.

```
resolvconf -u
```

29. mtr

- Network diagnostic tool.

```
mtr google.com
```

30. iwconfig

- Configure a wireless network interface.

```
Iwconfig
```

31. nc

- Arbitrary TCP and UDP connections and listens.

```
nc -l 8080
```

32. scp

- Copy files between hosts on a network.

```
scp file.txt user@host:/path/to/destination
```

33. ssh-keygen

- Generate, manage, and convert authentication keys for ssh.

```
ssh-keygen -t rsa
```

34. ss

- Show socket statistics.

```
ss -t -a
```

35. tcpdump

- Capture and display packets on a network.

```
tcpdump -i eth0 tcp port 80
```

36. route

- Add a new route.

```
route add -net 192.168.2.0 netmask  
255.255.255.0 gw 192.168.1.1
```

37. nmcli

- Command-line client for NetworkManager.

```
nmcli connection show
```

38. dig

- Perform DNS lookups.

```
dig +short A google.com
```

39. nload

- Visual representation of incoming and outgoing traffic.

```
Nload
```

40. iperf

- Tool for measuring TCP and UDP bandwidth performance.

```
iperf -c server_ip
```

41. fping

- Quickly ping multiple hosts.

```
fping -a -g 192.168.1.1 192.168.1.254
```

42. iftop

- Real-time console-based network bandwidth monitoring tool.

```
iftop -n
```

43. route

- Delete a route.

```
route del -net 192.168.2.0 netmask  
255.255.255.0
```

44. tcpdump

- Capture and display packets in ASCII.

```
tcpdump -A -i eth0
```

45. netcat

- Utility for reading from and writing to network connections.

```
nc -zv 192.168.1.1 22
```

46. nmtui

- Text User Interface for controlling NetworkManager.

```
Nmtui
```

47. ethtool

- Change the speed/duplex settings of an Ethernet device.

```
ethtool -s eth0 speed 100 duplex full
```

48. ss

- Show listening sockets.

```
ss -l
```

49. host

- DNS lookup utility.

```
host google.com
```

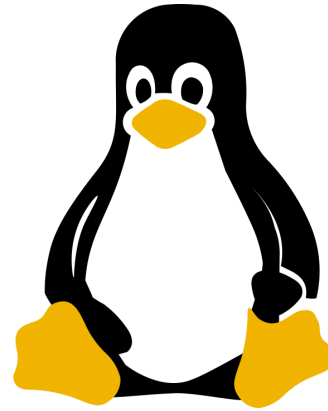
50. nmcli

- List available Wi-Fi networks.

```
nmcli device wifi list
```

These commands cover a wide range of networking tasks and can be useful for troubleshooting, monitoring, and managing network configurations in a DevOps environment.

Linux™



200+ Linux Technical Interview Questions:

1. Give any 3 examples of operating systems.
 - Windows, Linux, and macOS.
2. What is the root home directory?
 - The root home directory is `/root`.
3. Your company has terminated a server administrator. What is the first thing as an administrator you should do to enhance the security?
 - Change the root password.
4. How to check kernel version?
 - Use the command `uname -a`.
5. How to become a root user from a regular user?
 - Use `su -` or `sudo su` to switch to the root user.
6. How to check the computer name or hostname in Linux?
 - Use the `hostname` command.
7. List 3 basic commands to navigate the filesystem.
 - `cd`, `pwd`, and `ls`.
8. How to check network interfaces in Linux?
 - Use `ifconfig` or `ip addr` or `ip a`
9. List 3 different methods of adding a user.
 - Methods include using `useradd`, `adduser`, or manually editing `/etc/passwd` and `/etc/shadow` files.
10. What is the command to create a new user?
 - `useradd username`

11. How to change a user password?

- Use the command `passwd username`.

12. Which directory has all the configuration files?

- The `/etc` directory.

13. List any 4 Linux distributions.

- Kali, Red Hat, CentOS, Ubuntu, and SUSE.

14. How to log off from the system?

- Use the `exit` command.

15. How to create a directory?

- Use the `mkdir` command.

16. Explain the purpose of the “touch” command.

- `touch` is used to create an empty file or update the timestamp of an existing file.

17. If a command hangs, how to stop it and get the prompt back?

- Press `Ctrl + C`.

18. Which command is used to count words or lines?

- `wc`

19. How to rename a file or directory?

- Use the `mv` command.

20. How to change a hostname in Linux?

- Edit `/etc/hostname` and `/etc/hosts`, then restart the system or use `hostnamectl set-hostname newname`.

21. What is the command to change file/directory permissions?

- Use `chmod`.

22. What is the purpose of pipe (`|`)?

- To pass the output of one command as input to another command.

23. What is `/etc` directory used for?

- It contains configuration files for the system.

24. Which command is used to list files in a directory?

- `ls -l`

25. There is a command which gives you information about other commands, please explain that command and what is it used for?

- `man` is used to display the manual pages for other commands, providing detailed information on usage and options.
26. How to delete a file and a directory?
- Use `rm filename` for files and `rmdir dirname` for directories.
27. What is the difference between “tail” and “tail -10”?
- `tail` displays the last 10 lines of a file by default, while `tail -10` explicitly specifies to display the last 10 lines.
28. List 4 commands to display or read a file's contents.
- `cat`, `more`, `less`, `vi`.
29. Which command is used to read the top 5 lines of a file?
- `head -5 filename`
30. What are the different commands or methods to write to a file?
- `echo > filename`, `vi filename`, `cat > filename`.
31. What are the different types of shells?
- Sh, bash, ksh, csh, zsh
32. Which is the core of the operating system?
- Kernel
33. Which among the following interacts directly with system hardware?
- Kernel
34. List a few commands that are used in troubleshooting network-related issues?
- `netstat`: Displays network connections.
 - `tcpdump`: Captures and analyzes network traffic.
 - `ping`: Tests connectivity to another host.
 - `traceroute`: Traces the route packets take to a network host.
35. How to combine 2 files into 1? E.g., you have 3 lines in file “A” and 5 lines in file “B”, which command syntax to use that will combine into one file of 3+5 = 8 lines?*
- `cat fileA fileB > combinedfile`
36. What is the echo command used for?
- The ``echo`` command is used to display a line of text or output a string to the screen.
37. What does the following command do?

- `echo "This year the summer will be great" > file1`
 - It creates a new file named "file1" and writes the text "This year the summer will be great" into it. If the file already exists, it will be overwritten.
38. When you login you get "\$" prompt, what is the prompt for root?
- The prompt for the root user is typically #.
39. Explain the difference between grep and egrep.
- `grep` is used for basic regular expression pattern matching, while `egrep` (or `grep -E`) supports extended regular expressions, which allow more complex pattern matching.
40. What is the port number for DNS, NTP, and NFS?
- DNS: 53, NTP: 123, NFS: 111 (portmapper) and 2049 (NFS).
41. What is the configuration file name of DNS and where is it located?
- The DNS configuration file is named `named.conf` and is typically located in `/etc`.
42. How many new directories will be created after running the following command `mkdir {a..c}{1..3}`?
- 9 directories will be created: a1, a2, a3, b1, b2, b3, c1, c2, c3.
43. Your PC is configured with a DNS server address but not the default gateway. Can the PC access the internet?
- No, without a default gateway, the PC cannot access the internet.
44. What is the difference between IP and Gateway?
- An IP address is a unique identifier for a device on a network, while a gateway is a network point that acts as an entrance to another network, often used to connect local networks to external networks like the internet.
45. Can you assign one static IP to 2 computers, if not then why?
- No, assigning the same static IP to two computers will create an IP conflict, causing network communication issues.
46. How to change IP address to static?
- You can change the IP address to static by editing the network configuration files, such as `/etc/network/interfaces` on Debian-based

systems or `/etc/sysconfig/network-scripts/ifcfg-eth0` on Red Hat-based systems, and setting the IP address manually.

- `etc/netplan` for the network configuration file on Ubuntu.

47. You are trying to ping a server by hostname and you get an error message, “ping: unknown host ...”. What could be the reason and how to solve the problem so you can ping it by hostname?

- The issue could be due to a missing hostname-to-IP mapping in the `/etc/hosts` file or an incorrect DNS configuration. Check and update these files to resolve the issue.

48. Explain the difference between relative and absolute path.

- An absolute path starts from the root directory `/` and specifies the complete path to a file or directory. A relative path is based on the current directory and does not start with `/`.

49. What is the command to change file/directory ownership and group?

- Use `chown` to change ownership and `chgrp` to change the group.

50. List any 3 types of filesystem.

- ext4, NTFS, and FAT.

51. When you login you get a message on the screen. What is the name of that file and where is it located?

- The file is `/etc/motd` (Message of the Day).

52. What is the `/bin` directory used for?

- The `/bin` directory contains essential binary executables needed for booting and system repair.

53. What are the different types of DNS Server?

- Master (Primary) and Secondary (Slave) DNS servers.

54. Where are the zone files located for DNS service?

- Typically located in `/var/named/zonefiles`.

55. How many megabytes in 1 gigabyte?

- There are 1024 megabytes in 1 gigabyte.

56. What is the purpose of having different network ports?

- So the communication of each application goes through a dedicated port

- Different network ports allow multiple applications to communicate over the network simultaneously without interference.

57. How to display the first column of a file?

- Use `cat filename | awk '{print $1}'`
- `awk '{print $1}' filename.`

58. What is the name of the DNS rpm package?

- The package is called `bind`.

59. What is the difference between `nslookup` and `dig` commands?

- `nslookup` is a simple tool for querying DNS servers, while `dig` provides more detailed information and is more flexible for DNS troubleshooting.

60. How to check your user id and group id?

- Use the `id` command.

61. What is the difference between “kill” and “kill -9” command?

- `kill` sends a signal to terminate a process gracefully, allowing it to clean up resources. `kill -9` sends a SIGKILL signal, forcing the process to terminate immediately without cleanup.

62. What is a subnet?

- A subnet is a segmented piece of a larger network, designed to improve performance and security by grouping devices with similar network requirements.

63. You are troubleshooting an issue with Redhat support and they have asked you to send the contents of `/etc` directory. How and which method will you use to transfer the contents?

- Compress the `/etc` directory using `tar` and transfer it using `ftp` or `scp`.

64. What is `rsyslogd` daemon and its purpose?

- `rsyslogd` is a system utility providing support for message logging. It is an enhanced version of `syslogd`.

65. What is the command to untar a tarred file?

- Use `tar -xvf filename.tar`.

66. What is the `/proc` directory used for?

- The `/proc` directory contains virtual files that provide a view into the kernel's view of the system.

67. What is the purpose of the `nsswitch.conf` file?

- It specifies the sources from which to obtain name-service information in a range of categories and in what order.

68. Which service/daemon should be running on the server that allows you to connect remotely?

- `sshd` (SSH Daemon).

69. What is the purpose of a firewall?

- A firewall controls incoming and outgoing network traffic based on predetermined security rules.

70. List any 3 IT components.

- Hardware, Operating System, and Applications.

71. Which directory has all the commands we use, e.g., `ls`, `cd`, etc.?

- `/usr/bin` or `/bin`.

72. What is the difference between memory, virtual memory, and cache?

- Memory (RAM) is the physical hardware inside a computer that temporarily stores data.
- Virtual memory is a memory management capability that uses disk space as an extension of RAM.
- Cache is a smaller, faster memory component that stores copies of frequently accessed data for quick access.

73. Correct order of interaction:

- a. User >> Operating System >> Hardware

74. Which of the following is a communication command?

- `mail`

75. Why is the “`tail -f logfilename`” command used most often and what does it do?

- It outputs all incoming logs in real-time, useful for monitoring log files as they are updated.

76. How to sort a file in reverse order?

- Use `sort -r filename`.

77. List all byte sizes from smallest to largest.

- Bit, Byte, Kilobyte (KB), Megabyte (MB), Gigabyte (GB), Terabyte (TB), Petabyte (PB), Exabyte (EB).

78. How to check the total number of partitions in Linux?

- Use `fdisk -l`.

79. How to access a Linux system from a Linux system?

- Use `ssh`.

80. Explain the procedure of bonding 2 NICs or interfaces together.

- Use network bonding to combine two or more NICs into a single bonded interface for redundancy or increased throughput. This typically involves configuring `/etc/network/interfaces` or `/etc/sysconfig/network-scripts/ifcfg-bond0` and using the `bonding` kernel module.

81. What is the exact command syntax to list the 5th column of a file and cut the first 3 letters?

- `awk '{print $5}' filename | cut -c1-3`

82. What is the `/etc/hosts` file used for?

- It is used to resolve hostnames to IP addresses locally.

83. List any 3 options of 'df' command and what they are used for.

- `-h`: human-readable format, `-i`: inodes information, `-T`: file system type.

84. What is swap space and how to check swap space?

- Swap space is a portion of the hard drive used as an extension of RAM. Check swap space using `swapon -s` or `free`.

85. What is inode and how to find an inode of a file?

- An inode is a data structure on a filesystem that stores information about a file or directory. Use `ls -li filename` to find the inode number.

86. Which file to edit for kernel tuning?

- Edit `/etc/sysctl.conf` for kernel parameter tuning.

87. What is the latest version of Redhat?

- Search online for the most recent version as it frequently updates.

88. Name the command to find a specific word from a file.

- `grep word filename`

89. You have scheduled a job using crontab but it does not run at the time you specified, what could be the reason and how would you troubleshoot?

- Check system time, crontab entry syntax, and `/var/log/cron` for errors.

90. How to check system hardware information?

- Use `dmidecode`.

91. How to check network interface MAC address?

- Use `ifconfig` or `ip link`.

92. If I don't want others to read my file1, how do I do that?

- Remove read permission for others using `chmod o-r file1`.

93. What is the purpose of "uniq" and "sed" commands?

- `uniq` removes duplicate lines from sorted input, `sed` is a stream editor for filtering and transforming text.

94. Which command is used to list the contents of a directory in the most recent time and in reverse order, meaning the most updated file should be listed on the bottom?

- `ls -ltr`

95. What is the difference between tar, gzip, and gunzip?

- `tar` is used for archiving files, `gzip` compresses files, and `gunzip` decompresses files.

96. What are the different ways to install an OS?

- Using a DVD, DVD ISO, or network boot.

97. How to view the difference between two files?

- Use `diff file1 file2`.

98. You noticed that one of the Linux servers has no disk space left, how would you troubleshoot that issue?

- If using LVM, add more disk space and extend the logical volume. If not, add a new disk, create a partition, and link it to an existing filesystem.

99. How to check Redhat version release?

- Use `cat /etc/redhat-release` or `uname -a`.

100. What is the difference between TCP and UDP?

- TCP is connection-oriented and reliable, while UDP is connectionless and faster but less reliable.

101. What is a zombie process?

- A zombie process is a process that has completed execution but still has an entry in the process table, waiting for the parent process to read its exit status.
102. How do you search for a pattern/word in a file and then replace it in an entire file?
- Use `sed` for search and replace operations.
103. How to check the number of users logged in?
- Use `who` command.
104. What is the command to view the calendar of 2024?
- ```
cal 2024
```
105. Which command is used to view disk space?
- ```
df -h
```
106. How to create a new group in Linux?
- `groupadd groupname`.
107. What is the command to send a message to everyone who is logged into the system?
- Use `wall` command.
108. Which command is used to check the total number of disks?
- `fdisk -l`.
109. What is a mail server record in DNS?
- MX (Mail Exchange) record
110. What does the following command line do? `ps -ef | awk '{print $1}' | sort | uniq`
- Lists the first column of all running processes, sorts them, and removes duplicates.
111. You get a call that when a user goes to *www.yourwebsite.com* it fails and gets an error, how do you troubleshoot?
- Check user internet connectivity, DNS configuration, server status, and web service available
112. List 4 different directories in `/`?
- `/etc`, `/bin`, `/tmp`, `/home`.
113. What is the output of the following command:
- `$ tail -10 filename | head -1`

- It will show the first line from the last 10 lines of a file.

114. What are the different fields in `/etc/passwd` file?

The `/etc/passwd` file contains the following fields separated by colons (:):

- Username: The user's login name.
- Password: An 'x' character indicates that the password is stored in the `/etc/shadow` file.
- UID: User ID number.
- GID: Group ID number.
- GECOS: User's full name or other information.
- Home Directory: The path to the user's home directory.
- Shell: The user's default shell.

115. Which command is used to list the processes?

- `ps -ef`

The `ps -ef` command is used to list all the currently running processes.

116. What is the difference between “hostname” and “uname” commands?

- hostname: Displays or sets the system's hostname.
- uname: Prints system information, such as the kernel name, version, and other details. `uname -n` specifically prints the network node hostname, similar to the `hostname` command.

117. How to check system load?

- You can check system load using the `top` and `uptime` commands.

118. How to schedule jobs?

- You can schedule jobs using `crontab` for repetitive tasks and `at` for one-time tasks.

119. What is the 3rd field when setting up crontab?

- The third field in a crontab entry is the "Day of the month".

120. What is the “init #” for system reboot?

- The `init` level for system reboot is `6`.

121. How to restart a service?

- `systemctl restart servicename`
- `sudo service servicename restart`

122. How to shutdown a system?**

- `shutdown now`
- `init 0`
- `systemctl poweroff`

123. What is the “ftp” command used for?

- The ``ftp`` command is used to transfer files between a local system and a remote server using the FTP protocol.

124. Explain cron job syntax? First is minute, second is..?

- The cron job syntax consists of five fields followed by the command to be executed:
 - 1. Minute (0-59)
 - 2. Hour (0-23)
 - 3. Day of the month (1-31)
 - 4. Month (1-12 or JAN-DEC)
 - 5. Day of the week (0-6 or SUN-SAT).

125. How to delete a package in Linux?

- `rpm -e packagename`
- for RPM-based systems.

126. What is the file name where user password information is saved?

- The user password information is saved in the ``/etc/shadow`` file.

127. Which command would you use to find the location of the chmod command?

- `which chmod`

128. Which command is used to check if the other computer is online?

- `ping othercomputer`

129. Please explain about LAN, MAN and WAN?

- LAN (Local Area Network): Covers a small geographic area like a home, office, or building.
- MAN (Metropolitan Area Network): Spans a city or a large campus.
- WAN (Wide Area Network): Covers a large geographic area, such as a country or continent, connecting multiple LANs.

130. How to list hidden files in a directory?

- `ls -la`

131. What is the difference between telnet and ssh?
- SSH (Secure Shell): Provides encrypted communication and is secure.
 - Telnet: Does not provide encryption and is not secure.
132. How to run a calculator on Linux and exit out of it?
- Run ``bc`` to start the calculator and type ``quit`` to exit.
133. List any 4 commands to monitor the system?
- `top`
 - `df -h`
 - `iostat`
 - `dmesg`
134. You are notified that your server is down, list the steps you will take to troubleshoot?
- - Check the system physically.
 - - Login through the system console.
 - - Ping the system.
 - - Reboot or boot if possible.
135. What is the difference between static and DHCP IP?
- Static IP: Manually assigned and does not change.
 - DHCP IP: Automatically assigned by a DHCP server and can change over time.
136. How to write in vi editor mode?
- - ``i`` = insert
 - - ``a`` = insert after the cursor
 - - ``o`` = insert a new line below the current line
137. What is the `difference` between “crontab” and “at” jobs?
- `crontab`: Schedules repetitive jobs.
 - `at`: Schedules one-time jobs.
138. What is vCenter server in VMWare?
- vCenter Server is a centralized management tool for managing VMware vSphere environments, providing control over virtual machines, ESXi hosts, and other components.

139. What is the `dmidecode` command used for?
- The `dmidecode` command is used to retrieve system hardware information from the DMI (Desktop Management Interface) table.
140. What is the difference between SAN and NAS?
- SAN (Storage Area Network): Provides block-level storage and is typically used in enterprise environments.
 - NAS (Network Attached Storage): Provides file-level storage and is often used for simpler file sharing.
141. What is the location of system logs? E.g. messages
- System logs are typically located in the `/var/log` directory.
142. How to set up an alias and what is it used for?
- `alias aliasname="command"`
 - It is used to create shortcuts for long commands.
143. What is the purpose of the `netstat` command?
- The `netstat` command is used to display network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.
144. What are terminal control keys, list any 3?
- `Ctrl+C`: Interrupt a process.
 - `Ctrl+D`: End of input (EOF).
 - `Ctrl+Z`: Suspend a process.
145. Which command(s) you would run if you need to find out how many processes are running on your system?
- `ps -ef | wc -l`
146. How to delete a line when in vi editor mode?
- Press `dd` to delete a line.
147. How to save and quit from vi editor?
- Press `Shift ZZ` or type `:wq!` and press Enter.
148. What is the difference between a process and daemon?
- Process: A running instance of a program.
 - Daemon: A background process that starts at boot time and runs continuously.

149. What is the process or daemon name for NTP?
- The process or daemon name for NTP is `ntpd`.
150. What are a few commands you would run if your system is running slow?
- `top`: To view running processes and their resource usage.
 - `iostat`: To monitor system input/output device loading.
 - `df -h`: To check disk space usage.
 - `netstat`: To display network connections and statistics.
151. How to install a package in Redhat Linux?
- `yum install packagename`
152. What is the difference between “ifconfig” and “ipconfig” commands?
- `ifconfig`: Used in Linux to configure and display network interface parameters.
 - `ipconfig`: Used in Windows to display all current TCP/IP network configuration values.
153. What is the first line written in a shell script?
- The first line in a shell script is the shebang, which defines the shell to interpret the script, e.g., `#!/bin/bash`.
154. Where is the network (Ethernet) file located, please provide the exact directory location and file name?
- The network configuration file is located at `/etc/sysconfig/network-scripts/ifcfg-nic`, where `nic` represents the network interface card name (e.g., `eth0`).
155. Why do we use the “last” command?
- The `last` command is used to display a list of the last logged-in users, showing both active and logged-off sessions.
156. What does RHEL Linux stand for?
- RHEL stands for Red Hat Enterprise Linux.
157. To view your command history, which command is used and how to run a specific command?
- Use `history` to view the command history.
 - To run a specific command, use `!n`, where `n` is the command number from the history list.

158. What is NTP and briefly explain how it works and where are the config files and related commands of NTP?
- NTP (Network Time Protocol): Used to synchronize the clocks of computers over a network.
 - How it works: NTP servers provide time information to clients, which adjust their clocks accordingly.
 - Config files: Located at `/etc/ntp.conf`.
 - Related commands**: `ntpd` for the daemon, `ntpq` for querying NTP servers.
159. How to disable the firewall in Linux?
- To disable the firewall, you can use:
 - `systemctl stop firewalld`
 - `systemctl disable firewalld`
160. How to configure mail server relay for sendmail service?
- Edit the `/etc/mail/sendmail.mc` file and add the `SMART_HOST` entry to specify the relay host.
161. Where is the samba log file located?
- The Samba log file is located at `var/log/samba`.
162. What is the `mkfs` command used for?**
- The `mkfs` command is used to create a new filesystem on a device.
163. If you create a new group, which file does it get created in?
- The new group information is stored in the `/etc/group` file.
164. Which file has DNS server information (e.g., DNS resolution)?
- DNS server information is stored in the `/etc/resolv.conf` file.
165. What are the commands you would run if you need to find out the version and build date of a package (e.g., http)?
- `rpm -qi http`
166. On the file permissions, what are the first 3 bits for and who is it for?
- The first three bits represent the permissions for the file owner: read (r), write (w), and execute (x).
167. How to create a soft link?
- `ln -s target linkname`

168. How to write a script to delete messages in a log file older than 30 days automatically?
- You can use the `find` command in a script:
 - `find /path/to/logs -type f -mtime +30 -exec rm {} \;`
169. How to quit out of the “`man`” command?
- Press `q` to quit the manual page viewer.
170. Which command is used to partition a disk in Linux?
- The `fdisk` command is used to partition a disk.
171. What is the difference between the “shutdown” and “halt” command?
- `shutdown`: Gracefully shutdown the system, allowing processes to terminate properly.
 - `halt`: Stops all processes and halts the system without powering it off.
172. What is the exact syntax of mounting an NFS share on a client and also how to un-mount?*
- Mount: `mount -t nfs server:/path/to/share /mount/point`
 - Unmount: `umount /mount/point`
173. What experience do you have with scripting, explain?
- Experience with scripting includes using control structures like `if-then`, `do-while`, `case`, and `for` loops to automate tasks.
174. How to get information on all the packages installed on the system?
- `rpm -qa`
175. Explain VMWare?*
- VMWare is a company that provides cloud computing and virtualization technology, including products like vSphere, ESXi, and vCenter.
176. You are tasked to examine a log file in order to find out why a particular application keeps crashing. The log file is very lengthy, which command can you use to simplify the log search using a search string?
- Use the `grep` command to search for specific strings like “error”, “warning”, or “failure” in the log file:
 - `grep "error" /var/log/messages`

177. What is an `/etc/fstab` file and explain each column of this file?
- The `/etc/fstab` file contains information about filesystems and their mount points. Each line has six fields:
 - 1. **Device**: The block device or remote filesystem.
 - 2. **Mount Point**: Directory where the filesystem is mounted.
 - 3. **Filesystem Type**: Type of filesystem (e.g., ext4, nfs).
 - 4. **Options**: Mount options (e.g., defaults, ro).
 - 5. **Dump**: Backup utility flag (0 or 1).
 - 6. **Pass**: Filesystem check order at boot (0, 1, or 2).
178. What is the latest version of Windows server?
- The latest version as of 2023 is Windows Server 2022.
179. What is the exact command to list only the first 2 lines of history output?
- `history | head -2`
180. How to upgrade Linux from 7.3 to 7.4?
- `yum update`: This command updates all packages to their latest versions, including upgrading the distribution version.
181. How to tell which shell you are in or running?
- `echo $0`
182. You have tried to “cd” into a directory but you have been denied. You are not the owner of that directory, what permissions do you need and where?
- You need execute (`x`) permission on the directory to change into it.
183. What is CNAME record in DNS?
- A CNAME (Canonical Name) record is a type of DNS record that maps an alias name to a true or canonical domain name.
184. What is the name of the VMWare operating system?
- The VMWare operating system is called ESXi.
185. What is the client name used to connect to ESXi or vCenter server?
- The client used to connect to ESXi or vCenter is the vSphere Client.
186. You get a call from a user saying that I cannot write to a file because it says, permission denied. The file is owned by that user, how do you troubleshoot?
- Check the file permissions and ensure the user has write (`w`) permission.
187. What is the latest version of VMWare?
- As of 2023, the latest version of VMware vSphere is 8.0.

188. What is the name of the firewall daemon in Linux?
- The firewall daemon in Linux is called `firewalld`.
189. Which command syntax can you use to list only the 20th line of a file?
- `sed -n '20p' filename`
190. What is the difference between run level 3 and 5?
- Run level 3: Multi-user mode with networking, but without a graphical interface.
 - Run level 5: Multi-user mode with networking and a graphical interface (GUI).
191. What is the difference between domain and nameserver?
- Domain: A domain is a human-readable address used to access resources on the internet.
 - Nameserver: A server that translates domain names into IP addresses.
192. You open up a file and it has 3000 lines and it scrolls up really fast, which command will you use to view it one page at a time?
- Use `less` or `more` to view the file one page at a time.
193. How to start a new shell. E.g., start a new ksh shell?
- Simply type `ksh` or `bash` to start a new shell session.
194. How to kill a process?
- `kill processID`
195. How to check scheduled jobs?
- `crontab -l`
196. How to check system memory and CPU usage?
- Memory: Use `free`.
 - CPU Usage: Use `top` or `htop`.
197. Which utility could you use to repair the corrupted file system?**
- Use the `fsck` (File System Consistency Check) utility.
198. What is the command to make a service start at boot?**
- `systemctl enable servicename`
199. Which file to modify to allow users to run root commands?**
- Modify the `/etc/sudoers` file, typically using `visudo` for safe editing.
200. You need to modify the `httpd.conf` file but you cannot find it. Which command line tool can you use to find the file?**

- `find / -name "httpd.conf"`

201. Your system crashed and is being restarted, but a message appears indicating that the operating system cannot be found. What is the most likely cause of the problem?

- The `/boot`` partition or bootloader is most likely corrupted or missing.

202. What are the most essential [70+] Linux Commands you NEED to know

No	Command	Description	Example
1	<code>ssh</code>	Connects to a remote machine via SSH.	<code>ssh user@hostname</code> connects to the remote machine <code>hostname</code> as <code>user</code> .
2	<code>ls</code>	Lists directory contents.	<code>ls -l</code> lists files in long format, showing permissions, owner, size, and modification date.
3	<code>pwd</code>	Prints the current working directory.	<code>pwd</code> displays the full path of the current directory.
4	<code>cd</code>	Changes the current directory.	<code>cd /home/user</code> changes the directory to <code>/home/user</code> .
5	<code>touch</code>	Creates an empty file or updates the timestamp of an existing file.	<code>touch newfile.txt</code> creates an empty file named <code>newfile.txt</code> .
6	<code>echo</code>	Displays a line of text or a variable value.	<code>echo "Hello, World!"</code> prints <code>Hello, World!</code> to the terminal.
7	<code>nano</code>	Open the Nano text editor.	<code>nano file.txt</code> opens <code>file.txt</code> in the Nano editor.
8	<code>vim</code>	Open the Vim text editor.	<code>vim file.txt</code> opens <code>file.txt</code> in the Vim editor.

9	<code>cat</code>	Concatenates and displays file content.	<code>cat file.txt</code> displays the content of <code>file.txt</code> .
10	<code>shred</code>	Overwrites a file to hide its contents and optionally deletes it.	<code>shred -u file.txt</code> overwrites and deletes <code>file.txt</code> .
11	<code>mkdir</code>	Creates a new directory.	<code>mkdir newdir</code> creates a directory named <code>newdir</code> .
12	<code>cp</code>	Copies files or directories.	<code>cp file1.txt file2.txt</code> copies <code>file1.txt</code> to <code>file2.txt</code> .
13	<code>rm</code>	Removes files or directories.	<code>rm file.txt</code> deletes <code>file.txt</code> .
14	<code>rmdir</code>	Removes an empty directory.	<code>rmdir olddir</code> removes the empty directory <code>olddir</code> .
15	<code>ln</code>	Creates hard and symbolic links.	<code>ln -s target linkname</code> creates a symbolic link named <code>linkname</code> pointing to <code>target</code> .
16	<code>clear</code>	Clears the terminal screen.	<code>clear</code> clears the terminal display.
17	<code>whoami</code>	Displays the current logged-in user.	<code>whoami</code> shows the username of the current user.
18	<code>useradd</code>	Adds a new user.	<code>sudo useradd newuser</code> adds a new user named <code>newuser</code> .
19	<code>sudo</code>	Executes a command as another user, typically the superuser.	<code>sudo apt-get update</code> runs the <code>apt-get update</code> command with superuser privileges.

20	<code>adduser</code>	Adds a new user with a more interactive interface.	<code>sudo adduser newuser</code> interactively adds a new user named <code>newuser</code> .
21	<code>su</code>	Switch to another user account.	<code>su - user</code> switches to the <code>user</code> account.
22	<code>exit</code>	Exits the current shell or session.	<code>exit</code> logs out of the current session.
23	<code>passwd</code>	Changes a user's password.	<code>passwd</code> prompts to change the current user's password.
24	<code>apt</code>	Manages packages on Debian-based systems.	<code>sudo apt install package</code> installs the specified package.
25	<code>finger</code>	Displays information about system users.	<code>finger user</code> shows details about <code>user</code> .
26	<code>man</code>	Displays the manual page for a command.	<code>man ls</code> shows the manual for the <code>ls</code> command.
27	<code>whatis</code>	Displays a brief description of a command.	<code>whatis ls</code> provides a short description of the <code>ls</code> command.
28	<code>curl</code>	Transfers data from or to a server.	<code>curl -O http://example.com/file.txt</code> downloads <code>file.txt</code> from the specified URL.
29	<code>zip</code>	Compresses files into a zip archive.	<code>zip archive.zip file1 file2</code> compresses <code>file1</code> and <code>file2</code> into <code>archive.zip</code> .

30	<code>unzip</code>	Extracts files from a zip archive.	<code>unzip archive.zip</code> extracts files from <code>archive.zip</code> .
31	<code>less</code>	Views file content one screen at a time.	<code>less file.txt</code> displays <code>file.txt</code> content one screen at a time.
32	<code>head</code>	Displays the first part of a file.	<code>head -n 10 file.txt</code> shows the first 10 lines of <code>file.txt</code> .
33	<code>tail</code>	Displays the last part of a file.	<code>tail -n 10 file.txt</code> shows the last 10 lines of <code>file.txt</code> .
34	<code>cmp</code>	Compare two files byte by byte.	<code>cmp file1 file2</code> compares <code>file1</code> and <code>file2</code> .
35	<code>diff</code>	Compares files line by line.	<code>diff file1 file2</code> shows the differences between <code>file1</code> and <code>file2</code> .
36	<code>sort</code>	Sorts lines of text files.	<code>sort file.txt</code> sorts the lines in <code>file.txt</code> .
37	<code>find</code>	Searches for files in a directory hierarchy.	<code>find /home -name "*.txt"</code> finds all <code>.txt</code> files in the <code>/home</code> directory.
38	<code>chmod</code>	Changes file permissions.	<code>chmod 755 script.sh</code> sets the permissions of <code>script.sh</code> to <code>rw-r-xr-x</code> .
39	<code>chown</code>	Changes file owner and group.	<code>chown user:group file.txt</code> changes the owner and group of <code>file.txt</code> to <code>user</code> and <code>group</code> .
40	<code>ifconfig</code>	Configures network interfaces.	<code>ifconfig eth0</code> displays the configuration of the <code>eth0</code> interface.

41	<code>ip address</code>	Displays IP addresses and interfaces.	<code>ip address show</code> shows all IP addresses and network interfaces.
42	<code>grep</code>	Searches for patterns in files.	<code>grep "pattern" file.txt</code> searches for "pattern" in <code>file.txt</code> .
43	<code>awk</code>	A programming language for pattern scanning and processing.	<code>awk '{print \$1}' file.txt</code> prints the first field of each line in <code>file.txt</code> .
44	<code>resolvectl status</code>	Shows the current DNS settings.	<code>resolvectl status</code> displays the DNS configuration and status.
45	<code>ping</code>	Sends ICMP ECHO_REQUEST packets to network hosts.	<code>ping google.com</code> sends ping requests to <code>google.com</code> .
46	<code>netstat</code>	Displays network connections, routing tables, and interface statistics.	<code>netstat -tuln</code> shows listening ports and their status.
47	<code>ss</code>	Displays socket statistics.	<code>ss -tuln</code> shows listening sockets. <code>ss -l4p</code> displays all listening IPv4 sockets along with the associated processes
48	<code>iptables</code>	Configures packet filtering rules.	<code>sudo iptables -L</code> lists all current iptables rules.
49	<code>ufw</code>	Manages firewall with Uncomplicated Firewall.	<code>sudo ufw enable</code> enables the firewall.
50	<code>uname</code>	Prints system information.	<code>uname -a</code> displays all system information.

51	<code>neofetch</code>	Displays system information with an aesthetic layout.	<code>neofetch</code> shows system information in a visually appealing format.
52	<code>cal</code>	Displays a calendar.	<code>cal</code> shows the current month's calendar.
53	<code>free</code>	Displays memory usage.	<code>free -h</code> shows memory usage in a human-readable format.
54	<code>df</code>	Displays disk space usage of file systems.	<code>df -h</code> shows disk space usage in a human-readable format.
55	<code>ps</code>	Displays information about active processes.	<code>ps aux</code> shows detailed information about all running processes.
56	<code>top</code>	Displays real-time system resource usage.	<code>top</code> shows real-time processes and system resource usage.
57	<code>htop</code>	An interactive process viewer.	<code>htop</code> provides an interactive view of system processes.
58	<code>kill</code>	Terminates a process by PID.	<code>kill 1234</code> terminates the process with PID 1234.
59	<code>pkill</code>	Terminates processes by name.	<code>pkill firefox</code> terminates all processes named <code>firefox</code> .
60	<code>systemctl</code>	Manages systemd services.	<code>systemctl status nginx</code> shows the status of the <code>nginx</code> service.
61	<code>history</code>	Displays the command history.	<code>history</code> shows the list of previously executed commands.

62	<code>reboot</code>	Reboots the system.	<code>sudo reboot</code> restarts the system.
63	<code>shutdown</code>	Shuts down or reboots the system.	<code>sudo shutdown -h now</code> shuts down the system immediately.
64	<code>traceroute</code>	Traces the route packets take to a network host.	<code>traceroute google.com</code> shows the route to <code>google.com</code> .
65	<code>dig</code>	Queries DNS servers.	<code>dig example.com</code> retrieves DNS information for <code>example.com</code> .
66	<code>host</code>	Performs DNS lookups.	<code>host example.com</code> shows the IP address of <code>example.com</code> .
67	<code>arp</code>	Displays and modifies the ARP table.	<code>arp -a</code> shows the current ARP table.
68	<code>iwconfig</code>	Configures wireless network interfaces.	<code>iwconfig wlan0</code> shows the configuration of the <code>wlan0</code> wireless interface.
69	<code>hostname</code>	Displays or sets the system's hostname.	<code>hostname</code> shows the current hostname.
70	<code>whois</code>	Queries the WHOIS database for domain information.	<code>whois example.com</code> retrieves WHOIS information for <code>example.com</code> .

Linux Production Shell Scripts

1. File Backup Script:

```
#!/bin/bash

backup_dir="/path/to/backup"
source_dir="/path/to/source"

# Create a timestamped backup of the source directory
tar -czf "$backup_dir/backup_$(date +%Y%m%d_%H%M%S).tar.gz"
"$source_dir"
```

2. System Monitoring Script:

```
#!/bin/bash

threshold=90

# Monitor CPU usage and trigger alert if threshold exceeded
cpu_usage=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d. -f1)
if [ "$cpu_usage" -gt "$threshold" ]; then
    echo "High CPU usage detected: $cpu_usage%"
    # Add alert/notification logic here
fi
```

3. User Account Management Script:

```
#!/bin/bash

username="newuser"

# Check if user exists; if not, create new user
if id "$username" &>/dev/null; then
    echo "User $username already exists."
else
    useradd -m "$username"
    echo "User $username created."
```

Fi

4. Log Analyzer Script:

```
#!/bin/bash

logfile="/path/to/logfile.log"

# Extract lines with "ERROR" from the log file
grep "ERROR" "$logfile" > error_log.txt
echo "Error log created."
```

5. Password Generator Script:

```
#!/bin/bash

length=12

# Generate a random password
password=$(openssl rand -base64 $length)
echo "Generated password: $password"
```

6. File Encryption/Decryption Script:

```
#!/bin/bash

file="/path/to/file.txt"

# Encrypt file using AES-256-CBC
openssl enc -aes-256-cbc -salt -in "$file" -out "$file.enc"
echo "File encrypted: $file.enc"
```

7. Automated Software Installation Script:

```
#!/bin/bash

packages=("package1" "package2" "package3")
```

```
# Install listed packages using apt-get
for package in "${packages[@]}"; do
    sudo apt-get install "$package" -y
done

echo "Packages installed successfully."
```

8. Network Connectivity Checker Script:

```
#!/bin/bash

host="example.com"

# Check network connectivity by pinging a host
if ping -c 1 "$host" &>/dev/null; then
    echo "Network is up."
else
    echo "Network is down."
fi
```

9. Website Uptime Checker Script:

```
#!/bin/bash

website="https://example.com"

# Check if website is accessible
if curl --output /dev/null --silent --head --fail "$website"; then
    echo "Website is up."
else
    echo "Website is down."
fi
```

10. Data Cleanup Script:

```
#!/bin/bash

directory="/path/to/cleanup"
```

```
# Remove files older than 7 days in specified directory
find "$directory" -type f -mtime +7 -exec rm {} \;
echo "Old files removed."
```

11. CPU Usage Tracker Script:

```
#!/bin/bash

output_file="cpu_usage_log.txt"

# Log current CPU usage to a file with timestamp
echo "$(date) $(top -bn1 | grep 'Cpu(s)' | awk '{print $2}' | cut -d.
-f1)%" >> "$output_file"
echo "CPU usage logged."
```

12. System Information Script:

```
#!/bin/bash

output_file="system_info.txt"

# Gather system information and save to a file
echo "System Information:" > "$output_file"
echo "-----" >> "$output_file"
echo "Hostname: $(hostname)" >> "$output_file"
echo "OS: $(uname -a)" >> "$output_file"
echo "Memory: $(free -h)" >> "$output_file"
echo "Disk Space: $(df -h)" >> "$output_file"
echo "System info saved to $output_file."
```

13. Task Scheduler Script:

```
#!/bin/bash

scheduled_task="/path/to/your_script.sh"
schedule_time="0 2 * * *"
```

```
# Schedule a task using cron
echo "$schedule_time $scheduled_task" | crontab -
echo "Task scheduled successfully."
```

14. Disk Space Monitoring Script:

```
#!/bin/bash

threshold=90

# Monitor disk usage and trigger alert if threshold exceeded
disk_usage=$(df -h | grep "/dev/sda1" | awk '{print $5}' | cut -d% -f1)
if [ "$disk_usage" -gt "$threshold" ]; then
    echo "High disk usage detected: $disk_usage%"
    # Add alert/notification logic here
fi
```

15. Remote Server Backup Script:

```
#!/bin/bash

source_dir="/path/to/source"
remote_server="user@remoteserver:/path/to/backup"

# Backup files/directories to a remote server using rsync
rsync -avz "$source_dir" "$remote_server"
echo "Files backed up to remote server."
```

16. Environment Setup Script:

```
#!/bin/bash

# Customize for your specific environment setup
echo "Setting up development environment..."
# Install necessary packages, configure settings, etc.
echo "Development environment set up successfully."
```

17. File Compression/Decompression Script:

```
#!/bin/bash

file_to_compress="/path/to/file.txt"

# Compress a file using gzip
gzip "$file_to_compress"
echo "File compressed: $file_to_compress.gz"
```

18. Database Backup Script:

```
#!/bin/bash

database_name="your_database"
output_file="database_backup_$(date +%Y%m%d).sql"

# Perform database backup using mysqldump
mysqldump -u username -ppassword "$database_name" > "$output_file"
echo "Database backup created: $output_file"
```

19. Git Repository Updater Script:

```
#!/bin/bash

git_repo="/path/to/your/repo"

# Update a Git repository
cd "$git_repo"
git pull origin master
echo "Git repository updated."
```

20. Directory Synchronization Script:

```
#!/bin/bash

source_dir="/path/to/source"
destination_dir="/path/to/destination"
```

```
# Synchronize directories using rsync
rsync -avz "$source_dir" "$destination_dir"
echo "Directories synchronized successfully."
```

21. Web Server Log Analyzer Script:

```
#!/bin/bash

log_file="/var/log/apache2/access.log"

# Analyze web server log to count unique IP addresses
awk '{print $1}' "$log_file" | sort | uniq -c | sort -nr
echo "Web server log analyzed."
```

22. System Health Check Script:

```
#!/bin/bash

output_file="system_health_check.txt"

# Perform system health check and save results to a file
echo "System Health Check:" > "$output_file"
echo "-----" >> "$output_file"
echo "Uptime: $(uptime)" >> "$output_file"
echo "Load Average: $(cat /proc/loadavg)" >> "$output_file"
echo "Memory Usage: $(free -m)" >> "$output_file"
echo "System health check results saved to $output_file."
```

23. Automated Database Cleanup Script:

```
#!/bin/bash

database_name="your_database"
days_to_keep=7

# Clean up old database backups older than specified days
find /path/to/database/backups -name "$database_name*.sql" -mtime
+"$days_to_keep" -exec rm {} \;
echo "Old database backups cleaned up."
```


24. User Password Expiry Checker Script:

```
#!/bin/bash

# Check password expiry for users with bash shell
IFS=$'\n'
for user in $(cat /etc/passwd | grep "/bin/bash" | cut -d: -f1); do
    password_expires=$(chage -l "$user" | grep "Password expires" |
    awk '{print $4}')
    echo "User: $user, Password Expires: $password_expires"
done
unset IFS
```

25. Service Restart Script:

```
#!/bin/bash

service_name="your_service"

# Restart a specified service
sudo systemctl restart "$service_name"
echo "Service $service_name restarted."
```

26. Folder Size Checker Script:

```
#!/bin/bash

folder_path="/path/to/folder"

# Check and display the size of a specified folder
du -sh "$folder_path"
echo "Folder size checked."
```

27. Backup Rotation Script:

```
#!/bin/bash

backup_dir="/path/to/backups"
```

```
max_backups=5
```

```
# Rotate backups by deleting the oldest if more than max_backups
while [ $(ls -1 "$backup_dir" | wc -l) -gt "$max_backups" ]; do
    oldest_backup=$(ls -1t "$backup_dir" | tail -n 1)
    rm -r "$backup_dir/$oldest_backup"
done
echo "Backup rotation completed."
```

28. Remote Script Execution Script:

```
#!/bin/bash
```

```
remote_server="user@remote-server"
remote_script="/path/to/remote/script.sh"

# Execute a script on a remote server via SSH
ssh "$remote_server" "bash -s" < "$remote_script"
echo "Remote script executed."
```

29. Network Interface Information Script:

```
#!/bin/bash
```

```
network_interface="eth0"

# Display network interface information
ifconfig "$network_interface"
echo "Network interface information displayed."
```

30. Random Quotes Generator Script:

```
#!/bin/bash
```

```
quotes=("Quote 1" "Quote 2" "Quote 3" "Quote 4")

# Generate and display a random quote from the array
random_index=$((RANDOM % ${#quotes[@]}))
echo "Random Quote: ${quotes[$random_index]}"
```


Linux Production Shell Scripts

1. Log Analyzer Script

```
#!/bin/bash

Logfile= "/path/to/logfile.log"

# Extract lines with "ERROR" from the log file
grep "ERROR" "$logfile" > error_log.txt

echo "Error Log Created."
```

2. Automated Software Installation Scripts

```
#!/bin/bash

Packages= ("package1" "package2" "package3")

# Install listed packages using apt-get
for package in "${packages[@]}" : do
    sudo apt-get install "$package" -y
done

echo "Packages Installed Successfully."
```

3) Service Restart Script

```
#!/bin/bash

service_name = "your Service"

#Restart a specified service
sudo systemctl restart "$service_name"

Echo "Service $service_name restarted."
```

4) CPU Usage Tracker Script

```
#!/bin/bash
```

```
Output_file= "cpu_usage_log.txt"
```

```
# Log current CPU Usage to a file with timestamp
```

```
echo "$(date) $(top -bn1 | grep "CPU(s)" | awk '{print $2}' | cut -d. -f1)%" >> "$output_file"
```

```
echo "CPU usage logged."
```

5) System Information Script

```
#!/bin/bash
```

```
Output_file = "system_info.txt"
```

```
#Gather system information and save to a file
```

```
echo "System Information:" > "$output_file"
```

```
echo "-----" >> "$output_file"
```

```
echo "Hostname: $(hostname)" >> "$output_file."
```

```
echo "OS: $(uname -a)" >> "$output_file"
```

```
echo "Memory: $(free -h)" >> "$output_file"
```

```
echo "Disk Space: $(df -h)" >> "$output_file"
```

```
echo "System info saved to "$output_file"
```

6) Task Scheduled Script:

```
#!/bin/bash
```

```
Scheduled_task= "/path/to/your_script.sh"
```

```
Schedule_time= "0 2 * * *"
```

```
$schedule a task using cron
```

```
echo "$schedule_time $scheduled_task" | crontab -
```

```
echo "Task scheduled successfully."
```

7) Disk Space Monitoring Script:

```
#!/bin/bash
```

Threshold=90

\$monitor disk usage and trigger alert if threshold exceeded.

```
disk_usage=$(df -h | grep "/dev/sda1" | awk '{print $5}' | cut -d% -f1)
```

```
if
```

```
["$disk_usage" -gt "$threshold"]; then
```

```
echo "High disk usage detected: $disk_usage%"
```

```
# Add alert/notification logic here
```

```
fi
```

8) calculator

```
#!/bin/bash
```

```
# This is a simple script to calculate two numbers.
```

```
# Taking user input
```

```
echo "Enter two numbers:"
```

```
read a
```

```
read b
```

```
#input type of operation
```

```
echo "Enter your choice""
```

```
echo "1. Addition"
```

```
echo "2. Substraction"
```

```
echo "3. Multiplication"
```

```
echo "4. Division"
```

```
read ch
```

```
#Acting on the user input
```

```
case $ch in
```

```
1) res = 'echo $a + $b | bc'
```

```
::
```

```
2) res= 'echo $a - $b | bc'
```

```
::
```

```
3) res= 'echo $a \ * $b | bc'
```

```
::
```

```
4) res= ' echo "scale=2; $a / $b" | bc'
```

```
::
```

```
esac
```

```
#printing result
```

```
echo "Result: $res"
```

9) Convert All Uppercase Letters in a String to Lowercase

```
#!/bin/bash
```

```
Read -p "Enter a string: " str
```

```
echo "converted string:" $str | tr '[:upper:]' '[:lower:]'
```

10) Write a Shell Script to take user input from arguments and print the variables?

```
#!/bin/bash
```

```
# Take user input
```

```
echo "Enter your name:"
```

```
read name
```

```
#Take input from arguments
```

```
arg1=$1
```

```
arg2=$2
```

```
#print the variables
```

```
echo "your name is: $name"
```

```
echo "argument 1 is : $arg1"
```

```
echo "argument 2 is: $arg2"
```

Will keep you posted more

Linux Production Shell Script

These Scripts will help you to automate your manual work in production.

1. File Backup Script:

```
#!/bin/bash
backup_dir="/path/to/backup"
source_dir="/path/to/source"

tar-czf "$backup_dir/backup_$(date
+%Y%m%d_%H%M%S).tar.gz"
"$source_dir"
```

2. System Monitoring Script:

```
#!/bin/bash
threshold=90
```

```
cpu_usage=$(top-bn1 | grep "Cpu(s)" |  
awk '{print $2}' | cut-d.-f1)  
if [ "$cpu_usage"-gt "$threshold" ]; then  
    echo "High CPU usage detected:  
$cpu_usage%"  
    # Add alert/notification logic here  
fi
```

3. User Account Management Script:

```
#!/bin/bash  
username="newuser"  
  
if id "$username" &>/dev/null; then  
    echo "User $username already exists."  
else  
    useradd-m "$username"  
    echo "User $username created."  
fi
```

4. Log Analyzer Script:

```
#!/bin/bash  
logfile="/path/to/logfile.log"
```

```
grep "ERROR" "$logfile" > error_log.txt  
echo "Error log created."
```

5. Password Generator Script:

```
#!/bin/bash  
length=12  
  
password=$(openssl rand-base64 12)  
echo "Generated password: $password"
```

6. File Encryption/Decryption Script:

```
#!/bin/bash  
file="/path/to/file.txt"
```

```
openssl enc-aes-256-cbc-salt-in "$file"-out  
"$file.enc"  
echo "File encrypted: $file.enc"
```

7. Automated Software Installation

Script:

```
#!/bin/bash  
packages=("package1" "package2"  
"package3")  
  
for package in "${packages[@]}"; do  
    sudo apt-get install "$package"-y  
done  
  
echo "Packages installed successfully."
```

8. Network Connectivity Checker

Script:

```
#!/bin/bash
```

```
host="example.com"
```

```
if ping-c 1 "$host" &>/dev/null; then  
    echo "Network is up."  
else  
    echo "Network is down."  
fi
```

9. Website Uptime Checker Script:

```
#!/bin/bash  
website="https://example.com"  
  
if curl--output /dev/null--silent--head--fail  
"$website"; then  
    echo "Website is up."  
else  
    echo "Website is down."  
fi
```

10. Data Cleanup Script:

```
#!/bin/bash
directory="/path/to/cleanup"

find "$directory" -type f -mtime +7 -exec rm
{} \;
echo "Old files removed."
```

11. CPU Usage Tracker Script:

```
#!/bin/bash
output_file="cpu_usage_log.txt"

echo "$(date) $(top -bn1 | grep "Cpu(s)" |
awk '{print $2}' | cut -d. -f1)%" >>
"$output_file"
echo "CPU usage logged."
```

12. System Information Script:

```
#!/bin/bash
output_file="system_info.txt"

echo "System Information:" >
"$output_file"
echo "-----" >> "$output_file"
echo "Hostname: $(hostname)" >>
"$output_file"
echo "OS: $(uname-a)" >> "$output_file"
echo "Memory: $(free-h)" >>
"$output_file"
echo "Disk Space: $(df-h)" >>
"$output_file"
echo "System info saved to $output_file."
```

13. Task Scheduler Script:

```
#!/bin/bash
scheduled_task="/path/to/your_script.sh"
schedule_time="0 2 * * *"
```

```
echo "$schedule_time $scheduled_task" |  
crontab-  
echo "Task scheduled successfully."
```

14. Disk Space Monitoring Script:

```
#!/bin/bash  
threshold=90  
  
disk_usage=$(df-h | grep "/dev/sda1" |  
awk '{print $5}' | cut-d%-f1)  
if [ "$disk_usage"-gt "$threshold" ]; then  
    echo "High disk usage detected:  
$disk_usage%"  
    # Add alert/notification logic here  
fi
```

15. Remote Server Backup Script:

```
#!/bin/bash  
source_dir="/path/to/source"
```



```
remote_server="user@remote-  
server:/path/to/backup"
```

```
rsync-avz "$source_dir" "$remote_server"  
echo "Files backed up to remote server."
```

16. Environment Setup Script:

```
#!/bin/bash  
# Customize the following lines based on  
your development environment setup  
echo "Setting up development  
environment..."  
# Install necessary packages, configure  
settings, etc.  
echo "Development environment set up  
successfully."
```

17. File Compression/Decompression Script:

```
#!/bin/bash
file_to_compress="/path/to/file.txt"

gzip "$file_to_compress"
echo "File compressed:
$file_to_compress.gz"
```

18. Database Backup Script:

```
#!/bin/bash
database_name="your_database"
output_file="database_backup_$(date
+%Y%m%d).sql"

mysqldump-u username-ppassword
"$database_name" > "$output_file"
echo "Database backup created:
$output_file"
```

19. Git Repository Updater Script:

```
#!/bin/bash  
git_repo="/path/to/your/repo"
```

```
cd "$git_repo"  
git pull origin master  
echo "Git repository updated."
```

20. Directory Synchronization Script:

```
#!/bin/bash  
source_dir="/path/to/source"  
destination_dir="/path/to/destination"  
  
rsync-avz "$source_dir" "$destination_dir"  
echo "Directories synchronized  
successfully."
```

21. Web Server Log Analyzer Script:

```
#!/bin/bash  
log_file="/var/log/apache2/access.log"
```

```
awk '{print $1}' "$log_file" | sort | uniq-c |  
sort-nr  
echo "Web server log analyzed."
```

22. System Health Check Script:

```
#!/bin/bash  
output_file="system_health_check.txt"  
  
echo "System Health Check:" >  
"$output_file"  
echo "-----" >> "$output_file"  
echo "Uptime: $(uptime)" >>  
"$output_file"  
echo "Load Average: $(cat /proc/loadavg)"  
>> "$output_file"  
echo "Memory Usage: $(free-m)" >>  
"$output_file"  
echo "System health check results saved to  
$output_file."
```

23. Automated Database Cleanup

Script:

```
#!/bin/bash
database_name="your_database"
days_to_keep=7

find /path/to/database/backups-name
"$database_name*.sql"-mtime
+"$days_to_keep"-exec rm {} \;
echo "Old database backups cleaned up."
```

24. User Password Expiry Checker

Script:

```
#!/bin/bash

IFS=$'\n'
for user in $(cat /etc/passwd | grep
"/bin/bash" | cut-d:-f1); do
```

```
password_expires=$(chage-l "$user" |  
grep "Password expires" | awk '{print $4}')  
echo "User: $user, Password Expires:  
$password_expires"  
done  
unset IFS
```

25. Service Restart Script:

```
#!/bin/bash  
service_name="your_service"  
  
sudo systemctl restart "$service_name"  
echo "Service $service_name restarted."
```

26. Folder Size Checker Script:

```
#!/bin/bash  
folder_path="/path/to/folder"  
  
du-sh "$folder_path"
```

```
echo "Folder size checked."
```

27. Backup Rotation Script:

```
#!/bin/bash
backup_dir="/path/to/backups"
max_backups=5

while [ $(ls -1 "$backup_dir" | wc -l) -gt
"$max_backups" ]; do
    oldest_backup=$(ls -1t "$backup_dir" |
tail -n 1)
    rm -r "$backup_dir/$oldest_backup"
done
echo "Backup rotation completed."
```

28. Remote Script Execution Script:

```
#!/bin/bash
remote_server="user@remote-server"
```

```
remote_script="/path/to/remote/script.sh  
"
```

```
ssh "$remote_server" "bash-s" <  
"$remote_script"  
echo "Remote script executed."
```

29. Network Interface Information Script:

```
#!/bin/bash  
network_interface="eth0"
```

```
ifconfig "$network_interface"  
echo "Network interface information  
displayed."
```

30. Random Quotes Generator Script:

```
#!/bin/bash
```



```
quotes=("Quote 1" "Quote 2" "Quote 3"  
"Quote 4")
```

```
random_index=$((RANDOM %  
${#quotes[@]}))  
echo "Random Quote:  
${quotes[$random_index]}"
```

Linux 200+ Technical Interview Questions:

In some questions I am encouraging to search online because it will help you practice for searching more complicated questions in the future

(Good Luck)

1. When you login you get "\$" prompt, what is the prompt for root?
#
2. Explain the difference between grep and egrep?
Search online
3. What is the port # for DNS, NTP and NFS?
53,123 and 111/2049
4. What is the configuration file name of DNS and where is it located?
/etc/named.conf
5. How many new directories will be created after running the following command
mkdir {a..c}{1..3}
9
6. Your PC is configured with a DNS server address but not the default gateway. Can the PC access internet?
No
7. What is the difference between IP and Gateway?
Search online
8. Can you assign one static IP to 2 computers, if not then why?
No because it will create IP conflict
9. How to change IPs address to static?
ifconfig x.x.x.x
10. You are trying to ping a server by hostname and you get an error message, "ping: unknown host ...". What could be the reason and how to solve the problem so you can ping it by hostname?
Check for /etc/hosts or DNS to see if it has hostname to IP entry
11. Explain the difference between relative and absolute path?
Absolute path starts from / where relative path is your current directory
12. List 3 different methods of adding user?
Search online
13. What is the command to change file/directory ownership and group?
chown and chgrp
14. List any 3 type of filesystem?
ext4,NTFS and FAT

15. When you login you get a message on the screen. What is the name of that file and where is it located?
/etc/motd
16. What is /bin directory used for?
Search online
17. What are the different types of DNS Server
Master and secondary
18. How to change a user password?
passwd username
19. What is the version of Redhat Linux you have experience with?
7.4
20. List any 4 linux distributions?
Redhat, CentOS, Ubuntu and SUSE
21. How to logoff from the system?
exit
22. Give any 3 examples of operating systems?
Windows, Linux and MAC
23. How to create a directory?
mkdir
24. Where are the zone files located for DNS service?
/var/named/zonefiles
25. How to check kernel version?
uname -a
26. Which directory has all the configuration files?
/etc
27. How to become root user from a regular user?
su -
28. How many mega bytes in 1 giga bytes?
Search online
29. What is the purpose of having different network ports?
So the communication of each application goes through a dedicated port
30. How to display first column of a file?
cat filename | awk '{print \$1}'
31. What is the name of DNS rpm package?
bind
32. What is the difference between nslookup and dig commands?
Search online
33. How to check your user id and group id?

id

- 34. How to check a file's permission?
ls -l
- 35. What is the difference between "kill" and "kill -9" command?
Search online
- 36. What is subnet?
Search online
- 37. You are troubleshooting an issue with Redhat support and they have asked you to send the contents of /etc directory. How and which method you will use to transfer the contents?
tar (compress) the entire /etc directory and ftp
- 38. What is root home directory?
/root
- 39. What is rsyslogd daemon and its purpose?
Search online
- 40. Your company has terminated a server administrator. What is first thing as an administrator you should do to enhance the security?
Change root password
- 41. How to check the computer name or host name in Linux?
hostname
- 42. Which permission allows a user to run an executable with the permissions of the owner of that file?
First 3 bits should have x
- 43. What is the command to untar a tarred file?
untar
- 44. What is /proc directory used for?
Search online
- 45. What is the purpose of nsswitch.conf file
It tells the system where to go to resolve hostnames
- 46. List 3 basic commands to navigate the filesystem?
cd, pwd and ls
- 47. Which service/daemon should be running on the server that allows you to connect remotely?
sshd
- 48. What is the purpose of firewall?
Search online
- 49. List any 3 IT components?
Hardware, OS and Applications
- 50. Which directory has all the commands we use, e.g. ls, cd etc.?
/usr/bin or /bin

51. What is the difference between memory, virtual memory and cache?
Search online
52. Which of the following is correct?
a. **Hardware → Operating System → Users**
b. Operating System → Users → Hardware
c. Database → Hardware → Users
53. Which of the following is a communication command?
o grep
o **mail**
o touch
o cd
54. How to rename a file or directory?
mv
55. How to change a hostname in Linux?
Search online
56. How to check network interfaces in Linux?
ifconfig
57. Why is "tail -f logfilename" command used most often and what does it do?
It will output all incoming logs in real time
58. What type of hardware have you worked on?
You should get yourself familiar with Dell, HP and UCS hardware by going online and check the vendor websites
59. How to sort a file in reverse order?
cat filename | sort -r
60. What is the name of operating system that runs Unix?
Solaris, HP-UX etc.
61. List all byte sizes from smallest to largest?
Search online
62. How to check the total number of partition in Linux?
fdisk -l
63. How to access a linux system from a linux system?
ssh
64. Explain the procedure of bonding 2 NICs or interfaces together?
Search online
65. What is the exact command syntax to list the 5th column of a file and cut the first 3 letters?
cat filename | awk '{print \$5}' | cut -c1-3
66. What is /etc/hosts file used for?
To resolve hostnames with IP address

67. List any 3 options of 'df' command and what they are used for?
Search online
68. What is the command to change file/directory permissions?
chmod
69. What is the purpose of pipe (|)?
To combine multiple commands
70. What is /etc directory used for?
For configuration files
71. Which command is used to list files in a directory?
ls -l
72. There is a command which gives you information about other commands, please explain that command and what is it used for?
man
73. How to delete a file and a directory?
rm filename and rmdir dirname
74. What is the difference between "tail" and "tail -10"?
None
75. List 4 commands to display or read a file contents?
cat, more, less, vi
76. Which command is used to read the top 5 lines of a file?
head -5 filename
77. What are the different commands or methods to write to a file?
echo > filename and vi filename
78. What is swap space and how to check swap space?
Search online
79. What is inode and how to find an inode of a file?
Search online
80. Which file to edit for kernel tuning?
Search online
81. What is the latest version of Redhat?
Search online
82. Name the command to find specific word from a file?
grep word filename
83. You have scheduled a job using crontab but it does not run at the time you specified, what could be the reason and how would you troubleshoot?
Check your system time
Check your crontab entry
Check /var/log/messages
84. How to check system hardware information?

dmidecode

85. How to check network interface MAC address?
ifconfig
86. If I don't want others to read my file1, how to do that?
Remove r from the last 3 bits of file permission
87. What is the purpose of "uniq" and "sed" command?
Search online
88. Which command is used to list the contents of a directory in the most recent time and in reverse order, meaning the most updated file should be listed on the bottom?
ls -ltr
89. What is the difference between tar, gzip and gunzip?
Search online
90. What are the different ways to install an OS?
DVD, DVD iso and network boot
91. How to view difference between two files?
diff file1 and file2
92. You noticed that one of the Linux servers has no disk space left, how would you troubleshoot that issue?
If running LVM then add more disk and extend LVM
If not running LVM then add more disk, create a new partition and link the new partition to an existing filesystem
93. How to check Redhat version release?
uname -a or /etc/redhat-release
94. What is the difference between TCP and UDP?
Search online
95. What is a zombie process?
Search online
96. How do you search for a pattern/word in a file and then replace it in an entire file?
sed command
97. Explain the purpose of "touch" command?
To create an empty file
98. If a command hangs, how to stop it and get the prompt back?
Ctrl C
99. Which command is used to count words or lines?
wc
100. How to check the number of users logged in?
who
101. What is the command to view the calendar of 2011?

cal 2011

102. Which command is used to view disk space?
df -h
103. How to create a new group in Linux?
groupadd
104. What is the command to send a message to everyone who is logged into the system?
wall
105. Which command is used to check total number of disks?
fdisk -l
106. What is an mail server record in DNS?
MX
107. What does the following command line do?
`ps -ef | awk '{print $1}' | sort | uniq`
List the first column of all running processes, sort them and remove duplicates
108. You get a call that when a user goes to www.yourwebsite.com it fails and gets an error, how do you troubleshoot?
Check for user internet
Check to see if user computer has DNS for hostname lookup
Check to see if the server is up that is running that website
Check to see if the server's web service is running
Check for DNS availability which is resolving that website
109. List 4 different directories in /?
/etc, /bin, /tmp, /home
110. What is the output of the following command:
`$tail -10 filename | head -1`
It will show the first line from the last 10 lines of a file
111. What are the different fields in /etc/passwd file?
Search online
112. Which command is used to list the processes?
ps -ef
113. What is the difference between "hostname" and "uname" commands?
Hostname will give you system name and uname will give you OS information
114. How to check system load?
top and uptime command
115. How to schedule jobs?
crontab and at
116. What is the 3rd field when setting up crontab?
Day of the month

117. What is the command to create a new user?
useradd
118. What is the "init #" for system reboot?
6
119. How to restart a service?
systemctl restart servicename
120. How to shutdown a system?
shutdown or init 0
121. What is "ftp" command used for?
To transfer files from one computer to another
122. Explain cron job syntax? First is minute, second is..?
Min, house, day of the month, month, day of the week and command
123. How to delete a package in Linux?
rpm -e packagename
124. What is the file name where user password information is saved?
/etc/shadow
125. Which command you would use to find the location of chmod command?
which chmod
126. Which command is used to check if the other computer is online?
ping othercomputer
127. Please explain about LAN, MAN and WAN?
Search online
128. How to list hidden files in a directory?
ls -la
129. What is the difference between telnet and ssh?
ssh is secure where telnet is not
130. How to run a calculator on Linux and exit out of it?
bc and quit
131. List any 4 commands to monitor system?
top, df -h, iostat, dmesg
132. You are notified that your server is down, list the steps you will take to troubleshoot?
Check the system physically
Login through system console
Ping the system
Reboot or boot if possible
133. What is difference between static and DHCP IP?
Search online
134. How to write in vi editor mode?
i = insert, a = insert in next space, o = insert in new line

135. What is the difference between "crontab" and "at" jobs?
crontab is for repetitive jobs where at is for one time job
136. What is vCenter server in VMWare?
Search online
137. What is "dmidecode" command used for?
To get system information
138. What is the difference between SAN and NAS?
Search online
139. What is the location of system logs? E.g. messages
/var/log directory
140. How to setup an alias and what is it used for?
alias aliasname="command"
It is used to created short-cuts for long commands
141. What is the purpose of "netstat" command?
Search online
142. What are terminal control keys, list any 3?
Crtl C, D and Z
143. Which command(s) you would run if you need to find out how many processes are running on your system?
ps -ef | wc -l
144. What are the different types of shells?
sh, bash, ksh, csh etc.
145. How to delete a line when in vi editor mode?
dd
146. Which is the core of the operating system?
a) Shell
b) Kernel
c) Commands
d) Script
147. Which among the following interacts directly with system hardware?
a) Shell
b) Commands
c) Kernel
d) Applications
148. How to save and quit from vi editor?
Shift ZZ or :wq!
149. What is the difference between a process and daemon?
Search online
150. What is the process or daemon name for NTP?
ntpd

151. What are a few commands you would run if your system is running slow?
top, iostat, df -h, netstat etc.
152. How to install a package in Redhat Linux?
yum install packagename
153. What is the difference between "ifconfig" and "ipconfig" commands?
ifconfig for Linux and ipconfig for Windows
154. What is the first line written in a shell script?
Define shell
e.g. #!/bin/bash
155. Where is the network (Ethernet) file located, please provide exact directory location and file name?
/etc/sysconfig/network-scripts/ifcfg-nic
156. Why do we use "last" command?
To see who has logged in the system whether active or logged off
157. What is RHEL Linux stands for?
Search online
158. To view your command history, which command is used and how to run a specific command?
history and history #
159. What is NTP and briefly explain how does it work and where is the config files and related commands of NTP?
Search online
160. How to disable firewall in Linux?
Search online
161. How to configure mail server relay for sendmail service?
Edit /etc/mail/sendmail.mc file and add SMART_HOST entry
162. Where is samba log file located?
/var/log/samba
163. What is mkfs command used for?
To create a new filesystem
164. If you create a new group, which file does it get created in?
/etc/group
165. Which file has DNS server information (e.g. DNS resolution)?
/etc/resolv.conf
166. What are the commands you would run if you need to find out the version and build date of a package (e.g. http)?
rpm -qi http

167. On the file permissions? What are the first 3 bits for and who is it for?
Read, write and execute. They are used for the owner of the file
168. How to create a soft link?
ln -s
169. How to write a script to delete messages in a log file older than 30 days automatically?
Search online
170. How to quit out of "man" command?
q
171. Which command is used to partition disk in Linux?
fdisk
172. What is the difference between "shutdown" and "halt" command?
Search online
173. What is the exact syntax of mounting NFS share on a client and also how to un-mount?
Search online
174. What experience do you have with scripting, explain?
if-the, do-while, case, for loop scripts
175. How to get information on all the packages installed on the system?
rpm -qa
176. Explain VMWare?
Search online
177. You are tasked to examine a log file in order to find out why a particular application keep crashing. Log file is very lengthy, which command can you use to simplify the log search using a search string?
grep for error, warning, failure etc. in /var/log/messages file
178. What is /etc/fstab file and explain each column of this file?
Search online
179. What the latest version of Windows server?
Search online
180. What is the exact command to list only the first 2 lines of history output?
history | head -2
181. How to upgrade Linux from 7.3 to 7.4?
yum install update
182. How to tell which shell you are in or running?
\$0

183. You have tried to "cd" into a directory but you have been denied. You are not the owner of that directory, what permissions do you need and where?

- - - - - r - x

184. What is CNAME record in DNS?
Entry for hostname to hostname
185. What is the name of VMWare operating system?
ESXi
186. What is the client name used to connect to ESXi or vCenter server?
vSphere client
187. You get a call from a user saying that I cannot write to a file because it says, permission denied. The file is owned by that user, how do you troubleshoot?
Give write permission on the first 3 bits
188. What is the latest version of VMWare?
Search online
189. What is the name of firewall daemon in Linux?
firewalld
190. Which command syntax you can use to list only the 20th line of a file?
Search online
191. What is the difference between run level 3 and 5?
3 = Boot system with networking, 5 = boot system with networking and GUI
192. List a few commands that are used in troubleshooting network related issue?
netstat, tcpdump etc.
193. What is the difference between domain and nameserver?
Search online
194. You open up a file and it has 3000 lines and it scrolled up really fast, which command you will use to view it one page at a time?
more or less
195. How to start a new shell. E.g. start a new ksh shell?
Simply type ksh, or bash
196. How to kill a process?
kill processID
197. How to check scheduled jobs?
crontab -l

198. How to check system memory and CPU usage?
free and top
199. Which utility could you use to repair the corrupted file system?
fsck
200. What is the command to make a service start at boot?
systemctl enable servicename
201. How to combine 2 files into 1? E.g. you 3 lines in file "A" and 5 lines in file "B", which command syntax to use that will combine into one file of 3+5 = 8 lines
cat fileA >> fileB
202. What is echo command used for?
To output to a screen
203. What does the following command do?
echo This year the summer will be great > file1
It will create a new file "file1" with the content as "This year the summer will be great"
204. Which file to modify to allow users to run root commands
/etc/sudoers
205. You need to modify httpd.conf file but you cannot find it, Which command line tool you can use to find file?
find / -name "httpd.conf"
206. Your system crashed and being restarted, but a message appears, indicating that the operating system cannot be found. What is the most likely cause of the problem?
The /boot file is most likely corrupted

DevOps Shack

LINUX TROUBLESHOOTING GUIDE

1. System Boot Issues

1.1 Boot Loader Issues (GRUB Issues)

Symptoms:

- System does not boot into Linux.
- Stuck at GRUB prompt.
- Error: **grub-rescue>**

Causes:

- Corrupt or missing GRUB configuration.
- Wrong boot device.
- Kernel update failure.

Solution:

1. Boot using a Live CD/USB.

Mount the root partition:

```
sudo mount /dev/sdXn /mnt
```

Reinstall GRUB:

```
sudo grub-install --root-directory=/mnt /dev/sdX
```

Update GRUB:

```
sudo update-grub
```

1.2 Kernel Panic



Symptoms:

- Black screen with panic messages.
- **Kernel panic – not syncing** error.

Causes:

- Incompatible kernel update.
- Corrupt boot files.

Solution:

1. Reboot and select an older kernel from GRUB.

Check logs using:

journalctl -xb

2. Roll back the kernel update if necessary.

2. Login and User Issues

2.1 Forgotten Root Password

Symptoms:

- Cannot log in as root.

Solution:

1. Boot into recovery mode.

Remount root partition:

mount -o remount,rw /



Reset password:

`passwd root`

2. Reboot.

2.2 SSH Connection Refused

Symptoms:

- **Connection refused** when trying to SSH.

Causes:

- SSH service not running.
- Firewall blocking SSH.

Solution:

Check SSH status:

`sudo systemctl status sshd`

Start service if stopped:

`sudo systemctl start sshd`

Check firewall rules:

`sudo ufw allow ssh`

3. File System and Disk Issues

3.1 Disk Space Full

Symptoms:

- **No space left on device** error.

Solution:



Check disk usage:

```
df -h
```

Find large files:

```
du -ah / | sort -rh | head -20
```

1. Delete unnecessary files.

3.2 Read-Only File System

Symptoms:

- Read-only file system error.

Solution:

Remount the filesystem:

```
sudo mount -o remount,rw /
```

Check and repair disk errors:

```
sudo fsck -y /dev/sdXn
```

4. Process and Performance Issues

4.1 High CPU Usage

Symptoms:

- System slows down.
- High load average in top output.

Solution:

Identify culprit process:

```
top
```



Kill process:

```
kill -9 <PID>
```

5. Networking Issues

5.1 No Internet Connection

Solution:

Check network interfaces:

```
ip a
```

Restart networking:

```
sudo systemctl restart NetworkManager
```

6. Package Management Issues

6.1 Broken Package Dependencies

Symptoms:

- Unable to correct problems, you have held broken packages
- Package installation fails

Solution:

Identify broken packages:

```
sudo apt --fix-broken install
```

Clean package cache:

```
sudo apt clean && sudo apt autoremove
```

Reinstall dependencies:

```
sudo apt install -f
```

6.2 Slow Package Installation

Symptoms:

- Installing or updating packages takes too long

Solution:

Update repository lists:

```
sudo apt update
```

Switch to a faster mirror:

```
sudo sed -i  
's/http:\/\/archive.ubuntu.com/http:\/\/mirror.example.com/'  
/etc/apt/sources.list
```

7. Service and Daemon Issues

7.1 Service Not Starting

Symptoms:

- Failed to start service-name.service

Solution:

Check service logs:

```
sudo journalctl -xe | grep service-name
```

Restart the service:

```
sudo systemctl restart service-name
```

1. Check configuration files for errors.



7.2 Port Conflicts

Symptoms:

- Service fails to bind to a port

Solution:

Identify the conflicting process:

```
sudo netstat -tulnp | grep <port-number>
```

1. Kill the process or reconfigure services.

8. Kernel and Module Issues

8.1 Kernel Update Breaks System

Symptoms:

- System does not boot after update

Solution:

1. Reboot and select an older kernel in GRUB.

Remove faulty kernel:

```
sudo apt remove linux-image-<version>
```

8.2 Missing Kernel Modules

Symptoms:

- `modprobe: FATAL: Module not found`

Solution:

Check available modules:

```
lsmod | grep module-name
```

Load the module manually:

```
sudo modprobe module-name
```

9. Security and Access Control Issues

9.1 Unauthorized Access Attempts

Symptoms:

- Unusual login attempts in logs

Solution:

Check logs:

```
sudo cat /var/log/auth.log | grep "Failed password"
```

Block IP using firewall:

```
sudo ufw deny from <IP-ADDRESS>
```

9.2 SELinux Blocking Services

Symptoms:

- Service fails due to permission errors

Solution:

Check SELinux logs:

```
sudo ausearch -m AVC
```

Temporarily disable SELinux (for debugging only):

```
sudo setenforce 0
```



10. Hardware Issues

10.1 USB Device Not Recognized

Symptoms:

- USB device does not show up in **lsusb**
- **dmesg** shows errors related to USB

Solution:

Check if the USB device is detected:

Lsusb

Re-scan USB devices:

```
echo "1" | sudo tee /sys/bus/usb/devices/usb1/authorized
```

Restart the USB service:

```
sudo modprobe -r usb_storage && sudo modprobe usb_storage
```

10.2 Hard Drive Not Detected

Symptoms:

- **fdisk -l** does not show the disk
- **dmesg** shows disk-related errors

Solution:

List available disks:



lsblk

Re-scan the disk:

```
echo "- - -" | sudo tee /sys/class/scsi_host/host0/scan
```

11. Time and Date Issues

11.1 System Clock Out of Sync

Symptoms:

- Incorrect system time

Solution:

Sync time with NTP:

```
sudo timedatectl set-ntp on
```

Manually set time:

```
sudo timedatectl set-time "YYYY-MM-DD HH:MM:SS"
```

12. Firewall and Network Security Issues

12.1 Firewall Blocking Services

Symptoms:

- Cannot access services from another system

Solution:



List active firewall rules:

```
sudo ufw status
```

Allow required ports:

```
sudo ufw allow 80/tcp
```

12.2 Too Many Failed SSH Attempts

Symptoms:

- Too many authentication failures in SSH

Solution:

Edit SSH configuration:

```
sudo nano /etc/ssh/sshd_config
```

Increase allowed authentication retries:

```
MaxAuthTries 10
```

Restart SSH:

```
sudo systemctl restart sshd
```

13. Virtualization Issues

13.1 Virtual Machine Won't Start



Symptoms:

- **Failed to start the virtual machine** error

Solution:

Check virtualization support:

```
egrep -c '(vmx|svm)' /proc/cpuinfo
```

1. Enable virtualization in BIOS.

14. Miscellaneous Issues

14.1 Command Not Found

Symptoms:

- **command not found** error

Solution:

Check if the package is installed:

```
dpkg -l | grep package-name
```

Install missing package:

```
sudo apt install package-name
```

15. Logs and Monitoring Issues



15.1 Log Files Not Updating

Symptoms:

- `/var/log/syslog` or `/var/log/messages` is empty or not updating.

Solution:

Restart the logging service:

```
sudo systemctl restart rsyslog
```

Check disk space:

```
df -h
```

Verify SELinux settings:

```
sudo ausearch -m AVC
```

15.2 How to Monitor System Logs in Real-Time

Solution:

Use `tail` to follow log updates:

```
tail -f /var/log/syslog
```

Use `journalctl` for systemd logs:

```
journalctl -f
```



16. SSH Issues

16.1 SSH Connection Timeout

Symptoms:

- SSH session freezes or disconnects after inactivity.

Solution:

Edit SSH config:

```
sudo nano /etc/ssh/sshd_config
```

Set **ClientAliveInterval** and **ClientAliveCountMax**:
plaintext

```
ClientAliveInterval 60  
ClientAliveCountMax 5
```

Restart SSH service:

```
sudo systemctl restart sshd
```

16.2 Cannot SSH as Root

Symptoms:



- **Permission denied** when connecting as root.

Solution:

Enable root login in SSH config:

```
sudo nano /etc/ssh/sshd_config
```

Set:

plaintext

```
PermitRootLogin yes
```

Restart SSH:

```
sudo systemctl restart sshd
```

17. Printer and Device Issues

17.1 Printer Not Working

Symptoms:

- Printer not detected.

Solution:

Restart the CUPS service:

```
sudo systemctl restart cups
```

Check printer connections:

```
lpstat -p
```



18. User Management Issues

18.1 User Cannot Run sudo Commands

Symptoms:

- **user is not in the sudoers file** error.

Solution:

Add user to the sudo group:

```
sudo usermod -aG sudo username
```

18.2 User Cannot Write to a Directory

Symptoms:

- **Permission denied** error.

Solution:

Change ownership:

```
sudo chown username:groupname /path/to/directory
```

Set proper permissions:

```
sudo chmod 755 /path/to/directory
```

19. Software and Application Issues

19.1 Application Crashes Unexpectedly

Solution:

Check application logs:

```
journalctl -xe | grep application-name
```

1. Run application in debug mode if available.

19.2 Web Server Not Working

Symptoms:

- **502 Bad Gateway** or **503 Service Unavailable**.

Solution:

Restart the web server:

```
sudo systemctl restart apache2 # For Apache  
sudo systemctl restart nginx   # For Nginx
```

20. Advanced Security Issues

20.1 Detecting Unauthorized Root Access

Solution:

Check recent root logins:

```
sudo last -n 10
```



Check for unauthorized sudo commands:

```
cat /var/log/auth.log | grep sudo
```

20.2 Blocking Brute-Force Attacks

Solution:

Install Fail2Ban:

```
sudo apt install fail2ban
```

Enable SSH protection:

```
sudo systemctl enable fail2ban
```

21. Disk and Storage Issues

21.1 Unable to Mount a Disk

Symptoms:

- **mount: unknown filesystem type** error
- Disk not visible in **lsblk** or **fdisk -l**

Solution:

Identify the disk:

```
lsblk
```

Check filesystem type:



```
sudo blkid /dev/sdX
```

Manually mount the disk:

```
sudo mount /dev/sdX /mnt
```

If the filesystem is corrupted, repair it:

```
sudo fsck -y /dev/sdX
```

21.2 LVM Volume Not Found

Symptoms:

- **lvdisplay** does not show expected volumes

Solution:

Scan for missing volumes:

```
sudo vgscan --mknodes  
sudo lvscan
```

Activate volume group:

```
sudo vgchange -ay
```

Mount the logical volume:

```
sudo mount /dev/mapper/vgname-lvname /mnt
```

22. Kernel and Driver Issues

22.1 Missing Graphics Drivers

Symptoms:

- GUI does not load
- **NVIDIA driver not found** error

Solution:

Identify the graphics card:

```
lspci | grep -i vga
```

Install the correct driver:

```
sudo ubuntu-drivers autoinstall
```

Reboot the system:

```
sudo reboot
```



22.2 Sound Not Working

Symptoms:

- No audio output
- **No sound device found** error

Solution:

Check audio devices:

```
aplay -l
```

Restart PulseAudio:

```
pulseaudio --kill && pulseaudio --start
```

Unmute sound:

```
amixer set Master unmute
```

23. Network Troubleshooting

23.1 Wi-Fi Not Connecting

Symptoms:



- **No Wi-Fi adapter found** error

Solution:

List available network interfaces:

```
nmcli device
```

Restart NetworkManager:

```
sudo systemctl restart NetworkManager
```

If the Wi-Fi adapter is missing, install drivers:

```
sudo apt install firmware-iwlwifi
```

23.2 IP Address Not Assigned

Symptoms:

- **ip a** does not show an IP address

Solution:

Restart DHCP client:

```
sudo dhclient -r && sudo dhclient
```



Manually assign an IP address:

```
sudo ip addr add 192.168.1.100/24 dev eth0
```

24. Performance Optimization

24.1 System Running Slow

Symptoms:

- High load average
- Applications freezing

Solution:

Check system load:

```
top
```

Kill high CPU usage processes:

```
kill -9 <PID>
```

Clear cached memory:

```
sudo sync && sudo sysctl -w vm.drop_caches=3
```



24.2 High Memory Usage

Symptoms:

- **Out of memory** errors

Solution:

Check memory usage:

```
free -h
```

Enable swap:

```
sudo fallocate -l 2G /swapfile  
sudo chmod 600 /swapfile  
sudo mkswap /swapfile  
sudo swapon /swapfile
```

25. Backup and Recovery

25.1 Restoring Deleted Files

Symptoms:

- Important files deleted accidentally

Solution:

Recover files from the trash:

```
ls ~/.local/share/Trash/files/
```

If permanently deleted, use **extundelete**:

```
sudo extundelete /dev/sdX --restore-all
```

25.2 Creating a Full System Backup

Solution:

Use **rsync** for backup:

```
sudo rsync -av --progress /home/ /backup/home/
```

Restore files when needed:

```
sudo rsync -av /backup/home/ /home/
```

26. Advanced Security

26.1 Encrypting a Directory

Solution:

Install **ecryptfs-utils**:

```
sudo apt install ecryptfs-utils
```

Encrypt a directory:

```
ecryptfs-mount-private
```

26.2 Setting Up Automatic Security Updates

Solution:

Install unattended-upgrades:

```
sudo apt install unattended-upgrades
```

Enable automatic updates:

```
sudo dpkg-reconfigure unattended-upgrades
```

27. Miscellaneous Troubleshooting

27.1 How to Find Which Process is Using a Port

Solution:

Use **netstat**:



```
sudo netstat -tulnp | grep :<port>
```

Use **lsof**:

```
sudo lsof -i :<port>
```

Kill the process if needed:

```
sudo kill -9 <PID>
```

27.2 How to Check System Uptime

Solution:

Display uptime:

```
uptime
```

View uptime in human-readable format:

```
who -b
```

28. Debugging and Log Analysis



28.1 Checking Kernel Logs for Errors

Solution:

View logs in real-time:

```
dmesg -w
```

Filter errors:

```
dmesg | grep -i error
```

28.2 Finding Recently Modified Files

Solution:

Search for files modified in the last 24 hours:

```
find / -type f -mtime -1
```

29. Essential Linux Commands

29.1 Finding a File by Name

Solution:

Use **find**:



```
find / -name "filename"
```

29.2 Checking Disk Usage per Directory

Solution:

Use **du** to find large directories:

```
du -ah / | sort -rh | head -10
```

30. Common Errors and Fixes

30.1 Fixing **command not found** for Common Commands

Solution:

Refresh the PATH:

```
export PATH=$PATH:/usr/local/bin
```

Reinstall the package:

```
sudo apt install package-name
```

30.2 Fixing **permission denied** When Running a Script

Solution:



Make the script executable:

```
chmod +x script.sh
```

Run the script with **sudo** if needed:

```
sudo ./script.sh
```

31. Advanced File System Issues

31.1 Filesystem Goes into Read-Only Mode

Symptoms:

- Unable to write files
- **dmesg** shows **EXT4-fs error**

Solution:

Check disk errors:

```
dmesg | tail -50
```

Remount the filesystem:

```
sudo mount -o remount,rw /
```



Run filesystem check:

```
sudo fsck -y /dev/sdX
```

31.2 Recovering Deleted Files from EXT4 Filesystem

Solution:

Unmount the affected partition:

```
sudo umount /dev/sdX
```

Use **extundelete**:

```
sudo extundelete /dev/sdX --restore-all
```

32. Advanced Networking Issues

32.1 Identifying Slow Network Connections

Symptoms:

- Websites load slowly
- High latency on SSH

Solution:



Test network speed:

```
speedtest-cli
```

Identify packet loss:

```
ping -c 10 google.com
```

Check for dropped packets:

```
sudo netstat -s | grep "dropped"
```

32.2 Troubleshooting DNS Issues

Symptoms:

- `ping google.com` fails, but `ping 8.8.8.8` works

Solution:

Check DNS settings:

```
cat /etc/resolv.conf
```



Change to Google DNS:

```
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

33. Docker and Container Issues

33.1 Docker Container Won't Start

Symptoms:

- **docker run** fails with an error

Solution:

Check container logs:

```
docker logs container_id
```

Restart Docker:

```
sudo systemctl restart docker
```

Prune unused containers:

```
docker system prune -a
```



33.2 Docker Container Can't Access the Internet

Solution:

Restart the Docker network:

```
sudo systemctl restart docker
```

Add network to the container:

```
docker network connect bridge container_id
```

34. Kernel Panic and Boot Issues

34.1 Kernel Panic After Update

Symptoms:

- System does not boot after an update

Solution:

1. Boot into an older kernel from GRUB.

Reinstall the latest kernel:

```
sudo apt install --reinstall linux-image-$(uname -r)
```


34.2 System Stuck at GRUB Rescue Mode

Solution:

Identify the boot partition:

```
ls
```

Load the correct partition:

```
set prefix=(hd0,gpt2)/boot/grub
set root=(hd0,gpt2)
insmod normal
normal
```

35. High Availability and Load Balancing Issues

35.1 Load Balancer Not Distributing Traffic Evenly

Solution:

Check Nginx load balancer configuration:

```
sudo nano /etc/nginx/nginx.conf
```

Verify upstream servers:



```
sudo systemctl restart nginx
```

36. Virtualization and Cloud Issues

36.1 VM is Running Slowly

Solution:

Enable CPU virtualization:

```
egrep -c '(vmx|svm)' /proc/cpuinfo
```

1. Increase allocated RAM and CPU in the hypervisor.

36.2 Cloud Instance Fails to Boot

Solution:

1. Check the instance console logs.
2. Attach the disk to another instance and repair the OS files.

37. SELinux and AppArmor Issues

37.1 Application Blocked by SELinux

Symptoms:

- **Permission denied** errors even with correct permissions

Solution:

Check SELinux logs:

```
sudo ausearch -m AVC
```

Set the correct SELinux policy:

```
sudo semanage fcontext -a -t httpd_sys_content_t  
"/var/www/html(/.*)?"  
sudo restorecon -Rv /var/www/html
```

37.2 AppArmor Blocking a Service

Solution:

Check AppArmor logs:

```
sudo journalctl -xe | grep apparmor
```

Set AppArmor profile to complain mode:

```
sudo aa-complain /usr/sbin/nginx
```

38. System Recovery and Rescue Mode

38.1 How to Boot into Single User Mode

Solution:



1. Edit the GRUB boot entry.
2. Add `single` or `init=/bin/` to the kernel parameters.

38.2 Resetting Root Password from Recovery Mode

Solution:

1. Boot into recovery mode.

Mount the root filesystem:

```
mount -o remount,rw /
```

Reset password:

```
passwd root
```

39. Advanced Performance Tuning

39.1 Identifying Bottlenecks with iostat

Solution:

Install `sysstat`:

```
sudo apt install sysstat
```

Check disk I/O performance:

```
iostat -x 1 5
```

39.2 Reducing Swap Usage

Solution:

Check current swap usage:

```
swapon -s
```

Lower the swappiness value:

```
sudo sysctl vm.swappiness=10
```

40. Debugging and Profiling Applications

40.1 Debugging a Segmentation Fault

Solution:

Run the application with **gdb**:

```
gdb ./application  
run
```

40.2 Profiling CPU Usage with **perf**

Solution:

Install **perf**:

```
sudo apt install linux-tools-common
```

1. Profile CPU usage:

```
sudo perf top
```

1. User and Group Management

- `useradd` - Add a new user
- `usermod` - Modify user account
- `userdel` - Delete a user
- `groupadd` - Add a new group
- `groupdel` - Delete a group
- `chgrp` - Change group ownership of a file
- `passwd` - Change user password
- `Who` - Who is logged in

2. File and Directory Management

- `ls` - List directory contents
- `cd` - Change directory
- `pwd` - Print working directory
- `mkdir` - Make a new directory
- `rmdir` - Remove a directory
- `rm` - Remove files or directories
- `cp` - Copy files or directories
- `mv` - Move or rename files or directories
- `touch` - Create empty files
- `find` - Search for file in a directory hierarchy
- `locate` - Find files by name

3.File Management

- **cat** - Concatenate and display file contents
- **more** - View file content page by page
- **less** - View file content with navigation
- **head** - View the first part of a file
- **tail** - View the last part of a file
- **nano** - Text editor to edit files
- **vi** - Text editor to edit files
- **touch** - Create an empty file or update file timestamp
- **ln** - Create hard or symbolic links
- **file** - Determine file type
- **stat** - Display file or file system status
- **find** - Search for files in a directory hierarchy
- **locate** - Find files by name (using a database)
- **updatedb**-Updatethedatabaseusedbylocate

4.File Permissions and Ownership

- `chmod` - Change file permissions
- `chown` - Change file owner and group
- `chgrp` - Change group ownership
- `umask` - Set file creation mask
- `getfacl` - Get file access control lists (ACL)
- `setfacl` - Set file access control lists (ACL)
- `ls -l` - List files with detailed permissions, owner, and group
- `stat` - Display file or file system status
- `acl` - View or modify Access Control Lists (ACL) on files

5.Disk Management

- **df** - Display free disk space
- **du** - Estimate file space usage
- **fdisk** - Partition table manipulator
- **parted** - A tool for creating and managing disk partitions
- **lsblk** - List information about block devices
- **blkid** - Find or display block device attributes
- **mount** - Mount a file system
- **umount** - Unmount a file system
- **mkfs** - Create a file system
- **fsck** - Check and repair file systems
- **tune2fs** - Adjust tunable file system parameters
- **resize2fs** - Resize an ext2, ext3, or ext4 file system
- **lvcreate** - Create a logical volume
- **vgcreate** - Create a volume group
- **pvccreate** - Prepare a physical volume for LVM
- **lsattr** - List file attributes on a file system
- **chattr** - Change file attributes on a file system

6.Process Management

- **ps** - Display current processes
- **top** - Display dynamic process information
- **htop** - Interactive process viewer
- **kill** - Terminate a process by ID
- **killall** - Kill all processes by name
- **bg** - Resume a suspended process in the background
- **fg** - Bring a process to the foreground
- **nice** - Set process priority
- **renice** - Change the priority of running processes
- **jobs** - List active jobs
- **pgrep** - Search for processes by name or other attributes
- **pkill** - Kill processes by name or other attributes
- **ps tree** - Display processes in a tree format
- **lsof** - List open files and the processes that opened them
- **strace** - Trace system calls and signals
- **timeout** - Run a command with a time limit
- **watch** - Execute a program periodically and show output

7.Network Management

- `ifconfig` - Display and configure network interfaces
- `ip a` - Show all network interfaces and their IPs
- `ip r` - Display the routing table
- `ip link set <interface> up/down` - Enable or disable a network interface
- `ping <host>` - Send ICMP echo request to check connectivity
- `traceroute <host>` - Trace the route packets take to a network host
- `netstat -tulnp` - Show open ports and listening services
- `ss -tulnp` - Display socket statistics and listening services
- `hostname -I` - Show the system's IP address
- `dig <domain>` - Query DNS records
- `nslookup <domain>` - Lookup DNS details
- `whois <domain>` - Get domain registration details
- `curl -I <URL>` - Fetch HTTP headers from a website
- `wget <URL>` - Download files from the web
- `scp <file> user@host:/path` - Securely copy files over SSH
- `rsync -avz <src> <dest>` - Sync files between local and remote locations
- `tcpdump -i eth0` - Capture network packets on an interface
- `nmap <IP>` - Scan a network for open ports

8. System and Monitoring

- **uptime** - Show system uptime and load average
- **dmesg** - Print system boot and hardware logs
- **free -m** - Display memory usage in MB
- **vmstat** - Report system performance statistics
- **iostat** - Display CPU and disk I/O statistics
- **mpstat** - Show CPU usage per core
- **top** - Display dynamic process and system information
- **htop** - Interactive process and resource monitoring
- **sar -u 5 10** - Monitor CPU usage at 5-second intervals
- **iotop** - Display disk read/write usage per process
- **lsof** - List open files by processes
- **watch -n 2 <command>** - Run a command every 2 seconds
- **journalctl -xe** - View system logs with extended details
- **systemctl status <service>** - Check the status of a service
- **service <service> status** - View service status (legacy systems)
- **df -h** - Show disk space usage in a human-readable format
- **du -sh <directory>** - Display the size of a directory
- **ps aux --sort=-%mem** - List processes sorted by memory usage
- **netstat -tulnp** - Show active network connections and listening services

9.Text Processing

- **cat <file>** - Display file contents
- **tac <file>** - Display file contents in reverse order

• head	-n	<num> <file> - Show the first <num> lines of a file
• tail	-n	<num> <file> - Show the last <num> lines of a file
• tail	-f	<file> - Continuously display new lines in a file

- **grep '<pattern>' <file>** - Search for a pattern in a file
- **grep -r '<pattern>' <directory>** - Recursively search for a pattern in a directory
- **awk '{print \$1, \$2}' <file>** - Process and extract columns from a file
- **sed 's/old/new/g' <file>** - Replace text in a file
- **cut -d':' -f1 <file>** - Extract specific fields from a file
- **sort <file>** - Sort file contents
- **uniq <file>** - Remove duplicate lines from a file

• wc	-l	<file> - Count lines in a file
• wc	-w	<file> - Count words in a file
• wc	-c	<file> - Count bytes in a file

- **tr 'a-z' 'A-Z' <file>** - Convert lowercase to uppercase
- **diff <file1> <file2>** - Compare two files line by line
- **cmp <file1> <file2>** - Compare two files byte by byte
- **tee <file>** - Read from standard input and write to both a file and standard output

10. Package Management

For Debian-based systems (Ubuntu, Debian)

- **apt update** - Update package lists
- **apt upgrade** - Upgrade all installed packages
- **apt install <package>** - Install a package
- **apt remove <package>** - Remove a package
- **apt purge <package>** - Remove a package and its config files
- **dpkg -i <package>.deb** - Install a .deb package
- **dpkg -r <package>** - Remove a package installed via dpkg
- **dpkg -l** - List installed packages
- **apt-cache search <package>** - Search for a package
- **apt autoremove** - Remove unnecessary packages
- **apt clean** - Clear the package cache

For RHEL-based systems (CentOS, Rocky Linux, AlmaLinux, Fedora)

- **yum update** - Update installed packages (for older systems)
- **dnf update** - Update installed packages (for newer systems)
- **yum install <package>** - Install a package
- **dnf install <package>** - Install a package (newer distros)
- **yum remove <package>** - Remove a package
- **dnf remove <package>** - Remove a package (newer distros)
- **rpm -ivh <package>.rpm** - Install an .rpm package
- **rpm -e <package>** - Remove an installed .rpm package
- **yum list installed** - List installed packages
- **dnf list installed** - List installed packages (newer distros)
- **yum search <package>** - Search for a package
- **dnf search <package>** - Search for a package (newer distros)
- **yum clean all** - Clean cached package data

11. Archive and Compression

Working with tar files

• tar	-cvf	archive.tar <files> - Create a tar archive
• tar	-xvf	archive.tar - Extract a tar archive
• tar	-tvf	archive.tar - List files in a tar archive

- **tar -czvf archive.tar.gz <files>** - Create a compressed tar.gz archive
- **tar -xzvf archive.tar.gz** - Extract a tar.gz archive
- **tar -cJvf archive.tar.xz <files>** - Create a tar.xz archive
- **tar -xJvf archive.tar.xz** - Extract a tar.xz archive

Working with zip files

- **zip archive.zip <files>** - Create a zip archive
- **unzip archive.zip** - Extract a zip archive
- **unzip -l archive.zip** - List contents of a zip file
- **zip -r archive.zip <directory>** - Zip a directory

Working with gzip & bzip2

- **gzip <file>** - Compress a file using gzip
- **gunzip <file>.gz** - Decompress a gzip file
- **bzip2 <file>** - Compress a file using bzip2
- **bunzip2 <file>.bz2** - Decompress a bzip2 file

12.Task Scheduling

- **crontab -e** - Edit cron jobs for the current user
- **crontab -l** - List scheduled cron jobs
- **crontab -r** - Remove all cron jobs for the current user
- **crontab -u <user> -l** - List cron jobs for a specific user
- **at <time>** - Schedule a command to run at specified time

13. System Diagnostics and Troubleshooting

- **journalctl -xe** - View detailed system logs
- **journalctl -f** - Follow live logs in real-time
- **journalctl --since "1 hour ago"** - View logs from the last hour
- **dmesg | less** - View system boot logs
- **dmesg | grep error** - Check for errors in system logs
- **cat /var/log/syslog** - View general system logs (Debian-based)
- **cat /var/log/messages** - View general system logs (RHEL-based)
- **uptime** - Show system uptime and load
- **free -m** - Display available and used memory in MB
- **top** - Monitor real-time CPU, memory, and process usage
- **htop** - Interactive system monitoring (if installed)
- **vmstat 1 5** - Display system performance stats
- **iostat** - Show CPU and disk usage statistics

Linux 200+ Technical Interview Questions:

In some questions I am encouraging to search online because it will help you practice for searching more complicated questions in the future

(Good Luck)

1. When you login you get "\$" prompt, what is the prompt for root?
#
2. Explain the difference between grep and egrep?
Search online
3. What is the port # for DNS, NTP and NFS?
53,123 and 111/2049
4. What is the configuration file name of DNS and where is it located?
/etc/named.conf
5. How many new directories will be created after running the following command
mkdir {a..c}{1..3}
9
6. Your PC is configured with a DNS server address but not the default gateway. Can the PC access internet?
No
7. What is the difference between IP and Gateway?
Search online
8. Can you assign one static IP to 2 computers, if not then why?
No because it will create IP conflict
9. How to change IPs address to static?
ifconfig x.x.x.x
10. You are trying to ping a server by hostname and you get an error message, "ping: unknown host ...". What could be the reason and how to solve the problem so you can ping it by hostname?
Check for /etc/hosts or DNS to see if it has hostname to IP entry
11. Explain the difference between relative and absolute path?
Absolute path starts from / where relative path is your current directory
12. List 3 different methods of adding user?
Search online
13. What is the command to change file/directory ownership and group?
chown and chgrp
14. List any 3 type of filesystem?
ext4,NTFS and FAT

15. When you login you get a message on the screen. What is the name of that file and where is it located?
/etc/motd
16. What is /bin directory used for?
Search online
17. What are the different types of DNS Server
Master and secondary
18. How to change a user password?
passwd username
19. What is the version of Redhat Linux you have experience with?
7.4
20. List any 4 linux distributions?
Redhat, CentOS, Ubuntu and SUSE
21. How to logoff from the system?
exit
22. Give any 3 examples of operating systems?
Windows, Linux and MAC
23. How to create a directory?
mkdir
24. Where are the zone files located for DNS service?
/var/named/zonefiles
25. How to check kernel version?
uname -a
26. Which directory has all the configuration files?
/etc
27. How to become root user from a regular user?
su -
28. How many mega bytes in 1 giga bytes?
Search online
29. What is the purpose of having different network ports?
So the communication of each application goes through a dedicated port
30. How to display first column of a file?
cat filename | awk '{print \$1}'
31. What is the name of DNS rpm package?
bind
32. What is the difference between nslookup and dig commands?
Search online
33. How to check your user id and group id?

id

34. How to check a file's permission?
ls -l
35. What is the difference between "kill" and "kill -9" command?
Search online
36. What is subnet?
Search online
37. You are troubleshooting an issue with Redhat support and they have asked you to send the contents of /etc directory. How and which method you will use to transfer the contents?
tar (compress) the entire /etc directory and ftp
38. What is root home directory?
/root
39. What is rsyslogd daemon and its purpose?
Search online
40. Your company has terminated a server administrator. What is first thing as an administrator you should do to enhance the security?
Change root password
41. How to check the computer name or host name in Linux?
hostname
42. Which permission allows a user to run an executable with the permissions of the owner of that file?
First 3 bits should have x
43. What is the command to untar a tarred file?
untar
44. What is /proc directory used for?
Search online
45. What is the purpose of nsswitch.conf file
It tells the system where to go to resolve hostnames
46. List 3 basic commands to navigate the filesystem?
cd, pwd and ls
47. Which service/daemon should be running on the server that allows you to connect remotely?
sshd
48. What is the purpose of firewall?
Search online
49. List any 3 IT components?
Hardware, OS and Applications
50. Which directory has all the commands we use, e.g. ls, cd etc.?
/usr/bin or /bin

51. What is the difference between memory, virtual memory and cache?
Search online
52. Which of the following is correct?
a. **Hardware → Operating System → Users**
b. Operating System → Users → Hardware
c. Database → Hardware → Users
53. Which of the following is a communication command?
o grep
o **mail**
o touch
o cd
54. How to rename a file or directory?
mv
55. How to change a hostname in Linux?
Search online
56. How to check network interfaces in Linux?
ifconfig
57. Why is "tail -f logfilename" command used most often and what does it do?
It will output all incoming logs in real time
58. What type of hardware have you worked on?
You should get yourself familiar with Dell, HP and UCS hardware by going online and check the vendor websites
59. How to sort a file in reverse order?
cat filename | sort -r
60. What is the name of operating system that runs Unix?
Solaris, HP-UX etc.
61. List all byte sizes from smallest to largest?
Search online
62. How to check the total number of partition in Linux?
fdisk -l
63. How to access a linux system from a linux system?
ssh
64. Explain the procedure of bonding 2 NICs or interfaces together?
Search online
65. What is the exact command syntax to list the 5th column of a file and cut the first 3 letters?
cat filename | awk '{print \$5}' | cut -c1-3
66. What is /etc/hosts file used for?
To resolve hostnames with IP address

67. List any 3 options of 'df' command and what they are used for?
Search online
68. What is the command to change file/directory permissions?
chmod
69. What is the purpose of pipe (|)?
To combine multiple commands
70. What is /etc directory used for?
For configuration files
71. Which command is used to list files in a directory?
ls -l
72. There is a command which gives you information about other commands, please explain that command and what is it used for?
man
73. How to delete a file and a directory?
rm filename and rmdir dirname
74. What is the difference between "tail" and "tail -10"?
None
75. List 4 commands to display or read a file contents?
cat, more, less, vi
76. Which command is used to read the top 5 lines of a file?
head -5 filename
77. What are the different commands or methods to write to a file?
echo > filename and vi filename
78. What is swap space and how to check swap space?
Search online
79. What is inode and how to find an inode of a file?
Search online
80. Which file to edit for kernel tuning?
Search online
81. What is the latest version of Redhat?
Search online
82. Name the command to find specific word from a file?
grep word filename
83. You have scheduled a job using crontab but it does not run at the time you specified, what could be the reason and how would you troubleshoot?
Check your system time
Check your crontab entry
Check /var/log/messages
84. How to check system hardware information?

dmidecode

85. How to check network interface MAC address?
ifconfig
86. If I don't want others to read my file1, how to do that?
Remove r from the last 3 bits of file permission
87. What is the purpose of "uniq" and "sed" command?
Search online
88. Which command is used to list the contents of a directory in the most recent time and in reverse order, meaning the most updated file should be listed on the bottom?
ls -ltr
89. What is the difference between tar, gzip and gunzip?
Search online
90. What are the different ways to install an OS?
DVD, DVD iso and network boot
91. How to view difference between two files?
diff file1 and file2
92. You noticed that one of the Linux servers has no disk space left, how would you troubleshoot that issue?
If running LVM then add more disk and extend LVM
If not running LVM then add more disk, create a new partition and link the new partition to an existing filesystem
93. How to check Redhat version release?
uname -a or /etc/redhat-release
94. What is the difference between TCP and UDP?
Search online
95. What is a zombie process?
Search online
96. How do you search for a pattern/word in a file and then replace it in an entire file?
sed command
97. Explain the purpose of "touch" command?
To create an empty file
98. If a command hangs, how to stop it and get the prompt back?
Ctrl C
99. Which command is used to count words or lines?
wc
100. How to check the number of users logged in?
who
101. What is the command to view the calendar of 2011?

cal 2011

102. Which command is used to view disk space?
df -h
103. How to create a new group in Linux?
groupadd
104. What is the command to send a message to everyone who is logged into the system?
wall
105. Which command is used to check total number of disks?
fdisk -l
106. What is an mail server record in DNS?
MX
107. What does the following command line do?
`ps -ef | awk '{print $1}' | sort | uniq`
List the first column of all running processes, sort them and remove duplicates
108. You get a call that when a user goes to www.yourwebsite.com it fails and gets an error, how do you troubleshoot?
Check for user internet
Check to see if user computer has DNS for hostname lookup
Check to see if the server is up that is running that website
Check to see if the server's web service is running
Check for DNS availability which is resolving that website
109. List 4 different directories in /?
/etc, /bin, /tmp, /home
110. What is the output of the following command:
`$tail -10 filename | head -1`
It will show the first line from the last 10 lines of a file
111. What are the different fields in /etc/passwd file?
Search online
112. Which command is used to list the processes?
ps -ef
113. What is the difference between "hostname" and "uname" commands?
Hostname will give you system name and uname will give you OS information
114. How to check system load?
top and uptime command
115. How to schedule jobs?
crontab and at
116. What is the 3rd field when setting up crontab?
Day of the month

117. What is the command to create a new user?
useradd
118. What is the "init #" for system reboot?
6
119. How to restart a service?
systemctl restart servicename
120. How to shutdown a system?
shutdown or init 0
121. What is "ftp" command used for?
To transfer files from one computer to another
122. Explain cron job syntax? First is minute, second is..?
Min, house, day of the month, month, day of the week and command
123. How to delete a package in Linux?
rpm -e packagename
124. What is the file name where user password information is saved?
/etc/shadow
125. Which command you would use to find the location of chmod command?
which chmod
126. Which command is used to check if the other computer is online?
ping othercomputer
127. Please explain about LAN, MAN and WAN?
Search online
128. How to list hidden files in a directory?
ls -la
129. What is the difference between telnet and ssh?
ssh is secure where telnet is not
130. How to run a calculator on Linux and exit out of it?
bc and quit
131. List any 4 commands to monitor system?
top, df -h, iostat, dmesg
132. You are notified that your server is down, list the steps you will take to troubleshoot?
Check the system physically
Login through system console
Ping the system
Reboot or boot if possible
133. What is difference between static and DHCP IP?
Search online
134. How to write in vi editor mode?
i = insert, a = insert in next space, o = insert in new line

135. What is the difference between "crontab" and "at" jobs?
crontab is for repetitive jobs where at is for one time job
136. What is vCenter server in VMWare?
Search online
137. What is "dmidecode" command used for?
To get system information
138. What is the difference between SAN and NAS?
Search online
139. What is the location of system logs? E.g. messages
/var/log directory
140. How to setup an alias and what is it used for?
alias aliasname="command"
It is used to created short-cuts for long commands
141. What is the purpose of "netstat" command?
Search online
142. What are terminal control keys, list any 3?
Crtl C, D and Z
143. Which command(s) you would run if you need to find out how many processes are running on your system?
ps -ef | wc -l
144. What are the different types of shells?
sh, bash, ksh, csh etc.
145. How to delete a line when in vi editor mode?
dd
146. Which is the core of the operating system?
a) Shell
b) Kernel
c) Commands
d) Script
147. Which among the following interacts directly with system hardware?
a) Shell
b) Commands
c) Kernel
d) Applications
148. How to save and quit from vi editor?
Shift ZZ or :wq!
149. What is the difference between a process and daemon?
Search online
150. What is the process or daemon name for NTP?
ntpd

151. What are a few commands you would run if your system is running slow?
top, iostat, df -h, netstat etc.
152. How to install a package in Redhat Linux?
yum install packagename
153. What is the difference between "ifconfig" and "ipconfig" commands?
ifconfig for Linux and ipconfig for Windows
154. What is the first line written in a shell script?
Define shell
e.g. #!/bin/bash
155. Where is the network (Ethernet) file located, please provide exact directory location and file name?
/etc/sysconfig/network-scripts/ifcfg-nic
156. Why do we use "last" command?
To see who has logged in the system whether active or logged off
157. What is RHEL Linux stands for?
Search online
158. To view your command history, which command is used and how to run a specific command?
history and history #
159. What is NTP and briefly explain how does it work and where is the config files and related commands of NTP?
Search online
160. How to disable firewall in Linux?
Search online
161. How to configure mail server relay for sendmail service?
Edit /etc/mail/sendmail.mc file and add SMART_HOST entry
162. Where is samba log file located?
/var/log/samba
163. What is mkfs command used for?
To create a new filesystem
164. If you create a new group, which file does it get created in?
/etc/group
165. Which file has DNS server information (e.g. DNS resolution)?
/etc/resolv.conf
166. What are the commands you would run if you need to find out the version and build date of a package (e.g. http)?
rpm -qi http

167. On the file permissions? What are the first 3 bits for and who is it for?
Read, write and execute. They are used for the owner of the file
168. How to create a soft link?
ln -s
169. How to write a script to delete messages in a log file older than 30 days automatically?
Search online
170. How to quit out of "man" command?
q
171. Which command is used to partition disk in Linux?
fdisk
172. What is the difference between "shutdown" and "halt" command?
Search online
173. What is the exact syntax of mounting NFS share on a client and also how to un-mount?
Search online
174. What experience do you have with scripting, explain?
if-the, do-while, case, for loop scripts
175. How to get information on all the packages installed on the system?
rpm -qa
176. Explain VMWare?
Search online
177. You are tasked to examine a log file in order to find out why a particular application keep crashing. Log file is very lengthy, which command can you use to simplify the log search using a search string?
grep for error, warning, failure etc. in /var/log/messages file
178. What is /etc/fstab file and explain each column of this file?
Search online
179. What the latest version of Windows server?
Search online
180. What is the exact command to list only the first 2 lines of history output?
history | head -2
181. How to upgrade Linux from 7.3 to 7.4?
yum install update
182. How to tell which shell you are in or running?
\$0

183. You have tried to "cd" into a directory but you have been denied. You are not the owner of that directory, what permissions do you need and where?

- - - - - r - x

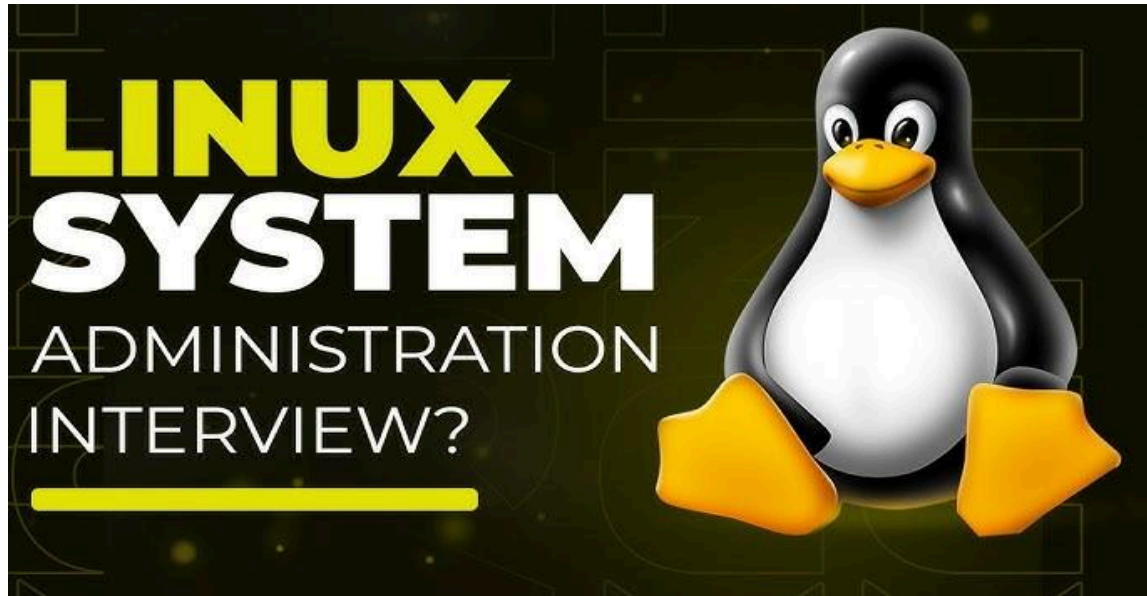
184. What is CNAME record in DNS?
Entry for hostname to hostname
185. What is the name of VMWare operating system?
ESXi
186. What is the client name used to connect to ESXi or vCenter server?
vSphere client
187. You get a call from a user saying that I cannot write to a file because it says, permission denied. The file is owned by that user, how do you troubleshoot?
Give write permission on the first 3 bits
188. What is the latest version of VMWare?
Search online
189. What is the name of firewall daemon in Linux?
firewalld
190. Which command syntax you can use to list only the 20th line of a file?
Search online
191. What is the difference between run level 3 and 5?
3 = Boot system with networking, 5 = boot system with networking and GUI
192. List a few commands that are used in troubleshooting network related issue?
netstat, tcpdump etc.
193. What is the difference between domain and nameserver?
Search online
194. You open up a file and it has 3000 lines and it scrolled up really fast, which command you will use to view it one page at a time?
more or less
195. How to start a new shell. E.g. start a new ksh shell?
Simply type ksh, or bash
196. How to kill a process?
kill processID
197. How to check scheduled jobs?
crontab -l

198. How to check system memory and CPU usage?
free and top
199. Which utility could you use to repair the corrupted file system?
fsck
200. What is the command to make a service start at boot?
systemctl enable servicename
201. How to combine 2 files into 1? E.g. you 3 lines in file "A" and 5 lines in file "B", which command syntax to use that will combine into one file of 3+5 = 8 lines
cat fileA >> fileB
202. What is echo command used for?
To output to a screen
203. What does the following command do?
echo This year the summer will be great > file1
It will create a new file "file1" with the content as "This year the summer will be great"
204. Which file to modify to allow users to run root commands
/etc/sudoers
205. You need to modify httpd.conf file but you cannot find it, Which command line tool you can use to find file?
find / -name "httpd.conf"
206. Your system crashed and being restarted, but a message appears, indicating that the operating system cannot be found. What is the most likely cause of the problem?
The /boot file is most likely corrupted

LINUX INTERVIEW QUESTIONS

Crack Your Next Interview with Confidence!

Author: [Umar Shahzad](#)



1. Filesystem Basics

1. What is the command to create an empty file?
 - `touch filename`
2. How do you create a new directory?
 - `mkdir directory_name`
3. How do you list all files in a directory?
 - `ls`
4. How do you list all files, including hidden ones?
 - `ls -a`
5. How do you display the current working directory?
 - `pwd`
6. How do you move or rename a file?
 - `mv source_file destination_file`
7. How do you copy a file?
 - `cp source_file destination_file`
8. What command is used to delete a file?
 - `rm filename`

9. How do you delete a directory and its contents?
 - `rm -r directory_name`
 10. How do you display the contents of a file?
 - `cat filename`
 11. How do you display the first 10 lines of a file?
 - `head filename`
 12. How do you display the last 10 lines of a file?
 - `tail filename`
 13. How do you check free disk space?
 - `df -h`
 14. How do you display disk usage of a directory?
 - `du -sh directory_name`
 15. How do you search for a file by name in a directory?
 - `find /path/to/directory -name filename`
 16. How do you compress a file using gzip?
 - `gzip filename`
 17. How do you decompress a gzip file?
 - `gunzip filename.gz`
 18. How do you archive multiple files using tar?
 - `tar -czvf archive.tar.gz files`
 19. How do you extract a tar.gz archive?
 - `tar -xzvf archive.tar.gz`
 20. How do you mount a filesystem?
 - `mount device_name mount_point`
 21. How do you unmount a filesystem?
 - `umount mount_point`
 22. How do you create a symbolic link?
 - `ln -s target_file link_name`
 23. How do you view file types in a directory?
 - `file filename`
 24. How do you remove an empty directory?
 - `rmdir directory_name`
 25. How do you check for file existence?
 - `test -f filename`
-

2. File Permissions and Ownership

26. How do you change file permissions?
 - `chmod permissions filename`

27. How do you change file ownership?
 - `chown user:group filename`
 28. How do you view file permissions in a directory?
 - `ls -l`
 29. How do you recursively change permissions for a directory?
 - `chmod -R permissions directory_name`
 30. What does the `chmod 777` command do?
 - Grants read, write, and execute permissions to everyone
 31. How do you view a file's access control list (ACL)?
 - `getfacl filename`
 32. How do you set a default ACL for a directory?
 - `setfacl -d -m u:user:permissions directory_name`
 33. How do you set permissions for a script to be executable?
 - `chmod +x script.sh`
 34. What is the significance of the `sticky bit` in Linux?
 - It restricts file deletion in shared directories like `/tmp`.
 35. How do you add the sticky bit to a directory?
 - `chmod +t directory_name`
-

3. Process Management

36. How do you view running processes?
 - `ps -aux`
37. How do you kill a process by PID?
 - `kill PID`
38. How do you find the PID of a running process by name?
 - `pgrep process_name`
39. How do you display processes in real-time?
 - `top` or `htop`
40. How do you stop a process gracefully?
 - `kill -15 PID`
41. How do you stop a process forcefully?
 - `kill -9 PID`
42. How do you start a background process?
 - `command &`
43. How do you bring a background job to the foreground?
 - `fg %job_id`
44. How do you monitor resource usage by a process?
 - `pidstat`

45. How do you view system logs for a process?
- `journalctl -u service_name`
46. How do you create a custom alias for a command?
- `alias alias_name='command'`
47. How do you remove an alias?
- `unalias alias_name`
48. How do you run a command as another user?
- `sudo -u username command`
49. How do you list all environment variables?
- `printenv` or `env`
50. How do you export a variable in Linux?
- `export VAR=value`
-

4. Networking

51. How do you find the IP address of your system?
- `ip addr` or `ifconfig`
52. How do you check active network connections?
- `ss -tuln` or `netstat -tuln`
53. How do you ping a host to check connectivity?
- `ping host`
54. How do you test port connectivity?
- `telnet hostname port`
55. How do you display routing information?
- `route`
56. How do you check open ports on your system?
- `lsof -i`
57. How do you download files from the internet?
- `wget URL`
58. How do you transfer files between systems using SCP?
- `scp source_file user@remote_host:/destination_path`
59. How do you use SSH to connect to a remote server?
- `ssh user@remote_host`
60. How do you set up a static IP address?
- Edit the network configuration file (e.g., `/etc/network/interfaces`)
-

5. User Management

- 61. How do you create a new user in Linux?
 - `adduser username`
 - 62. How do you delete a user?
 - `userdel username`
 - 63. How do you lock a user account?
 - `passwd -l username`
 - 64. How do you unlock a user account?
 - `passwd -u username`
 - 65. How do you switch to another user?
 - `su - username`
 - 66. How do you view currently logged-in users?
 - `who` or `w`
 - 67. How do you check user account information?
 - `id username`
 - 68. How do you list all users on the system?
 - `cat /etc/passwd`
 - 69. How do you create a group in Linux?
 - `groupadd group_name`
 - 70. How do you add a user to a group?
 - `usermod -aG group_name username`
 - 71. How do you change a user's default shell?
 - `chsh -s /bin/bash username`
 - 72. How do you change a user's home directory?
 - `usermod -d /new/home/dir username`
 - 73. How do you create a system user without a home directory?
 - `adduser --system --no-create-home username`
 - 74. How do you set or reset a user's password?
 - `passwd username`
 - 75. How do you check group memberships of a user?
 - `groups username`
-

6. Package Management

- 76. How do you install a package on Debian-based systems?
 - `apt install package_name`
- 77. How do you update all packages on a Debian-based system?
 - `apt update && apt upgrade`

78. How do you remove a package on Debian-based systems?
- `apt remove package_name`
79. How do you install a package on RHEL-based systems?
- `yum install package_name` or `dnf install package_name`
80. How do you remove a package on RHEL-based systems?
- `yum remove package_name` or `dnf remove package_name`
81. How do you search for a package in Debian repositories?
- `apt search package_name`
82. How do you list all installed packages?
- `dpkg -l` (Debian) or `rpm -qa` (RHEL)
83. How do you check which package a file belongs to?
- `dpkg -S filename` (Debian) or `rpm -qf filename` (RHEL)
84. How do you clean up cached packages on Debian-based systems?
- `apt clean`
85. How do you enable or disable a repository in RHEL-based systems?
- `yum-config-manager --enable repo_name` or `--disable repo_name`
-

7. System Monitoring and Performance

86. How do you check system uptime?
- `uptime`
87. How do you display memory usage?
- `free -h`
88. How do you monitor disk I/O in real-time?
- `iostat`
89. How do you check CPU usage?
- `top` or `htop`
90. How do you check running services?
- `systemctl list-units --type=service`
91. How do you enable a service to start on boot?
- `systemctl enable service_name`
92. How do you check system load averages?
- `uptime` or `cat /proc/loadavg`
93. How do you monitor network traffic in real-time?
- `iftop` or `nload`
94. How do you list hardware information?
- `lshw`
95. How do you check kernel version?
- `uname -r`

96. How do you check currently mounted filesystems?

- `mount` or `df -h`

97. How do you list active cron jobs?

- `crontab -l`

98. How do you check log files?

- `tail -f /var/log/syslog` or `journalctl`

99. How do you analyze disk usage by directories?

- `du -h --max-depth=1`

10. Advanced Networking

100. How do you configure a network interface?

- `ip addr add IP_ADDRESS dev interface`

101. How do you bring an interface up or down?

- `ip link set interface up` or `ip link set interface down`

102. How do you test DNS resolution?

- `nslookup domain_name`

103. How do you display ARP table entries?

- `arp -a`

104. How do you configure a firewall using iptables?

- `iptables -A INPUT -p tcp --dport port -j ACCEPT`

105. How do you save iptables rules?

- `iptables-save > /etc/iptables/rules.v4`

106. How do you check open ports on a system?

- `ss -tuln`

107. How do you trace the route to a host?

- `traceroute hostname`

108. How do you monitor packet flow?

- `tcpdump`

109. How do you edit the hosts file?

- `nano /etc/hosts`

11. Scripting and Automation

110. How do you create a simple shell script?

- `#!/bin/bash`

111. How do you make a script executable?

- `chmod +x script_name`

112. How do you run a script?

- `./script_name`

- 113. How do you use variables in a shell script?
 - `variable=value` and use with `$variable`
 - 114. How do you accept input in a script?
 - `read variable_name`
 - 115. How do you pass arguments to a script?
 - Use `$1`, `$2`, etc., for positional arguments
 - 116. How do you write an if-else condition in a script?
 - `bash if [ condition ]; then commands else commands fi`
 - 117. How do you write a for loop in a script?
 - `bash for i in {1..5}; do echo $i done`
 - 118. How do you schedule a script using cron?
 - `crontab -e` and add the schedule entry
 - 119. How do you debug a script?
 - `bash -x script_name`
-

12. Advanced Topics

- 120. What is SELinux?
 - Security-Enhanced Linux for access control
- 121. How do you check SELinux status?
 - `sestatus`
- 122. How do you change SELinux mode?
 - `setenforce 0` (permissive) or `setenforce 1` (enforcing)
- 123. How do you view running containers in Docker?
 - `docker ps`
- 124. How do you stop a running Docker container?
 - `docker stop container_id`
- 125. How do you create an SSH key pair?
 - `ssh-keygen`
- 126. How do you use rsync for file transfer?
 - `rsync -avz source destination`
- 127. How do you check the default gateway?
 - `ip route`
- 128. How do you monitor file changes in real-time?
 - `inotifywait`
- 129. How do you create a virtual environment in Python?
 - `python3 -m venv env_name`

13. Disk Management

- 130. How do you check swap usage?
 - `swapon --show` or `free -h`
- 131. How do you list disk partitions?
 - `lsblk`
- 132. How do you format a disk to ext4?
 - `mkfs.ext4 /dev/sdX`
- 133. How do you create a swap partition?
 - `mkswap /dev/sdX`
- 134. How do you check disk usage?
 - `df -h`
- 135. How do you check inodes usage?
 - `df -i`
- 136. How do you mount a disk?
 - `mount /dev/sdX /mount_point`
- 137. How do you unmount a disk?
 - `umount /mount_point`
- 138. How do you resize a partition?
 - Use `resize2fs` and `fdisk`
- 139. How do you check SMART status of a disk?
 - `smartctl -a /dev/sdX`

14. Process Management

- 140. How do you list all running processes?
 - `ps aux`
- 141. How do you kill a process by PID?
 - `kill PID`
- 142. How do you kill a process by name?
 - `pkill process_name`
- 143. How do you stop a process?
 - `kill -STOP PID`
- 144. How do you continue a stopped process?
 - `kill -CONT PID`
- 145. How do you view running processes in real-time?
 - `top` or `htop`

- 146. How do you check process memory usage?
 - `ps aux --sort=-%mem`
 - 147. How do you check process CPU usage?
 - `ps aux --sort=-%cpu`
 - 148. How do you check process status?
 - `ps -o stat= -p PID`
 - 149. How do you find a process by name?
 - `pgrep process_name`
 - 150. How do you check the number of processes running?
 - `ps aux | wc -l`
-

15. System Monitoring

- 151. How do you monitor CPU usage?
 - `top` or `htop`
 - 152. How do you check system load?
 - `uptime`
 - 153. How do you monitor disk space?
 - `df -h`
 - 154. How do you monitor memory usage?
 - `free -h`
 - 155. How do you monitor swap usage?
 - `swapon --show` or `free -h`
 - 156. How do you check network usage?
 - `ifstat`
 - 157. How do you monitor network interfaces?
 - `ip -s link`
 - 158. How do you monitor real-time log files?
 - `tail -f /var/log/syslog`
 - 159. How do you check system uptime?
 - `uptime`
 - 160. How do you check system resource utilization?
 - `vmstat`
-

16. Cloud and Automation Tools

- 161. How do you install AWS CLI?
 - `pip install awscli`

- 162. How do you configure AWS CLI?
 - `aws configure`
 - 163. How do you create an S3 bucket using CLI?
 - `aws s3 mb s3://bucket_name`
 - 164. How do you list running EC2 instances?
 - `aws ec2 describe-instances`
 - 165. How do you create an EC2 instance?
 - `aws ec2 run-instances --image-id ami_id --count 1 --instance-type t2.micro`
 - 166. How do you deploy an application using Ansible?
 - Write a playbook and run `ansible-playbook playbook.yml`
 - 167. How do you configure Ansible inventory?
 - Edit `/etc/ansible/hosts`
 - 168. How do you use Terraform to create infrastructure?
 - Write a `.tf` file and run `terraform apply`
 - 169. How do you start a Kubernetes cluster?
 - Use `kubeadm init`
 - 170. How do you install Docker on a system?
 - Use `apt-get install docker.io` or `yum install docker`
-

17. Advanced File Handling

- 171. How do you copy a directory recursively?
 - `cp -r source_directory destination_directory`
- 172. How do you move or rename a file?
 - `mv old_name new_name`
- 173. How do you remove a file?
 - `rm filename`
- 174. How do you remove a directory recursively?
 - `rm -r directory_name`
- 175. How do you find files by name?
 - `find / -name "filename"`
- 176. How do you search for text within files?
 - `grep "text" file_name`
- 177. How do you replace text in files?
 - `sed -i 's/old_text/new_text/g' file_name`
- 178. How do you concatenate two files?
 - `cat file1 file2 > new_file`
- 179. How do you compress a file?
 - `tar -czf file_name.tar.gz directory_name`

180. How do you decompress a file?
- `tar -xzf file_name.tar.gz`
-

18. Backup and Recovery

181. How do you create a backup of a directory?
- `tar -cvf backup_name.tar.gz directory_name`
182. How do you restore a backup?
- `tar -xvf backup_name.tar.gz`
183. How do you use `rsync` for incremental backups?
- `rsync -avz source destination`
184. How do you schedule backups with cron?
- `crontab -e` and add a backup command
185. How do you check backup logs?
- `cat /var/log/backup.log`
186. How do you create a full system backup with `dd`?
- `dd if=/dev/sda of=backup.img`
187. How do you restore a system from `dd` backup?
- `dd if=backup.img of=/dev/sda`
188. How do you create an incremental backup with `rsnapshot`?
- Configure `/etc/rsnapshot.conf` and run `rsnapshot daily`
189. How do you mount a backup disk?
- `mount /dev/sdX /mount_point`
190. How do you unmount a backup disk?
- `umount /mount_point`

19. Security and Permissions

191. How do you change file permissions in Linux?
- `chmod permissions file_name`
192. How do you change file ownership in Linux?
- `chown user:group file_name`
193. How do you create a new user in Linux?
- `useradd username`
194. How do you set a password for a user?
- `passwd username`
195. How do you delete a user in Linux?
- `userdel username`

196. How do you add a user to a group?
 - `usermod -aG group_name username`
197. How do you list groups in Linux?
 - `cat /etc/group`
198. How do you secure SSH access?
 - Disable root login by setting `PermitRootLogin no` in `/etc/ssh/sshd_config`
199. How do you configure SSH key-based authentication?
 - Copy the public key to `~/.ssh/authorized_keys` on the remote machine
200. How do you view system logs in Linux?
 - `journalctl` or `cat /var/log/syslog`

Tips for Cracking the Interview

- **Be practical:** Demonstrate how to solve real-world issues.
- **Explain clearly:** Don't just list commands; explain the reasoning behind them.
- **Share scenarios:** Mention situations where you applied these skills, like troubleshooting, securing systems, or optimizing performance.

Follow my [LinkedIn](#) for more insightful notes! Share your learning experiences and let me know which topic you'd like me to cover next. Your feedback matters!

200 LINUX SCENARIO BASED Q&A

By DevOps Shack

200 Linux Scenario Based Questions & Answers

DevOps Shack

Q1. User can see (ls) a file but cannot edit it.

Scenario:

User lists a file but gets Permission denied when opening it with an editor like vim or nano.

Root Cause:

They have **read** permission (r) but not **write** permission (w) on the file. Editing requires write access.

Solution:

1. Check ownership and permissions:

```
ls -l /path/to/file
```

2. Fix ownership (if needed):

```
sudo chown username:groupname /path/to/file
```

3. Or add write permission:

```
sudo chmod u+w /path/to/file
```

Tip:

Always use **group permissions** instead of giving **everyone** (others) write access for corporate systems.

Q2. Accidentally deleted a config file like /etc/ssh/sshd_config.

Scenario:

Important system config file is accidentally deleted, system services depending on it might break after restart.

Root Cause:

When you delete a file, only its metadata link is removed. If the file is open by a running service, recovery is possible.

Solution:

1. Find if any process is still using the deleted file:

```
lsof | grep deleted
```



2. Recover the file:

```
cp /proc/PID/fd/FD /path/to/newfile
```

3. If no running process:

- Need to restore from backup
- If no backup, use file system recovery tools (like extundelete).

Tip:

Always configure **automatic daily backups** of /etc to avoid system outage due to config loss.

Q3. "Read-only file system" error when creating or modifying files.

Scenario:

Commands like touch, mkdir, or editors fail with Read-only file system error.

Root Cause:

The file system was mounted read-only by the OS due to detecting disk or file system corruption to avoid further damage.

Solution:

1. Confirm filesystem is read-only:

```
mount | grep ro
```

2. Check disk errors:

```
dmesg | tail
```

3. Reboot into rescue mode and run fsck:

```
sudo fsck /dev/sdX
```

4. Allow fsck to fix errors and reboot.

Tip:

Monitor disk health regularly using tools like smartctl to detect hardware failure early.

Q4. "Permission denied" when creating file under /tmp/.

Scenario:

Even in /tmp/, user cannot create or write files and gets permission denied.



Root Cause:

Permissions on /tmp were accidentally changed. /tmp should have 1777 permissions (world-writable with sticky bit).

Solution:

1. Check permissions:

```
ls -ld /tmp
```

2. Restore correct permissions:

```
sudo chmod 1777 /tmp
```

Tip:

/tmp must always have 1777 permissions; without it, apps and users can face weird errors.

Q5. "No such file or directory" when executing a binary.**Scenario:**

A binary file exists, but running it gives No such file or directory.

Root Cause:

Either missing dependent shared libraries, or wrong binary compiled for another architecture.

Solution:

1. Check file architecture:

```
file ./binaryfile
```

2. Check dynamic dependencies:

```
ldd ./binaryfile
```

3. Install missing libraries if needed.

Tip:

Always compile binaries on the same OS and architecture when preparing for production.

Q6. Deleted /var/log/messages log file by mistake.**Scenario:**

Important system logs are removed; logging services may fail.

Root Cause:

Services like rsyslogd or journald rely on files under /var/log.



Solution:

1. Restart syslog service:

```
sudo systemctl restart rsyslog
```

2. Log files will be recreated automatically.

Tip:

Avoid manually deleting log files; use logrotate to manage logs safely.

Q7. "Text file busy" when replacing a binary.

Scenario:

Trying to overwrite an in-use binary results in Text file busy error.

Root Cause:

The binary is still being executed by a running process.

Solution:

1. Identify process:

fuser binaryfile

2. Kill the process:

kill -9 PID

3. Replace binary safely.

Tip:

Use safer deployment methods like versioned binaries + symlinks to avoid runtime issues.

Q8. Cannot run a newly created shell script — "Permission denied".**Scenario:**

Created a script but getting Permission denied even though it exists.

Root Cause:

Script is missing executable (x) permission.

Solution:

1. Make it executable:

```
chmod +x script.sh
```



2. Then run:

```
./script.sh
```

Tip:

Always verify both the shebang (#!/bin/bash) and permissions when troubleshooting scripts.

Q9. User can't access their own home directory.

Scenario:

User gets "Permission denied" when trying to cd into their home.

Root Cause:

Wrong ownership or permission settings on the home directory.

Solution:

1. Fix ownership:

```
sudo chown username:username /home/username
```

2. Fix permissions:

```
chmod 700 /home/username
```

Tip:

Corporate servers often set /home/username with 700 or 750 permissions for security.

Q10. Mistakenly set wrong permission on /etc/passwd, users can't login.

Scenario:

Users are unable to login after wrong permissions were set on /etc/passwd.

Root Cause:

Critical system files like /etc/passwd must have proper permissions.

Solution:

1. Boot into recovery mode.
2. Correct permissions:

```
chmod 644 /etc/passwd
```

Tip:

Use ls -l carefully when changing permissions on system files; mistakes can break authentication.



Q11. Created a new directory but can't cd into it — "Permission denied".

Scenario:

After creating a directory, the user cannot access it via cd, even though they own it.

Root Cause:

Directory is missing **execute (x) permission**, which is required to enter (cd) a directory.

Solution:

1. Check permissions:

```
ls -ld /path/to/dir
```

2. Add execute permission:

```
chmod +x /path/to/dir
```

Tip:

Directories need **execute** permission to enter, and **read** permission to list their contents.

Q12. A symbolic link shows "No such file" after reboot.

Scenario:

A previously working symlink is now broken, showing "No such file or directory".

Root Cause:

The symlink points to a file that was either on a temporary filesystem (e.g., /tmp) or deleted.

Solution:

1. Check where symlink points:

```
ls -l /path/to/symlink
```

2. Correct the target if needed:

```
ln -sf /correct/target/path /path/to/symlink
```

Tip:

Prefer absolute paths when creating symlinks for important files.



Q13. cp -r fails midway when copying a large directory.

Scenario:

Copying a huge directory with cp -r stops with errors like "No space left on device" or permission errors.

Root Cause:

- Disk may be full.
- Permission issues on some files.
- Filesystem limits like inode exhaustion.

Solution:

1. Check disk space:

```
df -h
```

2. Check inode usage:

```
df -i
```

3. If permission errors:

```
sudo cp -rp source/ destination/
```

```
(-p preserves ownerships.)
```

Tip:

When copying critical data, use rsync -a instead of cp -r for better control and resume support.

Q14. Tar backup created but getting error while extracting.

Scenario:

tar -xvf backup.tar throws errors like "Unexpected EOF" or "corrupt archive".

Root Cause:

- Tarball creation was interrupted.
- File got partially copied.
- Wrong tar command or compression mismatch.

Solution:



1. Check if tar is readable:

```
tar -tvf backup.tar
```

2. Ignore bad blocks if minor damage:

```
tar --ignore-failed-read -xvf backup.tar
```

Tip:

Always use gzip or bzip2 (tar -czvf) for compression + integrity checks.

Q15. Filesystem showing used space but no visible files.

Scenario:

df -h shows disk full but ls -lh shows no big files.

Root Cause:

Deleted files are still held by running processes and occupying space.

Solution:

1. Find deleted files:

```
lsdf | grep deleted
```

2. Kill the processes using those files:

```
kill PID
```

3. Disk space will be freed.

Tip:

On production, avoid killing critical daemons; restart them gracefully if possible.

Q16. "Argument list too long" when deleting thousands of files.

Scenario:

Running rm *.log on a folder with 100,000+ files throws "Argument list too long" error.

Root Cause:

Shell has a maximum command-line argument length (around 128KB).

Solution:

Use find instead of wildcards:

```
find . -name "*.log" -delete
```



Tip:

For mass file operations, find is always more reliable and scalable.

Q17. Accidentally moved files to the wrong location.**Scenario:**

`mv * /wrong/path/` by mistake; files need to be moved back.

Root Cause:

Wrong destination path supplied in mv command.

Solution:

Move back:

```
find /wrong/path/ -type f -exec mv {} /correct/path/ \;
```

Or manually move using mv.

Tip:

Use `mv -i` (interactive) to avoid accidental overwrites.

Q18. User created directory but team members can't write into it.**Scenario:**

One user creates a project folder, but others cannot create files inside it.

Root Cause:

Group write permissions missing.

Solution:

1. Set correct group ownership:

```
sudo chown :projectgroup /path/to/projectdir
```

2. Add group write permission:

```
chmod g+w /path/to/projectdir
```

Tip:

Use **setgid bit** (`chmod g+s`) on shared directories to automatically inherit group ownership for new files.



Q19. Finding out which file is filling up the disk.

Scenario:

Disk is almost full but unsure which files/folders are consuming space.

Root Cause:

Hidden huge logs, database dumps, or backup folders.

Solution:

Find largest directories/files:

```
du -ahx / | sort -rh | head -20
```

or

```
ncdu /
```

(ncdu gives an interactive disk usage view.)

Tip:

Schedule weekly disk audits in production servers to prevent sudden disk full incidents.

Q20. Mounted NFS share shows "stale file handle" error.

Scenario:

Accessing NFS share gives:

nginx

Stale file handle

Root Cause:

The file/folder on the server was deleted or modified while client still holds a reference.

Solution:

Force unmount:

```
sudo umount -f /mnt/nfs
```

Then remount:

```
sudo mount /mnt/nfs
```

Tip:

Avoid hardcoding paths on NFS-mounted systems; handle stale handles with retry logic if scripting.



Q21. System becomes very slow — commands take long to execute.

Scenario:

Basic commands like ls, cd, or top take 5–10 seconds to respond.

Root Cause:

- System may be out of **free RAM** and using **swap heavily**.
- High CPU or disk I/O wait also possible.

Solution:

1. Check memory usage:

```
free -h
```

2. Check swap usage:

```
swapon --show
```

3. Check CPU and load:

```
top
```

```
iostat -xz 1
```

Tip:

Set up swapoff -a temporarily to test if swap is the cause; add more RAM or reduce memory-hungry apps.

Q22. Disk is 100% full, can't create or write files.

Scenario:

App logs stop, can't save files, or services crash. df -h shows 100% usage.

Root Cause:

A large log, backup file, or temporary file has filled the disk.

Solution:

1. Find the largest files:

```
du -sh /* | sort -h
```

```
or use ncdu /
```

2. Remove or compress large files:

```
gzip huge-log.log
```

Tip:

Configure log rotation (logrotate) to prevent runaway log files.

Q23. Disk shows 0% free in df -h, but du -sh shows little usage.

Scenario:

Disk shows full, but there are no large files in sight.

Root Cause:

Files were **deleted** but are still held open by a running process.

Solution:

1. Find such processes:

```
lsof | grep deleted
```

2. Restart the process or:

```
kill -9 <PID>
```

Tip:

Always restart logging services after clearing large logs.

Q24. "Cannot allocate memory" error when starting services.

Scenario:

A service fails to start due to memory allocation errors.

Root Cause:

Out of RAM or **ulimit** limits exceeded.

Solution:

1. Check available RAM:

```
free -m
```

2. Check and increase memory limits:

```
ulimit -a
```

```
ulimit -v unlimited
```



Tip:

Set permanent limits in `/etc/security/limits.conf` for critical services.

Q25. Load average is high but CPU usage is low.**Scenario:**

`top` shows load average > 5, but CPU idle is still high.

Root Cause:

Load average includes processes waiting for **disk** or **I/O**, not just CPU.

Solution:

1. Check I/O wait:

```
iostat -xz 1
```

2. Check blocked processes:

```
vmstat 1
```

Tip:

Use SSDs and proper disk schedulers for high-I/O workloads.

Q26. Swap usage is always high even with free RAM.**Scenario:**

`free -h` shows high swap usage even when RAM is available.

Root Cause:

Kernel may have aggressively swapped out old memory pages.

Solution:

1. Reduce swappiness:

```
sysctl vm.swappiness=10
```

2. Persist it in `/etc/sysctl.conf`:

```
vm.swappiness=10
```

Tip:

Swappiness of 10 is ideal for most server workloads.

Q27. Disk I/O spikes suddenly — application slows down.

Scenario:

Random performance lag. iostat shows high %util and await.

Root Cause:

Too many read/write operations from one or more processes.

Solution:

1. Use iotop to identify culprit:

```
sudo iotop
```

2. Pause or throttle process:

```
ionice -c2 -n7 -p <PID>
```

Tip:

Use dedicated disk partitions for logs or data-heavy apps.

Q28. "No space left on device" but df -h shows free space.

Scenario:

df -h shows free space but still can't write to disk.

Root Cause:

Ran out of **inodes** (metadata structure used for files).

Solution:

1. Check inode usage:

2. Clean up folders with many small files (like /var/spool, /tmp):

```
find /var/spool -type f -delete
```

Tip:

Format file systems with more inodes if you expect millions of tiny files (e.g., mail servers).

Q29. top shows one process using 99% CPU.**Scenario:**

System is slow, one process is hogging the CPU.

Root Cause:

An infinite loop, runaway script, or bad query.



Solution:

1. Identify process:

top

```
ps aux --sort=-%cpu | head
```

2. Pause or kill process:

```
kill -STOP PID
```

```
kill -9 PID
```

Tip:

Use nice or cpublimit for non-critical CPU-bound processes.

Q30. df command hangs or freezes.**Scenario:**

Running df -h just hangs and never returns.

Root Cause:

A mounted NFS or remote filesystem is unavailable.

Solution:

1. List all mounts:

```
mount | grep nfs
```

2. Force unmount:

```
sudo umount -f /mnt/path
```

Tip:

Mount NFS with soft,timeo options to prevent such hangs.

Q31. Memory usage slowly increases over time.**Scenario:**

RAM usage creeps up daily until system swap is used.

Root Cause:

Memory leak in a service or cron job.

Solution:

1. Monitor usage:



top

```
ps aux --sort=-%mem
```

2. Restart the leaking service periodically.

Tip:

Use monitoring tools like prometheus + grafana to track memory trends.

Q32. Kernel OOM (Out of Memory) kills processes.

Scenario:

Processes suddenly die and dmesg shows OOM killer.

Root Cause:

System ran out of memory and the kernel killed the biggest consumer.

Solution:

1. View logs:

```
dmesg | grep -i kill
```

2. Tune OOM behavior:

```
echo -17 > /proc/<PID>/oom_score_adj
```

Tip:

Use OOM protection for critical apps like databases using systemd or memory cgroups.

Q33. Disk write speed is very low.

Scenario:

Copying large files is taking unusually long time.

Root Cause:

- Disk I/O contention.
- Wrong disk scheduler.
- Fragmentation.

Solution:

1. Benchmark:



```
dd if=/dev/zero of=testfile bs=1G count=1 oflag=dsync
```

2. Check scheduler:

```
cat /sys/block/sdX/queue/scheduler
```

Tip:

Use deadline or noop scheduler for SSDs.

Q34. Filesystem corruption detected on reboot.**Scenario:**

On boot, system enters emergency mode with fsck errors.

Root Cause:

Improper shutdown or hardware issues caused fs corruption.

Solution:

1. Enter rescue mode.
2. Run fsck manually:

```
fsck /dev/sdaX
```

Tip:

Always unmount disks cleanly or use journaling filesystems like ext4.

Q35. CPU always runs at 100% on all cores.**Scenario:**

Even idle system shows high CPU usage on all cores.

Root Cause:

Background process like cron, rsync, or malware.

Solution:

1. Inspect with:

```
top
```

```
htop
```

```
ps aux --sort=-%cpu
```

2. Kill the culprit or analyze further.

Tip:

Use auditd or psacct to track rogue activity.

Q36. Multiple zombie processes accumulate.**Scenario:**

ps aux shows <defunct> processes.

Root Cause:

Parent process failed to wait for child termination.

Solution:

1. Identify parent:

```
ps -ef | grep defunct
```

2. Restart or kill parent process.

Tip:

Zombie processes don't consume resources but too many indicate bad code logic.

Q37. RAM usage shown as full, but system is fast.**Scenario:**

free -h shows most of the memory used, but no lag.

Root Cause:

Linux caches file system I/O in memory.

Solution:

Check actual usage:

```
free -h
```

Look at "available", not "used".

Tip:

Don't panic if RAM is full — Linux intelligently caches it.

Q38. RAM is free, but app complains "cannot allocate memory".**Scenario:**

App errors out despite RAM being available.



Root Cause:

Per-process memory limit (ulimit) or overcommit policy.

Solution:

1. Increase limits:

```
ulimit -v unlimited
```

2. Tweak overcommit:

```
echo 1 > /proc/sys/vm/overcommit_memory
```

Tip:

Avoid setting overcommit=2 for memory-heavy applications.

Q39. Application performance drops when copying files.**Scenario:**

App slows down whenever a large file is copied.

Root Cause:

Disk I/O contention; both app and copy operation use same disk.

Solution:

1. Use ionice for background copy:

```
ionice -c3 cp file /target/
```

Tip:

Use dedicated storage for app vs backups/logs.

Q40. CPU usage randomly spikes — can't trace.**Scenario:**

System becomes slow randomly, CPU usage goes to 100% for a few minutes.

Root Cause:

- cron jobs
- Background scanning (like updatedb, mlocate, antivirus)

Solution:

Check cron:

```
cat /etc/crontab
```

Check logs:

```
journalctl --since "10 min ago"
```

Tip:

Disable or schedule heavy cron jobs during off-peak hours.

Q41. Ping to external websites fails, but internal network is reachable.

Scenario:

You can ping other internal servers, but external websites like google.com fail.

Root Cause:

Likely **missing or misconfigured DNS settings**.

Solution:

1. Check DNS in /etc/resolv.conf:

```
cat /etc/resolv.conf
```

2. Add valid DNS (e.g., Google DNS):

```
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

Tip:

For permanent DNS, configure via /etc/systemd/resolved.conf or /etc/netplan/ (Ubuntu) or /etc/sysconfig/network-scripts/ (RHEL).

Q42. SSH connection takes 30 seconds to respond.

Scenario:

SSH login delays for 30 seconds, then succeeds.

Root Cause:

Reverse DNS lookup delay or GSSAPI authentication hang.

Solution:

1. Edit SSH config:

```
sudo vi /etc/ssh/sshd_config
```

Add or update:

nginx



`UseDNS no`

`GSSAPIAuthentication no`

2. Restart SSH:

`sudo systemctl restart sshd`

Tip:

Disabling UseDNS is a common corporate tweak to speed up SSH logins.

Q43. SSH fails with "Permission denied (publickey)".

Scenario:

Trying to login via SSH with key, but always gets Permission denied (publickey).

Root Cause:

- Wrong key
- Missing authorized_keys file
- Incorrect permissions

Solution:

1. Ensure correct key is in:

`~/.ssh/authorized_keys`

2. Set proper permissions:

`chmod 700 ~/.ssh`

`chmod 600 ~/.ssh/authorized_keys`

Tip:

Always verify server-side logs:

Q44. SSH server is running but port 22 is not reachable from outside.

Scenario:

systemctl status sshd shows active, but remote users can't connect.

Root Cause:

Firewall or security group (cloud) blocking port 22.



Solution:

1. Check port open on local server:

```
sudo ss -tln | grep :22
```

2. Check firewall:

```
sudo ufw status # or sudo iptables -L
```

3. Open port 22:

```
sudo ufw allow 22
```

```
sudo systemctl restart sshd
```

Tip:

In AWS/GCP, check security group settings in addition to the OS firewall.

Q45. "Connection refused" when trying to SSH into a server.**Scenario:**

SSH fails with connection refused immediately.

Root Cause:

SSHD not running, or nothing is listening on port 22.

Solution:

1. Start SSH server:

```
sudo systemctl start sshd
```

2. Ensure it's enabled at boot:

```
sudo systemctl enable sshd
```

Tip:

Always check `ss -tln` to verify if any service is bound to port 22.

Q46. Server has no internet access, but IP and DNS look fine.**Scenario:**

ping 8.8.8.8 and curl fail, but IP and DNS are configured.

Root Cause:

Missing or incorrect **default gateway** route.

Solution:

1. View routing table:

```
ip route show
```

2. Add default gateway:

```
sudo ip route add default via 192.168.1.1
```

Tip:

Always validate both DNS and default route to confirm full internet access.

Q47. Network goes down after reboot.

Scenario:

After rebooting, server comes up without IP address or fails to connect.

Root Cause:

Static IP config missing or not applied properly in network configs.

Solution (Ubuntu - netplan):

1. Edit config:

```
sudo vi /etc/netplan/01-netcfg.yaml
```

2. Apply changes:

```
sudo netplan apply
```

Tip:

Always validate netplan configs with netplan try to prevent lockout.

Q48. Firewall blocks traffic after reboot even though ports were allowed.

Scenario:

Rules worked before reboot, but after restart, services are unreachable.

Root Cause:

Firewall rules were not **persisted**.

Solution:

1. On Ubuntu with ufw:

```
sudo ufw enable
```

```
sudo ufw allow 80
```

2. On iptables-based systems:



```
sudo iptables-save > /etc/iptables/rules.v4
```

Tip:

Use iptables-persistent or firewalld's --permanent flag to save rules.

Q49. Can't SSH as root user even though password is correct.**Scenario:**

Login as root gives access denied, but other users can login.

Root Cause:

Root SSH login disabled by default in many distributions.

Solution:

1. Edit /etc/ssh/sshd_config:

```
PermitRootLogin yes
```

2. Restart SSH:

```
systemctl restart sshd
```

Tip:

Allow root login only for emergency. Use regular users + sudo.

Q50. Port is open locally but not reachable from another server.**Scenario:**

App listens on port 3000, reachable via localhost, but not externally.

Root Cause:

Application bound to 127.0.0.1, not 0.0.0.0.

Solution:

1. Check with:

```
sudo ss -tln
```

2. Change app binding:
 - From: 127.0.0.1:3000
 - To: 0.0.0.0:3000
3. Restart the app.



Tip:

For Node.js, Flask, etc., always bind to 0.0.0.0 for external access.

Q51. Port 443 shows open, but HTTPS doesn't work.**Scenario:**

Website doesn't load over https://, even though port 443 is open.

Root Cause:

SSL certificate missing, expired, or web server misconfigured.

Solution:

1. Check certificate:

```
openssl s_client -connect yourdomain.com:443
```

2. Check Nginx/Apache SSL config.
3. Renew cert:

```
sudo certbot renew
```

Tip:

Use a cron job or systemd timer to automate SSL renewal with Certbot.

Q52. SSH sessions randomly freeze or timeout.**Scenario:**

SSH session hangs during idle or long data transfer.

Root Cause:

Firewall NAT timeout or session keepalive not configured.

Solution:

1. Add to /etc/ssh/sshd_config:

```
ClientAliveInterval 60
```

```
ClientAliveCountMax 3
```

2. Restart sshd.

Tip:

Also set ServerAliveInterval in client ~/.ssh/config.



Q53. Server cannot resolve internal hostname.**Scenario:**

ping myapp.internal fails, even though IP is correct.

Root Cause:

Missing or incorrect /etc/hosts entry or broken DNS resolver.

Solution:

1. Add entry manually:

```
echo "10.0.1.5 myapp.internal" | sudo tee -a /etc/hosts
```

2. Or fix internal DNS resolution.

Tip:

Avoid hardcoding hosts for dynamic services. Use proper internal DNS.

Q54. Network interface name changed after reboot.**Scenario:**

After upgrade or reboot, eth0 becomes ens33 or similar.

Root Cause:

Predictable Network Interface Names (systemd/udev feature).

Solution:

Use ip a to confirm interface, then fix config.

To revert:

```
sudo ln -s /dev/null /etc/udev/rules.d/80-net-setup-link.rules
```

Tip:

Stick with default naming unless you have automation depending on eth0.

Q55. Server reachable only via IP, not domain name.**Scenario:**

You can ping IP but not the domain.

Root Cause:

DNS resolution issue.

Solution:

1. Check:

`dig yourdomain.com`

`nslookup yourdomain.com`

2. Fix `/etc/resolv.conf`, or update DNS.

Tip:

Always check for DNS TTL expiration if domain recently changed.

Q56. Connection refused from one server, but works from another.

Scenario:

SSH or HTTP works from server A, fails from server B.

Root Cause:

Firewall is **blocking specific IP** or subnet.

Solution:

1. Check firewall:

```
sudo iptables -L
```

```
sudo ufw status
```

2. Allow that specific IP:

```
sudo ufw allow from <IP>
```

Tip:

Use centralized firewall management in corporate (e.g., firewalld zones or cloud firewalls).

Q57. iptables rules lost after reboot.

Scenario:

Custom firewall rules vanish after reboot.

Root Cause:

iptables rules not saved to persistent storage.

Solution:

1. Install persistence:

```
sudo apt install iptables-persistent
```

```
sudo netfilter-persistent save
```



Tip:

Always use iptables-save and iptables-restore in production.

Q58. Public IP changed after reboot (cloud server).**Scenario:**

SSH fails after reboot; public IP is different.

Root Cause:

Using **ephemeral public IP** not an elastic/static one.

Solution:

- In AWS/GCP: allocate **static IP** and attach to instance.

Tip:

Always assign static public IP for production servers.

Q59. SSH key authentication fails but password works.**Scenario:**

Password SSH works, but key-based login fails.

Root Cause:

sshd_config disables PubkeyAuthentication.

Solution:

1. Edit /etc/ssh/sshd_config:

PubkeyAuthentication yes

2. Restart SSH:

systemctl restart sshd

Tip:

Verify correct key is in ~/.ssh/authorized_keys with chmod 600.

Q60. After changing hostname, SSH gives “WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!”**Scenario:**

SSH gives security warning after host reinstallation or hostname change.



Root Cause:

SSH key mismatch; host key changed, but client stored old one.

Solution:

1. On client:

```
ssh-keygen -R server_ip
```

2. Try again:

```
ssh user@server_ip
```

Tip:

This is expected after reinstallation. Don't ignore blindly in real SSH MITM risks.

Q61. Application hangs, doesn't respond to input, but still runs in ps.**Scenario:**

App seems frozen. It's listed in ps, but no output or interaction possible.

Root Cause:

App is **stuck in an uninterruptible state** (D), usually due to I/O wait.

Solution:

1. Check process state:

```
ps -eo pid,stat,cmd | grep '^ *PID'
```

State D = Disk sleep (uninterruptible)

2. Check disk usage:

```
iotop
```

```
dmesg | tail
```

Tip:

Restarting the application is often the only fix if it's blocked on I/O.

Q62. High number of defunct (zombie) processes.**Scenario:**

ps aux shows many processes with <defunct> status.

Root Cause:

Parent process isn't collecting the exit status of child processes.



Solution:

1. Find parent:

```
ps -ef | grep defunct
```

2. Restart the parent:

```
kill -HUP <PARENT_PID>
```

Tip:

Well-coded apps should handle child exits with `wait()`; monitor bad processes.

Q63. A process refuses to die using kill.**Scenario:**

App doesn't stop after kill PID. Even kill -9 doesn't help.

Root Cause:

Kernel marks process as **uninterruptible sleep** — stuck on I/O or kernel-level operation.

Solution:

1. Confirm status:

```
ps -o pid,stat,cmd -p <PID>
```

2. Only fix:

Reboot the system

Tip:

This is rare but happens with disk/NFS issues — fix underlying I/O after reboot.

Q64. You started a long-running command and accidentally closed the terminal.**Scenario:**

You closed the terminal, and the command stopped.

Root Cause:

Foreground processes are terminated when the controlling terminal exits.

Solution:

Next time:

- Use `nohup`:

```
nohup long-running-command &
```



- Or use screen / tmux for detachable terminals.

Tip:

For production, always use terminal multiplexers for stability.

Q65. CPU is stuck at 100%, system lagging heavily.**Scenario:**

System nearly unresponsive, fans running loud, all CPU cores maxed.

Root Cause:

One or more processes are consuming full CPU in an infinite loop or runaway thread.

Solution:

1. Identify with:

```
top -o %CPU
```

2. Kill the culprit:

```
kill -9 <PID>
```

Tip:

For permanent control, use cpublimit or nice for background tasks.

Q66. Service crashes with segmentation fault (core dumped).**Scenario:**

A command crashes and prints: Segmentation fault (core dumped)

Root Cause:

Memory violation due to bad code or incompatible library.

Solution:

1. Enable core dumps:

```
ulimit -c unlimited
```

2. Analyze core:

```
gdb /path/to/bin core
```

Tip:

Recompile with debug symbols for deeper insight. Common in custom or C/C++ apps.



Q67. A script keeps running indefinitely and doesn't stop.

Scenario:

Shell script keeps running without output or finishing.

Root Cause:

Possible infinite loop or read command waiting for user input.

Solution:

1. Use strace to observe:

```
strace -p <PID>
```

2. Modify script to add debug logs (set -x, echo).

Tip:

Always put timeout logic in critical automation scripts.

Q68. Cron job runs but nothing happens.**Scenario:**

Job is listed in crontab, marked as run in logs, but no output or action.

Root Cause:

- Missing environment variables (e.g., PATH)
- Script paths incorrect when run from cron

Solution:

1. Add full paths to all commands inside the script.
2. Redirect output to a log file:

```
* * * * * /path/to/script.sh > /tmp/script.log 2>&1
```

Tip:

Use env at the top of your script to debug cron job environments.

Q69. Systemd service fails to start even though the binary works manually.**Scenario:**

Manual execution is fine, but systemctl start myapp fails.



Root Cause:

Wrong user, missing Environment, or incorrect WorkingDirectory in unit file.

Solution:

1. Check service logs:

```
journalctl -u myapp.service
```

2. Fix unit file:

```
ExecStart=/full/path/to/binary
```

```
WorkingDirectory=/path
```

```
User=appuser
```

Tip:

Run systemd-analyze verify file.service to lint systemd files.

Q70. You can't stop a service via systemd — says "failed".**Scenario:**

systemctl stop myapp fails with timeout or “failed to stop unit”.

Root Cause:

The process does not respond to SIGTERM or leaves behind child processes.

Solution:

1. Force kill:

```
systemctl kill -s SIGKILL myapp
```

2. Then restart:

```
systemctl restart myapp
```

Tip:

In unit file, add:

```
KillMode=control-group
```

To ensure child processes are killed too.



Q71. ps shows a process but you can't find the command or script behind it.

Scenario:

You see a process by PID but don't know what it is or what script it runs.

Root Cause:

Might be a forked shell, background script, or binary without obvious path.

Solution:

1. Use:

```
ps -p PID -o cmd
```

```
ls -l /proc/PID/exe
```

```
readlink /proc/PID/exe
```

Tip:

Combine strace with lsof for live debugging.

Q72. Process runs fine but doesn't log any output.

Scenario:

App runs, works, but you see no logs anywhere.

Root Cause:

- Stdout/stderr redirected to /dev/null
- Logging not configured

Solution:

1. Run in foreground:

```
./app
```

2. Check logging path in config.
3. Check systemd logs:

```
journalctl -u app
```

Tip:

Always configure both **file-based** and **journald-based** logging in production.



Q73. You need to run multiple processes in background from a script.**Scenario:**

Script launches 3–4 background jobs, but only the first runs.

Root Cause:

Process chaining or incorrect use of &, wait.

Solution:

Use:

```
command1 &
```

```
command2 &
```

Tip:

Use trap to clean up if script is interrupted.

Q74. A scheduled process keeps getting killed randomly.**Scenario:**

A background task or cronjob dies midway.

Root Cause:

OOM killer, memory limit, or wrong ulimit settings.

Solution:

1. Check:

```
dmesg | grep -i kill
```

2. Set memory limits properly:

```
ulimit -v unlimited
```

Tip:

Use cgroups for more granular resource control in critical systems.

Q75. System boots, but one service fails to start automatically.**Scenario:**

Manual start works, but service doesn't auto-start at boot.



Root Cause:

Service is not **enabled**, or has incorrect After= dependency.

Solution:

1. Enable it:

```
systemctl enable myservice
```

2. Check dependency timing in unit:

```
After=network.target
```

Tip:

Use systemd-analyze blame to see what's blocking boot.

Q76. Process binds to port, but logs say "address already in use".**Scenario:**

App fails to start, error: EADDRINUSE.

Root Cause:

Another process is using the port.

Solution:

1. Find the PID:

```
sudo lsof -i :PORT
```

2. Kill or reconfigure that process.

Tip:

Use ss -tln to list all open ports before binding new services.

Q77. Process always crashes after 1 minute of running.**Scenario:**

Service starts, runs fine, then crashes after 60 seconds.

Root Cause:

Could be systemd **WatchdogTimeout**, app memory leak, or config reload failure.

Solution:

1. Check logs:

```
journalctl -xe
```



2. Inspect unit file WatchdogSec=...
3. Run manually to confirm if app crashes outside systemd too.

Tip:

Avoid hardcoded timers unless watchdogs are explicitly needed.

Q78. Need to ensure a script auto-restarts on crash.**Scenario:**

You want a script to always restart if it crashes.

Root Cause:

Default shell scripts exit on failure and aren't managed.

Solution:

Wrap in a systemd service:

[Service]

ExecStart=/path/to/script.sh

Restart=always

RestartSec=5

Enable the service.

Tip:

This is more reliable than while true loops.

Q79. Background process blocks terminal until killed.**Scenario:**

You ran something with &, but it still ties up your terminal.

Root Cause:

It still outputs to the terminal or reads input.

Solution:

1. Use:

nohup yourcommand > /dev/null 2>&1 &

disown



Tip:

Use screen or tmux to avoid such issues entirely.

Q80. Process forks too many threads and slows down system.**Scenario:**

You notice 1000+ threads from one app, slowing things down.

Root Cause:

Thread leak or wrong concurrency setting.

Solution:

1. Check thread count:

```
ps -eLf | grep appname | wc -l
```

2. Reduce threads in app config.

Tip:

Use ulimit -u to cap total user-level processes and threads.

Q81. Server boots into emergency mode.

Scenario:

After reboot, the system shows Welcome to emergency mode and drops to a shell.

Root Cause:

- Fstab has invalid or missing mount entry
- Critical system file/directory is missing or corrupted

Solution:

1. Check fstab:

```
cat /etc/fstab
```

2. Comment the problematic mount (e.g., external drive), then:

```
mount -a
```

reboot



Tip:

Always use nofail in /etc/fstab for non-critical mounts to avoid boot failures.

Q82. Kernel panic on boot: "Unable to mount root fs".**Scenario:**

System shows a kernel panic and reboots or halts immediately.

Root Cause:

The kernel cannot find or mount the root filesystem.

Solution:

1. Boot from a live CD or rescue mode.
2. Check and repair:

```
fsck /dev/sdXn
```

3. Rebuild initramfs:

```
chroot /mnt
```

```
update-initramfs -u
```

Tip:

Keep a rescue ISO or recovery snapshot ready for fast recovery.

Q83. Grub menu doesn't show up at boot.**Scenario:**

System boots directly into OS, can't access grub menu.

Root Cause:

GRUB_TIMEOUT=0 or hidden menu config.

Solution:

1. Edit:

```
sudo vi /etc/default/grub
```

Set:

```
GRUB_TIMEOUT=5
```

```
GRUB_HIDDEN_TIMEOUT=0
```

2. Update grub:



`sudo update-grub`

Tip:

Hold **Shift** (BIOS) or **Esc** (UEFI) during boot to access GRUB temporarily.

Q84. Grub rescue prompt after reboot.

Scenario:

After booting, you see grub rescue> instead of the OS.

Root Cause:

Missing GRUB bootloader, corrupted MBR, or deleted boot partition.

Solution:

1. Boot from live CD.
2. Mount and chroot:

```
mount /dev/sdaX /mnt
```

```
grub-install --boot-directory=/mnt/boot /dev/sda
```

```
update-grub
```

Tip:

Take disk snapshots before resizing partitions or reinstalling bootloaders.

Q85. System stuck at "Loading initial ramdisk".

Scenario:

Boot hangs on the initramfs stage.

Root Cause:

Corrupted initrd.img or incorrect init binary location.

Solution:

1. Boot into recovery shell or live CD.
2. Mount root and rebuild initramfs:

```
update-initramfs -u -k all
```

Tip:

Avoid removing old kernel/initramfs versions unless space is low.

Q86. Boot takes over 5 minutes.

Scenario:

Boot process is unusually slow, hangs at certain services.

Root Cause:

Timed out services (e.g., network, NFS) or disk UUID mismatches.

Solution:

1. Analyze boot time:

```
systemd-analyze blame
```

2. Disable slow services:

```
systemctl disable service-name
```

Tip:

Check `journalctl -b` for detailed boot-time logs.

Q87. Kernel upgrade causes server to not boot.**Scenario:**

After upgrading kernel, boot fails — blank screen or crash.

Root Cause:

New kernel incompatible with modules or driver missing (e.g., for disk/NIC).

Solution:

1. Reboot into older kernel from GRUB.
2. Set default kernel:

```
sudo grub-set-default <menuentry>
```

```
update-grub
```

Tip:

Never remove older kernel versions until new one is fully tested.



Q88. Black screen after boot — no login prompt.

Scenario:

System boots, but shows only a black screen. No shell, no GUI.

Root Cause:

Wrong target (graphical instead of multi-user), X11/GDM issue, or no TTY active.

Solution:

1. Switch TTY: Press Ctrl + Alt + F2
2. Set correct target:

```
sudo systemctl set-default multi-user.target
```

```
sudo reboot
```

Tip:

Use multi-user target for servers; graphical is only for desktop systems.

Q89. init or systemd process missing or corrupted.

Scenario:

Boot fails with error like No init found.

Root Cause:

```
/sbin/init or /lib/systemd/systemd is missing or broken.
```

Solution:

1. Boot from live ISO.
2. Mount root and reinstall systemd:

```
sudo chroot /mnt
```

```
apt install --reinstall systemd
```

Tip:

Avoid deleting or modifying /sbin/init manually.

Q90. Reboot command doesn't work — system hangs.

Scenario:

reboot or shutdown -r now hangs and doesn't reboot the machine.

Root Cause:

A service or process is refusing to stop or a mounted NFS is blocking shutdown.

Solution:

1. Use force:

```
reboot -f
```

2. Or via magic sysrq:

```
echo b > /proc/sysrq-trigger
```

Tip:

Enable SysRq if disabled:

```
echo 1 > /proc/sys/kernel/sysrq
```

Q91. Boot fails with "UUID not found" error.

Scenario:

mount fails during boot due to missing UUID.

Root Cause:

Drive layout changed but /etc/fstab still uses old UUID.

Solution:

1. Get current UUIDs:

```
blkid
```

2. Update /etc/fstab with the correct UUID.

Tip:

Use LABEL= or /dev/sdX instead of UUIDs for removable disks.



Q92. Disk added but not detected during boot.

Scenario:

You attached a new disk, but it's not mounted at startup.

Root Cause:

Disk not present in /etc/fstab, or system doesn't wait for it.

Solution:

1. Add to fstab with nofail:

```
UUID=xxxx /mnt/data ext4 defaults,nofail 0 2
```

2. Run:

```
sudo mount -a
```

Tip:

Use nofail to avoid boot blocking if disk isn't present.

Q93. /boot partition full — can't install kernel updates.

Scenario:

apt upgrade or yum update fails due to no space in /boot.

Root Cause:

Too many old kernels.

Solution:

1. List installed kernels:

```
dpkg --get-architecture | grep linux-image
```

2. Remove old ones:

```
sudo apt remove linux-image-<version>
```

Tip:

Keep only **2 latest kernels** and remove others.

Q94. Kernel modules fail to load.

Scenario:

System says modprobe: FATAL: module not found.



Root Cause:

- Kernel module doesn't exist
- Wrong kernel version

Solution:

1. Check kernel version:

```
uname -r
```

2. Rebuild modules:

```
sudo depmod -a
```

Tip:

Use lsmod to list loaded modules; modprobe to add.

Q95. Kernel ring buffer (dmesg) shows I/O or memory errors.

Scenario:

You see disk or memory errors repeatedly in dmesg.

Root Cause:

Hardware-level faults — failing disk or RAM.

Solution:

1. For disk:

```
smartctl -a /dev/sdX
```

2. For memory:

- Reboot and run **memtest86+**

Tip:

Replace failing hardware immediately; don't just suppress warnings.

Q96. SELinux blocks services from starting after reboot.**Scenario:**

After reboot, web or DB service fails, logs show permission denied.

Root Cause:

SELinux denied access due to context mismatch.



Solution:

1. Temporarily disable:

```
setenforce 0
```

2. Fix context:

```
restorecon -Rv /var/www
```

Tip:

Use audit2allow to convert denial logs into policies if needed.

Q97. Kernel messages flooding logs (dmesg, /var/log/messages).**Scenario:**

Logs get filled rapidly with repetitive kernel messages.

Root Cause:

Driver or device generating excessive warnings (e.g., USB, NIC)

Solution:

1. Temporarily suppress:

```
dmesg -n 1
```

2. Permanently fix by updating kernel or blacklisting module.

Tip:

Don't ignore this; investigate the root hardware issue.

Q98. New kernel doesn't recognize a NIC or RAID device.**Scenario:**

NIC or disk not detected after kernel upgrade.

Root Cause:

Missing drivers or removed support in newer kernel.

Solution:

1. Boot into older kernel.
2. Check lspci and lsmod
3. Install proper kernel module or switch to known working version.



Tip:

Always test new kernels on staging systems first.

Q99. Kernel build fails — "missing headers".**Scenario:**

Trying to build a driver or module fails: missing kernel headers.

Root Cause:

Kernel headers not installed.

Solution:

Install matching headers:

```
sudo apt install linux-headers-$(uname -r)
```

Tip:

Always install build-essential package when compiling kernel modules.

Q100. Kernel upgrade silently skipped during update.**Scenario:**

You run apt upgrade, but kernel version stays the same.

Root Cause:

You need to run dist-upgrade or install explicitly.

Solution:

```
sudo apt update
```

```
sudo apt install linux-image-generic
```

Tip:

apt upgrade doesn't install new dependencies like kernels; use full-upgrade.

Q101. apt update fails with "Hash Sum mismatch".**Scenario:**

When running apt update, it fails with:

```
E: Failed to fetch ... Hash Sum mismatch
```

Root Cause:

Corrupted or out-of-sync APT cache or mirror issues.



Solution:

1. Clean local cache:

```
sudo rm -rf /var/lib/apt/lists/*
```

```
sudo apt update
```

Tip:

Switch mirrors if issue persists: use official or country-based mirrors.

Q102. yum install fails with "Cannot find a valid baseurl".**Scenario:**

YUM can't connect to any repo and fails with:

Cannot find a valid baseurl for repo

Root Cause:

- DNS resolution issue
- Broken repo URL
- Internet not reachable

Solution:

1. Check internet:

```
ping google.com
```

2. Check repo file:

```
cat /etc/yum.repos.d/*.repo
```

```
Fix or change baseurl.
```

Tip:

Use CentOS Vault or EPEL for older RHEL versions.

Q103. Package install fails with "Package is already installed".**Scenario:**

Installing a package gives:

```
dpkg: error processing package -- already installed
```



Root Cause:

Broken or half-configured install.

Solution:

1. Fix broken packages:

```
sudo dpkg --configure -a
```

```
sudo apt install -f
```

Tip:

This happens often after CTRL+C during install — always let apt finish.

Q104. Can't install .deb file due to dependency errors.**Scenario:**

Using `dpkg -i package.deb` fails with dependency errors.

Root Cause:

`dpkg` doesn't auto-resolve dependencies.

Solution:

1. Run:

```
sudo apt install -f
```

Tip:

Prefer `apt install ./package.deb` — it resolves dependencies automatically.

Q105. RPM install fails with "Failed dependencies".**Scenario:**

Installing .rpm package fails due to missing libraries or dependencies.

Root Cause:

YUM/DNF not used, or dependencies not in repo.

Solution:

1. Try:

```
sudo yum localinstall package.rpm
```

2. Or download all deps using:

```
sudo dnf install ./package.rpm
```



Tip:

Use yum provides to find which package offers the missing dependency.

Q106. apt install always installs old package version.**Scenario:**

APT pulls an outdated version, not the latest.

Root Cause:

- Repo cache not updated
- New version not available in default repo

Solution:

1. Update:

```
sudo apt update
```

2. Use versioned install:

```
sudo apt install pkgname=1.2.3-1
```

Tip:

Use apt policy pkgname to list available versions.

Q107. Trying to uninstall a package removes multiple dependencies.

Scenario:

Uninstalling a package triggers removal of many important ones.

Root Cause:

It's a dependency or part of a meta-package.

Solution:

1. Use apt remove carefully.
2. Simulate first:

```
sudo apt remove --simulate pkgname
```

Tip:

Never apt autoremove blindly — verify the list first.



Q108. yum or dnf gives GPG key warning.

Scenario:

Package install fails due to:

Public key for rpm not installed

Root Cause:

Repo's GPG key not imported.

Solution:

1. Import the key:

```
sudo rpm --import /etc/pki/rpm-gpg/RPM-GPG-KEY-CentOS-7
```

Tip:

Use --nogpgcheck only for testing — it's unsafe in production.

Q109. Installing one package breaks another.

Scenario:

Installing a package downgrades or breaks existing working software.

Root Cause:

Repo conflict or manual .deb/.rpm file overriding system-managed versions.

Solution:

1. Use pinning (APT):

```
echo -e "Package: pkgname\nPin: version 1.2.3\nPin-Priority: 1001" | sudo tee  
/etc/apt/preferences.d/pkgname
```

Tip:

For RHEL, use version-lock plugin:

```
yum versionlock add pkgname
```

Q110. dpkg package is broken and blocks all operations.

Scenario:

dpkg is stuck or corrupted. All apt/dpkg commands fail.

Root Cause:

Unfinished install or broken post-install script.



Solution:

1. Force fix:

```
sudo dpkg --configure -a
```

```
sudo apt install -f
```

Tip:

If package still stuck, remove .list and .postinst from /var/lib/dpkg/info/.

Q111. apt update is very slow.**Scenario:**

APT update takes minutes on a fast internet connection.

Root Cause:

Slow mirror or IPv6 issues.

Solution:

1. Switch mirror:

```
sudo sed -i 's|archive.ubuntu.com|mirror.xyz.com|g' /etc/apt/sources.list
```

2. Disable IPv6 (if needed):

```
echo 'Acquire::ForceIPv4 "true";' | sudo tee /etc/apt/apt.conf.d/99force-ipv4
```

Tip:

Use mirrors.ubuntu.com/mirrors.txt to find the fastest local mirror.

Q112. A package is installed but command not found.**Scenario:**

apt says package is installed, but running the command gives “command not found”.

Root Cause:

Package doesn't install binary in PATH, or needs login shell to expose it.

Solution:

1. Find binary:

```
dpkg -L pkgname | grep bin
```

2. Use full path or symlink:



```
sudo ln -s /opt/tool/bin/exe /usr/local/bin/exe
```

Tip:

Use which or type -a to check path visibility.

Q113. Need to downgrade a package.**Scenario:**

New version of package breaks functionality; want to rollback.

Root Cause:

APT/YUM does not downgrade by default.

Solution:

1. Use versioned install:

```
sudo apt install pkg=old_version
```

or for RPM:

```
sudo dnf downgrade pkgname
```

Tip:

Pin the version to prevent auto-upgrade in future.

Q114. Package upgrade fails due to held packages.**Scenario:**

APT blocks upgrade due to held packages.

Root Cause:

Manual hold set earlier.

Solution:

1. List held packages:

```
apt-mark showhold
```

2. Unhold:

```
sudo apt-mark unhold pkgname
```

Tip:

Use hold for critical packages like kernels or libraries you don't want changed.



Q115. After installing software, man pages not available.**Scenario:**

Command works, but man tool gives "No manual entry".

Root Cause:

Man pages are in a separate package.

Solution:

Install documentation package:

```
sudo apt install pkgname-doc
```

Tip:

Some packages use info instead of man. Try:

info tool

Q116. Want to remove all unused packages and dependencies.**Scenario:**

System is cluttered with unused libs and dependencies.

Root Cause:

Autoremove not run after uninstalling packages.

Solution:

1. Run cleanup:

```
sudo apt autoremove
```

```
sudo apt clean
```

Tip:

In RHEL, use:

```
dnf autoremove
```

Q117. Installed two versions of Python, package installs into wrong one.**Scenario:**

pip install puts package in wrong Python path.

Root Cause:

Default pip not pointing to desired version.

Solution:

1. Use version-specific pip:

```
python3.10 -m pip install flask
```

Tip:

Always use virtualenv for app-specific dependencies.

Q118. RPM installation reports “conflicts with file from package X”.

Scenario:

Installing .rpm reports file path already owned by another package.

Root Cause:

Two RPMs try to own same file.

Solution:

1. Remove conflicting package:

```
sudo rpm -e --nodeps conflictingpkg
```

2. Or skip file overwrite via rpm2cpio.

Tip:

Resolve properly, don't force installs in production.

Q119. Package says installed, but binary is missing after reboot.

Scenario:

App installed, works, then disappears on reboot.

Root Cause:

Installed on ephemeral or non-persistent mount (e.g., /tmp, overlay).

Solution:

1. Reinstall in persistent directory:

```
sudo apt install --reinstall pkgname
```

Tip:

Always verify mount output before installing in dynamic file systems.



Q120. Want to know which package provides a specific command or file**Scenario:**

Need to install a package that contains a specific binary.

Solution:

1. On Debian/Ubuntu:

```
apt-file search /usr/bin/command
```

2. On RHEL/CentOS:

```
yum provides /usr/bin/command
```

Tip:

Install apt-file with `sudo apt install apt-file && apt-file update`

Q121. New user created but cannot login via SSH.**Scenario:**

User exists, but SSH login fails with Permission denied.

Root Cause:

User's home directory or `.ssh/authorized_keys` missing or wrong permissions.

Solution:

1. Check home directory permissions:

```
ls -ld /home/username
```

```
chmod 700 /home/username
```

2. Fix `.ssh` permissions:

```
chmod 700 ~/.ssh
```

```
chmod 600 ~/.ssh/authorized_keys
```

Tip:

SSH is very strict about directory and file permissions for security.

Q122. sudo command says "user is not in the sudoers file".**Scenario:**

Trying to run sudo, but gets:

user is not in the sudoers file. This incident will be reported.

Root Cause:

User isn't added to the sudo group or explicitly allowed.

Solution:

1. Add to sudo group:

```
sudo usermod -aG sudo username
```

2. Or edit sudoers safely:

```
sudo visudo
```

Tip:

Never edit /etc/sudoers manually — always use visudo to avoid syntax errors.

Q123. Password expiry forces user logout immediately after login.**Scenario:**

User logs in but immediately sees "Your password has expired" and logout.

Root Cause:

Password expired but no chance to reset.

Solution:

1. Reset password:

```
sudo passwd username
```

2. Check expiry:

```
sudo chage -l username
```

Tip:

Set password policies carefully via /etc/login.defs.



Q124. After a user is deleted, their processes still run.**Scenario:**

Deleted a user account, but their processes are still active.

Root Cause:

Deleting a user doesn't auto-kill processes.

Solution:

1. Kill remaining processes:

```
kill -u username
```

Tip:

Always kill -u immediately after deleting a user to cleanly terminate all tasks.

Q125. Cannot su to another user — "Authentication failure".**Scenario:**

su - username fails with authentication error even with correct password.

Root Cause:

- Account locked
- Shell not valid

Solution:

1. Unlock account:

```
sudo passwd -u username
```

2. Check shell:

```
sudo usermod -s /bin/bash username
```

Tip:

Always ensure /etc/passwd has a valid shell for login users.



Q126. New user created but home directory missing.**Scenario:**

User created, but /home/username directory not present.

Root Cause:

User created without -m option or default configs changed.

Solution:

1. Create manually:

```
sudo mkdir /home/username
```

```
sudo chown username:username /home/username
```

2. Or recreate user properly:

```
sudo useradd -m username
```

Tip:

Always create home directory unless it's a system/service account.

Q127. id shows wrong groups after adding a user to a new group.**Scenario:**

User added to a new group but id username doesn't show updated group.

Root Cause:

Group membership cached in session. Logout/login required.

Solution:

1. User must logout and log back in.
2. Or re-exec shell:

```
exec su - username
```

Tip:

Use newgrp groupname for instant session group switch.



Q128. sudo asks for password every time.

Scenario:

User is in sudoers, but sudo always asks for password.

Root Cause:

Default sudo behavior.

Solution:

1. Edit sudoers:

```
sudo visudo
```

2. Add:

```
username ALL=(ALL) NOPASSWD:ALL
```

Tip:

Use NOPASSWD only for trusted accounts or specific commands.

Q129. Cannot delete a user: "user is currently logged in".**Scenario:**

userdel fails with "user is logged in".

Root Cause:

Active shell/session is preventing deletion.

Solution:

1. Kill user session:

```
pkill -u username
```

```
sudo userdel -r username
```

Tip:

Use who or w to see all active sessions before deletion.



Q130. /etc/passwd manually edited, now users can't login.**Scenario:**

Manually edited /etc/passwd, and now login fails.

Root Cause:

Syntax error or corrupted fields.

Solution:

1. Validate:

```
pwck
```

2. Restore from backup:

```
cp /etc/passwd- /etc/passwd
```

Tip:

Always use vipw to safely edit /etc/passwd.

Q131. LDAP user cannot use sudo.**Scenario:**

LDAP-authenticated user logs in but gets "not in sudoers" error.

Root Cause:

LDAP config missing sudo rules, or sudo schema not enabled.

Solution:

1. Update /etc/nsswitch.conf:

```
sudoers: files ldap
```

2. Or assign sudo rights manually in /etc/sudoers.

Tip:

Corporate LDAP setups should push sudo policies centrally.

Q132. /etc/shadow file is missing.**Scenario:**

System login completely broken — no password authentication possible.

Root Cause:

/etc/shadow accidentally deleted or corrupted.

Solution:

1. Boot into recovery mode.
2. Recreate or copy backup:

```
cp /etc/shadow- /etc/shadow
```

```
chmod 000 /etc/shadow
```

Tip:

Protect /etc/shadow — it's critical; restrict access to root-only.

Q133. User account is locked accidentally.**Scenario:**

User cannot login; gets "account locked" message.

Root Cause:

Account locked with `passwd -l` or failed login attempts.

Solution:

1. Unlock:

```
sudo passwd -u username
```

Tip:

Monitor /etc/shadow for accounts with ! mark indicating lock.

Q134. Users can su to root without password.**Scenario:**

Anyone can su to root without a password prompt.

Root Cause:

Root password is blank or /etc/pam.d/su misconfigured.

Solution:

1. Set root password:

```
sudo passwd root
```

2. Verify PAM config.



Tip:

Never leave root password blank, even if SSH is disabled.

Q135. User can login even after being deleted.**Scenario:**

Deleted a user, but they still can SSH/login.

Root Cause:

SSH key in ~/.ssh/authorized_keys allows access even without /etc/passwd entry.

Solution:

1. Remove home directory:

```
rm -rf /home/username
```

2. Remove cached sessions:

```
sudo loginctl terminate-user username
```

Tip:

Always delete /home/username after user deletion.

Q136. Cannot change user's shell — "Shell not found".**Scenario:**

Trying to set user's shell but error:

```
chsh: /new/shell: No such file
```

Root Cause:

Shell must exist in /etc/shells.

Solution:

1. Check available shells:

```
cat /etc/shells
```

2. Add new shell path if required.

Tip:

Only allow approved shells for security compliance.



Q137. User keeps getting disconnected after login.

Scenario:

User logs in successfully but immediately disconnected.

Root Cause:

- .bashrc, .profile, or shell scripts contain exit/logout command.

Solution:

1. Check user's .bashrc, .bash_profile, .profile.
2. Look for exit, logout, or fatal errors.

Tip:

Always test user startup scripts before deploying widely.

Q138. Need to expire user account after a specific date.**Scenario:**

You want a temporary user to auto-expire after 7 days.

Solution:

Set expiry:

```
sudo chage -E $(date -d "+7 days" +%Y-%m-%d) username
```

Tip:

Use chage -l username to review expiry settings anytime.

Q139. Need to force user password change at next login.**Scenario:**

Force users to reset passwords when they first login.

Solution:

Force change:

```
sudo chage -d 0 username
```

Tip:

Good practice when creating new corporate users.



Q140. How to allow a user to run only one command via sudo.

Scenario:

User should be allowed to run only specific commands with sudo.

Solution:

Edit sudoers:

```
username ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx
```

Tip:

Limit sudo carefully for security — never give unrestricted shells via sudo.

Q141. Service is enabled but not starting at boot.

Scenario:

systemctl enable servicename done, but after reboot service is inactive.

Root Cause:

- Wrong target
- Dependency service not ready
- Service failed early but not retried

Solution:

1. Check target:

bash

CopyEdit

```
systemctl list-dependencies multi-user.target
```

2. Add proper After= dependencies in the unit file if needed.
3. Analyze boot logs:

```
journalctl -b -u servicename
```

Tip:

Always match service timing with network.target or remote-fs.target if network mount involved.



Q142. systemctl start fails with "Unit not found".

Scenario:

Trying to start service, error:

Unit myapp.service not found.

Root Cause:

- Service file missing in /etc/systemd/system/
- Typo in service name

Solution:

1. List all units:

systemctl list-units --all

2. Reload daemon:

sudo systemctl daemon-reexec

Tip:

After adding new .service files, always run systemctl daemon-reload.

Q143. Edited systemd service file but changes don't apply.

Scenario:

Modified .service file, but behavior didn't change.

Root Cause:

Systemd caches units in memory; needs daemon reload.

Solution:

1. Reload:

sudo systemctl daemon-reload

2. Then restart service:

sudo systemctl restart myapp

Tip:

Use systemctl show servicename to verify active settings.



Q144. systemctl restart command hangs indefinitely.**Scenario:**

Restarting a service hangs forever.

Root Cause:

- Service script not handling stop properly
- PID file stale

Solution:

1. Force stop:

```
sudo systemctl kill servicename
```

2. Check ExecStop and PIDFile configs in .service.

Tip:

Always set reasonable TimeoutSec=30 in critical services.

Q145. Service keeps failing repeatedly (Restart loop).**Scenario:**

Service starts, crashes, auto-restarts in loop.

Root Cause:

- Restart settings too aggressive
- Actual binary error

Solution:

1. Set limits:

```
Restart=on-failure
```

```
RestartSec=5
```

2. Investigate real crash cause via logs.

Tip:

Don't use Restart=always without a good reason — it can hide real failures.



Q146. Status of service shows "inactive (dead)".

Scenario:

Systemd shows service as inactive after starting it manually.

Root Cause:

- Service exited immediately after starting
- Missing Type=forking or wrong ExecStart

Solution:

1. Set proper Type:

Type=simple

Or if it forks:

Type=forking

Tip:

Forking daemons must have PID tracking for systemd to manage them properly.

Q147. journalctl logs are missing after reboot.

Scenario:

Old service logs disappear after system restart.

Root Cause:

- Logs stored in memory (volatile)
- Persistent logging disabled

Solution:

1. Configure persistent logs:

sudo mkdir -p /var/log/journal

sudo systemctl restart systemd-journald

Tip:

Check /etc/systemd/journald.conf — set Storage=persistent.



Q148. Systemd service fails — "Exec format error".

Scenario:

Trying to start service, error:

Exec format error

Root Cause:

Wrong shebang (!) or executable permissions.

Solution:

1. Check script file:

```
head -1 /path/to/script.sh
```

```
chmod +x /path/to/script.sh
```

2. Fix first line:

```
#!/bin/bash
```

Tip:

Always verify scripts run manually before using in systemd.

Q149. Daemon is running but systemctl status shows failed.

Scenario:

Service actually running, but systemctl status shows "failed".

Root Cause:

Service started outside systemd (manual nohup, etc.) or PID tracking broken.

Solution:

- Always start/stop services using systemd, not manually.
- Fix PIDFile parameter in unit file if used.

Tip:

Use Type=notify if app supports sd_notify() for better tracking.



Q150. Service refuses to bind to port — permission denied.**Scenario:**

Service fails to bind to port below 1024 (e.g., port 80).

Root Cause:

Only root can bind ports < 1024.

Solution:

1. Run service as root, or
2. Use authbind / setcap:

```
sudo setcap 'cap_net_bind_service=+ep' /path/to/binary
```

Tip:

Safer to bind to high ports and use Nginx reverse proxy if possible.

Q151. System boot time increased after enabling a service.**Scenario:**

Boot takes longer after you enabled a new custom service.

Root Cause:

Service startup delays waiting for external resources.

Solution:

1. Add TimeoutStartSec=30 inside service unit.
2. Or move service to After=network-online.target or remote-fs.target.

Tip:

Use systemd-analyze blame to identify slow services.

Q152. Created custom service but it doesn't appear in systemctl list.**Scenario:**

New .service file created but systemctl list-units doesn't show it.

Root Cause:

New unit file placed wrong, or daemon-reload missed.

Solution:

1. Move unit file to:

`/etc/systemd/system/`

2. Then:

`sudo systemctl daemon-reload`

Tip:

Units should always go into `/etc/systemd/system/` (not `/usr/lib/systemd/` manually).

Q153. Cannot stop systemd service: "Job for xyz.service canceled".

Scenario:

`systemctl stop` immediately returns job canceled.

Root Cause:

Service stop script fails or exits early.

Solution:

- Check `ExecStop` command inside unit file.
- Use `kill` if badly hung.

Tip:

Graceful shutdown scripts are critical in writing services.

Q154. After disabling service, it still auto-starts.

Scenario:

Disabled a service, but it restarts automatically after reboot.

Root Cause:

- Service is socket-activated
- Timer unit triggers it

Solution:

1. Check for related sockets:

`systemctl list-sockets`

2. Check for timers:

`systemctl list-timers`



Disable them if needed.

Tip:

Timer-based activations need explicit disabling too.

Q155. A systemd service runs fine manually but fails under systemctl.

Scenario:

Manual command works but systemctl fails.

Root Cause:

Environment variables missing inside systemd.

Solution:

1. Set environment in service:

`Environment="ENV_VAR=value"`

Or load file:

`EnvironmentFile=/etc/myapp/env`

Tip:

Always test with `systemctl show-environment`.

Q156. System time wrong even after NTP enabled.

Scenario:

`timedatectl` status shows NTP active, but wrong time.

Root Cause:

- NTP server unreachable
- Firewall blocking NTP ports

Solution:

1. Verify NTP servers:

`chronyc sources`

2. Or force sync:

`sudo timedatectl set-ntp true`

`sudo ntpdate pool.ntp.org`



Tip:

Always whitelist UDP 123 for NTP in firewall.

Q157. Systemd journal files consuming too much disk space.**Scenario:**

/var/log/journal/ grows very large.

Root Cause:

Unlimited log retention policy.

Solution:

1. Edit /etc/systemd/journald.conf:

```
SystemMaxUse=500M
```

2. Restart journald:

```
systemctl restart systemd-journald
```

Tip:

Configure log rotation to control disk usage.

Q158. Cannot mask a service — "Masking is not permitted".**Scenario:**

Trying to mask a critical system service but getting permission error.

Root Cause:

- Insufficient privilege
- Trying to mask protected units

Solution:

1. Run as root:

```
sudo systemctl mask servicename
```

Tip:

Mask critical services (like systemd-udev) only when absolutely necessary.



Q159. systemctl commands are extremely slow.**Scenario:**

Running systemctl commands takes 10–20 seconds to return.

Root Cause:

- DNS resolution delays
- Systemd waits on unresponsive hosts

Solution:

1. Disable slow DNS resolution in /etc/hosts.
2. Set UseDNS=no in sshd config if SSH delay involved.

Tip:

Check /etc/hosts and hostnamed status for consistency.

Q160. Need to limit resource usage of a service.**Scenario:**

Want to cap memory, CPU for a specific service.

Solution:

Add into unit file:

MemoryMax=500M

CPUQuota=50%

Reload daemon and restart service.

Tip:

Use systemd-cgtop to monitor live cgroup resource usage.

Q161. Cronjob is scheduled but doesn't run.**Scenario:**

Added an entry to crontab, but script never executes.

Root Cause:

- Wrong cron syntax

- Path issues
- Script lacks execution permission

Solution:

1. Check cron syntax:
2. Add full paths inside scripts.
3. Make script executable:

```
chmod +x script.sh
```

Tip:

Cron uses a minimal environment — always specify absolute paths.

Q162. Cronjob runs but output is missing.**Scenario:**

Cron executes but no output saved.

Root Cause:

- No output redirection
- Silent failure

Solution:

Redirect output:

```
* * * * * /path/to/script.sh > /tmp/script.log 2>&1
```

Tip:

Always log both stdout and stderr for cronjobs.

Q163. Cron says "Permission denied" when running a script.**Scenario:**

Cron tries to run a script but permission error occurs.

Root Cause:

Script missing executable bit or ownership mismatch.

Solution:

1. Make executable:

```
chmod +x /path/to/script.sh
```



2. Ensure user has access.

Tip:

Cron runs as the user who owns the crontab — permissions must match.

Q164. crontab -e gives "no crontab for user".**Scenario:**

First time setting up cron, you see:

no crontab for user

Root Cause:

Normal behavior — no crontab yet.

Solution:

- Just create new entries after opening crontab -e.
- Save and exit.

Tip:

Use sudo crontab -e carefully — root's crontab is different from user's.

Q165. Cron job fails silently when using sudo in the script.**Scenario:**

Script with sudo commands fails without visible errors.

Root Cause:

Cron environment may not allow sudo without tty.

Solution:

- Edit /etc/sudoers:

Defaults:username !requiretty

Or better: avoid sudo inside cron scripts — pre-authorize permissions.

Tip:

Safer to schedule privileged jobs using root's crontab directly.



Q166. Email alerts from cron not received.**Scenario:**

Cron job has errors but no email notification is received.

Root Cause:

Mail system (sendmail or postfix) not configured.

Solution:

1. Install MTA:

```
sudo apt install postfix
```

2. Ensure cron sends emails.

Tip:

Or redirect cron output manually to a mail command.

Q167. A cronjob runs every minute instead of once daily.**Scenario:**

Job runs much more often than intended.

Root Cause:

Wrong crontab timing format.

Solution:

Correct format for daily at midnight:

```
0 0 * * * /path/to/script.sh
```

Tip:

Use crontab.guru website to validate cron expressions.

Q168. Cron runs script but reports "command not found".**Scenario:**

Script runs from shell but fails under cron.

Root Cause:

Environment variables like PATH missing in cron.

Solution:

Specify full path inside script:



```
/usr/bin/python3 /path/to/script.py
```

```
Or define PATH at top of script.
```

Tip:

Print environment inside cron script (env > /tmp/env.txt) to debug.

Q169. Cron job requires internet but fails after reboot.**Scenario:**

Script needs network but fails if cron runs early after boot.

Root Cause:

Network not yet ready at job execution.

Solution:

Delay job by adding sleep:

```
@reboot sleep 60 && /path/to/script.sh
```

Tip:

Or use systemd After=network-online.target units for critical network jobs.

Q170. Need to run cron only on weekdays.**Scenario:**

Script must run Monday–Friday only.

Solution:

Use:

```
0 9 * * 1-5 /path/to/script.sh
```

```
(Runs 9 AM Monday to Friday)
```

Tip:

Numbers 1-5 represent Monday (1) through Friday (5) in cron.

Q171. A cronjob modifies environment for future shells — doesn't work.**Scenario:**

Script sets variables but next shell doesn't see them.

Root Cause:

Cron jobs don't persist to user's shell environment.



Solution:

Use scripts to export into a file:

```
/path/to/script.sh > ~/.bash_profile
```

Reload manually later.

Tip:

Better to have static environment variables for crontab — avoid dynamic ones.

Q172. Want to schedule a cronjob every 15 minutes.**Scenario:**

Need periodic execution every 15 minutes.

Solution:

Add to crontab:

```
*/15 * * * * /path/to/script.sh
```

Tip:

*/15 covers minute 0, 15, 30, and 45 each hour.

Q173. Cron logs missing even after enabling logging.**Scenario:**

No cron activity visible in logs.

Root Cause:

Syslog not logging cron or cron daemon misconfigured.

```
sudo tail -f /var/log/syslog
```

1. Enable cron service:

```
sudo systemctl enable cron
```

```
sudo systemctl restart cron
```

Tip:

On RHEL, cron logs usually go to /var/log/cron.



Q174. Script runs manually but fails via cron — permissions error.**Scenario:**

Script works fine from terminal but fails when cron runs it.

Root Cause:

Different user context under cron.

Solution:

- Fix file permissions.
- Ensure script and files are readable/writable by cron job user.

Tip:

Always test cron jobs as the intended user (su - username).

Q175. Need to run a heavy cronjob during off-peak hours.**Scenario:**

Backup job needs to run only at night.

Solution:

Schedule at 2 AM:

```
0 2 * * * /path/to/backup.sh
```

Tip:

Backup/maintenance jobs should always be scheduled during lowest usage windows.

Q176. Cronjob scheduled but server reboot wipes all jobs.**Scenario:**

Cron entries disappear after reboot.

Root Cause:

Crontab file was not saved properly or running on ephemeral filesystem.

Solution:

- Always use crontab -e and save properly.
- Avoid editing /var/spool/cron/username manually.

Tip:

Check persistence settings if using cloud ephemeral disks.



Q177. Multiple crontabs for same user conflicting.

Scenario:

Two separate crontabs overwrite each other's jobs.

Root Cause:

Each crontab -e command edits the same user's crontab.

Solution:

- Combine all cron jobs into one crontab.
- Maintain a cron management script if needed.

Tip:

Use dedicated system cron files /etc/cron.d/ for complex cases.

Q178. Need to disable a cronjob temporarily without deleting it.

Scenario:

Want to stop cronjob but not remove it permanently.

Solution:

- Comment out the cron line by prefixing with #.
- Example:

```
# 0 3 * * * /path/to/script.sh
```

Tip:

Leave reason for disabling in comments for future reference.

Q179. Need a cron job that runs only once after reboot.

Scenario:

Script should run only once after boot.

Solution:

Use:

```
@reboot /path/to/script.sh && crontab -l | grep -v 'script.sh' | crontab -
```

Tip:

Or better: use systemd oneshot service for cleaner one-time startup jobs.

Q180. Need separate cron schedules for different users.

Scenario:

Different users need different cron schedules.

Solution:

Each user has their own crontab:

```
sudo crontab -u username -e
```

Tip:

Use `/etc/cron.d/` if you want central control over multiple users' cronjobs.

Q181. rsync backup fails with "Permission denied".

Scenario:

Trying to backup files using rsync, but gets:

```
rsync: send_files failed: Permission denied
```

Root Cause:

Destination directory not writable by user.

Solution:

1. Ensure destination permissions:

```
chmod -R u+w /backup/dir
```

2. If using remote, add sudo properly.

Tip:

When doing remote backups, prefer SSH key-based access.

Q182. tar extraction fails — "Cannot open: Permission denied".

Scenario:

Extracting a tar file throws permission errors.

Root Cause:

Extracting as non-root user into a root-owned directory.

Solution:

Use:

```
sudo tar -xvf archive.tar -C /target/dir
```



Tip:

Always ensure correct target folder permissions before extracting.

Q183. Need to create a compressed .tar.gz backup.**Scenario:**

Want to create a compressed backup file.

Solution:

Run:

```
tar -czvf backup.tar.gz /path/to/data
```

- c → create
- z → gzip compress
- v → verbose
- f → file output

Tip:

Add date in filenames: backup-\$(date +%F).tar.gz

Q184. Restore only one file from a .tar.gz archive.**Scenario:**

Want to extract a single file from a huge backup.

Solution:

List files:

```
tar -tzf backup.tar.gz
```

Extract one:

```
tar -xzf backup.tar.gz path/to/neededfile
```

Tip:

Always preview (-t) before extraction to avoid overwriting.

Q185. rsync shows 100% CPU usage during large sync.**Scenario:**

System slows down when doing large rsync backups.



Root Cause:

Rsync checksum calculations are CPU-heavy.

Solution:

Use `--whole-file` for local transfers:

```
rsync --whole-file src/ dest/
```

Tip:

For very large syncs, prefer `rsync -z` for compression only on slow networks.

Q186. Rsync copies everything again instead of just differences.**Scenario:**

Each rsync re-copies all files, not just changed ones.

Root Cause:

- Timestamps mismatch
- File permissions changed

Solution:

Use proper flags:

```
rsync -avz src/ dest/
```

- `-a` preserves timestamps and permissions.

Tip:

Always avoid modifying timestamps (`touch`) between syncs unless necessary.

Q187. Rsync deletes important files at destination.**Scenario:**

Accidentally added `--delete` flag, files lost.

Root Cause:

`--delete` removes files at destination not present at source.

Solution:

Avoid unless absolutely necessary.

For safe delete, use:

```
rsync --delete --backup --backup-dir=/path/to/recovery src/ dest/
```



Tip:

Always simulate first with `--dry-run` before risky `rsync` operations.

Q188. Tar backup is corrupt — can't extract.**Scenario:**

Extraction fails midway, saying unexpected EOF.

Root Cause:

Partial transfer, disk full, or network timeout.

Solution:

1. Check tar file size.
2. Recreate backup:

```
tar -czvf newbackup.tar.gz /data
```

Tip:

Always verify backup integrity immediately after creation.

Q189. Need incremental backup using tar.**Scenario:**

Full backup every day is too large; want only changes.

Solution:

Use:

```
tar --create --file=backup.tar --listed-incremental=snapshot.file /path
```

- `snapshot.file` tracks changes.

Tip:

Combine full + incremental backups smartly for faster restores.

Q190. Rsync between two remote servers without local download.**Scenario:**

Copy files between two remote servers without pulling locally.

Solution:

Direct transfer:

```
rsync -avz -e ssh user1@host1:/path user2@host2:/path
```



Tip:

SSH key authentication speeds up remote rsync significantly.

Q191. How to schedule daily automatic backups with rsync.**Scenario:**

Want to automate daily backups.

Solution:

Add crontab:

```
0 2 * * * rsync -avz /data/ /backup/
```

Runs daily at 2 AM.

Tip:

Redirect logs to file for troubleshooting cron-based backups.

Q192. Need to verify integrity of tar backup.**Scenario:**

Worried if backup tar is not corrupt.

Solution:

Test tar without extracting:

```
tar -tvf backup.tar.gz > /dev/null
```

If no error = healthy archive.

Tip:

Use checksums (md5sum, sha256sum) for extra verification.

Q193. Rsync skips hidden files.**Scenario:**

After rsync, hidden dotfiles missing at destination.

Root Cause:

Source path missing trailing slash or pattern filter wrong.

Solution:

Use:

```
rsync -avz /src/ /dest/
```



(trailing / matters!)

Tip:

Always specify source as /src/ not /src.

Q194. Tar extraction overwrites existing files.

Scenario:

Untarring a backup overwrites current files.

Solution:

Extract to a new directory:

```
mkdir newdir
```

```
tar -xvf backup.tar.gz -C newdir/
```

Tip:

Always extract cautiously in production environments.

Q195. Rsync consumes too much network bandwidth.

Scenario:

Rsync hogs all bandwidth, affecting other services.

Solution:

Throttle:

```
rsync --bwlimit=5000 -avz src/ dest/
```

(Limits to ~5MB/sec)

Tip:

Use --bwlimit during working hours; unthrottled during maintenance.

Q196. Need to backup an entire Linux system.

Scenario:

Want full Linux filesystem backup.

Solution:

Exclude runtime dirs:

```
rsync -aAXv --exclude={"/proc", "/tmp", "/dev", "/sys", "/run"} / /mnt/backup
```



Tip:

-AAX preserves ACLs, xattrs, hard links — crucial for system recovery.

Q197. Restore a Linux server from rsync backup.**Scenario:**

Server failed, need to restore from backup.

Solution:

- Boot rescue mode.
- Partition and mount disks.
- Rsync backup onto fresh system:

```
rsync -aAXv /mnt/backup/ /mnt/newdisk/
```

- Reinstall bootloader:

```
grub-install /dev/sda
```

```
update-grub
```

Tip:

Double-check /etc/fstab UUIDs after restore.

Q198. Rsync backup corrupts file permissions.**Scenario:**

Restored files have wrong permissions.

Root Cause:

Rsync without -a flag loses permissions.

Solution:

Use:

```
rsync -a src/ dest/
```

Tip:

-a == archive mode: preserves permissions, symlinks, timestamps.



Q199. Rsync transfer interrupted — need to resume.

Scenario:

Backup got interrupted midway due to network drop.

Solution:

Rerun rsync with the same command:

```
rsync -avzP src/ dest/
```

- -P enables partial resume.

Tip:

Rsync is intelligent — it resumes only missing parts!

Q200. Need to encrypt a tar backup for secure storage.

Scenario:

Want to backup data and encrypt it.

Solution:

Create backup:

```
tar -czf backup.tar.gz /data
```

Encrypt:

```
gpg -c backup.tar.gz
```

(Will prompt for passphrase)

Tip:

Use `gpg --batch --passphrase-file` for automated secure backups.



Top 300 Linux Commands Asked in Interviews



1. **ls** – Lists files and directories in the current directory.
2. **ls -al** – Lists all files, including hidden ones, with detailed information.
3. **pwd** – Prints the current working directory.
4. **cd /path/to/directory** – Changes the directory to the specified path.
5. **cd ..** – Moves up one directory level.
6. **mkdir new_directory** – Creates a new directory.
7. **rmdir empty_directory** – Removes an empty directory.
8. **rm -rf directory_name** – Deletes a directory and its contents recursively.
9. **touch file.txt** – Creates a new empty file.
10. **cat file.txt** – Displays the contents of a file.

11. **tac file.txt** – Displays the contents of a file in reverse order.

12. **nano file.txt** – Opens a file in the nano text editor.

13. **vim file.txt** – Opens a file in the Vim editor.

14. **vi file.txt** – Opens a file in the vi editor.

15. **echo "Hello, World!"** – Prints text to the terminal.

16. **echo "Hello" > file.txt** – Writes text to a file (overwrites existing content).

17. **echo "Hello" >> file.txt** – Appends text to a file.

18. **cp source.txt destination.txt** – Copies a file.

19. **cp -r source_directory destination_directory** – Copies a directory recursively.

20. **mv old_name.txt new_name.txt** – Renames a file.

21. **mv file.txt /path/to/destination/** – Moves a file to another directory.

22. **rm file.txt** – Deletes a file.



- 23. **find / -name "file.txt"** – Searches for a file by name starting from the root directory.
- 24. **find . -type f -name "*.log"** – Finds all log files in the current directory.
- 25. **locate file.txt** – Finds the location of a file using a pre-built index.
- 26. **updatedb** – Updates the locate command's index.
- 27. **grep "search_term" file.txt** – Searches for a term inside a file.
- 28. **grep -i "search_term" file.txt** – Case-insensitive search.
- 29. **grep -r "search_term" /path/to/search/** – Searches recursively in a directory.
- 30. **awk '{print \$1}' file.txt** – Prints the first column of a file.
- 31. **awk -F: '{print \$1}' /etc/passwd** – Prints the first field of the `/etc/passwd` file, separated by colons.
- 32. **sed 's/old/new/g' file.txt** – Replaces all occurrences of "old" with "new" in a file.
- 33. **sed -i 's/old/new/g' file.txt** – Replaces text in a file in place.
- 34. **sort file.txt** – Sorts lines in a file.

- 35. **sort -r file.txt**– Sorts lines in reverse order.
- 36. **uniq file.txt**– Removes duplicate lines from a sorted file.
- 37. **wc -l file.txt** – Counts the number of lines in a file.
- 38. **wc -w file.txt** – Counts the number of words in a file.
- 39. **wc -c file.txt** – Counts the number of bytes in a file.
- 40. **head -n 10 file.txt**– Displays the first 10 lines of a file.
- 41. **tail -n 10 file.txt** – Displays the last 10 lines of a file.
- 42. **tail -f file.txt** – Continuously monitors a file for changes.
- 43. **df -h** – Shows disk space usage in a human-readable format.
- 44. **du -sh directory_name** – Shows the size of a directory.
- 45. **free -m** – Displays memory usage in megabytes.
- 46. **uptime** – Shows system uptime and load average.
- 47. **who** – Displays currently logged-in users.
- 48. **whoami** – Displays the current logged-in username.

- 49.**id** – Displays the user ID (UID) and group ID (GID).
- 50.**groups username** – Displays groups a user belongs to.
- 51.**ps aux** – Displays running processes.
- 52.**top** – Displays real-time process information.
- 53.**htop** – An interactive process viewer (if installed).
- 54.**kill -9 PID** – Forcefully terminates a process.
- 55.**pkill process_name** – Kills processes by name.
- 56.**killall process_name** – Kills all processes with a specific name.
- 57.**jobs** – Lists background jobs.
- 58.**bg** – Resumes a background job.
- 59.**fg** – Brings a background job to the foreground.
- 60.**nohup command &** – Runs a command in the background, ignoring hangups.
- 61.**crontab -e** – Edits the crontab file to schedule tasks.

62.**crontab -l** – Lists scheduled cron jobs.

63.**crontab -r** – Removes all scheduled cron jobs.

64.**history** – Displays command history.

65.**!100** – Runs command number 100 from history.

66.**chmod 755 file.sh** – Changes file permissions.

67.**chown user:group file.txt** – Changes file ownership.

68.**chgrp group_name file.txt** – Changes file group ownership.

69.**ls -l | grep "^d"** – Lists only directories.

70.**df -i** – Shows inode usage.

71.**du -a** – Shows size of all files and directories.

72.**tar -cvf archive.tar directory/** – Creates a tar archive.

73.**tar -xvf archive.tar** – Extracts a tar archive.

74.**tar -czvf archive.tar.gz directory/** – Creates a compressed tar archive.



75. **tar -xzvf archive.tar.gz** – Extracts a compressed tar archive.

76. **zip -r archive.zip directory/** – Compresses a directory into a zip file.

77. **unzip archive.zip** – Extracts a zip file.

78. **scp file.txt user@remote:/path/** – Securely copies a file to a remote server.

79. **scp -r directory user@remote:/path/** – Securely copies a directory to a remote server.

80. **rsync -av source/ destination/** – Synchronizes directories.

81. **wget URL** – Downloads a file from a URL.

82. **curl -O URL** – Downloads a file from a URL.

83. **curl -I URL** – Retrieves HTTP headers from a URL.

84. **ping google.com** – Checks network connectivity.

85. **traceroute google.com** – Traces network route to a server.

86. **netstat -tulnp** – Shows network connections and listening ports.



87. **ss -tulnp** – Displays active connections (alternative to netstat).

88. **ip a** – Shows IP addresses.

89. **ifconfig** – Displays network interfaces (deprecated).

90. **hostname** – Displays the system hostname.

91. **uptime** – Shows system uptime.

92. **uname -a** – Displays system information.

93. **lscpu** – Shows CPU details.

94. **lsblk** – Lists information about storage devices.

95. **blkid** – Shows UUIDs of partitions.

96. **mount /dev/sdb1 /mnt** – Mounts a device.

97. **umount /mnt** – Unmounts a device.

98. **df -Th** – Displays file system types and disk usage.

99. **fdisk -l** – Lists partition tables.

100. **mkfs.ext4 /dev/sdb1** – Formats a partition with ext4.



- 101.**fsck /dev/sdb1** – Checks a filesystem for errors.
- 102.**echo \$?** – Displays the exit status of the last command.
- 103.**time command** – Measures command execution time.
- 104.**date** – Displays the current date and time.
105. **cal** – Displays a calendar.
- 106.**env** – Displays environment variables.
- 107.**export VAR=value** – Sets an environment variable.
- 108.**unset VAR** – Unsets an environment variable.
- 109.**alias ll='ls -al'** – Creates a command alias.
- 110.**unalias ll** – Removes an alias.
- 111.**basename /path/to/file.txt** – Extracts the filename from a given path.
112. **dirname /path/to/file.txt** – Extracts the directory path from a given file path.
- 113.**diff file1.txt file2.txt** – Compares two files line by line.

114. **cmp file1.txt file2.txt** – Compares two files byte by byte.
115. **stat file.txt** – Displays detailed information about a file.
116. **file file.txt** – Determines the file type.
117. **cut -d':' -f1 /etc/passwd** – Extracts the first field from a colon-separated file.
118. **paste file1.txt file2.txt** – Merges two files line by line.
119. **tee file.txt** – Writes output to both a file and the standard output.
120. **yes "text"** – Continuously outputs "text" until interrupted.
121. **watch -n 5 df -h** – Runs a command every 5 seconds.
122. **lsattr** – Lists file attributes.
123. **chattr +i file.txt** – Makes a file immutable (cannot be modified or deleted).
124. **chattr -i file.txt** – Removes immutability from a file.
125. **nohup command &** – Runs a command in the background and ignores hangups.

- 126. **jobs** – Lists active background jobs.
- 127. **fg %1** – Brings job number 1 to the foreground.
- 128. **bg %1** – Resumes a background job.
- 129. **disown -h %1** – Removes a job from the shell's job table.
- 130. **xargs** – Passes standard input as command arguments.
- 131. **ls | xargs rm** – Deletes all files in a directory.
- 132. **echo "file1 file2" | xargs rm** – Deletes specified files.
- 133. **uptime -p** – Shows how long the system has been running.
- 134. **uptime -s** – Shows the system start time.
- 135. **who -b** – Displays the last system boot time.
- 136. **last reboot** – Shows the system reboot history.
- 137. **dmesg | tail** – Displays the latest kernel messages.
- 138. **dmesg | grep error** – Searches the kernel logs for errors.
- 139. **journalctl -xe** – Views system logs.

- 140. `journalctl -f` – Monitors logs in real time.
- 141. `systemctl status service_name` – Checks the status of a systemd service.
- 142. `systemctl start service_name` – Starts a systemd service.
- 143. `systemctl stop service_name` – Stops a systemd service.
- 144. `systemctl restart service_name` – Restarts a systemd service.
- 145. `systemctl enable service_name` – Enables a service to start on boot.
- 146. `systemctl disable service_name` – Disables a service from starting on boot.
- 147. `systemctl list-units --type=service` – Lists all active system services.
- 148. `systemctl daemon-reload` – Reloads systemd configuration files.
- 149. `service service_name status` – Checks the status of a SysV service.
- 150. `service service_name start` – Starts a SysV service.

151. **service service_name stop** – Stops a SysV service.

152. **chkconfig --list** – Lists services managed by SysV init.

153. **chkconfig service_name on** – Enables a service on boot using SysV.

154. **chkconfig service_name off** – Disables a service from boot using SysV.

155. **modprobe module_name** – Loads a kernel module.

156. **lsmod** – Lists currently loaded kernel modules.

157. **rmmod module_name** – Removes a kernel module.

158. **insmod module.ko** – Inserts a module into the kernel.

159. **uname -r** – Displays the currently running kernel version.

160. **cat /proc/version** – Shows kernel version details.

161. **hostnamectl** – Displays and modifies the hostname.

162. **nmcli device status** – Shows network interfaces and their statuses.

163. **nmcli connection show** – Lists saved network connections.



- 164. `nmcli connection up eth0` – Brings up a network interface.
- 165. `nmcli connection down eth0` – Brings down a network interface.
- 166. `dhclient -r` – Releases the DHCP lease.
- 167. `dhclient eth0` – Obtains a new DHCP lease.
- 168. `ip link set eth0 up` – Brings up an interface.
- 169. `ip link set eth0 down` – Brings down an interface.
- 170. `tcpdump -i eth0` – Captures network packets on an interface.
- 171. `tcpdump -nn port 80` – Captures HTTP traffic.
- 172. `tcpdump -c 10 -i eth0` – Captures 10 packets and exits.
- 173. `iptables -L` – Lists firewall rules.
- 174. `iptables -A INPUT -p tcp --dport 22 -j ACCEPT` –
Allows SSH access.
- 175. `iptables -A INPUT -p tcp --dport 80 -j DROP` – Blocks
HTTP access.
- 176. `iptables -D INPUT -p tcp --dport 80 -j DROP` –



Removes a firewall rule.

177. `firewall-cmd --list-all` – Lists active firewall rules (Firewalld).

178. `firewall-cmd --permanent --add-port=443/tcp` – Opens port 443 permanently.

179. `firewall-cmd --reload` – Reloads the firewall rules.

180. `ufw status` – Checks UFW firewall status.

181. `ufw allow 22/tcp` – Allows SSH access.

182. `ufw deny 80/tcp` – Blocks HTTP access.

183. `ufw delete allow 22/tcp` – Removes an allowed rule.

184. `df -T` – Displays filesystem type.

185. `ls -lh` – Lists files with human-readable sizes.

186. `du -ch` – Shows total disk usage in human-readable format.

187. `htop` – Interactive process monitoring.

188. `top -o %MEM` – Sorts processes by memory usage.

- 189. **lsof -i :80** – Lists processes using port 80.
- 190. **strace -c ls** – Traces system calls used by a command.
- 191. **strace -e open ls** – Shows file open system calls used by ls.
- 192. **tcpdump -XX** – Captures packets with hex and ASCII output.
- 193. **watch -d -n 5 free -m** – Monitors memory usage every 5 seconds.
- 194. **iostat** – Monitors disk I/O usage by processes.
- 195. **lsusb** – Lists USB devices.
- 196. **lspci** – Lists PCI devices.
- 197. **uptime -p** – Shows how long the system has been running in a human-friendly format.
- 198. **dmidecode -t memory** – Displays RAM information.
- 199. **mpstat 1** – Displays CPU usage statistics.
- 200. **iostat -c 2 5** – Shows CPU statistics every 2 seconds for 5 iterations.
- 201. **vmstat 1 5** – Displays system performance statistics every second

for 5 iterations.

202. **sar -u 5 3** – Reports CPU usage every 5 seconds for 3 iterations.

203. **sar -r 5 3** – Reports memory usage every 5 seconds for 3 iterations.

204. **uptime -s** – Displays system startup time.

205. **iostat -o** – Shows processes doing the most disk I/O.

206. **dstat** – Displays system resource usage dynamically.

207. **mpstat -P ALL 5** – Displays CPU usage for all cores every 5 seconds.

208. **nice -n 10 command** – Runs a command with lower priority.

209. **renice -n 10 -p PID** – Changes priority of an existing process.

210. **ulimit -a** – Shows system resource limits.

211. **ulimit -n 10240** – Changes the maximum number of open file descriptors.

212. **getfacl file.txt** – Displays ACL (Access Control List) permissions of a file.



213. **setfacl -m u:username:rwx file.txt** – Grants a user additional file permissions.
214. **setfacl -x u:username file.txt** – Removes ACL permissions for a user.
215. **getsebool -a** – Lists all SELinux booleans and their statuses.
216. **setsebool -P httpd_can_network_connect on** – Allows Apache to make network connections in SELinux.
217. **semanage fcontext -l** – Lists default SELinux file contexts.
218. **restorecon -Rv /var/www/html** – Restores SELinux context for files.
219. **getenforce** – Displays the current SELinux mode (Enforcing/Permissive/Disabled).
220. **setenforce 0** – Switches SELinux to permissive mode.
221. **auditctl -l** – Lists all active audit rules.
222. **ausearch -m avc** – Searches SELinux denial messages.
223. **ausearch -m USER_LOGIN** – Searches authentication logs using audit logs.

224. **modinfo module_name** – Displays information about a kernel module.

225. **modprobe -r module_name** – Unloads a kernel module.

226. **ls -Z** – Displays SELinux contexts of files.

227. **ps -eZ** – Displays SELinux contexts of processes.

228. **firewall-cmd --list-services** – Lists allowed services in Firewalld.

229. **firewall-cmd --permanent --add-service=https** – Allows HTTPS traffic permanently.

230. **firewall-cmd --permanent --remove-service=https** – Removes HTTPS access.

231. **firewall-cmd --reload** – Reloads Firewalld rules.

232. **ufw enable** – Enables UFW firewall.

233. **ufw disable** – Disables UFW firewall.

234. **ufw status numbered** – Displays UFW rules with numbering.

235. **ufw delete 2** – Deletes UFW rule number 2.



236. **iptables -P INPUT DROP** – Sets default INPUT policy to DROP.
237. **iptables -P FORWARD DROP** – Drops forwarded packets by default.
238. **iptables -P OUTPUT ACCEPT** – Allows all outgoing traffic by default.
239. **iptables -A INPUT -p tcp --dport 22 -j ACCEPT** – Allows SSH access.
240. **iptables-save > rules.v4** – Saves iptables rules to a file.
241. **iptables-restore < rules.v4** – Restores iptables rules from a file.
242. **lsuf -p PID** – Lists open files by a process.
243. **lsuf -i :443** – Displays processes using port 443.
244. **lsuf /path/to/file** – Shows processes accessing a specific file.
245. **strace -o trace.log -p PID** – Traces system calls of a running process.
246. **strace -c ls** – Summarizes system calls used by a command.
247. **tcpdump -i eth0** – Captures network packets on interface eth0.

248. `tcpdump -nn -s0 -X -i eth0 port 80` – Captures and displays raw HTTP traffic.
249. `nc -zv google.com 443` – Checks if port 443 is open on google.com.
250. `nc -lvp 1234` – Starts a netcat listener on port 1234.
251. `rsync -avz /src/ user@remote:/dest/` – Syncs files securely over SSH.
252. `rsync -a --delete /src/ /dest/` – Synchronizes directories and removes extra files.
253. `scp -P 2222 file.txt user@remote:/path/` – Transfers a file using a non-default SSH port.
254. `ssh user@remote -p 2222` – Connects to a server using a different SSH port.
255. `ssh-keygen -t rsa -b 4096 -C "your_email@example.com"` – Generates an SSH key.
256. `ssh-copy-id user@remote` – Copies the SSH key to a remote host.
257. `ssh-agent bash` – Starts an SSH agent session.

258. **eval "\$(ssh-agent -s)"** – Initializes the SSH agent.
259. **chmod 600 ~/.ssh/id_rsa** – Sets secure permissions on an SSH private key.
260. **chage -l username** – Displays password expiration details for a user.
261. **chage -M 90 username** – Sets the password to expire every 90 days.
262. **passwd username** – Changes a user's password.
263. **useradd -m -s /bin/bash newuser** – Creates a new user with a home directory.
264. **usermod -aG sudo username** – Adds a user to the sudo group.
265. **deluser username** – Removes a user.
266. **groupadd newgroup** – Creates a new group.
267. **usermod -G groupname username** – Adds a user to a group.
268. **groupdel groupname** – Deletes a group.
269. **crontab -e** – Opens the user's crontab for editing.

270. **crontab -l** – Lists scheduled cron jobs.

271. **crontab -r** – Removes all cron jobs for a user.

272. **echo "0 2 * * * /path/to/script.sh" | crontab - -**
Schedules a cron job to run a script at 2 AM daily.

273. **at now + 10 minutes** – Schedules a command to run in 10 minutes.

274. **at -l** – Lists pending scheduled jobs.

275. **systemctl list-timers** – Lists active systemd timers.

276. **timedatectl** – Displays system time settings.

277. **timedatectl set-timezone America/New_York** – Changes system timezone.

278. **hwclock --systohc** – Synchronizes hardware clock with system clock.

279. **date "+%Y-%m-%d %H:%M:%S"** – Displays date and time in a specific format.

280. **find /var/log -type f -mtime +30 -delete** – Deletes log files older than 30 days.



281. **journalctl --vacuum-time=30d** – Removes journal logs older than 30 days.
282. **du -ah /var/log | sort -rh | head -10** – Lists the 10 largest log files.
283. **logrotate -d /etc/logrotate.conf** – Tests log rotation configuration.
284. **fsck -y /dev/sda1** – Checks and repairs a filesystem.
285. **tune2fs -m 5 /dev/sda1** – Reserves 5% of space for root user.
286. **blkid** – Lists partitions and their UUIDs.
287. **mount -o remount,rw /** – Remounts the root filesystem as read/write.
288. **mkfs.ext4 /dev/sdb1** – Formats a partition with ext4.
289. **tune2fs -c 100 /dev/sda1** – Forces a filesystem check every 100 mounts.
290. **swapoff -a && swapon -a** – Restarts the swap space.
291. **free -h** – Displays RAM and swap usage in human-readable format.
292. **grep -i error /var/log/syslog** – Searches syslog for errors.

293.**dmidecode -t memory** – Displays memory module details.

294.**systemctl poweroff** – Shuts down the system.

295.**systemctl reboot** – Reboots the system.

296.**shutdown -h now** – Immediately shuts down the system.

297.**shutdown -r +10** – Reboots the system in 10 minutes.

298. **wall "System maintenance in 5 minutes"** – Broadcasts a message to all users.

299.**uptime** – Displays system uptime and load average.

300. **exit** – Logs out of the shell session.



By DevOps Shack

100 LINUX ERRORS & SOLUTION



DEVOPS SHACK

100 Linux Errors & solution

1. Permission Denied

RCA: The user does not have the necessary permissions to execute or modify the file.

Solution: Use sudo or change file permissions.

Command:

```
sudo chmod +x script.sh
```

2. File Not Found (No such file or directory)

RCA: The specified file does not exist, or the path is incorrect.

Solution: Verify the correct path and check if the file exists.

Command:

```
ls -l /path/to/file
```

3. Directory Not Found

RCA: The specified directory does not exist or was deleted.

Solution: Create the missing directory.

Command:

```
mkdir -p /desired/paths
```

4. Read-Only File System

RCA: The filesystem is mounted as read-only due to disk corruption or improper shutdown.

Solution: Remount the filesystem in read/write mode.

Command:

```
mount -o remount,rw /
```



5. Disk Space Full (No space left on device)

RCA: The disk is full, preventing new files from being created.

Solution: Delete unnecessary files or expand disk space.

Command:

```
df -h
```

```
rm -rf /var/log/*
```

6. Symbolic Link Points to Non-Existent File (broken symlink)

RCA: The original file of the symlink was moved or deleted.

Solution: Find and recreate the symlink.

Command:

```
ls -l symlink_name
```

```
ln -sf /new/target/path symlink_name
```

7. Too Many Open Files

RCA: The system has reached its file descriptor limit.

Solution: Increase the open file limit.

Command:

```
Ulimit
```

```
-n 100000
```

8. Text file busy Error

RCA: The file is in use by another process while trying to modify or delete it.

Solution: Identify and stop the process using it.

Command:

```
lsof | grep filename
```

```
kill -9 <PID>
```



9. Stale NFS file handle

RCA: The NFS-mounted file was deleted or moved on the remote system.

Solution: Remount the NFS share.

Command:

```
umount -l /mnt/nfs
```

```
mount -a
```

10. Argument list too long

RCA: Too many arguments passed to a command, exceeding system limits.

Solution: Use find or xargs instead of rm *.

Command:

```
find /path -type f -delete
```

11. Process Killed Automatically

RCA: The Out-of-Memory (OOM) killer terminated the process.

Solution: Increase swap memory or optimize memory usage.

Command:

```
free -m
```

```
dmesg | grep -i 'oom'
```

12. Zombie Process Remains After Termination

RCA: The parent process has not cleaned up the child process.

Solution: Restart the parent process or kill the zombie process.

Command:

```
ps aux | grep Z
```

```
kill -9 <PID>
```

13. Segmentation Fault (core dumped)

RCA: A program attempted to access restricted memory.

Solution: Debug using gdb.

Command:

```
gdb ./program core
```

14. Too Many Processes Running (Fork Bomb)

RCA: A process is recursively creating child processes.

Solution: Use ulimit to prevent excessive process creation.

Command:

```
ulimit -u 5000
```

15. Device is Busy When Unmounting a Partition

RCA: A process is using the partition.

Solution: Find and kill the process before unmounting.

Command:

```
ls -l +D /mnt/partition
```

```
kill -9 <PID>
```

```
umount /mnt/partition
```

21. Network Unreachable

RCA: Incorrect network configuration or firewall rules.

Solution: Check network settings and routes.

Command:

```
ip a
```

```
route -n
```

22. Connection Timed Out

RCA: The remote server is not responding.

Solution: Use ping or telnet to diagnose.

Command:

```
ping -c 4 google.com
```

```
telnet google.com 80
```

23. SSH Connection Refused

RCA: SSH service is not running or firewall is blocking.

Solution: Start SSH and allow the port.

Command:

```
systemctl start ssh
```

```
ufw allow 22/tcp
```

24. Unknown Host

RCA: DNS resolution failed.

Solution: Check /etc/resolv.conf or use nslookup.

Command:

```
nslookup google.com
```

25. Port Already in Use

RCA: Another process is using the port.

Solution: Find and kill the process using the port.

Command:

```
netstat -tulnp | grep :80
```

```
kill -9 <PID>
```

26. User is Not in the sudoers File

RCA: The user does not have sudo privileges.

Solution: Add the user to the sudo group.

Command:

```
sudo usermod -aG sudo username
```

27. SSH Key Authentication Fails

RCA: Incorrect key permissions or missing public key in authorized_keys.

Solution: Ensure correct permissions and key placement.

Command:

```
chmod 600 ~/.ssh/authorized_keys
```

28. Account Locked Due to Too Many Failed Logins

RCA: Security settings locked the account after multiple incorrect attempts.

Solution: Unlock the user account.

Command:

```
faillock --reset --user username
```

29. Too Many Authentication Failures (SSH)

RCA: Exceeded max authentication attempts in SSH.

Solution: Increase the limit in sshd_config.

Command:

```
sudo nano /etc/ssh/sshd_config
```

```
MaxAuthTries 6
```

```
systemctl restart sshd
```

30. Unable to Change User Password

RCA: Password complexity requirements or expired account.

Solution: Reset the password and check account status.

Command:

```
sudo passwd username
```

```
sudo chage -l username
```

31. Kernel Panic – Not Syncing

RCA: The kernel cannot mount the root filesystem due to corruption or misconfiguration.

Solution: Boot into recovery mode and reinstall the kernel.

Command:

```
sudo apt update && sudo apt upgrade -y
```

```
sudo update-initramfs -u
```

32. Boot Stuck at GRUB> Prompt

RCA: GRUB bootloader is corrupted or missing.

Solution: Reinstall GRUB from a live CD.

Command:

```
grub-install /dev/sda
```

```
update-grub
```

33. initramfs Error at Boot

RCA: The system is unable to load the root filesystem.

Solution: Check the filesystem for corruption.

Command:

```
fsck -y /dev/sda1
```

34. No init found Error

RCA: The system init process is missing or corrupted.

Solution: Reinstall the init package.

Command:

```
sudo apt install --reinstall system
```

35. Slow Boot Time

RCA: Background services slowing down the boot.

Solution: Identify and disable unnecessary services.

Command:

```
systemd-analyze blame
```

```
systemctl disable unwanted-service
```

36. Boot Stuck at GRUB> Prompt

RCA: The GRUB bootloader is corrupted or missing.

Solution: Reinstall GRUB from a live CD.

Command:

```
grub-install /dev/sda
```

```
update-grub
```

37. initramfs Error at Boot

RCA: The system cannot load the root filesystem.

Solution: Boot into recovery mode and check filesystem integrity.

Command:

```
fsck -y /dev/sda1
```

38. No init found Error

RCA: The system init process is missing or corrupted.

Solution: Reinstall the init package.

Command:

```
sudo apt install --reinstall system
```

39. Kernel Module Failed to Load

RCA: The kernel module is missing or incompatible with the running kernel.

Solution: Manually load the module or install the required kernel headers.

Command:

```
modprobe module-name
```

40. modprobe: FATAL: Module Not Found

RCA: The module is missing from the `/lib/modules/` directory.

Solution: Update the kernel module list.

Command:

```
sudo depmod -a
```

41. High CPU Usage (top shows 100% CPU usage)

RCA: A process is consuming too much CPU.

Solution: Identify and stop the process.

Command:

```
top
```

```
kill -9 <PID>
```

42. Out of Memory (OOM Killer triggered)

RCA: The system ran out of available memory.

Solution: Add swap space or optimize memory usage.

Command:

```
sudo fallocate -l 2G /swapfile
```

```
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
```

43. System Running Slowly

RCA: High system load due to excessive background processes.

Solution: Check system load and stop unnecessary services.

Command:

```
uptime
```

```
systemctl stop unused-service
```

44. Disk I/O Bottleneck (iowait is high in top)

RCA: High disk usage causing slow performance.

Solution: Identify and stop I/O-intensive processes.

Command:

```
iostat -x 1 10
```

45. Too Many Open Files Error

RCA: The system has reached its file descriptor limit.

Solution: Increase the open file limit.

Command:

```
ulimit -n 100000
```

46. Frequent System Freezing

RCA: Resource exhaustion due to high CPU, memory, or disk usage.

Solution: Monitor and kill resource-hungry processes.

Command:

```
htop
```

```
kill -9 <PID>
```

47. High Disk Usage (df -h Shows 100% Used)

RCA: Large log files or unnecessary files filling up the disk.

Solution: Delete large unwanted files and free up space.

Command:

```
du -sh /var/log/*
```

```
rm -rf /var/log/old_logs
```

48. Slow File Read/Write Performance

RCA: Fragmented disk or faulty disk sectors.

Solution: Check the disk health and optimize filesystem performance.

Command:

```
sudo smartctl -a /dev/sda
```

49. System Clock is Incorrect

RCA: Time is not syncing properly due to NTP issues.

Solution: Restart NTP service and sync time.

Command:

```
sudo systemctl restart systemd-timesyncd
```

50. dmesg Shows Hardware Errors

RCA: Faulty hardware component (RAM, CPU, disk).

Solution: Run a hardware diagnostic test.

Command:

```
sudo dmesg | grep -i error
```

51. Wi-Fi Not Connecting

RCA: The network interface is down or the driver is missing.

Solution: Restart the network service and check drivers.

Command:

```
sudo systemctl restart NetworkManager
```

```
nmcli device wifi rescan
```

52. Unable to Obtain IP Address (DHCP Issue)

RCA: The DHCP client is not running or the router is not assigning an IP.

Solution: Restart DHCP and network services.

Command:

```
sudo dhclient -r && sudo dhclient
```

53. Internet Works but Local Network is Unreachable

RCA: The local network gateway is missing from the routing table.

Solution: Add a proper default gateway.

Command:

```
ip route add default via 192.168.1.1
```

54. Duplicate IP Address on Network

RCA: Another device is using the same IP.

Solution: Set a static IP or release/renew DHCP lease.

Command:

```
sudo dhclient -r && sudo dhclient eth0
```

55. VPN Connection Drops Frequently

RCA: Firewall rules or incorrect MTU settings.

Solution: Adjust MTU and check firewall.

Command:

```
sudo ip link set dev tun0 mtu 1400
```

56. Firewall Blocks Outgoing Traffic

RCA: UFW or iptables is too restrictive.

Solution: Allow outbound connections.

Command:

```
sudo ufw allow out on eth0
```

57. DNS Queries Failing Randomly

RCA: DNS cache corruption or ISP issues.

Solution: Clear cache and change DNS.

Command:

```
sudo systemctl restart systemd-resolved
```

58. High Network Latency

RCA: Network congestion or poor routing.

Solution: Identify slow hops using traceroute.

Command:

```
traceroute google.com
```



59. Packet Loss on Wired Connection

RCA: Faulty cable or network interference.

Solution: Check Ethernet cables and NIC status.

Command:

```
ethtool eth0
```

60. Port Forwarding Not Working

RCA: NAT or firewall rules preventing traffic.

Solution: Check router settings and firewall rules.

Command:

```
sudo iptables -t nat -L -n -v
```

61. SSH Public Key Authentication Not Working

RCA: Incorrect file permissions.

Solution: Fix permissions.

Command:

```
chmod 700 ~/.ssh
```

```
chmod 600 ~/.ssh/authorized_keys
```

62. User Cannot Write to /tmp Directory

RCA: Sticky bit or wrong permissions set.

Solution: Fix permissions.

Command:

```
sudo chmod 1777 /tmp
```

63. Password Authentication Failing for Local Users

RCA: PAM misconfiguration.

Solution: Restore PAM defaults.

Command:



```
sudo pam-auth-update --force
```

64. Cannot Unlock Encrypted Drive (LUKS)

RCA: Corrupt encryption headers.

Solution: Try backup headers.

Command:

```
cryptsetup luksHeaderRestore --header-backup-file /backup.img /dev/sdX
```

65. User's Home Directory Not Created Automatically

RCA: useradd command used without -m.

Solution: Create the directory manually.

Command:

```
sudo mkdir /home/username
```

```
sudo chown username:username /home/username
```

66. SSH Sessions Disconnecting Frequently

RCA: Keep-alive packets not enabled.

Solution: Enable keep-alive packets.

Command:

```
echo "ClientAliveInterval 60" | sudo tee -a /etc/ssh/sshd_config
```

67. sudo Command Takes a Long Time to Execute

RCA: Incorrect hostname resolution.

Solution: Fix /etc/hosts.

Command:

```
echo "127.0.0.1 localhost" | sudo tee -a /etc/hosts
```



68. Cannot Change User Shell

RCA: Shell is not listed in /etc/shells.

Solution: Add shell to /etc/shells.

Command:

```
echo "/bin/zsh" | sudo tee -a /etc/shells
```

```
chsh -s /bin/zsh
```

69. User Processes Exceed Limit (ulimit error)

RCA: Process limits exceeded.

Solution: Increase process limits.

Command:

```
ulimit -u 100000
```

70. Cannot Add User to Multiple Groups

RCA: Wrong syntax used in usermod.

Solution: Use correct usermod command.

Command:

```
sudo usermod -aG group1,group2 username
```

71. APT Repository Not Found

RCA: Incorrect repo URL.

Solution: Update sources list.

Command:

```
sudo nano /etc/apt/sources.list
```

```
sudo apt update
```

72. Software Crashes on Launch

RCA: Missing shared libraries.

Solution: Find and install missing dependencies.

Command:

```
ldd /usr/bin/software
```

73. Package Installation Hangs

RCA: Lock file stuck.

Solution: Remove lock files and retry.

Command:

```
sudo rm /var/lib/dpkg/lock
```

74. Compiled Binary Segfaults Immediately

RCA: Missing execution permissions or corrupt binary.

Solution: Recompile with debug flags.

Command:

```
gcc -g -o program program.c
```

75. PPA Removed but Packages Remain

RCA: Old packages still installed.

Solution: Remove manually.

Command:

```
sudo apt autoremove
```

76. Java Not Found Despite Installation

RCA: Incorrect PATH.

Solution: Set the correct JAVA_HOME.

Command:

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk
```

77. Kernel Headers Missing

RCA: Headers not installed.

Solution: Install matching kernel headers.

Command:

```
sudo apt install linux-headers-$(uname -r)
```

78. Snapd Not Running

RCA: Snap service is disabled.

Solution: Enable and start snapd.

Command:

```
sudo systemctl enable --now snapd
```

79. Package Dependency Issues

RCA: Missing dependencies.

Solution: Fix broken dependencies.

Command:

```
sudo apt --fix-broken install
```

80. Python Module Not Found Despite Installation

RCA: Module installed in wrong environment.

Solution: Install in the correct environment.

Command:

```
python3 -m pip install module-name
```

81. System Clock is Drifting

RCA: NTP service is not running or misconfigured.

Solution: Restart and sync time using NTP.

Command:

```
sudo systemctl restart systemd-timesyncd
```

```
sudo timedatectl set-ntp true
```

82. Laptop Battery Drains Fast in Linux

RCA: Power management settings are not optimized.

Solution: Enable power-saving mode.

Command:

```
sudo tlp start
```

83. Monitor Resolution Not Detected Correctly

RCA: Graphics driver issue or incorrect display settings.

Solution: Manually set resolution using xrandr.

Command:

```
xrandr --output HDMI-1 --mode 1920x1080
```

84. USB Device Not Recognized

RCA: The kernel does not detect the USB device.

Solution: Reload USB drivers.

Command:

```
sudo modprobe -r usb_storage
```

```
sudo modprobe usb_storage
```

85. Bluetooth Not Working

RCA: Bluetooth service is disabled or the adapter is not detected.

Solution: Enable Bluetooth and restart services.

Command:

```
sudo systemctl restart Bluetooth
```

86. Unable to Mount External Hard Drive

RCA: Unsupported file system or permission issues.

Solution: Check the filesystem type and mount with correct options.

Command:

```
sudo mount -t ntfs-3g /dev/sdb1 /mnt
```

87. Kernel Logs Flooded with Errors

RCA: A misbehaving driver or hardware failure.

Solution: Check logs and identify the issue.

Command:

```
dmesg | grep -i error
```

88. Keyring Keeps Asking for Password on Login

RCA: Keyring is not unlocked automatically.

Solution: Remove password protection from the keyring.

Command:

```
rm ~/.local/share/keyrings/*.keyring
```

89. System Boot Takes Too Long

RCA: A service is delaying startup.

Solution: Identify and disable unnecessary services.

Command:

```
systemd-analyze blame
```



90. Persistent Journal Logs Consuming Too Much Space

RCA: Journald logs are not rotating properly.

Solution: Limit the journal log size.

Command:

```
sudo journalctl --vacuum-size=500M
```

91. Unauthorized Root Login Attempts in Logs

RCA: A bot is attempting brute-force login.

Solution: Block IPs using fail2ban.

Command:

```
sudo apt install fail2ban
```

```
sudo systemctl enable --now fail2ban
```

92. System Security Hardening Required

RCA: Default Linux configurations may be insecure.

Solution: Use a security-hardening tool.

Command:

```
sudo apt install lynis
```

```
sudo lynis audit system
```

93. SELinux Preventing Software from Running

RCA: SELinux is blocking access.

Solution: Temporarily disable SELinux (for debugging only).

Command:

```
sudo setenforce 0
```

94. AppArmor Blocking Application Execution

RCA: AppArmor profile is too restrictive.

Solution: Set AppArmor profile to complain mode.

Command:

```
sudo aa-complain /usr/bin/application
```

95. SSH Brute Force Attacks Detected

RCA: Multiple failed SSH login attempts.

Solution: Restrict SSH to key-based authentication only.

Command:

```
sudo nano /etc/ssh/sshd_config
```

```
PasswordAuthentication no
```

```
sudo systemctl restart sshd
```

96. System Crashes Due to Corrupt Swap File

RCA: Swap file corruption causes kernel crashes.

Solution: Recreate the swap file.

Command:

```
sudo swapoff -a
```

```
sudo rm -f /swapfile
```

```
sudo fallocate -l 2G /swapfile
```

```
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
```

97. Files Deleted but Space Not Freed

RCA: A deleted file is still held open by a process.

Solution: Find and close the process using the file.

Command:

```
lsof | grep deleted
```



98. RAID Array Degraded or Failed

RCA: A disk in the RAID array has failed.

Solution: Identify and replace the failed disk.

Command:

```
cat /proc/mdstat
```

99. NVMe SSD Performance Degradation

RCA: TRIM is not enabled, leading to slower writes.

Solution: Enable TRIM.

Command:

```
sudo fstrim -av
```

100. System Freezes Randomly with No Logs

RCA: A hardware issue such as overheating, bad RAM, or power supply problems.

Solution: Run hardware diagnostics.

Command:

```
sudo memtester 2048M 5
```

DevOps Shack

250 Linux Scenario Based Interview Questions and Answers

Q1: How would you find all files larger than 100MB in a directory and its subdirectories?

Answer: Use the **find** command:

```
find /path/to/directory -type f -size +100M
```

- **-type f**: Finds only files.
- **-size +100M**: Specifies files larger than 100MB.

Q2: What does the command **chmod 755 file** do?

- Answer:
 - Changes the file's permissions to **755**.
 - **7**: Full permission for the owner (read, write, execute).
 - **5**: Read and execute for group and others.

Syntax breakdown:

r = 4, w = 2, x = 1 → rwx (7), r-x (5)

Q3: How do you recursively delete all **.log** files in a directory?

Answer:



```
find /path/to/directory -name "*.log" -type f -delete
```

- **-name "*.log"**: Matches files with **.log** extension.
- **-type f**: Targets only files.
- **-delete**: Deletes the files found.

Q4: How do you create a tarball of `/var/log` and compress it with gzip?

Answer:

```
tar -czvf logs.tar.gz /var/log
```

- **-c**: Creates a tarball.
- **-z**: Compresses using gzip.
- **-v**: Verbose output.
- **-f logs.tar.gz**: Specifies the tarball file name.

Q5: How do you troubleshoot a network issue where a server is unreachable?

- **Answer:**

Check network connectivity:

```
ping <server_ip>
```

Verify DNS resolution:

```
nslookup <server_name>
```

Check routes:

```
ip route
```



Verify firewall settings:

```
sudo iptables -L
```

Test specific port availability:

```
nc -zv <server_ip> <port>
```

Q6: How do you permanently assign an IP address in Linux?

- Answer: Edit the network configuration file:

For RHEL/CentOS:

```
vi /etc/sysconfig/network-scripts/ifcfg-eth0
```

Add or update:

```
IPADDR=192.168.1.100  
NETMASK=255.255.255.0  
GATEWAY=192.168.1.1
```

Restart the network service:

```
sudo systemctl restart network
```

Q7: How do you identify a process consuming high CPU?

Answer:

- Sort by CPU usage by pressing **Shift + P**.



Alternatively, use:

```
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head
```

Q8: How do you monitor disk I/O in real-time?

Answer:

```
iostat -x 2
```

- **-x**: Shows extended statistics.
- **2**: Refreshes every 2 seconds.

Q9: How do you kill a process by its name?

Answer:

```
pkill <process_name>
```

To verify:

```
pgrep <process_name>
```

Q10: How do you list all services and their statuses?

Answer:

```
systemctl list-units --type=service
```

Q11: How do you set up a basic firewall rule to allow SSH traffic?

Answer:

```
sudo ufw allow ssh
```



Check the status:

```
sudo ufw status
```

Q12: How do you check for unauthorized access attempts on your server?

Answer:

```
sudo cat /var/log/auth.log | grep "Failed password"
```

Q13: Write a script to check if a file exists and display its permissions.

Answer:

```
#!/bin/  
FILE=$1  
if [ -e "$FILE" ]; then  
    echo "File exists."  
    ls -l "$FILE" | awk '{print $1}'  
else  
    echo "File does not exist."  
fi
```

- Usage: `./script.sh filename`

Q14: How would you write a script to archive logs older than 7 days?

Answer:

```
#!/bin/  
find /var/log -type f -mtime +7 -exec tar -rvf old_logs.tar  
{ } \; -exec rm { } \;
```



- **-mtime +7**: Finds files older than 7 days.
- **-exec**: Executes **tar** to archive and **rm** to delete.

Q15: How do you add a new user with a specific home directory?

Answer:

```
sudo useradd -m -d /custom/home/username username
```

- **-m**: Creates the home directory.
- **-d**: Specifies the custom directory.

Q16: How do you change the default shell for a user?

Answer:

```
sudo usermod -s /bin/ username
```

Verify with:

```
cat /etc/passwd | grep username
```

Q17: What steps would you take to troubleshoot a slow server?

- **Answer:**

Check CPU usage:

Top

Analyze disk usage:

```
df -h
```



Inspect running processes:

```
ps aux --sort=-%cpu | head
```

Check I/O operations:

```
iostat -x 2
```

Verify memory usage:

```
free -m
```

Q18: How do you resolve a "permission denied" error for a script?

- Answer:

Check file permissions:

```
ls -l script.sh
```

Add execute permission:

```
chmod +x script.sh
```

Q19: How do you extend a mounted LVM partition?

- Answer:

Increase the logical volume:

```
sudo lvextend -L +10G /dev/mapper/vol_group-lv_name
```



Resize the filesystem:

```
sudo resize2fs /dev/mapper/vol_group-lv_name
```

Q20: How do you find the top 10 largest files in a directory?

Answer:

```
find /path/to/dir -type f -exec du -h {} + | sort -rh | head  
-n 10
```

Backup and Restore

Q21: How do you back up a directory using **rsync**?

Answer:

```
rsync -av /source/directory /destination/directory
```

- **-a**: Archive mode.
- **-v**: Verbose output.

Q22: How do you restore files from a tar backup?

Answer:

```
tar -xvf backup.tar -C /restore/path
```

Q23: How do you rotate logs manually?

- Answer:

Use **logrotate**:

```
sudo logrotate /etc/logrotate.conf --force
```

Q24: How do you view the last 50 lines of a log file?



Answer:

```
tail -n 50 /var/log/syslog
```

Q25: How do you check which kernel modules are currently loaded?

Answer:

```
lsmod
```

Q26: How do you load a kernel module manually?

Answer:

```
sudo modprobe <module_name>
```

Q27: How do you lock a user account?

Answer:

```
sudo passwd -l username
```

Q28: How do you check the groups a user belongs to?

Answer:

```
groups username
```

Q29: How do you check the installed Linux distribution?

Answer:

```
lsb_release -a
```

Or:

```
cat /etc/os-release
```

Q30: How do you list installed packages in RHEL/CentOS?

Answer:

```
yum list installed
```

Q31: How do you list all running virtual machines in KVM?

Answer:

```
virsh list
```

Q32: How do you create a virtual machine in VirtualBox from the command line?

Answer:

```
VBoxManage createvm --name VMName --register
```

Q33: How do you check running Docker containers?

Answer:

```
docker ps
```

Q34: How do you restart a Kubernetes pod?

Answer:

```
kubectl rollout restart deployment <deployment_name>
```

Q35: How do you troubleshoot if a Linux system fails to boot?

- Answer:
 1. Boot into recovery mode or single-user mode.
 2. Check logs in `/var/log/boot.log` or `/var/log/messages`.

Verify the GRUB configuration:

```
cat /boot/grub2/grub.cfg
```

Check disk integrity:



```
fsck /dev/sdX
```

Reinstall GRUB if required:

```
grub2-install /dev/sdX
```

Q36: What is the role of the **init** system in Linux?

- Answer:
 - The **init** system is the first process started by the kernel after booting.
 - It initializes the system by starting necessary services and managing runlevels (or targets in **systemd**).

Q37: How do you check the open ports on a Linux server?

Answer:

```
sudo netstat -tuln
```

Or using **ss**:

```
ss -tuln
```

Q38: How do you add a persistent static route in Linux?

- Answer:
 - Edit the network configuration file:

RHEL/CentOS:

```
vi /etc/sysconfig/network-scripts/route-eth0
```

Add:

```
192.168.2.0/24 via 192.168.1.1 dev eth0
```

Restart the network service:

```
sudo systemctl restart network
```



Q39: How do you list all installed packages on an Ubuntu system?

Answer:

```
dpkg --get-selections
```

Q40: How do you remove a package along with its configuration files in Debian/Ubuntu?

Answer:

```
sudo apt-get purge <package_name>
```

Q41: How do you check for disk usage by each directory in the current path?

Answer:

```
du -sh *
```

Q42: How do you create and mount an ext4 filesystem?

- **Answer:**

Create the filesystem:

```
mkfs.ext4 /dev/sdX
```

Mount the filesystem:

```
mount /dev/sdX /mnt
```

Q43: How do you find out which process is using the most memory?

Answer:

```
ps aux --sort=-%mem | head
```

Q44: How do you monitor live network traffic on an interface?

Answer:



```
sudo tcpdump -i eth0
```

Q45: How do you change the priority of a running process?

- Answer:

Use the **renice** command:

```
sudo renice -n 10 -p <PID>
```

- **10**: New priority value.

Q46: How do you restart a failed systemd service?

Answer:

```
sudo systemctl restart <service_name>
```

Q47: How do you generate an SSH key pair?

Answer:

```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

- **-t rsa**: Specifies the key type.
- **-b 4096**: Sets the key length.

Q48: How do you prevent a user from logging in via SSH?

- Answer:

Add the user to **/etc/ssh/sshd_config**:

```
DenyUsers username
```

Restart the SSH service:

```
sudo systemctl restart sshd
```



Q49: How do you find specific error messages in a log file?

Answer:

```
grep "ERROR" /var/log/syslog
```

Q50: How do you monitor a log file in real-time?

Answer:

```
tail -f /var/log/syslog
```

Q51: How do you check the current Linux kernel version?

Answer:

```
uname -r
```

Q52: How do you remove a kernel module?

Answer:

```
sudo rmmod <module_name>
```

Q53: How do you create a group and add a user to it?

- Answer:

Create the group:

```
sudo groupadd groupname
```

Add the user:

```
sudo usermod -aG groupname username
```

Q54: How do you delete a user and their home directory?

Answer:

```
sudo userdel -r username
```



Q55: Write a script to print the number of files in a directory.

Answer:

```
#!/bin/
DIR=$1
if [ -d "$DIR" ]; then
    echo "Number of files in $DIR: $(ls -1 $DIR | wc -l)"
else
    echo "$DIR is not a directory."
fi
```

Q56: How do you test if a specific port on a remote server is open?

Answer:

```
nc -zv <server_ip> <port>
```

- **-Z**: Scans without sending data.
- **-v**: Enables verbose output.

Q57: How do you flush the DNS cache in Linux?

- Answer:

For **systemd-resolved**:

```
sudo systemd-resolve --flush-caches
```

Verify:

```
sudo systemd-resolve --statistics
```

Q58: How do you check which services are enabled to start at boot?

Answer:

```
systemctl list-unit-files --type=service | grep enabled
```



Q59: How do you stop a service and ensure it doesn't start at boot?

Answer:

```
sudo systemctl stop <service_name>
sudo systemctl disable <service_name>
```

Q60: How do you check memory usage in Linux?

Answer:

```
free -m
```

- Or using **top** or **htop** for a real-time view.

Q61: How do you identify zombie processes?

Answer:

```
ps aux | grep 'Z'
```

- A zombie process will have a status of **Z**.

Q62: Write a script to monitor disk usage and send an email alert if usage exceeds 80%.

Answer:

```
#!/bin/
THRESHOLD=80
USAGE=$(df -h / | grep -v Filesystem | awk '{print $5}' |
sed 's/%//')
if [ "$USAGE" -gt "$THRESHOLD" ]; then
    echo "Disk usage is at ${USAGE}% on $(hostname)" | mail
-s "Disk Usage Alert" user@example.com
fi
```

Q63: Write a script to check if a service is running and restart it if it's not.



Answer:

```
#!/bin/
SERVICE=nginx
if ! systemctl is-active --quiet $SERVICE; then
    systemctl restart $SERVICE
    echo "$SERVICE was down and has been restarted."
else
    echo "$SERVICE is running."
fi
```

Q64: How do you unmount a busy filesystem?

- Answer:

Identify processes using the filesystem:

```
lsof | grep /path/to/mount
```

Kill those processes:

```
kill -9 <PID>
```

Unmount the filesystem:

```
sudo umount /path/to/mount
```

Q65: How do you create a swap file?

- Answer:

Create the file:

```
sudo dd if=/dev/zero of=/swapfile bs=1G count=2
```



Set permissions:

```
sudo chmod 600 /swapfile
```

Set up the swap space:

```
sudo mkswap /swapfile  
sudo swapon /swapfile
```

Make it persistent:

```
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Q66: How do you check for failed login attempts?

Answer:

```
sudo grep "Failed password" /var/log/auth.log
```

Q67: How do you restrict a user to their home directory using SSH?

- Answer:

Edit `/etc/ssh/sshd_config`:

```
Match User username  
    ChrootDirectory /home/username  
    AllowTCPForwarding no  
    X11Forwarding no
```

Restart the SSH service:

```
sudo systemctl restart sshd
```



Q68: How do you take a snapshot of an LVM volume?

- Answer:

Create a snapshot:

```
sudo lvcreate --size 1G --snapshot --name snap_name  
/dev/vol_group/lv_name
```

Mount the snapshot (optional):

```
sudo mount /dev/vol_group/snap_name /mnt
```

Q69: How do you schedule a daily backup with **cron**?

- Answer:

Edit the cron jobs:

```
crontab -e
```

Add the job:

```
0 2 * * * tar -czf /backup/${date +%F}.tar.gz /data
```

Q70: How do you compress log files older than 30 days?

Answer:

```
find /var/log -type f -mtime +30 -exec gzip {} \;
```

Q71: How do you rotate logs for a custom application?

- Answer:



Create a logrotate configuration file:

```
sudo vi /etc/logrotate.d/custom_app
```

Add the configuration:

```
/var/log/custom_app/*.log {  
    daily  
    rotate 7  
    compress  
    missingok  
    notifempty  
}
```

Test the configuration:

```
sudo logrotate -d /etc/logrotate.d/custom_app
```

Q72: How do you list all available kernel versions?

Answer:

```
dpkg --get-architecture | grep linux-image
```

Q73: How do you update the Linux kernel?

- Answer:

For Debian/Ubuntu:

```
sudo apt-get update  
sudo apt-get upgrade linux-image-$(uname -r)
```

Q74: How do you update all installed packages on a CentOS system?



Answer:

```
sudo yum update
```

Q75: How do you check dependencies for a package in Ubuntu?

Answer:

```
apt-cache depends <package_name>
```

Q76: How do you configure a static IP address on a Linux server?

- Answer:
 - For RHEL/CentOS:

Edit the network configuration file:

```
sudo vi /etc/sysconfig/network-scripts/ifcfg-eth0
```

Add or update:

```
makefile
```

```
BOOTPROTO=static  
IPADDR=192.168.1.100  
NETMASK=255.255.255.0  
GATEWAY=192.168.1.1  
DNS1=8.8.8.8
```

Restart the network:

```
sudo systemctl restart network
```

Q77: How do you check the default gateway on a Linux system?

Answer:



```
ip route | grep default
```

Q78: How do you find the parent process of a given process?

Answer:

```
ps -o ppid= -p <PID>
```

Q79: How do you kill all processes owned by a specific user?

Answer:

```
sudo pkill -u <username>
```

Q80: How do you generate a CPU usage report over time?

Answer:

```
sar -u 1 10
```

- **-u:** CPU usage.
- **1 10:** Sample every 1 second for 10 intervals.

Q81: How do you monitor real-time I/O usage per process?

Answer:

```
iotop
```

Q82: Write a script to display the top 5 largest files in a directory.

Answer:

```
#!/bin/  
du -ah $1 | sort -rh | head -n 5
```

Q83: Write a script to automate user creation and password assignment.

Answer:

```
#!/bin/
USER=$1
PASS=$2
sudo useradd -m $USER
echo "$USER:$PASS" | sudo chpasswd
echo "User $USER created with the given password."
```

Q84: How do you repair a corrupted filesystem?

- Answer:

Unmount the filesystem:

```
sudo umount /dev/sdX
```

Run **fsck**:

```
sudo fsck -y /dev/sdX
```

Q85: How do you check the inode usage on a filesystem?

Answer:

```
df -i
```

Q86: How do you disable root login over SSH?

- Answer:

Edit the SSH configuration file:

```
sudo vi /etc/ssh/sshd_config
```

Update or add:



perl

PermitRootLogin no

Restart the SSH service:

sudo systemctl restart sshd

Q87: How do you configure a Linux firewall to allow only HTTP and SSH traffic?

Answer:

```
sudo ufw allow ssh
sudo ufw allow http
sudo ufw enable
```

Q88: How do you schedule a weekly backup of the **/home** directory?

- Answer:

Create a backup script:

```
vi /usr/local/bin/backup.sh
```

Add:

```
#!/bin/
tar -czf /backup/home_$(date +%F).tar.gz /home
```

Make the script executable:

```
chmod +x /usr/local/bin/backup.sh
```

Add to cron:



```
crontab -e
```

Add:

```
0 3 * * 0 /usr/local/bin/backup.sh
```

Q89: How do you restore files from a compressed tar archive?

Answer:

```
tar -xvzf backup.tar.gz -C /restore/path
```

Q90: How do you clear logs without deleting the file?

Answer:

```
sudo truncate -s 0 /var/log/syslog
```

Q91: How do you archive logs older than 60 days?

Answer:

```
find /var/log -type f -mtime +60 -exec tar -rvf old_logs.tar  
{ } \; -exec rm { } \;
```

Q92: How do you rebuild the initramfs image?

Answer:

```
sudo dracut -f
```

Q93: How do you blacklist a kernel module?

- Answer:

Add the module name to `/etc/modprobe.d/blacklist.conf`:



```
echo "blacklist <module_name>" | sudo tee -a  
/etc/modprobe.d/blacklist.conf
```

Regenerate the initramfs:

```
sudo dracut -f
```

Q94: How do you check which package owns a specific file?

- Answer:

For Debian/Ubuntu:

```
dpkg -S /path/to/file
```

For RHEL/CentOS:

```
rpm -qf /path/to/file
```

Q95: How do you clean up cached package files in CentOS?

Answer:

```
sudo yum clean all
```

Q96: How do you list available virtual machines in KVM?

Answer:

```
virsh list --all
```

Q97: How do you take a snapshot of a running VM in KVM?

Answer:

```
virsh snapshot-create-as <vm_name> <snapshot_name>
```

Q98: How do you restart all Docker containers?

Answer:

```
docker restart $(docker ps -q)
```

Q99: How do you check the status of Kubernetes pods?

Answer:

```
kubectl get pods
```

Q100: How do you determine the Linux distribution and version?

Answer:

```
lsb_release -a
```

Or:

```
cat /etc/os-release
```

Q101: How do you test network connectivity using **ping** and set a timeout?

Answer:

```
ping -c 4 -W 2 <host>
```

- **-c 4**: Sends 4 packets.
- **-W 2**: Sets a timeout of 2 seconds.

Q102: How do you check active network connections on a system?

Answer:

```
netstat -an
```

Or:

```
ss -tunap
```

Q103: How do you schedule a process to run at a specific time using **at**?

- Answer:

Install **at** if not already available:

```
sudo apt install at
```

Use the **at** command:

```
echo "service apache2 restart" | at 02:00
```

Q104: How do you view the priority of a running process?

Answer:

```
ps -eo pid,ni,comm | grep <process_name>
```

- **ni**: Displays the process's nice value.

Q105: How do you monitor memory usage for a specific process?

Answer:

```
pmap <PID> | grep total
```



- **pmap**: Displays memory map of a process.

Q106: How do you find the top 5 CPU-consuming processes?

Answer:

```
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 5
```

Q107: Write a script to check if a directory exists and create it if not.

Answer:

```
#!/bin/  
DIR=$1  
if [ ! -d "$DIR" ]; then  
    mkdir -p "$DIR"  
    echo "Directory $DIR created."  
else  
    echo "Directory $DIR already exists."  
fi
```

Q108: Write a script to calculate the factorial of a given number.

Answer:

```
#!/bin/  
factorial=1  
for (( i=1; i<=$1; i++ ))  
do  
    factorial=$((factorial * i))  
done  
echo "Factorial of $1 is $factorial"
```



Q109: How do you display disk usage for a specific directory in a human-readable format?

Answer:

```
du -sh /path/to/directory
```

Q110: How do you check for filesystem errors on a specific partition?

Answer:

```
sudo fsck /dev/sdX
```

Q111: How do you enforce password expiration for a user?

Answer:

```
sudo chage -M 30 username
```

- **-M 30:** Sets the maximum number of days before password expiration.

Q112: How do you set up a firewall rule to block all incoming traffic except SSH?

Answer:

```
sudo ufw default deny incoming
sudo ufw allow ssh
sudo ufw enable
```

Q113: How do you compress and back up a MySQL database?

Answer:

```
mysqldump -u root -p database_name | gzip >  
database_backup.sql.gz
```

Q114: How do you restore a database from a compressed backup?

Answer:

```
gunzip < database_backup.sql.gz | mysql -u root -p  
database_name
```

Q115: How do you view only the last 10 lines of multiple log files?

Answer:

```
tail -n 10 /var/log/*.log
```

Q116: How do you delete log files older than 90 days?

Answer:

```
find /var/log -type f -mtime +90 -exec rm -f {} \;
```

Q117: How do you enable or disable a kernel module at runtime?

- **Answer:**

Enable:

```
sudo modprobe <module_name>
```

Disable:

```
sudo rmmod <module_name>
```

Q118: How do you check the kernel boot parameters?



Answer:

```
cat /proc/cmdline
```

Q119: How do you list all available updates for an Ubuntu system?

Answer:

```
sudo apt list --upgradable
```

Q120: How do you install a package from a **.rpm** file in CentOS?

Answer:

```
sudo rpm -ivh package.rpm
```

Q121: How do you shut down a running VM in KVM?

Answer:

```
virsh shutdown <vm_name>
```

Q122: How do you configure a network bridge for a VM in KVM?

- Answer:

Edit the network configuration file:

```
sudo vi /etc/sysconfig/network-scripts/ifcfg-br0
```

Add bridge configuration:

```
makefile
```



```
DEVICE=br0  
TYPE=Bridge  
BOOTPROTO=dhcp  
ONBOOT=yes
```

Q123: How do you check the logs of a specific Docker container?

Answer:

```
docker logs <container_id>
```

Q124: How do you delete all stopped Docker containers?

Answer:

```
docker container prune
```

Q125: How do you display the currently logged-in users?

Answer:

```
who
```

Q126: How do you set an environment variable permanently for a user?

- **Answer:**

Add the variable to the user's **.rc** file:

```
echo "export VAR_NAME=value" >> ~/.rc
```

Reload the shell:

```
source ~/.rc
```



Q127: How do you troubleshoot if a specific port is not accessible on a server?

- Answer:

Check if the service is running:

```
sudo systemctl status <service_name>
```

Verify if the port is open:

```
sudo netstat -tuln | grep <port>
```

Check firewall rules:

```
sudo ufw status
```

Test connectivity using **telnet** or **nc**:

```
telnet <server_ip> <port>
```

1. Inspect logs for errors in **/var/log**.

Q128: How do you configure DNS resolution in Linux?

- Answer:

Edit the **/etc/resolv.conf** file:

```
nameserver 8.8.8.8  
nameserver 8.8.4.4
```

Save and test using:



```
nslookup google.com
```

Q129: How do you restart a service automatically if it crashes?

- **Answer:**

Create a systemd service file or edit an existing one:

```
sudo vi /etc/systemd/system/<service_name>.service
```

Add the following lines under **[Service]**:

makefile

```
Restart=always
```

```
RestartSec=5
```

Reload systemd and enable the service:

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable <service_name>
```

Q130: How do you check all processes owned by a specific user?

Answer:

```
ps -u <username>
```

Q131: How do you monitor network bandwidth usage in real-time?

Answer:

```
iftop -i <interface>
```

Or:

```
nload <interface>
```

Q132: How do you generate a CPU usage report for the past hour?

Answer:

```
sar -u -s HH:MM -e HH:MM
```

- Replace **HH:MM** with the time range.

Q133: Write a script to check if a website is reachable.

Answer:

```
#!/bin/  
WEBSITE=$1  
if curl -Is $WEBSITE | grep "200 OK"; then  
    echo "$WEBSITE is reachable."  
else  
    echo "$WEBSITE is not reachable."  
fi
```

Q134: Write a script to back up a directory and log the output.

Answer:

```
#!/bin/  
DIR=$1  
BACKUP_DIR="/backup/$(date +%F)"
```



```
mkdir -p $BACKUP_DIR
tar -czf $BACKUP_DIR/backup.tar.gz $DIR >
/var/log/backup.log 2>&1
echo "Backup completed. Log available at
/var/log/backup.log."
```

Q135: How do you add a new disk to an existing Linux system without rebooting?

- Answer:

Scan for new disks:

```
sudo partprobe
```

Create a partition:

```
sudo fdisk /dev/sdX
```

Create a filesystem:

```
sudo mkfs.ext4 /dev/sdX1
```

Mount the partition:

```
sudo mount /dev/sdX1 /mnt
```

Q136: How do you increase the size of an existing filesystem?

- Answer:

Extend the logical volume:

```
sudo lvextend -L +10G /dev/vol_group/lv_name
```



Resize the filesystem:

```
sudo resize2fs /dev/vol_group/lv_name
```

Q137: How do you enable auditing on a file?

- Answer:

Install **auditd** if not already available:

```
sudo apt install auditd
```

Add a rule:

```
sudo auditctl -w /path/to/file -p rwx -k file_audit
```

Check logs:

```
sudo ausearch -k file_audit
```

Q138: How do you disable password-based SSH authentication?

- Answer:

Edit **/etc/ssh/sshd_config**:

```
PasswordAuthentication no
```

Restart the SSH service:

```
sudo systemctl restart sshd
```



Q139: How do you take an incremental backup with **rsync**?

Answer:

```
rsync -av --delete /source /destination
```

Q140: How do you restore a specific file from a tar archive?

Answer:

```
tar -xvzf backup.tar.gz path/to/file
```

Q141: How do you find the most frequent error in a log file?

Answer:

```
grep "ERROR" /var/log/syslog | awk '{print $NF}' | sort |  
uniq -c | sort -nr | head -n 1
```

Q142: How do you configure centralized log management using **rsyslog**?

- Answer:

On the client, edit **/etc/rsyslog.conf**:

```
*.* @logserver_ip:514
```

Restart the **rsyslog** service:

```
sudo systemctl restart rsyslog
```

Q143: How do you verify kernel parameters in Linux?

Answer:



```
sysctl -a
```

Q144: How do you temporarily modify a kernel parameter?

Answer:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Q145: How do you search for a package using **yum**?

Answer:

```
sudo yum search <package_name>
```

Q146: How do you install a package from a **.deb** file?

Answer:

```
sudo dpkg -i package.deb  
sudo apt-get install -f
```

Q147: How do you clone a virtual machine in KVM?

Answer:

```
virt-clone --original <vm_name> --name <new_vm_name> --file  
/var/lib/libvirt/images/new_vm.img
```

Q148: How do you check the virtualized environment of a Linux system?

Answer:

```
sudo dmidecode -s system-product-name
```

Q149: How do you delete unused Docker images?

Answer:

```
docker image prune
```

Q150: How do you deploy a Kubernetes pod using a YAML file?

Answer:

```
kubectl apply -f pod.yaml
```

Q151: How do you find the IP address of your Linux machine?

Answer:

```
ip addr show
```

Or for a specific interface:

```
ip addr show eth0
```

Q152: How do you capture network packets on a specific interface?

Answer:

```
sudo tcpdump -i eth0
```

Q153: How do you list all active and inactive services?

Answer:

```
systemctl list-units --type=service
```



Q154: How do you view the startup services and their statuses?

Answer:

```
sudo systemctl list-unit-files --type=service
```

Q155: How do you analyze disk I/O performance?

Answer:

```
iostat -d -x 2
```

- **-d:** Displays device utilization.
- **-x:** Displays extended statistics.

Q156: How do you display a summary of system performance?

Answer:

```
vmstat 1 5
```

- **1 5:** Collects data every second for 5 iterations.

Q157: Write a script to count the number of users currently logged in.

Answer:

```
#!/bin/  
echo "Number of logged-in users: $(who | wc -l)"
```

Q158: Write a script to check disk usage and send an email alert if it exceeds 90%.

Answer:




```
#!/bin/
THRESHOLD=90
USAGE=$(df -h / | grep / | awk '{print $5}' | sed 's/%//')
if [ "$USAGE" -gt "$THRESHOLD" ]; then
    echo "Disk usage is at ${USAGE}% on $(hostname)" | mail
-s "Disk Usage Alert" admin@example.com
fi
```

Q159: How do you remove a mount point that is in use?

- Answer:

Identify processes using the mount point:

```
lsof | grep /mount/point
```

Kill the processes:

```
kill -9 <PID>
```

Unmount:

```
sudo umount /mount/point
```

Q160: How do you find disk partitions and their usage?

Answer:

```
lsblk
```

Q161: How do you add a user to the **sudo** group?

Answer:



```
sudo usermod -aG sudo username
```

Q162: How do you check for open ports on your system?

Answer:

```
sudo netstat -tuln
```

Or:

```
sudo ss -tuln
```

Q163: How do you set up an automatic daily backup using **cron**?

- Answer:

Create a script:

```
vi /usr/local/bin/daily_backup.sh
```

Add:

```
#!/bin/  
tar -czf /backup/$(date +%F).tar.gz /data
```

Make it executable:

```
chmod +x /usr/local/bin/daily_backup.sh
```

Add to cron:

```
crontab -e
```

Add:



```
0 3 * * * /usr/local/bin/daily_backup.sh
```

Q164: How do you restore files from an incremental backup?

- Answer:

Extract the full backup first:

```
tar -xvzf full_backup.tar.gz -C /restore_path
```

Extract the incremental backups in order:

```
tar -xvzf incremental_backup.tar.gz -C /restore_path
```

Q165: How do you monitor log files in real-time for multiple files?

Answer:

```
multitail /var/log/file1 /var/log/file2
```

Q166: How do you rotate logs for a specific application?

- Answer:

Create a logrotate config:

```
sudo vi /etc/logrotate.d/app_name
```

Add:

```
/var/log/app_name/*.log {  
    daily  
    rotate 5
```

```
compress
missingok
notifempty
}
```

Test:

```
sudo logrotate -d /etc/logrotate.d/app_name
```

Q167: How do you enable IP forwarding temporarily?

Answer:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Q168: How do you list loaded kernel modules with descriptions?

Answer:

```
lsmod | column -t
```

Q169: How do you remove a package and its dependencies in Ubuntu?

Answer:

```
sudo apt-get autoremove --purge <package_name>
```

Q170: How do you upgrade a specific package in CentOS?

Answer:

```
sudo yum update <package_name>
```



Q171: How do you create a snapshot of a virtual machine in KVM?

Answer:

```
virsh snapshot-create-as <vm_name> <snapshot_name>
```

Q172: How do you configure networking for a VM to use NAT in KVM?

- Answer:

Edit the **virsh** network configuration:

```
virsh net-edit default
```

1. Ensure the **<forward mode='nat' />** tag is present.

Q173: How do you restart a Kubernetes deployment?

Answer:

```
kubectl rollout restart deployment <deployment_name>
```

Q174: How do you inspect the Docker network?

Answer:

```
docker network inspect <network_name>
```

Q175: How do you set a default editor for the terminal?

Answer:



```
export EDITOR=vim
```

To make it persistent:

```
echo "export EDITOR=vim" >> ~/.rc  
source ~/.rc
```

Q176: How do you schedule a one-time job using **at**?

Answer:

```
echo "shutdown -h now" | at 23:00
```

Q177: How do you add a static route in Linux?

- Answer:

Add a temporary route:

```
sudo ip route add 192.168.2.0/24 via 192.168.1.1 dev eth0
```

1. Add a persistent route:

For RHEL/CentOS, edit `/etc/sysconfig/network-scripts/route-eth0` and add:

```
192.168.2.0/24 via 192.168.1.1
```

Q178: How do you troubleshoot high network latency?

- Answer:

Check connectivity with **ping**:



ping <destination>

Trace the network path with **traceroute**:

traceroute <destination>

Inspect network traffic:

sudo tcpdump -i eth0

Monitor interface statistics:

ifconfig eth0

Q179: How do you set CPU affinity for a process?

Answer:

taskset -c 0,1 <PID>

- This assigns the process to CPUs 0 and 1.

Q180: How do you view the system logs of a specific service?

Answer:

journalctl -u <service_name>

Q181: How do you check the uptime of a system?

Answer:

Uptime

Q182: How do you monitor disk throughput in real-time?

Answer:

iostat

Q183: Write a script to monitor a directory and log changes.

Answer:

```
#!/bin/
inotifywait -m /path/to/directory -e create -e delete -e
modify |
while read path action file; do
    echo "[$(date)] $file was $action in $path" >>
/var/log/dir_changes.log
done
```

Q184: Write a script to send an email alert if a service is down.

Answer:

```
#!/bin/
SERVICE=nginx
if ! systemctl is-active --quiet $SERVICE; then
    echo "$SERVICE is down on $(hostname)" | mail -s
"Service Alert" admin@example.com
fi
```

Q185: How do you mount a filesystem at boot?

- **Answer:**



Add an entry in `/etc/fstab`:

```
/dev/sdX1 /mnt/data ext4 defaults 0 2
```

Test the entry:

```
sudo mount -a
```

Q186: How do you increase the size of a swap file?

- Answer:

Turn off the current swap file:

```
sudo swapoff /swapfile
```

Resize the swap file:

```
sudo dd if=/dev/zero of=/swapfile bs=1G count=4
```

Set it up again:

```
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
```

Q187: How do you configure SSH key-based authentication?

- Answer:

Generate a key pair:

```
ssh-keygen -t rsa
```



Copy the public key to the remote server:

```
ssh-copy-id user@remote_server
```

Verify login:

```
ssh user@remote_server
```

Q188: How do you block an IP address using **iptables**?

Answer:

```
sudo iptables -A INPUT -s <IP_ADDRESS> -j DROP
```

Q189: How do you create a snapshot of an LVM volume?

- Answer:

Create the snapshot:

```
sudo lvcreate --size 1G --snapshot --name snap_name  
/dev/vol_group/lv_name
```

Mount the snapshot:

```
sudo mount /dev/vol_group/snap_name /mnt
```

Q190: How do you restore a system from a tar backup?

Answer:

```
tar -xvzf backup.tar.gz -C /
```



Q191: How do you configure remote logging using **rsyslog**?

- Answer:

On the client machine, edit **/etc/rsyslog.conf**:

```
*.* @<log_server_ip>:514
```

Restart the **rsyslog** service:

```
sudo systemctl restart rsyslog
```

Q192: How do you analyze logs for the most common HTTP errors?

Answer:

```
cat /var/log/httpd/access.log | awk '{print $9}' | sort |  
uniq -c | sort -nr
```

Q193: How do you persist kernel parameter changes?

- Answer:

Add the parameter to **/etc/sysctl.conf**:

```
net.ipv4.ip_forward=1
```

Apply the changes:

```
sudo sysctl -p
```

Q194: How do you display CPU information?



Answer:

```
cat /proc/cpuinfo
```

Q195: How do you verify the integrity of a package in CentOS?

Answer:

```
rpm -V <package_name>
```

Q196: How do you list dependencies for a package in Ubuntu?

Answer:

```
apt-cache depends <package_name>
```

Q197: How do you migrate a VM to another host in KVM?

Answer:

```
virsh migrate --live <vm_name>  
qemu+ssh://<destination_host>/system
```

Q198: How do you resize a virtual disk in KVM?

- Answer:

Resize the disk:

```
qemu-img resize /path/to/disk.img +10G
```

1. Extend the filesystem inside the VM.



Q199: How do you update an image in a Kubernetes deployment?

Answer:

```
kubectl set image deployment/<deployment_name>  
<container_name>=<new_image>
```

Q200: How do you clean up unused volumes in Docker?

Answer:

```
docker volume prune
```

Q201: How do you configure a bonded network interface in Linux?

- Answer:

Install the bonding module:

```
sudo modprobe bonding
```

1. Configure the bond in `/etc/network/interfaces` (Debian/Ubuntu) or `/etc/sysconfig/network-scripts/ifcfg-bond0` (RHEL/CentOS).

Q202: How do you troubleshoot DNS resolution issues?

- Answer:
 1. Check `/etc/resolv.conf` for DNS server entries.

Test resolution:

```
nslookup google.com
```



Use **dig** for detailed DNS queries:

```
dig google.com
```

Q203: How do you debug a hung process in Linux?

- Answer:

Check the state of the process:

```
ps -o stat= -p <PID>
```

Use **strace** to trace system calls:

```
strace -p <PID>
```

Q204: How do you enable a service to start on boot?

Answer:

```
sudo systemctl enable <service_name>
```

Q205: How do you analyze system load averages?

Answer: Use the **uptime** or **top** command:

```
uptime
```

- Load averages represent the number of processes waiting for CPU over 1, 5, and 15 minutes.

Q206: How do you find disk latency using **iostat**?



Answer:

```
iostat -x
```

- Check **await** and **svctm** columns for latency.

Q207: Write a script to find the largest file in a directory.

Answer:

```
#!/bin/  
find $1 -type f -exec du -h {} + | sort -rh | head -n 1
```

Q208: Write a script to list all installed packages and save them to a file.

Answer:

```
#!/bin/  
dpkg --get-selections > installed_packages.txt
```

Q209: How do you check the health of a hard drive?

Answer:

```
sudo smartctl -H /dev/sdX
```

Q210: How do you enable quotas on a filesystem?

- Answer:

Mount the filesystem with quota options:

```
sudo mount -o remount,usrquota,grpquota /dev/sdX /mnt
```



Create quota files:

```
sudo touch /mnt/aquota.user /mnt/aquota.group
sudo chmod 600 /mnt/aquota.*
sudo quotacheck -cug /mnt
```

Enable quotas:

```
sudo quotaon /mnt
```

Q211: How do you monitor failed login attempts?

Answer:

```
sudo grep "Failed password" /var/log/auth.log
```

Q212: How do you lock a user account?

Answer:

```
sudo passwd -l <username>
```

Q213: How do you verify the integrity of a backup file?

- Answer:

Generate a checksum:

```
md5sum backup.tar.gz
```

1. Compare it with the original checksum.

Q214: How do you back up a MySQL database with compression?

Answer:

```
mysqldump -u root -p database_name | gzip > backup.sql.gz
```

Q215: How do you set up log rotation for custom application logs?

- Answer:

Create a custom configuration file:

```
sudo vi /etc/logrotate.d/custom_app
```

Add:

```
/var/log/custom_app/*.log {  
    daily  
    rotate 7  
    compress  
    missingok  
    notifempty  
}
```

Q216: How do you clear logs securely?

Answer:

```
sudo shred -u /var/log/<logfile>
```

Q217: How do you view active IRQs in Linux?

Answer:

```
cat /proc/interrupts
```

Q218: How do you list the system's block devices?

Answer:

```
lsblk
```

Q219: How do you downgrade a package in Ubuntu?

Answer:

```
sudo apt-get install <package_name>=<version>
```

Q220: How do you check the changelog of an installed package?

Answer:

```
apt changelog <package_name>
```

Q221: How do you start a VM using **virsh**?

Answer:

```
virsh start <vm_name>
```

Q222: How do you display the console output of a VM?

Answer:

```
virsh console <vm_name>
```



Q223: How do you view resource usage of a Kubernetes pod?

Answer:

```
kubectl top pod <pod_name>
```

Q224: How do you build a Docker image from a Dockerfile?

Answer:

```
docker build -t <image_name>:<tag> .
```

Q225: How do you display system architecture?

Answer:

```
uname -m
```

Q226: How do you display the last reboot time of the system?

Answer:

```
who -b
```

Q227: How do you configure a VLAN in Linux?

Answer:

```
sudo ip link add link eth0 name eth0.100 type vlan id 100
sudo ip addr add 192.168.1.2/24 dev eth0.100
sudo ip link set dev eth0.100 up
```

Q228: How do you test the MTU size of a network?

Answer:



```
ping -M do -s 1472 <destination>
```

Q229: Write a script to monitor a process and restart it if it stops.

Answer:

```
#!/bin/  
PROCESS="nginx"  
if ! pgrep $PROCESS > /dev/null; then  
    echo "$PROCESS is not running. Restarting..."  
    sudo systemctl restart $PROCESS  
fi
```

Q230: How do you migrate data from one disk to another?

Answer:

```
rsync -av /source/ /destination/
```

Q231: How do you prioritize a process using **nice**?

Answer:

```
nice -n 10 <command>
```

Q232: How do you set up incremental backups with **rsync**?

Answer:

```
rsync -av --link-dest=/previous/backup /source/ /new/backup/
```

Q233: How do you configure a Linux server as a router?

- Answer:



Enable IP forwarding:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Make it persistent:

```
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
```

Add routing rules using **iptables**:

```
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Q234: How do you capture only HTTP traffic using **tcpdump**?

Answer:

```
sudo tcpdump -i eth0 port 80
```

Q235: How do you set a process to run at a specific nice level permanently?

- Answer:

Use **nice** while starting the process:

```
nice -n 10 <command>
```

Use **renice** for running processes:

```
renice 10 -p <PID>
```

Q236: How do you restart all failed services?



Answer:

```
sudo systemctl list-units --failed | awk '{print $1}' |  
xargs sudo systemctl restart
```

Q237: How do you monitor real-time system statistics?

Answer: Use the **dstat** command:

dstat

- It provides real-time statistics for CPU, memory, disk I/O, and more.

Q238: How do you monitor user activity on a Linux system?

Answer: Use the **w** command:

w

- It shows logged-in users and their activity.

Q239: Write a script to list all files modified in the last 7 days.

Answer:

```
#!/bin/  
find $1 -type f -mtime -7
```

Q240: Write a script to backup logs and delete the originals after backup.

Answer:

```
#!/bin/  
LOG_DIR="/var/log"
```



```
BACKUP_DIR="/backup"
mkdir -p $BACKUP_DIR
tar -czf $BACKUP_DIR/logs_$(date +%F).tar.gz $LOG_DIR/*.log
rm $LOG_DIR/*.log
```

Q241: How do you move a partition to a new disk?

- Answer:

Copy data using **dd**:

```
sudo dd if=/dev/sdX1 of=/dev/sdY1 bs=64K conv=noerror,sync
```

1. Update **/etc/fstab** to reflect the new disk.

Q242: How do you defragment a filesystem?

- Answer:

For **ext4** filesystems:

```
sudo e4defrag /path/to/filesystem
```

Q243: How do you list all open files by a specific process?

Answer:

```
lsof -p <PID>
```

Q244: How do you block specific IP ranges using **iptables**?

Answer:

```
sudo iptables -A INPUT -m iprange --src-range
192.168.1.0-192.168.1.255 -j DROP
```



Q245: How do you create a differential backup using **rsync**?

Answer:

```
rsync -av --compare-dest=/path/to/previous/backup /source/  
/path/to/current/backup/
```

Q246: How do you restore permissions from a backup?

- Answer: Use **getfacl** and **setfacl**:

Backup permissions:

```
getfacl -R /path/to/directory > permissions.acl
```

Restore permissions:

```
setfacl --restore=permissions.acl
```

Q247: How do you filter logs for a specific date range?

Answer:

```
awk ' /2025-01-20/,/2025-01-21/' /var/log/syslog
```

Q248: How do you monitor changes to a log file?

Answer: Use **tail** with **-f**:

```
tail -f /var/log/syslog
```

Q249: How do you list all currently loaded kernel modules with their sizes?

Answer:



lsmod

Q250: How do you remove a kernel module that's in use?

- Answer:

Identify dependent processes:

```
lsmod | grep <module_name>
```

Stop dependent processes or unload dependencies first:

```
sudo modprobe -r <module_name>
```

Q-1 What is the difference between hard links and soft links?

🔗 **Hard Link:** Creates a duplicate reference to the same inode. Deleting the original file doesn't affect the hard link. Only works within the same filesystem.

🔗 **Soft Link (Symbolic Link):** Creates a pointer to the original file. If the original file is deleted, the soft link becomes broken. Works across different filesystems.

Q-2 Explain the significance of the 'root' user in Linux.

The **root** user is the **superuser** in Linux with **unrestricted access** to all system files and commands. It can install software, modify system configurations, and manage users. Misuse can cause system damage, so it's recommended to use **sudo** for administrative tasks.

Q-3 How do you find all files modified in the last 10 minutes in a directory and its subdirectories?

Use the find command:

find /path/to/directory -type f -mmin -10

This searches for files (-type f) modified in the last 10 minutes (-mmin -10) within the specified directory and its subdirectories.

Q-4 What is a kernel in Linux?

The **kernel** is the core of the Linux OS. It manages hardware resources, handles system processes, memory, and device communication. It acts as a bridge between software and hardware, ensuring efficient operation.

Q-5 Explain the purpose of the 'chmod' command.

The chmod command is used to change file permissions in Linux. It controls who can read (r), write (w), and execute (x) a file or directory.

Examples:

- **chmod 755 file.txt** → Owner: rwx, Group & Others: r-x
- **chmod u+x script.sh** → Adds execute permission to the user
- **chmod -R 644 /folder** → Recursively sets files to rw-r--r--

Q-6 How can you find out the current runlevel of a Linux system?

Use any of these commands to check the current runlevel:

runlevel

or

who -r

The output shows the previous and current runlevel (e.g., N 5 means the system is in runlevel 5).

Q-7 Explain the role of the 'grep' command in Linux.

The grep command is used to **search for a pattern in files or output**. It filters lines matching a specific string or regex.

Examples:

- `grep "error" logfile.txt` → Finds "error" in logfile.txt
- `ps aux | grep apache` → Searches for "apache" in running processes
- `grep -i "hello" file.txt` → Case-insensitive search for "hello"

Q-8 What is the purpose of the 'df' command?

The df command displays the **disk space usage** of file systems. It shows the amount of used and available space on mounted file systems.

df -h

Q-9 Explain the 'ps' command and how it is used to view processes.

The ps command is used to display **running processes** on a Linux system. It provides information like process ID (PID), CPU usage, and memory usage.

Examples:

- `ps` → Lists processes running in the current shell
- `ps aux` → Shows all processes with detailed information
- `ps -ef` → Displays processes in a full-format list

Q-10 How do you change the priority of a process in Linux?

Use the nice or renice command to change a process's priority:

To start a process with a specific priority:

`nice -n 10 command_name`

(The -n value ranges from -20 to 19, with lower values indicating higher priority.)

To change the priority of a running process:

`renice -n 10 -p <PID>`

(Replace <PID> with the process ID.)

Q-11 What is the purpose of the 'awk' command?

The awk command is used for **pattern scanning and processing**. It is typically used to **manipulate and analyze text files**, such as extracting or formatting specific columns of data.

Examples:

- `awk '{print $1}' file.txt` → Prints the first column from file.txt
- `ps aux | awk '{print $1, $3}'` → Prints the user and CPU usage from ps output
- `awk '/pattern/ {print $0}' file.txt` → Prints lines matching the pattern

Q-12 Explain the role of the 'tar' command in Linux.

The tar command is used to **archive** multiple files or directories into a single file, often for easier storage or transfer. It can also compress the archive using gzip or bzip2.

Examples:

- `tar -cvf archive.tar directory/` → Creates a .tar archive of a directory
- `tar -xvf archive.tar` → Extracts a .tar archive
- `tar -czvf archive.tar.gz directory/` → Creates a compressed .tar.gz archive

Q-13 How can you check the connectivity between two hosts using the 'ping' command?

Use the ping command to check the connectivity between two hosts:

ping <hostname_or_IP_address>

For example, to ping Google's DNS server:

ping 8.8.8.8

It sends ICMP echo requests to the target, and if the host is reachable, it will respond with echo replies.

Q-14 What is a cron job, and how do you create one?

A **cron job** is a scheduled task that runs automatically at specified intervals on a Linux system. It's managed by the **cron** daemon.

To create a cron job, edit the crontab file with `crontab -e` and add the desired schedule and command. The syntax is:

*** * * * * /path/to/command**

Where the five asterisks represent:

1. Minute (0-59)
2. Hour (0-23)
3. Day of the month (1-31)
4. Month (1-12)
5. Day of the week (0-6, Sunday=0)

Example:

To run a script every day at 2:30 AM:

30 2 * * * /path/to/script.sh

Q-15 Explain the purpose of the 'iptables' command.

The iptables command is used to configure and manage the **Linux firewall**. It controls incoming and outgoing network traffic by setting rules to **filter packets** based on criteria like IP addresses, ports, and protocols.

Examples:

- `iptables -L` → Lists all current firewall rules
- `iptables -A INPUT -p tcp --dport 80 -j ACCEPT` → Allows incoming traffic on port 80 (HTTP)
- `iptables -A INPUT -j DROP` → Blocks all incoming traffic

Q-16 How do you list all open ports on a Linux system?

To list all open ports on a Linux system, you can use the `netstat` or `ss` command:

- **Using `netstat`:**

`netstat -tuln`

Using `ss`:

`ss -tuln`

Both commands show a list of listening ports along with their corresponding services. `-t` for TCP, `-u` for UDP, `-l` for listening, and `-n` for numeric output.

Q-17 What is the purpose of the 'find' command in Linux?

The `find` command is used to **search for files and directories** within a specified location based on various criteria like name, size, type, and modification time.

Examples:

- `find /path/to/search -name "file.txt"` → Finds files named `file.txt`
- `find / -type d` → Lists all directories
- `find /home -mtime -7` → Finds files modified in the last 7 days

Q-18 Explain the significance of the 'passwd' file in Linux.

The `/etc/passwd` file in Linux contains essential information about system users. Each line in the file represents a user account with details such as:

1. **Username**
2. **Password** (encrypted, or a placeholder like `x` if stored in `/etc/shadow`)
3. **User ID (UID)**
4. **Group ID (GID)**
5. **User Full Name or Description**
6. **Home Directory**
7. **Login Shell**

It's critical for user authentication and system access.

Q-19 How can you check the available disk space on a specific filesystem using the 'du' command?

Use the `du` command with the `-h` option to check disk space usage for a specific filesystem or directory:

`du -sh /path/to/directory`

`-s` gives the total size of the directory.

`-h` makes the output human-readable (e.g., in MB, GB).

To check disk usage for all subdirectories:

`du -h /path/to/directory`

Q-20 Explain the purpose of the 'curl' command.

The `curl` command is used to **transfer data to or from a server** using various protocols (HTTP, HTTPS, FTP, etc.). It's commonly used to interact with APIs, download files, or test network connections.

Examples:

- `curl https://example.com` → Fetches the content of a webpage
- `curl -O http://example.com/file.txt` → Downloads a file
- `curl -X GET https://api.example.com/data` → Sends a GET request to an API

Q-21 What is the purpose of the 'dd' command in Linux?

The `dd` command is used for low-level copying and conversion of data. It can copy entire disks, partitions, or files, and is often used for creating disk images, backups, or restoring data.

Examples:

- `dd if=/dev/sda of=/path/to/backup.img` → Creates a disk image of `/dev/sda`
- `dd if=backup.img of=/dev/sda` → Restores the disk image to a disk
- `dd if=/dev/zero of=file.txt bs=1M count=100` → Creates a 100MB file filled with zeros

Q-22 How can you set environment variables in Linux?

You can set environment variables in Linux using the `export` command:

- **Temporarily (for the current session):**

`export VAR_NAME="value"`

Permanently (for all sessions):

Add the `export` command to the `~/.bashrc` (or `~/.bash_profile` for some distributions) file:

`echo 'export VAR_NAME="value"' >> ~/.bashrc`

`source ~/.bashrc`

This sets the environment variable for the current session or permanently for all future sessions.

Q-23 What is the difference between a process and a thread?

- **Process:** A process is an independent program running in its own memory space. It has its own resources (e.g., file descriptors, memory) and is isolated from other processes.
- **Thread:** A thread is the smallest unit of a process. Multiple threads within the same process share resources like memory but can execute independently. Threads are more lightweight compared to processes.

In short, processes are independent, while threads are part of a process and share the same memory space.

Q-24 Explain the purpose of the 'mount' command in Linux.

The mount command is used to **attach a filesystem** (e.g., a disk, partition, or network share) to a directory in the Linux file system hierarchy, making it accessible to the system.

Examples:

- `mount /dev/sda1 /mnt` → Mounts the disk/partition `/dev/sda1` to the `/mnt` directory
- `mount -t nfs server:/path /mnt` → Mounts a network file system (NFS) share
- `mount -o loop file.iso /mnt` → Mounts an ISO file as a virtual filesystem

Q-25 How do you find the top 5 memory-consuming processes on a Linux system?

You can use the `top` or `ps` command to find the top memory-consuming processes:

- **Using top:**
Run `top`, and press `M` to sort by memory usage. To limit it to the top 5, you can use:

`top -o %MEM`

- **Using ps:**

`ps aux --sort=-%mem | head -n 6`

This shows the top 5 processes consuming memory (including the header line).

Q-26 What is a symbolic link, and how do you create one using the 'ln' command?

A **symbolic link** (or **symlink**) is a type of file that points to another file or directory. Unlike hard links, symlinks can span across different filesystems and can link to directories.

To create a symbolic link:

`ln -s /path/to/target /path/to/symlink`

- `-s`: Creates a symbolic link.
- `/path/to/target`: The file or directory the symlink points to.
- `/path/to/symlink`: The name of the symlink.

Example:

`ln -s /home/user/file.txt /home/user/file_link.txt`

This creates a symlink named `file_link.txt` pointing to `file.txt`.

Q-27 Explain the purpose of the 'sed' command in Linux.

The sed (stream editor) command is used for **text manipulation** in Linux. It allows you to perform operations like search, replace, delete, and insert text in files or input streams.

Examples:

- `sed 's/old/new/' file.txt` → Replaces the first occurrence of "old" with "new" in each line of file.txt
- `sed -i 's/old/new/g' file.txt` → Replaces all occurrences of "old" with "new" in file.txt (in-place edit)
- `sed '/pattern/d' file.txt` → Deletes lines containing "pattern" in file.txt

Q-28 How can you run a Linux command in the background?

To run a Linux command in the background, append an **ampersand (&)** at the end of the command:

command &

This runs the command in the background, allowing you to continue using the terminal for other tasks.

For example:

sleep 60 &

This runs the sleep command for 60 seconds in the background.

Q-29 What is the 'sudo' command, and how is it used in Linux?

The sudo (superuser do) command allows a regular user to **execute commands with superuser (root) privileges**. It is used for tasks that require administrative permissions, such as installing software or modifying system settings.

Usage:

sudo command

For example:

- **sudo apt update** → Runs the apt update command with root privileges to update package lists.
- **sudo rm /protected_file** → Deletes a file that requires root access.

You'll usually be prompted to enter your user password for authentication.

Q-30 Explain the purpose of the 'journalctl' command.

The journalctl command is used to **view and manage logs** collected by the **systemd journal**. It displays logs related to system services, kernel messages, and applications running on the system.

Examples:

- **journalctl** → Displays the entire system log
- **journalctl -u service_name** → Shows logs for a specific service
- **journalctl -f** → Follows and displays real-time logs (similar to tail -f)
- **journalctl --since "1 hour ago"** → Shows logs from the past hour

Q-31 How would you troubleshoot a slow server?

To troubleshoot a slow server, start by checking system resources with tools like `top` or `htop`, which monitor CPU, memory, and processes. Use `free -m` to check memory usage, as insufficient RAM or high swap usage can cause slowness.

Next, examine disk performance with `iostat` or `vmstat` and ensure there's enough free space using `df -h`. Disk issues, like full disks or high I/O, can impact performance.

For network problems, run `netstat -tuln` to check open connections and use `ping` or `traceroute` to detect latency or connectivity issues. High network traffic could slow the server.

Check system logs with `journalctl` or `dmesg` to identify errors or warnings related to hardware, processes, or services. This can highlight issues affecting performance.

Review running processes with `ps aux` and check the load average with `uptime` to identify resource-heavy tasks. High load averages may indicate the server is overloaded.

Finally, check application logs and resource usage to ensure services aren't misconfigured or inefficient. Tools like `sar` or `collectd` can help monitor performance over time.

By evaluating resources, disk, network, logs, and processes, you can pinpoint the cause of the slowness and address it.

Q-32 Describe Linux file permissions and how to change them.

Permissions include read, write, and execute for the owner, group, and others. Use `chmod` to modify permissions (e.g., `chmod 755 filename`), `chown` to change ownership, and `chgrp` to change the group.

Q-33 What is LVM, and how do you manage it?

LVM (Logical Volume Management) is a system for managing disk drives and partitions in Linux. It allows you to create, resize, and manage logical volumes (LVs) dynamically, providing more flexibility than traditional partitioning.

LVM Components:

1. **Physical Volume (PV):** A disk or partition used by LVM.
2. **Volume Group (VG):** A group of physical volumes combined into one pool of storage.
3. **Logical Volume (LV):** A partition within the volume group, which can be resized and managed easily.

Managing LVM:

1. **Create Physical Volume:**
2. `pvcreate /dev/sda`

3. **Create Volume Group:**

4. `vgcreate my_volume_group /dev/sda`

5. **Create Logical Volume:**

6. `lvcreate -L 10G -n my_logical_volume my_volume_group`

7. **Resize Logical Volume:** To extend the LV:

8. `lvextend -L +5G /dev/my_volume_group/my_logical_volume`

To shrink the LV (after unmounting and filesystem checks):

`lvreduce -L -5G /dev/my_volume_group/my_logical_volume`

9. **Display Information:**

10. `pvdisk`

11. `vgdisplay`

12. `lvdisplay`

LVM allows for easy disk management, including resizing volumes, adding new disks, and moving data between disks without downtime.

Q-34 How do you secure a Linux server?

To secure a Linux server:

1. **Keep the system updated** with `sudo apt update` & `sudo apt upgrade`.
2. **Configure a firewall** (e.g., `ufw`) to block unwanted traffic.
3. **Disable root login** via SSH by setting `PermitRootLogin no` in `/etc/ssh/sshd_config`.
4. **Use SSH keys** instead of passwords for authentication.
5. **Set strong password policies** using `pam_pwquality`.
6. **Install Fail2ban** to prevent brute-force attacks.
7. **Limit user privileges** by granting only necessary permissions.
8. **Use SELinux or AppArmor** for mandatory access control.
9. **Disable unnecessary services** with `systemctl`.
10. **Monitor logs** regularly with tools like `logwatch` or `auditd`.

These steps help minimize vulnerabilities and secure the server from unauthorized access.

Q-35 How do you manage services in Linux?

To manage services in Linux, you can use the following commands, depending on your system's init system:

1. **Systemd (most modern Linux distributions):**

- **Start a service:**

- `sudo systemctl start service_name`
- **Stop a service:**
- `sudo systemctl stop service_name`
- **Enable a service to start on boot:**
- `sudo systemctl enable service_name`
- **Disable a service from starting on boot:**
- `sudo systemctl disable service_name`
- **Check the status of a service:**
- `sudo systemctl status service_name`

2. SysVinit (older systems):

- **Start a service:**
- `sudo service service_name start`
- **Stop a service:**
- `sudo service service_name stop`
- **Check the status of a service:**
- `sudo service service_name status`

These commands allow you to manage, monitor, and control services running on a Linux server.

Q-36 How do you add a user and manage permissions.

To add a user and manage permissions in Linux, follow these steps:

1. Add a User:

- To add a new user, use the `useradd` command:
- **`sudo useradd username`**
- To set a password for the user, use the `passwd` command:
- **`sudo passwd username`**

2. Assign User to Groups:

- To add a user to a group:
- **`sudo usermod -aG groupname username`**

3. Set File Permissions:

- Use `chmod` to change file permissions:
- **`chmod 755 filename`**

This gives read, write, and execute permissions to the owner, and read and execute permissions to others.

4. Change Ownership of a File/Directory:

- Use chown to change the owner and group of a file:
- **sudo chown owner:group filename**

5. Check File Permissions:

- Use ls -l to list the file permissions:
- **ls -l filename**

By adding users, assigning them to groups, and adjusting file permissions, you can control access and manage security on your Linux system.

Q-37 How do you monitor server performance?

To monitor server performance in Linux:

1. **CPU Usage:** Use top or htop to view CPU usage and processes.
2. **Memory Usage:** Use free -m to check memory usage.
3. **Disk Usage:** Use df -h to monitor disk space and iostat for disk performance.
4. **Network Activity:** Use netstat -tuln for active connections and iftop for real-time network monitoring.
5. **System Load:** Use uptime or top to check the system load averages.
6. **Logs:** Check system logs using journalctl or dmesg for errors and warnings.
7. **Process Monitoring:** Use ps aux to monitor running processes and resource consumption.

These tools help track system health and identify bottlenecks or performance issues.

Q-38 What steps would you take for filesystem corruption recovery?

To recover from filesystem corruption:

1. **Stop Using the Disk:** Unmount the filesystem using umount (or boot from a live CD/USB if it's the root filesystem).
2. **Run fsck:** Repair the filesystem with sudo fsck /dev/sdX.
3. **Remount:** After repair, mount the filesystem with sudo mount /dev/sdX /mount/point.
4. **Restore from Backup:** If fsck doesn't work, restore from a backup.
5. **Check Logs:** Analyze logs for hardware issues.

This approach helps recover data and fix filesystem issues.

Q-39 How do you automate tasks in Linux?

Use ``cron`` for scheduled tasks (``crontab -e``). For complex automation, use shell scripts and tools like ``Ansible`` or ``Puppet`` for consistent deployments across servers.

Q-40 What is the purpose of the `/etc/resolv.conf` file?

The `/etc/resolv.conf` file configures DNS settings in Linux. It defines the DNS servers to query (nameserver), the default domain to append (search), and other options like timeout settings (options). It ensures proper domain name resolution for the system.

Q-41 What is a mount point in Linux?

A **mount point** in Linux is a directory where a filesystem is attached to the system's directory tree. It allows access to files and directories stored on external devices, such as hard drives or USB drives, by linking them to the root filesystem. For example, `/mnt` or `/media` are common mount points.

Q-42 What is a kernel panic?

A **kernel panic** in Linux is a critical error that occurs when the kernel (the core part of the operating system) encounters an issue it cannot recover from, such as a hardware failure, software bug, or corrupted system file. When a kernel panic happens, the system stops, and a message is displayed on the screen, usually requiring a restart. It's similar to a system crash and indicates a severe issue that needs to be resolved.

Q-43 What is a daemon in Linux?

A **daemon** in Linux is a background process that runs without user interaction, typically starting during system boot and continuing to run until the system shuts down. Daemons handle system tasks like managing network connections, logging, and scheduling jobs. They often have names ending with "d", such as `sshd` (SSH daemon) or `httpd` (HTTP daemon).

Q-44 What is the purpose of the `/etc/fstab` file?

The `/etc/fstab` file in Linux is used to define how disk partitions and filesystems are mounted at boot time. It contains information about each filesystem, such as its device, mount point, filesystem type, and mount options. This file helps automate the mounting of filesystems during system startup, ensuring they are available without manual intervention.

Q-45 What is SELinux?

SELinux (Security-Enhanced Linux) is a security module for Linux that provides an additional layer of access control by enforcing mandatory access control (MAC). It restricts programs and users to the minimum privileges required for their tasks, enhancing security. SELinux policies define what actions are allowed for processes and users, preventing unauthorized access and limiting the impact of potential security breaches.

Q-46 What is the purpose of the cron daemon?

The **cron daemon** (`crond`) in Linux is used to schedule and automate tasks at specified times or intervals. It runs in the background and executes commands or scripts at predetermined times, such as daily backups, system updates, or log rotations. Cron jobs are defined in the `crontab` file, where users can specify the timing and the commands to run.

Q-47 Explain the difference between a Linux shell and a kernel.

The **Linux shell** is a command-line interface (CLI) that allows users to interact with the operating system by entering commands. It acts as a user interface, interpreting commands and passing them to the system for execution. Popular shells include Bash, Zsh, and Fish.

The **Linux kernel**, on the other hand, is the core component of the operating system responsible for managing hardware, system resources, and communication between software and hardware. It handles low-level tasks like memory management, process scheduling, and device management.

In short, the shell is the interface for user interaction, while the kernel is the heart of the system that manages the hardware and resources.

Q-48 What is the purpose of OS patching in Linux?

The purpose of **OS patching** in Linux is to update the operating system with security fixes, bug fixes, performance improvements, and new features. Patches are released to address vulnerabilities, prevent security breaches, and ensure the system runs smoothly. Regular patching helps maintain the stability, security, and functionality of the Linux system, protecting it from exploits and ensuring it remains up-to-date with the latest features and fixes.

Q-49 How can you list all running processes on a Linux system?

To list all running processes on a Linux system, you can use the following commands:

1. **ps command:**

- To view all processes running for the current user:
- **ps**
- To view all processes running on the system:
- **ps -ef**

2. **top command:**

- To view processes in real-time with resource usage:
- **top**

3. **htop command** (requires installation):

- A more interactive and user-friendly version of top:
- **htop**

These commands help you view and manage processes running on your Linux system.

Q-50 What is the difference between grep and awk?

The key differences between **grep** and **awk** are:

1. **Purpose:**

- **grep** is used for searching and filtering lines of text that match a specified pattern. It is focused on pattern matching.

- **awk** is a powerful text processing tool that can search for patterns but also allows for advanced text manipulation, including field-based operations, calculations, and data formatting.

2. **Functionality:**

- **grep** simply prints lines that match a pattern. Example:
- `grep "pattern" file.txt`
- **awk** can search for patterns but also perform operations like printing specific columns, performing calculations, and formatting output. Example:
- `awk '{print $1, $2}' file.txt`

3. **Use Cases:**

- **grep** is ideal for simple pattern searching.
- **awk** is better for complex text processing tasks, such as extracting specific columns from a file or performing arithmetic on data fields.

In short, **grep** is for searching, and **awk** is for more complex data extraction and manipulation.

Q-51 You need to find all files larger than 500MB under the /var/log directory. What command will you use?

To find all files larger than 500MB under the /var/log directory, you can use the `find` command with the `-size` option:

`find /var/log -type f -size +500M`

This command searches for regular files (`-type f`) in the /var/log directory that are larger than 500MB (`-size +500M`).

Q-52 A specific service has stopped responding. How will you find its process ID and kill it safely without affecting other processes?

To find the process ID (PID) of a specific service and kill it safely without affecting other processes, follow these steps:

1. **Find the PID:** Use the `ps` command combined with `grep` to search for the service. For example, if the service is `myservice`:
2. **`ps aux | grep myservice`**

This will return a list of processes related to `myservice`, along with their PIDs.

3. **Kill the Process:** Once you have identified the correct PID, use the `kill` command to terminate the process safely. For example, if the PID is 1234:
4. **`sudo kill 1234`**

If the process does not terminate with a normal kill, you can force it with:

`sudo kill -9 1234`

This method ensures you only kill the target service without affecting other running processes.

Q-53 How to identify the processes consuming the most CPU or memory?

To identify the processes consuming the most **CPU** or **memory**, you can use the following commands:

1. Using top:

- To view processes in real-time sorted by CPU or memory usage:
- **top**
- Press M to sort by memory usage or P to sort by CPU usage.

2. Using htop (if installed):

- A more user-friendly and interactive version of top:
- **htop**
- You can sort processes by CPU or memory usage directly from the interface.

3. Using ps:

- To list processes sorted by memory usage:
- **ps aux --sort=-%mem | head -n 10**
- To list processes sorted by CPU usage:
- **ps aux --sort=-%cpu | head -n 10**

These commands help identify the top processes that are consuming CPU or memory resources, allowing you to manage system performance.

Q-54 How to check the free disk space on a Linux machine? If a particular partition (e.g., /var) is 95% full, what steps will you take to resolve it?

To check free disk space on a Linux machine:

Use the **df** command:

```
df -h
```

This will display disk space usage in a human-readable format (e.g., MB or GB) for all mounted filesystems.

Steps to resolve a full partition (e.g., /var 95% full):

1. **Check Disk Usage by Directory:** Use the **du** command to identify which directories are consuming the most space under /var:
2. **sudo du -sh /var/***
3. **Clear Unnecessary Logs:**
 - Check the log files in /var/log and remove or compress large logs.
 - Use **journalctl** to clean up system logs:
 - **sudo journalctl --vacuum-size=100M**

4. **Remove Unused Packages:**

- Use apt-get (Debian/Ubuntu) or yum (CentOS/RedHat) to remove unnecessary packages and free up space.

5. **sudo apt-get autoremove**

6. **sudo apt-get clean**

7. **Move Files to Another Partition:**

- If possible, move large files to another partition with more space.

8. **Resize the Partition** (if applicable):

- If the partition is part of an LVM (Logical Volume Management) setup, you can resize it by extending the volume or adding more disk space.

By identifying the largest files and clearing unnecessary ones, you can effectively free up space in the /var partition.

Q-55 You suspect a service failure occurred last night. How to search for all error messages in the logs from yesterday using a single command?

To search for error messages in the logs from yesterday, you can use the journalctl command (for systems using systemd) and filter the logs by date and severity. Here's how you can do it:

sudo journalctl --since "yesterday" --priority=err

Explanation:

- **--since "yesterday"**: Filters logs from the previous day.
- **--priority=err**: Filters logs for error messages.

This will display all error messages logged since yesterday. If you're using traditional log files (like /var/log/syslog or /var/log/messages), you can use grep:

grep -i "error" /var/log/syslog* | grep "\$(date --date='yesterday' '+%b %d')"

This command searches for error messages in the log files, filtering those that occurred yesterday.

Q-56 A user reports they cannot execute a script named deploy.sh. What steps will you take to fix permissions so only the user and the group can execute it?

To fix the permissions so that only the user and the group can execute the deploy.sh script, follow these steps:

1. **Check current permissions:** First, check the current permissions of the script:
2. **ls -l deploy.sh**

This will show the file permissions in the format:

-rwxr-xr-x 1 user group 1234 Jan 1 12:34 deploy.sh

3. **Modify permissions:** Use the `chmod` command to ensure that only the owner and group have execute permissions. For example:

4. **chmod 750 deploy.sh**

- **7** (for owner): grants read, write, and execute permissions (rwx).
- **5** (for group): grants read and execute permissions (rx).
- **0** (for others): revokes all permissions (---).

5. **Verify the new permissions:** Check the permissions again to ensure the change was applied:

6. **ls -l deploy.sh**

The output should now show:

```
-rwxr-x--- 1 user group 1234 Jan 1 12:34 deploy.sh
```

7. **Check ownership** (if necessary): Ensure the user and group are correct. If not, you can change the owner and group using `chown`:

8. **sudo chown user:group deploy.sh**

Now, the script can only be executed by the user and the group. Other users will not have execution rights.

Q-57 Explain the output of `ls -l` and how file permission bits work.

The output of the `ls -l` command in Linux provides detailed information about files and directories. Here's an example of the output:

```
-rwxr-xr-- 1 user group 1234 Jan 1 12:34 deploy.sh
```

Breakdown of the output:

1. **File type and permissions (-rwxr-xr--):**
 - The first character represents the file type:
 - - = regular file
 - d = directory
 - l = symbolic link
 - The next 9 characters represent the file permissions, broken down into 3 groups of 3 characters:
 - **Owner permissions** (first 3 characters): rwx
 - r = read permission (owner can read the file).
 - w = write permission (owner can modify the file).
 - x = execute permission (owner can run the file as a program).
 - **Group permissions** (next 3 characters): r-x

- r = read permission (group members can read the file).
 - - = no write permission (group members cannot modify the file).
 - x = execute permission (group members can run the file).
 - **Other permissions** (last 3 characters): r--
 - r = read permission (others can read the file).
 - - = no write permission (others cannot modify the file).
 - - = no execute permission (others cannot run the file).
2. **Number of hard links** (1):
 - The number of hard links to the file. In this case, there is 1 hard link.
 3. **Owner** (user):
 - The name of the user who owns the file.
 4. **Group** (group):
 - The name of the group associated with the file.
 5. **File size** (1234):
 - The size of the file in bytes.
 6. **Modification date** (Jan 1 12:34):
 - The last modification date and time of the file.
 7. **Filename** (deploy.sh):
 - The name of the file or directory.

Q-58 You are asked to schedule a script to run every day at 3:00 AM. How to set up a cron job for this task?

To schedule a script to run every day at 3:00 AM using cron, follow these steps:

1. **Open the crontab file** for editing:
2. **crontab -e**
3. **Add the cron job** at the end of the crontab file:
4. **0 3 * * * /path/to/your/script.sh**

Explanation:

- 0: The minute when the job will run (0th minute).
- 3: The hour when the job will run (3 AM).
- *: Any day of the month.
- *: Any month.

- *: Any day of the week.

5. **Save and exit** the editor (for example, in vi, press Esc, then type :wq).

Key Points:

- This cron job will run /path/to/your/script.sh every day at 3:00 AM.
- Make sure the script has the correct permissions to execute:
- `chmod +x /path/to/your/script.sh`
- Ensure the path is absolute, and if necessary, use the full path for the script and any commands inside the script.

This will set up the cron job to automatically execute the script daily at 3:00 AM.

Q-59 You need to find all .log files modified in the last 7 days and archive them into a single .tar.gz file. What commands would you use?

To find all .log files modified in the last 7 days and archive them into a single .tar.gz file, you can use the following commands:

1. **Find the .log files modified in the last 7 days:**
2. `find /path/to/search -type f -name "*.log" -mtime -7`
 - **/path/to/search:** Replace with the directory where you want to search for .log files.
 - **-type f:** Ensures only files are selected (not directories).
 - **-name "*.log":** Filters files ending with .log.
 - **-mtime -7:** Finds files modified in the last 7 days.
3. **Archive the files into a .tar.gz file:** You can combine the find command with tar to create the archive:
4. `find /path/to/search -type f -name "*.log" -mtime -7 | tar -czvf archive_name.tar.gz -T -`
 - **tar -czvf archive_name.tar.gz:** Creates a compressed .tar.gz archive.
 - **-T -:** Tells tar to read the file list from standard input (which is provided by the find command).

Full Explanation:

- The find command searches for all .log files modified in the last 7 days.
- The tar command then compresses these files into a single .tar.gz archive.

This will create an archive named archive_name.tar.gz containing all .log files modified in the last 7 days.

If you want to first find the .log files and then tar them in a separate command, follow these steps:

Step 1: Find the .log files modified in the last 7 days and save the list to a file:

```
find /path/to/search -type f -name "*.log" -mtime -7 > log_files.txt
```

- This command will search for .log files modified in the last 7 days and save the list of these files in a file called log_files.txt.

Step 2: Create a tar archive of the found .log files:

```
tar -czvf archive_name.tar.gz -T log_files.txt
```

- **-T log_files.txt:** Tells tar to read the list of files from log_files.txt and add them to the archive.
- **archive_name.tar.gz:** The name of the resulting compressed tar archive.

Summary:

1. The first command generates the list of .log files modified in the last 7 days.
2. The second command tars those files into a .tar.gz archive.

This way, you can review or modify the list of files before creating the archive.

Q-60 A developer accidentally deleted an important file. You don't have a backup, but you suspect it is still open in a running process. How to recover the file?

If a file is deleted but still open in a running process, it can often be recovered because the process may still have the file descriptor open, keeping the file's data in memory. Here's how you can attempt to recover the file:

Steps to recover a deleted file:

1. **Identify the process using the deleted file:** You can use the lsof (List Open Files) command to identify processes that still have the file open. For example:
2. **lsof | grep deleted**

This will list all open files with "deleted" in their names. You need to look for the file you are trying to recover.

3. **Locate the process ID (PID):** In the output of lsof, note the **PID** of the process that is using the deleted file.
4. **Copy the file descriptor to a new file:** Once you identify the process, you can copy the contents of the file from the open file descriptor to a new file. Use the cp command with the file descriptor to recover it:
5. **cp /proc/<PID>/fd/<fd> /path/to/recovered_file**
 - **<PID>:** Replace with the process ID from the previous step.
 - **<fd>:** Replace with the file descriptor number associated with the deleted file.
 - **/path/to/recovered_file:** Specify the path and name for the recovered file.

6. **Alternative approach (if necessary):** If you cannot directly copy the file descriptor, you can also try using a tool like **testdisk** or **extundelete** (for ext3/ext4 file systems) to attempt recovery, but these methods are typically used for recovering deleted files from disk rather than from an open file descriptor.

Key Notes:

- **Time sensitivity:** The sooner you attempt recovery, the better. If the file is overwritten by other processes, recovery becomes much more difficult.
- **Disk usage:** If the file was large and the disk is actively being written to, it may overwrite the deleted data, making recovery impossible.

By copying the file from the open file descriptor before the process closes the file, you can recover the data even if it has been deleted from the filesystem.

Q-61 Check the current system load and CPU usage in Linux.

To check the current system load and CPU usage in Linux:

1. **top:**
2. **top**
 - Shows real-time system load and CPU usage.
3. **uptime:**
4. **uptime**
 - Displays system load averages for the last 1, 5, and 15 minutes.
5. **mpstat -P ALL** (from sysstat package):
6. **mpstat -P ALL**
 - Shows CPU usage for each core.
7. **vmstat 1:**
8. **vmstat 1**
 - Displays CPU and memory stats every second.
9. **htop:**
10. **htop**
 - Provides an interactive, colorful view of CPU and system stats.

Q-62 Purpose of the top and htop commands.

Both are used to monitor system processes and resource usage. htop is a more user-friendly, interactive version of top.

Q-63 Check memory usage on your Linux system

Use free -h or top to view memory usage.

Q-64 Write a shell script to back up files.

Example script:

```
#!/bin/bash
```

```
cp -r /path/to/source/* /path/to/destination/
```

Q-65 Purpose of the `#!/bin/bash` line at the beginning of a script.

The `#!/bin/bash` line at the beginning of a script is called a **shebang**. It tells the system which interpreter to use to execute the script.

- **#!/bin/bash:** Specifies that the script should be run using the **Bash shell**.
- It allows the script to be executed directly without needing to manually invoke Bash.

This line ensures the script runs with the correct shell, even if it's executed from another shell or environment.

Q-66 Create variables in a shell script.

To create a variable, simply assign a value: `variable_name="value"`. Refer to it as `$variable_name`.

Q-67 Difference between apt-get, yum, and dnf.

Here's a short comparison of apt-get, yum, and dnf:

1. **apt-get:**
 - **Used on:** Debian-based distributions (e.g., Ubuntu, Debian).
 - **Purpose:** A command-line tool for handling packages in .deb format.
 - **Key feature:** Handles installation, removal, and updates of software packages.
2. **yum:**
 - **Used on:** Older Red Hat-based distributions (e.g., CentOS, RHEL 7).
 - **Purpose:** A package manager for .rpm packages.
 - **Key feature:** Handles package management, repositories, and dependencies.
3. **dnf:**
 - **Used on:** Newer Red Hat-based distributions (e.g., CentOS 8, RHEL 8, Fedora).
 - **Purpose:** A more modern package manager, a replacement for yum.
 - **Key feature:** Faster, more efficient, and uses fewer resources compared to yum. Handles .rpm packages.

Summary:

- **apt-get:** Debian-based systems.
- **yum:** Older Red Hat-based systems.

- **dnf**: Newer Red Hat-based systems.

Q-68 Install a package in a Debian-based system and in a RedHat-based system.

To install a package:

1. **On a Debian-based system (e.g., Ubuntu):**
2. **sudo apt-get install package-name**
3. **On a Red Hat-based system (e.g., CentOS, RHEL):**
4. **sudo yum install package-name**

Or, on newer Red Hat-based systems (using dnf):

sudo dnf install package-name

Replace package-name with the name of the package you want to install.

Q-69 Update all packages on a Linux system.

To update all packages on a Linux system:

1. **On a Debian-based system (e.g., Ubuntu):**
2. **sudo apt-get update && sudo apt-get upgrade**
3. **On a Red Hat-based system (e.g., CentOS, RHEL):**
4. **sudo yum update**

Or, on newer Red Hat-based systems (using dnf):

sudo dnf update

These commands will update all installed packages to their latest available versions.

Q-70 Different types of file permissions in Linux:

Read (r): Permission to view the file content.

Write (w): Permission to modify the file.

Execute (x): Permission to execute the file.

Q-71 Secure SSH connections in Linux.

To secure SSH connections in Linux:

1. **Use Strong Passwords or SSH Keys:**
 - Prefer SSH key-based authentication over passwords.
 - Generate SSH keys using ssh-keygen and configure them in ~/.ssh/authorized_keys.
2. **Disable Root Login:**
 - Edit /etc/ssh/sshd_config and set:

- **PermitRootLogin no**
 - **Restart SSH: sudo systemctl restart sshd**
- 3. Change the Default SSH Port:**
- **Edit /etc/ssh/sshd_config and set a custom port:**
 - **Port 2222**
 - **Restart SSH: sudo systemctl restart sshd**
- 4. Use Firewall to Restrict Access:**
- Allow only trusted IP addresses through the firewall:
 - **sudo ufw allow from <trusted-ip> to any port 22**

Q-72 Where are system logs stored in Linux

Logs are stored in the `/var/log/` directory, with files like `/var/log/syslog`, `/var/log/auth.log`, etc.

Q-73 View logs in real-time

Use **tail -f /var/log/syslog** to view logs as they are written.

Q-74 Search for a specific event in system logs:

Use **grep** to search, for example: **grep "event" /var/log/syslog** to search for the event in syslog.

Q-75 What is process management?

Process management in Linux involves handling processes through creation, scheduling, termination, and resource allocation. It ensures processes run efficiently by assigning them resources, managing their states (running, waiting, etc.), and enabling communication between them.

Q-76 What is Zombie process?

A **zombie process** is a process that has completed execution but still has an entry in the process table. This happens because its parent process hasn't read its exit status. The process is essentially dead but hasn't been fully cleaned up yet.

Q-77 How do add a user to an Existing Group In Linux?

To add a user to an existing group in Linux, use the following command:

sudo usermod -aG groupname username

- **groupname:** The name of the existing group.
- **username:** The name of the user you want to add.

The **-aG** option appends the user to the group without removing them from other groups.

Q-78 What does the top command Display?

The **top** command displays real-time information about system processes, including:

- **Process ID (PID)** and **process name**.

- **CPU usage, memory usage, and swap usage.**
- **System load average** over the last 1, 5, and 15 minutes.
- **Uptime, number of users, and current time.**

It updates continuously to show the most resource-intensive processes.

Q-79 What is the use of lsblk command in Linux?

The lsblk command in Linux is used to display information about all block devices (e.g., hard drives, SSDs, USB drives) on the system. It shows details like:

- **Device name** (e.g., /dev/sda)
- **Size**
- **Type** (disk, partition, etc.)
- **Mount point**

It helps in identifying and managing storage devices.

Q-80 How to do Disk partition in Ubuntu?

To partition a disk in Ubuntu:

1. **Open Terminal** and use fdisk for MBR or gdisk for GPT partitions:
2. **sudo fdisk /dev/sdX** #Replace sdX with your disk (e.g., sda, sdb)
3. **Create a new partition:**
 - Type n to create a new partition.
 - Choose the partition type (primary or extended) and specify the size.
4. **Change partition type** (optional):
 - Type t and choose the partition type (e.g., Linux, swap).
5. **Write the changes:**
 - Type w to write the changes to disk.
6. **Format the partition:**
7. **sudo mkfs.ext4 /dev/sdX1** #Replace with your partition (e.g., /dev/sda1)
8. **Mount the partition:**
9. **sudo mount /dev/sdX1 /mnt**
10. **Add to /etc/fstab** (optional for auto-mount):
 - Edit /etc/fstab to add the partition for auto-mounting at boot.

This completes the basic disk partitioning process in Ubuntu.

Q-81 what is the linux boot process?

The Linux boot process involves several stages:

1. **BIOS/UEFI:**
 - Initializes hardware and loads the bootloader from the MBR (Master Boot Record) or UEFI partition.
2. **Bootloader (GRUB):**
 - GRUB loads the kernel (Linux) into memory and passes control to it.
3. **Kernel Initialization:**
 - The kernel sets up hardware, mounts the root filesystem, and starts essential system processes.
4. **Init System (Systemd):**
 - Systemd or another init system is launched to manage system services and processes.
5. **Runlevel/Target:**
 - The system enters a specific runlevel or target (e.g., multi-user, graphical).
6. **Login:**
 - User login prompt appears for authentication.

This is the basic sequence of events during the Linux boot process.

Q-82 What is a Linux filesystem? and what is /etc and /bin?

A **Linux filesystem** is a hierarchical structure used to organize and store files on a disk. It manages how data is stored, accessed, and organized, using directories and files.

- **/etc:** Contains system configuration files. These files control the system's behavior, settings, and services.
 - Example: /etc/passwd (user account information).
- **/bin:** Contains essential system binaries (executables) required for system booting and repair, accessible for all users.
 - Example: /bin/bash (shell command).

Q-83 What are Crontab and Cronjob?

- **Crontab:** A file that defines the schedule of tasks to be run automatically by the cron daemon. Each user can have their own crontab file.
- **Cronjob:** A scheduled task defined in the crontab file. It runs automatically at specified intervals, such as daily, weekly, or monthly.

For example, a cron job can be used to run a backup script every night at 2 AM.

Q-84 What is virtualization?

Virtualization is the creation of virtual (rather than physical) versions of resources like servers, storage devices, or operating systems. It allows a single physical machine to run multiple virtual machines (VMs), each with its own OS and applications, sharing the underlying hardware.

Benefits:

- Efficient resource utilization.
- Isolation between VMs.
- Easier management and scaling.

Q-85 How DNS works?

DNS (Domain Name System) translates human-readable domain names (e.g., www.example.com) into IP addresses that computers use to communicate.

How it works:

1. **DNS Query:** When you type a domain name into your browser, it sends a request to a DNS resolver.
2. **Recursive Resolver:** The resolver checks its cache; if not found, it asks a **root DNS server**.
3. **Root DNS Server:** Directs the request to a **TLD (Top-Level Domain)** server (e.g., .com).
4. **TLD Server:** Points to the **Authoritative DNS server** for the specific domain.
5. **Authoritative DNS Server:** Provides the correct IP address for the domain.
6. **Response:** The resolver sends the IP back to your browser, which connects to the website using that IP.

This entire process happens in milliseconds.

Q-86 What is Apache?

Apache (Apache HTTP Server) is an open-source web server software used to serve web content. It processes requests from clients (browsers) and delivers web pages, files, and other content over HTTP.

Key features:

- Handles dynamic and static web content.
- Supports various programming languages (PHP, Python, etc.).
- Highly configurable and customizable.
- Popular for hosting websites and web applications.

Q-87 What is PHPAdmin?

phpMyAdmin is a free, open-source web-based application used to manage MySQL and MariaDB databases. It provides a user-friendly interface to perform database operations like creating, modifying, and deleting databases, tables, and records.

Key features:

- **SQL querying:** Allows execution of SQL queries directly.
- **User management:** Manage MySQL user permissions.
- **Import/Export:** Import and export databases in various formats (e.g., SQL, CSV).
- **Multi-language support:** Available in many languages.

It's commonly used by developers for database management in a web environment.

Q-88 Do you know about firewalls?

Yes! A **firewall** is a network security system designed to monitor and control incoming and outgoing network traffic based on predetermined security rules. Its primary purpose is to establish a barrier between a trusted internal network and untrusted external networks, such as the internet.

Types of Firewalls:

1. **Network Firewalls:** Filter traffic between networks (e.g., between a local network and the internet).
2. **Host-based Firewalls:** Run on individual devices (servers, computers) to filter traffic entering or leaving that device.

Types of Filtering:

- **Packet Filtering:** Analyzes packets based on predefined rules (IP, port, protocol).
- **Stateful Inspection:** Tracks the state of active connections and ensures only valid packets are passed.
- **Proxy Firewalls:** Intercept and evaluate requests, acting as an intermediary between the client and the server.

Firewall Configuration in Linux:

- **iptables:** Used for packet filtering and managing network traffic.
- **firewalld:** A frontend for managing firewall rules with zones for different network types.

Firewalls protect systems from unauthorized access, cyberattacks, and malware.

Q-89 What is an SSL certificate? How we can create the SSL certificate and where we will do the configurations in apache for the SSL certificate?

An **SSL certificate** (Secure Sockets Layer) is used to establish an encrypted connection between a web server and a client (browser), ensuring data privacy and security during transmission. It also verifies the authenticity of the website.

How to Create an SSL Certificate:

1. **Generate a Private Key:**
2. `openssl genpkey -algorithm RSA -out /etc/ssl/private/server.key`

3. **Create a Certificate Signing Request (CSR):**
4. `openssl req -new -key /etc/ssl/private/server.key -out /etc/ssl/csr/server.csr`
5. **Obtain SSL Certificate:**
 - Send the CSR to a Certificate Authority (CA) to obtain the SSL certificate (or generate a self-signed certificate for testing):
6. `openssl req -x509 -key /etc/ssl/private/server.key -in /etc/ssl/csr/server.csr -out /etc/ssl/certs/server.crt -days 365`

Configuring SSL in Apache:

1. **Enable SSL Module:**
2. `sudo a2enmod ssl`
3. **Configure SSL in Apache:**
 - Edit the SSL configuration file (typically `/etc/apache2/sites-available/default-ssl.conf` or create a new one).
 - Set paths to the certificate and key files:
4. `SSLEngine on`
5. `SSLCertificateFile /etc/ssl/certs/server.crt`
6. `SSLCertificateKeyFile /etc/ssl/private/server.key`
7. **Enable the SSL Site:**
8. `sudo a2ensite default-ssl.conf`
9. **Restart Apache:**
10. `sudo systemctl restart apache2`

This configures SSL for encrypted communication in Apache.

Q-90 What is the path of apache configuration files?

The **Apache configuration files** are typically located in the following paths on a Linux system:

1. **Main configuration file:**
 - `/etc/httpd/conf/httpd.conf` (for RedHat/CentOS)
 - `/etc/apache2/apache2.conf` (for Debian/Ubuntu)
2. **Additional configuration directories:**
 - `/etc/httpd/conf.d/` (for RedHat/CentOS) – Directory for additional configurations, including SSL and virtual hosts.
 - `/etc/apache2/sites-available/` (for Debian/Ubuntu) – Directory for individual site configurations.

Q-91 What are the different commands available to check the disk usage?

Here are several commands to check disk usage in Linux:

1. **df** (Disk Free):
 - Shows the amount of free and used disk space on all mounted filesystems.
 - Example:
 - **df -h**
2. **du** (Disk Usage):
 - Displays the disk space used by files and directories.
 - Example:
 - **du -sh /path/to/directory**
3. **lsblk** (List Block Devices):
 - Shows block devices, including their size, mount points, and types.
 - Example:
 - **lsblk**
4. **fdisk -l**:
 - Lists disk partitions and their sizes.
 - Example:
 - **sudo fdisk -l**
5. **parted**:
 - A command-line tool to manage disk partitions and also shows disk usage.
 - Example:
 - **sudo parted -l**

These commands help monitor disk space usage and manage disks effectively.

Q-92 What are the different communication commands available in Unix/Shell?

Here are some common communication commands available in Unix/Shell:

1. **echo**:
 - Displays a message or output to the terminal.
 - Example:
 - **echo "Hello, World!"**
2. **write**:
 - Sends a message to another user logged into the same system.
 - Example:

- **write username**
- 3. **wall:**
 - Sends a message to all users on the system.
 - Example:
 - **wall "System maintenance at 2 PM"**
- 4. **talk:**
 - Initiates a real-time, two-way chat with another user.
 - Example:
 - **talk username**
- 5. **mail:**
 - Sends email messages from the command line.
 - Example:
 - **mail user@example.com**
- 6. **netcat (nc):**
 - A versatile network tool that can be used for communication between systems.
 - Example:
 - **nc -l 1234 # Listens on port 1234**
- 7. **scp** (Secure Copy Protocol):
 - Transfers files between systems securely over SSH.
 - Example:
 - **scp file.txt username@remote:/path/to/destination**
- 8. **rsync:**
 - A command used for syncing files and directories between two locations.
 - Example:
 - **rsync -avz /source/dir /destination/dir**

These commands are used for messaging, file transfer, and communication over networks in Unix/Shell environments.

Q-93 How to find out the total disk space used by a specific user, say for example username is John?

To find the total disk space used by a specific user (e.g., John), you can use the **du** command combined with **find** to target the user's directories.

Here's the command:

du -sh /home/John

Explanation:

- **du:** Disk usage command.
- **-s:** Summarize the total space used.
- **-h:** Human-readable format (e.g., MB, GB).

If the user's files are scattered across the system and you need to find files owned by John in all directories, you can use **find**:

find / -user John -exec du -ch {} + | grep total\$

This command:

- **find / -user John:** Finds all files owned by the user John.
- **du -ch:** Shows the disk usage of each file, then the total.
- **grep total\$:** Filters and shows the total disk usage.

These commands will help you determine the total disk usage by the user.

Q-94 What is Run level in Linux

A **runlevel** in Linux defines the state or mode of the system, specifying which processes and services are running. Different runlevels correspond to different system states, such as single-user mode or multi-user mode with networking.

Q-95 What are the different runlevels, and what do they mean?

Here are the different **Linux runlevels** and their meanings:

1. **Runlevel 0: Halt** – Shuts down the system.
2. **Runlevel 1: Single-user mode** – For system maintenance; no networking.
3. **Runlevel 2: Multi-user mode without networking** – Multiple users can log in, but no networking services.
4. **Runlevel 3: Multi-user mode with networking** – Allows multiple users with networking enabled (common for servers).
5. **Runlevel 4: Unused/Custom** – Reserved for custom configurations (not typically used).
6. **Runlevel 5: Multi-user mode with GUI** – Multi-user mode with a graphical user interface (common for desktops).
7. **Runlevel 6: Reboot** – Reboots the system.

These runlevels define which services are running at a given time on the system. Modern Linux distributions using **systemd** use **targets**, which are similar to runlevels.

Q-96 How can you change runlevels in Linux?

You can change runlevels in Linux using the following commands:

1. **init command:**

- To change to a specific runlevel, use:
- **sudo init <runlevel>**

Example:

sudo init 3 #Switches to runlevel 3 (multi-user with networking)

2. **telinit command:**

- telinit is another command used to change the runlevel:
- **sudo telinit <runlevel>**

Example:

sudo telinit 5 #Switches to runlevel 5 (multi-user with GUI)

In modern systems using **systemd**, runlevels are replaced by **targets**, but you can still use the **systemctl** command to change the system state:

3. **systemctl command:**

- Example to change to a target:
- **sudo systemctl isolate multi-user.target** #Similar to runlevel 3

These methods allow you to change the system's state and services.

Q-97 How can you check the current runlevel?

To check the current runlevel in a Linux system, you can use the following commands:

1. **runlevel command:**
2. **runlevel**

This will display the previous and current runlevels.

3. **who -r command:**
4. **who -r**

This will display the current runlevel as well as other related information.

5. **systemctl command** (for systems using systemd):
6. **systemctl get-default**

This will show the default target (similar to the runlevel) in systems using **systemd**.

These commands will help you find the current runlevel or target in your system.

Q-98 What is the difference between runlevels 2 and 3 in Linux?

The main difference between **runlevel 2** and **runlevel 3** in Linux is the availability of **networking**:

1. **Runlevel 2:**

- **Multi-user mode without networking.**
- Multiple users can log in, but **networking services are not started.**
- Typically used in environments where networking is not required (e.g., isolated systems).

2. Runlevel 3:

- **Multi-user mode with networking.**
- Multiple users can log in, and **networking services are enabled.**
- Common for servers where networking is essential.

In summary, **runlevel 2** does not have networking, while **runlevel 3** includes networking.

Q-99 What's the purpose of runlevel 6, and when should it be used?

Runlevel 6 is used to **reboot** the system. When the system is in runlevel 6, it will restart the machine.

Purpose:

- **Reboots the system** cleanly and gracefully.

When to use:

- Use runlevel 6 when you need to **restart** the system, either after making significant system changes or during regular maintenance or updates that require a reboot.

You can switch to runlevel 6 using the `init` or `telinit` command:

sudo init 6

In modern systems with **systemd**, you can use the command:

sudo systemctl reboot

Q-100 How would you configure a static IP address on a Linux system?

For **Red Hat** and **CentOS** servers, follow these steps to configure a static IP address:

Steps:

1. Edit network configuration file:

- Open the configuration file for your network interface:
- `sudo vi /etc/sysconfig/network-scripts/ifcfg-eth0`

2. Configure static IP:

- Set the following parameters:
- `BOOTPROTO=none`
- `ONBOOT=yes`
- `IPADDR=192.168.1.100`

- NETMASK=255.255.255.0
- GATEWAY=192.168.1.1
- DNS1=8.8.8.8

3. Restart network service:

- Apply changes by restarting the network service:
- `sudo systemctl restart network`

This configures a static IP on your CentOS or RHEL system.

QUESTION

What is Linux and why is it so popular?

ANSWER

Linux is an operating system based on UNIX, and was first introduced by Linus Torvalds commonly used in servers. It is based on the Linux Kernel, and can run on different hardware platforms manufactured by Intel, MIPS, HP, IBM, SPARC and Motorola. Another popular element in Linux is its mascot, a penguin figure named Tux.

Popularity of Linux is because of the following reasons

- It is free and open source. We can download Linux for free and customize it as per our needs.
- It is very robust and adaptable.
- Immense amount of libraries and utilities

QUESTION

What is BASH?

ANSWER

BASH is short for Bourne Again SHell. It was written by Steve Bourne as a replacement to the original Bourne Shell (represented by `/bin/sh`). It combines all the features from the original version of Bourne Shell, plus additional functions to make it easier and more convenient to use. It has since been adapted as the default shell for most systems running Linux.

QUESTION

What is the core of Linux Operating System?

ANSWER

The core of the Linux operating system is Kernel. It is broken down into Shell, Command, Script, and Terminal. Shell is a command Line Interpreter, Command is user Instruction to Computer, Script is collection of commands stored in a file, and Terminal is a command Line Interface.

QUESTION

What is the basic difference between UNIX and Linux Operating System?

ANSWER

Linux is free and open-source software (allowing programmers to program with Linux not around it), the kernel of which is created by Linus Torvalds and community. UNIX, on the other hand, is UNIX is copyrighted name only big companies are allowed to use the UNIX copyright and name, so IBM AIX and Sun Solaris and HP-UX all are UNIX operating systems.

QUESTION

What is LILO?

ANSWER

LILO is a boot loader for Linux. It is used mainly to load the Linux operating system into main memory so that it can begin its operations.

QUESTION

What is an INODE?

ANSWER

All files have its description stored in a structure called 'inode'. The inode contains info about the file-size, access and modification time, permission and so on. In addition to descriptions about the file, the inode contains pointers to the data blocks of the file.

QUESTION

What is a swap space?

ANSWER

A swap space is a certain amount of space used by Linux to temporarily hold some programs that are running concurrently. This happens when RAM does not have enough memory to hold all programs that are executing.

QUESTION

What is the advantage of open source?

ANSWER

Open source allows you to distribute your software, including source codes freely to anyone who is interested. People would then be able to add features and even debug and correct errors that are in the source code. They can even make it run better, and then redistribute these enhanced source code freely again. This eventually benefits everyone in the community.

QUESTION

What is the basic difference between BASH and DOS?

ANSWER

The key differences between the BASH and DOS console lies in 3 areas:

- BASH commands are case sensitive while DOS commands are not;
- under BASH, / character is a directory separator and \ acts as an escape character. Under DOS, / serves as a command argument delimiter and \ is the directory separator
- DOS follows a convention in naming files, which is 8 character file name followed by a dot and 3 character for the extension. BASH follows no such convention.

QUESTION

What is the difference between TCP and UDP?

ANSWER

The basic difference is that TCP establishes a connection before sending data and this allows it to control the dataflow and guarantee that all packets get delivered. UDP simply chucks datagrams onto the wire and if some get lost or arrive in bad order there's no way to request a resend. However UDP has low network overhead so some services such as DNS resolution, SNMP, DHCP, RIP and VOIP use UDP for its speed and any errors are usually dealt with on the application layer rather than network layer.

QUESTION

What is the importance of the GNU project?

ANSWER

This so-called Free software movement allows several advantages, such as the freedom to run programs for any purpose and freedom to study and modify a program to your needs. It also allows you to redistribute copies of a software to other people, as well as freedom to improve software and have it released to the public.

QUESTION

How does DNS resolution work?

ANSWER

A client application requests an IP address from the name server usually by connecting to UDP port 53. The name server will attempt to resolve the FQDN based on its resolver library, which may contain authoritative information about the host requested or cached data about that name from an earlier query. If the name server does not already have the answer, it will turn to root name servers to determine the authoritative for the FQDN in question. Then, with that information, it will query the authoritative name servers for that name to determine the IP address.

QUESTION

Describe the root account.

ANSWER

The root account is like a systems administrator account, and allows you full control of the system. Here you can create and maintain user accounts, assigning different permissions for each account. It is the default account every time you install Linux.

QUESTION

What is CLI?

ANSWER

CLI is short for Command Line Interface. This interface allows user to type declarative commands to instruct the computer to perform operations. CLI offers an advantage in that there is greater flexibility. However, other users who are already accustomed with using GUI find it difficult to remember commands including attributes that come with it.

QUESTION

What is GUI?

ANSWER

GUI, or Graphical User Interface, makes use of images and icons that users click and manipulate as a way of communicating with the computer. Instead of having to remember and type commands, the use of graphical elements makes it easier to interact with the system, as well as adding more attraction through images, icons and colors.

QUESTION

How can you find out how much memory Linux is using?

ANSWER

From a command shell, use the “concatenate” command: `cat /proc/meminfo` for memory usage information. You should see a line starting something like: `Mem: 64655360`, etc. This is the total memory Linux thinks it has available to use.

QUESTION

What is an MX record?

ANSWER

An MX record numerically ranks the mail servers you would prefer to receive email for a domain. The MX record with the lowest number is preferred over the others, but you can set multiple email servers with the same value for simple load balancing.

QUESTION

Please describe the Linux boot-up sequence.

ANSWER

There are seven steps to the boot-up sequence. 1) BIOS (basic input/output system) – executes the MBR where Boot Loader sits, 2) MBR- Master boot reads Kernel into memory, 3) GRUB (Grand Unified Bootloader) Kernel starts Init process, 4) Kernel – Kernel executes the `/sbin/init` program. Init reads `inittab`, executes `rc.sysinit`, 5) Init – the rc script then starts services to reach the default run level and 6) Run level programs – these programs are executed from `/etc/rc.d/rc*.dl/`

QUESTION

What is typical size for a swap partition under a Linux system?

ANSWER

The preferred size for a swap partition is twice the amount of physical memory available on the system. If this is not possible, then the minimum size should be the same as the amount of memory installed.

QUESTION

How do you search for a pattern and then replace it in an entire file?

ANSWER

You use Sed, or in Vi editor, the search uses character 's' slash the pattern to be searched, slash the pattern to replace it with, slash 'g' which stands for entire file.

QUESTION

Does the Ctrl+Alt+Del key combination work on Linux?

ANSWER

Yes, it does. Just like Windows, you can use this key combination to perform a system restart. One difference is that you won't be getting any confirmation message and therefore, reboot is immediate.

QUESTION

How do you list and flush all Iptables?

ANSWER

First you use the `-L` switch to view all the currently present rules and then `-F` to flush them.

QUESTION

How do you change permissions under Linux?

ANSWER

Assuming you are the system administrator or the owner of a file or directory, you can grant permission using the `chmod` command. Use `+` symbol to add permission or `-` symbol to deny permission, along with any of the following letters: `u` (user), `g` (group), `o` (others), `a` (all), `r` (read), `w` (write) and `x` (execute). For example the command `chmod go+rw FILE1.TXT` grants read and write access to the file `FILE1.TXT`, which is assigned to groups and others.

QUESTION

What is a shell? What are their names?

ANSWER

The shell is the part of the system with which the user interacts. A Unix shell interprets commands such as “pwd”, “cd” or “traceroute” and sends the proper instructions to the actual operating system itself. The shells currently available are Ans SH, BASH, CSH, TCSH, NOLOGIN, KSH. Other functions of a shell include scripting capability, path memory, multitasking, and file handling.

QUESTION

In Linux, what names are assigned to the different serial ports?

ANSWER

Serial ports are identified as `/dev/ttyS0` to `/dev/ttyS7`. These are the equivalent names of COM1 to COM8 in Windows.

QUESTION

What is a zombie?

ANSWER

Cheeky answers get bonus points for this one. But in the Linux world, a zombie process is the process output of 'ps' by the presence of 'Z' in the STAT column. Zombies are essentially the premature processes whose mature parent processes died without reaping its children. Note that zombies can't be killed with the usual 'kill' signal.

QUESTION

How do you access partitions under Linux?

ANSWER

Linux assigns numbers at the end of the drive identifier. For example, if the first IDE hard drive had three primary partitions, they would be named/numbered, /dev/hda1, /dev/hda2 and /dev/hda3.

QUESTION

You need to define a macro, a key binding for the existing command. How would you do it?

ANSWER

There is a command called `bind`, in bash shell which is capable of defining macro, or binding a key. In order to bind a key with an existing command, we need to generate Character Sequence emitted by the key. Press `Ctrl+v` and then key `F12`, I got `^[[24~`

QUESTION

Your assistant needs to print directory stack, what would you suggest?

ANSWER

The Linux command 'dirs', will print the directory stack.

For Example: [root@localhost ~]# dirs

/usr/share/X11

QUESTION

What is the maximum length for a filename under Linux?

ANSWER

Any filename can have a maximum of 255 characters. This limit does not include the path name, so therefore the entire pathname and filename could well exceed 255 characters.

QUESTION

What does the command 'hash' is used for in bash Shell?

ANSWER

Linux command 'hash' manages internal hash table, finds and remembers full path of the specified command, displays used command names and number of times the command is used.

```
[root@localhost ~]# hash
```

```
hits  command
```

```
2    /bin/l
```

```
2    /bin/su
```

QUESTION

Which built-in Linux command performs arithmetic operation of Integers in Bash?

ANSWER

The 'let' command that performs, arithmetic operation of integer in bash shell.

```
#!/bin/bash
```

```
...
```

```
...
```

```
let c=a+b
```

```
...
```

```
...
```

QUESTION

Explain virtual desktop.

ANSWER

This serves as an alternative to minimizing and maximizing different windows on the current desktop. Using virtual desktops, each desktop is a clean slate where you can open one or more programs. Rather than minimizing/restoring all those programs as needed, you can simply shuffle between virtual desktops with programs intact in each one.

QUESTION

Differentiate multiuser from multitask.

ANSWER

Multiuser means that more than one person can use the computer at the same time. Multitask means that even a single user can have the computer work on more than one task or program at the same time.

QUESTION

You have a large text file, and you need to see one page at a time. What will you do?

ANSWER

You can achieve the above result by pipeling the output of 'cat file_name.txt' with 'more' command.

```
[root@localhost ~]# cat file_name.txt | more
```

QUESTION

Who own the data dictionary?

ANSWER

The user 'SYS' owns the data dictionary. Users 'SYS' and 'SYSEM' are created by default, automatically.

QUESTION

How do you share a program across different virtual desktops under Linux?

ANSWER

To share a program across different virtual desktops, in the upper left-hand corner of a program window look for an icon that looks like a pushpin. Pressing this button will “pin” that application in place, making it appear in all virtual desktops, in the same position onscreen.

QUESTION

What command should you use to check the number of files and disk space used by each user's defined quotas?

ANSWER

The command 'repquota' comes to rescue here. Command repquota summaries quotas for a file system.

QUESTION

What are daemons?

ANSWER

Daemons are services that provide several functions that may not be available under the base operating system. Its main task is to listen for service request and at the same time to act on these requests. After the service is done, it is then disconnected and waits for further requests.

QUESTION

You have lots of running jobs, how would you remove all the running processes, without restarting the machine?

ANSWER

The Linux command 'disown -r' will remove all the running Processes.

QUESTION

A user is new to Linux and he wants to know full list of available commands, what would you suggest him?

ANSWER

A command 'compgen -c' will show a full list of available commands.

```
[root@localhost ~]$ compgen -c
```

I.	else	select
II	elif	while
Is	fi	until
which	case	do
if	esac	done
then	for	...

QUESTION

What are the kinds of permissions under Linux?

ANSWER

There are 3 kinds of permissions under Linux:

- Read: users may read the files or list the directory
- Write: users may write to the file or new files to the directory
- Execute: users may run the file or lookup a specific file within a directory

QUESTION

You need to track events on your system. What will you do?

ANSWER

For tracking the events on the system, we need a daemon called syslogd. The syslogd daemon is useful in tracking the information of system and then saving it to specified log files.

Running 'syslogd' application in terminal generates log file at the location '/var/log/syslog'. The syslogd application is very useful in troubleshooting Linux systems. A sample log file looks similar to below.

QUESTION

How will you restrict IP so that the restricted IP's may not use the FTP Server?

ANSWER

We can block suspicious IP by integrating tcp_wrapper. We need to enable the parameter "tcp_wrapper=YES" in the configuration file at '/etc/vsftpd.conf'. And then add the suspicious IP in the 'host.deny' file at location '/etc/host.deny'.

QUESTION

What are the different modes when using vi editor?

ANSWER

There are 3 modes under vi:

- Command mode – this is the mode where you start in
- Edit mode – this is the mode that allows you to do text editing
- Ex mode – this is the mode wherein you interact with vi with instructions to process a file

QUESTION

Is it possible to use shortcut for a long pathname?

ANSWER

Yes, there is. A feature known as filename expansion allows you do this using the TAB key. For example, if you have a path named `/home/iceman/assignments` directory, you would type as follows: `/ho[tab]/ice[tab]/assi[tab]` . This, however, assumes that the path is unique, and that the shell you're using supports this feature.

QUESTION

Tell us the difference between Telnet and SSH?

ANSWER

Telnet and SSH both are communication protocol which are used to manage remote system. SSH is Secured, which requires exchanging of key opposite of telnet which transmit data in plain text, which means telnet is less secure than SSH.

QUESTION

You need to stop your X server. When you tries to kill your X server, You got an error message that you cannot quit X server. What will you do?

ANSWER

When killing a X server, it won't work normal way like doing `/etc/init.d/gdm stop`. We need to execute a special key combination `Ctrl+ Alt+ Back Space` which will force X server to restart.

QUESTION

What is the difference between command 'ping' and 'ping6'?

ANSWER

Both the commands are same and used for the same purpose except that the fact that ping6 is used with ipv6 IP address.

QUESTION

What is redirection?

ANSWER

Redirection is the process of directing data from one output to another. It can also be used to direct an output as an input to another process.

QUESTION

You want to search for all the *.tar files in your Home directory and wants to delete all at once. How will you do it?

ANSWER

We need to use find command with rm command to delete all “.tar” files.

```
# find /home/ -name '*.tar' | xargs rm -rf
```

QUESTION

What is the difference between locate and slocate command?

ANSWER

The slocate looks for the files that user have access whereas locate will search for the file with updated result.

QUESTION

What is the SMTP?

ANSWER

SMTP is the most common protocol for an e-mail server.

QUESTION

What Samba service do?

ANSWER

The Samba service provides Microsoft SMB support in Linux so that Linux machines can connect to Microsoft network resources.

QUESTION

Which service can be used with NFS to provide remote file access for UNIX systems?

ANSWER

NIS is used with NFS to provide remote file access for UNIX systems.

QUESTION

What is MTA?

ANSWER

MTA is a service of Sendmail server, which transfer mail from one network or host to another network or host.

QUESTION

Name a connection-less protocol for file transfer.

ANSWER

TFTP is a connection-less protocol for file transfer.

QUESTION

You company have slow internet connection. Which Linux service you can use to manage internet connection ?

ANSWER

You should use Squid proxy server, which allows to manage the web contents and also cache the web pages to decrease the amount of traffic going to Internet.

QUESTION

Which port should you open in your host firewall to run web server?

ANSWER

By default web server use port 80 and 443 which you need to open in firewall.

QUESTION

Which IP addressing should you use for Server?

ANSWER

For server you should always use static IP address.

QUESTION

Which ports should you open in host firewall for an email server?

ANSWER

Open port 110 which is used by the POP3 e-mail protocol.

Open port 25 which is used by the SMTP e-mail protocol.

Open port 143 which is used by the IMAP e-mail protocol.

QUESTION

Which IP address should you use for client desktop?

ANSWER

For client you can use both static and dynamic method. For easy management using DHCP to assign IP address would be the best option.

QUESTION

What is initrd image and what is its function in the linux booting process ?

ANSWER

The initial RAM disk (initrd) is an initial root file system that is mounted prior to when the real root file system is available. The initrd is bound to the kernel and loaded as part of the kernel boot procedure. The kernel then mounts this initrd as part of the two-stage boot process to load the modules to make the real file systems available and get at the real root file system. Thus initrd image plays a vital role in linux booting process.

QUESTION

Explain the terms suid, sgid and sticky bit ?

ANSWER

In addition to the basic file permissions in Linux, there are few special permissions that are available for executable files and directories.

SUID : If setuid bit is set, when the file is executed by a user, the process will have the same rights as the owner of the file being executed.

SGID : Same as above, but inherits group privileges of the file on execution, not user privileges. Similar way when you create a file within directory, it will inherit the group ownership of the directories.

Sticky bit : Sticky bit was used on executables in linux so that they would remain in the memory more time after the initial execution, hoping they would be needed in the near future. But mainly it is on folders, to imply that a file or folder created inside a stickybit enabled folder could only be deleted by the owner. A very good implementation of sticky bit is /tmp ,where every user has write permission but only users who own a file can delete them.

QUESTION

List out few of the differences between Softlink and Hardlink ?

ANSWER

- a) Hardlink cannot be created for directories. Hard link can only be created for a file.
- b) Symbolic links or symlinks can link to a directory.
- c) Removing the original file that your hard link points to does not remove the hardlink itself; the hardlink still provides the content of the underlying file.
- d) If you remove the hard link or the symlink itself, the original file will stay intact.
- e) Removing the original file does not remove the attached symbolic link or symlink, but without the original file, the symlink is useless

QUESTION

What is the difference between umask and ulimit ?

ANSWER

umask stands for 'User file creation mask', which determines the settings of a mask that controls which file permissions are set for files and directories when they are created. While ulimit is a linux built in command which provides control over the resources available to the shell and/or to processes started by it.

You can limit user to specific range by editing `/etc/security/limits.conf` at the same time system wide settings can be updated in `/etc/sysctl.conf`

QUESTION

What is the functionality of a Puppet Server ?

ANSWER

Puppet is an open-source and enterprise application for configuration management tool in UNIX like operating system. Puppet is an IT automation software used to push configuration to its clients (puppet agents) using code. Puppet code can do a variety of tasks from installing new software, to check file permissions, or updating user accounts and lots of other tasks

QUESTION

What is SeLinux?

ANSWER

SELinux is an acronym for Security-enhanced Linux. It is an access control implementation and security feature for the Linux kernel. It is designed to protect the server against misconfigurations and/or compromised daemons. It put limits and instructs server daemons or programs what files they can access and what actions they can take by defining a security policy.

QUESTION

Explain briefly the procedure for re-installing Grub in Linux?

ANSWER

- 1) Download Ubuntu Installation / Live cd
 - 2) Boot from Ubuntu Installation / Live cd - usb, burned cd etc.
 - 3) During boot select “Try Ubuntu” , Don’t select install !
 - 4) Mount your Linux root partition
`sudo mount /dev/sda6 /mnt` (Assuming /dev/sda6 is the Linux root partition)
 - 5) Install / reinstall grub
`$ sudo grub-install --root-directory=/mnt/ /dev/sda` (where /dev/sda is your primary disk)
Installation finished. No error reported.
 - 6) Reboot your system, remove bootable CD and we should have the boot menu ready when the system starts.
- Note: There would be slight difference when using with other distros

QUESTION

Explain the different Scenarios involved in TCP 3 way handshake?

ANSWER

The TCP three way handshake is the process for establishing a TCP connection. We can explain 3 way handshake with a simple scenario where we assume a client computer is contacting a server to send it some information.

- a) The client sends a packet with the SYN bit set and a sequence number of N.
- b) The server sends a packet with an ACK number of N+1, the SYN bit set and a sequence number of X.
- c) The client sends a packet with an ACK number of X+1 and the connection is established.
- d) The client sends the data.

The first three steps in the above process is called the three way handshake.

QUESTION

What is rDNS and explain its benefits in the Linux Domain Name Systems?

ANSWER

A typical DNS lookup is used to determine which IP address is associated with a hostname, and this is called Forward DNS lookup. A reverse DNS lookup is used for the opposite, to determine which hostname is associated with an IP address. Sometimes reverse DNS lookups are required for diagnostic purposes. Today, reverse DNS lookups are used mainly for security purposes to trace a hacker or spammer. Many modern mailing systems use reverse mapping to provide simple authentication using dual lookup: hostname-to-address and address-to-hostname. The rDNS (reverse DNS) is implemented using a specialized zone record for reverse lookups called PTR record. PTR records always resolve to names, never IP addresses.

QUESTION

Why is LVM required?

ANSWER

LVM stands for Logical Volume Manager, to resize filesystem's size online we required LVM partition in Linux. Size of LVM partition can be extended and reduced using the `lvextend` & `lvreduce` commands respectively.

QUESTION

How to check Memory stats and CPU stats?

ANSWER

Using 'free' & 'vmstat' command we can display the physical and virtual memory statistics respectively. With the help of 'sar' command we see the CPU utilization & other stats.

QUESTION

How to increase the size of LVM partition?

ANSWER

Below are the Logical Steps:

- Use the `lvextend` command (`lvextend -L +100M /dev/<Name of the LVM Partition>`, in this example we are extending the size by 100MB.
- `resize2fs /dev/<Name of the LVM Partition>`
- check the size of partition using `'df -h'` command

QUESTION

How to create partition from the raw disk?

ANSWER

Using fdisk utility we can create partitions from the raw disk. Below are the steps to create partition from the raw disk:

- `fdisk /dev/hd*` (IDE) or `/dev/sd*` (SCSI)
- Type `n` to create a new partition
- After creating partition, type `w` command to write the changes to the partition table.

QUESTION

What is load average in a linux?

ANSWER

Load Average is defined as the average sum of the number of process waiting in the run queue and number of process currently executing over the period of 1,5 and 15 minutes. Using the 'top' and 'uptime' command we find the load average of a linux sever.

QUESTION

What is the role of udev daemon?

ANSWER

The udev demon used to create and remove all these device nodes or files in `/dev/` directory.

QUESTION

What is RAID?

ANSWER

RAID, stands for Redundant Array of Inexpensive Disks. RAID is a method by which same data or information is spread across several disks, using techniques such as disk striping (RAID Level 0), disk mirroring (RAID Level 1), and disk striping with parity (RAID Level 5) to achieve redundancy, lower latency, increased bandwidth, and maximized ability to recover from hard disk crashes.

QUESTION

What is CUPS? What is the location of log files for CUPS?

ANSWER

CUPS stands for "Common UNIX Printing System". CUPS is a open source printing system developed by Apple Inc.

CUPS uses the Internet Printing Protocol (IPP) to allow local printing and print sharing.

The log files for the CUPS printing system are located in the `/var/log/cups/` directory.

QUESTION

What is YUM?

ANSWER

YUM stands for Yellow dog Updater, Modified because it is based on YUP, the Yellow dog Updater. Where does the name Yellow dog come from? Yellow Dog is a version of Linux for the Power Architecture hardware and is RPM-based, just like Red Hat Enterprise Linux and Fedora. YUP, and later YUM, were written by the Linux community as a way to maintain an RPM-based system.

QUESTION

Differentiate relative path from absolute path.

ANSWER

Relative path refers to the path relative to the current path. Absolute path, on the other hand, refers to the exact path as referenced from the root directory.

QUESTION

What is piping?

ANSWER

Piping, represented by the pipe character “|”, is used to combine two or more commands together. The output of the first command serves as input the next command, and so on.

QUESTION

What is a superuser?

ANSWER

A superuser is a special type user who has open access to all files and commands on a system. Note that the superuser's login is usually root, and is protected by a so-called root password.

QUESTION

What is the use of the tee command?

ANSWER

The tee command does two things: one is to get data from the standard input and send it to standard output; the second is that it redirects a copy of that input data into a file that was specified.

QUESTION

What is parsing?

ANSWER

Parsing is the process of breaking up of a command line into words. This is made possible by using delimiters and spaces. In the event that tabs or multiple spaces are part of the command, these are eventually replaced by a single space.

QUESTION

What is pid?

ANSWER

Pid is short for Process ID. It is used primarily to identify every process that runs on the UNIX system, whether it runs on the foreground or runs at the background. Every pid is considered unique.

QUESTION

What is ldd?

ANSWER

List dynamic dependencies – print shared library dependencies

Eg: – # vim new.c

gcc new.c -o new

ldd new

QUESTION

What is the command to uninstall processes in Linux?

ANSWER

`rpm -e sendmail`

`-e` – remove

QUESTION

What are the Linux boot files?

ANSWER

- 1./boot/grub/grub.conf: contains boot disk parameters
- 2./etc/fstab: contains File systems which need to mount at boot time
- 3./etc/initab: Contains default run level
- 4./etc/init.d/rc.d/rcN.d: This is a dir it contains

QUESTION

Difference between nfs soft and hard mounting points?

ANSWER

Hard mount option: – If the client fails to access the server, then the connection hangs and once the system is up then it will again access the server.

Soft: – If the client failed to connect the server, it immediately gives the error report and closes the connection.

QUESTION

What is the status code 403,404 represented in apache server?

ANSWER

403 represent forbidden error, means if a file misses some selinux security context.

404 represent that there is a cgi script missing or web pages missing.

QUESTION

WHAT IS THE MEANING OF AIX

ANSWER

AIX (Advanced Interactive eXecutive) is an open operating system from IBM which is based on a version of UNIX. AIX/ESA was designed for IBM's System/390 or large server hardware platform. AIX/6000 is an operating system that runs on IBM's workstation platform, the RISC System/6000.

QUESTION

Who invented unix?

ANSWER

Unix was created in 1969 by Ken Thompson and Dennis Ritchie at Bell Laboratories.

QUESTION

What is the command to make a process to run in the foreground from background?

ANSWER

When the process is running, press “control+Z”. Now at the command prompt, type “bg”

QUESTION

I want to see how many interfaces (Ethernet cards) are working using single command?

ANSWER

```
Ifconfig
```

QUESTION

A file which is not deleted by normal user and also root (using rm), for that type of file how we delete it?

ANSWER

Using chattr command, we need to change the attributes and then we remove using rm command.

Eg: – chattr -ilu example

rm -rf example

QUESTION

What is wild-card interpretation?

ANSWER

When a command line contains wild-card characters such as '*' or '?', these are replaced by the shell with a sorted list of files whose pattern matches the input command. Wild-card characters are used to setup a list of files for processing, instead of having it specified one at a time.

QUESTION

What does this command do? `cat food 1 > kitty`

ANSWER

It redirects the output of `cat food` into the file `kitty`; the command is the same as:

`cat food > kitty`

QUESTION

What "neat" command will do?

ANSWER

“Neat” command provides Graphical interface to change network settings for network devices.

QUESTION

Is there any relation between modprobe.conf file and network devices?

ANSWER

Yes, this file assigns a kernel module to each network device.

For Example:-

```
[root@localhost ~]# cat /etc/modprobe.conf
```

```
alias eth0 b44
```

Here b44 is the kernel module for network device eth0.

We can Confirm by following command (This module “b44” is present or not).

```
[root@localhost ~]# lsmod |grep b44
```

```
b44                29005  0
```


<https://www.udemy.com/blog/linux-interview-questions/>

<http://www.masteringinlinux.com/linux-questions/top-linux-interview-question-answer.html>

<http://computernetworkingnotes.com/rhce-interview-questions/basic-linux-interview-questions-for-freshers.html>

<http://www.tecmint.com/core-linux-interview-questions-and-answers/>

<http://www.linuxtechi.com/experience-linux-admin-interview-questions/>

<http://www.01world.in/p/linux.html>

<http://linoxide.com/linux-how-to/linux-interview-questions-answers/>

<http://career.guru99.com/top-50-linux-interview-questions/>

<http://www.tecmint.com/useful-random-linux-interview-questions-and-answers/>

<http://certcollection.org/forum/topic/154299-linux-system-administration-interview-questions-and-answers/>

<http://linuxserverworld.com/linux-interview-questions-part-i/>